elan1 Govern

TECHNOLOGY MEMORANDUM · REVISED EDITION

How Elan1 Govern is built, end to end — the decision before a tool call runs, the proof it produces, and the plan to finish it

IN ONE LINE

A model may propose a tool call; it may never produce an allow. This memorandum sets out how Elan1 Govern makes that sentence true in software — the rulebook, the decision path, the receipt chain, the approvals, the connectors, the governance layer and the build plan — and states plainly which parts exist in working code, which are frozen specification, and which are proposals.

REVISED EDITION · 27 OCTOBER 2026

The product formerly called elan1 is now **Elan1 Govern**, and it now exists as working code. The Revision supplement (pages R-1 onward, after the Contents) records what is built, maps the specification to the code, and lists the original sections it supersedes. Read it first.

ENTITY	elan1 Private Limited — Elan1 Govern and its industry and regulatory packs
COMPANION TO	Elan1 Govern Series A Information Memorandum, revised 27 October 2026
PREPARED FOR	Investors' technical diligence teams and the Elan1 Govern build team
OWNER	M K Elango, Founder
STATUS	Revised edition — draft for review
DATE	27 October 2026 (first issued 23 September 2026)
SCOPE	23 chapters · 32 figures · 3 appendices, plus the Revision supplement
ALSO ISSUED	Elan1 Govern Architecture Atlas — revised, with new figures from 33

How to read this document

This is the technical companion to the elan1 Series A Information Memorandum [IM]. The Information Memorandum explains why elan1 should exist and what it is worth. This document explains how it is built: the kernel contract, the nine-rung write path, the Declaration ontology, the console and approval path, the data architecture and proof record, the seal, the evidence bundle and offline verifier, key custody and timestamping, the regulatory packs, Level 3, the Action Receipt standard, the assurance ladder, the runtime, security, metering, the technology stack, the engineering plan, operations and the technical risks. It is written for an investor's technical diligence team and for the engineers who will build the system, and it reconciles the elan1, group, stack1 and rung1 document series into one buildable plan.

> **Designed is not shipped (DR-010).** As at 23 September 2026 elan1 has no external customer and no capability runs for a customer. A prototype exists; the Level 1 kernel is being built; Level 2 and Level 3 are frozen specifications. Every latency, cost, throughput and capacity figure in this document is a target unless it is explicitly marked as measured.

Conventions

Source identifiers. Every substantive statement carries a source identifier in square brackets, with a section where one exists — for example [E02 §05] or [IM §7.3]. E01–E27 are the elan1 documents, O01–O12 the group documents, S01–S19 the stack1 documents and R01–R24 the rung1 documents. IM is the Information Memorandum. The product-folder and strategy documents are listed in Appendix B.

Build recommendations. Where the sources are silent and this memorandum has to choose an implementation in order to explain how something is built, the choice is marked **(build recommendation)**. These are proposals. None of them is a decision the Company has recorded.

Conflicts. Where sources disagree in a way that changes the build, the chapter follows one reading and Appendix A records the conflict, the treatment and who must close it. Appendix A opens with sixteen principal recurring conflicts.

Naming. Brand names are lower case: elan1, rung1, stack1, verna1, ledger1, invoice1, collect1. rung1's product lines are rung1 Enterprise, rung1 Revenue, rung1 Voice and rung1 Commerce.

Callouts. Teal panels state rules and invariants; amber panels state qualifications, caveats and unresolved questions.

Colour language of the figures

Each figure carries a status tag in its top-right corner, such as “L1 — BEING BUILT” or “L2 — SPECIFIED, NOT BUILT”, and its sources along the foot.

Colour	Meaning
Teal	The elan1 kernel, gates, Declarations, proof and packs (dashed border: Level 2 work)
Blue	Consumers of the kernel: rung1, verna1, third-party agents, SDK and MCP callers
Purple	stack1 runtime and infrastructure; dashed purple marks Level 3
Amber	People who decide: approvers, operators, tranches
Pink	Refusals, failures, blocked paths and rejected choices
Grey	External systems and parties: systems of record, timestamp authorities, auditors, regulators

Contents

PART I — OVERVIEW AND ARCHITECTURE

1. Executive technical summary 1

1.1 What this memorandum is, and how to read it 1
1.2 The system in one page 1
1.3 The kernel in numbers 3
1.4 Three assurance levels, one declaration 4
1.5 What is built, what is specified and what is parked, as at 23 September 2026 4
1.6 The build plan in one table 5
1.7 Ten things a diligence team should check first 7
1.8 Top technical risks 7
1.9 Chapter map 8

2. System context, design principles and the perimeter 9

2.1 What elan1 is, technically 9
2.2 Actors and external systems 10
2.3 Design principles and invariants 12
2.4 The perimeter 13
2.5 The Three-Brand Integration Contract 14
2.6 Dependency direction 17
2.7 What elan1 refuses to build 18
2.8 How the architecture evolved, and why the narrowed kernel is the right cut 19
2.9 Naming rules as architectural constraints 20

3. Reference architecture — planes, components and repositories 22

3.1 The plane model 22
3.2 Other vocabularies, mapped 24
3.3 Component catalogue 24
3.4 Internal APIs 31
3.5 Repositories and packages 32
3.6 Deployment units and runtime dependencies on stack1 34
3.7 Reconciling the product-folder architecture with the E/O series 35
3.8 The existing prototype, and what carries forward 36

PART II — THE KERNEL AND THE GOVERNED WRITE

4. The kernel contract — KernelPort, six calls and five objects 38

4.1 What the contract is, and the port rule 38
4.2 The eight invariants 41
4.3 The five objects 42
4.4 Tenancy, tokens and attribution 47
4.5 The six calls 49
4.6 The refusal register at L1 55
4.7 Versioning and deprecation 57
4.8 Conformance tests 58
4.9 Decisions and open questions 59

5. The Right Path — nine rungs and the governed action end to end 60

5.1 Nine rungs and a stop set 60
5.2 Rung 0b — STOP 62
5.3 Rung 1 — IDENTITY 63

5.4 Rung 2 — WRITE DOOR 63
5.5 Rung 3 — PRE-GATES 63
5.6 Rung 4 — evaluate() 65
5.7 Rung 5 — GUARDRAIL 65
5.8 Rung 6 — AUTONOMY 66
5.9 Rung 7 — ACT 67
5.10 Rung 8 — PROOF 68
5.11 Rung 9 — METERED 69
5.12 The rungs in one table 69
5.13 One governed action, end to end — an agent posts a journal entry under ledger1 70
5.14 Idempotency and retries 74
5.15 Dependency failure — fail closed 74
5.16 Concurrency and ordering 75
5.17 Durable execution on stack1's workflow engine 76
5.18 Latency — what is targeted and what is measured 77
5.19 Decisions and open questions 77

6. Surfaces and enforcement — the SDK, the MCP server and the two modes 79
6.1 One submission surface, two ways to reach it 79
6.2 The two modes 79
6.3 How enforcement is made physical 81
6.4 The in-process SDK 83
6.5 The MCP server 84
6.6 The HTTP API 86
6.7 Sandbox key, production key and the shadow sandbox 87
6.8 The sales rule — advisory is never called enforcement 87
6.9 Third-party onboarding and conformance 88
6.10 Decisions and open questions 89

7. The Declaration — the ontology, controls, regulations and policy as data 91
7.1 Why policy is data, not code 91
7.2 The ontology claim, stated precisely 92
7.3 The seven blocks 92
7.4 Identifiers and closed enumerations 98
7.5 The three dispositions — never a blank 99
7.6 Validation — schema, eleven semantic rules and seven lint rules 100
7.7 Policy rules — what rung 4 evaluates 101
7.8 Controls and Regulations — the two families that complete the ontology 102
7.9 The five-link chain, worked end to end 104
7.10 When a clause changes 108
7.11 The pipeline — author, validate, simulate, certify, compile, resolve 108
7.12 Versioning and pinning of declaration_version 110
7.13 How packs extend declarations without code 110
7.14 The Palantir comparison 111
7.15 Tests and acceptance criteria 112
7.16 What is in Level 1, and what is not 112

8. The console and the human approval path 114
8.1 What the console is 114
8.2 The screens, and who uses them 114
8.3 The approval as data, not UI 115
8.4 The state machine 116
8.5 Dual control, quorum, delegation and step-up 118

8.6 What the approver sees — the Decide screen 119
8.7 Against rubber-stamping 120
8.8 Notification and approval channels 120
8.9 Accessibility 121
8.10 The audit of approvals 122
8.11 The APIs behind the console 122
8.12 Front-end build recommendation 123
8.13 Tests and acceptance criteria 124

PART III — DATA, EVIDENCE AND CRYPTOGRAPHY

9. Data architecture and the Level 1 proof record 125
9.1 What Level 1 proves, and what it does not 125
9.2 The stores 126
9.3 Tenancy: row-level security or schema per tenant 129
9.4 The declaration registry 130
9.5 The action store: intents, decisions, gate results and the outbox 131
9.6 Approvals and obligations 132
9.7 The proof log: the hash-chained Level 1 record 133
9.8 The meter ledger and the constraint that makes refusals free 138
9.9 Keys, secrets and the salt store 140
9.10 Retention and residency 140
9.11 Append-only, enforced by grants 141
9.12 Deletion under DPDP against a log that is never rewritten 142
9.13 Backups, restore, and why a restore is an integrity event 142
9.14 The suppression attack, and why a single writer closes it 143
9.15 The audit note: what an auditor is given at Level 1 144
9.16 Acceptance criteria for the data layer 145

10. Seal Spec v6 — canonical fields, signatures and test vectors 146
10.1 What the specification freezes, and why now 146
10.2 The sixteen fields 146
10.3 Canonicalisation rules 147
10.4 The preimage and the digest 149
10.5 Level 1 and Level 2 records, and the Ed25519 signature 151
10.6 Anchoring at two authorities 152
10.7 The seal object: how a record travels 153
10.8 The sixty-four test vectors 153
10.9 The two-implementation rule, and how to run it in CI 154
10.10 Negative cases and error handling 156
10.11 Versioning: why v6, what changed, and what is frozen 157
10.12 Code sketch: seal and verify 158
10.13 Acceptance criteria 159

11. The evidence bundle, the offline verifier and verifier profiles 160
11.1 What a bundle is 160
11.2 Layout 160
11.3 What is included, and what is deliberately not 162
11.4 Exporting a bundle 163
11.5 The offline verifier 164
11.6 Distribution, reproducible builds and no phone-home 168
11.7 Verifier profiles 169
11.8 Who verifies what 171
11.9 Auditor acceptance and the pilot 172

11.10 Performance targets 173
11.11 Test plan 174
11.12 What exists at Level 1 in the meantime 174

12. Key management, custody and timestamp authorities 175
12.1 What exists at each level 175
12.2 The design principle: a key is a capability to be accounted for 176
12.3 The key inventory 176
12.4 Generation, storage and the custody module 178
12.5 Residency domains and per-tenant isolation 179
12.6 The key register and rotation windows 180
12.7 Compromise, revocation and the unsigned interval 182
12.8 Dual control and the audit of key use 183
12.9 Proving the key was used only by the kernel, and the insider limitation 184
12.10 Publication of public keys and trust anchors 185
12.11 Key ceremonies 186
12.12 Customer-held keys at L3 186
12.13 Timestamp authorities: what an anchor is for 187
12.14 The RFC 3161 exchange 188
12.15 Selecting two authorities 189
12.16 Cadence, batching and Merkle aggregation 190
12.17 Flow 1 — anchoring a batch, and Flow 2 — when an authority fails 191
12.18 Long-term validity 192
12.19 Cost, and how it enters the meter 192
12.20 Evidence of time, and its standing in India 192
12.21 Acceptance criteria 193

PART IV — PACKS, LEVEL 3 AND THE STANDARD

13. Packs — the pack factory, ledger1, invoice1 and collect1 195
13.1 What a pack is 195
13.2 The five pack gates and the eight rejections 196
13.3 The pack definition template, field by field 198
13.4 Pack testing and the evaluation set 199
13.5 ledger1 in full 200
13.6 invoice1 — GST e-invoicing 205
13.7 collect1 — RBI collections conduct 206
13.8 The pack factory 209
13.9 A regime is a pack; a country is a set of packs 210
13.10 Packaging, versioning, signing and distribution 210
13.11 What is in Level 1, and what comes later 211

14. Level 3 — cross-organisation verification, selective disclosure and earned autonomy 212
14.1 Scope freeze and the entry gate 212
14.2 Cross-organisation verification 214
14.3 Selective disclosure 221
14.4 Precedent and earned autonomy 225
14.5 Customer-held keys and sovereign deployment 231
14.6 Build sequencing and dependencies on L2 231
14.7 What is still not true at L3 232

15. The Action Receipt standard, certification and regulator engagement 233
15.1 Publish the format, keep the write path 233
15.2 Scope of the standard 234

15.3 Receipt structure and its relationship to Seal v6 and the bundle 235
15.4 The suite registry and versioning 236
15.5 The verification procedure 236
15.6 Conformance for other implementers 237
15.7 Licensing and the reference verifier 238
15.8 Governance of the standard 239
15.9 Alignment with existing work 239
15.10 Timing, and the case against publishing 240
15.11 The certification plan 240
15.12 Regulator engagement 242
15.13 What elan1 can lawfully claim, and what it cannot 244
15.14 The overall standard and certification plan, and the partner and verifier programme 245
15.15 Acceptance criteria 247

16. Assurance levels and the upgrade path — L1 GA, L1→L2 and L2→L3 248
16.1 Why levels exist, and the rule that makes them honest 248
16.2 The three levels and what each may truthfully claim 249
16.3 L1 Guarded — the scope freeze 251
16.4 L1 general availability and third-party onboarding 252
16.5 L2 Attested — the scope freeze 254
16.6 The L1 → L2 upgrade runbook 256
16.7 L2 → L3 — the entry gate 260
16.8 Dates and gates, tied to the funding tranches 260
16.9 Acceptance tests per level 261
16.10 Open questions carried by this chapter 262

PART V — PLATFORM, SECURITY AND DELIVERY

17. Runtime and deployment — stack1, hosting profiles and sovereign deployment 263
17.1 What elan1 takes from stack1 263
17.2 The kernel's deployable units 264
17.3 Durable execution — the governed write as a workflow 266
17.4 Secrets, credentials and observability 267
17.5 Hosting profiles for the kernel 267
17.6 Sovereign deployment — a deployment profile, not a model strategy 269
17.7 Tenancy and isolation in the runtime 271
17.8 Capacity and cost model 271
17.9 Scale and multiple regions 272
17.10 The substitution protocol, as it touches the kernel 273
17.11 Environments, infrastructure as code, release and rollback 273
17.12 The latency budget on the write path 274
17.13 Failure domains and kernel-unreachable behaviour 275
17.14 Open questions 276

18. Security, residency, data protection and open-source licences 277
18.1 What is being protected 277
18.2 Threat model for a governance kernel 278
18.3 Identity and token classes 280
18.4 Secrets and target credentials 281
18.5 Keys and encryption 281
18.6 Tenant isolation inside the kernel 282
18.7 Residency 282
18.8 Logging, time and retention 283
18.9 The DPDP Act, personal data in proof records, and erasure 283

18.10 CERT-In incident reporting 285
18.11 RBI draft model-risk guidance and FREE-AI 285
18.12 The open-source licence register and its CI enforcement 286
18.13 Security testing, penetration testing and bug bounty 287
18.14 Certification posture 288
18.15 The go-live bar, elan1 column 288
18.16 Open security, residency and data-protection items 289

19. Metering, pricing plumbing and billing 290
19.1 The Governed Action as the billable unit 290
19.2 Where the meter lives: rung 9 291
19.3 What produces a meter event, and what does not 291
19.4 The meter event schema 292
19.5 The quantity-zero rule, enforced by the database 294
19.6 Emission at rung 9 295
19.7 Idempotency across the pipeline 296
19.8 Reconciliation between the proof log and the meter ledger 296
19.9 Evidence line and pack line metering 299
19.10 Price book mechanics as the rating engine sees them 300
19.11 Aggregation, rating and invoicing 301
19.12 Internal transfer billing to rung1 302
19.13 Cost per governed action and its denominator 303
19.14 The billing stack: build recommendation 304
19.15 Tests and acceptance criteria 305
19.16 What is in L1, what waits, and what is open 305

20. The technology stack — choices, alternatives and what was rejected 307
20.1 How the choices were made 307
20.2 The stack, layer by layer 309
20.3 Licences 316
20.4 Reconciling the product-folder stack with the E and O series 316
20.5 What was rejected, and why 317
20.6 Build versus buy 319
20.7 Supply-chain constraints 319
20.8 Open stack decisions 320

21. Engineering plan — teams, phases, build sequence and quality gates 322
21.1 The plan on one page 322
21.2 Team topology 323
21.3 Phases and milestones, October 2026 to March 2031 325
21.4 The L1 build sequence 326
21.5 Dependencies on stack1 and rung1 329
21.6 Definition of done 330
21.7 The CI pipeline and quality gates 331
21.8 Test strategy 334
21.9 Release process 335
21.10 Agent-assisted development 335
21.11 Engineering metrics 336
21.12 Open questions the plan depends on 337

22. Operations, reliability and incident response 338
22.1 Operating principles 338
22.2 Service level objectives and indicators 339
22.3 What "kernel unreachable" means for a consumer 341

22.4 Observability without leaking tenant data 343
22.5 Paging, on-call and escalation 344
22.6 Incident classes and runbooks 346
22.7 CERT-In and DPDP reporting timelines 349
22.8 Backup, disaster recovery, RPO and RTO 350
22.9 Capacity management 351
22.10 Support model for design partners and third parties 352
22.11 Operational readiness before L1 GA 352

PART VI — RISKS

23. Technical risks, open questions and diligence answers 353
23.1 The state of the evidence 353
23.2 The technical risk register 353
23.3 Platform absorption, read technically 355
23.4 The open-questions register 356
23.5 Contradictions in the sources that change the build 360
23.6 Diligence questions and answers 360
23.7 What would falsify the technical thesis 365
23.8 What a diligence team should check first 365

APPENDICES

Appendix A. Source conflicts and how this memorandum treats them 367

Appendix B. Source documents 386

Appendix C. Glossary 390

List of figures

Figure 1 — elan1 in context 10
Figure 2 — The perimeter and the seams 15
Figure 3 — Planes P0–P8 and who owns each 23
Figure 4 — Components, repositories and internal seams 27
Figure 5 — KernelPort v1 — six calls, five objects 40
Figure 6 — The Right Path — nine rungs in one order 61
Figure 7 — One governed action, as a sequence 71
Figure 8 — Two modes, two claims 80
Figure 9 — The Declaration — seven blocks and their relations 93
Figure 10 — From clause to proof field — the five-link chain 105
Figure 11 — The approval path — obligation state machine 117
Figure 12 — Data architecture — stores, flows, tenancy and retention 127
Figure 13 — The Level 1 proof record — a per-tenant hash chain 134
Figure 14 — Seal Spec v6 — sixteen fields to a signed, anchored seal 150
Figure 15 — The evidence bundle and the offline verifier 165
Figure 16 — Key management and custody 177
Figure 17 — Two-authority timestamp anchoring 188
Figure 18 — The pack factory: five gates, eleven blocks, eight tests 197
Figure 19 — ledger1 at runtime 203
Figure 20 — collect1: four obligation layers, fourteen gates 207
Figure 21 — Cross-organisation verification 216
Figure 22 — Selective disclosure: revealed versus committed 222
Figure 23 — Precedent and earned autonomy 227
Figure 24 — Action Receipt standard, certification and regulators 234
Figure 25 — The assurance ladder and the upgrade path 249
Figure 26 — Runtime and deployment topology 268
Figure 27 — Security, residency and DPDP controls by layer 278
Figure 28 — Metering pipeline: rung 9 to the invoice 297
Figure 29 — Technology stack by layer 309
Figure 30 — Build roadmap, October 2026 to March 2031 323
Figure 31 — CI pipeline and quality gates 332
Figure 32 — Operations: SLOs, failure behaviour and incidents 339

REVISION SUPPLEMENT · 27 OCTOBER 2026

How to read the revised edition

This supplement revises the Technology Memorandum of 23 September 2026 for two events. The founder has renamed the product. And the product now exists as working code. The original 405 pages follow this supplement unchanged. Where an original page is out of date, it carries a stamp that points to the R-page that replaces or qualifies it.

The rename — how to read "elan1" in the original pages

In the original pages, "elan1" meaning the product or kernel should be read as **Elan1 Govern**; "elan1 Private Limited" remains the entity. The founder confirmed on 27 October 2026 that only the product is renamed [FU]. The sister brands keep their names: rung1, stack1 and verna1, and the regulatory packs ledger1, invoice1 and collect1.

Status as at 27 October 2026 — built is not shipped (DR-010, restated)

Elan1 Govern is working code and the current, partial Level 1 implementation (no meter; enforcement only on the MCP route). Its guard, approvals, receipts, offline verifier, console and governance layer are present in the code at commit 22388ad. The repository's own screenshots, generated by its Playwright script against seeded demo data, show them working end to end; the audit did not itself run `pnpm demo`, because the database engine could not be installed in the sandbox [GF §2, §7]. **No customer, pilot or design partner uses it.** It has no production deployment, no usage telemetry, no certification and no independent code or security review [GF §8–9]. The Gmail and Zoho Books connectors have never been run against real accounts [GF §3 area 7]. Every latency, cost, throughput and capacity figure in this memorandum, original or revised, is still a target. Test results quoted here were observed in a sandbox on 27 October 2026. They are not product measurements.

Precedence

Where this supplement and a stamped original page disagree, the supplement prevails. Unstamped original pages stand as written, read through the rename rule above. The original pages remain the record of the 23 September 2026 plan and of the frozen specification. The frozen specification — KernelPort, Seal Spec v6 with its two-implementation rule, RFC 3161 anchoring, verifier profiles, and Levels 2 and 3 — remains the target. It is also the bar any Level 2 claim must clear. BUILD RECOMMENDATION

Conventions added

Status marks. For capabilities: BUILT means present in the repository at commit `22388ad` and confirmed by an independent read of the code; the demo was not run in the audit; PARTIAL means code exists but differs in a way that matters; SPECIFIED means frozen specification with no code; ABSENT means neither specified nor built. A specification item with no code is always Specified, never Absent. For screens: BUILT, NEEDS LOGIC, DESIGN ONLY. Provenance marks are unchanged: MEASURED is used only for measurements, and nothing in this supplement is one.

Mode names. BUILD RECOMMENDATION "Governed run" is the MCP proxy route, which is enforced (the corpus's Enforced mode). "Check and record" is the SDK and REST route, which is advisory (the corpus's Attested advisory mode). The supplement uses the new names throughout (R-32).

Proposals. BUILD RECOMMENDATION marks a choice this memorandum makes where the sources are silent. PROPOSED DECISION — UNSIGNED marks a position put to the founder that the Company has not recorded. Neither is a decision.

New source identifiers. [GF] is the code audit of 27 October 2026 (`work/facts/govern-facts.md`). It is the only source for what is built and for every count, and it audits commit `22388ad` of `Hasif-Elan1/elan1-govern`. [GOV *path*] is a file in that repository. [HB] is Hasif's brief to Ashoke on what has been built, forwarded by the founder on 27 October 2026. [FU] is the founder's update of 27 October 2026, 14:36 IST (the rename). [SM] is the screen-to-code map (`work/facts/screen-map.md` and `.csv`). [TM §x] cites an original section of this memorandum.

Numbering. New conflicts continue the Appendix A numbering: principal conflicts from P17, the register from A194. New technical risks continue the Chapter 23 register from T23. New open questions continue the open-questions register from Q105. Figures continue the Architecture Atlas numbering from 33. The figures are drawn in the revised Atlas and reproduced here.

Contents of the supplement

	How to read the revised edition	R-1
	What changed since 23 September 2026	R-4
	Supersession register	R-5
R1	Elan1 Govern as built — architecture, packages and stack	R-6
R2	The decision path and the data model	R-10
R3	Receipts, checkpoints, the auditor pack and offline verify	R-14
R4	Rulebooks — rules, limits, testing, suggestions and industry packs	R-18
R5	Approvals — states, links, Slack and WhatsApp	R-21
R6	Connectors and the MCP proxy	R-25
R7	The governance layer — inventory, assessments, frameworks and controls	R-26
R8	Alerts, auto-pause and the emergency stop	R-28
R9	Identity, roles, organisations and security	R-30
R10	Developer surfaces and the two modes	R-32
R11	How the built code was checked	R-34
R12	Spec-to-code crosswalk	R-36
R13	Vocabulary — one name per concept	R-38
R14	One information architecture — thirteen areas	R-39
R15	Revised build plan and what remains	R-42
R16	New technical risks	R-43
R17	Conflicts, proposed decisions and open items	R-44
	Glossary and source additions	R-46

Figures in the supplement

33	Elan1 Govern in context	R-6
34	The decision path for one tool call	R-10
35	Receipts, checkpoints, auditor pack and offline verify	R-14
36	Rulebook lifecycle and the suggestions loop	R-20

37	Approval state machine, links and chat	R-22
38	Governance layer	R-26
39	Alerts, auto-pause and emergency stop	R-28
40	One information architecture: thirteen areas	R-39
41	Spec-to-code crosswalk	R-36
42	Deployment and package topology	R-8

What changed since 23 September 2026

The original memorandum described a prototype, a Level 1 kernel being built, and frozen specifications for Levels 2 and 3 [TM §1.5]. On 26 and 27 October 2026 a separate codebase, Elan1 Govern, was written. Its 47 commits span about 15.5 hours under one identity [GF §1]. The founder then renamed the product to match [FU]. The twelve changes below are what a reader of the original pages needs to know first.

Topic	As at 23 September 2026 (original)	As at 27 October 2026 (revised)	See
Name	elan1, the governance kernel [TM §2.1]	Elan1 Govern. The entity elan1 Private Limited is unchanged [FU].	R-1
What exists	A prototype (`elan1-platform`); no signing, no export; "the E02 kernel as not yet built" [TM §1.5, §3.8]	Working code: about 33,600 lines in one monorepo (about 31,300 of TypeScript and Python, of which about 8,600 are tests), 27 console pages, a guard service, two SDKs and a CLI [GF §2]. Still no customer.	R-6
Scope	Consequential writes into a system of record, ERPNext first, Indian finance first [TM §2.1]	Every tool call an agent routes through Elan1 Govern, reads and writes, in any industry, through risk-tag rules. Built-in tools exist only for Gmail and Zoho Books; other systems join through upstream MCP servers [GF §3].	R-10
Decisions	allow · refuse · escalate, after nine rungs [TM §5]	allow · ask a person · refuse, after an eight-step guard. Default refuse; strictest rule wins [GF §3].	R-10
Record	Sixteen-field record, length-prefixed preimage, SHA-256, **unsigned at L1**; no key material at L1 [TM §9.7, §12.1]	Receipt: canonical JSON, SHA-256, **Ed25519-signed**, hash-chained per organisation, with hourly signed checkpoints [GF §3]. Seal Spec v6 remains the L2 target.	R-14
Verification	No bundle and no verifier at L1; a zero-dependency Go verifier with six exit codes at L2 [TM §11]	An auditor pack (bundle and PDF) and an offline `govern verify` in TypeScript, with exit codes 0, 1 and 2. The verifier trusts the key in the bundle unless one is supplied [GF §3].	R-14
Policy object	Seven-block Declaration; author, validate, simulate, certify [TM §7]	One signed rulebook per agent: tool or tag rules, conditions, limits, Try an action, Test on past actions, sign and publish, rule suggestions [GF §3].	R-18
Console	Four Doors compiled from declarations, "30 screens, 0 screen files" [TM §8.1]	Thirteen hand-built areas; 27 page routes. Of 275 design screens, 113 now sit on working code [SM].	R-39
Approvals	Obligation; resubmission, never resumption; 4-hour default; WhatsApp reply-approval specified [TM §8]	Waiting, approved, rejected, expired. Signed single-use links. Slack and WhatsApp buttons only on rules marked for chat. Default expiry 24 hours. An approved MCP call runs at once [GF §3].	R-21
Governance	Controls and Regulations as declaration families; no regulator mapping at L1 [TM §7.8, §2.7]	AI inventory, three assessment templates, 10 frameworks and 13 controls with coverage counted from receipts — all "Draft — needs legal review" [GF §3].	R-26
Operations	Stop set at rung 0b [TM §5.2]	Per-agent pause, six behaviour detectors, optional auto-pause, and an emergency stop with an incident report [GF §3].	R-28
Stack	Python 3.12 / FastAPI kernel, React 18 console, Go verifier, Temporal on stack1 [TM §3.5, §20]	TypeScript: Fastify guard, Next.js 15 / React 19 console, Prisma 6 on PostgreSQL 16, pg-boss jobs. It runs on its own, with no stack1 dependency [GF §2].	R-6

What did not change

No customer, no revenue, no certification. No external timestamp, no key custody module, no second implementation, no metering. Levels 2 and 3 remain specifications. Every date in the original build plan predates the code, and the founder has not yet re-set them (R-42).

Supersession register

Chapter 14 and sections 15.1–15.12 (except 15.3) are not superseded. For the rest: original section, first PDF page (printed page), treatment and replacement. **Supersede**: no longer true of what exists (the specification may stand as a target). **Amend**: right but qualified. **Add**: built material added. Stamps name the same R-page.

Original section	PDF (pr.)	Treatment and reason	See
Cover; How to read	1–2	Supersede: new cover; status and naming conventions revised	R-1
1.1 What this memorandum is	12 (1)	Amend: DR-010 status; "signed" is now true of L1 receipts	R-1
1.2 The system in one page	12 (1)	Supersede: every tool call, eight-step guard, signed receipt	R-10
1.3 The kernel in numbers	14 (3)	Add: built counts (tags, decisions, frameworks, controls, industry packs)	R-6
1.4 Three levels, one declaration	15 (4)	Supersede status row: signing exists before custody and anchoring	R-14
1.5 What is built	15 (4)	Supersede: replaced by the crosswalk	R-36
1.6 Build plan; 16.8, 22.11 gates	16 (5)	Supersede: sequence overtaken; all dates to be re-set	R-42
2.1 What elan1 is; the frozen claim	20 (9)	Supersede: scope and the L1 claim sentence (proposed)	R-17
2.2, 2.4, 2.6 Actors, perimeter, dependencies	21 (10)	Amend: built actors and one-way package dependencies	R-6
2.3 Invariants I1–I8	23 (12)	Amend: seven built invariants mapped to I1–I8	R-10
2.5, 2.7–2.9 Contract; refusals; DR-006; naming	25 (14)	Amend: code takes positions the corpus forbade — conflicts P17–P23	R-44
3.3–3.6 Components, APIs, repositories, deployment	35 (24)	Supersede as a description of what exists	R-6
3.7–3.8 Product folder; prototype	46 (35)	Supersede: prototype replaced by Elan1 Govern	R-36
4 KernelPort (4.1–4.8)	49 (38)	Amend: no port in code; check/record and MCP are the surfaces	R-36
5.1–5.13 Nine rungs; end to end	71 (60)	Amend: rung-by-rung mapping to the guard; new worked example	R-10
5.14–5.17 Idempotency, fail closed, Temporal	85 (74)	Amend: no idempotency keys; fail-closed with one gap; pg-boss	R-12
6 Surfaces and the two modes	90 (79)	Supersede: built MCP, REST, SDKs; modes renamed and inverted	R-32
7.1–7.7, 7.11–7.13 The Declaration	102 (91)	Supersede as the L1 object: the rulebook	R-18
7.8–7.9 Controls and Regulations	113 (102)	Supersede as built: frameworks and controls	R-26
8.1–8.2 Console, Four Doors, screens	125 (114)	Supersede: thirteen areas	R-39
8.3–8.8, 8.10 Approval path, channels	126 (115)	Supersede: built approval states, links, chat	R-21
8.11–8.12 APIs; front end	133 (122)	Supersede: built API, webhooks and front-end stack	R-32
9.1–9.2, 9.5 Stores and action store	136 (125)	Supersede: one PostgreSQL schema, 30 tables	R-12
9.3, 9.12 Tenancy; DPDP deletion	140 (129)	Amend: application-level tenancy, no RLS; no erasure path	R-30
9.7, 9.9, 9.15 Proof log, keys, audit note	144 (133)	Supersede: signed receipts, checkpoints, auditor pack	R-14
9.8 Meter ledger	149 (138)	Amend: absent in code	R-36
10.1–10.5 Seal v6 status, fields, signature	157 (146)	Amend: Seal v6 stays the L2 target beside the built receipt	R-14
11.1, 11.5, 11.12 Bundle, verifier, L1	171 (160)	Supersede: auditor pack and `govern verify` exist	R-14
12.1, 12.6 Keys at L1; rotation	186 (175)	Supersede: per-organisation keys, rotation built, no custody	R-14
12.9 Insider limitation	195 (184)	Amend: now the live limitation of the built record	R-43
13.1, 13.11 What a regulatory pack is	206 (195)	Amend: industry packs (built) vs regulatory packs (specified)	R-18
15.3, 15.13 Receipt standard; lawful claims	246 (235)	Amend: which format the standard covers; compliance labels	R-44
16.1–16.4 Levels; L1 scope freeze; L1 GA	259 (248)	Amend: the level the code may claim (proposed)	R-17
17.1–17.3 Runtime on stack1	274 (263)	Supersede: standalone services, pg-boss	R-6
18.2–18.9 Threats, identity, isolation, DPDP	289 (278)	Supersede/amend: built auth, SSO, roles, security posture	R-30
19.1, 19.16 Billable unit; L1 meter	301 (290)	Amend: no meter in code; unit to be confirmed	R-42
20.1–20.2, 20.5 Stack; rejected choices	318 (307)	Supersede: built stack; conflict A195	R-6
21.1–21.4, 21.7 Plan, sequence, CI	333 (322)	Supersede: revised plan; built CI and tests	R-42
1.7–1.8, 23.1–23.5 Checks, risks	18 (7)	Add: code provenance and checks, T23–T34, P17–P23	R-34
Appendix A conflicts	378 (367)	Add: P17–P23, A194–A205; updates to P2, P8, P14	R-44
Appendices B and C	397 (386)	Add: new sources and terms	R-46

R1. Elan1 Govern as built — architecture, packages and stack

Elan1 Govern has two halves that share one data model. Runtime control decides every tool call an agent routes through it and records a signed receipt. Governance keeps a register of AI systems, assessments, frameworks and controls, and counts its evidence from those receipts [GOV docs/ARCHITECTURE.md]. Everything in this chapter is at commit `22388ad` [GF §1].

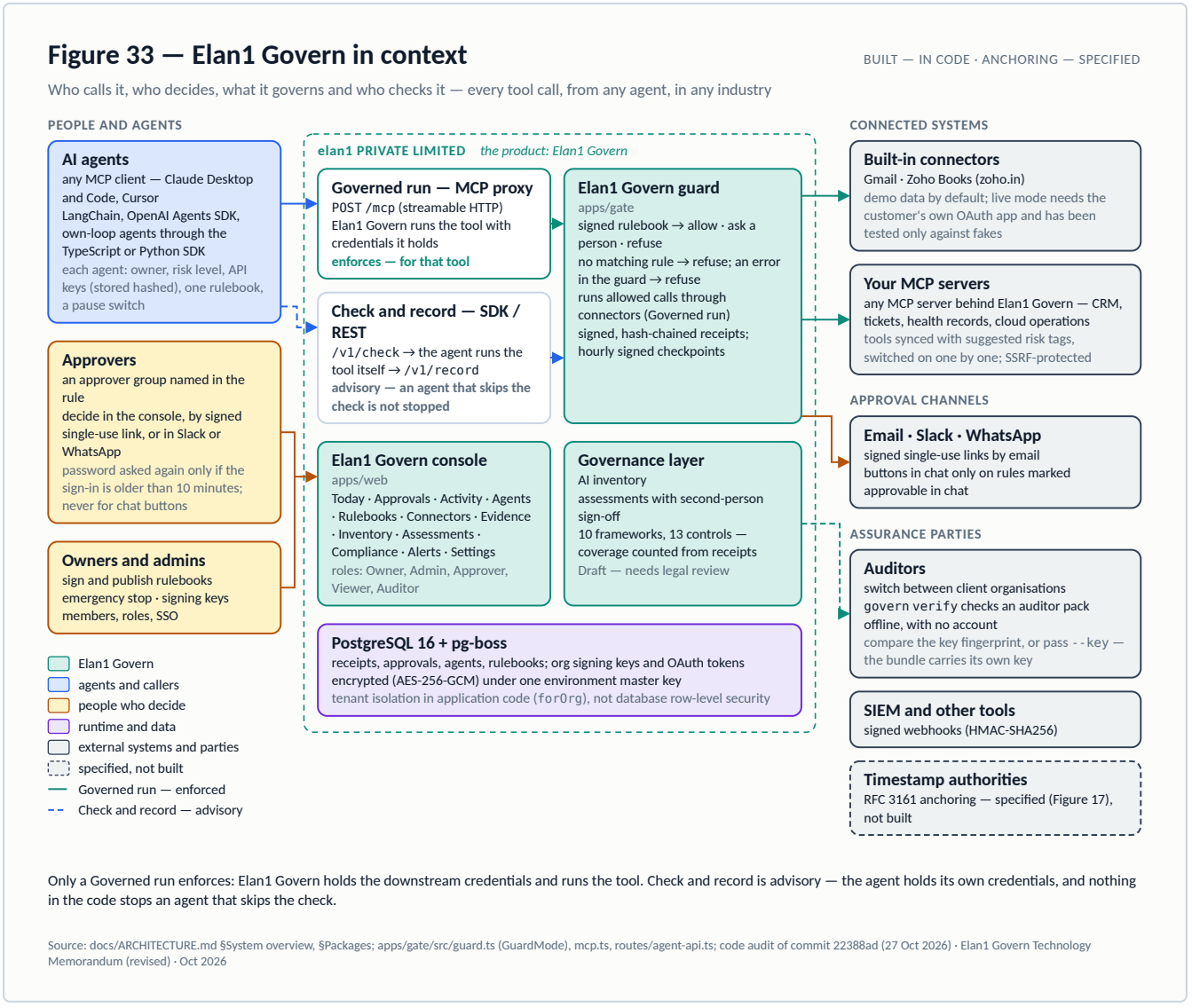


Figure 33 — Elan1 Govern in context: agents reach the guard by MCP, REST or SDK. People use the console. Connectors and upstream MCP servers sit behind the guard. Auditors check an exported auditor pack offline. Status: built, demo data only.

R1.1 Two services and one database

The **guard service** (`apps/gate` , Fastify) is the only component that decides, signs or holds secrets. It runs the MCP endpoint, the REST Event API, approvals, chain writes, evidence export, connectors, webhooks, chat messages and background jobs. The **console** (`apps/web` , Next.js) has 27 pages and 53 API route handlers. It reads through a tenant-scoped client and changes chain-affecting state only by calling the guard's internal API, which uses HMAC-signed requests with a 60-second skew [GF §2–3]. Both services are stateless apart from **PostgreSQL 16**. Background work runs on **pg-**

boss queues inside the same database. The design allows several guard instances, but rate limits and the tool-registry cache are held per process, and no multi-instance deployment has been tested [GF §2, §5; GOV docs/ARCHITECTURE.md].

The guard holds the organisation signing keys, the OAuth tokens for built-in connectors and the upstream MCP authorisation headers. All of them are encrypted with AES-256-GCM under one environment master key, `GATE_MASTER_KEY`. The console cannot sign anything [GF §3 Security mechanics].

R1.2 Packages and dependency direction

Package	Role	Source lines	Test lines
`apps/web`	Console: 27 pages, 53 API handlers; auth, SSO, roles, CSRF	11,041	849
`apps/gate`	Guard: MCP server, REST API, approvals, chain writes, evidence, connectors, webhooks, chat, jobs, demo and seed	5,312	2,794
`packages/core`	Pure logic, no I/O: canonical JSON, crypto, rulebook schema and signing, rule engine, receipts, verifier, bundle, behaviour detectors, suggestions, replay, assessments, frameworks and industry packs	4,336	2,755
`packages/db`	Prisma schema and migrations, `forOrg()` tenant client, serialised chain writer, checkpoints	1,891	212
`packages/connectors`	Gmail, Zoho Books (zoho.in) and in-memory mock connectors	822	289
`packages/sdk`	TypeScript SDK: `check`, `record`, `waitForApproval`, `verifyWebhook`	125	90
`packages/sdk-python`	Standard-library-only Python SDK with the `guarded` decorator	419	470
`packages/cli`	`govern verify | sign | keygen`	111	122

Source: [GF §2], line counts by `git ls-files`; all code about 33,567 lines, excluding lockfiles, JSON, Markdown and images.

Dependencies run one way: `core ← db ← apps` and `core ← connectors ← gate`. Core performs no I/O, and the console never imports connectors, so it can never hold an OAuth token [GOV docs/ARCHITECTURE.md]. This is the built counterpart of the original's import-graph rule [TM §2.6, §3.5.3]. It is enforced by package boundaries and one static test (not run in the audit), not by an import-linter contract. `packages/core` is also the code the verifier uses, which matters for the two-implementation rule (P19).

R1.3 The stack, as built

Layer	Built (version)	Original plan [TM §20.2]
Language	TypeScript (226 `.ts`, 75 `.tsx` files); Python for the SDK only	Python 3.12 kernel; Go verifier; TypeScript second seal
Guard / API	Fastify with helmet and rate-limit; `@modelcontextprotocol/sdk`, stateless Streamable HTTP; run with `tsx`	FastAPI with uv
Console	Next.js 15.5 App Router, React 19.2, Tailwind, radix-ui, lucide	React 18, Vite, compiled from declarations
Database	PostgreSQL 16; Prisma 6.19; 11 migrations	PostgreSQL, INSERT-only evidence tables
Jobs	pg-boss queues and cron in PostgreSQL	Temporal on stack1 (DR-002)
Crypto	`node:crypto` only: Ed25519, SHA-256, AES-256-GCM, HMAC-SHA256	pyca/cryptography; KMS/HSM at L2
Auth	argon2id, TOTP (otpauth), OIDC (openid-client)	Consume identity; own token issuer
Other	zod 4, yaml, fflate, pdf-lib, nodemailer, pino	—
Tests	Vitest 4.1, fast-check, Playwright 1.64, Python unittest	Conformance suite C1–C14
Deployment	Dockerfile and docker-compose (postgres, mailpit, web, gate)	stack1 runtime, hosting profiles

Source: [GF §2]. The divergence from the planned stack and from stack1 is recorded as P19 and A195.

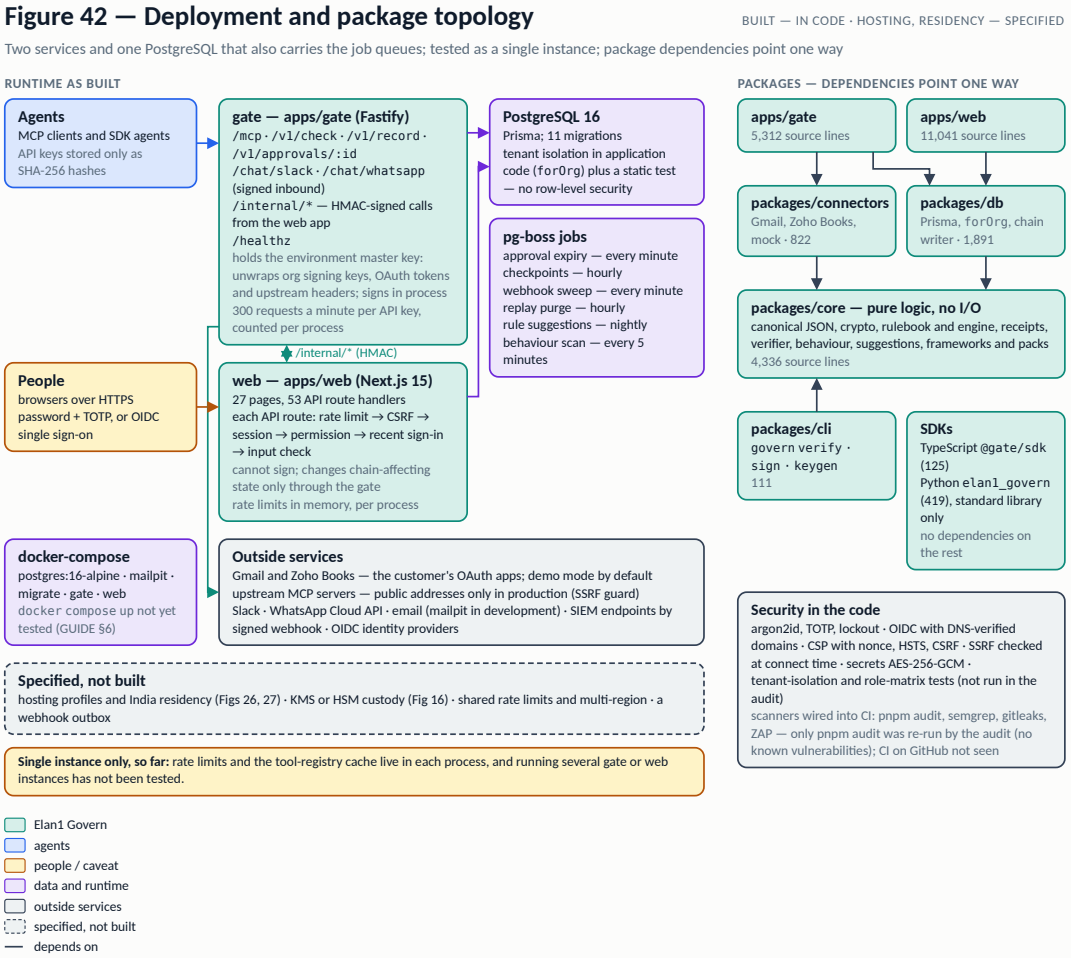


Figure 42 — Deployment and package topology: two stateless services, one PostgreSQL database holding the application tables and the pg-boss queues, and the package dependency direction. Status: built; `docker compose up` untested [GF §6].

R1.4 Elan1 Govern in numbers

These counts replace the "kernel in numbers" as a description of what exists [TM §1.3]; the original numbers remain the specification. Source: [GF §2–3]; tables counted in `schema.prisma`.

Quantity	Built	What it is
Decisions	3	allow · ask · refuse; no rule matched means refuse
Risk tags	8	reads_data, writes_data, moves_money, sends_external, personal_data, deletes, changes_access, changes_production
Condition operators	8	eq, lt, lte, gt, gte, in, to_domain_in, fields_include
System refusal ids / receipt kinds / fields	6 / 4 / 22	agent-paused, no-rulebook, rulebook-invalid, default, invalid-arguments, gate-error; decision, approval, outcome, admin
Approval states / channels / verifier exit codes	4 / 4 / 3	pending, approved, rejected, expired; web, email, slack, whatsapp; exit 0 intact, 1 broken, 2 usage
Frameworks / controls	10 / 13	All labelled "Draft — needs legal review"
Industry packs / assessment templates	7 / 3	Starter rulebooks by industry; triage, impact, data protection
Detectors / roles / permissions	6 / 5 / 22	Statistical alert rules; Owner, Admin, Approver, Viewer, Auditor

Quantity	Built	What it is
Tables / pages / screenshots	30 / 27 / 23	Prisma models (26 tenant-scoped); console pages; screenshots of seeded demo data

R2. The decision path and the data model

Every MCP `tools/call` and every `POST /v1/check` runs the same function, `guard.ts` [GOV apps/gate/src/guard.ts]. It is the built counterpart of `assert` and the Right Path [TM §4.5, §5]. It is shorter, and it governs reads as well as writes.

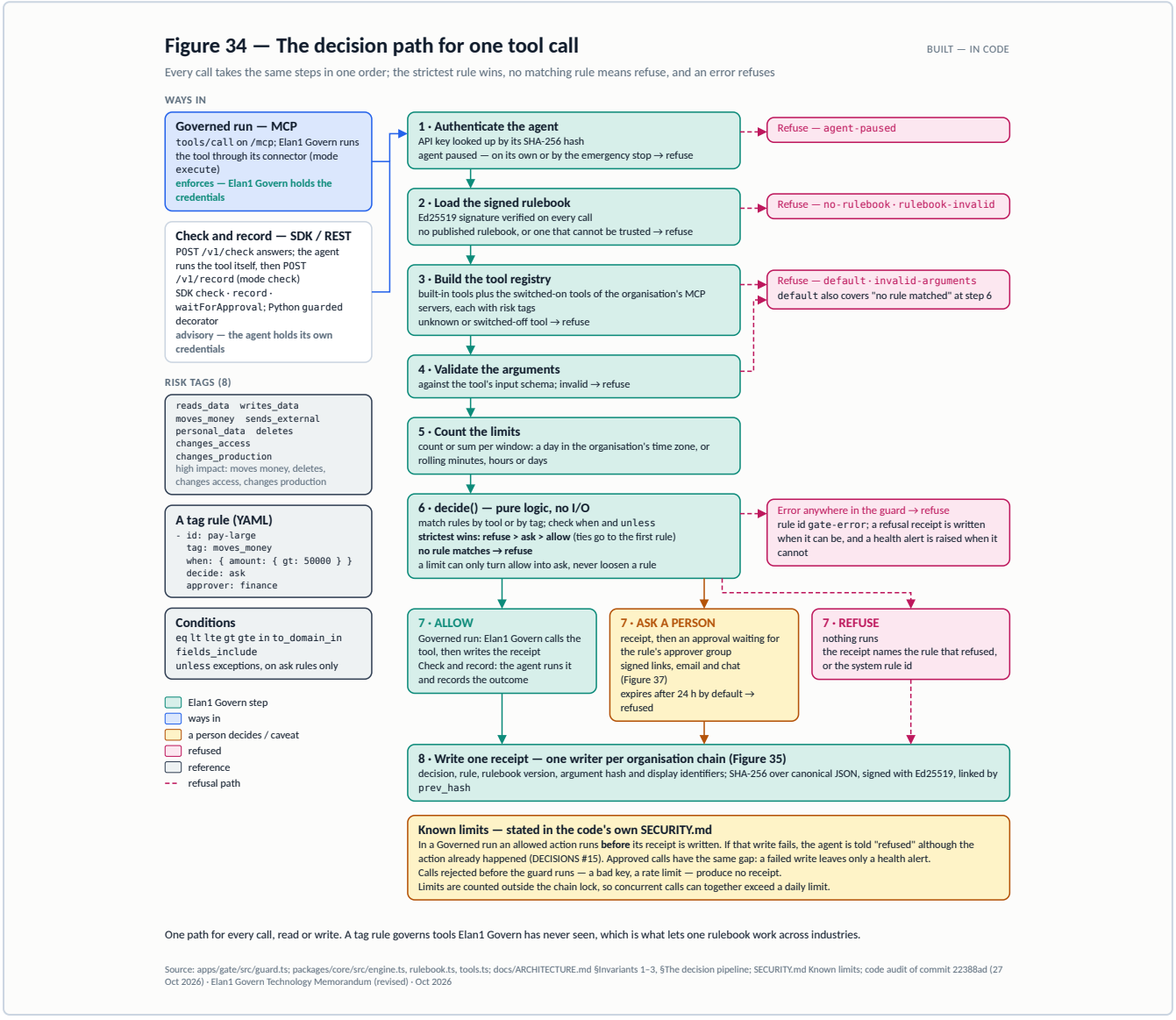


Figure 34 — The decision path for one tool call: identify the agent, load and verify the signed rulebook, resolve the tool, validate the arguments, compute limits, decide, then run, hold or refuse — and write the receipt. Status: built.

R2.1 The eight steps

1. **Identify.** The agent's API key (prefix `gk_`) is looked up by SHA-256 hash with a constant-time compare. A paused agent is refused with rule id `agent-paused`.
2. **Load the rulebook** and verify its Ed25519 signature against the organisation key. A missing or unverifiable rulebook is refused (`no-rulebook`, `rulebook-invalid`).
3. **Resolve the tool** in the organisation's tool registry: built-in tools plus enabled tools synced from upstream MCP servers, each with a schema and risk tags. Unknown, disabled and removed tools are refused before any rule runs (`default`).

4. **Validate the arguments** against the tool's schema (`invalid-arguments`).
5. **Compute limits.** Each applicable quota gets one parameterised SQL query, counting or summing over a day in the organisation's time zone or a rolling window.
6. **Decide.** `decide()` is a pure function in core. It matches rules by tool or by tag, evaluates conditions, picks the strictest (refuse over ask over allow) and applies limits. A limit can only turn an allow into an ask, or into a refuse when there is no approver group. If no rule matches, the call is refused.
7. **Act.** On allow in MCP execute mode, the connector runs the tool and one receipt is written. On ask, a receipt, an approval row, single-use signed links and an email are written before anything runs. On refuse, a receipt names the rule.
8. **Notify.** Held approvals queue Slack and WhatsApp messages as background jobs. After commit, webhooks and an inline behaviour check run, also in the background.

Source: [GOV docs/ARCHITECTURE.md "The decision pipeline"; GF §3].

R2.2 The nine rungs, mapped

Rung [TM §5]	Built equivalent	Status
0b STOP	Per-agent pause and organisation-wide emergency stop → `agent-paused`	BUILT
1 IDENTITY	Agent API key (hash lookup); human sessions in the console	BUILT
2 WRITE DOOR	None: reads and writes take the same path; no write-type descriptor	SPECIFIED
3 PRE-GATES	Rulebook present and signed; tool known and enabled; arguments valid. No G1–G9 gate set	PARTIAL
4 evaluate()	`decide()`, pure, no model	BUILT
5 GUARDRAIL	Quotas (count or sum per window); no blast-radius values	PARTIAL
6 AUTONOMY	None (A0–A5 absent). Ask rules name an approver group; rule suggestions are the nearest analogue	SPECIFIED
7 ACT	Connector call in MCP execute mode only; no snapshot, shadow sandbox, per-write credential or rollback	PARTIAL
8 PROOF	Signed receipt in the per-organisation chain	BUILT
9 METERED	None	SPECIFIED

Source: [GF §4].

R2.3 Invariants as built, against I1–I8

The built system states seven invariants: default refuse; strictest wins; only signed rulebooks run; one writer per chain; tenant scoping through `forOrg`; secrets only in the guard; and minimal receipts [GOV docs/ARCHITECTURE.md]. Against the original eight [TM §2.3]:

- **I1** (no model decides) holds: no model is called anywhere on the decision path.
- **I3** (fail closed) holds in the guard with one gap, set out below.
- **I4** (refusals free and typed) is half met. Refusals are receipted, but there is no meter and no closed refusal-code register: six system rule ids plus free-form customer rule ids.
- **I5** (no fast path or decision cache) is qualified. The tool registry is cached for 10 seconds per organisation and dropped on every change to an upstream server.
- **I6** (no warn) holds: there are three outcomes.
- **I7** (nothing resolves to "latest") is replaced. The guard loads the agent's active rulebook, and each receipt records the rulebook version and hash it was decided under.

- **I8** (determinism) is supported by a property test. Replaying the active rulebook over its own history reproduces every decision [GOV DECISIONS.md #66]. But an `as_of` instant is not supplied by the caller.
- **I2** (the single writer holds the only target credential) is met only on the MCP path, and only if the agent has no other route to the system (see R-32).

> **Fail-closed, with one disclosed gap (DECISIONS #15)**
>
> Unknown tools, unloadable rulebooks, invalid arguments, a paused agent and any exception in the guard all end in refuse. An exception writes a refusal receipt with rule id `gate-error`. If even that write fails, the guard raises a health alert and still answers refused [GF §3]. The exception: in MCP execute mode an **allowed action runs before its receipt is written**. If that write then fails, the agent is told "refused" although the action has happened. A health alert names the tool and the connector's reference so that a person can reconcile it [GOV DECISIONS.md #15; SECURITY.md "Known limits"]. The same order applies when an approver approves a held governed run: the call runs, then the approval receipt is written, and a failed write raises a health alert [GOV apps/gate/src/approvals.ts]. Refusals, asks and check-and-record decisions write their receipt before any answer is given. Calls rejected before the guard runs — an invalid or missing API key, or a rate limit — produce no receipt. This is the original memorandum's commit-then-seal gap (`acted_unproven`) [TM §1.8], without the outbox that was designed to close it.

R2.4 One governed tool call, end to end

This replaces the ledger1 journal-entry walk-through [TM §5.13] with the demonstration the code actually runs: an accounts-payable agent in the seeded organisation Demo Traders Pvt Ltd. The agent reads an invoice email through the Gmail connector (demo mailbox) and creates a bill in Zoho Books (demo books). It then pays it. The demo rulebook allows a payment up to ₹50,000 (`pay-small`), asks a person above that (`pay-large`), refuses a change to vendor bank details, and caps payments per day. The demo cap is ₹20,00,000 rather than the brief's ₹5,00,000, so the demo can be repeated [GOV DECISIONS.md #22]. A ₹2,00,000 payment to Sharma Traders therefore waits for a person (R-21, Figure 37). Once approved, the held call runs, and a second receipt (kind `approval`) records who decided, where and when. `govern verify` on the exported pack reports "Chain intact" [GOV README.md]. Every name, amount and vendor in this example is seed data.

R2.5 The data model

One PostgreSQL schema holds 30 Prisma models, 26 of them tenant-scoped through `forOrg(orgId)`, which ANDs the organisation into every query; a static isolation test exists but was not run in the audit, and CI was not seen. There is no Postgres row-level security [GF §3 Security mechanics]. pg-boss keeps its own tables in a separate schema.

Group	Tables	Notes
Identity and organisations	User, Session, Membership, Org, Invite, SsoConnection	Membership carries the role per organisation; Org holds industry, regions, time zone, currency (no UI writes currency), replay-store switch
Agents and keys	Agent, ApiKey, OrgKey	API keys stored as SHA-256 hashes; OrgKey holds active and retired signing keys, wrapped
Policy	Rulebook, RuleSuggestion, ReplayCall	Signed rulebook versions; ReplayCall holds encrypted arguments for 30 days
Decisions and proof	Receipt, Checkpoint, Approval, ApprovalLink	Receipts are the chain; links are single-use
Connectors	Connector, UpstreamServer, UpstreamTool	Tokens and headers encrypted under the master key
Governance	AiSystem, Assessment, OrgFramework	Framework library itself is code, not rows
Integrations	Webhook, WebhookDelivery, ChatChannel, ChatNumber, ChatMessage	Retry state per delivery and per chat message

Group	Tables	Notes
Monitoring	BehaviourAlert, HealthAlert, Incident	Incident is written by the emergency stop

Source: [GOV packages/db/prisma/schema.prisma; GF §3]. The original's separate stores — action store, outbox, meter ledger, salt store [TM §9.2] — have no counterparts. There is no outbox: allowed actions run before their receipt, and a webhook event can be lost if the process dies between commit and enqueue [GF §5].

R3. Receipts, checkpoints, the auditor pack and offline verify

The original memorandum said that Level 1 holds no key material and that its record is not signed [TM §9.7, §12.1]. Elan1 Govern signs every receipt from the first one. This chapter says exactly what is signed, by whom, and what the signature does and does not prove. All statements are at commit `22388ad` [GF §3].

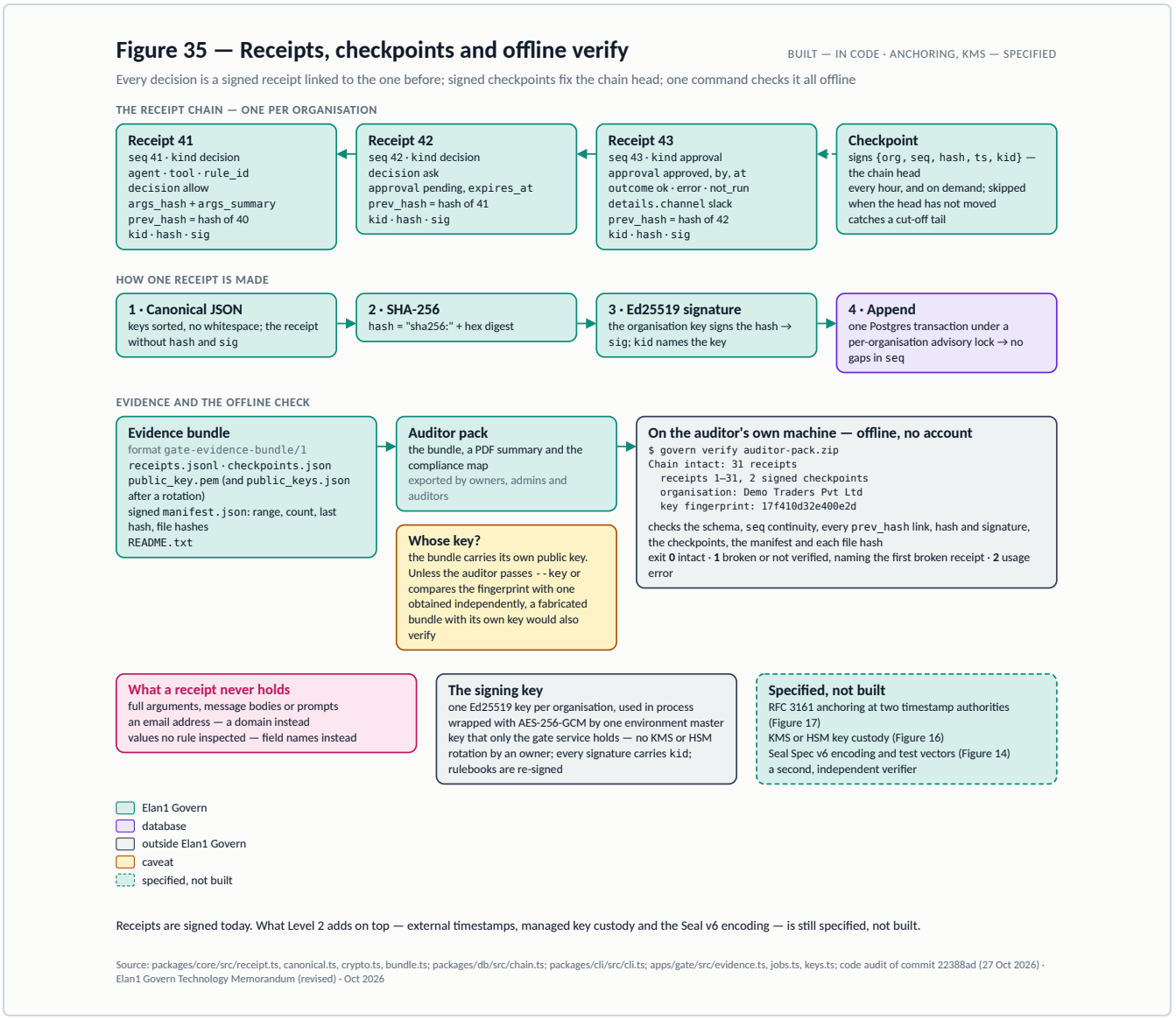


Figure 35 — Receipts, checkpoints, auditor pack and offline verify: each receipt is hashed over canonical JSON, signed with the organisation's Ed25519 key and linked to the previous one; an hourly checkpoint signs the chain head; the auditor pack carries receipts, checkpoints, keys and a signed manifest for `govern verify`. Status: built; no external anchor.

R3.1 The receipt

A receipt is a strict schema of 22 top-level fields, most of them optional [GOV packages/core/src/receipt.ts]: `seq, ts, org, kind, agent, tool, args_summary, args_hash, rulebook{version,hash}, rule_id, decision, reason, approval{…}, outcome{…}, ref_seq, actor, action, details, prev_hash, kid, hash, sig`. The `kind` is one of `decision` (the guard's answer), `approval` (the second receipt of an ask), `outcome` (an SDK agent reporting what happened) or `admin` (a rulebook signed, a connector added, a role changed, a key rotated).

Full arguments are never stored. `args_hash` is SHA-256 over the canonical form of the tool and its arguments. `args_summary` keeps only display identifiers and the values the rules inspected. For an email that means the recipient's domain, not the address. For a record change it means the names of changed fields, not their values [GF §3; GOV DECISIONS.md #6, #24].

R3.2 Hash, signature, chain and checkpoint

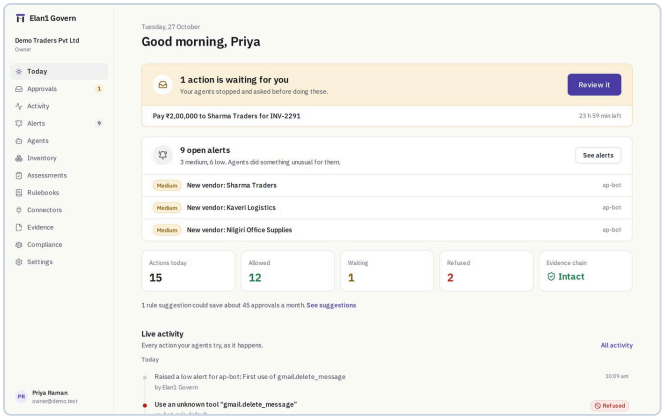
Element	Built	Seal Spec v6 [TM §10]
Encoding	Custom canonical JSON: sorted keys, no whitespace, undefined members dropped, −0 as 0, non-finite numbers rejected. Not claimed to conform to RFC 8785; no cross-language vectors	Sixteen fixed fields, length-prefixed preimage, no JSON in the digest
Digest	`"sha256:" + hex(SHA-256(canonical(receipt without hash and sig)))`	SHA-256 over the domain-separated preimage
Signature	Ed25519 over the UTF-8 text of the hash string, from the first receipt; `kid` = first 16 hex of SHA-256 of the public-key PEM	None at L1; pure Ed25519 at L2
Chain	Per organisation; `prev_hash` from a genesis of 64 zeros; gap-free `seq` under a per-organisation Postgres advisory lock	Per tenant; F02 `seq`; chain writer from an outbox
Checkpoint	Signs `{org, seq, hash, ts, kid}` hourly at :00 (skipped if the head has not moved) and on demand	Merkle batch anchored at two RFC 3161 authorities (L2)
Implementations	One (TypeScript, `packages/core`)	Two, independent, agreeing on 64 vectors before L2

Source: [GF §3 "Receipt, chain, checkpoint and verifier details"; GF §4].

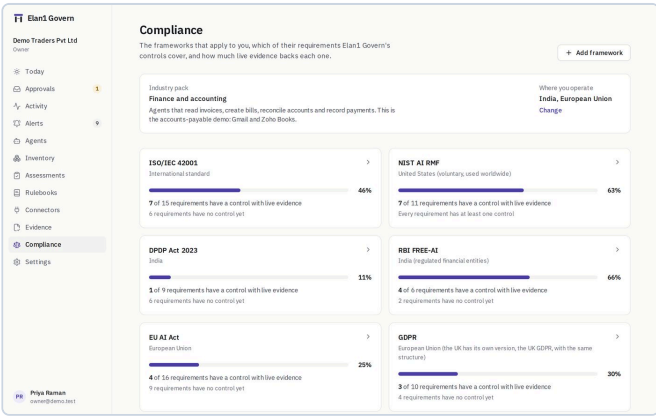
R3.3 The auditor pack and `govern verify`

The **bundle** (`gate-evidence-bundle/1`) contains `receipts.jsonl`, `checkpoints.json`, `public_key.pem`, optional `public_keys.json` (key id to PEM), a signed `manifest.json` and a `README.txt`. The **auditor pack** wraps the bundle with a PDF summary generated from the same receipts, including the compliance map for the frameworks the organisation has turned on. Every export writes an `evidence.exported` receipt [GF §3; GOV apps/gate/src/evidence.ts].

`govern verify` runs offline; the CLI contains no network code. It checks that the archive is readable and complete. If `--key` is given (a public-key PEM file), it checks that the bundle key matches it. It checks that key ids equal their fingerprints. Then, for each receipt, it checks the strict schema, sequence continuity (gaps, duplicates, reordering), the `prev_hash` link, a recomputed hash, and the Ed25519 signature with the key named by `kid`. It rejects any key used after its retirement. Finally it checks the checkpoint signatures, the manifest signature, counts and last hash, and the SHA-256 of every listed file. It names the first broken receipt. Its exit codes are **0** (intact), **1** (broken or not verified) and **2** (usage error or missing file). The specification has six [GF §3; TM §11.5]. In the console, every receipt in Activity has "Verify this receipt", which re-checks its hash, signature and both chain links [GF §3 area 4].



Today, with the "Evidence chain: Intact" tile and one waiting approval [GOV docs/screenshots/02-today.png]. Seeded demo data.



Compliance coverage counted from receipts, labelled draft [GOV docs/screenshots/17-compliance.png]. Seeded demo data; coverage is a count of receipts, not a test of controls.

R3.4 Keys

Each organisation has one active Ed25519 key and any number of retired ones. The active key signs receipts, checkpoints, manifests and rulebooks [GOV DECISIONS.md #4, #56]. Private keys are wrapped with AES-256-GCM under the environment master key through a `KeyProvider` interface. **Only the environment provider exists**: there is no KMS and no HSM, signing happens in the guard process, and the decrypted key is cached in memory [GF §3 Security mechanics].

Rotation is owner-only and needs a sign-in within the last 10 minutes. It writes a `key.rotated` receipt signed by the old key, retires that key at the receipt's time and re-signs every rulebook with the new key. Bundles carry every key id, and the verifier rejects any signature made after its key was retired [GOV DECISIONS.md #57–58; GF §3]. The key inventory, custody module, ceremonies, dual control of key use and out-of-band publication of fingerprints [TM §12.3–12.11] remain specification. The built per-organisation key answers Appendix A conflict P2 in the direction of O05 (per-tenant keys) rather than E16 (one key per residency domain in KMS). That is recorded as A196.

R3.5 What the signature proves, and the claim it permits

A valid `govern verify` proves that the receipts were signed by the holder of the private key named in the bundle, and that none has since been edited, removed, inserted or reordered without breaking the chain or a checkpoint. It does not prove three things.

- **It does not exclude the key holder.** The operator who holds `GATE_MASTER_KEY` can unwrap the organisation key and re-sign a rewritten chain, checkpoints included, because the checkpoints are self-signed and nothing is anchored to an outside time source.
- **It does not authenticate the bundle by itself.** The verifier trusts the key inside the bundle unless the auditor passes `--key` with the organisation's public-key PEM file obtained separately, or compares the printed fingerprint with one obtained independently. A fabricated bundle with its own key would also verify [GF §3].
- **It does not show what happened downstream.** It shows what Elan1 Govern decided and recorded, not that the system of record changed. In check-and-record mode the agent acts on its own credentials (R-32).

The phrase in the brief, "verify an exported bundle offline … without trusting us" [HB], therefore over-claims and is withdrawn from this memorandum. The L1 copy rule that forbade "signed" [TM §2.9, C14] is also out of date, because the receipts are signed. The proposed replacement for the frozen L1 claim [TM §2.1.1] is:

> *"At L1, Elan1 Govern decides whether a tool call is allowed before it runs. On its MCP route it holds the call for a named person where the rulebook requires it and refuses anything not explicitly allowed; on its SDK and REST routes it advises and records, and the agent acts. It records each decision it makes in a per-organisation hash chain, signed with a key Elan1 Govern holds, with hourly signed checkpoints. The record is tamper-evident. It is not yet anchored to an external time source, and not yet verifiable by a party who does not trust us."*

PROPOSED DECISION — UNSIGNED The sentence is recorded as P17 in this memorandum and as DR-017 in the revised Information Memorandum, in identical words. Until it is signed, the words "tamper-proof", "immutable", "court-ready", "independently verifiable" and "without trusting us" stay forbidden, and "signed" is used only with the key holder named. Elan1 Govern is the current, partial Level 1 implementation (no meter; enforcement only on the MCP route). L2 is not claimed; two of its ingredients — Ed25519 signing and an offline verifier — exist in code. L2 still requires key custody, two-authority anchoring, two implementations agreeing on the vectors, and verifier profiles [TM §16.5]. The screen map reaches the same conclusion: investor documents should not call Elan1 Govern L2 or L3 [SM (g) 9].

Two record formats (P20)

The built receipt is the L1 record. Seal Spec v6 remains the L2 target. BUILD RECOMMENDATION The original plan froze the record shape on 30 November 2026 [TM §1.6]. Whether that freeze now applies to the built receipt (re-basing Seal v6 and the Action Receipt standard on it), to v6 (with a migration), or to neither is a founder decision, and it is left open.

R4. Rulebooks — rules, limits, testing, suggestions and industry packs

The rulebook is the built policy object. Like the Declaration, it is data, not code. It is far smaller: one signed YAML document per agent, with top-level keys `version, agent, owner, default` (always `refuse`), `approvers, lists, limits, quotas, rules, signature` [GF §4]. The seven-block Declaration remains the L2 specification the rulebook would have to grow into [TM §7]. BUILD RECOMMENDATION

R4.1 Rules and limits

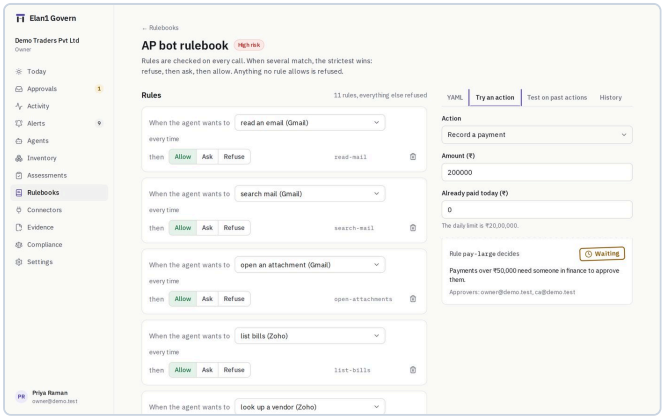
A rule matches **either** one tool **or** one of the eight risk tags. It may add conditions (`when`) using eq, lt, lte, gt, gte, in, to_domain_in and fields_include. It decides allow, ask (naming an approver group of email addresses in the rulebook) or refuse. It can carry a reason, an expiry for asks (`expires_in`, default 24 hours, at most 30 days) and `approve_in_chat`. Ask rules alone may carry an exception, `unless`: the ask stands aside only when every exception condition holds, and a missing value never holds. The strictest matching rule wins, ties go to the first rule, and no match means refuse [GF §3 area 6].

Tag rules let one rulebook govern tools Elan1 Govern has never seen. That is the mechanism behind the claim that it works in any industry [GOV docs/ARCHITECTURE.md]. **Limits** (quotas) count calls or sum a field over a day (local midnight) or a rolling window, for example "50 emails an hour" or "₹5 lakh a day". A breached limit turns an allow into an ask, or into a refuse when no approver group exists. It never loosens. Limits are checked outside the chain lock, so concurrent calls can jointly exceed one [GF §5 Known limits]; see T27.

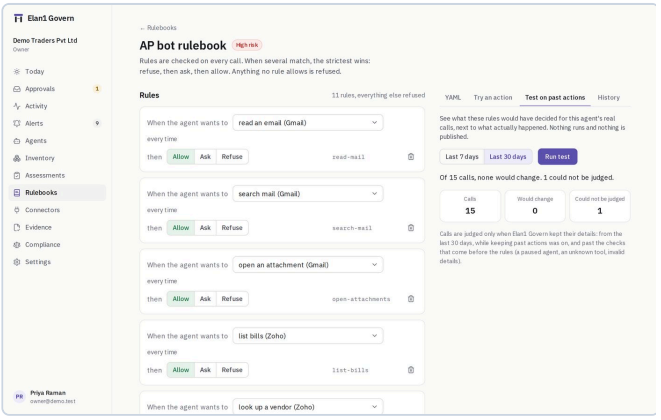
R4.2 The rule builder, testing and publishing

The builder at `/rulebooks/[slug]` keeps a visual editor and a YAML view in sync [GF §3]. **Try an action** decides one made-up call against the draft. **Test on past actions** replays the draft over the agent's real calls from the last 7 or 30 days. It reports how many more would be refused or wait for a person, and lists up to 200 calls whose decision changes.

The replay uses a separate store, not the receipts. The full validated arguments are encrypted with AES-256-GCM under the master key, kept for 30 days and purged hourly. The store is **on by default** and can be switched off in Settings, which deletes the rows. Replays run in the guard, are rate-limited, cover at most the 50,000 most recent calls, and write no receipt [GF §3; GOV DECISIONS.md #64–67]. **Publishing** validates tool names strictly and refuses high-risk agents without an ask rule on high-impact tags. It shows a diff, then signs the version. The guard verifies that signature on every call [GF §3].



Rule builder with Try an action: a ₹2,00,000 payment is decided by rule `pay-large` — waiting for a person [GOV docs/screenshots/08-rule-builder.png]. Seeded demo data.



Test on past actions over 7 or 30 days: "Of 15 calls, none would change. 1 could not be judged." [GOV docs/screenshots/22-test-on-past-actions.png]. Seeded demo data.

R4.3 Rule suggestions

When people approve the same kind of action every time, Elan1 Govern drafts a rule that would stop asking. A nightly job runs at 02:00 India time, and "Look for suggestions" runs it on demand. It reads 30 days of answered approvals and proposes a draft only under these conditions:

- at least 10 approvals, spread over at least 7 days, the latest within 14 days;
- none of them rejected or expired;
- a cap no higher than the largest approved amount;
- never a change to refuse rules, or to tools tagged deletes, changes_access or changes_production.

Every draft is replayed against past approvals before it is shown. It may only turn approved asks into allows. "Open as draft" loads it into the builder; nothing changes until someone with permission signs and publishes it. Opening and dismissing are receipted [GF §3; GOV DECISIONS.md #79–87]. This is the nearest built analogue of the Level 3 idea of earned autonomy [TM §14.4]. A person decides; nothing graduates on its own.

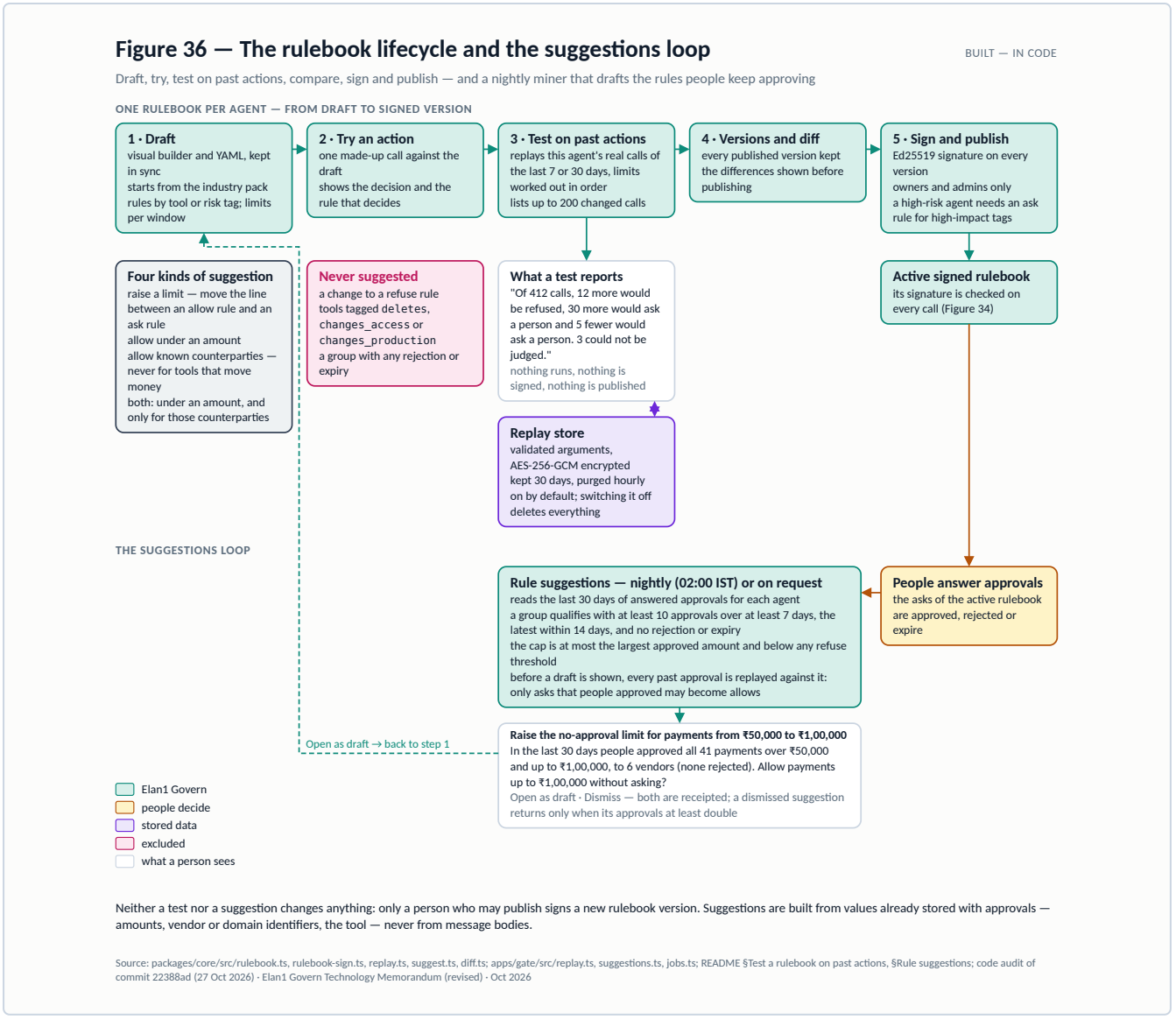


Figure 36 — Rulebook lifecycle and the suggestions loop: draft in the builder, try an action, test on past actions, sign and publish; approvals accumulate; a suggestion is mined, replay-checked and offered back as a draft. Status: built.

R4.4 Industry packs and regulatory packs

Onboarding asks for the organisation's industry and the regions where it operates (India, EU, UK, US). It does **not** ask for currency or time zone, although the brief says it does [GF §3 area 1]. The industry picks one of seven **industry packs**: Finance and accounting, Healthcare, HR and recruiting, Customer support, Sales and marketing, Engineering and DevOps, and General. Each industry pack carries typical risks, recommended frameworks, the controls that matter most, wording for the risk questions, and a starter rulebook written mostly as tag rules.

Only the finance industry pack names built-in tools: the Gmail and Zoho Books accounts-payable rulebook. The other six use tag rules, for example refuse `deletes`, ask on `changes_access`, 50 messages an hour in healthcare, 20 refunds a day in support and 10 deploys a day in engineering. Every industry pack turns on ISO/IEC 42001 and NIST AI RMF, plus frameworks for the chosen regions [GF §3 "Industries at onboarding"].

An industry pack is not a regulatory pack in the original sense: "one declaration, one certified verifier profile and one regime-specific export mapping … under one dated obligation" [TM §13.1]. It passes none of the five admission gates. The two are therefore named apart. **Industry pack** means the built starter set. **Regulatory pack** means ledger1, invoice1 and collect1, which remain SPECIFIED. The bare word "pack" is not used (R-38).

R5. Approvals — states, links, Slack and WhatsApp

The original's strongest Level 1 claim was that a person approved the actions that needed one, having seen what they were approving [TM §8]. The built approval path keeps the idea and changes most of the mechanics.

R5.1 States and what the approver sees

An ask creates a receipt and an approval in state **waiting** (`pending`). It ends as **approved**, **rejected** or **expired**. Expiry defaults to 24 hours, the maximum is 30 days, and a job runs every minute. An approval past its time is refused at once, even before the job runs, and its stored arguments are wiped [GF §3 area 3]. The approval page shows:

- the agent, the tool and the arguments;
- the evidence, for example the invoice, the vendor and the amount;
- the rule that caused the ask, who must approve, and when it expires.

There is no field-by-field diff, no gate-by-gate result list and no "Reverse this (30 days)", which the original Decide screen specified [TM §8.6]. Approve and Reject go through one atomic claim on the approval row, so double clicks and replays decide once [GOV DECISIONS.md #69].

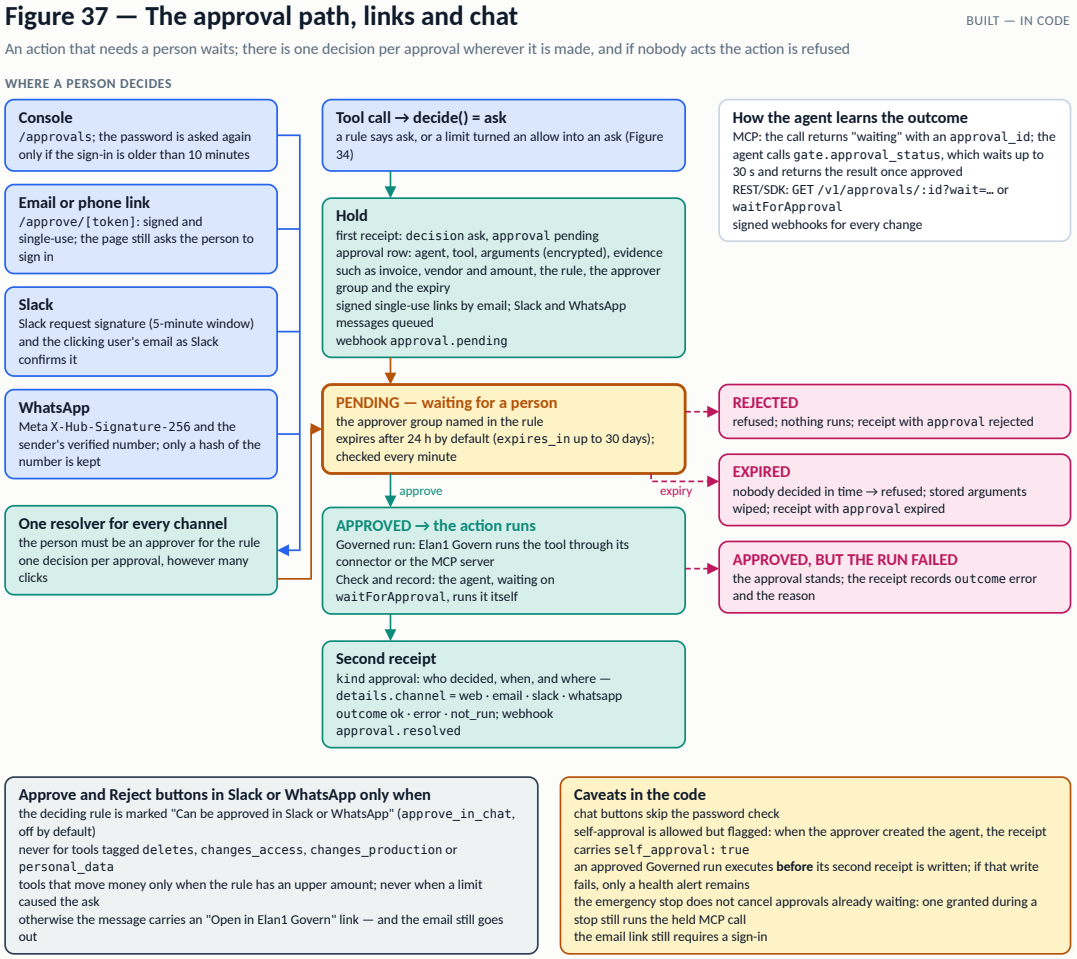


Figure 37 — Approval state machine, links and chat: waiting → approved, rejected or expired; decided on the web, through a signed single-use email or phone link, or by a Slack or WhatsApp button where the rule allows it; the receipt records the channel. Status: built.

R5.2 Re-authentication, links and self-approval

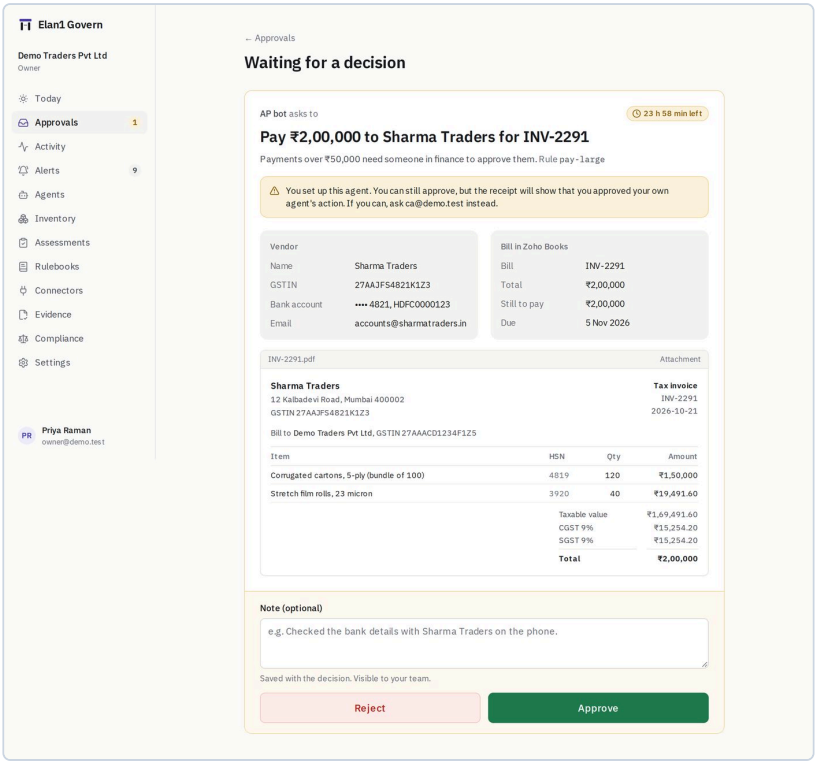
- **Re-authentication is conditional.** Approving asks for the password only if the approver's last sign-in was more than 10 minutes ago. Chat buttons skip the check entirely [GF §3].
- **Email and phone links** carry an HMAC token and are single-use (`ApprovalLink.usedAt`). The link page still requires sign-in.
- **Self-approval is allowed but flagged.** When the approver created the agent, the approval receipt carries `self_approval: true` , and control GATE-03 counts such approvals [GF §3 "Approval states"]. This departs from the original rule that the proposer cannot approve [TM §8.7]. The proposer here is an agent and agents never approve, but the person who set up the agent may, and the receipt then carries `self_approval: true` .
- **No dual control.** There is no quorum, delegation or re-routing to a backup approver. All are recorded in A194.

R5.3 Run on approval, not resubmission

The original design re-submitted an approved write, which climbed every rung again: "resubmission, never resumption" [TM §8.3, §2.3.2]. The built behaviour differs by surface:

- **MCP (governed run).** When an approver says yes, Elan1 Govern runs the held call at once and writes the second receipt. The agent sees the result next time it asks, through Elan1 Govern's own tool `gate.approval_status`, which long-polls for up to 30 seconds.
- **SDK and REST (check and record).** The agent waits with `waitForApproval`, runs the tool itself, then calls `record()`, which writes an outcome receipt [GOV DECISIONS.md #13–14].

The approved call is not re-decided against the rulebook in force at run time, and on the MCP route it runs before its approval receipt is written (R-12). This is recorded in A194.



Approval detail for a ₹2,00,000 payment, with the self-approval warning ("You set up this agent… the receipt will show that you approved your own agent's action") [GOV docs/screenshots/14-approval-phone.png]. Seeded demo data.

R5.4 Slack and WhatsApp

Chat is "a doorbell, not a replacement" [GOV DECISIONS.md #69]. Approvers are notified on Slack or WhatsApp, and email still goes out. Approve and Reject buttons appear only on rules marked "Can be approved in Slack or WhatsApp" (`approve_in_chat`). That flag is off by default and is refused at publish for any rule that reaches:

- a tool tagged deletes, changes_access, changes_production or personal_data;
- a money-moving tool without an upper amount.

The guard checks again when it holds the action. Asks caused by a limit are never chat-approvable. Everything else gets a link back to the console. This matches the original specification's bounded WhatsApp reply-approval [TM §8.8], now built and extended to Slack.

Identity of the person who clicked is checked differently on each platform:

- **Slack** users are matched to members only by an email Slack marks as confirmed. Clicks from another workspace are ignored.
- **WhatsApp** senders must match a number the member verified with a 6-digit code.
- **Inbound requests** are verified by Slack's v0 HMAC (5-minute window) and by WhatsApp's `X-Hub-Signature-256` [GF §3 area 3; GOV DECISIONS.md #70–72].

The record of the decision. The approval row and the approval receipt record the channel (web, email, slack, whatsapp) and the channel user. WhatsApp users appear only as a keyed hash; numbers are stored encrypted. Sending is a background job with up to six attempts [GOV DECISIONS.md #74–75]. The console does not yet show "decided in Slack" or "decided in WhatsApp" as a label [SM (g) 4].

R6. Connectors and the MCP proxy

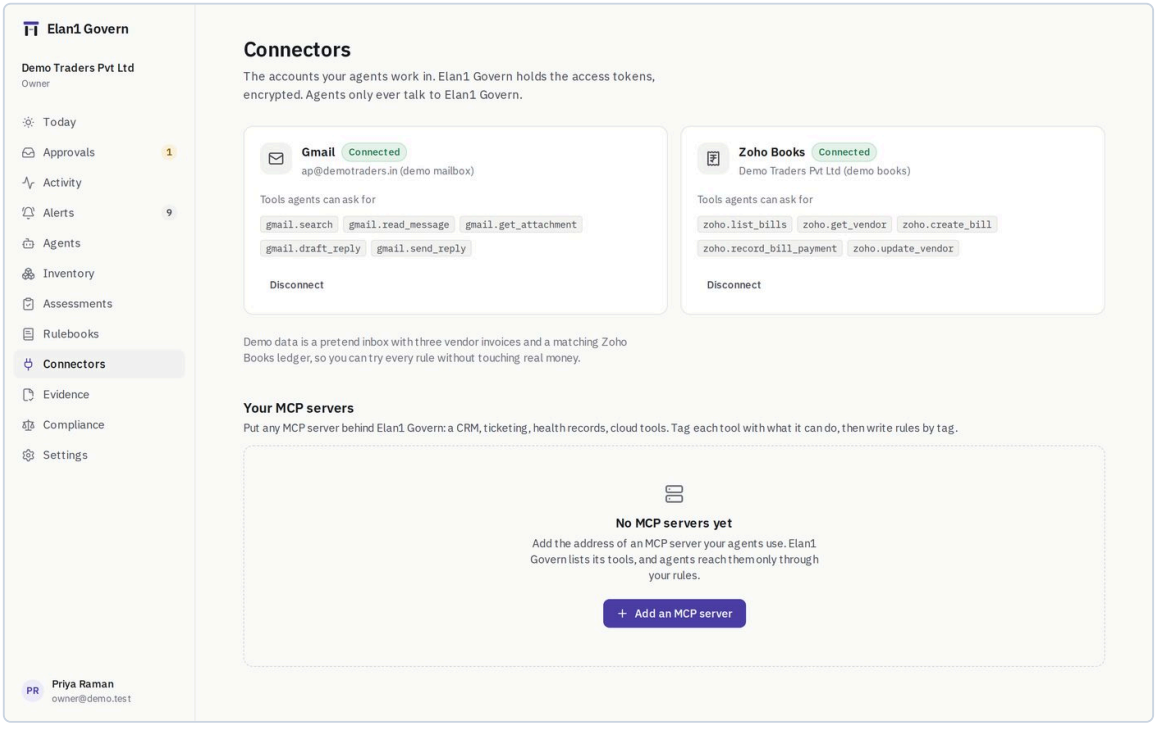
The original memorandum left every system-of-record adapter to consumers [TM §2.8.2]. Elan1 Govern ships two built-in connectors and a proxy that puts any MCP server behind the guard. Status is at commit `22388ad` [GF §3 area 7].

Connector	What is built	Limits
Gmail	Read and send tools with schemas and risk tags; customer's own OAuth app in live mode	**Live mode has never been run against a real account**; tested only against fakes. Default is demo (`CONNECTOR_MODE=mock`), held in memory and reset on restart [GOV DECISIONS.md #16]
Zoho Books	Bills and payments against the India data centre (zoho.in); customer's own OAuth app in live mode	
Your MCP servers	Any MCP server behind Elan1 Govern. Its tools are synced, given suggested risk tags from a fixed word list, and start **disabled** until a person switches each on. Tools are named `<server>.<tool>`; `gate`, `gmail` and `zoho` are reserved. One MCP session per upstream call; upstream errors reach agents only as generic text; registry cached 10 s and dropped on change	Upstream authorisation is one `Authorization` header; no OAuth to upstreams. Elan1 Govern cannot stop an agent calling the upstream server directly

Source: [GF §3 area 7, §4; GOV DECISIONS.md #34–42].

Secrets. OAuth tokens and upstream headers are encrypted under the master key and decrypted only in the guard's memory to make a call. Agents never see them on the MCP path. This is control GATE-09, "Agents never hold credentials", and it is true only on that path. On the SDK and REST path the agent holds its own credentials [GF §3 "13 controls"].

Server-side request forgery. Upstream URLs must be http or https and may not carry credentials. A block list covers the private, loopback, link-local, CGNAT, multicast, documentation and reserved IPv4 and IPv6 ranges, and IPv4-mapped IPv6 addresses are refused. The address is checked inside the socket's DNS lookup, so the address checked is the address connected, with no DNS-rebinding window. Redirects are not followed, and responses are time- and size-capped. Private addresses are allowed by default outside production (`UPSTREAM_ALLOW_PRIVATE`) [GF §3 Security mechanics].



Connectors: Gmail (demo mailbox) and Zoho Books (demo books) connected; "Your MCP servers" empty [GOV docs/screenshots/09-connectors.png]. Seeded demo data.

R7. The governance layer — inventory, assessments, frameworks and controls

The original memorandum refused "a regulator mapping at L1" because the chain could not yet support it [TM §2.7]. Elan1 Govern has built one, as code-level data labelled "Draft — needs legal review" on every page and export [GF §3 area 11]. Its evidence is a count of receipts. It is not proof that a control works.

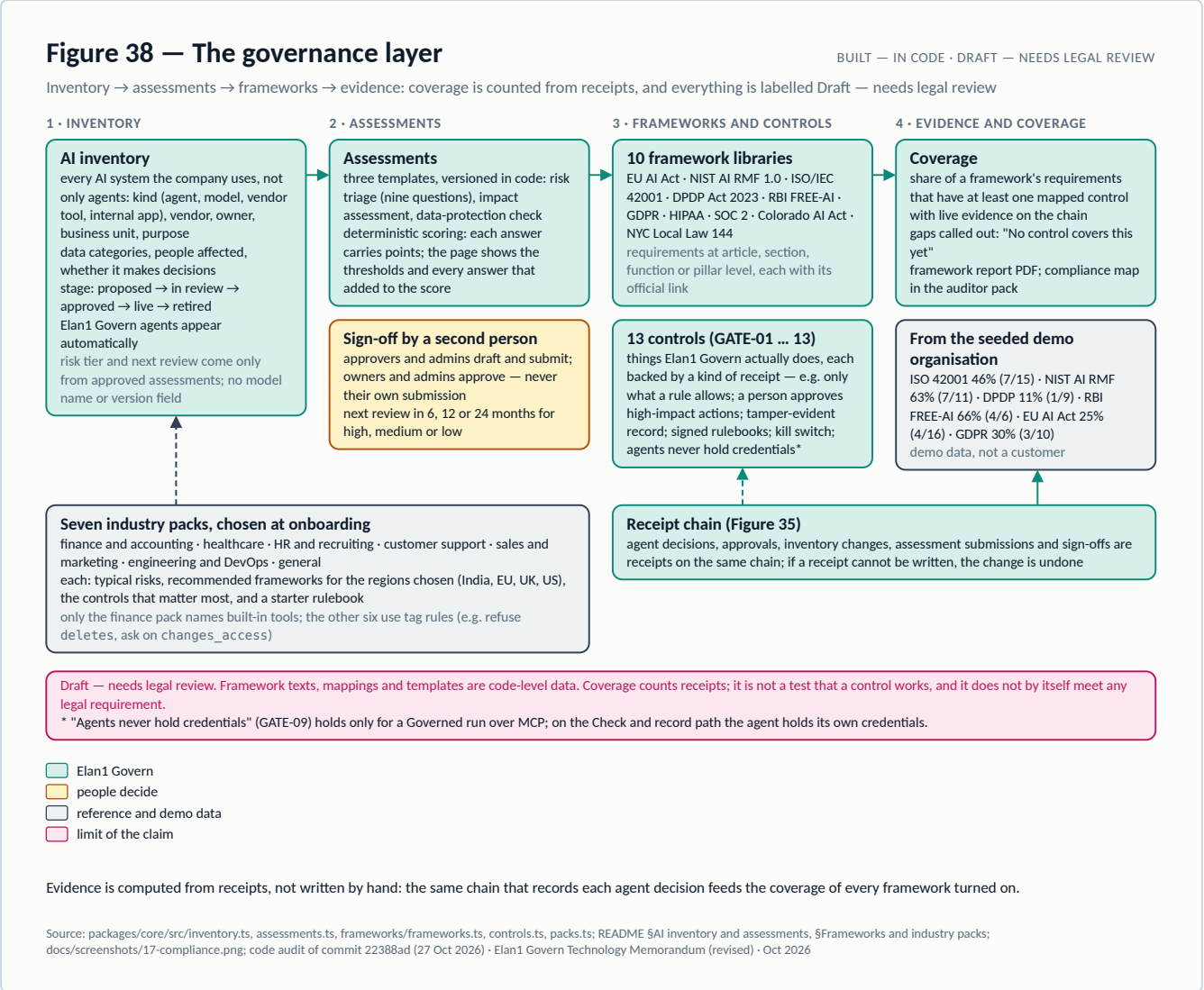


Figure 38 — Governance layer: AI inventory and assessments set each system's risk tier; frameworks map requirements to thirteen controls; coverage is counted from receipts on the chain; gaps are named. Status: built; every mapping a draft needing legal review.

R7.1 AI inventory

The inventory holds every AI system the organisation uses, not only agents. It records name, kind (agent, model, vendor tool, internal app), vendor, purpose, owner, business unit, lifecycle stage (proposed, in review, approved, live, retired), data categories, affected people, decision impact, linked agent and next review. There is **no model name or version field**, which the brief describes. The risk tier and next review **cannot be set by hand**: they come only from approved assessments. Agents join the inventory automatically [GF §3 area 9; GOV DECISIONS.md #29–31]. Automatic discovery of unregistered AI is not built [SM (c)].

R7.2 Assessments

There are three templates, versioned in code: risk triage, impact assessment and data-protection check. Scores come with explanations. The submitter can never sign off: separation of duties is checked by user id, including against racing reviews. Approvers may draft and submit; only owners and admins sign off. Approval sets the system's tier to the highest latest result and the next review to 6, 12 or 24 months for high, medium or low [GF §3 area 10; GOV DECISIONS.md #28, #31–32].

R7.3 Frameworks, controls and coverage

Ten frameworks are mapped at the level of article, section, function or pillar: EU AI Act, NIST AI RMF, ISO/IEC 42001, DPDP Act 2023, RBI FREE-AI (August 2025 report, pillar level only), GDPR, HIPAA, SOC 2 (2017 criteria), Colorado AI Act and NYC Local Law 144. Thirteen controls, each something Elan1 Govern does and backed by a kind of receipt, map to their requirements. Coverage is the share of a framework's requirements that have at least one mapped control with live evidence, rounded down. A requirement with no control is shown as a gap. A framework report PDF is generated by the guard and receipted [GF §3 area 11; GOV DECISIONS.md #43–46, #49].

Id	Control	Evidence counted	Id	Control	Evidence counted
GATE-01	Agents only do what a rule allows	default refusals	GATE-08	Minimal personal data in logs	decisions
GATE-02	A person approves high-impact actions	approvals	GATE-09	Agents never hold credentials	connectors
GATE-03	Separation of duties on approvals	self-approvals	GATE-10	Retention is set and recorded	settings
GATE-04	Tamper-evident record of every decision	receipts	GATE-11	Access is role-based, changes recorded	member changes
GATE-05	Signed, versioned rulebooks	rulebooks	GATE-12	AI systems are inventoried	systems
GATE-06	Kill switch for every agent	kill switch	GATE-13	Risk assessments approved before go-live	approved assessments
GATE-07	Registry of agents, owners and risk tiers	agents			

Source: [GF §3 "13 controls"]. GATE-09 holds only on the MCP path (R-25).

> **Draft — needs legal review**
>
> No mapping has been reviewed by counsel. Nothing in the product claims compliance, and nothing in this memorandum should. The original's rule on what may lawfully be claimed [TM §15.13] applies to the compliance pages and the framework report without exception.

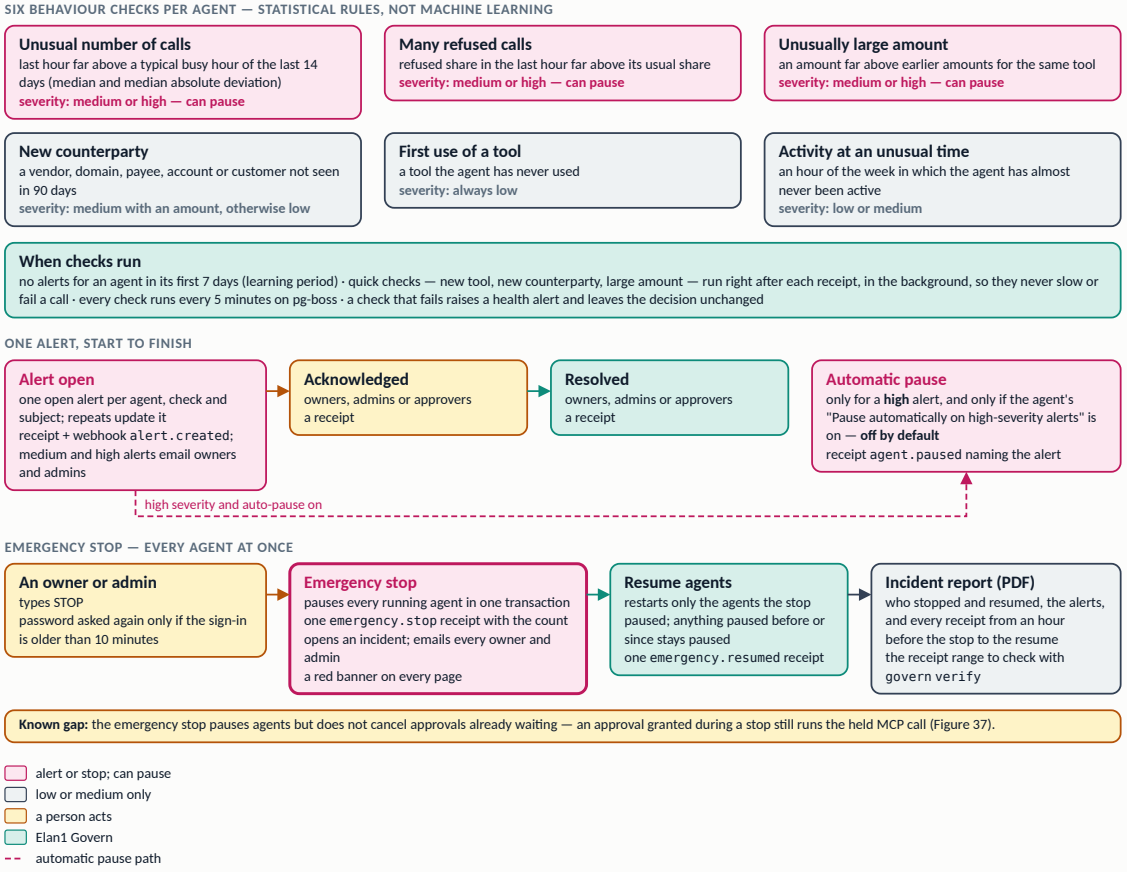
R8. Alerts, auto-pause and the emergency stop

The original's rung 0b stop set [TM §5.2] is built as three things: a per-agent pause, statistical behaviour alerts that can pause an agent, and an organisation-wide emergency stop. All three are at commit `22388ad` [GF §3 area 12].

Figure 39 — Alerts, automatic pause and the emergency stop

BUILT — IN CODE

Elan1 Govern learns what is normal for each agent; a high alert can pause one agent if its owner switched that on; STOP pauses them all



A paused agent is refused at the first step of every call (Figure 34). Only 3 of the 6 checks can raise a high alert, so only those three can pause an agent automatically, and only where its owner switched automatic pause on.

Source: packages/core/src/behaviour.ts (DETECTORS, BEHAVIOUR_DEFAULTS); apps/gate/src/behaviour.ts, incidents.ts, jobs.ts; apps/web/src/app/api/emergency-stop/route.ts; README §Unusual-behaviour alerts and the emergency stop; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 39 — Alerts, auto-pause and emergency stop: detectors run inline after each decision and every five minutes; a high finding pauses the agent only if auto-pause is on; the emergency stop pauses every agent in one transaction and writes an incident and its report. Status: built; auto-pause off by default.

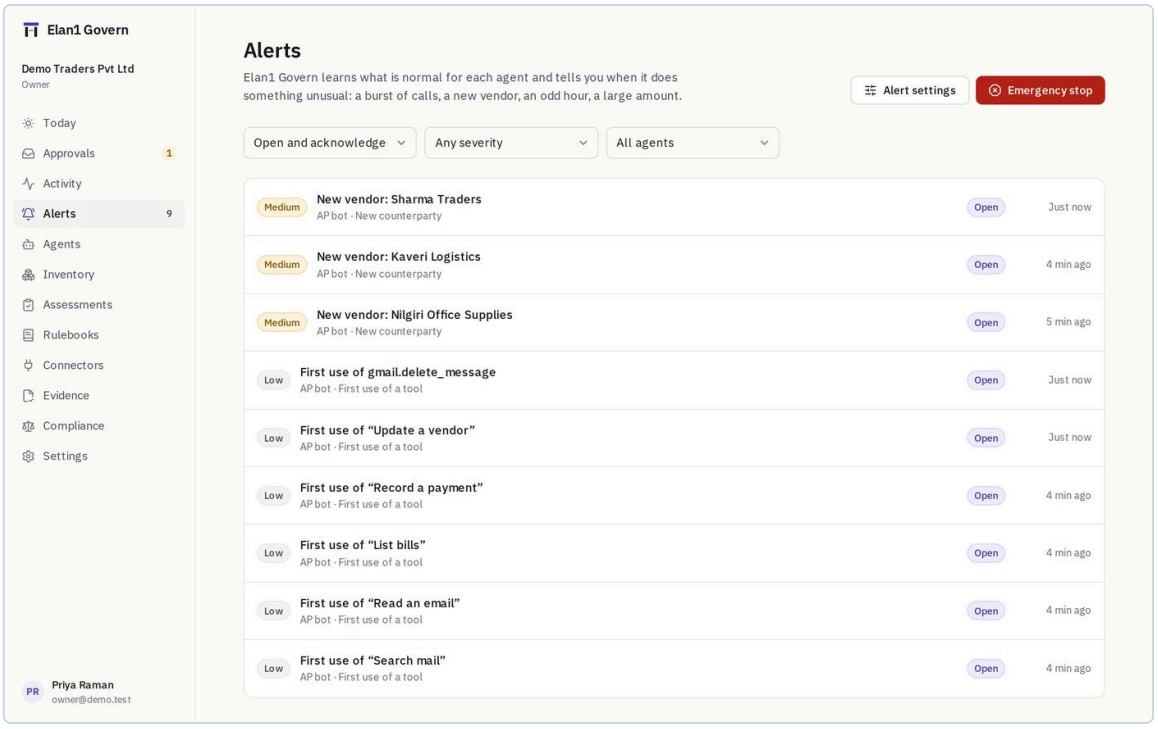
R8.1 Detectors

Detector	Trigger (defaults)	Can reach high and auto-pause
volume_spike	Robust z-score of the last hour's calls; at least 20 calls; medium at 6, high at 12	Yes
refusal_surge	Refused share above baseline; at least 10 calls; medium at 0.3, high at 0.6	Yes
amount_outlier	Robust z-score of the amount; at least 5 samples; medium at 6, high at 15	Yes
new_counterparty	Vendor, domain, payee, account or customer not seen in 90 days	No (medium with an amount, else low)
new_tool	First use of a tool	No (low)

Detector	Trigger (defaults)	Can reach high and auto-pause
off_hours	Hour of week holding at most 0.5% of baseline calls	No (low or medium)

Source: [GF §3 "Alert detectors"]. Learning period 7 days (no alerts), 14-day baseline, 90-day memory. These are statistical rules, not machine learning.

Alerts are deduplicated per agent, detector and key. Owners, admins and approvers acknowledge and resolve them. Each alert is written in the same transaction as its receipt, so an alert never exists without one [GOV DECISIONS.md #94–96]. **Auto-pause** acts only on a high finding **and** only when the agent's `autoPauseOnHigh` switch is on. That switch is **off by default** [GF §3 area 12]. The brief's "it can auto-pause an agent on a serious alert" is true only after someone turns it on.



Alerts: three medium "New vendor" and six low "First use of…" findings, with Alert settings and Emergency stop [GOV docs/screenshots/20-alerts.png]. Seeded demo data; the "New vendor" alert is produced by the normal guard path, not written by hand.

R8.2 The emergency stop

Only owners and admins can stop. Stopping needs the typed word `STOP`, and a password only if the sign-in is more than 10 minutes old. One chain transaction pauses every active agent, creates an `Incident` with the ids it paused, and writes one `emergency.stop` receipt. Only one stop can be open at a time. Resume restarts only the agents the stop paused. The console shows a red banner on every page while the stop lasts. The stop refuses every new call. It does not cancel approvals already waiting: if a person approves a held MCP action during a stop, the action still runs. Expiring or blocking pending approvals on stop is a build item. The incident report is a PDF from the guard covering an hour before the stop to the resume, with every receipt up to 300 rows and the range to export for `govern verify` [GOV DECISIONS.md #97–98; GF §3].

The original specified the stop as a governed write under dual control [TM §8.2]. The built stop needs one person. Recorded in A194. The incident list exists inside Alerts, but an incident detail page does not (R-39).

R9. Identity, roles, organisations and security

The original said Elan1 Govern would "consume identity, differentiate on enforcement" and never build authentication [TM §2.1, §2.7]. The built code has its own sign-in, adds OIDC single sign-on and enforces five roles across organisations. Everything here is at commit `22388ad` [GF §3, §5].

R9.1 Sign-in and single sign-on

Mechanism	Built	Not built
Password	argon2id at library defaults; at least 12 characters; dummy hash for unknown emails; lockout after 5 failures (1 minute, doubling, capped at 1 hour)	Tuned argon2 parameters
Two-step code	Optional TOTP (SHA1, 6 digits, 30 s, ±1 window), secret encrypted	WebAuthn
Single sign-on	Generic OIDC with PKCE, state and nonce; one connection per organisation; domains proved by DNS TXT `gate-verify=…` ; optional SSO-only; users created on first sign-in. Invitations are single-use links	SAML, SCIM; provider-specific code
Fresh sign-in	Approvals, emergency stop and key rotation ask for a password if the sign-in is over 10 minutes old	Step-up per action

Source: [GF §3 area 1, area 3].

R9.2 Roles and organisations

A user has one role per organisation: Owner, Admin, Approver, Viewer or Auditor, across 22 permissions. Everyone can view. Owners, admins and approvers decide approvals. Owners and admins manage agents, rulebooks, connectors, settings, compliance, members, webhooks, notifications, suggestions and alerts, and can trigger the emergency stop. Owners, admins and auditors export evidence. Only owners manage single sign-on and rotate keys [GF §3 "Roles and permissions"]. Any user with several memberships can switch organisations, which is how an auditor such as a chartered accountant works across client companies [GOV DECISIONS.md #17]. Organisations replace the original's tenants.

R9.3 Security posture

Area	Built	Limit or finding
Tenant isolation	`forOrg()` ANDs the organisation into every query on 26 models; a static test (`packages/db/test/isolation.test.ts`) is designed to fail on raw-client access; it was not run in the audit, and CI results could not be seen	No Postgres row-level security [TM §9.3 recommended it]
Secrets	Signing keys, OAuth tokens, upstream headers and webhook secrets under AES-256-GCM with one environment master key; the console cannot sign	No KMS or HSM; one master key
Web	CSP with per-request nonce and `strict-dynamic` ; frame-ancestors none; HSTS; double-submit CSRF cookie plus same-origin check	ZAP accepted `style-src 'unsafe-inline'` and a non-HttpOnly CSRF cookie (self-reported)
Rate limits	Guard: 300 a minute per API key on `/mcp` and `/v1/*` . Console: per-route limits	Guard and console limits in memory, per process; multi-instance running untested; guard hard-codes `trustProxy: true`
Privacy	Minimal receipts; replay store purged at 30 days; WhatsApp numbers encrypted with keyed-hash matching	No erasure, crypto-shredding or salts [TM §9.12] (A203); no residency control
Supply chain	pnpm 10 with 7-day minimum release age, trust policy, install-script allowlist; CI actions pinned to SHAs	No LICENSE file anywhere

Area	Built	Limit or finding
Scanners	`pnpm audit` (run by the audit: no known vulnerabilities); semgrep, gitleaks, OWASP ZAP baseline in CI	Other results self-reported in SECURITY.md (STRIDE model over 14 areas); not re-run; no independent review or penetration test

R10. Developer surfaces and the two modes

The original specified Mode A (the in-process SDK, enforced) and Mode B (MCP or HTTP, advisory), with KernelPort behind both [TM §6]. The built surfaces differ, and the enforcement mapping is the other way round.

R10.1 What an agent calls

Surface	Built
MCP endpoint	`POST /mcp`, stateless Streamable HTTP. Exposes the governed tools themselves plus `gate.approval_status`. A refusal is a tool result with `isError: true` carrying the decision, rule id and reason, so the model sees why [GOV DECISIONS.md #12]
REST Event API	`POST /v1/check` (returns allow, ask with an approval id and link, or refuse — a refusal is a 200, not a 4xx), `POST /v1/record` (outcome of an allowed check; a duplicate is 409), `GET /v1/approvals/{id}`
OpenAPI	OpenAPI 3.1 generated from the same zod schemas, served at `/v1/openapi.json` and checked in; 5 paths. It covers only the agent-facing API, not the console or internal API
TypeScript SDK	`check()`, `record()`, `waitForApproval()`, `verifyWebhook()` (95 lines)
Python SDK	`elan1_govern`, standard library only. `guarded` decorator; fails closed (no usable answer, a timeout or a redirect raises `GovernError`); `RecordError` carries the result if recording fails, so a payment is not run twice
CLI	`govern verify | sign | keygen` (`gate` kept as an alias)
Webhooks	Envelope with `receipt.created`, `approval.pending`, `approval.resolved`, `alert.created`. Header `gate-signature: t=…,v1=HMAC-SHA256`; up to 8 attempts; private addresses blocked in production. No outbox
Examples	LangChain, OpenAI Agents SDK, plain Python; MCP set-up for Claude Desktop, Claude Code and Cursor

Source: [GF §3 "Exist only in code", §4; GOV DECISIONS.md #61–63]. There are no idempotency keys on `/v1/check` or `/mcp` [GF §4]; see T29. None of these surfaces has a screen yet. The Developer area is KEEP (R-39).

R10.2 The two modes, renamed

BUILD RECOMMENDATION The modes take the names of what actually happens.

	Governed run	Check and record
Surface	MCP proxy (built-in connectors and upstream MCP servers)	REST Event API and the TypeScript and Python SDKs
Who runs the tool	Elan1 Govern, with credentials only it holds	The agent, with its own credentials
What it is	**Enforcing for that tool**, provided the agent has no other route to the system; network-proxy enforcement, not in-process	**Advisory.** An agent that skips `check()` is not stopped, and is seen only if it later calls `record()`
Receipt timing	Allowed and approved actions run **before** their receipt (DECISIONS #15 gap, R-12)	Decision receipt first; outcome receipt if the agent records
Corpus term	Enforced mode (the spec tied it to Mode A, the in-process SDK)	Attested advisory mode (the spec tied it to Mode B, MCP or HTTP)

Source: [GF §4 "Can an agent bypass Govern?"].

The sales rule, re-pointed

The original rule stands: advisory is never called enforcement [TM §6.8]. It now applies to **check and record**. Copy may not say "blocked", "prevented" or "cannot" of an SDK or REST integration. Even a governed run is enforcing only for tools reached through Elan1 Govern, and the code cannot stop an agent that also holds a direct credential. There is no in-process enforced mode [GF §4]. The original tied Mode A to the SDK and Mode B to MCP; the code inverts that pairing. Recorded as P21.

R11. How the built code was checked

This chapter says how much weight the "built" marks in this supplement can bear. Every statement comes from the code audit of 27 October 2026, which read the code and ran the tests that could run at commit `22388ad` [GF].

R11.1 Provenance

- **Author and designer.** Per the brief, Hasif is the author of the codebase (`github.com/Hasif-Elan1/elan1-govern`), and Ashoke is the designer of the elan1 screen system (`github.com/Ashoke-elan1/elan1-govern`) [HB]. Their roles, terms and any assignment of intellectual property to elan1 Private Limited are open items (Q117).
- **History.** 47 commits, 9 of them merges, from 26 October 2026 18:39 IST to 27 October 2026 10:10 IST — about 15.5 hours. All are under one identity, `elan1 <hello@elan1.ai>`, with no commits from a personal identity. The owner's guide reads as the output of an AI coding-agent session on one machine [GF §1].
- **Review.** There has been no independent code review and no independent security review. The scanner results other than `pnpm audit` are self-reported, and CI status on GitHub could not be seen [GF §5].
- **Licence.** There is no LICENSE file and no licence field in any package. Ownership of the code is open [GF §9 item 11].
- **Data and connectors.** All 23 screenshots and every example here come from seeded demo organisations (Demo Traders Pvt Ltd, Patel Agro Foods) [GF §7]. The Gmail and Zoho Books connectors have never been run against real accounts, and `docker compose up` is untested [GF §6].
- **Customers.** No customer, pilot or design partner uses Elan1 Govern, and there is no deployment beyond a developer machine [GF §8].

R11.2 Tests, as the audit observed them

Suite	Observed in the audit sandbox, 27 Oct 2026
Vitest (unit and integration)	**397 passed, 1 failed, of 398 collected.** 41 test files: 21 passed and 20 failed. 19 of 41 test files could not load, because they need the database and the Prisma engine download was blocked; a 20th file ran with the one failure, which needs a `.env` file and passes when the example values are loaded
Per package	core 245/245 · cli 6/6 · sdk 8/8 · connectors 18/18 · web 118/118 (the 3 of 9 files that need no database) · gate 3/3 (environment tests)
Python SDK	**40/40 passed**
Static counts	437 Vitest `it(` / `test(` statements in 41 files; 6 core files use fast-check property tests; **23** Playwright tests in 9 specs (static count)
Not run	Database-backed integration tests, Playwright end-to-end tests, coverage, lint, typecheck
Unverified	The owner's guide claims "580 automated tests … plus 23 browser tests". The 580 cannot be confirmed without a database; the 23 browser tests exist but were not run

Source: [GF §2 "Tests"]. These are sandbox observations of a test suite, not measurements of the product. No latency, throughput or cost of Elan1 Govern has been measured.

R11.3 What a diligence team should run first

These checks replace items 3, 4, 8 and 9 of the original list [TM §1.7, §23.8]. The other items stand as specification checks.

1. Clone at `22388ad` and run `pnpm demo`. Watch an allow, an ask and a refuse, then export an auditor pack.

2. Run `govern verify` on the pack. Run it again with `--key` pointing to the organisation's public-key PEM file obtained separately, or compare the printed key fingerprint with one obtained separately. Then edit one receipt and confirm the verifier names it.
3. Run the database-backed suites against PostgreSQL 16, and check the claimed 580 tests and the Playwright specs.
4. Read the isolation and authorisation-matrix tests and try a cross-organisation request; force a receipt-write failure after an allowed MCP action; send concurrent payments against a daily limit.
5. Connect a real Gmail or Zoho Books sandbox account, which nobody has done yet.
6. Ask the founder for the commit identity, review history, CI runs and an IP assignment.

R12. Spec-to-code crosswalk

The original "what is built" table [TM §1.5] and the prototype reading [TM §3.8] are replaced by this crosswalk. Each row is a concept from the frozen specification, its built counterpart at commit `22388ad`, and its status [GF §4]. The specification remains the target: "Specified" means frozen specification with no code yet, not withdrawn.

Figure 41 — Spec-to-code crosswalk

BUILT vs SPECIFIED — AS AT 27 OCT 2026

Each concept of the September specification beside its Elan1 Govern name and its place in the code (paths under the Elan1 Govern repository)

SPECIFIED (SEPT 2026 EDITION)	ELAN1 GOVERN — BUILT NAME	WHERE IN THE CODE	STATUS
Declaration, seven blocks (Fig 9)	Rulebook — one per agent, YAML, signed	`core/src/rulebook.ts` · `rulebook-sign.ts`	BUILT DIFFERENTLY — the seven-block ontology is not in the code
Write type	Tool + 8 risk tags	`core/src/tools.ts`	BUILT DIFFERENTLY — every call, reads included
Studio: simulate · certify	Try an action · Test on past actions · Sign and publish	`core/src/replay.ts` · `gate/src/replay.ts`	BUILT
Proof record — L1 hash chain (Fig 13)	Receipt: SHA-256 + Ed25519 signature	`core/src/receipt.ts` · `db/src/chain.ts`	BUILT — signed today; the L1 spec was unsigned
Chain standing	Receipt chain + hourly signed checkpoints	`db/src/chain.ts` · `gate/src/jobs.ts`	BUILT
Evidence bundle + offline verifier (Fig 15)	Auditor pack + `govern verify`	`core/src/bundle.ts` · `packages/cli`	PARTIAL — exit codes 0/1/2, not six; no verifier profiles
Approval obligation (Fig 11)	Approval: pending · approved · rejected · expired	`gate/src/approvals.ts`	BUILT DIFFERENTLY
WhatsApp approval	Slack and WhatsApp approvals	`gate/src/chat/` · `core/src/chat.ts`	BUILT
Stop switch — rung 0b	Emergency stop + per-agent pause	`web api/emergency-stop` · `gate/src/incidents.ts`	BUILT
Agent registry · Tools catalogue	Agents · Tool registry	`web (app)/agents` · `core/src/tools.ts`	BUILT
Connected systems	Connectors: Gmail, Zoho Books, Your MCP servers	`packages/connectors` · `gate/src/upstream.ts`	BUILT — live Gmail and Zoho never run against real accounts
Controls & regulations (Fig 10)	Compliance: 10 frameworks, 13 controls	`core/src/frameworks/`	BUILT — Draft — needs legal review
Regulatory packs (Fig 18)	Industry packs (7) — a different thing	`core/src/frameworks/packs.ts`	BUILT DIFFERENTLY — regulatory packs ledger1, invoice1, collect1 remain specified
Tenants	Organisations	`db/src/tenant.ts (forOrg)`	BUILT — app-level isolation; no Postgres RLS
MCP server, 34 tools in three scopes (spec)	MCP server `/mcp`: built-in, upstream and status tools	`gate/src/mcp.ts`	PARTIAL — not the specified 34-tool set
KernelPort v1 — six calls (Fig 5)	MCP `/mcp` + REST `/v1/*` (OpenAPI 3.1)	`gate/src/mcp.ts` · `routes/agent-api.ts`	BUILT DIFFERENTLY — no single port abstraction
Nine rungs (Fig 6)	Eight-step guard (Figure 34)	`gate/src/guard.ts`	PARTIAL — no write door, autonomy or metering rung
Mode A enforced / Mode B advisory (Fig 8)	Governed run (MCP proxy) enforces · Check and record (SDK/REST) is advisory	`gate/src/guard.ts (GuardMode)`	BUILT DIFFERENTLY — no in-process enforced mode
Single writer holds the target credential	Govern holds connector credentials in a Governed run	`gate/src/connectors.ts` · `upstream.ts`	PARTIAL — nothing stops an agent holding its own
Durable execution — Temporal	pg-boss jobs in Postgres	`gate/src/jobs.ts`	BUILT DIFFERENTLY
Seal Spec v6, 64 vectors, two implementations (Fig 14)	—		SPECIFIED — frozen spec; no code
RFC 3161 two-authority anchoring (Fig 17)	— (checkpoints are self-signed)		SPECIFIED
KMS / HSM key custody (Fig 16)	One env master key wraps org keys	`gate/src/keys.ts (KeyProvider)`	SPECIFIED — a code seam only
Earned autonomy (Fig 23)	nearest: rule suggestions, published by a person	`core/src/suggest.ts`	SPECIFIED
Metering and billing (Fig 28) · assurance levels and L3 (Figs 21–25) · residency, erasure, idempotency keys	—		SPECIFIED

WHERE THE BUILT CODE SITS AGAINST THE SEPTEMBER ASSURANCE LADDER



Where the code and the September specification differ, the code is what exists and the specification is what is planned. SPECIFIED means a frozen specification with no code. Status follows the code audit of 27 October 2026 (commit 22388ad).

Source: code audit of commit 22388ad (27 Oct 2026); Hasif brief 27 Oct 2026 (name table); docs/ARCHITECTURE.md; Atlas Figures 5–28 (Sept 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 41 — Spec-to-code crosswalk: each specification concept against its built counterpart, coloured built, partial, specified or absent. Status: as at commit 22388ad.

Specification [TM]	Built counterpart	Status
KernelPort, six calls (§4)	No port. Analogues: define ≈ sign and publish a rulebook; assert ≈ `guard()` / `decide()`; seal ≈ `buildReceipt`; verify ≈ `verifyChain` / `govern verify`; scope ≈ `forOrg`; refuse ≈ refusal receipts	SPECIFIED
Five objects (§4.3)	Tool call, decision, approval, receipt; no meter event	PARTIAL
Nine rungs (§5)	Eight-step guard; write door, autonomy and meter absent (R-10)	PARTIAL
Fail closed (§5.15)	Yes in the guard; act-before-receipt gap in governed runs	PARTIAL
Idempotency keys (§5.14)	None; duplicate `record()` refused	SPECIFIED

Specification [TM]	Built counterpart	Status
Durable execution (§5.17)	pg-boss queues and cron; no workflow engine on the action path	PARTIAL
Mode A / Mode B (§6)	Governed run (MCP) and check and record (SDK, REST); no in-process enforced mode	PARTIAL
MCP server, 34 tools (§6.5)	MCP proxy exposing 10 built-in tools, enabled upstream tools and `gate.approval_status`; no scopes, descriptions not pinned	PARTIAL
Declaration, seven blocks (§7)	Rulebook YAML, one per agent	PARTIAL
Simulate / certify (§7.11)	Try an action; Test on past actions; sign and publish	BUILT
Controls and Regulations (§7.8)	10 frameworks, 13 controls, coverage from receipts (draft)	BUILT
Refusal register, 43 codes (§4.6)	6 system rule ids plus customer rule ids	SPECIFIED
Console, Four Doors compiled (§8.1)	Thirteen hand-built areas, 27 pages	PARTIAL
Approval and obligation (§8.3–8.4)	Approval with four states, links, expiry; run on approval	BUILT
Dual control, quorum, delegation (§8.5)	None; self-approval flagged	SPECIFIED
Decide screen diff and gates (§8.6)	Approval detail with evidence; no diff or gate list	PARTIAL
WhatsApp approval (§8.8)	Slack and WhatsApp buttons on chat-approvable rules	BUILT
Hash-chained proof log (§9.7)	Per-organisation receipt chain, signed	BUILT
Meter, refusals at zero (§9.8, §19)	None	SPECIFIED
Row-level security (§9.3)	Application-level `forOrg` plus static test	PARTIAL
Seal Spec v6, 64 vectors (§10)	Receipt format, one implementation, no vectors	SPECIFIED
Two-implementation rule (§10.9)	Verifier shares `packages/core` with the writer	SPECIFIED
Bundle and offline verifier (§11)	Auditor pack; `govern verify`, 3 exit codes	PARTIAL
Verifier profiles (§11.7)	None	SPECIFIED
Key custody, KMS/HSM (§12.4)	`KeyProvider` seam; environment provider only	SPECIFIED
Key rotation (§12.6)	Owner rotation, `key.rotated` receipt, retired keys verified	BUILT
RFC 3161 anchoring (§12.13)	None; checkpoints self-signed	SPECIFIED
Regulatory packs (§13)	None; seven industry packs instead	SPECIFIED
Level 3: cross-organisation, disclosure, earned autonomy (§14)	None; rule suggestions are the nearest analogue	SPECIFIED
Assurance levels (§16)	No level concept in code	SPECIFIED
Residency, sovereign deployment (§17.6, §18.7)	Regions select frameworks only	SPECIFIED
DPDP erasure (§9.12, §18.9)	Minimal receipts; replay store purge; no erasure	SPECIFIED
Identity (§18.3)	Own auth, TOTP, OIDC SSO, five roles	BUILT
Conformance suite C1–C14 (§4.8)	Internal tests only; no published suite	SPECIFIED

Source: [GF §3–4; SM]. "Built" means present in the repository at commit `22388ad` and confirmed by an independent read of the code; the demo was not run in the audit. It does not mean shipped or reviewed (R-34).

R13. Vocabulary — one name per concept

The built name wins, because it is what ships; the specification's meaning is kept where it differs [SM (f)]. This table also governs the revised IM, Atlas and financial model.

Concept	One name now	Earlier names and notes
The product	Elan1 Govern	elan1 (product), the elan1 kernel or layer; entity stays elan1 Private Limited. Internal `gate` names stay in code only [GOV DECISIONS.md #60]
Policy object	Rulebook; rule builder	Declaration, Declaration Studio. The seven-block Declaration remains the L2 specification
What is governed	Tool (with risk tags)	Write type
The three outcomes	Allow · ask a person · refuse	allow · escalate · refuse; needs_approval. Wire values `allow`, `ask`, `refuse`
Waiting human decision	Approval: waiting, approved, rejected, expired	Obligation, escalation; decline
Testing a draft	Try an action · Test on past actions	Studio: simulate
Making a draft live	Sign and publish	Studio: certify, compile
The record of a decision	Receipt; Activity is the list	Proof record, proof log. "Action Receipt" stays the standard's name; design "Action receipts" become "Counterparty receipts (L3)"
Chain integrity	Receipt chain and signed checkpoints	Chain standing
What an auditor receives	Auditor pack (bundle, PDF, compliance map)	Evidence bundle; the eight-member Seal v6 bundle remains the L2 specification
The offline checker	`govern verify`	Offline verifier, elan1-verify
Stopping	Pause (an agent) · emergency stop (all agents)	Stop switch, stop set, rung 0b
External systems; agents	Connectors, incl. "Your MCP servers"; agents; tool registry	Connected systems, adapters; agent registry; MCP catalogue
Starter set by industry	Industry pack	Never the bare word "pack"
Regime-specific set	Regulatory pack: ledger1, invoice1, collect1	Pack. All three specified, none built
Compliance	Compliance → frameworks and controls	Controls and Regulations. Always "Draft — needs legal review"
Customer	Organisation	Tenant. Auditors switch between client organisations
Agent risk	Risk tier (from approved assessments)	Autonomy A0–A5 stays a Level 3 specification
Modes	Governed run · check and record	Mode A / Mode B; Enforced / attested advisory (R-32)
Billing unit	Governed action: one tool call decided by Elan1 Govern	Committed governed write. No meter is built; the unit is a proposal pending A197
Levels	L1 Guarded · L2 Attested · L3 Proven (unchanged)	Elan1 Govern is at most L1, and partial (no meter; enforcement on the MCP route only). Do not call it L2 or L3 (R-17)

R13.1 Claim language

- **Allowed**, with the conditions in R-16: "signed", "hash-chained", "tamper-evident", "checkable offline by anyone with the organisation's public key".
- **Forbidden** until L2: "tamper-proof", "immutable", "court-ready", "independently verifiable", "without trusting us". Lower-case brand names remain the rule for sister brands; "Elan1 Govern" is the one exception [FU].
- **Check and record**: never "blocked", "prevented" or "cannot". **Compliance**: never "compliant"; always "Draft — needs legal review".

R14. One information architecture — thirteen areas

The original console presented Four Doors — Work, Records, Agents, Assurance — and screens compiled from declarations [TM §8.1]. The built console has thirteen areas of hand-written pages. Ashoke's 275 design screens have been mapped onto those areas, so that every screen can be built on working code [SM].

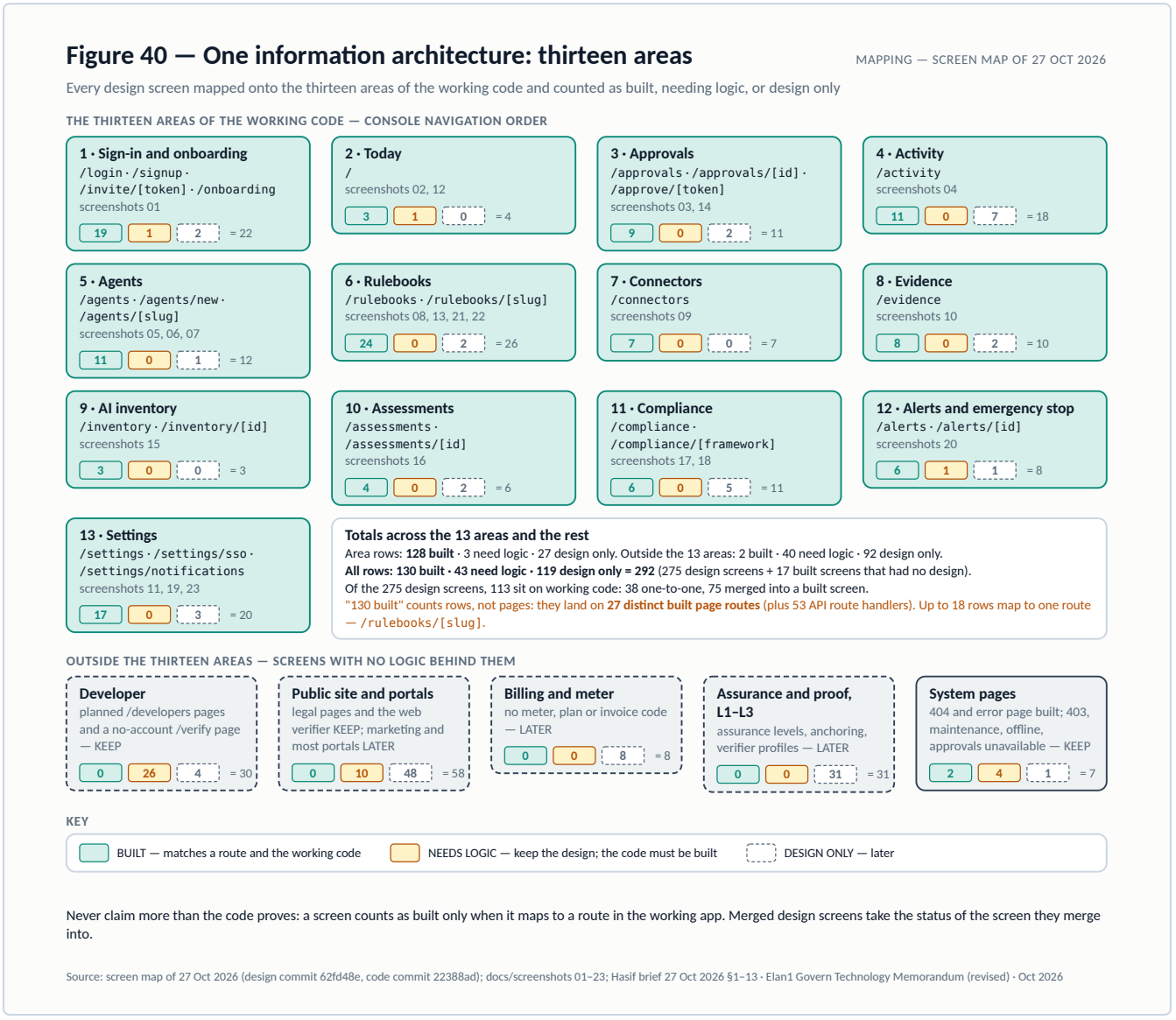


Figure 40 — One information architecture: thirteen areas of the console, with the developer, public, billing, assurance and system groups outside it; each area coloured by how many of its design screens are built, need logic or remain design only. Source: screen map at design commit 62fd48e and code commit 22388ad.

R14.1 Totals

Of 275 design screens, **113 sit on working code**: 38 map one-to-one to a built route, and 75 are merged into a built screen. A further 26 are KEEP (build next) and 116 are LATER. The remaining 20 merge into screens that are not built yet (17 need logic, 3 design only). Seventeen built screens have no design yet. The built code has **27 distinct page routes**; the "130 built" in the table counts rows, not pages [SM (a)].

Area	Design rows	New rows	Built	Needs logic	Design only	Total
1 Sign-in and onboarding	21	1	19	1	2	22

Area	Design rows	New rows	Built	Needs logic	Design only	Total
2 Today	4	0	3	1	0	4
3 Approvals	11	0	9	0	2	11
4 Activity	18	0	11	0	7	18
5 Agents	10	2	11	0	1	12
6 Rulebooks	23	3	24	0	2	26
7 Connectors	6	1	7	0	0	7
8 Evidence	10	0	8	0	2	10
9 AI inventory	1	2	3	0	0	3
10 Assessments	3	3	4	0	2	6
11 Compliance	11	0	6	0	5	11
12 Alerts and emergency stop	6	2	6	1	1	8
13 Settings	18	2	17	0	3	20
Developer	30	0	0	26	4	30
Public site and portals	58	0	0	10	48	58
Billing and meter	8	0	0	0	8	8
Assurance and proof (L1–L3)	31	0	0	0	31	31
System pages	6	1	2	4	1	7
Total	275	17	130	43	119	292

Source: [SM (a)], output of the totals script on `screen-map.csv`, unedited. Status rules: BUILT disposition → Built; KEEP → Needs logic; LATER → Design only; MERGE takes the status of its target.

R14.2 The thirteen areas

Area	Routes (built)	Gaps
1 Sign-in and onboarding	`/login`, `/signup`, `/invite/[token]`, `/onboarding` (company → industry and regions → demo); organisation switcher in the shell	Password recovery (KEEP); no currency or time-zone inputs
2 Today	`/` : waiting for me, today's numbers, live activity, suggestions, alerts, chain status	Due-dates panel (merged obligations calendar, needs logic)
3 Approvals	`/approvals` (waiting, decided), `/approvals/[id]`, `/approve/[token]`	No "decided in Slack/WhatsApp" label; no re-routing
4 Activity	`/activity` : receipts, filters, "Verify this receipt"	—
5 Agents	`/agents`, `/agents/new` (3 steps), `/agents/[slug]` (owner, risk, keys, rulebook, pause, baseline, auto-pause)	No automatic discovery
6 Rulebooks	`/rulebooks` (with suggestions), `/rulebooks/[slug]` (builder, tags, limits, try, test on past actions, versions, diff, sign and publish)	—
7 Connectors	`/connectors` (Gmail, Zoho Books, Your MCP servers)	—
8 Evidence	`/evidence` (auditor pack, checkpoints, compliance map)	Export-history list unconfirmed
9 AI inventory	`/inventory`, `/inventory/[id]`	No model field
10 Assessments	`/assessments`, `/assessments/[id]`	—
11 Compliance	`/compliance`, `/compliance/[framework]`	Verifier-profile screens LATER
12 Alerts and emergency stop	`/alerts`, `/alerts/[id]` ; stop button in the shell	Incident detail page (KEEP)

Area	Routes (built)	Gaps
13 Settings	`/settings`, `/settings/sso`, `/settings/notifications`	—

Source: [SM (b)]. Outside the console: a Developer area (KEEP, public documentation for the SDKs, OpenAPI, MCP, webhooks and CLI, plus a no-account web verifier); System pages (404 and error built; 403, maintenance, offline and "approvals unavailable" KEEP). Pages have no loading states yet: there is no `loading.tsx` [SM (g) 3].

R14.3 Screens with no logic behind them

Design item	Verdict	Reason
Public site	LATER	Marketing copy must be rewritten for Elan1 Govern; legal pages (privacy, terms, DPA, cookies) are KEEP because sign-up is live
Verifier portal	KEEP	The proof story: a no-account web verifier on top of `govern verify`
Auditor portal	MERGE	Auditor role and organisation switcher already work inside the app
Regulator, counterparty, partner portals; trust centre	LATER	Depend on Level 3 or a partner programme
Developer portal	KEEP	SDKs, OpenAPI, MCP, webhooks and CLI exist with no screen
Assurance levels, L2 upgrade wizard, anchoring, verifier profiles	LATER	Nothing to upgrade to: anchoring, profiles and L3 are not built
Meter, invoices, plan	LATER	No billing code
Erasure requests; lexicon; objects and changesets; residency; agent map; break-glass	LATER	No model in code; break-glass would weaken the "no admin mode" claim before dual control exists
Autonomy and graduation; escalations; discovered agents; review queue	MERGE	Into rulebook outcomes and rule suggestions; the Approvals "Decided" list; the AI inventory; assessment sign-off
Obligations calendar	MERGE (needs logic)	Due-dates panel on Today; the data exists, the view does not
Incidents	KEEP (detail)	The list exists under Alerts; the incident record page is missing
System pages	KEEP	An honest "approvals unavailable" page for a fail-closed product

Source: [SM (c)].

R14.4 New screens, and the states every screen must show

Seventeen built screens have no design yet: AI inventory and AI system; assessments, assessment and assessment sign-off; rule suggestions; test on past actions; the risk-tag rule builder; alerts with auto-pause and the alert page; Your MCP servers; single sign-on set-up; multi-industry onboarding; the three-step Add agent; the replay-store switch; the agent behaviour baseline; and the error page [SM (d)].

Two sets of states apply wherever they occur:

- **Approval:** waiting (with expiry), approved, rejected, expired (the action is refused), and decided on the web, by email or phone link, on Slack or on WhatsApp.
- **Decision:** allow, ask a person, refuse — where a refusal can come from a rule, from no matching rule, or from an Elan1 Govern failure.

Ashoke's design tokens and proof language are kept [HB].

R15. Revised build plan and what remains

The original plan began with signing six specification documents and reached signing in Q3 2027 [TM §1.6, §21]. The code took the other road: product first, specification later. This chapter records what is done, what remains, and which gates the founder must re-set. **It proposes no dates.** Every original date — 31 October, 30 November and 31 December 2026; L1 GA 30 June 2027; L2 30 April 2028 — predates the code and is "to be re-set" until the founder signs new ones (A204).

R15.1 Done, as at commit `22388ad`

The repository's plan lists 14 phases, all ticked: scaffold; core; guard service; console; demo, seed and live connectors; security; polish; foundations (tags, tag rules, quotas); any MCP server; inventory and assessments; frameworks and industry packs; platform (jobs, webhooks, OpenAPI, key rotation); single sign-on; and test on past actions. Slack and WhatsApp approvals, rule suggestions, alerts and the emergency stop, and the Python SDK were built after that and are not in the plan file [GF §6].

R15.2 What remains

Workstream	Items	Gate it serves
Make the L1 claim true in the field	Independent code and security review; database-backed tests and CI made visible; live Gmail and Zoho runs; tested deployment; IP assignment and licence; the first design partner	Any external use
Close L1 gaps in the guard	Write the receipt before an allowed or approved governed run, or an outbox (DECISIONS #15); expire or block pending approvals on emergency stop; limits inside the chain lock; idempotency keys; shared rate limits; `trustProxy` setting; webhook outbox; a closed refusal-code register	Payment use cases
Approval controls	Dual control and quorum for high-impact rules and the emergency stop; delegation and re-routing; a decision to forbid or keep self-approval; channel labels	Original §8.5 bar
Product completion	KEEP screens: developer portal and web verifier, incident detail, system pages, password recovery, legal pages, due-dates panel; onboarding currency and time zone; loading states	Self-serve onboarding
Enterprise	SAML and SCIM; KMS or HSM key provider; more connectors (Slack, Salesforce, Jira, Tally, Razorpay); rulebooks as code in CI; SOC 2 Type I [GOV docs/PRODUCT.md]	Procurement
The L2 bar (unchanged)	Key custody per residency domain; two-authority RFC 3161 anchoring; the record-format decision (P20); a second, independent implementation and the 64 vectors; a verifier with the specified exit codes; verifier profiles; out-of-band key publication	Any L2 claim
Commercial plumbing	Meter and billing for the governed action (A197)	First invoice
Scope decisions	Whether rung1, ERPNext and the December gate remain; whether stack1 remains the runtime (P18, P19)	Founder

Ordering within each row is a BUILD RECOMMENDATION. The workstreams follow the original four [TM §1.6]: kernel engineering becomes guard engineering; evidence and cryptography owns the L2 bar; pack and regulatory engineering owns industry and regulatory packs; platform, SRE and assurance owns review, CI, deployment and the security baseline.

> **The original gates, as they now stand**
>
> "Six coding blockers signed by 31 Oct 2026": the code was written before them. "Record shape frozen 30 Nov 2026": open (P20). "First real governed write in a design partner's ERPNext with the bypass failing, 31 Dec 2026": not built, and ERPNext is not a connector. Whether it survives is P18. L1 GA, signing, anchoring and the funding-tranche gates [TM §16.8]: to be re-set by the founder.

R16. New technical risks

These risks continue the Chapter 23 register [TM §23.2], which ends at T22. Likelihood and impact are this memorandum's judgement BUILD RECOMMENDATION. The original risks stand. T5 (the L1 chain rewritten) and T8 (a key-holder insider) now describe the built system directly.

#	Risk	L	I	Response	Trigger
T23	Claim overreach: "without trusting us", "enforced" or "compliant" used of the built product	H	H	Sign the L1 claim sentence (P17); the claim-language rules in R-38; review every external document	Any such phrase in copy
T24	The key holder rewrites history: master key and organisation keys in software; checkpoints self-signed; nothing anchored	M	H	KMS/HSM provider; publish fingerprints out of band; anchoring at L2	A buyer or auditor asks who can re-sign
T25	A fabricated bundle verifies, because the verifier trusts the bundle key	M	H	Make `--key` or a fingerprint check the documented default for auditors	An auditor runs verify without a key
T26	An allowed or approved governed run happens with no receipt (DECISIONS #15)	L	H	Receipt-before-act or outbox; nightly reconciliation of health alerts	Any `receipt-write-failed` health alert
T27	Concurrent calls jointly exceed a daily money limit	M	H	Evaluate limits inside the chain lock	First payment customer
T28	Check-and-record integrations sold or read as enforcement	M	H	Mode naming and sales rule (R-32); governed run as default for money-moving tools	An SDK customer claims an agent "could not" act
T29	Retries double an action: no idempotency keys	M	M	Idempotency keys on `/mcp` and `/v1/check`	A duplicate payment in testing
T30	Unreviewed, agent-written code: one identity, 15.5 hours, no independent review	H	H	Independent code and security review before any customer data	Before first design partner
T31	IP is unclear: no licence; code under developers' GitHub accounts	M	H	Assignment to elan1 Private Limited; licence decision	Term sheet
T32	Connectors fail on first live use (never run against real Gmail or Zoho)	H	M	Live sandbox accounts in CI	First live connection
T33	Single-process limits: in-memory rate limits; untested container deployment; no residency	M	M	Shared store; deployment profile; region decision	Second instance or first non-Indian customer
T34	Two record formats and two plans diverge (built receipt vs Seal v6; built stack vs specified stack)	H	M	Founder decisions P19 and P20; one plan of record	30 Nov 2026 freeze date

Sources: [GF §3–5, §8–9]. L = likelihood, I = impact (H, M, L).

R17. Conflicts, proposed decisions and open items

Appendix A records where sources disagree in a way that changes the build. The code is now a source, and it disagrees with the frozen specification in several places. None of the entries below is a decision the Company has recorded. Each is a PROPOSED DECISION — UNSIGNED or an open item, and each names who must close it.

R17.1 Principal conflicts added

#	Topic	What the sources say	Proposed treatment	Close by
P17	The L1 claim and "signed"	E01, E02 C14, DR-006, E12, E16: L1 is unsigned, holds no key and never says "signed". Code and [HB]: Ed25519 receipts and checkpoints; "verify … without trusting us"	Sign the sentence on R-17 (= DR-017 in the IM); "signed" allowed with the key holder named; "without trusting us" withdrawn; the level stays L1	Founder
P18	Scope and first consumer	E-series: writes into a system of record, ERPNext and rung1 first, stack1 runtime. Code: every tool call, any industry, Gmail, Zoho Books and MCP; no rung1 or stack1 dependency	The built code is the current L1 implementation; the founder decides whether the ERPNext gate, rung1 and stack1 remain	Founder with rung1 and stack1 leads
P19	Stack, repositories, durable execution	ETS, E-series: Python, Go verifier, eight repositories, Temporal on stack1 (DR-002). Code: TypeScript monorepo, pg-boss, verifier sharing `packages/core`	The built stack is the stack of record for L1; an independent verifier is an L2 build item	Founder, evidence lead
P20	Two record formats	E12: Seal v6, frozen record shape 30 Nov 2026. Code: 22-field canonical-JSON receipt	Built receipt is the L1 record; Seal v6 the L2 target; the freeze is the founder's decision — left open	Founder, evidence lead
P21	Modes inverted	O03, E05: SDK in-process is enforced (Mode A); MCP/HTTP advisory (Mode B). Code: MCP proxy enforces; SDK/REST is advisory; no in-process mode	Governed run / check and record (R-32); sales rule re-pointed	Founder, sales
P22	Check/record vs KernelPort	E02: six calls, five objects, frozen 30 Nov 2026. Code: `/mcp`, `/v1/check`, `/v1/record`, `/v1/approvals`	The built API is the L1 interface; KernelPort the target the API should grow into, or be retired	Founder, guard lead
P23	Console model and Gate 2	E06, O09: Four Doors compiled from declarations, zero screen files; Gate 2. Code: hand-built thirteen areas	Thirteen areas govern the IA; Gate 2 re-scoped or retired	Founder, design

R17.2 Register entries added and updated

#	Topic	What the sources say	Proposed treatment
A194	Approval semantics	E04, E06: resubmission, 4-hour expiry, proposer ≠ approver, dual control, stop under dual control. Code: run on approval, 24-hour default, self-approval flagged, single approver, one-person stop	Record as built; dual control and quorum are build items; decide on self-approval. Updates P8
A195	Two-implementation rule	E12, IM §7.3: the verifier shares no code with the writer. Code: both in `packages/core`	Holds for any L2 claim; not met today
A196	Signing keys (updates P2)	E16: one key per residency domain in KMS. O05: per-tenant keys. Code: per-organisation keys under one environment master key	Per-organisation keys stand; a KMS/HSM provider before any external claim
A197	Billing unit	O01: a committed governed write. Revised IM and financial model (working definition): one tool call decided. Code: no meter; reads are governed too	Founder to fix the unit (decisions, or writes only) before the financial model relies on it
A198	Internal names	O10: retired names do not reappear; E05 header `X-Elan1-Signature`. Code: `gate` internally, `gate-signature` header; "governance platform" in docs (O01 retired "platform")	Internal identifiers accepted, never in copy; "platform" left to the founder
A199	Identity	IM §13.2: consume identity, never build it. Code: own auth plus OIDC SSO	Accept for L1; SSO is the consumption path; record the change
A200	Regulator mapping at L1	E01 §07: none at L1. Code: 10 frameworks, draft	Keep, labelled; counsel review before customer use

#	Topic	What the sources say	Proposed treatment
A201	"Pack"	E18: pack = declaration + certified profile + export mapping. Code: industry packs	Two names; bare "pack" retired
A202	Tenancy enforcement	EPA, TM §9.3: RLS recommended. Code: application-level only	Accept for L1 with the static test; RLS a hardening item
A203	DPDP, retention, residency (updates P3)	E02, E11: salted F14, erasure, retention ≥ 8 years. Code: minimal receipts, no erasure, replay store 30 days on by default, receipt retention unset, no residency	Retention policy and erasure design needed before personal data
A204	Dates (updates P1, P14)	Every original gate predates the code	"To be re-set"; no date invented here
A205	Brief vs code	[HB]: onboarding asks currency and time zone; inventory has a model field; password on every approval; auto-pause on serious alerts. Code: none of these as stated	The code audit prevails; brief corrected in all documents

R17.3 Open items

These continue the open-questions register [TM §23.4], which ends at Q104.

#	Question	Owner
Q105	Sign, amend or reject the L1 claim sentence on R-17 (P17 = IM DR-017)	Founder
Q106	Does the 30 Nov 2026 record-shape freeze apply to the receipt, to Seal v6, or to neither (P20)?	Founder, evidence lead
Q107	Does the Declaration survive as the L2 form of the rulebook, or is it retired?	Founder
Q108	Do rung1, ERPNext and the December gate remain; does stack1 remain the runtime (P18)?	Founder
Q109	Adopt "governed run" and "check and record" (P21)?	Founder, sales
Q110	Thirteen areas and Gate 2 (P23)	Founder, design
Q111	Which "refuses to build" items still hold [TM §2.7]?	Founder
Q112	Self-approval: forbid or keep flagged; when do dual control and quorum arrive (A194)?	Guard lead
Q113	Is the built stack the stack of record (P19)?	Founder
Q114	When does a KMS/HSM key provider replace the environment master key (A196)?	Evidence lead
Q115	What is the billing unit (A197)?	Founder, finance
Q116	Internal `gate` names and the word "platform" (A198)	Founder
Q117	Roles and terms of Hasif and Ashoke; IP assignment to elan1 Private Limited; licence (T31)	Founder, counsel
Q118	How was the code produced, and who reviewed it; can CI runs and the 580-test claim be shown?	Founder
Q119	Industry vs regulatory pack naming (A201)	Founder
Q120	Counsel review of the 10 framework mappings (A200)	Counsel
Q121	Retention, erasure and residency policy (A203)	Security owner, counsel
Q122	New dates for every gate (A204)	Founder
Q123	Consistency of names, numbers and dates across the revised IM, Atlas and financial model	Document owner

Glossary and source additions

Terms added to Appendix C

Term	Meaning
Elan1 Govern	The product, formerly "elan1" the kernel; renamed 27 October 2026 [FU]. The entity remains elan1 Private Limited
Guard	The service (`apps/gate`) that decides every tool call, holds keys and writes receipts
Rulebook	One signed YAML policy per agent: tool or tag rules, limits, approver groups; default refuse
Risk tag	One of eight labels on a tool (for example moves_money) that tag rules match
Limit (quota)	A count or sum per window that can turn an allow into an ask
`unless`	An exception on an ask rule; the ask stands aside only when every condition holds
Ask a person	The decision that holds a call for a named approver group
Receipt	The signed, hash-chained record of one decision, approval, outcome or admin event
Checkpoint	An hourly signed statement of an organisation's chain head
Auditor pack	Evidence bundle plus PDF summary and compliance map
`govern verify`	The offline CLI check of a bundle or auditor pack; exit 0, 1 or 2
Try an action · Test on past actions	Testing a draft rulebook on one made-up call, or on 7 or 30 days of real calls
Rule suggestion	A draft rule mined from repeated approvals, replay-checked, opened or dismissed by a person
Governed run · check and record	Elan1 Govern runs the tool (MCP proxy, enforcing for that tool) · the agent runs it after a check (advisory)
Connector · Your MCP servers	Built-in integration (Gmail, Zoho Books) · any upstream MCP server behind the guard
Industry pack	Starter rulebook, frameworks and risk questions for one of seven industries
Behaviour alert · auto-pause	A statistical finding about an agent · pausing it on a high finding, if switched on
Emergency stop	Pausing every agent of an organisation in one transaction, with an incident report
Organisation · Auditor	The customer (formerly tenant) · the role that reads and exports across client organisations
KeyProvider · kid	The seam for wrapping signing keys (environment only today) · the key id, a fingerprint of the public key

Sources added to Appendix B

ID	Source
GF	Elan1 Govern code facts for investor documents — code audit of `Hasif-Elan1/elan1-govern` at commit `22388ad` , 27 October 2026
GOV	The Elan1 Govern repository at that commit, cited by path: README.md, DECISIONS.md (99 decisions), PLAN.md, SECURITY.md, docs/ARCHITECTURE.md, docs/PRODUCT.md, docs/GUIDE.md, docs/openapi.json, docs/screenshots
HB	Hasif's brief to Ashoke on what has been built in Elan1 Govern, forwarded by the founder, 27 October 2026
FU	Founder's update, 27 October 2026, 14:36 IST: the product is renamed Elan1 Govern; the entity is unchanged
SM	Screen-to-code map: 275 design screens (Ashoke, design commit `62fd48e`) mapped to Elan1 Govern (code commit `22388ad`)

1. Executive technical summary

1.1 What this memorandum is, and how to read it

This is the technical companion to the elan1 Series A Information Memorandum dated 22 September 2026 [IM]. The Information Memorandum explains why elan1 should exist and what it is worth. This memorandum explains how elan1 is built, end to end: the architecture, the components, the data, the interfaces, the governed write path, the evidence and its cryptography, the packs, Levels 2 and 3, the runtime and deployment, security, metering, the technology stack, the build order and the teams, continuous integration and quality, operations, and the technical risks. It is written for two readers at once. One is an investor's technical diligence team, who need to know whether the design is sound and whether its claims are honest. The other is the build team, who need a single reference that reconciles the corpus into one buildable plan.

The entity raising is **elan1 Private Limited: the governance kernel and the regulatory packs, and nothing else** [IM §2.1]. rung1 and stack1 are separate ventures under common founder control. In this memorandum rung1 appears only as the kernel's first production consumer, and stack1 only as its runtime supplier, at arm's-length prices [IM §2.2]. verna1 is outside the perimeter and parked [O01 DR-007].

Source identifiers. Every substantive statement carries a source identifier in square brackets. E01–E27 are the elan1 phase documents; O01–O12 are the group documents; S01–S19 and R01–R24 are the stack1 and rung1 phase documents, cited only where they fix the seam; IM is the Information Memorandum, cited by section (for example [IM §6.4]). The earlier product-folder and strategy documents are cited by short code: ETS (technology stack), EPA and ETA (platform architecture flows), EBB (Build Book), ECB (Claude Build Dossier), ERD (requirement-to-deployment checklist), ETR and EAP (team kit and Agenthon playbook), GIP, GAM, HFS, HV2, BOOMI, MMS and WD. Where the product folder and the E/O series disagree, **the E/O series prevail**, and the disagreement is recorded in the conflicts block at the end of the chapter concerned. Appendix B lists every source.

Build recommendations. Where the sources are silent and this memorandum has to choose an implementation in order to explain how something is built, the choice is marked **(build recommendation)**. Those choices are this memorandum's proposals. None of them is a decision the Company has recorded, and none of them overrides a closed entry in the decision register [O01 §05].

> **Designed is not shipped (DR-010).** As at 23 September 2026 elan1 has no external customer, no signed order, no invoice, no certification and no evidence bundle verified by anyone outside the company. Zero governed actions have been committed or refused in a production tenant [IM §2.5; IM App. E.1]. Every latency, cost, throughput and capacity figure in this memorandum is a **target** unless it is explicitly marked as measured, and none has been measured under a real tenant's load [O03 §08; E04 §02]. The Company's own register scores a designed-but-unshipped capability as zero, and applies that rule to itself first [O01 DR-010].

Three reading conventions follow from that rule. First, specifications are described in the present tense of the specification ("rung 8 appends the record"), and their build status is stated separately in §1.5 and in each chapter. Second, "tamper-evident" is the only integrity word used for Level 1; "signed", "tamper-proof", "immutable", "court-ready" and "independently verifiable" are used only of Level 2 or later, and only in the future tense [E01 §08; E07 §02]. Third, "Enforced" is used only where the kernel holds the target credential or the target accepts only kernel-issued tokens; everything else is "attested advisory" [O03 §02; E05 §02].

1.2 The system in one page

elan1 is a **governance kernel on the write path** between an AI agent (or a person, or a scheduled job) and a system of record. It types the intent, evaluates policy gates held as data, forces a human to approve where the declaration requires it, executes the write through a single writer, records the outcome in an append-only hash-chained log, and meters it [IM

§4.1; E01 §01]. The billable unit is the **Governed Action**: the one write that was permitted, executed, recorded and billed, and the same object at all four points [O01 §02].

> *A model may produce a write intent. It may never produce an allow. [E02 I1]*

One governed action, from agent intent to receipt, runs as follows. The numbers in brackets are the rungs of the Right Path [E04 §03].

1. **Intent.** An agent composes a write intent against a declared write type — for example `erp.sales_invoice.submit` in declaration `erp.ar` at version 7 — using the `stage.write` tool, which validates the request object and has no side effects. The intent carries a client-generated `intent_id` (the idempotency key), an explicit integer declaration version (never "latest") and a supplied `as_of` instant [E02 §03; E05 §04].
2. **Submission.** The consumer calls `assert` — in-process through the SDK (Mode A) or as the `act.write` tool or `POST /v1/writes` (Mode B). Both land on the same first rung. Tenant and actor come from the token, never from an argument [E05 §01, §05].
3. **Decision (rungs 0b–6).** STOP checks the tenant's stop set; IDENTITY sets `actor_kind` (human, agent or system) from the token; WRITE DOOR resolves the declaration at its version and the write-type descriptor; PRE-GATES run every declared gate in declared order as pure functions of the context and `as_of`; `evaluate()` applies versioned rules over a projection of declared fields, with no record read and no model; GUARDRAIL checks the blast radius against the declared value; AUTONOMY resolves the actor's grant, A0 to A5, with a missing grant resolving to A0 [E02 §05; E04 §03].
4. **Escalation, if required.** Where the actor's grant is below what the write type needs, or the write type is irreversible, rung 6 escalates: an obligation is created with a named resolver set and an expiry (four hours by default), and the ladder stops. A named human approves in the console; the approval is itself a governed write with its own record. The original intent is then resubmitted with the obligation reference and climbs the whole ladder again [E04 §04 Flow 3].
5. **Act (rung 7).** In Enforced mode the single writer takes a snapshot, applies the change to a per-tenant shadow sandbox restored from it, re-runs the gates on the result, applies the identical bytes to the live system through a single-use credential with a ten-minute life scoped to one write, re-runs the gates on live, and commits the record version and exactly one outbox row in one transaction. In Phase 1 the live apply is an ERPNext write through Frappe's own API, never the ORM, and the adapter returns the inverse operation from which the 30-day rollback is built [E04 §03 rung 7].
6. **Proof (rung 8).** The chain writer consumes the outbox in commit order, assigns the tenant's next sequence number, assembles the sixteen frozen fields, chains them to the previous digest and appends the record. At Level 1 the record is SHA-256 over a length-prefixed preimage and is not signed; at Level 2 the same preimage is signed with pure Ed25519 and anchored to two RFC 3161 timestamp authorities [E02 §03; E12; E14 §02].
7. **Meter (rung 9).** Exactly one meter event is written, its basis taken normatively from the declared blast radius. A refusal meters at quantity zero, and a database `CHECK` makes a non-zero refusal row impossible to insert [E02 I4; E05 §06].
8. **Return.** The caller receives `committed`, `refused` or `escalated` with the sequence number and digest, synchronously. A refusal is an HTTP 200, never a 4xx, because clients retry 4xx and a retried refusal is an attempt to get a different answer [E02 §04; E05 §07]. Webhooks carry the same `seq` and `digest`, so a consumer can detect a gap in its own delivery [E05 §09].

Three layers carry this, and the boundaries between them are enforced in continuous integration and network policy rather than left to review [O03 §10; S03 §03]:

Layer	Role	Owns	May never change or build	Repositories today	Entity
elan1	The kernel and the packs	KernelPort; the Right Path; the Declaration schema; gates; approval; the proof format; the meter; the refusal register; the MCP server and SDKs; the verifier (L2); packs	A second write path; a special case for any product; an ERP, a CRM or a UI shell for someone else's product	`elan1-core`, `elan1-declarations`	elan1 Private Limited

Layer	Role	Owns	May never change or build	Repositories today	Entity
rung1	First production consumer	Its Declarations, its ERPNext adapters, its screens within the Four Doors, its price list, its hosted tenants	Gate logic, approval logic, the proof-record shape, the meter, the autonomy ladder; it may never build a gates engine, approval store, seal, meter or verifier	`rung1-enterprise` (recorded in O03/O10 under its earlier name)	Separate venture
stack1	Runtime supplier	Component choices behind the `fusion.*/v1` contracts; deploy topology; observability; secrets; the sandbox pool	Anything the kernel decides; anything on the governed write path; reading a tenant record from the control plane	`stack1-contracts`, `stack1-runtime`	Separate venture

Sources: [O03 §10, §12; IM §2.1–2.3, §6.2; S03 §02; O10 §06]. The entity column follows the IM perimeter rather than the older Holdco/Opco structure in O11; Chapter 2 records the difference.

1.3 The kernel in numbers

The kernel is small and closed by construction. The numbers below are fixed by specification; none is a measurement.

Quantity	Value	What it means	Source
KernelPort calls	**6** — `define`, `assert`, `seal`, `verify`, `scope`, `refuse`	Identical signatures for every consumer, product and level; frozen for Phase 1	[E02 §01, §05; O03 §09]
Objects on the port	**5** — write intent, decision, approval, record, meter event	Consumers hold these, never a kernel internal	[E02 §03]
Invariants	**8** — I1 to I8	Where a call and an invariant disagree, the call is a defect	[E02 §02]
Rungs of the Right Path	**9**, plus the 0b STOP check	Fixed order; no fast path, batch variant, admin mode or decision cache	[E04 §03; IM §4.3]
Pre-gates at rung 3	**9** (G1–G9), **6** never skippable	Consent, residency, retention, dual control, counterparty and precondition always run	[E04 §03]
Autonomy levels	**6** — A0 Observe to A5 Act broad	Missing grant resolves to A0; there is no A6	[O01 §02; E04 §03]
Blast-radius values	**5** — `one_record`, `one_counterparty`, `batch`, `tenant`, `cross_boundary`	Also the normative meter basis	[E02 §03; E05 §06]
Declaration blocks	**7** — identity, shape, permissions, write_types, gates, proof_binding, channels	Policy as data; JSON, under 900 lines, never a programming language	[O01 §02; IM §6.6]
Declaration semantic rules	**11** (L0-S01–S11), plus 7 lint rules	Run in CI with no platform present	[E03 §05; IM §6.6]
Refusal register	**43** codes (policy 14, gate 19, platform 10); **28** active at L1	Closed enumeration; each code maps to one rung, one class, one operator action	[E02 §06; IM §6.4]
Assurance levels	**3** — L1 Guarded, L2 Attested, L3 Proven	A level is a property of the proof and the claim, never of the policy	[E07 §01; IM §4.7]
Proof-record fields	**16** (F01–F16), frozen at L1, unsigned until L2	Length-prefixed preimage; no JSON in the digest; no optional fields	[E02 §03; IM §7.2]
Seal test vectors	**64** — 12 structure, 12 chain, 12 Merkle/anchor, 28 negative	Two independent implementations must agree on every byte	[IM §7.3; E12]
Evidence-bundle members	**8**	Self-contained, inert, claim-bearing	[IM §7.4; E15]
Offline-verifier exit codes	**6** — 0 PASS (three variants), 1 FAIL_INTEGRITY, 2 FAIL_PROFILE, 3 INCOMPLETE, 4 UNSUPPORTED, 5 USAGE	Integrity, profile and completeness failures are never collapsed	[IM §7.5; E15]
Timestamp authorities	**2** per batch, one India-based; interval ≤ 15 minutes	Anchoring never refuses a write	[IM §7.6; E17]

Quantity	Value	What it means	Source
Consumer conformance tests	**14** — C1 to C14	"The suite is the claim"	[E02 §08]
MCP tools (Phase 1)	**34** — 18 read-only, 10 stage, 6 act	Tool descriptions pinned by hash in the declaration	[E05 §04]
Pack admission gates	**5** — G-A dated hook, G-B named auditor, G-C reversibility declared truthfully, G-D evidence sold separately, G-E elan1 owns the write	Eight candidate packs rejected by name	[E18 §02; IM §5.2]

Two latency figures sit beside these and are **targets only**: a kernel budget of about 64 ms p50 across the nine rungs, and an end-to-end governed write of at most 2 s p95 once the sandbox and live applies (budgeted separately at ≤ 1,200 ms p95) are included [E04 §02]. The group integration contract sets a kernel-side envelope of p50 ≤ 120 ms and p99 ≤ 450 ms, and the elan1 kernel's first kill criterion is failing to hold that envelope at ten design partners' load, measured [O03 §08; O02 §06]. A second internal model puts the kernel at about 91 ms with the proof rung at 40 ms; the two are unreconciled [IM §6.10; EPA §07]. Chapter 5 carries the budget in full.

1.4 Three assurance levels, one declaration

The same declaration carries across all three levels. What changes is what happens at rung 8 and after it, and what may be said about the result [E07 §01; E14 §02].

	L1 Guarded	L2 Attested	L3 Proven
Adds	Nine rungs; declared gates; versioned rules; approval; single writer; shadow sandbox; 30-day rollback; hash-chained record; meter	Pure Ed25519 over the frozen preimage; key custody per residency domain; Merkle batching to two RFC 3161 authorities; eight-member bundle; zero-dependency offline verifier; executing verifier profiles	Cross-organisation verification; selective disclosure; customer-held keys; sovereign deployment; precedent intelligence; earned autonomy; the Action Receipt open standard
May truthfully claim	Control and reconstruction, for a party willing to trust elan1's infrastructure	A party who distrusts elan1 can verify a record offline with a fingerprint obtained out of band	Verification between organisations, and autonomy granted because it was earned
Status at 23 Sept 2026	Specified; being built	Frozen specification	Frozen specification; entry gated on an event
Plan date	GA targeted Q2 CY2027 (30 June 2027)	Signing live 30 Sept 2027; GA 30 April 2028	Entry October 2029 at the earliest, only after an external auditor accepts an L2 bundle in writing

Sources: [E01; E07; E14; E20; IM §1.3, §4.7, §16.5]. The corpus gives three different dates for L2; §1.5 and the conflicts block explain the treatment.

Two rules make the ladder a retrofit rather than a migration. The sixteen-field layout is frozen at L1 even though it is unsigned, so L2 adds a signature over the same bytes [E01 decision 2]. And a chain written at L1 is never retroactively sealed: signatures start at the sequence number where signing starts, and the bundle states the unsigned prefix [E01 decision 3; E07 §06].

1.5 What is built, what is specified and what is parked, as at 23 September 2026

The Company's own position is blunt: "We have the design and nobody's books" [IM §4.9]. The table separates four states. **Built** means code exists and passes its own tests; **prototype** means code exists in the earlier `elan1-platform` tree but does not implement the current specification; **specified** means a frozen or draft E/O document exists; **parked** means suspended in writing with a named reopen condition.

Capability	State	Evidence
KernelPort v1, the Right Path, the refusal register, conformance C1–C14	**Specified** (draft for review, 21 Sept 2026)	[E02; E04]. The six documents that block coding must be signed by 31 Oct 2026 [IM §16.2.1]

Capability	State	Evidence
A governed-write chokepoint (`sor_write.governed()`), hash-chained proofs, policy rules as rows, a React console shell	**Prototype**	Derived by execution on 9–12 Sept 2026: every act writes to elan1's own Postgres, not an external system of record; the sealed tuple has 13 fields and no sequence number; no signing library; no export route [HFS Part 1; HV2 §11]
Declaration schema (seven blocks, eleven rules)	**Specified**; four divergent dialects in the prototype	[E03; HFS "Worst seam"]
L1 single writer against a real ERPNext site	**Not built.** Target: first real governed write in a design partner's books, with the bypass attempt failing, by 31 Dec 2026	[IM §16.2.3; O02 §06]
Meter with refusals at zero, two-way reconciliation	**Specified**; the prototype's reconciliation compares counts without a join	[E02 §03; HV2 §11 item 5]
MCP server (34 tools) and SDK	**Specified**; third-party GA is Phase 2	[E05]
Seal Spec v6 and 64 vectors	**Specified** (Phase 2 document); not implemented, and no seal may be shown outside the company until two implementations agree	[E12; IM §4.9]
Evidence bundle, offline verifier, verifier profiles, key custody, TSA anchoring	**Specified** (L2)	[E13; E15; E16; E17]
Packs: ledger1 (first), invoice1, collect1	**Specified**; collect1 is a sketch needing counsel	[E18; E27]
L3 capabilities and the Action Receipt standard	**Specified**; engineering begins on an event, not a date	[E20–E25]
Certification (SOC 2, ISO/IEC 27001 or 42001)	**None held**	[IM §4.9]
rung1 Revenue, rung1 Commerce, verna1	**Parked** (outside the perimeter in any case)	[O02 §03]

The Information Memorandum states that "an implementation of the L1 kernel against a single system of record" exists [IM §4.9]. The derived readings of the working tree do not support that phrase as written: the tree's acts all insert into elan1's own database, and HFS records that the frontend already asserts `writesReachSystemOfRecord: true` where the backend cannot [HFS "Worst seam"]. This memorandum therefore describes the existing code as a prototype and the E02 kernel as not yet built. The conflicts block records the difference, and Chapter 3 (§3.8) sets out what in the prototype carries forward.

> **The corpus is a plan written in one pass.** The Phase 2–4 folders were recorded as empty on 21 September 2026, the day the Phase 2–4 documents are dated [IM App. E.2 item 16]. Every phase after the first describes work not begun. A five-day agentic hackathon on 17–21 September 2026 targeted two seal implementations and a zero-dependency verifier against draft vectors; the corpus records its plan, not its outcome, and its genesis vector (311 octets) predates the current encoding table (329 octets) [EAP §01; IM §7.3]. Its output does not count towards any L2 gate.

1.6 The build plan in one table

elan1 builds **only Level 1 now**. L2 and L3 are frozen specifications, and the Moonshot allocation buys specification, not implementation: "any spend on L2 implementation voids the defence and returns the allocation to zero" [O02 decision 4; E01 decision 1]. stack1 is being built at the same time by its own team, and rung1 builds rung1 Enterprise on Frappe/ERPNext first [O02 §02; IM §2.2].

Period	Phase and objective	elan1 ships	Gate and date	Consumer and supplier dependencies
Oct 2026	Phase 1 — sign the coding blockers	E02, E03, O01 signed (with R01, R03, S01)	Six coding blockers SIGNED, 31 Oct 2026	rung1 and stack1 sign their own blockers
Nov 2026	Phase 1 — freeze the record shape	KernelPort shapes frozen; sixteen-field preimage frozen; Declaration schema major (`doors` → `channels`) published; E12 drafted, not implemented	Record shape frozen, 30 Nov 2026	—
Dec 2026	Phase 1 — customer zero is real	Rungs 0b–9 on one write type, single writer against a real ERPNext site	First real governed write with the bypass attempt failing, 31 Dec 2026	rung1 hosted ERPNext; stack1 runtime live (stack1 kill criterion)
Jan–Mar 2027	Phase 1 — meter and baseline	Meter live; two-way reconciliation; O05 thirteen-row security baseline; E09 readiness rows worked	Meter reconciles over a full month, 31 Jan 2027; tranche 1 (₹63 cr), Mar 2027	rung1 pays ₹0.40 per committed action from 1 Jan 2027
Apr–Jun 2027	Phase 2 — L1 GA	Sandbox keys self-serve; production keys by review; conformance suite published; status page	L1 GA, 30 Jun 2027; a third party integrated under a signed agreement (elan1 kill criterion 5)	—
Jul–Sep 2027	Phase 3 — signing	Ed25519 signing over the frozen preimage; key custody per residency domain	Key custody 31 Aug 2027; L2 signing live, 30 Sep 2027, only after two implementations agree on all 64 vectors	stack1 residency-domain topology
Oct–Dec 2027	Phase 3 — anchoring and the first pack	TSA contract; Merkle anchor job; RFC 3161 live; bundle and offline verifier shipped as source; **ledger1** live	TSA contract 31 Oct 2027; anchor job 30 Nov 2027 (on a blocking-green `fusion.run/v1`); anchoring live 31 Dec 2027	stack1 GA
Jan–Mar 2028	Phase 3 — the two gates	L1→L2 upgrade executed on a real tenant	**Gate 1** 31 Jan 2028 (a paying customer's write verifies offline); **Gate 2** 31 Mar 2028 (app three renders with zero screen code)	A design partner and its auditor
Apr 2028–Mar 2029	Phase 4 entry — evidence as a line item	**L2 GA** 30 Apr 2028; **invoice1** Apr 2028; **collect1** Oct 2028	Gate A / tranche 2 (₹72 cr), Oct 2028; Gate 3 (evidence on its own line), 31 Dec 2028	—
Apr 2029–Mar 2030	L3 entry, on an event	Second seal implementation kept in agreement; L3 capabilities begin only if the entry condition is met	Gate B / tranche 3 (₹45 cr), Jul 2029; **L3 entry** Oct 2029	First sovereign deployment contracted Jul 2029
Apr 2030–Mar 2031	The format outlives the company	Action Receipt v1 with external implementers	EBITDA positive for the year	—

Sources: [IM §1.7–1.8, §16.2–16.5; O09 §02–§04; E14 §07; E20 §02].

Teams. Four elan1 engineering workstreams build in parallel, alongside the separate rung1 and stack1 teams: (1) **kernel engineering** (KernelPort, the rungs, gates, registry, meter), four people in October 2026 rising to seven by March 2027; (2) **evidence and cryptography** (the frozen record, the chain writer, and the L2 specification only), one rising to three; (3) **pack and regulatory engineering** (declarations, the ledger1 profile), one rising to two; and (4) **platform, SRE and assurance** (conformance suite, CI invariants, the security baseline, restore drills), two rising to five with the first security hire in December 2026 [IM §15.2–15.4, §16.2]. The grouping of functions into four build workstreams is this memorandum's **(build recommendation)**; Chapter 21 sets out the epics, sprints and ownership.

The Phase 3 critical path is six items in series — L2 signing, key custody, the TSA contract, the Merkle anchor job, the offline verifier and the first India evidence pack — with about one quarter of float end to end. The longest-lead item is the timestamp-authority contract, with a counterparty not yet named [O09 §03–§04].

1.7 Ten things a diligence team should check first

These are ordered by how much of the thesis each one tests, not by effort.

#	Check	Why it matters	Where
1	**Read E02 and O03 side by side.** They describe `define`, `assert` and `seal` differently (E02: `assert` runs all nine rungs and `seal` is kernel-only; O03: `define` registers an action, `assert` stops at rung 6 and a Mode B caller calls `seal`)	The port is "the commercial asset"; it cannot be frozen on 30 Nov 2026 while its two governing documents disagree	Ch. 2 §2.5; Ch. 4
2	**Watch the bypass test (C2) against the real ERPNext site**	elan1 kill criterion 4 fires if no ERPNext deployment mode lets the kernel hold the credential or accept only kernel tokens	Ch. 5, 6
3	**Ask to see the import-graph check fail** in a consuming repository (C1): a module above P3 importing a kernel type, and a model client reachable from rungs 3–8	I1 is enforced by a build failure, not by review	Ch. 3 §3.5; Ch. 21
4	**Read the grant catalogue and the meter `CHECK`**: INSERT-only on the evidence tables, one INSERT principal, `CHECK (outcome <> 'refused' OR quantity = 0)`	"Refusals are free" is a constraint, not a policy	Ch. 9, 19
5	**Run C14 on rung1's rendered copy**	The L1 claim boundary is a failing test	Ch. 6, 15
6	**Inspect the determinism replay (C4)**	A stage that reads a clock, a random value or an undeclared input fails the same night	Ch. 5, 21
7	**Confirm no L2 implementation spend** before the seal specification is frozen, and ask for the disagreement log when it exists	O02 decision 4; the two-implementation rule	Ch. 10, 21
8	**Ask for measured latency** (E01 open question 2, due 30 Nov 2026) and which of the 64 ms and 91 ms models it replaces	Kill criterion 1 is a measured envelope	Ch. 5, 22
9	**Re-derive the state of the tree** (HFS method): does any act reach an external system of record; is `seq` sealed; is the declaration versioned in its key	The distance from prototype to E02 kernel is the Phase 1 plan	Ch. 3 §3.8
10	**Read counsel's answer on DPDP erasure against the hash chain**, and the TSA and key-custody decisions	The evidence thesis leans on all three; none is closed	Ch. 12, 18

1.8 Top technical risks

Chapter 23 carries the full register with triggers. The eight below are the ones that change the build.

Risk	What would happen	Mitigation in the design	Status
The seam is not settled (§1.7 item 1)	Two teams build against two readings of `assert` and `seal`; the freeze on 30 Nov 2026 freezes an ambiguity	This memorandum adopts E02 for the port and records O03's differences (Chapter 2 conflicts)	Open
Enforcement cannot be held outside ERPNext	elan1 sells advice, not control (kill criterion 4)	Credential custody in hosted mode; token acceptance explored (O03 open question 1)	Open; due 31 Oct 2026
Sandbox-first cost	The shadow sandbox and double gate run miss the 2 s p95 end-to-end target	Target-system call has its own SLO outside the kernel budget [E04 decision 3]	Unmeasured
Commit-then-seal gap	A live write with no chained record (`acted_unproven`)	Same-transaction outbox; ordered retry; nightly write-to-record join, non-zero is a P0	Specified
Declaration dialect drift	Four incompatible encodings of the same seven blocks, as found in the prototype	One schema, generated types, a CI arm that loads the canonical fixture	Open in prototype
DPDP erasure against an append-only chain	Retention model of the evidence plane is affected if a regulator reads erasure as reaching the record	Per-record salts outside the preimage; `PASS — payload unbound`	Counsel has not answered [IM §1.9]
Two-implementation staffing	L2 signing slips; Gate 1 moves a quarter	Evidence and cryptography lead is hire one; second implementation in a different language	Hiring

Risk	What would happen	Mitigation in the design	Status
Single-founder and one-bet concentration	rung1, elan1 and stack1 share the kernel team and the founder	Written boundaries; kill criteria; key hires	Disclosed [O09 §10]

1.9 Chapter map

If you want...	Read	Figures
The perimeter, the design principles and how the architecture narrowed to a kernel	Chapter 2	1, 2
Planes, components, repositories and deployment units	Chapter 3	3, 4
KernelPort, the five objects and the refusal register	Chapter 4	5
The nine rungs and one governed action end to end	Chapter 5	6, 7
The SDK, the MCP server and the two modes	Chapter 6	8
The Declaration, Controls and Regulations	Chapter 7	9, 10
The console and the human approval path	Chapter 8	11
Data architecture and the L1 proof record	Chapter 9	12, 13
Seal Spec v6, the bundle, the verifier, keys and timestamps	Chapters 10–12	14–17
Packs: the factory, ledger1, invoice1, collect1	Chapter 13	18–20
Level 3, the Action Receipt standard and certification	Chapters 14–15	21–24
The assurance ladder and upgrade runbooks	Chapter 16	25
Runtime, deployment, security, residency and licences	Chapters 17–18	26, 27
Metering, pricing plumbing and billing	Chapter 19	28
Technology choices and alternatives	Chapter 20	29
Teams, build sequence, CI and quality gates	Chapter 21	30, 31
Operations, reliability and incident response	Chapter 22	32
Risks, open questions and diligence answers	Chapter 23	—

Appendix A collects every source conflict recorded at the end of each chapter, with its treatment and the date by which it must be closed.

2. System context, design principles and the perimeter

This chapter places elan1 in its surroundings before Chapter 3 takes it apart: what it is as a technical system, whom and what it deals with, the invariants every component keeps, the perimeter of the entity being funded, the seam with rung1 and stack1, what elan1 refuses to build, and how the architecture narrowed to a governance kernel and its packs. Nothing described here runs for a customer. Under DR-010 elan1's shipped-capability score is zero [O01 DR-010; IM §4.9].

2.1 What elan1 is, technically

elan1 is **one write function, reached through six calls, that decides whether a consequential change to a system of record may happen, makes it happen through a single writer when it may, and leaves the same record behind whether it did or not** [E02 §01; O01 §02]. Everything else in the corpus — the Declaration, the console, the MCP server, the evidence bundle, the packs — either feeds that function, renders it, or exports what it recorded.

The unit that flows through the system is the **Governed Action**: the one unit that is permitted, executed, recorded and billed. It is the same unit at all four points. If something is billed that was not permitted, or recorded at a different granularity from what was billed, that is a defect and not a pricing choice [O01 §02]. One governed action is one row in the chain, one meter event, one line on the invoice and one thing an auditor can open [IM §4.1].

Four properties separate elan1 from an authorisation service, a guardrail proxy or an audit log:

Property	What it means in the build	Source
The decision is not a model's	Rungs 3, 4, 5, 6 and 8 are unreachable from any model client, enforced by an import-graph check. `evaluate()` is versioned rules over a projection of declared fields, with no record read and no model, at every level	[E02 I1; E04 §03 rung 4; E01 §02 item 2]
The kernel is on the write, not beside it	In Enforced mode the kernel's single writer holds the only credential to the target, minted per write with a ten-minute life; a refusal means the write did not happen	[E02 I2; E05 §02]
The refused case is recorded as carefully as the allowed one	A refusal is an HTTP 200 with a full sixteen-field record (record identifier zeroed) and a meter event of quantity zero, enforced by a database `CHECK`	[E02 I4, §04]
Policy is data	What may be changed, by whom, under which gates and with which proof is a certified, versioned seven-block Declaration; a new regime needs a declaration and a profile, not kernel code	[E03; IM §4.5, §6.6]

What elan1 is not is equally load-bearing. It is not a model, an agent framework or an orchestration engine: stack1 runs durable execution and the model gateway, and the kernel calls no model [S03 §01–02]. It is not an identity provider: elan1 consumes agent and workforce identity and differentiates on enforcement, approval and evidence, never on authentication [IM §13.2]. It is not a system of record: the customer's ERP, ledger or core system remains the only place a record changes [O01 §02 P2]. And it is not a platform at Level 1: the retired phrase "Governed Intelligence Platform" over-claims, and the written form is "the elan1 layer" or "the elan1 kernel" [O01 §04; O10 §05].

2.1.1 The claim, frozen

The Company froze, verbatim, what it may say about the level it is building first:

> "At L1, elan1 decides whether a write is permitted, forces a human to approve the writes that need it, executes it through a single writer, records the decision in a hash-chained append-only log, and meters it. The log is tamper-evident. It is not yet signed, not yet anchored to an external time source, and not yet verifiable by a third party who does not trust us." [E01 §01]

The sentence is an engineering constraint as well as a sales one. It describes operations rather than a category; it names three negatives that later levels remove one at a time; and it is enforced mechanically by conformance test C14, a string check over a consumer's rendered copy that fails the build on "signed", "tamper-proof" or "independently verifiable" while the consumer runs at L1 [E02 §08 C14; IM §4.1].

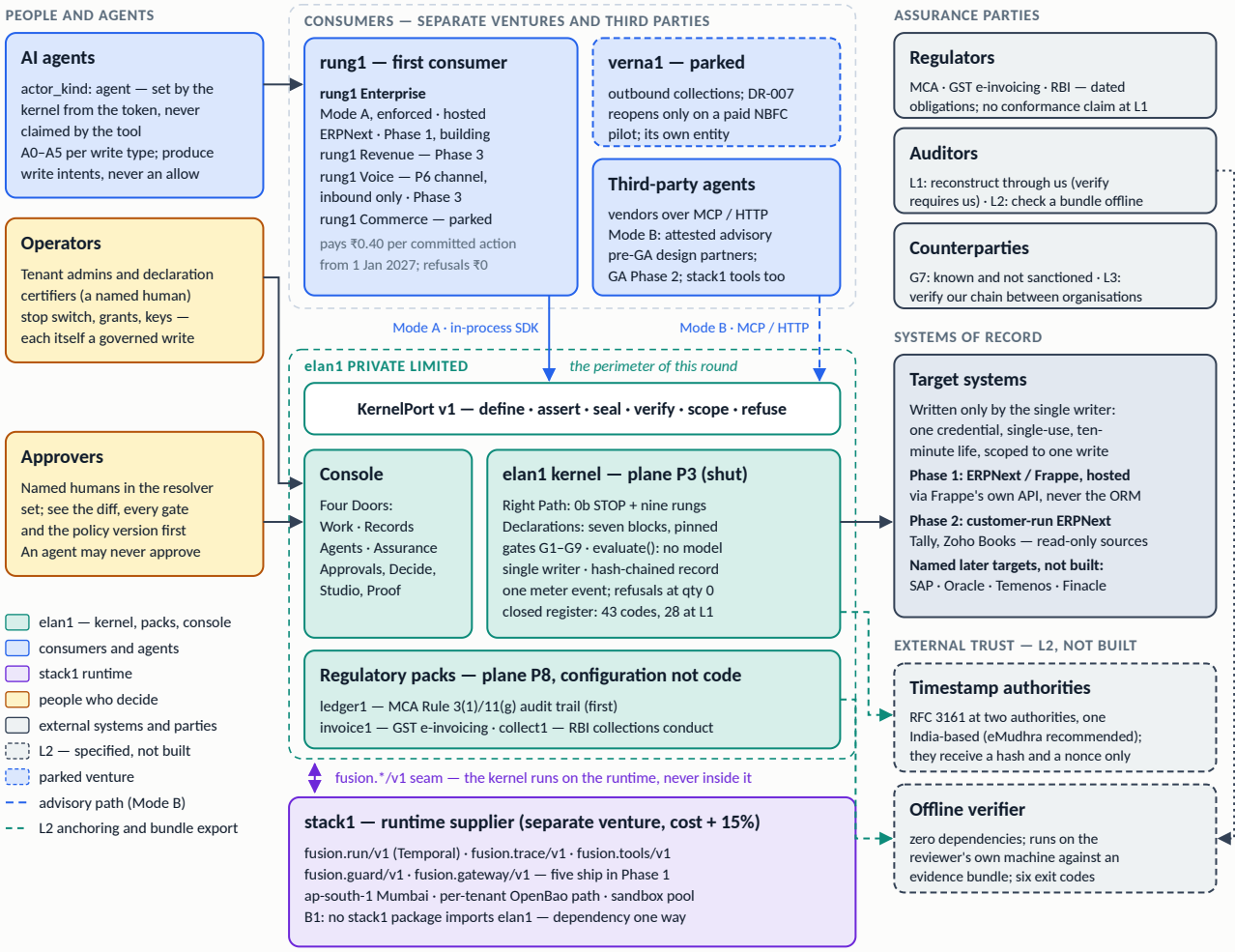
2.2 Actors and external systems

elan1 has five kinds of counterpart: the **people and agents** who ask, approve, certify, operate and audit; the **consumers** who build against KernelPort; the **supplier** beneath it; the **systems of record** whose writes it governs; and the **trust and identity services** it relies on for time, keys and identity. Figure 1 draws them.

Figure 1 — elan1 in context

L1 — BEING BUILT · L2/L3 — NOT BUILT

Who acts, who consumes the kernel, what it runs on, and the external systems it writes to or is checked by



A model may produce a write intent; it may never produce an allow. In Mode A only the kernel's single writer holds a credential to the books.

Source: E01 §02–05, E02 §01, E04, E05 §02, E06, E17 §03, O03 §01–02, O01 §02, S03 §02, R10, IM §1.3–1.4, §2.1–2.3, §6.1, App. B · elan1 Technology Memorandum · Sept 2026

Figure 1 — elan1 in context: the actors who ask, approve and audit; the consumers that build against KernelPort; stack1 beneath as the runtime; the systems of record written through the single writer; and the trust services used from Level 2.

2.2.1 Actors

Actor	What they do	How the kernel knows them	Constraint	Source
Agent	Composes write intents; stages and submits within its autonomy grant	`actor_kind: agent`, set by the kernel from the token, never inferred from the payload	Can never approve; A0–A5 grant per write type; missing grant is A0	[E02 §03; E04 rung 1, rung 6; E02 decision 7]
Human requester	Submits a write (a form, a console action, a request routed by a consumer)	`actor_kind: human` from a tenant token	Same ladder as an agent: "exactly as a human clicking a button does"	[E04 §04 Flow 2]
Approver	Resolves an escalation; the approval is itself a governed write	`approver_kind: human` only at L1; must differ from the requester where G4 applies	Approves on a digest of what they were shown (`basis`)	[E02 §03, decision 7]
Declaration author and certifier	Authors, validates, simulates and certifies a declaration version	Control token; certification is a governed write under dual control	Certifier must not be the author (`GATE.DUAL_CONTROL_SAME_ACTOR`)	[E02 §05 define; E09 §04]
Operator (elan1 or consumer)	Runs the tenant; reads refusals, meter and chain; throws a tenant stop	Ops token: read-only, no tenant data	Break-glass is a declared write type with dual control, recorded in the operator's own chain and visible to the tenant	[E05 §05; E02 decision 9]
Auditor	Asks what happened and whether it can be checked	At L1, through the tenant ("verify requires us"); at L2, offline with a fingerprint obtained out of band	L1 evidence is internal-control evidence only	[E02 §05 verify; E07 §02]
Regulator or supervisor	Sets the obligation a pack compiles; may issue a directive that enters the stop set	Rung 0b stop set includes "regulator directive"	No conformance claim is made at L1	[O01 §02 Right Path; E01 §07]
Counterparty organisation	The other side of a cross-boundary write	`cross_boundary` blast radius; from L3, a counterparty verifier	Cross-boundary forces dual control and a non-empty proof binding	[IM §6.6; E22]

2.2.2 Consumers, supplier and external systems

Group	System	elan1's relationship	Direction	Phase	Source
Consumer	**rung1 Enterprise** (hosted ERPNext)	First production consumer; Mode A, in-process SDK; Enforced	Calls KernelPort	Phase 1	[O03 §02; E01 §04]
Consumer	stack1 tooling	Mode B over MCP; attested advisory; the second consumer	Calls KernelPort	Phase 1	[O03 §04; WD]
Consumer	Third-party software vendors	Mode B over MCP or HTTP under a pre-GA agreement, then GA; attested advisory until promoted per write type	Calls KernelPort	Design partners in Phase 1; GA Phase 2	[E05 §01; E09]
Consumer	Regulatory packs	Configuration consumed by the kernel: declaration, profile, export mapping, approver model	Loaded by the registry	ledger1 Oct 2027	[E18; IM §5.1]
Supplier	**stack1** runtime	Compute, durable execution (`fusion.run/v1`), tracing, secrets, sandbox pool, deploy pipeline	elan1 depends on it; never the reverse	Phase 1	[S02 §1; S03 §02]
System of record	ERPNext on Frappe	Governed write through Frappe's own API (never the ORM) by the single writer, inside rung1 hosted mode	Write	Phase 1	[E04 rung 7; IM §6.10]
System of record	Tally	In rung1, a read-only migration source and never written; any elan1 claim over Tally-held books would be attested advisory	Read (via consumer)	rung1 Phase 2	[R09 §02; E05 §02]

Group	System	elan1's relationship	Direction	Phase	Source
System of record	SAP, Oracle, Temenos, Finacle	Named as the write paths platform vendors do not govern; reached only through a consumer's declared adapter; no adapter is specified	Write, future	Not scheduled	[IM §1.6, §6.1]
Statutory endpoints	GSP/IRP (e-invoicing)	The invoice1 regime; the IRN is produced by a government endpoint and reaches the tenant as a response, which is why G-E partly fails for it	Read / validate	invoice1, Apr 2028	[E18 §04; IM §16.3.3]
Trust service	RFC 3161 timestamp authorities	Two per batch, one India-based (eMudhra named, plus one of DigiCert, Sectigo, GlobalSign or Entrust); only the Merkle root and a nonce leave	Outbound, by hostname	L2, contract 31 Oct 2027	[IM §7.6; E17]
Trust service	Managed KMS / HSM	Signing keys per residency domain; no key material anywhere at L1	Internal	L2	[E16; E01 §07]
Trust service	SCITT transparency service	Optional publication of COSE receipts; never mandatory	Outbound	L3	[IM §13.4; ETS §02]
Identity	Workforce and agent identity (Entra Agent ID, Okta/Auth0 for MCP, Keycloak in rung1)	Consumed to establish the actor behind a token; never competed with	Inbound assertion	Phase 1 onward	[IM §13.2; S03 §04]
Model providers	Hosted LLMs, self-hosted open weights	Upstream of the intent only, through the consumer and `fusion.gateway/v1`; never on rungs 3–8	None from the kernel	—	[E02 I1; S02 §1]

Three rules govern every row. **Adapters, not substrates**: a channel or connector adapts a request and never decides; "a channel adapts; it never decides" [IM §6.6]. **Egress by declaration**: a connector's hostname reaches the record-plane allow-list only because a certified declaration names it, so a new egress destination is a new declaration version and therefore a certification [EPA §03]. **Residency is never overridden**: `GATE.RESIDENCY_MISMATCH` has no override, and an applicant in a residency domain elan1 does not run is refused onboarding [E02 §06; E09 §08].

2.3 Design principles and invariants

The kernel contract states eight invariants, and "where a call and an invariant disagree, the invariant wins and the call is a defect" [E02 §02]. Around them sit a smaller number of structural principles that shape components outside the kernel. Each is listed with the failure it exists to prevent and the mechanism that enforces it, because "an instruction in a document is advice and a failing build is a control" [O01 §02].

2.3.1 The eight kernel invariants

> **I1 — A model may produce a write intent. It may never produce an allow.** A model may sit upstream of `assert` and compose the intent. Rungs 3, 4, 5, 6 and 8 are unreachable from any model. Enforced by an import-graph check, not by review. [E02 I1]

#	Invariant	Why it exists	How it is enforced	Source
I1	A model may produce an intent, never an allow	A decision a model made cannot be replayed, versioned or defended to a regulator; a model that can be talked out of a refusal is not a control	C1 import graph: no module reachable from rungs 3–8 imports a model client; `evaluate()` has no model path at any level	[E02 I1, C1; E01 §02 item 2]
I2	Single writer, one ten-minute single-use credential per write	Any second path to the target turns enforcement into advice; a long-lived credential is "a credential that will be found in a log"	C2 bypass attempt; credential scope tests; nightly check that zero records were mutated outside the kernel writer	[E02 I2, C2; O03 §06; E01 §06 item 4]

#	Invariant	Why it exists	How it is enforced	Source
I3	Fail closed: every stage's default is refuse	A governance layer that fails open is worse than none, because it reports controls as working	C5 fail-closed matrix (registry, rule engine, autonomy table, target each unavailable in turn); C6 missing grant → A0 over 1,000 fuzzed cases	[E02 I3, C5, C6]
I4	Refusals are free and typed; zero-quantity meter enforced by `CHECK`	Removes the vendor's incentive to say no, and makes every refusal actionable	Register lint (one rung, one class, one operator action per code); C8 refusal shape; database constraint	[E02 I4, C8; E01 §06 item 6]
I5	No fast path, batch variant, admin mode, decision cache or second implementation	"A cached allow is an allow at an unknown policy version"; any bypass becomes the path attackers use	Rung-skip suite (C3); break-glass only as a declared dual-control write type	[E02 I5, C3, decision 9]
I6	No warn: allow, refuse, escalate	"Succeeded with warnings" is how controls decay; escalate is the honest version of warn	`disposition` enum has three members, and no fourth	[E02 I6, §03]
I7	Nothing resolves to "latest"	A decision against an unpinned version cannot be reconstructed	C13 version pinning; `define` rejects a version that is not `last + 1`	[E02 I7, C13]
I8	Determinism: same intent, declaration version, policy version and `as_of` give the same decision forever	Makes a 2027 refusal replayable in 2035; separates policy from time	C4 nightly replay of every reference-tenant refusal; a stage that acquires a clock, a random value or an undeclared read fails that night	[E02 I8, C4; E01 §06 item 5]

2.3.2 Structural principles

One write function, and no second write path, including for demonstrations [O01 §02]. Migration is a declared write type with blast radius `batch`, through the kernel, at a declared rate; an admin fix is a declared write type under dual control [EPA §02]. The principle's cost is visible and accepted: four proof entries for one escalated business decision, because the kernel has no suspended-and-resumed state [E04 §04 Flow 3].

Policy is data; a declaration is never a programming language. "If a declaration needs a loop, the design is wrong" [IM §6.6]. This is what makes a pack possible: a new regime needs a declaration, gate parameters, a verifier profile and an approver model, and no kernel code [IM §4.5]. Compilation is deterministic and content-addressed, so "the screen an approver saw in 2027 can be rebuilt in 2035" [IM §6.6].

A level is a property of the proof and the claim, never of the policy. Any design that would require re-declaring on upgrade "is rejected on sight" [E07 §01]. This is why the sixteen-field record is frozen at L1 although unsigned [E01 decision 2].

Stopping allows is the safe direction. The kernel and the runtime fail in different directions: a runtime outage stops writes; a kernel outage stops allows [S03 §06]. Every elan1 component is designed so that its failure produces a refusal, a platform-class code or a paused obligation, never a write. Platform-class codes are "the honest ones": a governance layer that reports its own failures as policy refusals "makes them believe their controls are working when our infrastructure fell over" [E02 §06].

Residency and key custody have no migration path. Plane P0 is one of two planes labelled shut; a tenant is created in a residency domain and stays there, and moving one means a new tenant, a fresh chain, and the old chain closed with a sealed record [O01 §02; EPA §03].

2.4 The perimeter

The entity raising is elan1 Private Limited, holding "the governance kernel, the Declaration ontology, the seal and evidence specifications, the verifier, the pack portfolio and the Action Receipt work" [IM §2.1]. The narrowing is deliberate: "an investor in a kernel should be underwriting a kernel, not a portfolio" [IM §2.1].

Inside the perimeter (elan1)	Outside the perimeter
The kernel and KernelPort; the nine-rung path; the refusal register	**rung1** — rung1 Enterprise (built first), rung1 Revenue, rung1 Commerce, rung1 Voice; its hosted tenants, adapters, console screens and price list

Inside the perimeter (elan1)	Outside the perimeter
The Declaration ontology, including the Controls and Regulations families	**stack1** — the `fusion.*/v1` runtime contract layer and Day-2 operation
The seal, the evidence bundle, the offline verifier and the verifier-profile mechanism	**verna1** — regulated outbound collections voice; parked under DR-007
The regulatory packs — ledger1, invoice1, collect1 and those that follow	Retired names: core1, rev1, commerce1, gtm1, fusion1, start1, run1, academy1
The MCP server and the SDKs through which third parties call the kernel	Customer systems of record; identity providers; model providers
The Action Receipt specification and the certification programme	—

Sources: [IM §2.1; O10 §05; O01 §04].

Two related-party relationships cross the line, and both are contracts rather than code dependencies [IM §2.2]:

- **rung1 is the first production consumer.** From 1 January 2027 it pays elan1 ₹0.40 per committed governed action, refusals at zero, reset quarterly against measured cost. The money is small; the load is the point: "a real write path under a real ERP with real customers eighteen months before an external tenant sees it", and the first measured cost per governed action. The internal line is modelled separately so it can be stripped out in one subtraction [IM §2.2; O11 §05].
- **stack1 supplies the runtime**, billed at observed internal cost plus 15%, appearing in elan1's accounts only as cost of revenue. "If stack1's own round does not close, elan1's exposure is a supplier substitution, not a capital call" [IM §2.2].

verna1 is outside for a technical reason as well as a commercial one: regulated outbound collections brings TCCCPR/DLT duties, a carrier and NBFC counterparties, and putting that perimeter around the entity that holds the kernel is "how a licence question becomes an existential one" [O11 §02]. rung1 Voice is different: it is an inbound approval channel on a tenant's own number, a channel on plane P6 that needs a `voice` channel kind in Declaration block 7 and "does not reopen verna1" [O09 §03 item 9, decision 7].

The collections pack, collect1, is inside the perimeter because a pack is configuration on the kernel; it is not verna1. Its own sketch records the honest limit: "the write into the lender's collection system may belong to their core software. If it does, this pack is Attested advisory on that path, not Enforced" [IM App. E.1; E27].

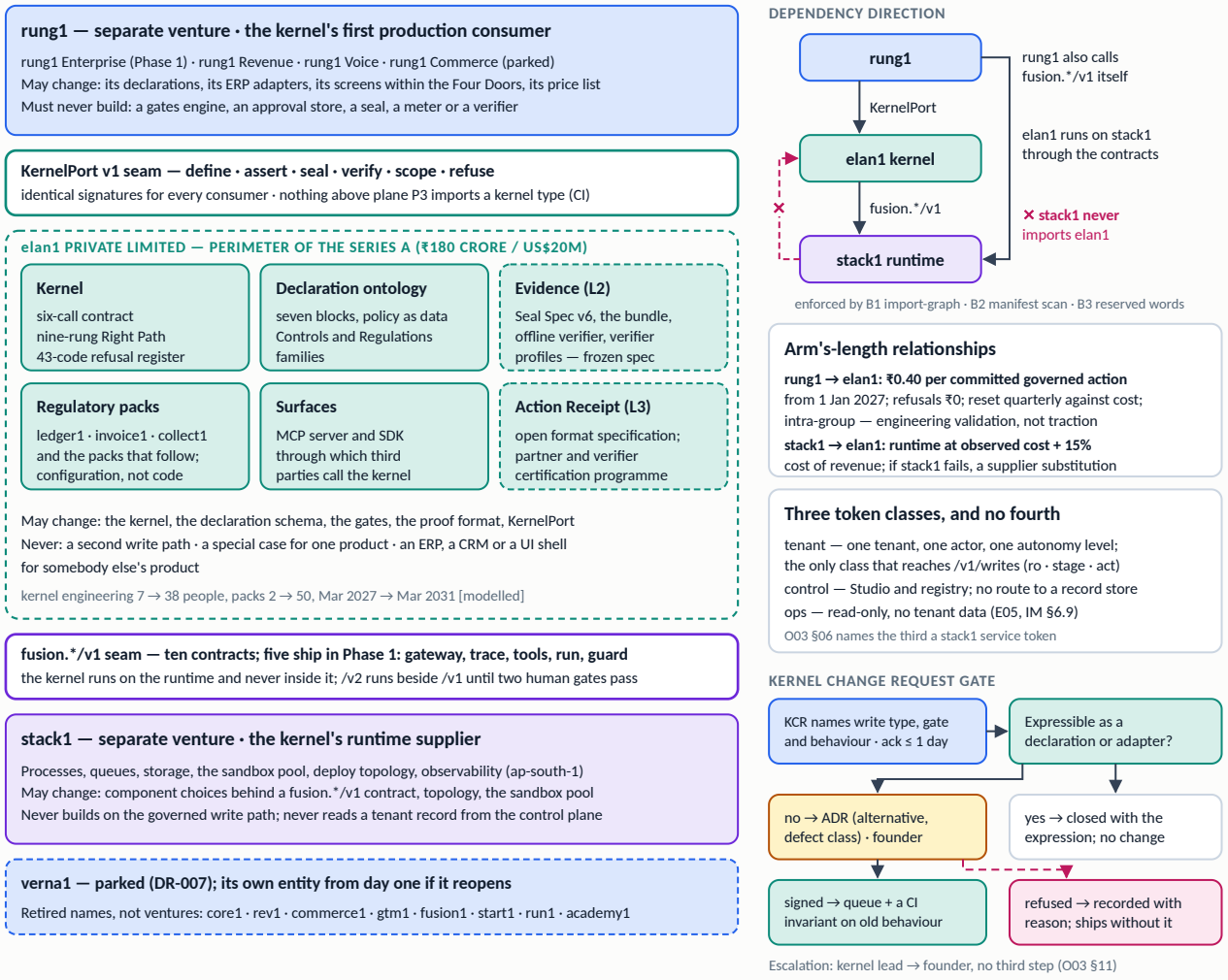
2.5 The Three-Brand Integration Contract

O03 is the seam document: "There is one write function reached through six calls; rung1 and stack1 bring Declarations and adapters and change nothing else" [O03 §01]. It fixes six things that every engineer on either side of the line must know. Chapter 4 specifies the calls field by field; Chapter 6 specifies the surfaces.

Figure 2 — The perimeter and the seams

L1 — BEING BUILT

What elan1 Private Limited holds, the two separate ventures on either side, the two seams and the gate on kernel change



One kernel, reached through six calls, running on a runtime it never lives inside. A product team that ships its own gate has built a second kernel.

Source: IM §2.1–2.3, §6.1–6.2, §6.9, App. B; O03 §02, §06, §09–12; O11 §02; E02 §01, §04; E05 §05; S01 §02; S03 §02–03 · elan1 Technology Memorandum · Sept 2026

Figure 2 — The perimeter and the Three-Brand seam: elan1 Private Limited (kernel and packs) between rung1 above and stack1 below, with the one-way dependency direction, the KernelPort and fusion.*/v1 seams, and the three token classes.

2.5.1 One port

rung1, stack1 and any third party reach the kernel only through KernelPort — `define` · `assert` · `seal` · `verify` · `scope` · `refuse` — with identical signatures for every consumer [O01 §02; E02 §01]. Every call is attributed to a tenant (an unattributable call is rejected, not defaulted) and to an actor; declarations are pinned by explicit integer version; all six calls are idempotent [O03 §05; E02 §04]. KernelPort call shapes are frozen for Phase 1, including `seal`, whose contents change at L2 but whose signature does not [O03 decision 5; O01 DR-013].

The port is described differently in its two governing documents. E02 (the kernel contract) has `define` register a write type at a declaration version, `assert` run all nine rungs and return `committed`, `refused` or `escalated` with `seq` and `digest`, and `seal` callable only by the kernel [E02 §05, decision 3]. O03 (the integration contract) has `define` register a Governed Action in state `proposed`, `assert` run rungs 0b–6 and return `allow`, `refuse` or `needs_approval`, the kernel then execute rung 7, and — in Mode B — the caller report the outcome through `seal` [O03 §03–§05]. The IM follows E02 in §4.2 and O03 in its December 2026 demonstration script [IM §4.2, §16.2.3]. This memorandum follows **E02** for the port, uses O03's `needs_approval` only as a screen label for `escalated`, and treats a Mode B outcome report as a separate declared write type rather than a consumer call to `seal` **(build recommendation)**. Chapter 4 carries the full reconciliation; the conflicts block below records it.

2.5.2 Two modes, two truthful claims

	Mode A — SDK in-process	Mode B — MCP / HTTP
Phase 1 user	rung1 Enterprise, hosted	stack1 tooling; third parties from Phase 2
How the kernel is reached	Library call to KernelPort in the caller's process	MCP tool call or HTTPS to the kernel service
Who holds the target credential	The kernel, via the per-tenant credential broker	The caller, unless the target accepts only kernel-issued tokens
Enforcement class	**Enforced**	**Attested advisory** by default; Enforced only by token acceptance
Added latency (target)	≤ 15 ms over the governed-write envelope	≤ 40 ms p95 for the contract call, excluding the target's work
On kernel failure	The write does not happen	The write does not happen if the caller honours the refusal; nothing physically stops it otherwise
What rung 7 does	Performs the write	Nothing; the consumer acts
What the chain contains	The decision and the act	The decision, plus a flag that the caller, not the kernel, performed the act
Truthful claim	"This write could not have occurred without passing the gates."	"Every proposed write was evaluated against a versioned policy and the decision is recorded."

Sources: [O03 §02, §04; E05 §02; E01 §05]. There are exactly two ways to be Enforced: **credential custody**, in which the kernel holds the target credential in the secrets broker and mints a single-use, ten-minute, one-write credential at the moment of the write; and **token acceptance**, in which the target is configured to accept only kernel-issued tokens for the write types in question [E05 §02]. In Phase 1 Enforced is true for rung1 Enterprise in hosted mode and nothing else [O03 §02]. `enforcement_mode` is declared per write type (as `channels[].mode`), sealed into every record, rendered on the Assurance door and printed in every export, and a tool call cannot set it [E05 decision 1; E09 §04]. Promotion from attested advisory to Enforced is a per-write-type review with 30 days of production traffic, a bypass proof against the real target, and a rollback drill; a later unrecorded write reverts the mode at once [E09 §09].

"MCP alone gives advice, not enforcement." A tool description is a suggestion; a model that ignores it breaks nothing. Writing "enforced" on a slide about an MCP integration "is the one claim that would end a first enterprise deal." [E01 §05; O03 §02]

2.5.3 Three token classes

O03 fixes "three token classes and no fourth": a **tenant token** (one tenant, one actor, one autonomy level); a **control token** for platform operations, never able to write a tenant record; and a **service token** for stack1 components calling each other, never able to reach KernelPort [O03 §06]. E05, written for external callers, names the third class differently — an **ops token**, read-only with no tenant data — and IM §6.9 follows E05 [E05 §05]. The two lists are reconciled by scope: the kernel accepts exactly three classes (tenant, control, ops), and stack1's service token exists below the port and is rejected by it

(build recommendation). Within a tenant token, E02 grants one of three scopes — `ro`, `stage`, `act` — with `act` required for `assert` and `refuse` [E02 §04]. Target-system credentials are not tokens at all: they live in the per-tenant credential broker and are minted per call [O03 §06].

2.5.4 Metering and billing

"The kernel meters." rung1 bills its end customer on its own price list; elan1 bills rung1 internally per Governed Action from Phase 1 "so that the kernel's economics are visible before a third party ever buys it"; stack1 bills nobody in Phase 1 and states its cost at observed unit cost [O03 §07; S02 §9]. Refusals meter at quantity zero "so that the count is honest and the bill is not" [O03 §07]. Where two brands quote different governed-action counts, "the kernel's count is the count and the other is a report defect" [O11 §06]. stack1's own six-unit cost meter is a different meter and never counts Governed Actions [S02 §9]; Chapter 19 covers the pipeline.

2.5.5 What each brand may and may not change

Brand	May change	May never change
rung1	Its Declarations, its ERP adapters, its screens within the Four Doors, its price list	Gate logic, approval logic, the proof-record shape, the meter, the autonomy ladder
stack1	Component choices behind a `fusion.*/v1` contract, deploy topology, observability, the sandbox pool	Anything the kernel decides; may not read a tenant record from the control plane
elan1	The kernel, the Declaration schema, the gates, the proof format, KernelPort	May not add a second write path; may not special-case a product

Sources: [O03 §10; IM §6.2]. Three rules are enforced in CI rather than prose: nothing above plane P3 imports a kernel type; exactly one write credential exists per environment; the control plane has no route to the record store [O03 §10; IM §6.1].

2.5.6 Kernel change requests

A product team that needs different kernel behaviour opens a **Kernel Change Request** naming the write type, the gate and the exact behaviour. The kernel lead acknowledges within one working day and applies one test: *can this be expressed as a Declaration or an adapter?* If yes, the request closes with that expression and no kernel change happens. If no, the kernel lead writes an ADR naming the alternative considered and the defect class prevented; the founder signs or refuses; on signature a CI invariant is added that fails if the old behaviour returns. Escalation stops at the founder, "and no product deadline is a reason to skip step 2" [O03 §11]. The ADR number is "the artefact an auditor asks for" [IM §6.11]. A product team that ships its own gate "has built a second kernel, and DR-001 exists to prevent exactly that" [O03 §11; O01 DR-001].

2.6 Dependency direction

The direction is fixed and one-way: **rung1 → elan1 → stack1**, and rung1 → stack1 directly for its own runtime needs. rung1 depends on elan1 only through KernelPort; elan1 depends on stack1 only through `fusion.*/v1` contracts and operational commitments; **stack1 never depends on elan1** [S03 decision 2; O03 §01].

Rule	Statement	Enforced by	Source
B1	No package under `stack1/` imports any package under `elan1/`	Import-graph check in CI; fails the build	[S03 §03]
B2	No elan1 package name appears in any stack1 manifest, image layer or Helm chart	Manifest scan of built images	[S03 §03]
B3	`approve`, `allow`, `gate`, `refuse`, `proof`, `seal` are reserved in stack1 symbol and endpoint names	Symbol-name check in CI	[S03 §03]
B5	The stack1 control plane never calls the tenant plane; the tenant plane writes to the evidence plane only through the transactional outbox	Default-deny network policy at deploy time	[S03 §03]
B7	No domain noun (`invoice`, `approval`, `ledger_entry`) in a `fusion.*` descriptor	Descriptor lint	[S03 §03]

Rule	Statement	Enforced by	Source
P3	Nothing above plane P3 imports a kernel type	Import-graph check in every consuming repository's CI (C1)	[O01 §02; E02 §01]

The reasoning is one sentence: "infrastructure gets swapped; the write path must not" [S03 §02]. A stack1 substitution — LiteLLM to Bifrost behind `fusion.gateway/v1`, Temporal to another engine behind `fusion.run/v1` — is a routine, conformance-gated change. If the kernel were a runtime component, such a change "could alter refusal semantics, approval semantics or what a proof record contains", and a tenant would have two write paths in one quarter with the same name [S03 §02]. A `fusion.guard/v1` verdict (content is third-party; a string looks like an injection) is therefore an **input** to the elan1 gate check and never the allow [S03 decision 4].

The asymmetry has a planning consequence: stack1 slippage reaches elan1 (key custody and the anchor job depend on it) but not rung1, and "that asymmetry is the reason the boundary exists" [O09 §06].

2.7 What elan1 refuses to build

The things elan1 will not build shape the architecture as much as the things it will. The IM compresses the boundary into one line each: "rung1 must never build a gates engine, an approval store, a seal, a meter or a verifier; stack1 must never build anything on the governed write path; elan1 must never build an ERP, a CRM, or a user-interface shell for somebody else's product" [IM §6.2]. The fuller list below is assembled from the kernel contract, the scope freeze and the onboarding rules.

elan1 will not build	Why	What the architecture does instead	Source
An ERP, a CRM or a UI shell for another product	Ties the kernel to one consumer's release train and competes with the consumers it serves	Consumers render the elan1 surfaces inside their own Four Doors; elan1 owns only the shared P5 renderers and its own Platform console	[IM §2.3; O01 §02 P5]
A second write path — for bulk import, migration, demonstrations or "just this once"	Two paths mean two definitions of "allowed"	Migration is a declared `batch` write type through the kernel at a declared rate	[O01 §02; EPA §02]
A fast path, batch variant, admin mode or decision cache	Each becomes the path that is used under pressure	Break-glass is a declared write type with `dual_control: true`, in the operator's own chain	[E02 I5, decision 9]
A model in the decision path, or a proprietary model	I1; a model cannot be versioned or replayed like a rule	Models stay upstream of the intent, in the consumer, through `fusion.gateway/v1`	[E02 I1; GAM §1]
Identity and authentication	Consolidating fast around Entra Agent ID, Okta for MCP and SPIFFE; "consume identity, differentiate on enforcement, approval and evidence"	The token establishes tenant and actor; the kernel sets `actor_kind`	[IM §13.2]
A per-customer meter basis	"A per-customer kernel arriving through the finance department"	Basis mapped normatively from the declared blast radius	[E09 §08; E05 §06]
A switch for any of the six never-skippable gates	That is "a different product"	Only rate, budget and schedule may be declared off, with an ADR	[E09 §08; IM §6.6]
Key material, a verifier, a bundle or a regulator mapping at L1	Each promises something the chain cannot yet support	L1 states its limits; L2 adds them under E14	[E01 §07]
A priced verifier	"A verifier you have to buy from the vendor whose records it checks is not a verifier"	Source, free, to anyone, at every level	[E07 decision 3; IM §7.5]
A pack as a code fork, a codebase or a brand	Packs must scale with regulatory fragmentation; forks do not	A pack is configuration: declaration, profile, export mapping, approver model	[O10 §04; IM §5.1]
A sovereign deployment before a defect-propagation process exists	"The capability most likely to be sold before it is ready"	Held as an L3 deployment model, specified in Phase 4	[IM §8.5; E20 §03]

Two runtime exclusions bind elan1 as a consumer of stack1: n8n and Inngest are excluded because neither licence permits them inside a paid hosted service, and Windmill is not adopted in v1 [O01 DR-003, DR-004]. Temporal owns anything touching a customer's books [O01 DR-002].

2.8 How the architecture evolved, and why the narrowed kernel is the right cut

The present design is the fourth reframing of one idea in roughly three years. Each reframing kept a core and changed the frame around it [MMS §02; IM §2.4].

Version	Dates	Frame	What it built or specified	What survived	What was dropped
ELAN 0 — "Enterprise 1"	to Aug 2026	An agentic ERP	Ten applications, ten industry packs, the Four Doors grammar, a Declaration ontology, a nine-category execution path; scrapped after about a hundred days	The Declaration and the execution path — "recognisably the two things elan1 sells today"	Building the system of record
ELAN 1.0 — GAM and GIP	GIP 3 Sep; Horizon 1 9 Sep 2026	A Governed Intelligence Platform: "an ERP layer, not a replacement"	GAM: "governed writes generate the attested ontology that trains the small models that win the writes"; GIP: a clean-sheet shell over the existing core and a four-tier price ladder; Horizon 1 derived the working tree by execution	The Governed Action as the unit that is typed, gated, executed, recorded and billed; fail closed at the write boundary; proof as a first-class object	The model flywheel inside the product; the word "platform"; the tier ladder
ELAN 2.0 — Boomi and Horizon 2	11–12 Sep 2026	A governed AI operating system	Boomi read as the closest live analogue — governance outside the agent, billable atom equal to audited atom — with "the absence of a proof primitive" as "the most significant unclaimed space"; Horizon 2 produced a status ledger and a programme	Governance outside the agent; the meter atom equal to the evidence atom; proof as the differentiator	"Govern everyone's agents" as a moat (a claim "most likely to be copied within 12 months")
ELAN 3.0 — product pack	16–20 Sep 2026	A governed write plus proof	Build Book and Strategy Blueprint: sixteen-field Ed25519 seal, 64 vectors, a Go verifier, twenty apps, ten evidence packs, five zones; MMS made rung1 the Milk and the elan1 proof kernel the one Mountain	The sixteen fields, the vectors, the two-implementation rule, the bundle and verifier design	elan1-owned apps; the seal-first build order
v3.0 E/O series and the IM	21–22 Sep 2026	The governance kernel and its packs, at L1 first	KernelPort v1, the Right Path runtime, Declaration spec, three assurance levels with L1 only in Phase 1, the Three-Brand contract, the decision register, the perimeter	Everything above that survives the "designed is not shipped" rule	The apps (to consumers); L2 implementation in Phase 1

Sources: [IM §2.4; MMS §02, §08; GAM §1–§2; GIP §0; HFS Part 1; BOOMI §20–§22; HV2 §13; EPA §01, §14; O01 §05].

Across the rewrites three ideas never moved: **one governed write path, declarations instead of code, and proof as the product** [MMS §02]. Everything else — ERP apps, verticals, UI grammar, pricing — changed from version to version. MMS's most valuable engineering finding was that "the same governance kernel [was being] designed separately each time" in rung1, rev1, commerce1, verna1 and stack1, each already exposing the same six-call KernelPort: "Build it once, inside the elan1 Mountain, and ship rung1 on it" [MMS §08].

2.8.1 The decision register entries that bind the architecture

The register is closed; reopening an entry needs a superseding numbered entry [O01 §05].

Entry	Decision	Consequence for elan1's build
DR-001	One kernel; products vary only in Declarations, adapters and, from L3, verifier profiles	elan1 owns the only gates engine, approval store, proof log and meter in the group
DR-002	Temporal owns anything touching a customer's books	Rung-7 execution and approval signals ride `fusion.run/v1`; replay must not re-run side effects

Entry	Decision	Consequence for elan1's build
DR-006	L1 only in Phase 1: no seal, no RFC 3161, no verifier profiles	`seal` returns a hash-chain receipt; no key material exists
DR-011	One numbering scheme: planes P0–P8, autonomy A0–A5, gates G1–G9, register tiers REG-T1–T7	Identifiers in code; the colliding v2.0 "L0" meanings are gone
DR-012	Declaration block 7 is `channels`	A breaking schema change, published as a schema major before L1 GA
DR-013	`seal` stays in KernelPort at L1 and returns a hash-chain receipt	No breaking port change between Phase 1 and Phase 3

2.8.2 Why the narrowed kernel is the right technical cut

It concentrates the build on the part nobody else has. Boomi's own analysis finds governance and audit logs everywhere and "nothing … describes a cryptographically signed, hash-chained, independently verifiable proof of a permitted write" [BOOMI §22]. The platform vendors govern what runs inside them: none governs a write into SAP, Oracle, Temenos, Finacle or Tally, "and none produces evidence a party that does not trust the vendor can verify" [IM §1.6]. A kernel that sits on the write, outside the system of record, is the piece that cannot be bought from any of them.

It is where the prototype's hardest defects are, and where they are cheapest to fix. Horizon 1 derived the existing tree by execution and found that its costliest foreclosures sat on the write path: no sealed sequence number and no anchor ("the only item … where delay destroys evidence that cannot be recovered"), run-record attribution written as constants, the act inside the same database transaction as the proof, and a declarations table whose unique key forbids versioning [HFS Part 2]. Each maps to an E02 invariant — `seq` in F02, `actor_kind` from the token, the outbox between rung 7 and rung 8, versions immutable and `last + 1`. The costliest seam outside the write path — four incompatible declaration dialects feeding hand-written screens [HFS "Worst seam"] — is exactly the product-surface work the narrowing hands to consumers.

A frozen interface amortises. "Freezing that interface is why elan1 is a kernel with consumers rather than a product" [IM §4.2]. Kernel engineering is planned to grow from 7 to 38 people by March 2031 while pack and regulatory engineering grows from 2 to 50, because "a frozen surface needs a team that maintains and hardens it, not one that grows with revenue"; if the two lines ever cross back, "the kernel is being reopened" [IM §6.2, §15.4].

Determinism is also the economic choice. Deterministic gates and rules cost almost nothing per run and can be replayed for ever; intelligence is spent upstream, once, by the consumer that composes the intent [E02 I8; IM §4.3].

The boundary makes failures separate. A runtime outage stops writes; a kernel outage stops allows [S03 §06]. A consumer outage stops requests. No single component's failure turns into an unrecorded write.

What the cut costs, stated. elan1 depends on consumers for every system-of-record adapter and on rung1 for its first real load; if no ERPNext deployment mode lets the kernel hold the credential or accept only kernel tokens, elan1 sells advice rather than control, and its fourth kill criterion fires [O02 §06]. It forgoes application revenue entirely. And one question from the older frame survives uneasily: Gate 2 — "app three renders its four doors from its Declaration with zero lines of hand-written screen code" — remains in the gate calendar, owned by product and design, although elan1 may not build a UI shell for another product [O09 §07; IM §16.5]. This memorandum reads Gate 2 as a test of the elan1-owned shared renderers at plane P5, exercised by a consumer's declaration, not as elan1 building apps; Chapter 8 carries it.

2.9 Naming rules as architectural constraints

In this group naming is enforced in the build, because "the boundary erodes through naming before it erodes through code" [S03 §03].

Rule	Where it bites	Enforcement	Source
Brand names are lower-case always: elan1, rung1, stack1; pack ids lower-case; retired names do not reappear	Code, repositories, docs, READMEs	Lint on new identifiers **(build recommendation)**	[O10 §05, §09]

Rule	Where it bites	Enforcement	Source
The consumer line is **rung1 Enterprise**, and its voice line is **rung1 Voice**	Declaration identities, repository names, rendered copy	Group naming instruction; the O10 rule against "Enterprise" in a product name needs a dated superseding entry	Group naming instruction; [IM App. E.2 item 1]
L1 copy never says "signed", "tamper-proof", "immutable", "court-ready" or "independently verifiable"	Every rendered string, the Assurance door, exports	Conformance test C14 fails the build	[E02 C14; O10 §09]
Mode B deployments say "attested advisory", never "enforced", "prevented", "blocked" or "cannot"	Copy; `enforcement_mode` rendered per write type	Sealed field; claim-boundary review	[O03 §02; E05 decision 1; O10 §09]
elan1 appears on a rung1 screen in two places only: the proof record ("Checked and recorded by the elan1 layer") and the footer (rung1 · by elan1); at L2 the export bundle carries it prominently	Consumer console shell and renderer templates	Renderer template	[O10 §07]
"Governed Action" is a unit of account, never a button or a heading	UI copy	Copy lint	[O10 §07]

Chapter 3 now opens the box: the planes, every elan1 component with its interfaces and owner, the repositories and import rules, and the deployment units that run on stack1.

3. Reference architecture — planes, components and repositories

Chapter 2 described elan1 from the outside. This chapter describes it from the inside: the plane model and the boundary rules that make it load-bearing; how that model maps to the older strata, zone and tier vocabularies still found in the product folder; every elan1 component with its responsibility, interfaces, state, owning team and assurance level; the internal APIs between them; the repositories and packages and the import-graph rules that police them; the deployment units and what each needs from stack1's `fusion.*/v1` contracts; the reconciliation of the product-folder architecture with the E/O series; and what in the existing prototype carries forward. The governed write itself — the order of the calls, the rungs and the refusal exits — is specified in Chapters 4 and 5. Where the sources are silent the text says **(build recommendation)**. Nothing described here runs for a customer.

3.1 The plane model

The group's canonical architecture has nine planes, P0–P8, read bottom-up as a cost gradient: the lower the plane, the more expensive a mistake there is to undo [O01 §02; DR-011]. They replace the v2.0 "strata L0–L8", whose "L0" collided with the lowest autonomy level and with the declaration schema [O01 §03]. Only two planes are labelled shut — residency and tenancy, because a residency or key-custody decision has no migration path, and the kernel, because everything depends on it [IM §6.1].

Plane	Name	What it holds	How changeable	Owner	What elan1 builds or decides here
P8	Packs	Vertical and regulatory packs: declaration plus verifier profile each	Yes — intended churn	elan1	ledger1, invoice1, collect1 and later packs, as configuration; the pack template and admission gates
P7	Apps	Horizontal apps, one declaration each	Yes	Consumers	Nothing. rung1 Enterprise's declarations are rung1's
P6	Channels	Web, API, messaging, MCP, voice	Yes, per channel	Consumers; elan1 for its own surfaces	The MCP server and HTTP API as channels onto the kernel; the channel kinds allowed in Declaration block 7
P5	Surfaces	Shared renderers and the surface registry; screens are compiled	Hard — one team	elan1	The shared renderers, the surface registry, the Platform console (Studio, Decide, Assurance)
P4	Declaration plane	Declaration schema, write-type descriptors, lexicon, policy versions	Forward only, never migrated	elan1 (schema); authors (declarations)	The schema, the validator and compiler, the registry, the lexicon, the policy-version store
P3	Kernel	The one write function, the nine rungs, `evaluate()`, proof, meter	**Shut**	elan1	The whole of KernelPort and everything behind it, including the evidence store
P2	System of record	Write API and record store; the only place a record changes	Hard	The customer; rung1 hosts ERPNext	Nothing. Reached only by the single writer through a declared adapter
P1	Platform runtime	Compute, queues, object store, secrets, transactional outbox	Yes — deliberately boring	stack1	Nothing of its own. elan1 runs here as a consumer of `fusion.*/v1`
P0	Residency and tenancy	Region, tenant, key custody, erasure salt store	**No migration path**	elan1 (policy); stack1 (operation)	Residency domains, tenant identity, key-custody policy (L2), salt-store policy

Sources: [O01 §02; O03 §10; O10 §03–§04; S03 §01; IM §6.1].

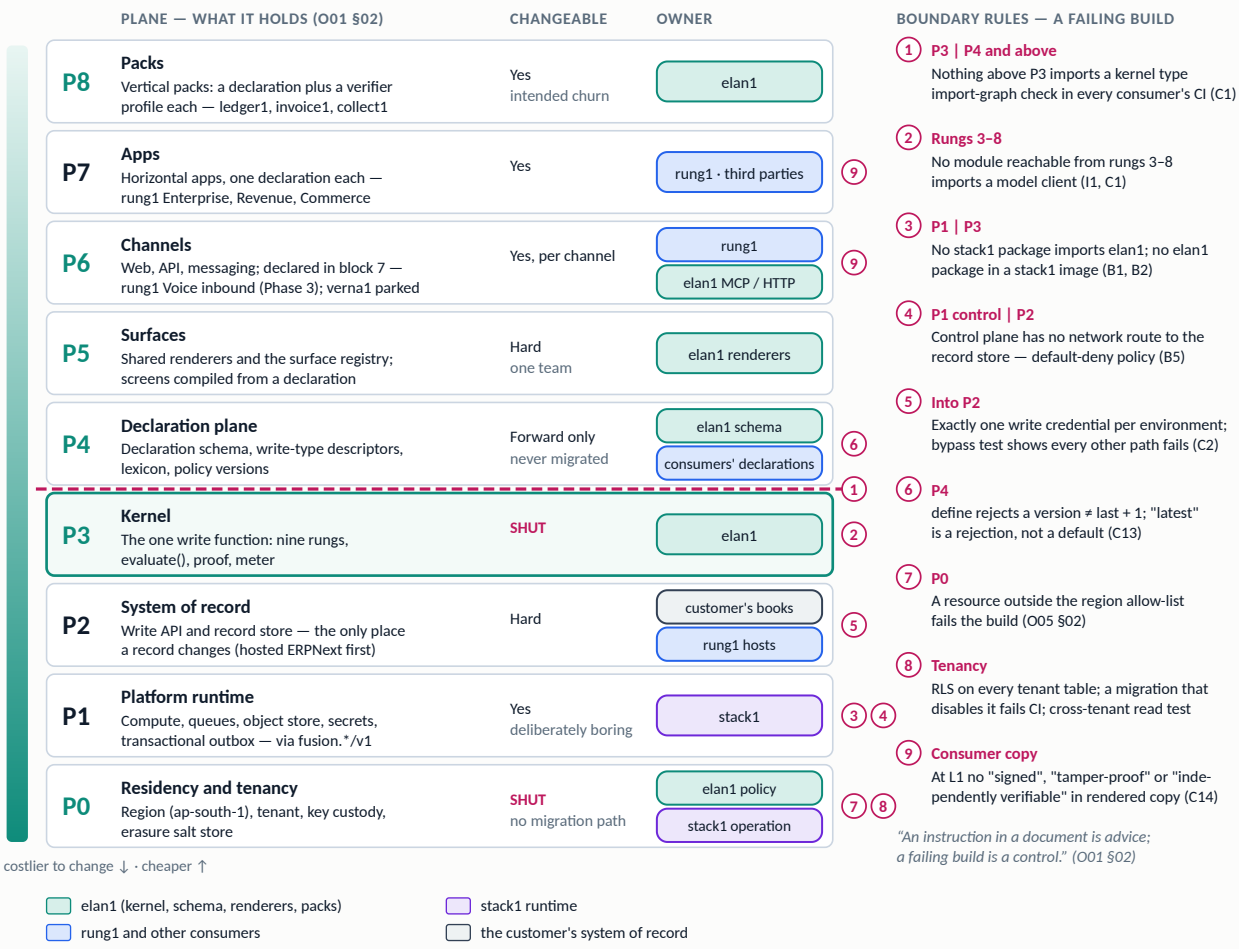
Three readings of the table matter for the build. First, **proof and meter live in P3**, not in a plane of their own: the evidence store is a kernel component, deployed per residency domain, and S03's failure-domain table already calls it "Evidence plane (P3)" [O01 §02; S03 §06]. Second, **elan1 decides P0 and stack1 operates it**: a spun-out stack1 takes "nothing at plane P3 or

below", which this memorandum reads as ownership of the kernel, residency policy and key custody rather than operation of the P1 runtime [O10 decision 5]. Third, **P5 is the only surface elan1 owns**; everything a user of rung1 sees is rung1's screen with elan1's renderers embedded inside the Four Doors [O10 §07].

Figure 3 — Planes P0–P8 and who owns each

L1 — BEING BUILT

Nine planes read bottom-up as a cost gradient, the brand that owns each, and the boundary rules a failing build enforces



Only two planes are shut: P0, because a residency or key-custody choice has no migration path, and P3, because everything depends on it.

Source: O01 §02, DR-011; O03 §10, §12; IM §6.1–6.2, §6.5, §6.10; E02 §01, §08; S03 §01, §03; O05 §02; O10; RTM Ch.3 · elan1 Technology Memorandum · Sept 2026

Figure 3 — The nine planes, P0 to P8, with the owner of each, the planes elan1 owns (P0 policy, P3, P4, P5, P8), and the boundary rules that are enforced in CI or by network policy rather than in review.

3.1.1 Boundary rules on the plane model

Eight rules turn the plane table into an architecture. Each has a mechanical enforcement, and each is a check a diligence engineer can ask to see fail.

#	Boundary	Rule	Enforced by	Source
BR1	P3 / everything above	Nothing above P3 imports a kernel type; consumers hold identifiers, enumerations and the five objects	Import-graph check in every consuming repository's CI (C1)	[O01 §02; E02 §01, C1]
BR2	Inside P3	No module reachable from rungs 3–8 imports a model client	Import-graph check (C1); a forbidden-module contract (§3.5.3)	[E02 I1, C1]
BR3	P1 / P3	The kernel never lives inside the runtime: no stack1 package imports elan1, no elan1 package appears in a stack1 image	Import graph (B1); manifest scan of built images (B2)	[S03 §03]

#	Boundary	Rule	Enforced by	Source
BR4	Control plane / record store	`ctl`-class paths have no read path to any record store — "not a restricted one, none"	Default-deny network policy at deploy time; no route Z2 → Z3	[E05 §04; EPA §16 C05; S03 B5]
BR5	Evidence / record store	The evidence store reads the outbox and never reads a record	No route Z4 → Z3; no foreign key from evidence tables into record tables	[EPA §13 rule 7, §16 C06]
BR6	Into P2	One write credential per write, minted only at rung 7 against an allow bound to a `write_id`	Credential-scope tests; bypass test (C2); nightly "zero records mutated outside the kernel writer"	[E02 I2, C2; E01 §06 item 4]
BR7	P4	Declarations are forward only; `define` rejects any version other than `last + 1`; old governed actions keep resolving against their own version	`define` idempotency and ordering rules; C13	[E02 §05; O03 §09]
BR8	P0	No resource outside the region allow-list; a tenant never changes residency domain	CI fails the build on an out-of-region resource; residency has no migration API	[IM §16.2.4; EPA §16 C13]

Two of the eight are absences rather than code — a missing network route and a missing grant — and the product folder's security review order starts there for that reason: "Nine of the twenty-two controls are a missing route, grant or API — which the next application cannot bypass" [EPA §16].

3.2 Other vocabularies, mapped

Engineers will meet four other ways of slicing the same system in the product folder and in the prototype. None of them is wrong; each describes something different. The P-planes are the canonical vocabulary and the only one used in code identifiers [O01 DR-011].

Vocabulary	Where it appears	What it describes	Mapping to this memorandum
Strata L0–L8	Strategy Blueprint, Build Book, EPA §02	The same nine layers as P0–P8, under the retired letter	L*n* = P*n*. "Nothing above L0 imports a kernel type" (E02 §01, EPA §20 note 7) means "nothing above P3" [O01 §02]
Runtime planes C / S / E	EPA §02, ETA	Where things run: control (authoring, registry, policy), system of record (write API, kernel worker, record store, outbox), evidence (seal worker, evidence store, anchor, bundle)	A security view across P3–P5; "plane E is not a stratum" because a seal binds tenancy (P0), record (P2), decision (P3), declaration version (P4) and pack profile (P8)
Zones Z1–Z5	EPA §03, ETA, ETS	Deployment: Z1 edge and channels (shared, stateless); Z2 control plane (global, no tenant records); Z3 record plane (per residency domain); Z4 evidence plane (per residency domain); Z5 customer and auditor (not ours)	The deployment view used in §3.6; Z2 → Z3 and Z4 → Z3 routes do not exist
Module tiers T1–T4	EPA §14, Strategy Blueprint ch. 7	Package dependency tiers: T1 core (`kernel`, `seal`, `meter`, `verify`), T2 semantics (`declare`, `lexicon`, `policy`, `autonomy`), T3 composition (`doors`, `renderers`, `registry`, `studio`), T4 products (apps, packs, channels)	Dependencies point down only; §3.5 keeps the rule and moves T4 apps out of elan1

The product folder and the prototype still use `sor_write.governed()` as the name of the one write function [EBB; ERD; HFS]. This memorandum keeps the name for the internal entry point behind `assert` (§3.4).

3.3 Component catalogue

Figure 4 shows the components and the repositories that hold them. The table lists every component with its responsibility, its state, the team that owns it (using the four elan1 workstreams of §1.6), and the assurance level at which it is built. Service names follow the product folder's `e1-*` convention where one exists [EPA §03]; the rest are this memorandum's **(build recommendation)**.

#	Component	Responsibility	State it owns	Plane / zone	Team	Level
1	**Write API** (`e1-write-api`)	The only public entry to a governed write: tenant token → one tenant, one actor, one autonomy level; idempotency on `intent_id`	Idempotency index (30 days)	P3 / Z3	Kernel	L1
2	**Kernel core** (`e1-kernel`) — stage pipeline	Runs rungs 0b–9 as stages S01–S12 over an append-only write context	`write` rows	P3 / Z3	Kernel	L1
3	**Gate engine** (S05)	Runs the declared pre-gates G1–G9 in declared order as pure functions of context and `as_of`; runs every gate even after the first failure	`gate_result` rows	P3 / Z3	Kernel	L1
4	**Policy evaluator** (`e1-policy`, S06)	`evaluate(request, declaration_version, policy_version)` over a projection of declared fields; allow, refuse or escalate	`policy_version`, `policy_rule` (versioned, append-only)	P3–P4 / Z2	Kernel	L1
5	**Declaration registry and compiler** (`e1-registry`)	`define`: validate (eleven rules), simulate, certify, compile to surface plan, registry entries, meter basis and verifier-profile stub; resolve by explicit version	`declaration`, `write_type`, `certification`, `surface_plan` (append-only)	P4 / Z2	Kernel (schema); Pack (content)	L1
6	**Lexicon service** (`e1-lexicon`)	Holds term sets by hash; supplies F15 `lexicon_hash`	`lexicon`, `lexicon_term`	P4 / Z2	Pack	L1
7	**Obligation and approval service**	Creates obligations at rung 6; renders the approval basis; records approvals and declines as governed writes; expires obligations	`obligation`, `escalation`, `approval`	P3 / Z3	Kernel	L1
8	**Stop service**	The rung-0b stop set: tenant kill switch, revocation, blacklist, regulator directive	`stop_event`	P3 / Z3	Kernel	L1
9	**Single writer and credential broker** (`e1-connect`, `e1-keys`)	Rung 7: snapshot, shadow-sandbox apply, gate re-run, identical bytes to live through a single-use ten-minute credential, gate re-run on live, record + outbox in one transaction	`record_version`, `outbox`, snapshot references (30 days)	P3 → P2 / Z3	Kernel	L1
10	**Chain writer** (`e1-seal`) and **proof store** (`e1-evidence`)	Rung 8: consume the outbox in commit order; assign `seq`; chain `prev_digest`; SHA-256 over the frozen preimage; append	`seal` (the record), INSERT-only to one principal	P3 / Z4	Evidence	L1 (signing at L2)
11	**Meter** (`e1-meter`)	Rung 9: one meter event per governed write; basis from blast radius; refusals at zero by `CHECK`; nightly two-way reconciliation; invoice feed	`meter_event`, `meter_basis`	P3 / Z3	Kernel	L1
12	**MCP server**	34 tools in three scopes (18 `ro`, 10 `stage`, 6 `act`); descriptions pinned by hash in the declaration	None (stateless)	P6 / Z1	Kernel	L1 (external GA Phase 2)
13	**HTTP API**	Eight endpoint families (`/v1/writes`, `/obligations`, `/records`, `/evidence`, `/meter`, `/declarations`, `/actors`, `/tenants`)	None	P6 / Z1–Z2	Kernel	L1
14	**SDKs**	Mode A in-process port (Python); Mode B clients (Python, TypeScript)	None	Consumer process	Kernel	L1
15	**Webhook dispatcher**	HMAC-signed, at-least-once events carrying `seq` and `digest`	Delivery log	P3 / Z3	Kernel	L1

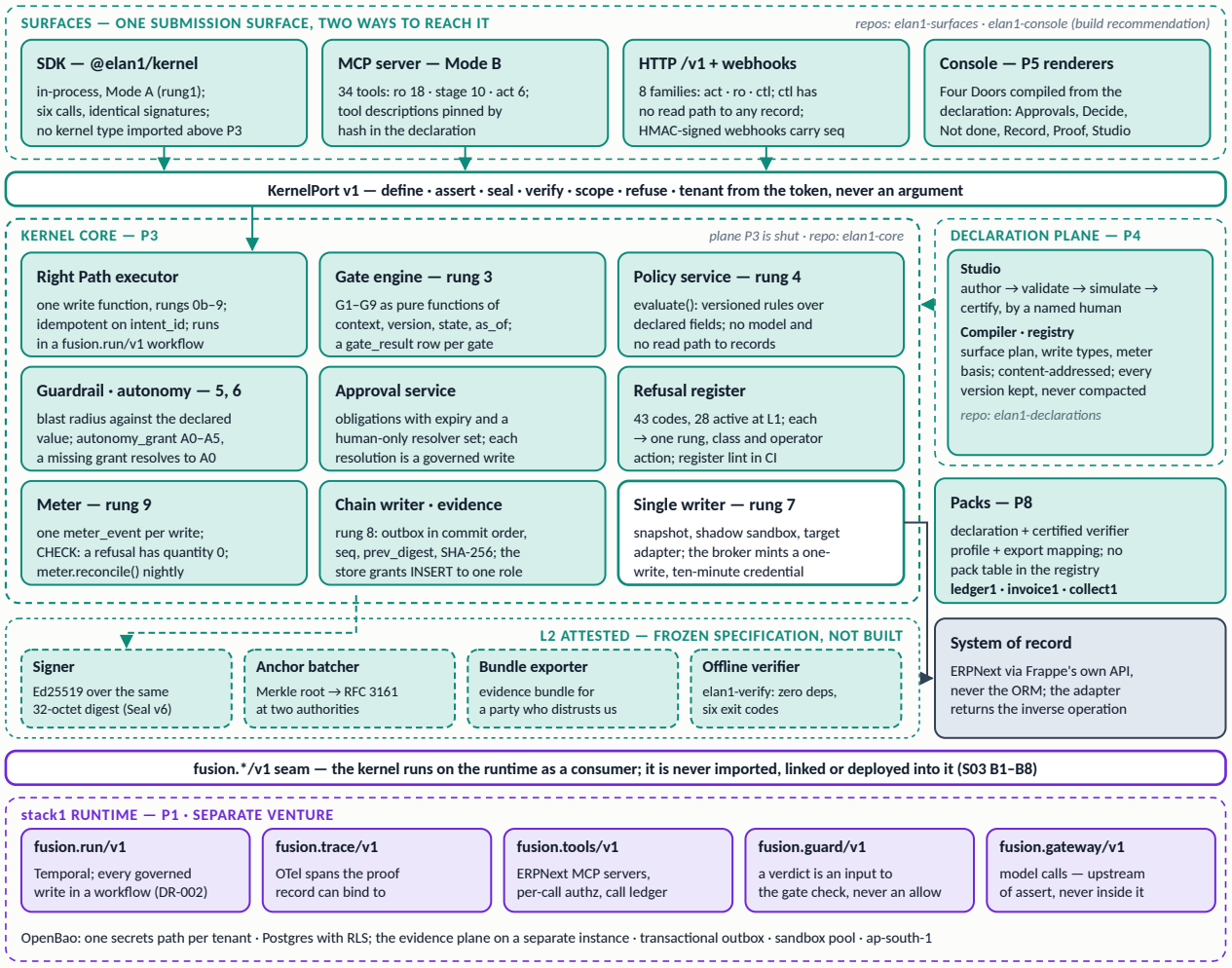
#	Component	Responsibility	State it owns	Plane / zone	Team	Level
16	**Shared renderers and Platform console**	P5 renderers embedded in consumers' Four Doors; elan1's own console: Studio, Decide, Assurance, Meter	`surface_plan` (read)	P5 / Z2	Platform & assurance	L1
17	**Integrity and reconciliation jobs**	Nightly chain verify; write-to-record join in both directions; meter reconciliation; determinism replay (C4)	Job results; status-page component	P3 / Z3–Z4	Platform & assurance	L1
18	**Reference tenant, conformance suite and sandbox provisioning**	Synthetic fixtures; adversarial declaration; C1–C14 runnable by consumers; self-serve sandbox keys	Fixtures; result-file hashes	P3–P4	Platform & assurance	L1
19	**Pack artefacts and profile registry**	Pack template (eleven blocks), admission-gate scores, verifier profiles (dry-run at L1, executing at L2)	Portfolio register; `verifier_profile`	P8 / Z2	Pack	L1 (dry run); L2
20	**Anchor service, bundle service, key register, salt store**	Merkle batching and RFC 3161 tokens; eight-member export; key windows; per-record salts	`anchor_batch`, `anchor_token`, `inclusion_path`, `key`, `key_window`, `erasure_salt`, `bundle`	P0, P3 / Z4	Evidence	**L2, specified only**
21	**Offline verifier** (`elan1-verify`)	Zero-dependency verification of a bundle; six exit codes	None	Z5 (shipped, not hosted)	Evidence	**L2, specified only**

Sources: [E02 §03–§05; E04 §03; E05 §03–§09; E09 §02; EPA §03, §07, §13; IM §5.1, §7.4–7.5].

Figure 4 — Components, repositories and internal seams

L1 — BEING BUILT · L2 — NOT BUILT

What elan1 builds for L1, where each part sits, the frozen L2 parts beside it, and the two seams in and out



Repositories: elan1-core and elan1-declarations are recorded (O03 §12); elan1-surfaces, elan1-console, elan1-packs and elan1-verify are this memorandum's proposal (build recommendation). One writer per file; a pull request crossing two repositories needs both owners.

Every submission lands on one function; the only credential to the books sits at rung 7, and the control plane cannot read a record.

Source: E02 §03–06; E04 §03; E05 §03–09; E06 §01–07; E18 §01; O03 §05–06, §10, §12; O01 DR-002; S02 §02; S03 §01–03; IM §6.9–6.10 · elan1 Technology Memorandum · Sept 2026

Figure 4 — The elan1 components by zone, from the submission surfaces through the kernel stages, single writer, chain writer and meter, with the declaration registry and policy service in the control plane, the Level 2 components drawn as specified-only, and the repository that holds each.

3.3.1 Submission: the write API, the MCP server, the HTTP API and the SDKs

There is **one submission surface and two ways to reach it** [E05 §01]. In-process consumers link the kernel behind KernelPort (Mode A); out-of-process consumers reach the same entry over MCP or HTTPS (Mode B). Both land on rung 0b, and no interface can set `actor_kind`, `seq`, `policy_version` or the proof timestamp [E05 §01; IM §6.9]. The write API is therefore thin: it authenticates the token, rejects an unattributable call (4xx, no record), rejects a body whose `tenant_id` differs from the token's (a rejection and an alert, not a refusal), deduplicates on `intent_id`, and calls the kernel entry point [E02 §04; E05 §08].

The **MCP server** publishes 34 tools in Phase 1: 18 `ro` tools that cannot reach rung 7 by any path, 10 `stage` tools that produce a validated request object with no side effects, and 6 `act` tools that submit a governed write [E05 §04]. The `stage` scope exists so that an A1 or A2 agent can do useful work without producing a stream of refusals; "autonomy is a kernel concern, not a tool-catalogue concern" [E05 §04]. Tool descriptions are pinned by hash in the declaration, so a description

change is a new declaration version and needs certification by a named human [E05 decision 5]. The server is stateless and its tool list is generated from the declaration's `write_types` and `channels` blocks rather than hand-maintained **(build recommendation)**, which is the Horizon 2 proposal to "generate MCP tools from block 4" [HV2 §13 P6].

The **HTTP API** has eight endpoint families in three classes: `act` (`/v1/writes`, `/v1/obligations`), `ro` (`/v1/records`, `/v1/evidence`, `/v1/meter`) and `ctl` (`/v1/declarations`, `/v1/actors`, `/v1/tenants`) [E05 §04]. `/v1/evidence` pages by `seq`, "so contiguity is the pagination contract and the integrity property at the same time" [E05 §09]. Refusals return HTTP 200; throttling returns 429 with `Retry-After` and produces no proof entry and no meter event [E05 §06–§07].

The **SDKs** are two artefacts. The Mode A port is a Python package, because the first consumer runs on Frappe/ERPNext and the prototype kernel is Python 3.12 on FastAPI [ETS §01; HFS Part 1]; it exposes the six calls and the five objects and nothing else **(build recommendation)**. The Mode B clients (Python and TypeScript) wrap the HTTP surface, generate `intent_id` values, retry 5xx and 429 with exponential back-off and jitter to a maximum of five attempts with the same key, and never retry a refusal [E05 §08].

3.3.2 Decision: the kernel core, the gate engine and the policy evaluator

The kernel core implements the nine rungs plus 0b as twelve stages, each with one responsibility and one owner [EPA §07]:

Rung	Stage	Kind	Reads	Writes	Budget p50 (target)
0b STOP	S01 `resolve_tenant`	Deterministic	Stop set, residency	— (a stopped tenant writes no chain entry)	4 ms for 0b–2
1 IDENTITY	S02 `resolve_identity`	Deterministic	Token, delegation	`write` row begins; `actor_kind`	
2 WRITE DOOR	S03 `resolve_declaration`, S04 `resolve_write_type`	Deterministic	Registry at explicit version	Resolved versions, descriptor	
3 PRE-GATES	S05 `run_gates`	Deterministic	Context, record state, `as_of`	One `gate_result` per declared gate	8 ms
4 `evaluate()`	S06 `evaluate`	Rule, deterministic at a fixed policy version	Projection of declared fields	Decision, `policy_version`	12 ms
5 GUARDRAIL	S07 `check_blast_radius`	Deterministic	Resolved payload, descriptor	Measured radius	8 ms for 5–6
6 AUTONOMY	S08 `resolve_autonomy`	Deterministic	`autonomy_grant`	Obligation on escalate	
7 ACT	S09 `apply`, S10 `enqueue_outbox`	I/O	Target via adapter	`record_version`, one `outbox` row, same transaction	18 ms (kernel transaction only)
8 PROOF	S11 `seal`	Deterministic	Outbox row, `seq − 1`	`seal` record	12 ms (E04) or 40 ms (EPA)
9 METERED	S12 `meter`	I/O	Descriptor	One `meter_event`	2 ms

Sources: [E04 §02–§03; EPA §07]. The stage contract is `stage(ctx: WriteContext) -> StageResult`. The context is append-only within a write — a stage adds fields and never modifies another's — so all sixteen record fields are present when S11 runs without a second read; a `StageResult` is `continue`, `refuse(code)` or `escalate(obligation)`, with no skip and no retry; and a stage may not call another stage [EPA §07; E02 §05 seal step 2].

The **gate engine** executes the gates named in the write type's declared order. The six never-skippable gates (consent, residency, retention, dual control, counterparty, precondition) run "whatever the declaration says": a declaration that omits one fails validation, "and if a validator bug let it through the kernel runs it anyway" [E04 rung 3]. Every declared gate runs even after the first failure, and the refusal carries the first failure's code, so an operator sees every problem at once [E02 decision 5]. A gate that returns no result is a failure, never a pass [E02 I3]. Whether running all gates leaks a map of the gate set to an attacker is an open question [E02 open question 4].

The **policy evaluator** is a rules-as-data interpreter over subject, action, resource and context, where the resource is assembled from the context because "the policy service has no read path to records"; it returns allow, refuse or escalate, never a score [E04 rung 4]. The product folder recommends an own evaluator with Cedar as the fallback, because rung 4 must be deterministic, clock-free and versioned [ETS §01]. An unavailable evaluator is `PLATFORM.POLICY_UNAVAILABLE`, which pages and never falls back to a default allow [E02 §06].

3.3.3 The declaration plane: registry, compiler and lexicon

The registry implements `define` and the resolution that rung 2 performs. A declaration passes through six stages that cannot be skipped — author, validate, simulate, certify, compile, resolve [IM §6.6; EPA §04]. Validation runs the eleven semantic rules (L0-S01 to L0-S11) in CI with no platform present; simulation replays write types against the reference tenant's synthetic fixtures and never touches production records; certification is itself a governed write under `dual_control: true`, climbing the nine rungs and appending its own record before the version becomes resolvable [E03 §05; E09 §04]. The version must be exactly `last + 1`; idempotency is keyed on `(declaration_id, version, compiled_digest)`, and a different digest at the same version is a rejection because versions are immutable [E02 §05].

The compiler's outputs are deterministic and content-addressed: the surface plan (components, order and field bindings), the write-type registry entries rung 2 resolves against, the meter basis for each write type, and a verifier-profile stub [E02 §05; EPA §04]. "A compile producing two digests from two runs fails the build as a P0" [IM §6.6]. Every version is retained verbatim for ever; the registry has no migration API and no garbage collection [EPA §14 R4; IM §6.6].

The **lexicon** holds term sets by hash. Every declaration references a lexicon by hash, and the hash is sealed into every record as F15, "so that 'credit limit' means in 2033 what it meant on the day" [IM §6.7].

3.3.4 Escalation: the obligation and approval service

At rung 6 an escalation creates an obligation row with the `write_id`, the autonomy level that triggered it, a declared expiry (four hours by default, "asserted, not derived") and the set of actors who may resolve it; the ladder stops and rung 7 is never reached [E04 rung 6; E04 open question 3]. An approval "is itself a governed write. It is not a flag": it records the approver, a `basis` naming the surface-plan version and the digest of the rendered payload the approver was shown, and the platform-clock time [E02 §03]. The approver must differ from the requester where G4 applies, and must be human at L1 [E02 §03, decision 7]. A decline refuses the original write with `POLICY.APPROVAL_DECLINED`; expiry is `POLICY.OBLIGATION_EXPIRED`, "a governed event with its own proof entry, not a row that quietly goes stale" [E04 §04 Flow 3].

The kernel keeps no suspended state: the original intent is resubmitted with `obligation_ref` and climbs the whole ladder again [E04 decision 4]. The consumer's workflow, by contrast, does wait — and on stack1 an approval "arrives back in the workflow as a durable Temporal signal, not a callback that can be lost" [S03 §04 step 8, decision 6]. The service therefore emits `obligation.resolved` both as a webhook and as a `fusion.run/v1` signal to the waiting run **(build recommendation)**.

3.3.5 Act: the single writer and the credential broker

Rung 7 checks nothing by way of policy — "by this point everything has been decided" [E04 rung 7]. It performs six steps in order: snapshot; apply to the per-tenant sandbox restored from the snapshot; re-run the gates against the result; apply the identical bytes to live through a single-use credential minted at that moment, scoped to one write, with a ten-minute life; re-run the gates on live; then `BEGIN` · record version · enqueue exactly one outbox row · `COMMIT` [E04 rung 7]. "Record-or-nothing is that shared transaction and nothing else" [E04 rung 7].

The single writer is split into three parts with three owners **(build recommendation)**:

Part	Owner	What it does	Why it is split this way
Rung-7 executor	elan1 (kernel code)	Sequences the six steps, holds the per-tenant writer lock, writes the record version and the outbox row	The sequence is kernel behaviour and may not vary by product [O03 §10]

Part	Owner	What it does	Why it is split this way
Target adapter	The consumer (rung1 for ERPNext), declared in the declaration	Implements `snapshot`, `restore`, `apply(target, changeset)`, `recheck`, and returns the inverse operation; for ERPNext, writes through Frappe's own API, never the ORM	rung1 "may change … its ERP adapters" [O03 §10]; the adapter adapts and never decides [IM §6.6]
Credential broker	elan1 logic over stack1 secrets	Holds the target credential in custody; mints a one-write, ten-minute credential only against an allow bound to the `write_id`; never returns a long-lived secret	Credential custody is how Enforced mode works [E05 §02]; stack1 secrets hold values, and a consumer "never receives a secret value" [S03 §01]

Failure is typed. A live re-run that disagrees with the sandbox restores the snapshot and refuses with `GATE.SANDBOX_DIVERGED`; an unreachable target after the sandbox apply is `PLATFORM.TARGET_UNREACHABLE`, nothing reaches live and the retry is safe; a failed mint is `PLATFORM.TOKEN_MINT_FAILED`; a lost writer lock is `PLATFORM.WRITER_LOCK_LOST`, the snapshot is restored and the on-call engineer is paged [E02 §05; E04 rung 7]. Whether `SANDBOX_DIVERGED` is gate-class or platform-class is open [E02 open question 3].

3.3.6 Proof and meter: the chain writer, the proof store and the meter

The chain writer is the only principal with `INSERT` on the proof store, and no principal has `UPDATE` or `DELETE` [E04 rung 8]. It consumes the outbox in commit order, assigns the tenant's next `seq` (from 1), reads `prev_digest` from `seq − 1` (32 zero octets at genesis), assembles the sixteen fields and appends [E02 §05 seal]. The preimage is `LP(DOMAIN) || LP(enc(F01)) || … || LP(enc(F16))` with `LP(x) = uint32_BE(len) || x`; the digest is SHA-256 of it; there is no JSON in the digest path and no optional field [E02 §03]. At L1 the record carries `assurance_level: 1` and has no signature member at all — absent, not stubbed [E02 §03; IM §4.7]. Two chain writers for one tenant is `PLATFORM.CHAIN_HEAD_CONFLICT`, a P0 that stops the tenant; a gap in `seq` "is a P0, not a warning" [E02 §05]. Per-tenant ordering is enforced by a per-tenant lock in the chain writer; the prototype already serialises proof appends with `pg_advisory_xact_lock(hashtext(tenant_id))` [HFS Part 1], which is the obvious mechanism to keep **(build recommendation)**.

The meter writes exactly one event per governed write — `quantity` 1 on commit, 0 on every refusal, enforced by `CHECK (outcome <> 'refused' OR quantity = 0)` — with the basis mapped normatively from the declared blast radius, and reconciles nightly in both directions [E02 §03, C10; EPA §13 rule 4; E05 §06]. Chapter 19 carries the pipeline.

3.3.7 Surfaces: the shared renderers and the Platform console

Every module presents exactly four doors — Work · Records · Agents · Assurance — "and no fifth" [O01 §02]. elan1 owns the shared renderers that compile those doors from a declaration's surface plan, and its own Platform console, in which the developer journeys of E09 happen: Studio (author, validate, simulate, certify), the Decide screen for approvals, the Assurance door (proof, meter, mode boundaries) and the Meter view that traces every invoice line to a record [E09 §03–§07; O10 §07]. Inside a consumer's product the elan1 name appears in two places only: the proof record and the footer [O10 §07]. Chapter 8 specifies the approval screens.

3.3.8 Assurance jobs, the reference tenant and the conformance suite

Four scheduled jobs make the invariants observable: the nightly chain integrity verifier (a broken link is a P0) [E01 §02 item 7]; the write-to-record join in both directions (a committed write without a record is a P0) [E01 §06 item 3]; the meter reconciliation [E02 C10]; and the determinism replay of every reference-tenant refusal [E02 C4]. Each drives its own component on the public status page, and "a missed nightly integrity job flips the component without anyone pressing anything" [E09 §02 item 4].

The **reference tenant** holds synthetic fixtures and an adversarial declaration; its owner (kernel team or a separate assurance owner) is an open question [E02 open question 5]. The **conformance suite** runs C1–C14 against it from a consumer's own CI and writes a machine-readable result file, which is a required attachment to a production-key request [E09 §05, decision 3]. A clean clone of the suite must pass on elan1's reference implementation and fail on a deliberately broken one [E09 §02 item 8].

3.3.9 Level 2 components: specified, not built

The anchor service (Merkle batches per tenant per interval under RFC 6962, odd node promoted; RFC 3161 tokens from two authorities), the bundle service (eight members, manifest written last), the key register (key identifiers and windows, never key material), the salt store (per-record salts outside the preimage, whose destruction unbinds a payload while every digest still verifies) and the offline verifier (zero dependencies, empty lockfile, reproducible build, six exit codes) are specified in E12–E17 and are not built in Phase 1 [E01 §03; E14 §02; IM §7.4–7.7]. Two of them constrain the L1 build now. The salting scheme for F14 must be chosen at L1 because "a scheme chosen now must survive the L2 retrofit" [E02 open question 2]. And the salt store sits at P0 with its own grants, because erasure destroys salts and never rows [EPA §16 C22].

3.4 Internal APIs

The external contract is KernelPort (Chapter 4). Behind it sit five internal interfaces, each a seam between two components that a different engineer can own. The sketches are **(build recommendation)** in Python, the kernel's language [ETS §04]; field names follow E02.

The port is the only module a consumer may import. It exposes the six calls and the five objects, and nothing from the kernel's internals [E02 §01].

```
# elan1_port/__init__.py — the only import a consumer is allowed (BR1)
from typing import Protocol
from elan1_port.objects import (WriteIntent, Decision, Approval, Record,
                                MeterEvent, DeclarationHandle, ChainResult,
                                ScopeResult, RefusalCode)

class KernelPort(Protocol):
    def define(self, declaration: dict, version: int, certifier_id: str,
               supersedes: int | None = None) -> DeclarationHandle: ...
    def assert_(self, intent: WriteIntent) -> Decision: ...  # 0b..9, sync
    def seal(self, write_id: str) -> Record: ...             # kernel-only
    def verify(self, seq_from: int, seq_to: int) -> ChainResult: ...
    def scope(self, actor_id: str, write_type: str) -> ScopeResult: ...
    def refuse(self, intent: WriteIntent, code: RefusalCode) -> Decision: ...
```

`seal` appears on the protocol so that the call shape is fixed now (DR-013), but the consumer-facing implementation raises for any caller other than the kernel [E02 decision 3]. `assert_` carries a trailing underscore only because `assert` is a Python keyword; the wire name is `assert`.

The stage contract is the seam inside the kernel. It makes each rung testable alone and makes the rung-skip test (C3) a property of the pipeline rather than of each stage [EPA §07].

```
class StageResult(Enum): CONTINUE = 1; REFUSE = 2; ESCALATE = 3

@dataclass(frozen=True)
class Outcome:
    result: StageResult
    code: RefusalCode | None = None          # REFUSE only; register code
    obligation: Obligation | None = None     # ESCALATE only

class Stage(Protocol):
    rung: str                                 # "0b", "1", ... "9"
    def __call__(self, ctx: WriteContext) -> Outcome: ...
    # WriteContext is append-only: ctx.add(field, value) raises on overwrite.
    # No stage reads a clock, draws randomness or performs an undeclared read.
```

The outbox and chain-writer contract is the transaction boundary. Rung 7 commits the record version and exactly one outbox row together; the chain writer consumes rows in commit order [E02 §05; E04 rung 7]. The product folder names the relay pattern [EPA §07].

```
-- Rung 7, one transaction (the "record-or-nothing" boundary)
BEGIN;
  INSERT INTO record_version (tenant_id, record_id, version, payload_hash) VALUES (...);
```

```
  INSERT INTO outbox (tenant_id, write_id, committed_at) VALUES (...);
COMMIT;

-- Chain writer, per tenant, in commit order
SELECT pg_advisory_xact_lock(hashtext(:tenant_id));
SELECT * FROM outbox WHERE tenant_id = :tenant_id AND sealed = false
  ORDER BY outbox_id FOR UPDATE SKIP LOCKED LIMIT 1;
-- assemble F01..F16 from the write context; seq = head.seq + 1;
-- digest = sha256(LP(domain) || LP(F01) || ... || LP(F16)); INSERT INTO seal ...
```

A chain append that fails leaves the outbox row in place and the writer retries it in order — "no lost write, no gap in the sequence" [E02 §05 assert failure paths]. The caller of `assert` waits for rung 8 synchronously [E04 rung 8]; if the append has not completed within the SLO, the caller receives the committed outcome with the proof marked pending, the condition O03 calls `acted_unproven`, and the nightly join treats any committed write still without a record as a P0 [O03 §03; E01 §06 item 3] **(build recommendation for the timeout behaviour)**.

The registry and policy interfaces are read-only at runtime and version-pinned.

```
class Registry(Protocol):
    def resolve(self, declaration_id: str, version: int) -> CompiledDeclaration: ...
    def write_type(self, compiled: CompiledDeclaration, name: str) -> WriteTypeDescriptor: ...
    # never: resolve("latest"); no list-and-pick; no mutation at runtime

class PolicyEvaluator(Protocol):
    def evaluate(self, request: Projection, declaration_version: int,
                 policy_version: int) -> PolicyDecision: ...  # allow|refuse|escalate
```

The credential broker and adapter interfaces are the only path into P2.

```
class CredentialBroker(Protocol):
    def mint(self, tenant_id: str, write_id: str, decision_digest: bytes,
             target: str) -> ScopedCredential: ...   # single use, ttl = 600 s
    # refuses to mint unless (write_id, decision_digest) is an allow on record

class TargetAdapter(Protocol):          # supplied by the consumer, declared
    def snapshot(self, target: str) -> SnapshotRef: ...
    def restore(self, snap: SnapshotRef) -> None: ...
    def apply(self, target: str, changeset: bytes,
              cred: ScopedCredential | None) -> ApplyResult: ...  # returns inverse
    def recheck(self, target: str, gates: list[str]) -> list[GateResult]: ...
```

Events leave the kernel as signed webhooks carrying `seq` and `digest` [E05 §09] and, for consumers on stack1, as `fusion.run/v1` signals to a workflow waiting on an approval [S03 decision 6].

3.5 Repositories and packages

3.5.1 What exists and what is proposed

Three descriptions of the code layout exist and disagree. O03 and O10 record two elan1 repositories, `elan1-core` (kernel, Right Path, gates, proof, meter, KernelPort) and `elan1-declarations` (schema, lexicon, policy versions), with "one writer per file" and a two-owner rule for any pull request that crosses them [O03 §12; O10 §06]. The Build Book designs one monorepo with eleven shared packages, twenty apps, ten packs, `/spec` and `/vectors` [EPA §14]. The prototype is a single `elan1-platform` tree [HFS; GIP].

This memorandum keeps O03's two recorded repositories as the ownership boundary and adds the repositories that the specifications themselves require to be separate **(build recommendation)**:

Repository	Holds	Why separate	Writers	Readers	Created
`elan1-core`	Packages `elan1_port` (the six calls and five objects), `elan1_kernel` (stages, gate engine, policy evaluator, registry service, obligations, single writer, chain writer, meter), the write API, the MCP server, the Mode B clients	O03 §12	Kernel team; evidence team for the chain writer	Consumers import `elan1_port` only	Oct 2026
`elan1-declarations`	JSON Schema (2020-12, `additionalProperties: false`), the eleven rules, the compiler, the lexicon, the reference tenant fixtures and adversarial declaration	O03 §12; declarations are read by consumers and must not pull kernel code with them	Kernel team (schema); pack team (lexicon)	Consumers, packs, CI	Oct 2026
`elan1-packs`	ledger1, invoice1, collect1: declarations, profiles, export mappings, approver models, eval sets — configuration only, no `src/`	"The diff of a new app package: one declaration file, a fixtures directory and a README. If it has a `src/` directory, something has gone wrong upstream" [EPA §14]	Pack team	Registry	Before ledger1 build, Q2 2027
`elan1-conformance`	C1–C14, fixtures, expected outcomes, result-file format	Published for consumers to clone and run in their own CI [E09 §02 item 8]	Platform & assurance	Every consumer	Before L1 GA, Q1 2027
`elan1-console`	Shared P5 renderers and the Platform console	One team owns P5 [O01 §02]	Platform & assurance	Consumers embed renderers	Q1 2027
`elan1-spec`	E-series normative texts, the 64 vectors, the disagreement log	"A specification in a wiki is one nobody diffs" [EPA §14]; the text is fixed before any implementation [IM §7.3]	Evidence team	All	Oct 2026
`elan1-verify`, `elan1-seal-b`	The offline verifier (Go, standard library only) and the second seal implementation	The verifier "shares no code with the chain writer" and ships with an empty lockfile; the second implementation is written from the text alone by people who did not talk to the first team [IM §7.3, §7.5]	Separate people from `elan1-core`	Public (verifier)	Phase 3 only

No repository is named after a pack or a retired brand, and where a repository's history carries a retired name its README's first line states the current name [O10 §06].

3.5.2 Language and package choices

The kernel, the write API and the workers are Python 3.12 on FastAPI with `uv`, reusing the prototype's toolchain, because the kernel budget "is dominated by the seal and I/O, not interpreter speed"; Go is the named alternative if Gate 1 measurements miss the envelope [ETS §01, §04]. The console is React 18, Vite, TypeScript and Tailwind. At L2 the offline verifier is Go on the standard library alone, and the second seal implementation is TypeScript on `node:crypto`, so that the two implementations and the verifier share no language or library [ETS §04]. Chapter 20 carries the alternatives.

3.5.3 Import-graph rules

The import rules are contracts checked on every pull request, using `import-linter` for Python and `dependency-cruiser` for TypeScript, the tools the product folder names [ETA; ETS §01].

```
# elan1-core/.importlinter  (build recommendation)
[importlinter]
root_packages = elan1_port
                elan1_kernel

[importlinter:contract:port-is-leaf]
name = The port imports nothing from the kernel
type = forbidden
source_modules = elan1_port
```

```
forbidden_modules = elan1_kernel

[importlinter:contract:no-model-in-decision]
name = No model client reachable from rungs 3-8 (I1, C1)
type = forbidden
source_modules = elan1_kernel.stages.gates
                 elan1_kernel.stages.evaluate
                 elan1_kernel.stages.guardrail
                 elan1_kernel.stages.autonomy
                 elan1_kernel.chain
forbidden_modules = openai
                    anthropic
                    litellm
                    elan1_kernel.integrations.models

[importlinter:contract:layers]
name = Dependencies point down only
type = layers
layers = elan1_kernel.api
         elan1_kernel.stages
         elan1_kernel.chain | elan1_kernel.meter
         elan1_kernel.store
```

```
# rung1-enterprise/.importlinter — the consumer side of BR1
[importlinter:contract:no-kernel-types]
name = Nothing above P3 imports a kernel type
type = forbidden
source_modules = rung1_core
forbidden_modules = elan1_kernel
```

Four further checks sit beside the import graph: the empty-lockfile check on `elan1-verify` (L2), the "no screen files" glob that is Gate 2's instrument, stack1's descriptor lint (B7), and the claim-boundary check C14 [EPA §17; S03 §03; E02 C14]. Chapter 21 specifies how each is proved able to fail before it is trusted.

3.6 Deployment units and runtime dependencies on stack1

3.6.1 Deployment units

elan1 runs on stack1 as a consumer: stack1 hosts the processes, and elan1 is "never imported into it, never linked into it, and never deployed as part of it" [S03 §02]. The kernel's own units are therefore ordinary tenant-plane services on stack1's runtime, each in its own zone. The zone assignment follows EPA; the unit boundaries are **(build recommendation)**.

Unit	Zone	Scope	Stateful?	Scales by	Notes
`e1-write-api` + MCP server	Z1 → Z3	Per residency domain	No	Request rate	TLS and WAF at the edge; tenant token checked here
`e1-kernel` worker (stages S01–S10, single writer)	Z3	Per residency domain	Idempotency index; writer lock	Tenants, writes	In Mode A the same code runs inside the consumer's process, and the single writer runs where the credential broker can reach the target
`e1-seal` chain writer	Z4	Per residency domain	Chain head per tenant	Tenants (one ordered consumer per tenant)	Only principal with INSERT on `seal`
`e1-meter`	Z3	Per residency domain	`meter_event`	Writes	Reconciliation job beside it
`e1-registry`, `e1-policy`, `e1-lexicon`, Studio	Z2	Global; holds no tenant records	Declarations, rules, lexicons	Authors	Read by Z3 at explicit versions
Obligation service, webhook dispatcher	Z3	Per residency domain	Obligations; delivery log	Escalations	Emits `fusion.run/v1` signals
Integrity, reconciliation and replay jobs	Z3–Z4	Per residency domain	Results	Nightly	Drive the status page

Unit	Zone	Scope	Stateful?	Scales by	Notes
Console	Z1–Z2	Global	None	Users	Reads evidence and meter through `ro` APIs

One Z3, one Z4, one key-custody boundary and one salt store exist per residency domain; adding a residency domain is a deployment, not a migration [EPA §03]. Phase 1 has one domain, `ap-south-1` Mumbai, across three availability zones, and no cross-region failover — an accepted and stated Phase 1 risk [IM §16.2.4; S03 decision 7].

The product folder notes that the egress table it inherited has no rows for three flows the design relies on — Z3 reading the registry and policy service in Z2, Z4 reading the registry and lexicon at export, and channels in Z1 reaching the write API in Z3 — and suggests explicit read-only allow rows per service [EPA §20 note 4]. This memorandum adopts that settlement **(build recommendation)**: the absences that carry security weight are Z2 → Z3 and Z4 → Z3, and every permitted route is an explicit row in the network definition.

3.6.2 What elan1 needs from stack1

stack1 surface	elan1's use	Level	Source
`fusion.run/v1` (Temporal namespace per tenant)	Durable execution of the consumer's governed-write workflow; approvals delivered as durable signals; the L2 Merkle anchor job, which must run on a blocking-green `fusion.run/v1` because "replay must not re-run the side effect"	L1; anchor job L2	[S03 decision 6; O09 §03 item 4; O01 DR-002]
`fusion.trace/v1`	OpenTelemetry spans per rung for latency measurement; never a payload and never the proof, because trace retention is 90 days by default	L1	[S02 §1, open question 4; EPA §18]
`fusion.guard/v1`	Provenance and injection verdicts arrive as declared inputs to rung-3 gates (for example a precondition that third-party-ingested content cannot trigger an act without a human); a verdict is never the allow	L1	[S03 decision 4]
`fusion.tools/v1`	Registration of the elan1 MCP server, pinned by hash, so that stack1 tooling reaches it as a Mode B consumer	L1	[S02 §1; O03 §04]
`fusion.gateway/v1`	**None.** The kernel calls no model	—	[E02 I1]
Secrets (OpenBao, per-tenant paths, short-lived tokens)	Custody of target-system credentials and webhook secrets behind the credential broker; never L2 signing keys	L1	[S03 §01; E16 §03]
Postgres instances with separate credential sets	The record-plane and evidence-plane databases; elan1 owns the schema and the grants, stack1 operates the instance **(build recommendation)**	L1	[S03 §06, open question 2]
Sandbox pool	Consumer integration and stage rehearsal; it is not the L1 shadow sandbox, which the target adapter provides	L1	[S03 §01; E05 §05]
Metering plumbing	stack1's six cost units for its own statement; not the Governed Action meter	L1	[S02 §9; O11 §06]
Environments `dev`, `ci`, `stage`, `prod`	Four only; `prod` availability 99.5% monthly per contract; change windows Tue/Thu 21:00–23:00 IST	L1	[S02 §2–§5]

The reference tenant and self-serve sandbox tenants are tenants, not a fifth environment: they run in `prod` with their own chains marked `mode: sandbox`, metered at quantity zero and never billed, because third parties must reach them to run the conformance suite [E05 §05; E09 §03] **(build recommendation)**.

Chapter 22 sets out the failure domains and what survives each; the rule that matters here is S03's: a runtime outage stops writes, an elan1 outage stops allows, "and stopping allows is the safe direction" [S03 §06].

3.7 Reconciling the product-folder architecture with the E/O series

The product folder — the Build Book, the Strategy Blueprint, the platform architecture flows (EPA, ETA), the technology stack (ETS) and the Claude Build Dossier (ECB) — was written between 16 and 18 September 2026, before the E/O series of 21 September narrowed the perimeter and the phase plan. It is the most detailed engineering material in the corpus and most of it carries forward. Where it disagrees with the E/O series, the E/O series prevail (§1.1).

Topic	Product folder	E/O series	Treatment
Scope of elan1	Twenty apps, ten evidence packs, thirty compiled screens; "a governed-write platform" [EPA §01, §14]	Kernel and regulatory packs; never an app or a UI shell for another product [IM §2.3]	Apps move to consumers; shared renderers stay (§3.3.7)
Build order	Seal specification and vectors, then the offline verifier, then kernel rungs (W0–W2) [EPA §19]	L1 kernel first; L2 specification only in Phase 1 [E01; O02 decision 4]	Kernel first (Chapter 1 conflicts)
Rung 8	Signs with Ed25519 and enqueues the anchor; "no seal, no commit" [EPA §07, §16 C15]	Hash chain only at L1; signing at L2; commit then ordered chain append [E02 §05; E04 decision 1]	E02; signing retrofitted at L2
Autonomy and block 7	L0–L5; `doors` [EPA §04, §07]	A0–A5; `channels` [O01 DR-011, DR-012]	E/O names
Deployment	elan1 operates its own AWS VPCs per zone per domain, Terraform, ECS Fargate, KMS; hosting decision OD-28 open [ETS §01]	stack1 hosts processes; elan1 is a consumer of `fusion.*/v1` [S03 §02]	stack1 operates; elan1 states the required absences (Z2 → Z3, Z4 → Z3) and the region allow-list as requirements on stack1's deploy specification

Four things in the product folder have no equivalent in the E/O series and are adopted as build recommendations because they close gaps the E-series leaves open: the twelve-stage decomposition of the nine rungs (§3.3.2); the append-only-seven table set with no `UPDATE` or `DELETE` grant to any principal including the writer (Chapter 9); the explicit egress rows for Z3 → Z2 and Z4 → Z2 reads (§3.6.1); and the "three numbers on the wall" that must read zero — writes committed without a record, records mutated outside the kernel writer, and (from L2) dependencies in the verifier [EPA §01, §13, §18].

3.8 The existing prototype, and what carries forward

The working tree was derived by execution on 3, 9 and 12 September 2026 [GIP; HFS; HV2]. Its authors warn that its own status table "went stale within four hours" and that every number must be re-derived before it is quoted [HFS "How to read"]. With that caveat, the readings that matter for the build are these:

Finding	Reading at 9–12 Sep 2026	Carries forward?
A governed-write chokepoint	`sor_write.governed()` refuses or seals every governed write at one point; the seal is recomputable from the payload	**Yes** — as the internal entry point behind `assert`
Per-tenant serialisation	Proof appends take `pg_advisory_xact_lock(hashtext(tenant_id))` before reading the head	**Yes** — the chain writer's ordering mechanism
The seal	Domain `elan1.proof.seal.v2`, a closed 13-field tuple with no `seq`; a test asserts that a chain with its two newest proofs removed still verifies	**No** — replaced by the sixteen-field v6 preimage with F02 `seq`; the passing test is inverted
Where acts land	All four act call sites insert into elan1's own Postgres; no external system of record is written	**No** — rung 7 through a declared adapter to a real target is the December 2026 gate
Act and proof in one database transaction	The act receives the same writable session as the proof insert	**No** — outbox between rung 7 and rung 8; a remote write cannot be rolled back by a Postgres `ROLLBACK`
Declarations	Four dialects of the seven blocks in the tree (camelCase TypeScript, snake_case fixture, pydantic model, JSON Schema), with two blocks typed differently; the table's unique key excludes `version`	**Partly** — the pydantic snake_case model is the recommended seed; the schema is generated from one source; `version` enters the key
Reconciliation	`meter.reconcile()` compares `count(*)` to `count(*)` with no join, then marks billing rows reconciled	**No** — a two-way join on `write_id` and `seq`, and billing rows are append-only
An in-browser verifier page	A dependency-free SHA-256 implementation in the console, "the most valuable artefact in the tree"	**Yes, as generated output of the specification** — not as a third opinion on the encoding

Sources: [HFS Part 1, Part 2 items 1–7; HV2 §11]. GIP's reading of 3 September measured a wider tree — 390 registered routes rendered from a single 23,375-line screen file — and recommended "a clean-sheet shell over the existing core" rather than a rebuild [GIP §0]. The narrowed perimeter goes further: the product shell belongs to consumers, and elan1 keeps the core. The distance from this prototype to the E02 kernel is the content of the Phase 1 plan in Chapter 21, and its first measurable step is the December 2026 gate: one governed write in a design partner's own ERPNext site, through the kernel, with the bypass attempt shown failing [IM §16.2.3].

4. The kernel contract — KernelPort, six calls and five objects

KernelPort is the contract the rest of this memorandum hangs from. It is "the single seam between anything that wants to change a system of record and the thing that decides whether it may" [E02 §01]. It has six calls — `define` · `assert` · `seal` · `verify` · `scope` · `refuse` — five objects, eight invariants, one writer and a closed refusal register [E02, IM §4.2]. Every consumer builds against it: rung1 Enterprise as the first production consumer, stack1 tooling over MCP, third-party vendors from Phase 2 and the regulatory packs thereafter [IM §6.1, O03 §01]. This chapter specifies the contract from the kernel's side: what each call does, what it must never do, what it returns, how it fails, and how a consumer proves it is using it correctly.

Chapter 5 walks the nine rungs that `assert` runs. Chapter 6 covers the surfaces through which the calls are reached (the in-process SDK, the MCP server and the HTTP API) and the two enforcement modes. Chapter 9 covers the storage of the objects defined here, and Chapter 10 the L2 seal over the sixteen-field record.

> **Status.** KernelPort v1 is a draft contract dated 21 September 2026 and owned by the elan1 lead [E02]. As at 23 September 2026 it is not running for any customer; no governed action has been committed or refused in a production tenant [IM §4.9]. What exists is the specification and an implementation of the L1 kernel against a single system of record [IM §4.9]. Every latency figure below is a design target, not a measurement [E04 §02]. Where this chapter adds wire detail or type sketches that the sources do not give, they are marked **(build recommendation)**.

4.1 What the contract is, and the port rule

4.1.1 One seam, identical for every consumer

KernelPort has "identical signatures for every consumer, in every product, at every Assurance Level" [E02 §01, decision 1]. That sentence carries three commitments.

- **Every consumer.** rung1, stack1 and an outside developer call the same six calls with the same argument shapes. No consumer gets a private call, a convenience variant or a batch endpoint [E02 I5, IM §4.2].
- **Every product.** A product varies only in its Declarations, its adapters and, from L3, its verifier profiles. It never varies the kernel. That is DR-001: "One kernel. No product builds its own gates, approval, proof or meter" [O01 §05 DR-001].
- **Every Assurance Level.** The call shapes are frozen for Phase 1. At L2 the contents of `seal` change (it also signs) but its signature does not [O03 decision 5, O01 DR-013, IM §6.3]. Behaviour that exists only to keep the L2 retrofit cheap is marked `[L2]` in the contract, with its L1 behaviour stated [E02 §01].

The commercial reason for freezing the interface is stated in the IM: every consumer builds against the same six calls, so the kernel is built once and amortised across all of them [IM §4.2]. The engineering reason is sharper. A governance layer with two write paths has two definitions of "allowed", and "neither is defensible to a regulator" [O01 §05 DR-001]. A frozen port makes a second write path visible as a build failure rather than a design review finding.

4.1.2 The port rule

> **The port rule.** Nothing above plane P3 imports a kernel type. Consumers hold identifiers, enumerations and the five objects defined here — never a kernel internal. Enforced by an import-graph check in the CI of every consuming repository, not just elan1's [E02 §01, O01 §02].

E02 states the rule as "nothing above L0 imports a kernel type". O01 corrects that wording: in the v2.0 vocabulary L0 meant three different things, and the kernel is the fourth stratum, so the rule is restated as "nothing above P3 imports a kernel type" [O01 §02–03]. This memorandum uses the P3 form throughout.

What a consumer may hold, and what it may not, is therefore exact:

A consumer may hold	A consumer may never hold
Identifiers: `tenant_id`, `write_id`, `intent_id`, `declaration_id`, `obligation_id`, `seq`	A gate implementation, a gate-result row type, a rule-engine type
Enumerations: disposition, actor kind, blast radius, autonomy level, refusal code	The chain writer, the outbox, the snapshot store
The five objects: write intent, decision, approval, record, meter event	Any type from `elan1-core` other than the published port package
Its own Declarations, adapters and screens	A target-system credential (Mode A)

Sources: E02 §01, §03; O03 §10, §12. The check is enforced as conformance test C1, which also asserts that no module reachable from rungs 3–8 imports a model client [E02 §08]. The same CI rule appears in the group's boundary list alongside two others — exactly one write credential exists per environment, and the control plane has no network route to the record store [IM §6.2, O03 §10].

(Build recommendation) The import-graph check is implemented per language. For Python consumers (rung1 on Frappe), an `import-linter` contract forbids any module outside the published `elan1_kernelport` package from importing `elan1_core.*`, and a second contract forbids any module in the rung packages `elan1_core.rungs.r3`–`r8` from importing a model client (`openai`, `anthropic`, the stack1 `fusion.gateway/v1` client, or any package on a maintained deny-list). For TypeScript consumers, `dependency-cruiser` enforces the same two rules against `@elan1/kernel` internals. Both rules run on every pull request and fail the build; they are published as part of the conformance suite so that a consumer runs the kernel team's rule, not its own paraphrase.

```
# .importlinter  (build recommendation — shape only)
[importlinter:contract:port-rule]
name = Nothing above P3 imports a kernel type
type = forbidden
source_modules = rung1_core, rung1_adapters, rung1_agent
forbidden_modules = elan1_core
allow_indirect_imports = False

[importlinter:contract:no-model-near-a-decision]
name = Rungs 3-8 never import a model client (E02 I1, C1)
type = forbidden
source_modules = elan1_core.rungs.r3, elan1_core.rungs.r4, elan1_core.rungs.r5,
                 elan1_core.rungs.r6, elan1_core.rungs.r7, elan1_core.rungs.r8
forbidden_modules = openai, anthropic, litellm, fusion_gateway, elan1_core.model
```

4.1.3 The six calls at a glance

Figure 5 shows the six calls, the five objects they exchange, the eight invariants and the refusal register.

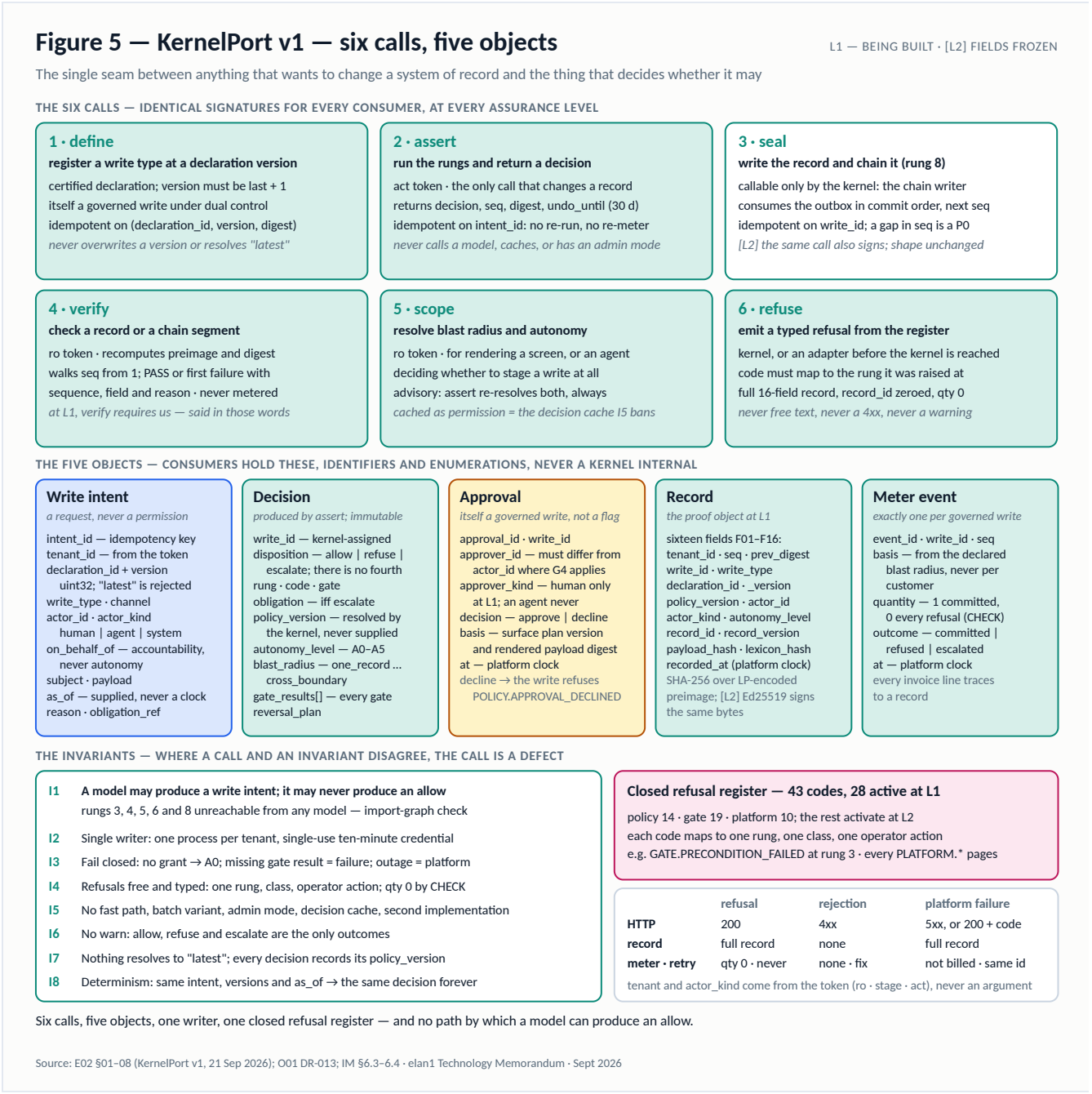


Figure 5 — KernelPort v1: six calls, five objects, eight invariants and the closed refusal register, with the token scope each call requires and the one call (assert) that can change a record.

Call	What it does	Caller	Token scope	Changes a record?	Idempotency key	L1 → L2
`define`	Registers a declaration version and its write types	Declaration author via Studio or repository, with a named certifier	control	No (registers a declaration; itself a governed write)	`(declaration_id, version, compiled_digest)`	Adds a verifier-profile stub [L2]
`assert`	Runs the rungs and returns a decision	Consumer write path, for a human, agent or job	tenant, `act`	Yes — the only call that can	`intent_id`	Unchanged shape
`seal`	Assigns `seq`, chains, appends the record	Kernel only	—	No	`write_id`	Also signs and enqueues the anchor batch [L2]

Call	What it does	Caller	Token scope	Changes a record?	Idempotency key	L1 → L2
`verify`	Recomputes digests and walks the chain	Consumer, operator, nightly job	tenant, `ro`	No	Naturally idempotent	Adds signature, Merkle and RFC 3161 checks; answerable offline [L2]
`scope`	Resolves blast radius and autonomy for rendering	Consumer building a surface	tenant, `ro`	No	Naturally idempotent	Unchanged
`refuse`	Emits a typed refusal with a full record	Kernel; a consumer adapter that has already decided	tenant, `act`	No	`write_id` (build recommendation)	Unchanged

Sources: E02 §04–05, §09; E05 §03; IM §4.2. "`assert` is the only call that can change a record. An SDK consumer that never calls `assert` is a reader" [E05 §03]. The IM names the `define` caller as holding a control-plane token [IM §4.2]; E02 §04 lists only the tenant scopes `ro`, `stage` and `act`. The two are consistent once `define` is read as a control-plane call on `/v1/declarations`, which E05 classes `ctl` [E05 §04].

4.2 The eight invariants

E02 states eight invariants and sets their priority over everything else in the contract: "Every call below is an instance of these. Where a call and an invariant disagree, the invariant wins and the call is a defect" [E02 §02]. The IM counts the same eight [IM §6.3].

#	Invariant	Why it exists	How it is enforced and tested
I1	A model may produce a write intent. It may never produce an allow. Rungs 3, 4, 5, 6 and 8 are unreachable from any model	The company's lead claim; a decision a model can influence is not a control	Import-graph check, not review (C1); no model client reachable from rungs 3–8
I2	Single writer. Exactly one process mutates the target for a tenant, holding a single-use credential with a ten-minute life scoped to one write	A second path to the target makes every refusal advisory	Bypass attempt (C2); CI rule "one write credential per environment"; `PLATFORM.CHAIN_HEAD_CONFLICT` and `PLATFORM.WRITER_LOCK_LOST` as P0/P1 signals
I3	Fail closed. Every stage defaults to refuse. A missing grant is A0; a missing gate result is a failure; an unavailable dependency is a platform-class refusal	An unanswered question that resolves permissively is the commonest governance failure	Fail-closed matrix (C5); missing-grant fuzzing (C6)
I4	Refusals are free and typed. Every refusal carries a register code mapping to one rung, one class and one operator action; it meters at zero, enforced by a database CHECK, and produces a full record	Removes the vendor's incentive to refuse; makes refusals evidence	Refusal shape (C8); meter reconciliation (C10); `CHECK` constraint on `meter_event`
I5	No fast path, no batch variant, no admin mode, no decision cache, no second implementation	"A cached allow is an allow at an unknown policy version. There is no principal who can skip rung 3"	Rung completeness (C3); break-glass declared as a dual-control write type
I6	No warn. The only outcomes are allow, refuse and escalate	"Succeeded with warnings" hides a decision from the record	The disposition enum has three values; adding a fourth is a breaking change [O06 §03]
I7	Nothing resolves to "latest". Every declaration reference is an identifier plus an explicit integer; every decision records its policy version	Replay and audit require the exact rules in force	Version pinning (C13)

#	Invariant	Why it exists	How it is enforced and tested
I8	Determinism. Same intent, declaration version, policy version and `as_of` give the same decision forever; no stage reads a clock, takes a random value or performs an undeclared external read	Replay is the audit; a decision that cannot be replayed cannot be defended	Nightly determinism replay (C4)

Sources: E02 §02, §08; IM §4.3, §6.3.

Four points about the invariants shape the build more than the table suggests.

I1 is enforced structurally. The claim is not that the kernel "does not use" a model at rung 4; it is that no model client is importable from the rung packages, and the check fails the build if one appears [E02 I1, C1]. This matters because an earlier group document marks rung 4 as "the only rung at which a model may be called" [O01 §02]. E02 I1, E04 rung 4 ("There is no model in this path and no configuration that puts one there") and the IM ("No model, at any level, ever") override it [E04 §03, IM §4.3]; the difference is recorded in the conflicts block. A model may sit upstream of `assert` and compose the intent. Chapter 5 §5.6 shows how the policy engine is kept model-free.

I2 is an invariant, not a deployment choice. "A consumer that can reach the target by another path is in Attested advisory mode and must be labelled as such" [E02 I2]. The contract does not pretend that the kernel can stop a caller who holds its own credential. It defines the label that applies when it cannot. Chapter 6 §6.3 describes how the single writer is made physical.

I3 and I5 together forbid the usual availability shortcuts. There is no decision cache to fall back on when the rule engine is down, and there is no default allow: the only answer is `PLATFORM.POLICY_UNAVAILABLE`, which pages [E02 §06]. A consumer that caches a `scope` result and treats it as permission "has rebuilt the decision cache I5 forbids" [E02 §05.5]. The availability of the kernel is therefore the availability of the write path; Chapter 22 carries the operational consequences.

I8 makes the gates pure functions. Every gate is a function of `(context, declaration_version, record_state, as_of)`, and `as_of` is supplied in the intent, never taken from a clock [E04 §03 rung 3, E02 §03]. The platform clock appears in exactly two places: the approval's `at` and the record's `recorded_at` (F16), both of which are outputs, not inputs, to a decision [E02 §03].

(Build recommendation) Each invariant is carried as a row in a machine-readable traceability file in `elan1-core`, naming the conformance tests and unit suites that would fail if the invariant were broken. A CI job fails when an invariant has no failing-capable test attached. This follows the group's rule that an instruction in a document is advice and a failing build is a control [IM §6.1], and it answers the diligence question "which test would catch a violation of I3?" with a file rather than a recollection.

```
# elan1-core/invariants.yaml  (build recommendation)
- id: I3
  statement: "Fail closed. Every stage's default is refuse."
  source: "E02 §02"
  tests:
    conformance: [C5, C6]
    unit: [tests/rungs/test_r6_missing_grant.py, tests/rungs/test_r3_missing_result.py]
  mutation_proof: required      # each listed test must be shown to fail on a seeded defect
- id: I8
  statement: "Determinism: no clock, no random value, no undeclared read in any stage."
  source: "E02 §02"
  tests:
    conformance: [C4]
    static: [lint/no_clock_in_rungs.py]   # bans datetime.now, time.time, uuid4 in rungs 0b-6
```

4.3 The five objects

KernelPort exchanges five objects and no others: the **write intent**, the **decision**, the **approval**, the **record** (the proof object at L1) and the **meter event** [E02 §03]. Two further kernel rows — the gate result and the obligation — appear only nested inside a decision; a consumer reads them through the decision, never as independent types.

4.3.1 The write intent

Produced by a consumer, possibly composed by a model. "A request, never a permission" [E02 §03].

Field	Type	Req	Meaning
`intent_id`	uuid	yes	Client-generated; the idempotency key for the whole write
`tenant_id`	uuid	yes	From the token, cross-checked against the body; a mismatch is a rejection
`declaration_id`	uuid	yes	Which declaration governs this
`declaration_version`	uint32	yes	Explicit integer. `"latest"` is a rejection
`write_type`	string	yes	Must exist in that declaration version's `write_types`
`actor_id`	uuid	yes	Who is asking
`actor_kind`	enum	yes	`human` | `agent` | `system`. Never inferred
`on_behalf_of`	uuid	no	Delegation. Confers accountability, never autonomy
`subject`	object	yes	Target record reference, or absent for a create
`payload`	object	yes	Proposed field values, conforming to the declaration's shape
`as_of`	RFC 3339	yes	The business instant the gates evaluate against. Supplied, never taken from a clock
`channel`	string	yes	Must be listed in the declaration's `channels` block for this write type
`reason`	string	no	Free text. Recorded, never evaluated
`obligation_ref`	uuid	no	Present on the resubmission of an escalated write

Source: E02 §03.

Three fields need interpretation because the sources pull in different directions.

- **`actor_kind`.** E02 lists it as a required intent field "never inferred", but also says that at rung 1 it "is taken from the token, never inferred from the payload", and E04 says it "is set by the kernel and can never be supplied by a caller" [E02 §03, §05.2; E04 rung 1]. The coherent reading, and the one this memorandum adopts, is that the token is authoritative: the intent carries `actor_kind` so that a consumer states what it believes, and the kernel compares it with the token's value. A mismatch is a rejection and an alert, by analogy with the `tenant_id` rule [E02 §04]. **(Build recommendation.)**
- **`intent_id` versus `idempotency_key`.** E02 makes `intent_id` the idempotency key; E05 and O03 name a separate `idempotency_key` on `act.write` and `define` [E05 §08, O03 §05]; R04 defines rung1's key as `sha256(changeset_id ‖ base_schema_hash)` [R04 §04]. **(Build recommendation)** On the wire, `intent_id` is the one key. HTTP and MCP callers that supply `idempotency_key` have it mapped to `intent_id` as a name-based UUID (UUIDv5 over the tenant namespace), so the same key always yields the same `intent_id` and E02's semantics hold.
- **Identifier format.** E02 types identifiers as `uuid`; R04 uses ULIDs for rung1's own identifiers [R04 §04]. The port uses UUIDs; a consumer's ULIDs travel inside `subject` or `payload`, or are mapped to UUIDs by the consumer's adapter.

(Build recommendation) The port package publishes the intent as a frozen type in both SDK languages. The Python sketch uses a frozen dataclass so an intent cannot be mutated after construction; the TypeScript sketch uses `readonly` fields.

```
# elan1_kernelport/objects.py  (build recommendation — shapes from E02 §03)
from dataclasses import dataclass
from enum import Enum
from typing import Any, Mapping, Optional
from uuid import UUID

class ActorKind(str, Enum):
    HUMAN = "human"
    AGENT = "agent"
    SYSTEM = "system"

@dataclass(frozen=True)
class WriteIntent:
    intent_id: UUID                # idempotency key for the whole write
    tenant_id: UUID                # cross-checked against the token
```

```
    declaration_id: UUID
    declaration_version: int        # uint32; "latest" is unrepresentable by type
    write_type: str
    actor_id: UUID
    actor_kind: ActorKind           # compared with the token; token wins
    subject: Optional[Mapping[str, Any]]
    payload: Mapping[str, Any]
    as_of: str                      # RFC 3339, supplied by the caller
    channel: str
    on_behalf_of: Optional[UUID] = None
    reason: Optional[str] = None
    obligation_ref: Optional[UUID] = None
```

```
// @elan1/kernel/objects.ts  (build recommendation — shapes from E02 §03)
export type ActorKind = "human" | "agent" | "system";

export interface WriteIntent {
  readonly intent_id: string;            // uuid; idempotency key
  readonly tenant_id: string;            // uuid; cross-checked against the token
  readonly declaration_id: string;       // uuid
  readonly declaration_version: number;  // uint32; never "latest"
  readonly write_type: string;
  readonly actor_id: string;
  readonly actor_kind: ActorKind;
  readonly on_behalf_of?: string;
  readonly subject: Readonly<Record<string, unknown>> | null;
  readonly payload: Readonly<Record<string, unknown>>;
  readonly as_of: string;                // RFC 3339
  readonly channel: string;
  readonly reason?: string;
  readonly obligation_ref?: string;
}
```

4.3.2 The decision

Produced by `assert`. Immutable [E02 §03].

Field	Type	Meaning
`write_id`	uuid	Kernel-assigned; stable for the life of the intent
`disposition`	enum	`allow` | `refuse` | `escalate`. There is no fourth
`rung`	uint8	The rung that produced the disposition
`code`	string	Refusal code. Present iff `disposition = refuse`
`gate`	string	Gate name. Present iff the code is gate-class
`obligation`	object	Present iff `disposition = escalate`
`policy_version`	uint32	Resolved by the kernel, never supplied by the caller
`declaration_version`	uint32	Echoed from the intent
`autonomy_level`	enum	A0–A5 held by this actor at this instant
`blast_radius`	enum	`one_record` | `one_counterparty` | `batch` | `tenant` | `cross_boundary`
`gate_results`	array	One row per declared gate, in declared order, each with name, outcome and inputs hash
`reversal_plan`	object	From the write type. Absent only when the write type is flagged irreversible

Source: E02 §03.

The disposition is the kernel's decision; the consumer-facing **outcome** is what happened. E05 and E04 describe `assert` as returning `committed · refused · escalated` [E05 §03, E04 Flow 2], while E02's decision carries `allow · refuse · escalate` [E02 §03]. These are two views of one result: `allow` followed by a successful rung 7 is reported as `committed`; `refuse` at any rung, including a platform-class refusal at rung 7 after an `allow` at rung 6, is `refused`; `escalate` is `escalated`. The decision object records what the kernel decided; the response envelope records the outcome and the proof reference (§4.5.2).

The obligation (nested in an escalating decision) carries the `write_id`, the autonomy level that triggered it, an expiry, and the set of actors who may resolve it [E04 §03 rung 6, E02 §05.2]. The resolver set contains human actors only [E06 §03]. **(Build recommendation)** Field names: `obligation_id` (uuid), `write_id`, `triggered_at_rung` (uint8), `autonomy_level`, `expires_at` (RFC 3339, platform clock), `resolver_ids` (uuid array) and `resolver_roles` (string array, from the declaration's `permissions` block).

The gate result (nested in every decision) records which gate ran, what it decided, at which declaration version and at which `as_of` [E04 §03 rung 3]. **(Build recommendation)** Field names: `name` (one of the nine gate names), `position` (declared order), `outcome` (`pass` | `fail`), `code` (present on fail), `inputs_hash` (SHA-256 over the canonicalised gate inputs), `declaration_version`, `as_of`.

4.3.3 The approval

"An approval is itself a governed write. It is not a flag" [E02 §03].

Field	Type	Meaning
`approval_id`	uuid	Kernel-assigned
`write_id`	uuid	The write being approved
`approver_id`	uuid	Must differ from `actor_id` when gate G4 applies
`approver_kind`	enum	`human` only at L1. An agent may never approve
`decision`	enum	`approve` | `decline`
`basis`	string	What the approver saw: the surface plan version and the rendered payload digest
`at`	RFC 3339	Platform clock

Source: E02 §03. Declining is a governed outcome, not an error: the original write refuses with `POLICY.APPROVAL_DECLINED` [E02 §03]. The `basis` field is what makes an approval evidence rather than a click: it binds the approval to the exact compiled screen and the exact payload the approver was shown, so that "the screen an approver saw in 2027 can be rebuilt in 2035" [IM §4.5, E06 §01]. Conformance test C11 asserts that an approver equal to the requester is refused and that an agent approver is refused at L1 [E02 §08]; decision 7 of E02, owned by the founder, makes "an agent may never be an approver at L1" a closed rule [E02 §09].

4.3.4 The record — the proof object at L1

Sixteen fields, hash-chained, not yet signed. "Field names, order and encoding are frozen now so that L2 adds a signature over the same preimage rather than a new layout" [E02 §03].

#	Field	Type	Binds
F01	`tenant_id`	uuid	Whose chain this is
F02	`seq`	uint64	Per-tenant monotonic from 1; assigned only by the chain writer
F03	`prev_digest`	32 octets	32 zero octets at genesis
F04	`write_id`	uuid	The write this records
F05	`write_type`	string	From the declaration
F06	`declaration_id`	uuid	Which declaration governed it
F07	`declaration_version`	uint32	Which version. Never "latest"
F08	`policy_version`	uint32	Which rules evaluated it
F09	`actor_id`	uuid	Who asked
F10	`actor_kind`	enum	`human` | `agent` | `system`. Never inferred
F11	`autonomy_level`	uint8	A0–A5 held at the moment of the write
F12	`record_id`	uuid	Zero-UUID on a refusal

#	Field	Type	Binds
F13	`record_version`	uint32	0 on a refusal
F14	`payload_hash`	32 octets	Over the salted, canonicalised payload
F15	`lexicon_hash`	32 octets	Over the term set in force
F16	`recorded_at`	RFC 3339	Platform clock at chain-write time, never the caller's

Source: E02 §03.

Encoding. `preimage = LP(DOMAIN) || LP(enc(F01)) || … || LP(enc(F16))`, `digest = SHA-256(preimage)`, with `LP(x) = uint32_BE(len) || x`. There is no JSON in the digest, length prefixes rather than delimiters, and no optional fields [E02 §03]. At `[L2]` the record adds `signature = Ed25519(digest)` — pure Ed25519, not the pre-hashed or context variants — plus a Merkle root and an RFC 3161 token. At L1 all three are absent and the record states `assurance_level: 1` [E02 §03]. Absent means absent: not null, not an empty string, not thirty-two zero octets, because "a stub is a value, values get copied, and a copied stub is eventually read as a signature by something that does not check" [IM §4.7]. Chapter 10 specifies the per-field encodings (`enc`) and the domain string under Seal Spec v6; this chapter does not restate them.

Three properties of the record matter to the contract rather than to the cryptography.

- **Refusals and commits are structurally identical.** A refusal seals with `record_id` at the zero UUID and `record_version` 0, "so a refusal and a commit are structurally identical and semantically distinct" [E04 §03 rung 8]. A verifier walks one record type.
- **The record does not carry the outcome or the refusal code as a field.** The sixteen fields bind the outcome indirectly: F12/F13 at zero mark a non-commit, and the refusal code, gate results and obligation live in the write row the record references by F04. The console's Proof screen nonetheless displays the outcome, the refusal code and `mode: live | sandbox` beside the record [E06 §05], and E05 says the `mode` field and `enforcement_mode` are "sealed into" every record [E05 §02, §05]. Neither `mode` nor `enforcement_mode` appears among the sixteen frozen fields, and O03 also requires a caller-performed flag on Mode B records [O03 §04]. This is a real gap in the frozen layout and is recorded in the conflicts block. **(Build recommendation)** Until the owner closes it, the kernel binds `mode`, `enforcement_mode`, `outcome` and the Mode B `performed_by` flag into the salted payload canonicalisation that produces F14, as reserved kernel-owned members that precede the declaration's `proof_binding` fields. This commits them into the digest without adding a seventeenth field. The alternative — a new field — is a port major version and is "not permitted at v1" [E02 §07].
- **F14's salt is not yet fixed.** "What is the exact salting scheme for F14 at L1, given that the erasure story only matters at L2? A scheme chosen now must survive the L2 retrofit" is open with the elan1 lead, due 30 November 2026 [E02 OQ2]. The declaration fixes `salt_scope: per_record` in Phase 1 and `canonicalisation: elan1_lp_v1` [E03 §02 block 6].

```
# elan1_core/proof/preimage.py  (build recommendation — encoding rule from E02 §03;
# field encoders enc_* are specified in Seal Spec v6, Chapter 10)
import hashlib
import struct

def lp(x: bytes) -> bytes:
    return struct.pack(">I", len(x)) + x          # uint32 big-endian length prefix

def preimage(domain: bytes, fields: list[bytes]) -> bytes:
    if len(fields) != 16:
        raise ValueError("the record has exactly sixteen fields; none is optional")
    return lp(domain) + b"".join(lp(f) for f in fields)

def digest(domain: bytes, fields: list[bytes]) -> bytes:
    return hashlib.sha256(preimage(domain, fields)).digest()   # 32 octets
```

4.3.5 The meter event

Field	Type	Meaning
`event_id`	uuid	Kernel-assigned
`write_id`	uuid	Always present; every invoice line traces to a record
`seq`	uint64	The record this event refers to
`basis`	enum	From the write type's declared blast radius, never configurable per customer
`quantity`	uint32	1 for a committed write; 0 for every refusal, enforced by CHECK
`outcome`	enum	`committed` | `refused` | `escalated`
`at`	RFC 3339	Platform clock

Source: E02 §03. The basis mapping is normative: `one_record` meters per record, `one_counterparty` per counterparty, `batch` per batch, `tenant` per tenant, `cross_boundary` per crossing [E05 §06]. Because each committed governed action is one unit of its basis, quantity is 1 for a commit; the basis tells the invoice what kind of unit it is. Escalations also meter at zero [E05 §07]. The constraint is in the database, not the application:

```
-- kernel-owned; shape per E02 §03; constraint per E02 I4 and E05 §06 (DDL is a sketch)
CREATE TABLE meter_event (
  event_id  uuid        PRIMARY KEY,
  tenant_id uuid        NOT NULL,
  write_id  uuid        NOT NULL,
  seq       bigint      NOT NULL CHECK (seq >= 1),
  basis     text        NOT NULL CHECK (basis IN
              ('one_record','one_counterparty','batch','tenant','cross_boundary')),
  quantity  integer     NOT NULL CHECK (quantity IN (0, 1)),
  outcome   text        NOT NULL CHECK (outcome IN ('committed','refused','escalated')),
  at        timestamptz NOT NULL,
  CONSTRAINT refusal_is_free CHECK (outcome = 'committed' OR quantity = 0),
  CONSTRAINT one_event_per_write UNIQUE (tenant_id, write_id)
);
```

The `one_event_per_write` constraint is a **build recommendation** that turns "exactly one meter event per governed write" [E04 §03 rung 9] and the idempotency rule of C7 ("produces no second meter event") into a database property [E02 §08]. The `quantity IN (0, 1)` constraint follows from E02's field definition. Chapter 19 covers the pipeline from meter event to invoice line.

4.4 Tenancy, tokens and attribution

4.4.1 Tenancy comes from the token

"Tenancy comes from the token, never from a call argument. A call that cannot be attributed to a tenant is rejected, not defaulted" [E02 §04]. A token for one tenant presented with another tenant's `tenant_id` in the body "is a rejection and an alert, not a refusal" [E02 §04]. The distinction matters: a refusal is a governed outcome with a record in the tenant's chain; a cross-tenant body is a malformed or hostile request that never entered the kernel, so it has no record in any tenant's chain and goes to the security alert stream instead.

Below the kernel, multi-tenancy is enforced by row-level security on every tenant table; a migration that disables it fails CI, and a cross-tenant read test runs in CI and expects a failure. Caches, search indexes and vector stores are partitioned by tenant rather than filtered at query time [IM §6.10]. Chapter 9 covers the storage side.

4.4.2 Scopes on the tenant token

Tokens are per tenant with one of three scopes [E02 §04]:

Scope	Sufficient for	Cannot
`ro`	`verify`, `scope`, reads of records, evidence, declarations and the meter	Reach rung 7 by any path

Scope	Sufficient for	Cannot
`stage`	Composing an intent: `stage.write` returns a validated request object with no side effects	Submit it
`act`	`assert` and `refuse`; `act.write`, `act.escalate`	Assert its own autonomy level

Sources: E02 §04, E05 §04. The `stage` scope exists "so that an A1 or A2 agent can do useful work without producing a stream of refusals" [E05 §04]. Autonomy is not a token scope: the same `act` token held by an A2 agent and an A4 agent reaches the same rungs, and the difference is decided entirely at rung 6 from the `autonomy_grant` table [E05 §04]. "A token that could assert its own autonomy level would make rung 6 decorative, so it cannot" [E05 §05].

4.4.3 Token classes

The sources agree there are three token classes "and no fourth", but disagree on the third [E05 §05, O03 §06, IM §6.9]:

Class	Scope	May never	Source
Tenant token	One tenant, one actor, one autonomy level; the only class that can reach `/v1/writes`	Assert its own autonomy level; carry another tenant's id	E05 §05, O03 §06
Control token	Studio and registry only (`/v1/declarations`, `/v1/actors`, `/v1/tenants`)	Reach a record store — "not a restricted one, none", enforced at the network layer	E05 §04–05
Ops token	Read-only, no tenant data; break-glass is a governed write with dual control	Read tenant data	E05 §05
Service token	stack1 components calling each other	Reach KernelPort	O03 §06

E05 names an **ops token**; O03 and the IM name a **service token** [E05 §05, O03 §06, IM §6.9]. They are different things: an ops token is a human operator's read-only credential into elan1; a service token is a stack1-internal workload credential that must never reach the kernel at all. **(Build recommendation)** The kernel recognises three classes — tenant, control and ops — and treats any stack1 service token presented to KernelPort as a rejection and an alert. The service token is stack1's class, not elan1's, and "three classes and no fourth" is read per publisher. The conflict is recorded for the owners to close.

Tokens are short-lived; "a credential with a long life is a credential that will be found in a log" [O03 §06]. Target-system credentials are not tokens in this sense: they live in the per-tenant credential broker and are minted per write [O03 §06, E05 §02].

(Build recommendation) A tenant token is a signed JWT (or PASETO) issued by elan1's token service after the actor authenticates through the tenant's identity provider. The claims below are the minimum. The kernel reads `actor_kind` from the token and never from the body (§4.3.1). The kernel resolves autonomy from `autonomy_grant` at rung 6 on every call; the token carries no autonomy claim the kernel would trust.

```
{
  "iss": "https://auth.elan1.example/in",
  "sub": "actor:6f1c…",                 // actor_id
  "tid": "3a9e…",                       // tenant_id — the only source of tenancy
  "akd": "agent",                       // actor_kind — authoritative
  "scp": "act",                         // ro | stage | act
  "cls": "tenant",                      // tenant | control | ops
  "obo": "actor:12ab…",                 // optional delegation: accountability only
  "cnf": { "jkt": "…" },                // proof-of-possession key thumbprint
  "iat": 1790150400, "exp": 1790151300   // 15-minute life
}
```

4.4.4 Attribution

Three attribution rules carry the security review.

1. **`actor_kind` answers the question the reviewer asks first.** It is the field that answers "show me everything your agents did, separately from what people did" [E04 §03 rung 1]. It is set at rung 1 from the token and sealed into F10.

2. **Delegation confers accountability, never autonomy.** `on_behalf_of` records whose authority the actor claims; the autonomy level is always the acting actor's own grant, and a missing grant is A0 — "not to tenant default, not inherited from the delegation chain" [E02 §03, E04 §03 rung 6]. An invalid delegation refuses at rung 1 with `POLICY.DELEGATION_INVALID` [E02 §06].
3. **An unattributable call does not exist.** A call without a resolvable tenant and actor is rejected before rung 0b [E02 §04]. R04 requires rung1's `actor_id` to be "the human who raised the request — never an agent identity" [R04 §04]; the kernel contract permits `actor_kind: agent` [E02 §03]. The two are compatible only if rung1 registers its agents as actors and sets `on_behalf_of` to the requesting human (build recommendation); otherwise rung1's agent activity is attributed to people, and the "everything your agents did" query returns nothing. The difference is recorded in the conflicts block.

4.4.5 Refusal, rejection and platform failure

The kernel separates three kinds of "no", and consumers must too [E02 §04]:

	Refusal	Rejection	Platform failure
HTTP	200	4xx	5xx, or 200 carrying a `PLATFORM.*` code if the kernel was reached
Meaning	A governed outcome	A malformed request; nothing entered the kernel	We failed
Record	Full record written	None	Full record written, platform class
Meter	Quantity 0	None	Not billable
Retry	Never retry a refusal	Fix and resubmit	Retry with the same `intent_id`

Source: E02 §04. "Clients retry 4xx. That is why a refusal is never a 4xx" [E02 §04]. Chapter 6 §6.6 gives the full HTTP taxonomy including throttling (429) and the stopped tenant.

4.5 The six calls

4.5.1 The port in two languages

(Build recommendation) The port is published as one interface in each SDK language. Signatures below follow E02 §05; the names of request and response types are this memorandum's. Every call takes the caller's credentials from the transport (the token), never from an argument, so no signature carries `tenant_id` except inside the intent where E02 requires it for the cross-check.

```
# elan1_kernelport/port.py  (build recommendation — semantics from E02 §05)
from typing import Protocol, Sequence
from uuid import UUID
from .objects import (WriteIntent, Decision, RecordRef, DeclarationAccepted,
                      VerifyResult, ScopeResult, RefusalOutcome, Declaration,
                      AssertResponse, WriteContext)

class KernelPort(Protocol):
    def define(self, declaration: Declaration, version: int, certifier_id: UUID,
               supersedes: int | None = None) -> DeclarationAccepted: ...

    def assert_(self, intent: WriteIntent) -> AssertResponse: ...   # 'assert' is reserved

    def seal(self, write_id: UUID) -> RecordRef: ...                # kernel-only; not exported

    def verify(self, seq_from: int, seq_to: int) -> VerifyResult: ...

    def scope(self, actor_id: UUID, write_types: Sequence[str]) -> ScopeResult: ...

    def refuse(self, code: str, context: WriteContext) -> RefusalOutcome: ...
```

```
// @elan1/kernel/port.ts  (build recommendation — semantics from E02 §05)
export interface KernelPort {
  define(req: {
    declaration: Declaration; version: number; certifier_id: string; supersedes?: number;
  }): Promise<DeclarationAccepted>;
  assert(intent: WriteIntent): Promise<AssertResponse>;
  // seal is callable only by the kernel (E02 §05.3, decision 3); it is not on the
  // consumer interface and exists only in the reference implementation's internal module.
  verify(req: { seq_from: number; seq_to: number }): Promise<VerifyResult>;
  scope(req: { actor_id: string; write_types: readonly string[] }): Promise<ScopeResult>;
  refuse(req: { code: RefusalCode; context: WriteContext }): Promise<RefusalOutcome>;
}
```

The Python method is spelt `assert_` because `assert` is a reserved word; the wire name remains `assert`. `seal` appears on the Python protocol for completeness of the reference implementation, and E05 lists it among the SDK calls "exposed for the reference implementation" [E05 §03]; the published consumer package omits it, because "seal is callable only by the kernel; consumers reach the chain through assert and read it through verify" [E02 decision 3].

4.5.2 `define` — register a write type at a declaration version

Actor. A declaration author, through Studio or a file in the consumer's repository. **Trigger.** A declaration reaches CERTIFY. **Preconditions.** It passed VALIDATE; a named human certified it; the version integer is one greater than the last [E02 §05.1].

Behaviour [E02 §05.1]:

1. The author submits the certified declaration. The kernel validates the seven blocks against the schema and the semantic rules. The author sees "Accepted at version N", or the first failing rule with its block and field.
2. The kernel compiles it into a surface plan, write-type registry entries, the meter basis per write type and `[L2]` a verifier-profile stub. The author sees the compiled artefact digest.
3. The kernel writes the registry entries append-only and the version becomes resolvable. Registering a declaration is itself a governed write under `dual_control: true`: "it climbs the rungs like any other and produces its own record."

Request field	Type	Req	Meaning
`declaration`	object	yes	The seven blocks, verbatim
`version`	uint32	yes	Must be `last + 1`. Gaps are a rejection
`certifier_id`	uuid	yes	The named human
`supersedes`	uint32	no	Present when this version deprecates a predecessor

Response field	Type	Meaning
`declaration_id`	uuid	Stable across versions
`version`	uint32	As accepted
`compiled_digest`	32 octets	Content-addressed; the same declaration compiles to the same digest forever
`write_types`	array	Registered names with their resolved meter basis and blast radius

Source: E02 §05.1.

Errors. `POLICY.DECLARATION_INVALID` with the first failing rule; `POLICY.VERSION_NOT_SEQUENTIAL`; `POLICY.NOT_CERTIFIED`; `PLATFORM.REGISTRY_UNAVAILABLE` [E02 §05.1]. When the certifier is also the author, the registration write refuses at G4 with `GATE.DUAL_CONTROL_SAME_ACTOR`, and the Studio offers "Send to" with eligible human approvers only [E09 §04].

Idempotency and ordering. Keyed on `(declaration_id, version, compiled_digest)`. The same digest returns the original acceptance; a different digest at the same version is a rejection, because versions are immutable. Versions are strictly increasing per declaration, with no concurrent version N [E02 §05.1]. A retry after `PLATFORM.REGISTRY_UNAVAILABLE` "with the same content returns the same acceptance" [E09 §04].

`define` must never overwrite a version, resolve "latest", register a write type whose gates name a gate absent from the declaration, or apply retroactively to writes already recorded [E02 §05.1]. The compile must be deterministic: "a compile producing two digests from two runs fails the build as a P0" [IM §6.6].

L1 and [L2]. At L1 the compile produces the surface plan, registry entries and meter basis. At L2 it also emits the verifier-profile stub; the call shape does not change [E02 §05.1].

> **`define` has two meanings in the corpus.** E02 and R04 use `define` to publish a declaration version — rung1 calls it "at tenant onboarding and on every rung1 release that changes the ERPNext write-type set" [R04 §02]. O03 §05 uses it per action: `define(tenant, actor, declaration@version, write_type, payload, idempotency_key) → GovernedActionRef`, registering "a Governed Action in state proposed" before `assert` [O03 §03 step 3, §05]. This memorandum follows E02, the kernel's own contract, and R04, the first consumer's: `define` publishes declarations, and an intent is registered by `assert` itself. The O03 "proposed" state is served by `stage.write`, which returns a validated request object with no side effects [E05 §04]. Recorded in the conflicts block.

4.5.3 `assert` — run the rungs and return a decision

"The central call; everything else supports it" [E02 §05.2].

Actor. A consumer's write path, on behalf of a human, an agent or a scheduled job. **Trigger.** A write intent is complete. **Preconditions.** A valid `act` token; the declaration version resolvable; the tenant not stopped [E02 §05.2].

Behaviour. The kernel runs, in order, rung 0b STOP, 1 IDENTITY, 2 WRITE DOOR, 3 PRE-GATES, 4 `evaluate()`, 5 GUARDRAIL, 6 AUTONOMY, then on allow 7 ACT, 8 PROOF and 9 METERED, and returns the decision [E02 §05.2]. Chapter 5 specifies every rung. Four rules from the contract bind the implementation regardless of rung detail:

- rung 3 runs every declared gate even after the first failure, and the first failure sets the code [E02 decision 5];
- a refusal at any rung goes straight to rung 8 as a refusal-shaped record and to rung 9 at quantity zero [E04 §03 rung 3, rung 8];
- an escalation stops at rung 6; rung 7 is never reached for an escalated write, and the write is later resubmitted, never resumed [E04 decision 4];
- rung 8 returns `seq` and digest synchronously on every outcome [E04 §03 rung 8].

Response [E02 §05.2]:

Field	Type	Meaning
`decision`	object	As defined in §4.3.2
`record`	object	`seq`, `digest`, `assurance_level`
`record_id` / `record_version`	uuid / uint32	Target-system identifiers on commit; zero-UUID and 0 on refusal
`undo_until`	RFC 3339	Commit only. 30 days from apply at L1

(Build recommendation) The wire envelope adds `outcome` (`committed` | `refused` | `escalated`), the refusal's rendered operator action, and the `X-Kernel-Port: v1` header [O06 §02]. Three examples:

```
// committed — HTTP 200
{ "outcome": "committed",
  "decision": { "write_id": "0b7e…", "disposition": "allow", "rung": 7,
                "policy_version": 11, "declaration_version": 7, "autonomy_level": "A4",
                "blast_radius": "one_counterparty",
                "gate_results": [ { "name": "consent", "outcome": "pass",
                                    "inputs_hash": "…" } ],
                "reversal_plan": { "method": "compensating_entry", "window_hours": 720 } },
  "record": { "seq": 4097, "digest": "9f2c…", "assurance_level": 1 },
  "record_id": "5d0a…", "record_version": 3, "undo_until": "2027-08-14T10:02:11Z" }

// refused at rung 3 — HTTP 200, never 4xx
{ "outcome": "refused",
```

```
  "decision": { "write_id": "7c41…", "disposition": "refuse", "rung": 3,
                "code": "GATE.PRECONDITION_FAILED", "gate": "precondition",
                "policy_version": 11, "declaration_version": 7, "autonomy_level": "A3",
                "gate_results": [ "…every declared gate, pass or fail…" ] },
  "operator_action": "Fix the record state",
  "record": { "seq": 4098, "digest": "…", "assurance_level": 1 },
  "record_id": "00000000-0000-0000-0000-000000000000", "record_version": 0 }

// escalated at rung 6 — HTTP 200
{ "outcome": "escalated",
  "decision": { "write_id": "e2a9…", "disposition": "escalate", "rung": 6,
                "policy_version": 11, "declaration_version": 7, "autonomy_level": "A2",
                "obligation": { "obligation_id": "ob_91…", "expires_at": "…",
                                "resolver_roles": ["ar-approvers"] } },
  "record": { "seq": 4099, "digest": "…", "assurance_level": 1 } }
```

(The `//` comments are annotations; the wire form is plain JSON.)

Refusal paths. The user sees the code's operator action "in the words of the declaration's surface plan — never a stack trace, never a raw code alone" [E02 §05.2]. A gate-class refusal names the gate; a policy-class refusal names the rule version; a platform-class refusal says "We could not complete this. Nothing was changed. We have been paged" [E02 §05.2].

Failure paths [E02 §05.2]:

Failure	Where	Effect	Retry
Target unreachable after the sandbox apply	Rung 7	Nothing reaches live; sandbox discarded; `PLATFORM.TARGET_UNREACHABLE`	Safe, same `intent_id`
Chain append fails	Rung 8	The rung-7 transaction already committed, so the outbox row survives and the chain writer retries in order; no lost write, no gap	Kernel-internal
Writer lock lost mid-apply	Rung 7	`PLATFORM.WRITER_LOCK_LOST`; snapshot restored; pages	Safe, same `intent_id`

Idempotency. Keyed on `intent_id`. "A repeat returns the original decision including `seq` and `digest`; it does not re-run the rungs and does not meter again" [E02 §05.2]. E02 also says that after a refusal "the user resubmits with the same `intent_id` once the cause is fixed" [E02 §05.2], which cannot both hold for a governed refusal: a repeat of a refused `intent_id` would return the refusal. **(Build recommendation)** The idempotency table stores the outcome class with the key. A committed, governed-refused or escalated key is terminal and replays its original response. A platform-class refusal is not terminal: a retry with the same `intent_id` re-enters at rung 0b under the same `write_id`, and the platform-class record stays in the chain as evidence of the failure. A resubmission after a governed refusal, or after an approval, carries a new `intent_id` — for the approval case with `obligation_ref` set — because it is a new intent in the sense that matters: its context has changed. Keys are retained 30 days [E05 §08]. This reading is recorded in the conflicts block for the elan1 lead to confirm.

Ordering. "Within a tenant, records are in commit order, not arrival order. Across tenants there is no ordering guarantee and none is implied" [E02 §05.2]. Two concurrent writes get `seq` values in commit order, "which is the order that actually happened" [E05 §08]. Chapter 5 §5.16 describes the mechanics.

`assert` must never call a model, read a clock in any gate, return allow without a `policy_version`, commit without an outbox row, cache a decision, or acquire an "admin" or "emergency" variant. Break-glass is a declared write type with `dual_control: true`, recorded in the operator's own chain and visible to the tenant [E02 §05.2, decision 9].

Synchronous or not. E02 leaves open whether `assert` returns synchronously through rung 9 or at rung 8 with metering asynchronous: "Synchronous is simpler to reason about and costs latency" [E02 OQ1, due 31 October 2026]. O03 sets a meter-lag target of p99 ≤ 30 s, which implies an asynchronous meter pipeline beyond the event itself [O03 §07–08]. **(Build recommendation)** Rung 9 inserts the `meter_event` row synchronously, in the same transaction as the rung-8 chain append, so the CHECK and uniqueness constraints hold at commit; the downstream aggregation into billing lines is asynchronous and is what the 30-second lag target measures. The 2 ms rung-9 budget [E04 §02] is consistent with a single insert.

L1 and [L2]. The call shape is identical at L2; only the record returned by rung 8 gains a signature and an anchor reference [E02 §03, O03 decision 5].

4.5.4 `seal` — write the record and chain it

"At L1 this is the chain writer; at L2 the same call also signs. Callable only by the kernel" [E02 §05.3].

Actor. The kernel, at rung 8. **Trigger.** An outbox row exists. **Preconditions.** The rung-7 transaction committed (or, for a refusal or escalation, the refusal's write row committed).

Behaviour [E02 §05.3]:

1. The chain writer consumes the outbox in commit order, assigns the tenant's next `seq` and reads `prev_digest` from `seq - 1`.
2. It assembles the sixteen fields from the write context, "which is append-only within a write, so every field is present without a second read".
3. It computes the preimage and digest and appends the record; `[L2]` it signs and enqueues the anchor batch.

Response field	Type	Meaning
`seq`	uint64	The assigned sequence
`digest`	32 octets	SHA-256 over the preimage
`assurance_level`	uint8	1 at L1

Errors. `PLATFORM.LOG_APPEND_FAILED` (retried in order, never skipped); `PLATFORM.CHAIN_HEAD_CONFLICT` (two writers for one tenant — "a P0, since I2 says there is one") [E02 §05.3]. **Idempotency.** Keyed on `write_id`: a re-consumed outbox row returns the existing `seq`. **Ordering.** Strict per tenant; "a gap in `seq` is a P0, not a warning" [E02 §05.3]. The evidence store "grants INSERT to one principal and UPDATE to none" [E04 §03 rung 8].

`seal` must never be optional, per-tenant configurable or disabled for performance; assign a sequence out of order; omit a field for a refusal; or be callable by a consumer [E02 §05.3].

> **Consumers calling `seal`.** Two consumer-side documents have the caller invoke `seal`. O03 §04 has a Mode B caller perform the write itself and then report "the outcome through `seal`", after which the kernel writes the proof record with a caller-performed flag [O03 §04, decision 3]. R04 has rung1 call `seal` "immediately after the promoter's final verification step, and again on rollback" [R04 §02, §03 step 8]. Both contradict E02 decision 3. **Treatment in this memorandum:** `seal` stays kernel-only. A Mode B caller reports its outcome through an `act`-scoped outcome report (`POST /v1/writes/{write_id}/outcome`, build recommendation), which the kernel validates against the original `allow` and then seals itself, setting the caller-performed flag. rung1 in Mode A does not call `seal` at all, because rung 7 and rung 8 run inside the kernel (Chapter 6 §6.3). Recorded in the conflicts block.

4.5.5 `verify` — check a record or a chain segment

Actor. A consumer, an operator, or the nightly integrity job. **Trigger.** An audit question, or the nightly run. **Preconditions.** An `ro` token; at L1 the caller is inside the tenancy boundary [E02 §05.4].

Behaviour [E02 §05.4]:

1. The caller supplies a sequence range; the kernel recomputes each preimage and digest and compares octet for octet.
2. The kernel walks the chain: `seq` contiguous from 1, each `prev_digest` matching its predecessor's digest.
3. The kernel resolves every referenced declaration version and lexicon hash, and returns PASS or the first failure with sequence, field and reason.

E05 describes the result as "three-valued" [E05 §03], and the console renders chain standing as "continuous / break at seq N / not checked over this range, with the scope and count printed next to the word" [E06 §05, decision 3]. **(Build recommendation)** The response carries `result: pass | fail | not_checked`, the checked range, the count of records and of distinct write types in scope, and on failure `{seq, field, reason}`. `not_checked` is returned for any part of a requested range outside the caller's tenancy or beyond the current chain head, so a partial check can never read as a full one.

```
{ "result": "fail",
  "range": { "seq_from": 1, "seq_to": 10000 }, "records_checked": 5003, "write_types": 7,
  "failure": { "seq": 5004, "field": "F03 prev_digest",
               "reason": "does not equal digest(seq 5003)" } }
```

`[L2]` adds signature verification against the key register, Merkle inclusion under RFC 6962 and an RFC 3161 anchor check, "and makes the call answerable offline by a party who does not trust us. At L1, verify requires us — state that limitation to the customer in those words" [E02 §05.4]. **`verify` must never** be metered, return PASS with an unexplained gap, or be the only integrity check: the nightly job runs whether or not anyone calls it [E02 §05.4]. A missed nightly integrity job flips the chain-integrity component on the public status page without anyone pressing anything [E09 §02 item 4].

4.5.6 `scope` — resolve blast radius and autonomy

Actor. A consumer building a surface. **Trigger.** Rendering a screen, or an agent deciding whether to stage a write at all. **Preconditions.** An `ro` token [E02 §05.5].

Behaviour. The consumer names an actor and a write type; the kernel returns the declared blast radius, the actor's autonomy level, and whether the combination would reach rung 7 or stop at approval. The consumer renders accordingly: "a control the actor may not use is not rendered, and a control that will need approval says so before it is pressed, not after" [E02 §05.5].

```
// POST /v1/scope   (build recommendation — tenant and actor come from the token)
{ "actor_id": "6f1c…", "write_types": ["journal_entry.submit", "payment_entry.submit"] }
// → 200
{ "declaration_version": 7,
  "permitted": [
    { "write_type": "journal_entry.submit", "blast_radius": "batch",
      "autonomy_level": "A2", "reaches": "approval", "dual_control": true },
    { "write_type": "payment_entry.submit", "blast_radius": "one_counterparty",
      "autonomy_level": "A0", "reaches": "none" } ] }
```

"`scope` is advisory and the kernel does not trust its own answer: `assert` re-resolves both" [E02 §05.5, decision 4]. **`scope` must never** be treated as an allow, mutate anything, or be reachable without a tenant [E02 §05.5]. R04 has rung1 use `scope` at the start of every agent run to fix the agent's tool allow-list for that run [R04 §02–03]; that is a legitimate use because it narrows what the agent can attempt, and it does not substitute for `assert`.

4.5.7 `refuse` — emit a typed refusal

Actor. The kernel; and a consumer's adapter when it detects a register condition before the kernel is reached. **Trigger.** Any stage returning refuse. **Preconditions.** The code exists in the register [E02 §05.6].

Behaviour [E02 §05.6]:

1. The caller supplies the code and the write context. The kernel checks that the code is registered and maps to the rung it was raised at. "A code raised at the wrong rung is a defect and fails CI."
2. The kernel records the refusal as a full sixteen-field record, `record_id` zeroed and `record_version` 0, meters at quantity 0 and returns HTTP 200.

"`refuse` is a call rather than a return value because a refusal is a product surface: designed once, for every consumer, rendered from the declaration's surface plan" [E02 §05.6]. **`refuse` must never** accept a code outside the register, accept free text in place of a code, meter above zero, return 4xx, or be softened into a warning [E02 §05.6].

R04 uses `refuse` for rung1-side stops: an approver declines, the operator presses Stop, or a pre-flight check fails inside rung1; it also calls `refuse` with `APPROVAL_EXPIRED` when an escalation lapses [R04 §02, §06]. `APPROVAL_EXPIRED` is not in the kernel register; the kernel code is `POLICY.OBLIGATION_EXPIRED`, and E04 makes the expiry a kernel-raised governed event rather than a consumer call [E04 Flow 3, E02 §06]. A declined approval is likewise recorded by the kernel when the approval write resolves, not by a consumer `refuse` [E04 Flow 3]. **(Build recommendation)** The kernel raises both. A consumer's `refuse` is accepted only for codes whose register entry marks them `consumer_raisable: true`. At L1 that set is empty until a code is registered through the procedure in §4.6.4, because every active L1 code is raised by a kernel rung.

4.5.8 How the consumer documents use the calls

O03 and R04 were written from the consumers' side and use four calls differently from E02: O03 has `define` register a per-action Governed Action, `assert` stop at rung 6 with `needs_approval`, and a Mode B caller report its outcome through `seal`; R04 has rung1's promoter perform the live apply and call `seal` afterwards [O03 §03–05, R04 §02–07]. Each difference changes who writes which code. This memorandum follows E02, the kernel's own contract, throughout, and carries every difference into the conflicts block.

4.6 The refusal register at L1

4.6.1 Shape of the register

The register is a closed enumeration of **43 codes in three classes — policy 14, gate 19, platform 10**. Twenty-eight are active at L1; the remainder "concern signing, anchoring, bundles and verifier profiles and activate at L2" [E02 §06, decision 6]. The IM adds that six more are added at L3 "through the register's own procedure rather than as ad hoc strings" [IM §6.4]. E02 "fixes the wire spelling for the first time": `POLICY.*`, `GATE.*`, `PLATFORM.*` [E02 §06]. Every code maps to exactly one rung, one class and one operator action [E02 I4]. Codes are never renumbered and never reused; a retired code stays retired [E05 §10].

4.6.2 The codes E02 enumerates

All refusals below return HTTP 200 with `outcome: refused`, write a full sixteen-field record with `record_id` zeroed (except `POLICY.TENANT_STOPPED`, which writes none), and meter at quantity 0 [E02 §04–06, E04 decision 5]. Platform-class codes are not billable and every one pages [E02 §06].

Code	Class	Rung	Means	Operator action
`POLICY.TENANT_STOPPED`	policy	0b	The tenant is stopped	Resolve the stop event; it names its cause class
`POLICY.ACTOR_NOT_PERMITTED`	policy	1	`permissions` does not grant this actor kind this write type	Grant it in a new declaration version, or resubmit as a permitted actor
`POLICY.DELEGATION_INVALID`	policy	1	`on_behalf_of` is not a live delegation	Renew the delegation
`POLICY.WRITE_TYPE_UNKNOWN`	policy	2	Not in `write_types` at that version	Publish a version that declares it
`POLICY.CHANNEL_NOT_PERMITTED`	policy	2	`channels` does not carry this write type	Use a permitted channel or declare this one
`POLICY.DECLARATION_UNRESOLVED`	policy	2	Version absent or uncertified	Certify it
`POLICY.DENIED_BY_RULE`	policy	4	A named rule denied at `policy_version`	Read the named rule; change the request or the rule
`POLICY.NO_RULE_MATCHED`	policy	4	No rule covers this case	Author a rule. Never widen a default
`POLICY.BLAST_RADIUS_EXCEEDED`	policy	5	Wider than declared	Narrow the request, or raise the declared radius (forces dual control)
`POLICY.AUTONOMY_INSUFFICIENT`	policy	6	Grant below what this write type needs	Raise the grant (a governed write) or route to approval

Code	Class	Rung	Means	Operator action
`POLICY.APPROVAL_DECLINED`	policy	6	An approver declined	Address the approver's basis and resubmit
`POLICY.OBLIGATION_EXPIRED`	policy	6	The escalation expired unresolved	Resubmit; expiry is itself a recorded governed event
`GATE.CONSENT_MISSING`	gate	3	No consent covers this purpose	Capture consent
`GATE.CONSENT_EXPIRED`	gate	3	Consent lapsed	Re-capture
`GATE.RESIDENCY_MISMATCH`	gate	3	Record's residency domain ≠ tenant's	Never override. Escalate to the residency runbook
`GATE.RETENTION_WINDOW_CLOSED`	gate	3	Record outside its declared retention window	Do not write. Check the retention declaration
`GATE.DUAL_CONTROL_PENDING`	gate	3	No second approver yet	Request approval
`GATE.DUAL_CONTROL_SAME_ACTOR`	gate	3	Approver equals requester	Find a second named actor
`GATE.RATE_EXCEEDED`	gate	3	Above the declared per-actor rate	Wait, or raise the declared rate
`GATE.BUDGET_EXHAUSTED`	gate	3	Above the declared per-period budget	Raise the budget; never truncate silently
`GATE.COUNTERPARTY_UNKNOWN`	gate	3	Counterparty not registered	Register it
`GATE.COUNTERPARTY_SANCTIONED`	gate	3	Counterparty on a sanctions list	Never override. Escalate to compliance
`GATE.OUTSIDE_SCHEDULE`	gate	3	Outside the declared operating window	Wait, or change the window
`GATE.PRECONDITION_FAILED`	gate	3	A declared record state does not hold	Fix the record state
`GATE.SANDBOX_DIVERGED`	gate	3 (raised at 7)	Live re-run disagreed with the sandbox result	Snapshot restored. Investigate before resubmitting
`PLATFORM.REGISTRY_UNAVAILABLE`	platform	2	Declaration registry down	Page. Retry with the same `intent_id`
`PLATFORM.POLICY_UNAVAILABLE`	platform	4	Rule engine down	Page. Never fall back to a default allow
`PLATFORM.TARGET_UNREACHABLE`	platform	7	Target system unreachable	Page. Nothing was applied
`PLATFORM.TOKEN_MINT_FAILED`	platform	7	Could not mint the short-lived credential	Page
`PLATFORM.WRITER_LOCK_LOST`	platform	7	Single-writer lock lost	Page. Snapshot restored
`PLATFORM.LOG_APPEND_FAILED`	platform	8	Chain append failed	Page. Outbox retries in order
`PLATFORM.CHAIN_HEAD_CONFLICT`	platform	8	Two chain writers for one tenant	P0. Stop the tenant

Source: E02 §06. `define` raises three further codes not in the table: `POLICY.DECLARATION_INVALID`, `POLICY.VERSION_NOT_SEQUENTIAL` and `POLICY.NOT_CERTIFIED` [E02 §05.1, E09 §04].

4.6.3 Three inconsistencies the build must close

1. **The count.** E02's table enumerates 32 codes (12 policy, 13 gate, 7 platform), `define` adds three policy codes, and E02 states that 28 are active at L1 and that the full register has 14 policy and 10 platform codes [E02 §06]. Thirty-five named codes cannot be 28 active codes. **(Build recommendation)** The register file is the source of truth, each entry carries `active_from: L1 | L2 | L3`, and a CI test asserts the per-class and active counts against the published numbers; the numbers in prose are corrected to whatever the file says. E04 separately asks which of the active codes are reachable with only ERPNext declared, and whether the unreachable ones need fixtures anyway (due October 2026) [E04 OQ5].
2. **`GATE.SANDBOX_DIVERGED`.** E02 registers it at rung 3, E04 raises it at rung 7 after the live re-run, and E02 leaves open whether it is gate-class or platform-class: "It is our machinery disagreeing with itself, which argues platform; it protects the customer's books, which argues gate" [E02 §06, OQ3, due 31 October 2026; E04 rung 7]. Because a code raised at the wrong rung fails CI [E02 §05.6], the register entry must say rung 7 before the kernel can raise it. This memorandum lists it as gate class, raised at rung 7, pending the decision.

3. **The spelling scheme.** O01 DR-011 renames the refusal register `REF-nn`; E02 fixes dotted wire spellings; O03 speaks of "a REF code" [O01 §05 DR-011, E02 §06, O03 §03]. **(Build recommendation)** Each register entry carries both: a permanent numeric `ref` (`REF-01`…`REF-43`, never reused) for documents and support, and the dotted `wire` spelling for APIs. Neither is ever reassigned.

4.6.4 Adding a code, and the register as data

"Adding a code takes a decision, not a commit: name what it protects, write its contract, register it with rung, class and operator action, implement it as a pure function, add positive and negative fixtures to the reference tenant, and add it to the adversarial declaration. The negative fixtures matter more" [E02 §06]. A new code in an existing class is a register version increment with 30 days' internal notice and 60 days' external notice, operator action published first [E02 §07, O06 §04].

```
# elan1-core/register/refusal_codes.yaml  (build recommendation — one entry)
- ref: REF-24
  wire: GATE.PRECONDITION_FAILED
  class: gate            # policy | gate | platform
  rung: 3
  active_from: L1
  billable: false        # every refusal meters at quantity 0 (E02 I4)
  pages: false           # true for every platform-class code (E02 §06)
  consumer_raisable: false
  operator_action: "Fix the record state"
  fixtures: { positive: fx/g9_pass_*.json, negative: fx/g9_fail_*.json }
  adversarial_declaration: adv/decl_precondition_edges.json
```

4.6.5 Why platform-class codes exist

"Platform-class codes are the honest ones, and the ones most vendors do not have. A governance layer that reports its own failures as policy refusals is lying to its customer in the most damaging way available: it makes them believe their controls are working when our infrastructure fell over" [E02 §06]. The build consequence is that every dependency the kernel calls has a named platform code, and a dependency without one cannot be added: the fail-closed matrix (C5) takes each dependency down in turn and expects a platform refusal, never an allow and never a policy code [E02 §08].

4.7 Versioning and deprecation

KernelPort's versioning is set by E02 §07 and extended to external consumers by O06, which states that "E02's ninety days is the internal floor. E05's twelve months is the external floor. Both stand" [O06 §04].

Kind of change	Version effect	Internal notice	External notice
New optional response field	None; consumers must ignore unknown fields	None	None
New refusal code in an existing class	Register version increments	30 days, operator action first	60 days, operator action first
New required request field	New port major	90 days, two releases in parallel	12 months from dated notice, headers throughout
Change to a rung's order or presence	New port major	180 days plus a written argument	12 months, argument published
New value in the disposition enum	New port major (breaking)	As above	As above
Change to the sixteen fields or their encoding	Not permitted at v1	—	—

Sources: E02 §07, O06 §03–04. E05 lists "a new refusal code inside an existing class" among changes that "ship without notice" [E05 §10], which contradicts E02's 30 days; O06's table, which is later and explicitly reconciles the two, prevails.

Where a consumer reads the version [O06 §02]: the port major (`kernelport/v1`) is the SDK package major and the `X-Kernel-Port: v1` header on every kernel response; the HTTP surface is the `/v1` path segment with `Deprecation` and `Sunset` headers once superseded; the MCP tool surface is versioned by the declaration version that exposes it; the refusal register has its own integer version. `GET /v1/versions` returns the port major, the Declaration schema major, the register version and the live policy version, and a consumer's build fails if the responses do not echo its pins [O06 §05].

What is not a version. The proof preimage is frozen; changing it invalidates every digest written under the old shape, and when L2 adds a signature it signs the same bytes [E05 §10, O06 §02]. A declaration version never expires: an action written under version 4 resolves against version 4 forever, and "latest" exists only in sandbox [O06 §02, E05 §10].

Anti-degradation. "A deprecated version keeps answering, unchanged, for its notice period. It is never made slower, never rate-limited into uselessness, never silently mapped onto its successor" [E02 §07]; O06 adds "never given a worse error message to encourage movement" and prohibits degradation by name [O06 §04]. The publisher carries the migration: a named engineer, a mechanical diff, a codemod or worked examples, and a run of the consumer's own conformance suite against the new major before the notice goes out [O06 §04]. At Sunset the old major stops answering only for consumers with both notices recorded; a pinned major past Sunset returns 410 with the notice reference [O06 §05–06].

4.8 Conformance tests

"A consumer may not claim 'runs on the elan1 layer' until all of these pass in its own CI against the reference tenant. The suite is the claim" [E02 §08].

#	Test	Passes when	Invariant
C1	Import-graph check	No consumer module imports a kernel type; no module reachable from rungs 3–8 imports a model client	Port rule, I1
C2	Bypass attempt	Every known path to the target other than the single writer is shown to fail, with the attempt recorded	I2
C3	Rung completeness	For 100 sampled intents, all nine rungs executed in order, with a gate-result row per declared gate	I5
C4	Determinism replay	Every reference-tenant refusal replays to the identical decision; a stage that acquired a clock, a random value or an undeclared read fails the same night	I8
C5	Fail-closed matrix	Registry, rule engine, autonomy table and target each made unavailable in turn; every case refuses, none allows	I3
C6	Missing grant	An actor with no autonomy row resolves to A0 in 1,000 fuzzed cases	I3
C7	Idempotency	A repeated `intent_id` returns the original decision, `seq` and digest, and produces no second meter event	I4, I5
C8	Refusal shape	Every refusal returns HTTP 200, a register code, a rung, a full record with zeroed record identifier, and a meter event of quantity 0	I4
C9	Chain integrity	10,000 records verify; a flipped octet, a re-ordered pair, a duplicated sequence and a gap each fail with the correct sequence and field named	I2, I4
C10	Meter reconciliation	Every committed write has one record, every record one write, every meter event a write that exists, every refusal quantity 0	I4
C11	Approval separation	An approver equal to the requester is refused; an agent approver is refused at L1	— (E02 decision 7)
C12	Rollback drill	A committed write is undone inside the window by both the fast and the forward path, each producing its own governed record	I2
C13	Version pinning	An intent carrying `"latest"`, or a version that does not exist, is rejected — not defaulted	I7
C14	Claim boundary	The consumer's user-facing copy contains no "signed", "tamper-proof" or "independently verifiable" while running at L1	— (E02 decision 8)

Source: E02 §08. "C14 is a test, not a guideline. It runs as a string check over the consumer's rendered copy, and it fails the build" [E02 §08]. The invariant column is this memorandum's mapping.

C13 and O06 differ on one point: C13 says a non-existent pinned version is "rejected — not defaulted", while O06 Flow 1 says it "refuses with `POLICY.DECLARATION_UNRESOLVED`, recorded and metered at zero", reserving rejection for the literal `"latest"` [E02 §08, O06 §05]. **(Build recommendation)** Follow O06: `"latest"` or a non-integer is a 4xx rejection (it cannot be typed as a `uint32`); a well-formed integer that does not resolve is a governed refusal at rung 2, which is what E02's own register says `POLICY.DECLARATION_UNRESOLVED` is for. C13 then asserts both halves.

Running the suite. E09 publishes C1–C14 "runnable by a consumer against the reference tenant, with fixtures, expected outcomes and a machine-readable result file", and requires that a clean clone passes on elan1's reference implementation and fails on a deliberately broken one [E09 §02 item 8]. C1 and C2 run first because they are cheap and disqualifying; C8 requires a real refusal and a run that produced none "did not exercise the product" [E09 §05]. The result file is a required attachment to a production-key request [E09 decision 3]. Chapter 6 §6.9 covers onboarding. Who owns the reference tenant and its adversarial declaration — the kernel team or a separate assurance owner — is open with the founder, due 31 October 2026 [E02 OQ5].

```
{ "suite": "kernelport-conformance", "suite_version": "1.0.0", "port": "kernelport/v1",
  "consumer": "rung1-enterprise", "commit": "8c1f…", "reference_tenant": "ref-in-01",
  "declaration": { "id": "…", "version": 7 },
  "results": [ { "id": "C1", "status": "pass" }, { "id": "C8", "status": "pass",
                 "evidence": { "seq": 212, "code": "GATE.PRECONDITION_FAILED" } } ],
  "summary": { "passed": 14, "failed": 0 } }
```

(Result file shape is a build recommendation.)

4.9 Decisions and open questions

E02 records nine decisions, all dated 21 September 2026 and cited where they apply above: the six calls (decision 1), the frozen layout (2), kernel-only `seal` (3), advisory `scope` (4), all gates run (5), the wire spellings (6), no agent approver at L1 (7, founder), C14 as a build failure (8, founder) and break-glass as a declared write type (9) [E02 §09]. O01 DR-013 keeps `seal` in the port at L1 as a hash-chain receipt [O01 §05]. The open questions that bind this chapter are:

#	Open question	Owner, by	Source
1	Does `assert` return synchronously through rung 9, or at rung 8 with metering asynchronous?	elan1 lead, 31 Oct 2026	E02 OQ1
2	The F14 salting scheme at L1 that survives the L2 retrofit	elan1 lead, 30 Nov 2026	E02 OQ2
3	Is `GATE.SANDBOX_DIVERGED` gate-class or platform-class?	elan1 and rung1 leads, 31 Oct 2026	E02 OQ3
4	Does running every gate after the first failure let an attacker map the gate set?	elan1 lead, 30 Nov 2026	E02 OQ4
5	Who owns the reference tenant and adversarial declaration?	Founder, 31 Oct 2026	E02 OQ5
6	Where does an `acted_unproven` action go — retry queue or manual reconciliation?	elan1 kernel lead, 15 Oct 2026	O03 OQ3
7	Must the Mode B caller-performed flag be visible to the end customer, or only in the export?	elan1 kernel lead, 30 Nov 2026	O03 OQ4
8	If the preimage ever must change, what notice period is meaningful?	elan1 kernel lead, before L2 scoping	O06 OQ5

This chapter adds three questions of its own, each carried into the conflicts block: how `mode`, `enforcement_mode` and the caller-performed flag are bound into a record whose sixteen fields do not name them; which reading of idempotency applies to a resubmission after a governed refusal; and whether the third token class is ops or service.

5. The Right Path — nine rungs and the governed action end to end

`assert` is one function, and the Right Path is what it does. "Every governed write climbs the same checks in the same order, and each one leaves a row behind" [E04 §01]. There is one write function and no second write path, "including for demonstrations" [O01 §02]. This chapter specifies the path as a runtime: for each rung, what it reads, what it decides, what the caller gets back, what lands in the database, which refusal codes it can raise, how long it may take, how it fails and what a model can and cannot reach [E04 §02]. It then walks one governed action end to end — an agent posting a journal entry through rung1 Enterprise under the ledger1 pack — including escalation, approval, single-writer execution, proof and meter, and closes with idempotency, dependency failure, concurrency and durable execution on stack1's workflow engine.

> **Status and scope.** The path is specified for Phase 1, Assurance Level 1 Guarded: policy gates, human approval, sandbox, 30-day rollback and a hash-chained log. There is no signature, no RFC 3161 anchor and no verifier profile in Phase 1 [E04 §01]. Every budget below is "a p50 design target, not a measurement"; each is marked unverified until Gate 1 replaces the table with readings [E04 §02]. No governed action has yet run for a customer [IM §4.9]. Details the sources do not give are marked **(build recommendation)**.

5.1 Nine rungs and a stop set

5.1.1 The order

Rung	Name	What it decides	Deterministic?	Budget p50 (target)
0b	STOP	Is this tenant, actor or target in the stop set?	yes	4 ms (with 1 and 2)
1	IDENTITY	Who is acting, of what kind, on whose behalf	yes	(in the 4 ms)
2	WRITE DOOR	Does this write type exist at this explicit version, on this channel?	yes	(in the 4 ms)
3	PRE-GATES	Do the declared gates pass, as pure functions of `as_of`?	yes	8 ms
4	`evaluate()`	Do the versioned rules allow, refuse or escalate?	yes, at a fixed policy version	12 ms
5	GUARDRAIL	Is the measured blast radius within the declared one?	yes	8 ms (with 6)
6	AUTONOMY	May this actor act alone, or must a human approve?	yes	(in the 8 ms)
7	ACT	Kernel transaction: record version plus one outbox row	no — I/O	18 ms
8	PROOF	Chain the sixteen-field record, return `seq` and digest	yes	12 ms
9	METERED	Emit exactly one meter event	no — I/O	2 ms
	Kernel total			**≈ 64 ms**

Source: E04 §02–03. Figure 6 shows the rungs with their refusal exits.

Figure 6 — The Right Path — nine rungs in one order

L1 — BEING BUILT · BUDGETS ARE TARGETS

Every governed write climbs the stop check and nine rungs in the same order; every exit is typed, and each leaves a row

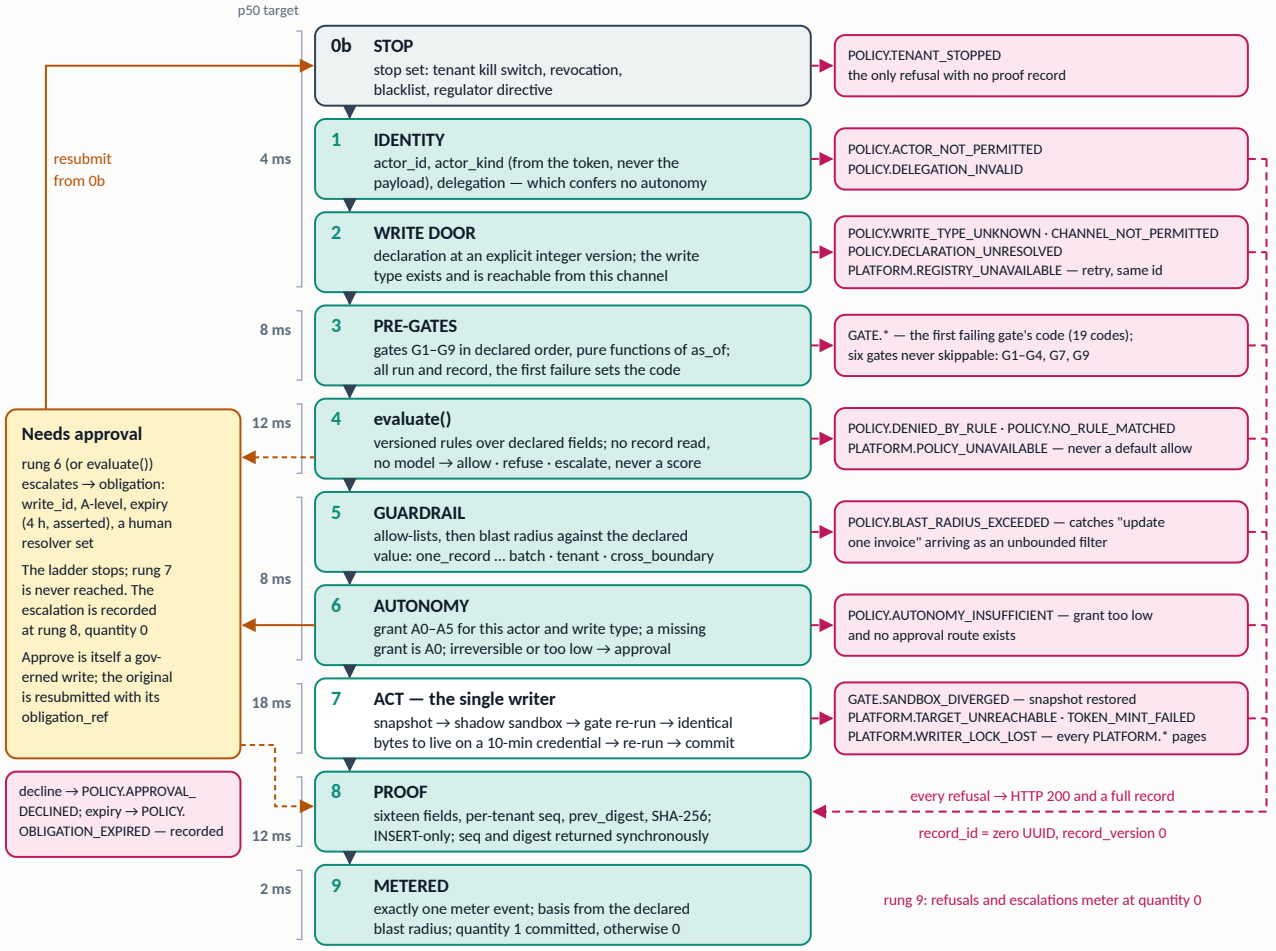


Figure 6 — The Right Path: the stop set at 0b and the nine rungs in fixed order, each with its refusal exit to the proof rung, the escalation exit at rung 6, the sandbox and live applies drawn outside the kernel budget, and the one rung (0b) whose refusal writes no chain entry.

5.1.2 Nine or ten

The corpus counts the path two ways. The IM, E02 conformance test C3 and the working-draft brief say nine rungs, with the stop set at 0b in front of them ("nine rungs… plus Stop") [IM §4.3, E02 §08, WD]. E04 titles its section "The ten rungs", and O01 says "eight of the ten rungs make no model call" [E04 §03, O01 §02]. This memorandum says **nine rungs and a stop set**: rungs 1–9 are the ladder, and 0b is a gate in front of it whose refusal is the only one without a chain entry. C3's "all nine rungs executed, in order" is read as rungs 1–9 plus a passed 0b check, so a test harness counts ten stage executions per committed write. The difference is recorded in the conflicts block because a conformance harness that counts nine will fail on a correct kernel.

5.1.3 Rules that apply to every rung

- **No skip, no fast path.** "There is no configuration that skips a rung and no fast path" [E04 §02]. I5 forbids a batch variant, an admin mode and a decision cache [E02 I5].

- **Fail closed.** Every stage's default is refuse; a missing gate result is a failure; an unavailable dependency is a platform-class refusal [E02 I3].
- **Refusal exits go to rung 8.** A refusal at rungs 1–7 jumps to rung 8, which writes a refusal-shaped record, and to rung 9, which meters at zero [E04 §03 rungs 3, 8, 9]. An escalation at rung 6 does the same with an obligation attached.
- **No model on rungs 3–8.** Rungs 3, 4, 5, 6 and 8 are unreachable from any model, enforced by the import-graph check [E02 I1]; rung 7 applies bytes that were fixed before it started. A model may sit upstream of `assert` and compose the intent [E02 I1].
- **The write context is append-only within a write.** Each rung adds to it and none rewrites it, so rung 8 can assemble all sixteen fields "without a second read" [E02 §05.3].

(Build recommendation) The kernel implements the ladder as a fixed list of stage objects executed by one loop, with no conditional that can omit a stage other than the refusal and escalation exits. A unit test asserts the list's contents and order against a constant, and the constant is the same file the conformance suite reads for C3.

```
# elan1_core/right_path.py  (build recommendation)
RUNGS: tuple[Stage, ...] = (Stop(), Identity(), WriteDoor(), PreGates(), Evaluate(),
                            Guardrail(), Autonomy(), Act(), Proof(), Metered())

def run(intent: WriteIntent, ctx: WriteContext) -> AssertResponse:
    for stage in RUNGS[:7]:                    # 0b .. 6: decide
        verdict = stage.check(intent, ctx)     # pure: reads ctx and as_of only
        ctx = ctx.append(stage.name, verdict)  # append-only
        if verdict.kind in ("refuse", "escalate"):
            if stage.name == "0b":             # stopped: no chain entry (E04 dec. 5)
                return stopped_response(verdict)
            return finish(ctx, verdict)        # straight to rung 8 and 9
    ctx = RUNGS[7].act(intent, ctx)            # rung 7: may refuse (PLATFORM.*, divergence)
    return finish(ctx, ctx.final_verdict())

def finish(ctx, verdict) -> AssertResponse:
    record = RUNGS[8].seal(ctx)                # rung 8: synchronous seq and digest
    RUNGS[9].meter(ctx, record)                # rung 9: quantity 0 unless committed
    return response(ctx, verdict, record)
```

5.2 Rung 0b — STOP

Purpose. Refuse everything for a tenant that has been stopped. The stop set holds the tenant kill switch, revocation, a blacklist and a regulator directive [IM §4.3, O01 §02]. E02 describes the check as "tenant, residency, stop flag" [E02 §05.2].

Inputs	Tenant token; the tenant's stop state
Pass	Continue to rung 1
Output on fail	`POLICY.TENANT_STOPPED`, carrying the reason class and the time the stop was set
State written	An operational log line and the stop event — **no chain entry**
Budget	Inside the 4 ms shared with rungs 1 and 2 (target)
Model reach	None

Source: E04 §03 rung 0b. This is "the one refusal in the system with no proof record, and the stop event that caused it is itself a governed write with a record" [E04 §03]. The reason is that a stopped tenant writes nothing, and a chain entry is a write [E04 decision 5]. Throwing the stop is a governed write under dual control on the Tenants admin screen, and "it stops all writes including refusals" [E06 §07]. `PLATFORM.CHAIN_HEAD_CONFLICT` at rung 8 is the one kernel event that sets the stop automatically: "P0. Stop the tenant" [E02 §06].

Failure behaviour. If the stop state cannot be read, the kernel cannot know the tenant is live, and I3 requires a refusal. E02 has no platform code for an unreadable stop set. **(Build recommendation)** The stop set is held in the same transactional store as the write rows and read in the same connection, so its unavailability surfaces as the store being down; the request

fails as a 5xx before rung 1 and is retried by the caller with the same `intent_id`. A dedicated platform code would be cleaner and needs a register decision.

5.3 Rung 1 — IDENTITY

Purpose. Establish who is acting, of which kind, and on whose behalf.

Inputs	Tenant token, delegation chain
Pass	`actor_id`, `actor_kind` (human · agent · system) and the delegation resolved
Refusals	`POLICY.ACTOR_NOT_PERMITTED`, `POLICY.DELEGATION_INVALID`
State written	The write row begins; `actor_kind` is set by the kernel
Budget	Inside the 4 ms (target)
Model reach	None

Source: E04 §03 rung 1. "`actor_kind` is set by the kernel and can never be supplied by a caller" [E04 §03]. It is the field that answers "show me everything your agents did, separately from what people did" [E04 §03]. `POLICY.ACTOR_NOT_PERMITTED` means the `permissions` block does not grant this actor kind this write type, and its operator action is to grant it in a new declaration version or resubmit as a permitted actor [E02 §06]. An unattributable call never reaches rung 1: it is a rejection at the transport [E02 §04].

5.4 Rung 2 — WRITE DOOR

Purpose. Resolve the write type at an explicit declaration version and confirm the channel may carry it. "A write type that is not declared cannot be attempted — there is no generic write" [E04 §03 rung 2].

Inputs	Write type name, channel, declaration registry
Pass	`declaration_id` + `declaration_version` + the write-type descriptor (blast radius, reversal plan, gate list, dual control)
Refusals	`POLICY.WRITE_TYPE_UNKNOWN`, `POLICY.CHANNEL_NOT_PERMITTED`, `POLICY.DECLARATION_UNRESOLVED`; platform `PLATFORM.REGISTRY_UNAVAILABLE`
State written	Write row updated with the resolved versions
Budget	Inside the 4 ms (target)
Model reach	None

Sources: E04 §03 rung 2, E02 §06. The descriptor fields come from the Declaration's `write_types` block: name, blast radius, dual control, ordered gate list, and exactly one of a reversal plan or `irreversible: true` [E03 §02 block 4]. "Latest" never reaches rung 2: it is a rejection at the transport [E02 I7, O06 §05]. The channel check is where an MCP-originated write meets its `channels[].mode`, so the enforcement mode the record will carry is fixed here [E03 §02 block 7]. Tool descriptions shown to an agent are pinned by hash in the declaration, so a changed description is a new version and a pinned intent resolves against the description its agent was shown [E05 §04].

(Build recommendation) The registry is append-only and every certified version is immutable, so the write door caches compiled declaration versions in process memory keyed by `(declaration_id, version, compiled_digest)`. This is not the decision cache I5 forbids: it caches inputs whose identity is their digest, never an outcome. A cache miss that cannot reach the registry is `PLATFORM.REGISTRY_UNAVAILABLE`, never a fall-back to another version.

5.5 Rung 3 — PRE-GATES

Purpose. Run the declared gates, in declared order, as pure functions of `(context, declaration_version, record_state, as_of)`. "No gate calls `now()`" [E04 §03 rung 3].

Gate	Skippable	Asserts	Refusal codes
G1 consent	never	A recorded, unexpired consent covering this purpose	`GATE.CONSENT_MISSING`, `GATE.CONSENT_EXPIRED`
G2 residency	never	The record's residency domain matches the tenant's	`GATE.RESIDENCY_MISMATCH`
G3 retention	never	The record is inside its declared retention window	`GATE.RETENTION_WINDOW_CLOSED`
G4 dual_control	never	A second named actor has approved this `write_id`	`GATE.DUAL_CONTROL_PENDING`, `GATE.DUAL_CONTROL_SAME_ACTOR`
G5 rate	yes	Within the declared per-actor rate	`GATE.RATE_EXCEEDED`
G6 budget	yes	Within the declared per-period budget	`GATE.BUDGET_EXHAUSTED`
G7 counterparty	never	The counterparty is known and not sanctioned	`GATE.COUNTERPARTY_UNKNOWN`, `GATE.COUNTERPARTY_SANCTIONED`
G8 schedule	yes	Inside a declared operating window	`GATE.OUTSIDE_SCHEDULE`
G9 precondition	never	A declared record state holds	`GATE.PRECONDITION_FAILED`

Sources: E04 §03 rung 3, E02 §06.

Pass. Every gate returns pass. **Fail.** The refusal carries the first failing gate's code, "but every declared gate still runs and records — so an operator sees every problem at once instead of one per resubmission" [E04 §03]. The ladder then goes straight to rung 8 as a refusal. **Caller sees** the code, the gate name, a plain sentence and the full list of gate results. **State written:** one `gate_result` row per declared gate, pass or fail — which gate, what it decided, at which declaration version, at which `as_of` [E04 §03]. **Budget:** 8 ms p50 target [E04 §02].

The six never-skippable gates are enforced twice. "A declaration that omits one fails validation, and if a validator bug let it through the kernel runs it anyway" [E04 §03]. Only rate, budget and schedule may be declared off, "because only those three are commercial rather than protective" [IM §4.3], and only with an architecture decision record [IM §6.6].

Rate and budget are governed, throttling is not. A declared G5 or G6 failure is a governed refusal with a proof entry, "because the tenant declared that limit and the refusal is evidence the limit worked". Exceeding the per-token transport limit is HTTP 429 with `Retry-After`, no proof entry and no meter event [E05 §06].

Where gates get their inputs. A gate reads `record_state`, which for a consumer-hosted system of record is not in the kernel. rung1's five Phase 1 gates (ledger, stock, permissions, tax/GST, MCA audit trail) are computed by rung1 against the sandbox and bound to the kernel's `precondition` gate, with rung1's own code carried as detail [R04 §03–04, RTM Ch.5 §5.7]. **(Build recommendation)** The kernel's gate functions take a `record_state` snapshot that the adapter supplies in the write context, hashed into `inputs_hash`; the gate re-run at rung 7 is where the kernel checks that snapshot against the sandbox result and then against live. A gate that needs data the context does not carry is a declaration defect, not a reason for the gate to read the store.

```
# elan1_core/gates/contract.py  (build recommendation — purity rule from E04 rung 3, E02 I8)
from typing import Protocol

class Gate(Protocol):
    name: str                                          # one of the nine
    def evaluate(self, ctx: GateContext, params: GateParams,
                 declaration_version: int, as_of: str) -> GateResult: ...
    # Forbidden in implementations (static lint + C4 replay): datetime.now, time.time,
    # random, uuid4, network or database clients. GateContext carries record_state.
```

Information leakage. Whether running every gate after the first failure "leak[s] information an attacker can use to map the gate set" is open, due 30 November 2026 [E02 OQ4]. **(Build recommendation)** The full gate list is returned to tokens whose actor holds at least `stage` on the write type and to the console; a token with only `act` for a different write type receives the first failing gate alone. The record keeps every row either way, so the question affects disclosure, not evidence.

Dual control and the rung order. G4 asserts that a second named actor has approved this write, which on a first submission is never true. Read literally, every dual-control write would refuse at rung 3 with `GATE.DUAL_CONTROL_PENDING` before rung 6 could escalate it, yet E04's approval walkthrough escalates at rung 6 and says the resubmission passes "because the dual-control gate G4 now finds a second named actor" [E04 Flow 3, E02 §06]. **(Build recommendation)** On a submission with no `obligation_ref`, G4 records the outcome `pending` rather than `fail`; `pending` does not set a refusal code, and rung 6 treats a pending G4 as requiring approval and escalates. On a resubmission, G4 passes only if `obligation_ref` resolves to an approved obligation whose approver is a named human different from the actor; an open obligation fails with `GATE.DUAL_CONTROL_PENDING`, and an approver equal to the actor fails with `GATE.DUAL_CONTROL_SAME_ACTOR`. `pending` is a gate-result value, not a fourth disposition, so I6 is untouched. This needs the elan1 lead's confirmation and is recorded in the conflicts block.

5.6 Rung 4 — `evaluate()`

Purpose. Apply the tenant's versioned rules. `evaluate(request, declaration_version, policy_version)` runs "against a projection of declared fields only" [E04 §03 rung 4].

Inputs	subject · action · resource · context; the resource is assembled from the context, not read from the record store
Outcomes	`allow`; `refuse` as `POLICY.DENIED_BY_RULE` or `POLICY.NO_RULE_MATCHED`; `escalate`
Platform refusal	`PLATFORM.POLICY_UNAVAILABLE` — never a default allow
State written	The decision with its `policy_version`
Budget	12 ms p50 (target)
Model reach	None, at any level

Sources: E04 §03 rung 4, E02 §06. "The policy service has no read path to records" [E04 §03]. "Caller sees one of three outcomes, never a score." "There is no model in this path and no configuration that puts one there" [E04 §03]. `POLICY.NO_RULE_MATCHED` means nothing covers this case, and its operator action is "author a rule, never widen a default" [E04 §03, E02 §06]. Escalate is "the honest version of warn — a human must decide and the write does not proceed until they do" [E04 §03]. **(Build recommendation)** An escalate at rung 4 is carried forward; rungs 5 and 6 still run, and rung 6 is the single place that creates obligations, so an approval flow has one entry point whichever rung asked for it.

Engine. The product-folder stack document recommends elan1's own rules-as-data evaluator (`@elan1/policy`), with Cedar, CEL and OPA/Rego as alternatives, under the constraint that the engine "must read only the projection and `as_of`, never a clock or the network. No model, ever. An ADR records the choice" [ETS §08]. For OPA the constraint means disabling `time.now_ns` and `http.send` [ETS §08]. **(Build recommendation)** Cedar or CEL is preferred over an own evaluator if either can be shown to satisfy C4 determinism replay under the kernel's projection, because an analysable, non-Turing-complete language answers the diligence question "can a rule loop or call out?" by construction; the ADR records the result.

A declared skip. E03 defines `policy_exempt: boolean` on a write type, which "skips rung 4", requires an ADR and is never valid on an irreversible write type [E03 §02 block 4]. That contradicts "there is no configuration that skips a rung" [E04 §02] and I5 [E02 §02]. **(Build recommendation)** A `policy_exempt` write type still passes through rung 4, which records `allow` with `policy_version` and a rule identifier `exempt:ADR-nnnn`, so the rung executes, the record names why no rule applied, and C3 continues to see every rung. Recorded in the conflicts block.

5.7 Rung 5 — GUARDRAIL

Purpose. Measure the blast radius of the resolved payload and compare it with the value declared for the write type: `one_record · one_counterparty · batch · tenant · cross_boundary` [E04 §03 rung 5]. The IM and O01 describe the rung as "allow-lists first, then the blast-radius check" [IM §4.3, O01 §02].

Inputs	The resolved payload; the write-type descriptor
Pass	Continue

Refusal	`POLICY.BLAST_RADIUS_EXCEEDED`
State written	The measured radius on the write row
Budget	8 ms p50 with rung 6 (target)
Model reach	None

Source: E04 §03 rung 5. "This is the rung that catches 'update one invoice' arriving as an unbounded filter" [E04 §03]. The caller sees what was attempted and what was declared.

(Build recommendation) The allow-list is the write type's `payload_schema` — "which shape fields this write type may set, and their bounds" [E03 §02 block 4] — applied to the resolved payload, so a field outside the schema or a value outside its bounds refuses here with `POLICY.BLAST_RADIUS_EXCEEDED` and a detail naming the field. The radius measure is computed from the resolved subject set, not from the request's own claim: the count of distinct target records, distinct counterparties, whether the set spans the tenant, and whether any target belongs to another organisation. The radii are ordered (`one_record` < `one_counterparty` < `batch` < `tenant` < `cross_boundary`), and the measured value must not exceed the declared one. Raising a declared radius to `tenant` or `cross_boundary` forces dual control through a declaration rule [E02 §06, IM §4.5].

5.8 Rung 6 — AUTONOMY

Purpose. Decide whether this actor may act alone on this write type now, from `autonomy_grant` [E04 §03 rung 6].

Level	What rung 6 permits
A0 Observe	Reads only; cannot reach rung 7 by any path
A1 Draft	Produces a write intent a human must submit
A2 Propose	Submits; a human approves before rung 7
A3 Act reversible	Acts alone where a reversal plan exists
A4 Act bounded	Acts alone inside a declared blast radius and budget
A5 Act broad	Acts alone at tenant scope; requires dual control on the declaration itself

Source: E04 §03 rung 6. "A missing grant resolves to A0. Not to tenant default, not inherited from the delegation chain" [E04 §03]. There is no A6 and no flag creates one [O01 §02].

Inputs	Actor, write type, `autonomy_grant`, the descriptor's reversibility and radius, rung 3's G4 result
Pass	Continue to rung 7
Escalate	An obligation row is created with the `write_id`, the level that triggered it, an expiry and the resolver set; the ladder stops
Refusals	`POLICY.AUTONOMY_INSUFFICIENT` where the grant is below what the write type needs and no approval route exists; `POLICY.APPROVAL_DECLINED` and `POLICY.OBLIGATION_EXPIRED` on the escalated write's lifecycle
State written	Obligation row, then the escalation-shaped proof entry at rung 8
Budget	Inside the 8 ms with rung 5 (target)
Model reach	None

Sources: E04 §03 rung 6, E02 §06. "Rung 7 is never reached for an escalated write" [E04 §03]. An irreversible write type "can never be auto-approved", and "an agent may never approve" [IM §4.3]. E02's version of the rung says an irreversible write type or an insufficient grant "escalates to approval rather than refusing" [E02 §05.2]; E04 keeps `POLICY.AUTONOMY_INSUFFICIENT` for the case with no approval route. The two are consistent if "approval route" means a non-empty resolver set in the declaration: with one, the write escalates; without one, it refuses.

(Build recommendation) The rung is a small decision table evaluated in this order: (1) grant resolved, missing → A0; (2) A0 → refuse `POLICY.AUTONOMY_INSUFFICIENT`; (3) A1 on an `act` call → refuse `POLICY.AUTONOMY_INSUFFICIENT` (an A1 actor stages, a human submits [E05 §04]); (4) escalate if any of: A2; write type irreversible; write type `dual_control` and G4 `pending`; rung 4 returned `escalate`; A3 and no reversal plan; A4 and measured radius or budget above the grant's bound; (5) escalate is only possible if the resolver set is non-empty, otherwise refuse `POLICY.AUTONOMY_INSUFFICIENT`; (6) else continue. On a resubmission carrying `obligation_ref`, an approved obligation satisfies step 4 for exactly the `write_type`, payload digest and declaration version it was granted against, so an approval cannot be replayed onto a different payload.

Obligation expiry. E04 sets a four-hour default, stated as "asserted, not derived from any observed approver behaviour" [E04 §04 Flow 3, OQ3]. R04 uses 72 hours and O03 keeps an unapproved action open for 24 hours [R04 §06, O03 §03]. Expiry is paused during a platform outage [E06 decision 6]. This memorandum treats expiry as a per-write-type declaration parameter with a four-hour kernel default, so rung1 can declare 72 hours without a kernel change; the difference is recorded in the conflicts block.

5.9 Rung 7 — ACT

Purpose. Execute. "Checks nothing by way of policy — by this point everything has been decided" [E04 §03 rung 7].

The rung does, in order [E04 §03 rung 7]:

1. take a snapshot;
2. apply to the per-tenant sandbox restored from that snapshot;
3. re-run the gates against the result;
4. apply the identical bytes to live through a single-use credential minted at that moment, scoped to one write, with a ten-minute life;
5. re-run the gates on live;
6. `BEGIN` · record version · enqueue exactly one outbox row · `COMMIT`.

In Phase 1 "the live apply is an ERPNext write through Frappe's own API (never the ORM), so hooks, validations and permissions run, and the adapter returns the inverse operation the 30-day rollback is built from" [E04 §03]. "Record-or-nothing is that shared transaction and nothing else" [E04 §03].

Inputs	The decided intent; the write-type descriptor's reversal plan; the adapter for the target
Output	`committed` with the record reference; `undo_until` 30 days from apply
Refusals	`GATE.SANDBOX_DIVERGED` (live re-run disagrees with the sandbox; snapshot restored); `PLATFORM.TARGET_UNREACHABLE`; `PLATFORM.TOKEN_MINT_FAILED`; `PLATFORM.WRITER_LOCK_LOST` (snapshot restored) — all platform codes page
State written	Record version and one outbox row in one transaction; the snapshot reference kept 30 days
Budget	18 ms p50 for the kernel's own transaction only; the four target operations sit outside it
Model reach	None; the bytes were fixed before the rung began

Sources: E04 §02–03, E02 §06.

The expensive decision. The 18 ms "is the kernel's own transaction only. The sandbox apply, the gate re-run, the live apply and the live re-run all sit outside it: for rung1's hosted ERPNext that is budgeted separately at ≤ 1,200 ms p95 [unverified] for the pair of applies, so the end-to-end governed write SLO is ≤ 2 s p95" [E04 §02]. "Sandbox-first is the most expensive decision in the Phase 1 design and it is the one that makes the 30-day rollback real" [E04 §02]. The target-system call has its own SLO [E04 decision 3].

Two sandboxes. The shadow sandbox here is a kernel mechanism inside every governed write, production included. A sandbox *key* is a developer credential for an isolated tenant. The two are never abbreviated to "sandbox" in consumer-facing text [E05 §05, decision 7].

Reversal. The descriptor's reversal plan names the method — `snapshot_restore`, `compensating_entry` or `target_native_undo` — its window and what happens after the window [E03 §02 block 4]. Conformance test C12 requires a committed write to be undone inside the window "by both the fast and the forward path, each producing its own governed record" [E02 §08]. A reversal is itself a governed write. Whether the 30-day window needs a per-write-type override is open, due November 2026 [E04 OQ4].

(Build recommendation) The adapter interface the kernel calls at rung 7 is narrow, and the adapter is kernel-owned code for each system of record, parameterised by the consumer's declaration rather than written by the consumer. That keeps the single writer inside the kernel (I2) while letting rung1 own "its ERP adapters" in the sense O03 means — the mapping from write type to Frappe API call — as declared data [O03 §10].

```
# elan1_core/act/adapter.py  (build recommendation)
class TargetAdapter(Protocol):
    def snapshot(self, tenant: TenantRef, subject: Subject) -> SnapshotRef: ...
    def apply(self, site: SiteRef, cred: SingleUseCredential,
              write: ResolvedWrite, idempotency_key: str) -> ApplyResult: ...
    def read_back(self, site: SiteRef, cred: SingleUseCredential,
                  ref: RecordRef) -> RecordState: ...
    def inverse(self, applied: ApplyResult) -> InverseOperation: ...
    def restore(self, snapshot: SnapshotRef) -> None: ...
```

5.10 Rung 8 — PROOF

Purpose. Chain the record. The rung consumes the outbox in order, assigns `seq` per tenant from 1, chains `prev_digest`, computes SHA-256 over the frozen sixteen-field preimage and appends to the evidence store [E04 §03 rung 8].

Inputs	Outbox row; the append-only write context; the tenant's chain head
Output	`seq` and digest, synchronously, on every outcome
Refusals	`PLATFORM.LOG_APPEND_FAILED` (retried in order, never skipped); `PLATFORM.CHAIN_HEAD_CONFLICT` (two writers for one tenant — P0, stops the tenant)
State written	One evidence row; the store grants INSERT to one principal and UPDATE to none
Budget	12 ms p50 (target)
Model reach	None

Source: E04 §03 rung 8, E02 §05.3. "Refusals seal too — with `record_id` at the zero UUID and `record_version` 0, so a refusal and a commit are structurally identical and semantically distinct" [E04 §03].

> **The L1 claim.** "In L1 this record is tamper-evident to us and to the tenant: anyone holding the chain can detect an edit. It is not yet tamper-evident to a stranger who distrusts us, because nothing is signed. That claim arrives at L2. Do not let it into a Phase 1 deck" [E04 §03 rung 8].

The budget conflict. E04 budgets the proof rung at 12 ms within a ≈64 ms kernel; the pricing model's cost side assumes about 91 ms of which about 40 ms is the proof rung; O03 targets a proof-record write at p50 ≤ 40 ms. "Both are modelled, neither measured, and a diligence engineer should treat latency as an open item" [IM §6.10, O03 §08, ETS §04]. At L1 there is no signature, so the 12 ms figure (one hash and one insert) is the one that applies to the L1 build; the 40 ms figure is plausible only once L2 signing through key custody is in the path.

When the append fails after a commit. E02 says the rung-7 transaction has committed, the outbox row survives and the chain writer retries in order, so there is "no lost write, no gap in the sequence" [E02 §05.2]. E04 says rung 8 is "synchronous to the caller" and the caller sees `seq` and digest on every outcome [E04 §03]. O03 marks such an action `acted_unproven`, which "alarms" and "is never silently completed", and R04 holds it in `SEAL_PENDING` and treats a promoted write without a proof record as a P1 that pages after five minutes [O03 §03, R04 §07]. **(Build recommendation)** `assert` waits for the chain

append up to its deadline. If the deadline passes, it returns HTTP 200 with `outcome: committed`, the `write_id`, `record_id`, `undo_until`, and `record: { "status": "pending" }` rather than a `seq`. It never returns a platform refusal code, because the write happened and a platform code would say it had not. The `write.committed` webhook carries `seq` and digest once the append lands [E05 §09]. The pending state pages at five minutes, following R04. Where an `acted_unproven` action finally goes — retry queue or manual reconciliation — is open [O03 OQ3].

5.11 Rung 9 — METERED

Purpose. Emit exactly one meter event per governed write, with the basis taken from the declared blast radius [E04 §03 rung 9].

Inputs	Outcome; write-type descriptor; `seq`
Output	Nothing visible to the caller
State written	One `meter_event`; quantity 1 for a commit, 0 for every refusal and escalation
Budget	2 ms p50 (target)
Model reach	None

Source: E04 §03 rung 9, E02 §03. "Refusals meter at zero, enforced by a database constraint — a refusal row with a non-zero quantity cannot be inserted" [E04 §03]. The mapping from radius to basis is normative, "not configurable" [E05 §06]. Platform-class refusals "are not billed" [E04 Flow 1]. **(Build recommendation)** Governed writes that the platform raises on its own behalf — obligation resolutions, declaration registrations, tenant stops, reversals — are declared in an elan1 platform declaration whose write types carry `billable: false`, so they meter at quantity 0 with outcome `committed`. Without that rule an approval would bill the customer twice for one business decision. Whether a refusal caused by a rung1 defect rather than tenant policy should be charged to anyone is open [R04 OQ2].

5.12 The rungs in one table

Rung	State written	Codes it can raise	Budget p50 (target)	A model can…
0b STOP	Stop event (no chain entry)	`POLICY.TENANT_STOPPED`	4 ms (0b–2)	…not reach it
1 IDENTITY	Write row begins; `actor_kind`	`POLICY.ACTOR_NOT_PERMITTED`, `POLICY.DELEGATION_INVALID`		…not set `actor_kind`
2 WRITE DOOR	Resolved versions	`POLICY.WRITE_TYPE_UNKNOWN`, `POLICY.CHANNEL_NOT_PERMITTED`, `POLICY.DECLARATION_UNRESOLVED`, `PLATFORM.REGISTRY_UNAVAILABLE`		…choose a declared write type, nothing more
3 PRE-GATES	One `gate_result` per declared gate	Twelve `GATE.*` codes	8 ms	…not influence a gate
4 `evaluate()`	Decision + `policy_version`	`POLICY.DENIED_BY_RULE`, `POLICY.NO_RULE_MATCHED`, `PLATFORM.POLICY_UNAVAILABLE`	12 ms	…not be called
5 GUARDRAIL	Measured radius	`POLICY.BLAST_RADIUS_EXCEEDED`	8 ms (5–6)	…not widen the radius
6 AUTONOMY	Obligation on escalate	`POLICY.AUTONOMY_INSUFFICIENT`, `POLICY.APPROVAL_DECLINED`, `POLICY.OBLIGATION_EXPIRED`		…not approve
7 ACT	Record version + outbox row; snapshot kept 30 days	`GATE.SANDBOX_DIVERGED`, `PLATFORM.TARGET_UNREACHABLE`, `PLATFORM.TOKEN_MINT_FAILED`, `PLATFORM.WRITER_LOCK_LOST`	18 ms + target SLO	…not hold the credential

Rung	State written	Codes it can raise	Budget p50 (target)	A model can...
8 PROOF	Evidence row	`PLATFORM.LOG_APPEND_FAILED`, `PLATFORM.CHAIN_HEAD_CONFLICT`	12 ms	...not reach it
9 METERED	Meter event	none	2 ms	...not reach it

Sources: E04 §02–03, E02 §06; the model column follows E02 I1 and E05 §01 ("no interface in this document can set `actor_kind`, `seq`, `policy_version` or the proof timestamp").

5.13 One governed action, end to end — an agent posts a journal entry under ledger1

This walkthrough follows one month-end accrual from an agent's proposal to an invoice line. It uses the ledger1 pack's `journal_entry.submit` write type, because it exercises every mechanism at once: an agent actor, a `batch` blast radius, dual control, a compensating reversal, the budget gate and a statutory purpose. The sources walk a payment and a sales invoice [E04 §04, IM §4.4]; the journal-entry values below are illustrative and marked where they are this memorandum's.

5.13.1 Setting

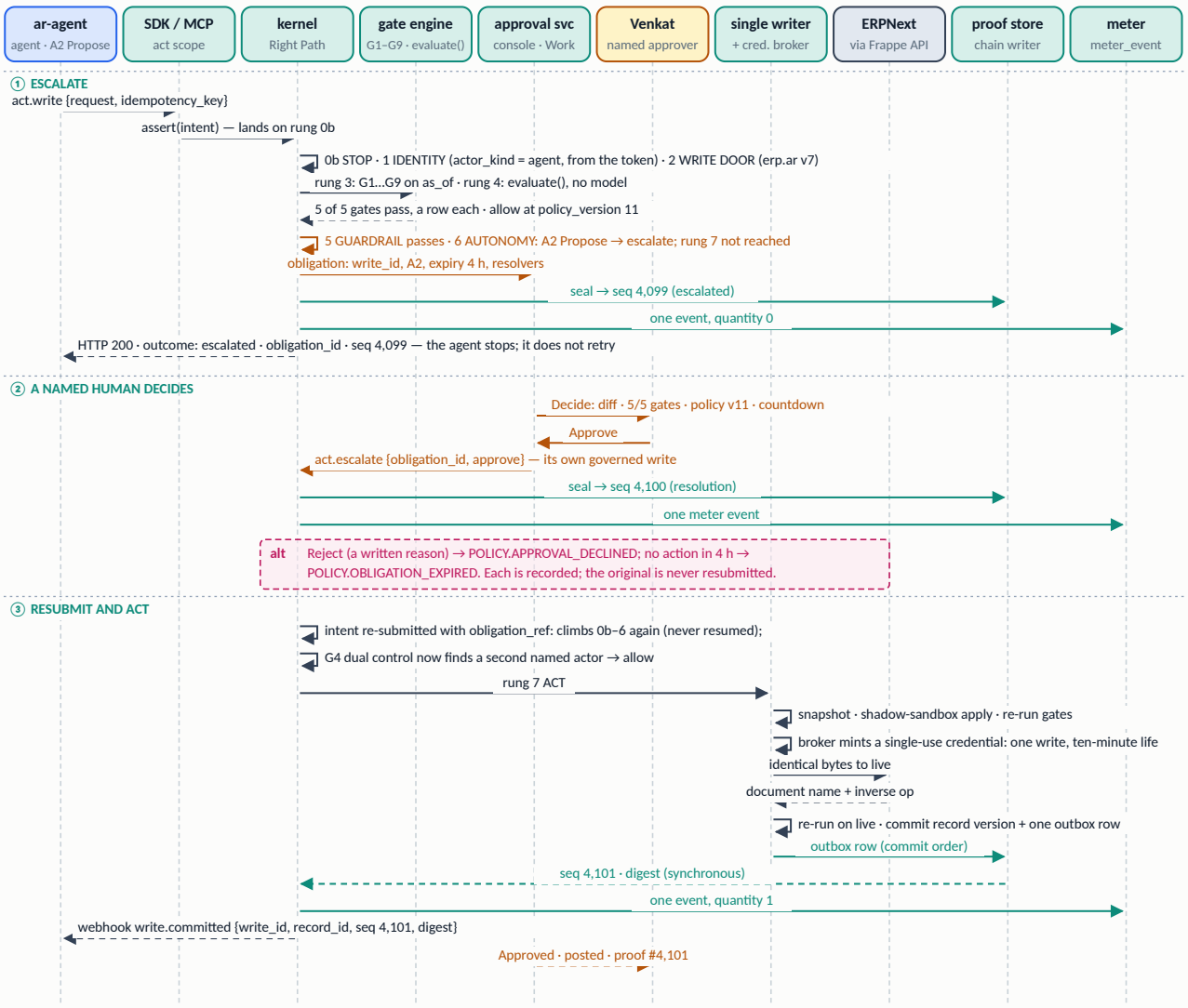
Element	Value	Source
Tenant	A mid-sized manufacturer on rung1 Enterprise, hosted ERPNext, residency Mumbai; enforced mode through credential custody	E04 Flow 1; O03 §02
Declaration	rung1's ERPNext declaration at an explicit version (here v12, illustrative); ledger1 references it and adds no new ontology	E18 §06
Write type	`journal_entry.submit` — blast radius `batch`, dual control always, reversal by compensating entry	E18 §06, E03 §02
Gates, in order	consent, residency, retention, counterparty, precondition, dual_control, budget	E18 §06
Pack profile	`in-mca-audit-trail` v1, `min_assurance_level: 2`; at L1 a dry run reports "not applicable — assurance level 1" on the face of the report	E18 §06, IM §6.7
Actor	`gl-agent`, `actor_kind: agent`, grant A2 Propose on `journal_entry.submit`, `on_behalf_of` the accounts executive who runs month-end	E04 §03 rung 6 (levels); illustrative
Resolver set	Role `gl-approvers`: the finance controller and the CFO, both human	E06 §03; illustrative
Policy	Rule set at `policy_version` 14 (illustrative)	—

Figure 7 shows the sequence across components.

Figure 7 — One governed action, as a sequence

L1 — DESIGNED, NOT YET BUILT

E04 Flow 3: an agent at A2 proposes a ₹2,40,000 sales invoice, a named human approves, the single writer commits



Three proof entries for one approved decision (a fourth, the lapse, on expiry): the kernel keeps no suspended-and-resumed state.

Source: E04 §03–04 (Flow 3); E02 §03, §05; E05 §04, §08–09; E06 §02–03; IM §4.4 · elan1 Technology Memorandum · Sept 2026

Figure 7 — One governed action end to end: the agent stages and submits through the SDK, the kernel escalates at rung 6, a human approves on the Decide screen (itself a governed write), the resubmission climbs the rungs again, the single writer applies to the shadow sandbox and then to live, and the proof and meter rungs close the action; three chain entries and three meter events result.

5.13.2 Step 1 — the agent composes and stages the intent

rung1's month-end workflow asks `gl-agent` to book an accrual for unbilled freight: debit Freight Inward, credit Accrued Expenses, ₹6,40,000 across three lines, two of which carry a transporter as party (illustrative). The model's contribution ends with the intent. rung1's workflow calls `scope` for `journal_entry.submit` and learns `reaches: approval`, so the console shows "needs approval" before anything is submitted [E02 §05.5]. It then calls `stage.write`, which returns a validated request object "with no side effects at all" [E05 §04].

```
{ "intent_id": "5c8e1f0a-7b1d-5d8e-9c2a-2f8f6e1a0c41",
  "tenant_id": "3a9e…", "declaration_id": "d-rung1-erpnext", "declaration_version": 12,
  "write_type": "journal_entry.submit", "actor_id": "gl-agent", "actor_kind": "agent",
  "on_behalf_of": "u-accounts-exec", "channel": "console",
  "subject": null,
  "payload": { "voucher_type": "Journal Entry", "posting_date": "2027-09-30",
               "accounts": [
```

```
                { "account": "Freight Inward - MFG", "debit": 640000 },
                { "account": "Accrued Expenses - MFG", "credit": 420000,
                  "party_type": "Supplier", "party": "Sai Logistics" },
                { "account": "Accrued Expenses - MFG", "credit": 220000,
                  "party_type": "Supplier", "party": "Konkan Carriers" } ],
              "user_remark": "Accrual for September freight, invoices pending" },
  "as_of": "2027-09-30T18:00:00+05:30",
  "reason": "Month-end accrual proposed by gl-agent" }
```

The `intent_id` is a name-based UUID of rung1's idempotency key, `sha256(changeset_id ‖ base_schema_hash)` [R04 §04]; the mapping is a build recommendation (Chapter 4 §4.3.1). `as_of` is supplied, not read from a clock [E02 §03].

5.13.3 Step 2 — first submission escalates at rung 6

rung1 calls `assert` through the SDK [O03 §03 step 4].

Rung	What happens	Row written
0b	Tenant live; no stop	—
1	`actor_kind: agent` taken from the token; delegation to the accounts executive is live	Write row W1 begins
2	`journal_entry.submit` resolves at v12; `console` carries it	W1: `declaration_id`, v12
3	consent, residency (Mumbai), retention, counterparty (both transporters registered and not sanctioned), precondition (period open; accounts exist), budget (inside the monthly journal budget) pass; dual_control has no obligation yet and records `pending` (build recommendation, §5.5)	Seven `gate_result` rows
4	`evaluate()` returns `allow` at `policy_version` 14	Decision
5	Measured radius: one journal voucher, three lines, two counterparties, inside `batch`	Measured radius
6	A2 Propose, dual control required, irreversible? no — escalate; resolver set `gl-approvers` non-empty; obligation O1 created, expiry per declaration	Obligation O1
8	Escalation-shaped record, `record_id` zero, `seq` 7,301 (illustrative)	Evidence row
9	Meter event, outcome `escalated`, quantity 0, basis `batch`	Meter row

"Rung 7 is not reached" [E04 Flow 3]. The agent receives HTTP 200 with `outcome: escalated`, `obligation_id` and `seq` 7,301. "It stops; it does not retry" [E04 Flow 3]. The kernel emits `write.escalated` with `write_id`, `obligation_id`, `autonomy_level` and expiry [E05 §09].

5.13.4 Step 3 — a human decides, and the decision is itself a governed write

The finance controller sees the item at the top of the Approvals queue in the Work door with a countdown, "Agent" stated in words, the amount and the radius, and "6 of 7 checks passed · dual control: awaiting you" [E06 §02]. On the Decide screen she sees the field-by-field diff with nothing summarised away, every declared gate's result with the declaration version it ran against, the decision "allow at policy version 14", and the reason it reached a person: "the proposing agent is at A2 Propose; a person approves before anything is written" [E06 §03]. She presses Approve.

The console submits `act.escalate` with `{obligation_id, decision: "approve", note}` under her own tenant token [E05 §04]. "Resolving an obligation is itself a governed write, so it climbs the ladder and appends its own record" [E06 §03]: it runs rungs 0b–9 as write W2 of a platform write type, is checked by G4 for approver ≠ proposer and approver kind human [E02 §03, §08 C11], commits the approval object with its `basis` — the surface plan version and the rendered payload digest she was shown — and seals at `seq` 7,302, metered at zero as a platform write type (build recommendation, §5.11). The kernel emits `obligation.resolved` with the outcome and her `actor_id` [E05 §09].

5.13.5 Step 4 — resubmission, and single-writer execution

rung1's workflow receives `obligation.resolved` and resubmits: a new `intent_id`, identical payload, `obligation_ref: O1` [E04 Flow 3, E02 §03]. The ladder runs again from 0b; nothing is resumed [E04 decision 4]. Rungs 0b–2 pass as before. At rung 3 G4 now finds an approved obligation for this write type, this payload digest and v12, by a named human who is not

the actor, and passes; the other six gates re-run against the same `as_of` and pass. Rung 4 allows at `policy_version` 14. Rung 6 finds A2 with a satisfied obligation and continues.

Rung 7 then executes [E04 §03 rung 7]:

1. **Snapshot.** The adapter records a snapshot reference covering the affected accounts' ledger state.
2. **Shadow sandbox apply.** The per-tenant sandbox site is restored from the snapshot and the Journal Entry is inserted and submitted through Frappe's API, so ERPNext's own validations run (debits equal credits, period open, account permissions).
3. **Gate re-run on the sandbox result.** The kernel re-runs the declared gates against the resulting record state; rung1's ledger-integrity checks run as the precondition's detail [R04 §03].
4. **Mint and apply to live.** The broker mints a single-use credential scoped to this one write with a ten-minute life; the adapter posts the identical bytes to the live site with the write's idempotency key [E04 §03, S04 §05].
5. **Gate re-run on live.** The live result is compared with the sandbox result.
6. **Commit.** `BEGIN` · record version (the voucher name, say `ACC-JV-2027-00412`, and its version) · one outbox row · `COMMIT`. The adapter returns the inverse operation: a compensating Journal Entry with debits and credits swapped [E03 §02 block 4].

Rung 8 appends the committed record at `seq` 7,303 with `prev_digest` equal to the digest of 7,302; rung 9 emits one meter event, outcome `committed`, basis `batch`, quantity 1. The response carries the decision, `seq` 7,303 and digest, `record_id`, `record_version` and `undo_until` thirty days out [E02 §05.2]. `write.committed` fires with `write_id`, `record_id`, `seq` and digest [E05 §09]. The controller sees "Approved · posted · proof #7,303", with Open the record and Reverse this (30 days) [E06 §03].

5.13.6 What was recorded

Row	Count	Detail
Write rows	3	W1 escalated, W2 approval, W3 committed
Gate results	7 + 7 + those of W2's platform write type	Every declared gate, both submissions, pass or fail
Obligation	1	O1, resolved `approve`
Approval object	1	With `basis`: surface plan version and rendered payload digest
Record version and outbox row	1 each	W3 only
Snapshot reference	1	Kept 30 days
Proof records	3	`seq` 7,301 (escalation), 7,302 (resolution), 7,303 (commit)
Meter events	3	0 (escalated), 0 (platform approval), 1 (`batch`)

E04 counts "four proof entries for one business decision: the escalation, the resolution, the resubmission, and (on expiry instead) the lapse" [E04 Flow 3]; on the approve path there are three. "One extra row is the price of having no suspended-and-resumed state inside the kernel, and we take that trade every time" [E04 Flow 3]. The invoice carries one billed line for this decision; the two zero-quantity events appear on the same table [E09 §07].

5.13.7 The other endings

Ending	Where	What the actors see	Records
Controller rejects (with a sentence of reason)	W2 resolves `decline`; the original write is never resubmitted	Agent told `POLICY.APPROVAL_DECLINED`	Escalation, resolution; no commit [E04 Flow 3, E06 §03]
Nobody acts before expiry	Kernel raises `POLICY.OBLIGATION_EXPIRED` as a governed event	Agent receives `obligation.expired`	Escalation, lapse [E04 Flow 3, E05 §09]

Ending	Where	What the actors see	Records
The approver is the executive on whose behalf the agent acted	W2 refuses at G4, which treats `on_behalf_of` as a proposer (build recommendation)	"You proposed this, so you cannot also approve it. Send it to: [names]"	Escalation, refused resolution [E06 §03]
Live re-run disagrees with the sandbox	Rung 7; snapshot restored	"Your approval was recorded. The write itself did not complete, and nothing was changed. We have been paged."	`GATE.SANDBOX_DIVERGED` refusal record [E06 §03, E04 §03]
ERPNext unreachable after the sandbox apply	Rung 7; nothing reaches live; sandbox discarded	"We could not complete this. Nothing was changed."	`PLATFORM.TARGET_UNREACHABLE`; retry is safe with the same `intent_id` [E02 §05.2]
Period closed between approval and resubmission	Rung 3 precondition fails on resubmission	"Not done — period closed" with the operator action	Refusal record; the approval stands but binds nothing further

The last row shows why the approval binds a payload digest and a declaration version rather than a `write_id` alone: an approval granted against one state of the books is not an approval of whatever the resubmission finds. The resubmission always faces the gates afresh.

5.14 Idempotency and retries

The kernel's rules are Chapter 4's; this section states how they compose across a governed action.

1. **One key per intent.** Every `act` call carries a client-supplied key scoped to the tenant and retained 30 days; a repeat returns the original outcome with the original `seq` and digest, "not a second write" [E05 §08]. "A duplicate write is a duplicate proof entry, and a chain with two entries for one business event is a chain the customer has to explain" [E05 §08].
2. **Retry transport failures, never refusals.** Retry 5xx and 429 with exponential backoff and jitter, at most five attempts, always with the same key; "never retry a refusal. A retried refusal is an attempt to get a different answer, and it is visible to us as exactly that" [E05 §08]. R04 has rung1 retry three times with jitter inside a 20-second budget before parking [R04 §07]; both are within E05's ceiling.
3. **The target honours a key too.** "Every write activity against a system of record carries an idempotency key the target honours" [S04 decision 6]. Whether ERPNext honours one on document submission natively, or rung1's adapter must implement the dedupe, is open, due 15 October 2026 [S04 OQ5]. **(Build recommendation)** The kernel adapter writes the `write_id` into a custom field on the target document and, before any live apply, queries for an existing document with that value; a hit returns the existing document instead of posting again. This closes S04's "second PINV" failure mode — the response lost after ERPNext committed — regardless of the answer to S04 OQ5 [S04 §02].
4. **Resubmission is not retry.** After an approval, or after fixing the cause of a governed refusal, the consumer submits a new intent (Chapter 4 §4.5.3). After a platform-class refusal, it retries the same intent.
5. **Webhooks are at least once.** Consumers must be idempotent on `write_id`, and can detect a missed delivery by watching `seq` contiguity per tenant [E05 §09].

5.15 Dependency failure — fail closed

"When a dependency is down the system fails closed, and the codes say so" [IM §6.10].

Dependency down	Rung	Outcome	Nothing reached live?	Pages?
Declaration registry	2	`PLATFORM.REGISTRY_UNAVAILABLE`	Yes	Yes
Rule engine	4	`PLATFORM.POLICY_UNAVAILABLE`; "never fall back to a default allow"	Yes	Yes
Autonomy table	6	Refuse (C5 requires it); E02 names no code	Yes	Yes

Dependency down	Rung	Outcome	Nothing reached live?	Pages?
Credential broker	7	`PLATFORM.TOKEN_MINT_FAILED`	Yes	Yes
Target system	7	`PLATFORM.TARGET_UNREACHABLE`; sandbox discarded	Yes	Yes
Writer lock	7	`PLATFORM.WRITER_LOCK_LOST`; snapshot restored	Restored	Yes
Evidence store	8	`PLATFORM.LOG_APPEND_FAILED`; outbox retries in order	Write committed; proof pending	Yes
Key custody (L2 only)	8	Domain stops signing entirely	—	Yes

Sources: E02 §05.2, §06, §08 C5; IM §6.10. At L2, "a platform that stops sealing is recoverable; a platform that seals badly is not" [IM §6.10]. E02's fail-closed matrix names the autonomy table as a dependency C5 takes down, but the register has no platform code for it; **(build recommendation)** register `PLATFORM.AUTONOMY_UNAVAILABLE` at rung 6 through the §4.6.4 procedure rather than overloading `POLICY.AUTONOMY_INSUFFICIENT`, which would report our failure as the customer's policy.

The kernel itself unreachable. The consumer must fail closed.

- **Mode A (rung1).** "rung1 fails closed. There is no degraded mode, no cache of yesterday's allow, no local gate that stands in" [R04 §07]. After three jittered retries inside 20 seconds the workflow parks in `BLOCKED_KERNEL_UNREACHABLE`; the user sees "We cannot apply changes right now. Your site has not been touched."; when the layer returns, the workflow "resumes at the same `assert` with the same `idempotency_key`… Nothing is re-planned and nothing is re-charged" [R04 §07]. Because the credential sits in the kernel's broker, the consumer could not write if it tried [O03 §02].
- **Mode B (third party, advisory).** "The caller must fail closed by its own contract, and we cannot make it. Timeout on `assert`: treat as refuse" [O03 §04]. The write does not happen "if the caller honours the refusal; nothing physically stops it otherwise" [O03 §02]. Chapter 6 covers what may be claimed.

Obligation expiry is paused during a platform outage, so an approver does not lose an item to downtime [E06 decision 6].

5.16 Concurrency and ordering

One writer per tenant. I2 says exactly one process mutates the target for a tenant [E02 I2]. Two chain writers for one tenant is `PLATFORM.CHAIN_HEAD_CONFLICT`, "a P0 that stops the tenant" [E04 §03 rung 8].

Commit order, not arrival order. "Ordering within a tenant is established by the outbox, not by request arrival. Two concurrent writes get `seq` values in commit order, which is the order that actually happened" [E05 §08]. Across tenants there is no ordering guarantee [E02 §05.2].

(Build recommendation) The mechanics:

- **Decision in parallel, action in series.** Rungs 0b–6 are pure and run concurrently for any number of intents. Rung 7 takes a per-tenant writer lease (a row lock or advisory lock keyed on `tenant_id`, with a fencing token), so live applies for a tenant are serialised. Loss of the lease mid-apply is `PLATFORM.WRITER_LOCK_LOST`, and the fencing token prevents a stale holder from committing its outbox row.
- **One chain head.** Rung 8 assigns `seq` inside a transaction that locks the tenant's chain-head row (`SELECT … FOR UPDATE`), reads `prev_digest`, inserts the record and advances the head. A unique constraint on `(tenant_id, seq)` makes a second writer's insert fail, which is then raised as `PLATFORM.CHAIN_HEAD_CONFLICT`.
- **Two writes to one record.** The gates evaluated at rungs 3–6 used a `record_state` that may be stale by the time rung 7 runs. The live apply uses the target's optimistic concurrency check (Frappe compares the document's `modified` timestamp), and the gate re-run on live catches a changed state. Either failure restores the snapshot and refuses; the write is resubmitted with a fresh `as_of` and faces the gates again.

- **Refusals do not wait for the writer.** Refusals and escalations do not take the writer lease; they queue only for the chain head. A tenant under a burst of refusals therefore does not delay its committed writes by more than the chain append.

5.17 Durable execution on stack1's workflow engine

DR-002 is closed: "Temporal owns anything touching a customer's books. Every governed write runs inside a Temporal workflow", because "replay must not re-run side effects" [O01 §05 DR-002]. S04 binds Temporal as the primary implementation of `fusion.run/v1`, with Prefect and Dagster as licence-distant substitutes, and n8n and Inngest excluded [S04 §01, decisions 1–5]. The safety property is Temporal's workflow/activity split: "Replay reconstructs a decision, not an action" [S04 §02].

Two statements in the corpus pull against each other. S04 requires that "the approval must be durable state inside the run, not a callback… a Temporal signal is part of the run's history" [S04 §02], and stack1's runtime note has the approval arrive in the workflow "as a durable Temporal signal" [S03 §04 step 8]. E04 decides that "an escalated write is re-submitted, never resumed" and that the kernel has "no suspended-and-resumed state" [E04 decision 4, Flow 3]. Both hold if the durable wait lives in the **consumer's** workflow and never in the kernel.

(Build recommendation) Two workflows, owned by two teams:

1. **The consumer's workflow** (rung1's, in rung1's Temporal namespace) composes the intent, calls `assert` as an activity, and on `escalated` waits on a signal raised from the `obligation.resolved` or `obligation.expired` webhook. On approval it calls `assert` again as a new activity with `obligation_ref`. The wait is durable and replay-safe, and the kernel holds no suspended state.
2. **The kernel's writer workflow** (elan1's, one per tenant, in an elan1-owned namespace and task queue) executes rung 7. It is started by `assert` with the `write_id` as the workflow ID, so Temporal's workflow-ID uniqueness is a second idempotency barrier. Its activities are snapshot, sandbox apply, sandbox re-run, credential mint, live apply, live re-run and outbox commit, each idempotent on `write_id`. Only this worker's workload identity can obtain a credential from the broker (Chapter 6 §6.3). One writer workflow per tenant, processing writes in order, is the single writer of I2 made concrete.

```
# rung1 side — consumer workflow (build recommendation; Temporal Python SDK shape)
@workflow.defn
class PostJournalEntry:
    def __init__(self) -> None:
        self.resolution: str | None = None

    @workflow.signal
    def obligation_resolved(self, outcome: str) -> None:   # raised from the webhook
        self.resolution = outcome

    @workflow.run
    async def run(self, req: JournalRequest) -> str:
        intent = await workflow.execute_activity(stage_intent, req, start_to_close_timeout=T5)
        res = await workflow.execute_activity(kernel_assert, intent,
                                              start_to_close_timeout=T5, retry_policy=R_5XX)
        if res.outcome == "escalated":
            await workflow.wait_condition(lambda: self.resolution is not None)
            if self.resolution != "approve":
                return "not_posted"                        # declined or expired
            again = intent.resubmission(obligation_ref=res.obligation_id)  # new intent_id
            res = await workflow.execute_activity(kernel_assert, again,
                                                  start_to_close_timeout=T5,
                                                  retry_policy=R_5XX)
        return res.outcome                                 # committed | refused
```

The `assert` activity timeout follows R04's 5 seconds [R04 §07]; `R_5XX` retries only transport failures, never a refusal [E05 §08]. The consumer workflow never touches the target and holds no credential, which is what keeps rung1's integration Enforced.

This arrangement also resolves a difference in who executes the live write. R04 has rung1's promoter mint the ten-minute token and apply the change, then call `seal` [R04 §03 steps 7–8]. S03 has rung1's workflow call the ERPNext MCP tool through `fusion.tools/v1` with a sealed credential reference [S03 §04 step 9]. E02, E04 and O03 Mode A put rung 7 in the kernel, using a credential only the kernel holds [E02 I2, E04 rung 7, O03 §03 step 6]. The memorandum follows E02: rung1 plans, tests and proposes; the kernel's writer workflow applies. The rung1 promoter's sandbox and golden-gate work becomes the input to the kernel's rung 7 (the resolved payload and the precondition detail), not a second write path. The difference is recorded in the conflicts block because it moves code between teams.

5.18 Latency — what is targeted and what is measured

Nothing on this path has been measured [IM §6.10].

Segment	Target	Source	Status
Kernel, rungs 0b–9	≈ 64 ms p50	E04 §02	Design target; Gate 1 to measure
Kernel, alternative model	≈ 91 ms, proof ≈ 40 ms	Pricing model via IM §6.10; ETS §04	Modelled; unreconciled with E04
Sandbox + live applies (hosted ERPNext)	≤ 1,200 ms p95	E04 §02	Unverified
Governed write end to end (with target)	≤ 2 s p95	E04 §02	"MEASURE THIS"
Governed write end to end (group contract)	p50 ≤ 120 ms, p99 ≤ 450 ms	O03 §08	"MEASURE THIS"; not in any customer document until measured
Mode A added latency	≤ 15 ms over the envelope	O03 §02	Target
Mode B contract call	≤ 40 ms p95 excluding the target	O03 §02, §08	Target
Meter lag	p99 ≤ 30 s	O03 §07	Target

O03's ≤ 120 ms p50 cannot be an end-to-end figure that includes a sandbox apply and a live apply budgeted at up to 1,200 ms; it is read as the kernel-side governed-write figure ("Kernel-side, per write type" is how O03 says it is measured) [O03 §08]. The two E04 open questions — measured p50 for rungs 0b–9 on the reference tenant, and measured p95 for the ERPNext adapter on hosted Frappe at Mumbai — are both due at Gate 1 [E04 OQ1–2]. Whether the shadow sandbox's second gate run changes the end-to-end SLO enough that third parties need a different latency promise from rung1's is open, due December 2026 [E05 OQ6]. ETS says the kernel language (Python 3.12 with FastAPI) is revisited "only if Gate 1 measurements miss the ≤ 120 ms p50 SLO" [ETS §04].

5.19 Decisions and open questions

E04 records seven decisions, all dated 21 September 2026: rung 8 at L1 is a hash chain with no signature field over a frozen preimage (1); the autonomy ladder is A0–A5 (2); the target-system call sits outside the kernel budget with its own SLO (3); an escalated write is re-submitted, never resumed (4); a stopped tenant produces no chain entry (5); rung 3 runs every declared gate (6); rung 7 is sandbox-first (7) [E04 §05].

#	Open question	Owner, by	Source
1	Measured p50 for rungs 0b–9 on the reference tenant	Kernel engineer, Gate 1	E04 OQ1
2	Measured p95 for the ERPNext adapter on hosted Frappe at Mumbai	ERPNext engineer, Gate 1	E04 OQ2
3	Default obligation expiry (4 hours asserted, not derived)	elan1 lead, first design partner	E04 OQ3, E06 OQ3
4	Per-write-type override of the 30-day rollback window	elan1 lead, Nov 2026	E04 OQ4
5	Which of the 28 active codes are reachable with only ERPNext declared	Kernel engineer, Oct 2026	E04 OQ5
6	Does ERPNext honour an idempotency key on submission natively?	rung1 lead, 15 Oct 2026	S04 OQ5

#	Open question	Owner, by	Source
7	Where an `acted_unproven` action goes	elan1 kernel lead, 15 Oct 2026	O03 OQ3
8	Does running every gate leak the gate set?	elan1 lead, 30 Nov 2026	E02 OQ4

This chapter adds three for the elan1 lead: the `pending` outcome for G4 on a first submission (§5.5); metering of platform write types such as approvals (§5.11); and a platform code for an unavailable autonomy table or stop set (§5.2, §5.15).

6. Surfaces and enforcement — the SDK, the MCP server and the two modes

"There is one submission surface, two ways to reach it, and only one of the two modes lets you say the write could not have happened without us" [E05, in one line]. This chapter specifies the ways a consumer reaches KernelPort — the in-process SDK, the MCP server and the HTTP API — and the two enforcement modes that determine what the consumer may afterwards claim. It then describes how enforcement is made physical rather than contractual, the rule that advisory is never sold as enforcement, and the path a third-party developer walks from a sandbox key to a production key.

> **Status.** Nothing in this chapter is generally available. Phase 1 is Assurance Level 1 Guarded; third-party general availability is Phase 2 (Q1–Q2 2027), and in Phase 1 "the only external consumers are design partners under an explicit pre-GA agreement" [E05 §01]. In Phase 1 the Enforced claim is true for rung1 Enterprise in hosted mode "and nothing else" [O03 §02]. The latency figures are targets marked "MEASURE THIS" [O03 §08]. Sketches the sources do not give are marked **(build recommendation)**.

6.1 One submission surface, two ways to reach it

Two kinds of consumer reach the kernel. In-process consumers link the kernel library behind KernelPort; out-of-process consumers reach it over MCP or HTTP. "Both land on rung 0b. There is no second submission surface, and no interface in this document can set `actor_kind`, `seq`, `policy_version` or the proof timestamp" [E05 §01, IM §6.9].

Two ideas are easily run together and must be kept apart:

- **The transport** — how the call arrives: Mode A (SDK in-process) or Mode B (MCP or HTTP) [O03 §02].
- **The enforcement class** — who holds the credential to the target: Enforced (the kernel, or a target that accepts only kernel-issued tokens) or Attested advisory (the caller) [E05 §02].

In Phase 1 the two line up — Mode A is Enforced and Mode B is Attested advisory by default — but they are not the same axis. O03 allows Mode B to be Enforced "only where the target accepts kernel-issued tokens" [O03 §02], and E03's worked declaration shows rung1's own MCP server as an `mcp` channel with `mode: enforced`, because the write it submits is still executed by the kernel's single writer [E03 §04 channels]. The claim follows the credential, not the wire.

6.2 The two modes

	Mode A — SDK in-process	Mode B — MCP / HTTP
Phase 1 user	rung1 Enterprise (hosted)	stack1 tooling; third parties from Phase 2
How the kernel is reached	Library call to KernelPort inside the caller's process	MCP tool call or HTTPS to the kernel service
Holder of the target credential	The kernel, via the per-tenant credential broker	The caller, unless the target accepts only kernel-issued tokens
Enforcement class	Enforced	Attested advisory by default; Enforced only with token acceptance
What rung 7 does	Performs the write	Nothing; the consumer acts
What a refusal means	"The write did not happen"	"elan1 evaluated this and said no"
Chain contains	The decision and the act	The decision; the act only as reported, flagged caller-performed
Rollback	Kernel's reversal plan, 30 days	Consumer's problem
Added latency (target)	≤ 15 ms over the governed-write envelope	≤ 40 ms p95 for the contract call, excluding the target's own work

	Mode A — SDK in-process	Mode B — MCP / HTTP
If the kernel fails	The write does not happen	The write does not happen *if the caller honours the refusal*; nothing physically stops it otherwise
Truthful claim	"This write could not have occurred without passing the gates."	"Every proposed write was evaluated against a versioned policy and the decision is recorded."

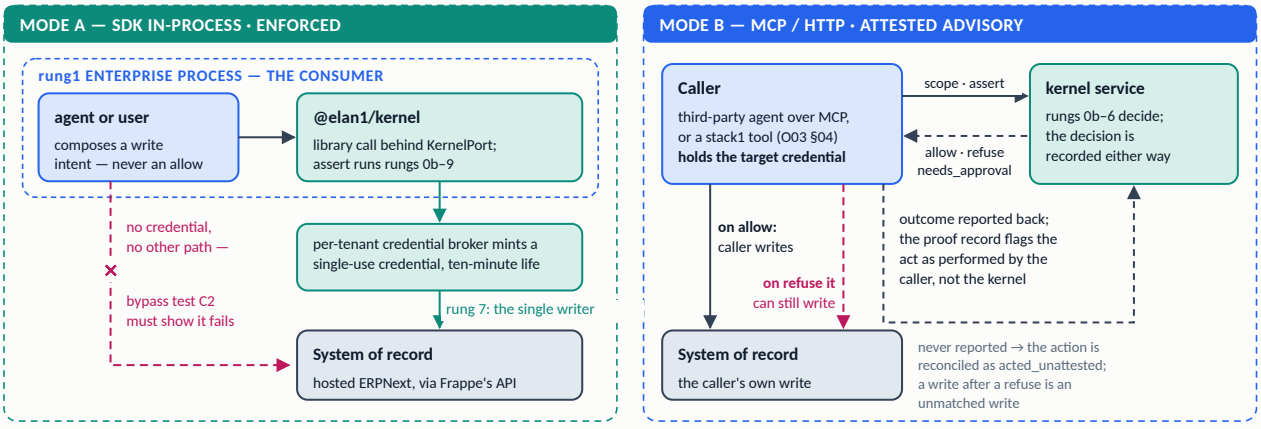
Sources: O03 §02, E05 §02, IM §4.6.

Figure 8 sets the two side by side.

Figure 8 — Two modes, two claims

L1 — BEING BUILT · LATENCIES ARE TARGETS

Mode A: the kernel holds the credential and performs the write. Mode B: the caller asks, then writes on its own



	MODE A — ENFORCED	MODE B — ATTESTED ADVISORY
Target credential	the kernel, in the per-tenant credential broker; no long-lived key exists where a consumer, agent or operator can reach it	the caller or its agent — Enforced only where the target accepts kernel-issued tokens (open for ERPNext, O03 OQ1)
Rung 7	performs the write: snapshot, sandbox, identical bytes	does nothing; the caller acts
A refusal means	"the write did not happen"	"elan1 evaluated this and said no"
May claim	*"This write could not have occurred without passing the gates."*	*"Every write intent is evaluated against policy and recorded, and the record is exportable."*
May not say	anything implying L2: "signed", "tamper-proof" (C14)	"we block", "we prevent", "cannot", "nothing gets through without us"
Added latency	≤ 15 ms over the governed-write envelope [target]	≤ 40 ms p95 for the contract call, excluding the target [target]
Kernel fails	the write does not happen; rung1 fails closed: "Nothing was changed."	the caller must fail closed by its own contract; we cannot make it. A timeout on assert is treated as refuse
Proof record	enforcement_mode: enforced — the decision and the act	enforcement_mode: attested_advisory — the decision, plus the caller-performed flag, which must survive into a bundle
Rollback	the kernel's reversal plan, 30 days	the consumer's problem
Phase 1 use	rung1 Enterprise, hosted ERPNext — and nothing else	stack1 tooling; third parties at GA (Phase 2), pre-GA partners

enforcement_mode is set per write type at declaration time, sealed into every record, rendered and exported — never set in a tool call.
Promotion B → A is per write type, never per tenant: 30 days of advisory traffic, a bypass proof, no long-lived credential anywhere, and a rollback drill by both paths; a later write with no record reverts it at once. O03 §04 calls the report-back seal; E02 keeps seal kernel-only.

MCP alone gives advice, not enforcement: a caller that ignores a refusal still writes, and the record says only that it was told not to.

Source: E05 §02–06; O03 §02–04, §08, §13; E01 §05; IM §4.6, §6.9 · elan1 Technology Memorandum · Sept 2026

Figure 8 — Mode A, SDK in-process and Enforced, against Mode B, MCP or HTTP and Attested advisory: where the target credential lives, what rung 7 does, what a refusal means, what the proof record carries, and the one sentence each mode may claim.

What Attested advisory is worth. "MCP alone gives advice, not enforcement. If your agent holds the credential to the target system, elan1 can tell it 'no' and the agent can ignore us and write anyway. What elan1 produces in that arrangement is an honest, hash-chained record that a decision was made — which is worth money, and is not control" [E05 §02]. The IM calls the two-mode design "the company choosing to sell less" [IM §4.6].

The Mode B flow. A stack1 tool calls `scope` over MCP with its tenant token, submits the intent, and the kernel runs rungs 0b–6 and returns allow, refuse or needs-approval, "and records the decision either way". On allow, "the caller performs the write itself using its own credential" and reports the outcome; the kernel writes the proof record and meters. On refuse, "the tool is expected to stop. Nothing prevents it from writing anyway, and the record says only that it was refused" [O03 §04]. A caller that writes after a refusal produces "an unmatched write: an act with no allow behind it. Reconciliation catches this after the fact, which is exactly the limit of advisory mode" [O03 §04]. If no outcome is ever reported, the action is reconciled as `acted_unattested` and flagged [O03 §04]. Chapter 4 §4.5.4 explains why the outcome report is a separate `act`-scoped call rather than `seal`, which stays kernel-only.

The caller-performed flag. A Mode B record carries "the same fields as Mode A, plus a flag on the proof record stating that the act was performed by the caller and not by the kernel. That flag is what makes the export honest, and it must survive into any bundle a customer shows a third party" [O03 §04, decision 3]. Whether the end customer sees it, or only the export bundle does, is open, due 30 November 2026 [O03 OQ4]. Chapter 4 §4.3.4 records how the flag is committed into a record whose sixteen fields do not name it.

Declared, not claimed. "Every integration record carries `enforcement_mode: attested_advisory | enforced` per write type, set at declaration time, rendered on the Assurance door and printed in any export. A consumer cannot set it in a tool call" [E05 §02, decision 1]. E03 declares the mode per channel (`channels[].mode`), "not per tenant, so a tenant may be enforced on one surface and attested advisory on another — and must be told so" [E03 §02 block 7, decision 8]. E09 runs promotion "per write type, never per tenant, because `channels[].mode` is declared per channel" [E09 §09]. **(Build recommendation)** The effective mode of a governed write is the mode of the `(channel, write_type)` pair it arrived on: the channel declares a default, and a write type may be promoted to Enforced on a channel only through the promotion review (§6.9.5). The rung-2 write door resolves the pair and fixes the mode on the write context, so it is committed with the record and cannot be supplied by the caller. The per-write-type and per-channel wording is recorded as a conflict.

Promotion. Moving a write type from Attested advisory to Enforced "is a formal review, run per write type and never per tenant". It requires thirty days of production traffic in advisory mode, a bypass proof, confirmation that no long-lived credential exists anywhere a consumer, agent or operator can reach (including the consumer's CI), and a rollback drill by both paths. "If a promoted write type later shows a write to the target with no record, the mode reverts immediately and the boundary date is rendered in both directions" [IM §4.6, E09 §09]. "Records written before the promotion keep the mode they were written under, forever. We do not relabel a chain" [E09 §09].

6.3 How enforcement is made physical

An Enforced claim is a claim about where a credential can and cannot be. "There are exactly two ways to be Enforced" [E05 §02].

6.3.1 Credential custody

"The kernel holds the target credential in the secrets broker and mints a single-use credential at the moment of the write, scoped to one write, with a ten-minute life; no long-lived key to the customer's books exists anywhere a consumer, an agent or a human operator can reach. This is how rung1's hosted ERPNext works in Phase 1" [E05 §02]. Tenant credentials for target systems "live in the per-tenant credential broker and are minted per call" [O03 §06]. stack1 supplies the secrets substrate: OpenBao, "one path per tenant, short-lived tokens, mTLS workload identities between planes", with consumers receiving "sealed references only; a consumer never receives a secret value" [S03 §01].

Three CI-enforced rules back the arrangement: nothing above P3 imports a kernel type; exactly one write credential exists per environment; the control plane has no network route to the record store [IM §6.1–6.2, O03 §10].

(Build recommendation) The chain of custody for a Phase 1 hosted ERPNext write:

1. At site provisioning the ERPNext API key and secret for a dedicated integration user are written once to the tenant's broker path by the provisioning workflow, and the provisioning identity's write access to that path is revoked after the write. No human reads the value.

2. The broker policy on that path grants `read` to exactly one workload identity: the kernel's per-tenant writer worker (Chapter 5 §5.17). rung1's services, the stack1 gateway and every human operator role are denied, and a CI test over the broker policy files asserts that the grant list for every tenant's target path has exactly one member.
3. At rung 7 the writer worker obtains a response-wrapped, single-use token with a ten-minute TTL and use count of one, bound to the `write_id`; the wrapped token is unwrapped inside the activity that performs the live apply and never leaves it. A failure here is `PLATFORM.TOKEN_MINT_FAILED` [E02 §06].
4. The ERPNext integration user's role permits only the DocTypes and actions declared in the tenant's `write_types`, so a leaked token could not exceed the declared surface even inside its ten minutes.
5. Conformance test C2 attempts every known path to the site other than the single writer — the rung1 console's own session, the rung1 agent's tools, direct REST with any credential found in rung1's environment and CI, and database access — and expects each to fail with the attempt recorded [E02 §08].

ERPNext's API keys are long-lived by default, so step 3 needs either a short-lived credential scheme on the Frappe side or an egress gateway that holds the key and accepts the single-use token; which is feasible on hosted Frappe is a rung1 engineering question and is recorded as open.

6.3.2 Token acceptance

"The target system is configured to accept only kernel-issued tokens for the write types in question; useful where a customer will not hand over a credential but will change their own system's trust configuration" [E05 §02]. Whether ERPNext hosted mode can be configured this way — which would make Mode B Enforced too — is open with the rung1 lead, due 31 October 2026, and whether token acceptance needs a reference implementation for ERPNext in Phase 1 at all is open, due December 2026 [O03 OQ1, E05 OQ3].

(Build recommendation) A kernel-issued write token is a short-lived signed token (Ed25519 over a JWS, key from the kernel's token-signing key, not the L2 seal key) carrying `tenant_id`, `write_id`, `write_type`, the payload digest, `exp` at ten minutes and a single-use nonce. The target-side verifier (for ERPNext, a small Frappe app installed on the customer's site) rejects any write to a declared DocType that does not carry a valid, unspent token whose payload digest matches the request body. The verifier's public key is pinned at install. The bypass proof then has to show that every other write path to those DocTypes — including the site's own administrator — is refused, which on a customer-operated site is the hard part and is why this mechanism suits customers willing "to change their own system's trust configuration" [E05 §02].

6.3.3 Where the single writer runs when the SDK is "in-process"

E05 says in-process consumers "link `@elan1/kernel` behind KernelPort" [E05 §01]; O03 says the kernel is reached by "library call to KernelPort inside the caller's process" and that in Mode A the kernel holds the credential [O03 §02]. If the whole kernel, including rung 7 and the broker grant, ran in rung1's process, rung1's code would share an address space with a credential, and I2's "a consumer that can reach the target by another path is in Attested advisory mode" would apply to rung1 itself [E02 I2].

(Build recommendation) The SDK linked into a consumer's process contains the port types, a client, and the pure rungs' shared validation (so a consumer gets early, local rejection of malformed intents), but not rung 7, rung 8 or any broker grant. `assert` crosses a process boundary to the kernel service over mTLS; rungs 0b–9 execute there; rung 7 runs in the kernel's per-tenant writer worker. "In-process" then describes the programming model and the ≤ 15 ms overhead target [O03 §02], not the location of the credential. This is recorded as a conflict because it determines what the ≤ 15 ms target can include and what C2 must prove.

6.3.4 The control plane cannot see records

"`ctl`-class paths have no read path to any record store — not a restricted one, none, enforced at the network layer. A control endpoint that returned record data is a boundary violation the network makes impossible rather than the code making unlikely" [E05 §04]. The Studio's Simulate step therefore runs against the reference tenant's synthetic fixtures, "never production records" [E06 §06]. **(Build recommendation)** The control-plane services run in a network segment

whose egress policy denies the record-store segment, expressed in reviewed infrastructure-as-code; a CI test over the network policy asserts the absence of any route, and a runtime probe from the control plane attempts a connection nightly and expects failure.

6.4 The in-process SDK

6.4.1 Packages and languages

E05 names one package, `@elan1/kernel` [E05 §01]. The first production consumer is Python: rung1 builds on Frappe/ERPNext, and the product-folder stack recommends Python 3.12 with FastAPI for the kernel and write API [ETS §04]. **(Build recommendation)** Two SDKs ship, generated from one interface definition and tested by one conformance suite:

Package	Language	First consumer	Contents
`@elan1/kernel`	TypeScript (Node 20+)	stack1 tooling, third parties	Port types, client, webhook verifier, retry policy
`elan1-kernel` (import `elan1_kernelport`)	Python 3.12	rung1 Enterprise	Same, plus Frappe-friendly synchronous client

The SDK package major is the port major (`kernelport/v1`), so a consumer pins the contract by pinning the package, and every response carries `X-Kernel-Port: v1`, which the SDK asserts [O06 §02, §05]. `seal` is not exported (Chapter 4 §4.5.1).

6.4.2 API surface and error model

Call	Returns	Raises
`define(...)`	`DeclarationAccepted`	`Rejected` (4xx); `PlatformUnavailable`
`assert(intent)`	`Committed | Refused | Escalated`	`Rejected`; `Throttled`; `PlatformUnavailable` (5xx before the kernel was reached)
`verify(range)`	`VerifyResult` (pass / fail / not_checked)	`Rejected`
`scope(actor, write_types)`	`ScopeResult`	`Rejected`
`refuse(code, context)`	`RefusalOutcome`	`Rejected` (unregistered or non-raisable code)

(Build recommendation) A refusal and an escalation are returned values, never exceptions, because a refusal is a governed outcome and an exception invites a retry loop — the same reasoning that makes a refusal an HTTP 200 [E02 §04]. A platform-class refusal (the kernel was reached and failed) is also a returned `Refused` whose `code` starts `PLATFORM.`, and the SDK's retry helper retries only those and transport failures, always with the same `intent_id` and at most five attempts with exponential backoff and jitter [E05 §07–08].

6.4.3 Example: submitting a governed write

```
// Build recommendation — @elan1/kernel usage
import { KernelClient, outcomeOf } from "@elan1/kernel";

const k = new KernelClient({ token: process.env.ELAN1_TENANT_TOKEN!, port: "v1" });

const s = await k.scope({ actor_id: agentId, write_types: ["sales_invoice.submit"] });
if (s.permitted[0].reaches === "none") return renderNotAvailable();

const res = await k.assert({
  intent_id: uuidv5(changesetKey, TENANT_NS), tenant_id, declaration_id,
  declaration_version: 7, write_type: "sales_invoice.submit",
  actor_id: agentId, actor_kind: "agent", channel: "api",
  subject: null, payload, as_of: businessInstant,
});
```

```
switch (res.outcome) {
  case "committed": return showDone(res.record.seq, res.undo_until);
  case "escalated": return awaitObligation(res.decision.obligation!.obligation_id);
  case "refused":   return res.decision.code.startsWith("PLATFORM.")
                      ? retryLater(res)                // our failure: same intent_id
                      : showOperatorAction(res);       // never retry a governed refusal
}
```

```
# Build recommendation — elan1_kernelport usage inside a rung1 Temporal activity
from elan1_kernelport import KernelClient, Committed, Escalated, Refused

k = KernelClient(token=tenant_token, port="v1", timeout_s=5.0)   # R04 §07: assert 5 s

res = k.assert_(intent)
match res:
    case Committed(record=r, undo_until=u):
        return {"seq": r.seq, "undo_until": u}
    case Escalated(decision=d):
        return {"await": d.obligation.obligation_id}      # wait for the webhook signal
    case Refused(decision=d) if d.code.startswith("PLATFORM."):
        raise RetryableKernelFailure(d.code)             # Temporal retries, same intent_id
    case Refused(decision=d):
        return {"refused": d.code, "action": res.operator_action}
```

6.5 The MCP server

6.5.1 Scopes and the tool catalogue

Scope	Count (Phase 1)	Meaning
`ro`	18	Reads only; cannot reach rung 7 by any path
`stage`	10	Produces a request object with no side effects at all
`act`	6	Submits a governed write; every call lands on rung 0b

Source: E05 §04. "An A1 agent stages and a human submits; an A2 agent stages and submits and rung 6 stops it at approval; an A3 agent stages, submits and proceeds. Same tools, same descriptions — the difference is entirely in rung 6. Autonomy is a kernel concern, not a tool-catalogue concern" [E05 §04].

Seven tools carry nearly all Phase 1 traffic [E05 §04, IM §6.9]:

Tool	Scope	Args	Returns
`read.records`	ro	`{type, filter, as_of?}`	Permission-filtered rows
`read.evidence`	ro	`{seq_from, seq_to}`	Proof entries
`read.declaration`	ro	`{declaration_id, version?}`	The seven blocks
`read.meter`	ro	`{period}`	Events and totals
`stage.write`	stage	`{write_type, payload, as_of}`	Request object, validated, no side effects
`act.write`	act	`{request, idempotency_key}`	committed · refused · escalated
`act.escalate`	act	`{obligation_id, decision, note}`	Resolution outcome

An `act.escalate` call from an agent token is refused: "an agent may never be an approver at L1" [E06 §03, E02 decision 7]. Which of the 34 tools actually ship in Phase 1 with only ERPNext declared is open, due October 2026 [E05 OQ5].

Protocol baseline. The product-folder stack pins the MCP revision of 28 July 2026 (stateless), with "stage handles as arguments", implemented with the official MCP SDK in the API language rather than hand-rolled JSON-RPC [ETS §02, §08]. The stateless revision matters because `stage.write` returns a request object that the agent passes back to `act.write` as an argument, so no session state is needed on the server.

6.5.2 Pinned tool descriptions

"Tool descriptions are pinned by hash in the declaration. Changing a description changes the hash, which is a new declaration version, which requires certification by a named human" [E05 §04]. The reason is a gap in the protocol: MCP's `tools/list` "has no description pinning and no re-consent requirement", so "an agent can be shown one description on Monday and another on Tuesday with nothing recording the difference" [E05 §04; E05 marks its read of the 2026-07-28 revision unverified]. "We pay a certification for a typo fix and consider that the correct price" [E05 §04]. The Agents admin screen shows "the hash-pinned descriptions it was shown", and a changed description "shows as a version change, not a silent update" [E06 §07]. The IM cites independent research reporting 82% of 2,614 surveyed MCP implementations vulnerable to path traversal, 8.5% using OAuth against 53% on static API keys, and 22.6% of 31,309 registry servers carrying no source repository [IM §6.9]; the design response is to control what a server may do at the write path rather than to inspect the supply chain.

(Build recommendation) The `tools/list` response carries, per tool, the `declaration@version` that exposes it and the description hash, under `_meta`; the server refuses to start if the description text it would serve does not hash to the value in the declaration version it is serving.

```
{ "name": "act.write",
  "description": "Submit a governed write. A refusal is final: never retry it.",
  "inputSchema": {
    "type": "object", "additionalProperties": false,
    "required": ["request", "idempotency_key"],
    "properties": {
      "request": { "type": "object", "description": "The object returned by stage.write" },
      "idempotency_key": { "type": "string", "minLength": 16, "maxLength": 128 } } },
  "_meta": { "elan1/declaration": "d-acme-billing@7",
             "elan1/description_sha256": "4be1…", "elan1/scope": "act" } }
```

6.5.3 How a third-party agent uses it

Authentication. The MCP server accepts a tenant token as an OAuth 2.1 bearer token; tenant and actor come from the token, never from a tool argument [E05 §05]. **(Build recommendation)** Static API keys are not accepted on the MCP surface; a sandbox or production key (§6.7) is exchanged for a short-lived tenant token by a client-credentials grant, so the token the agent holds expires in minutes.

A staged write, then its submission, as JSON-RPC over the MCP transport (build recommendation for the payload detail; tool names and arguments from E05 §04):

```
{ "jsonrpc": "2.0", "id": 41, "method": "tools/call",
  "params": { "name": "stage.write",
    "arguments": { "write_type": "sales_invoice.submit", "as_of": "2027-08-14T10:00:00Z",
      "payload": { "customer": "ACME Textiles", "posting_date": "2027-08-14",
                   "items": [ { "item_code": "YARN-40S", "qty": 800, "rate": 300 } ] } } } }
```

```
{ "jsonrpc": "2.0", "id": 41, "result": {
    "structuredContent": { "request": { "request_id": "rq_7f…", "declaration_version": 7,
        "write_type": "sales_invoice.submit", "payload_digest": "c1d9…", "valid": true,
        "channel": "mcp", "enforcement_mode": "attested_advisory" } },
    "content": [ { "type": "text",
                   "text": "Staged. No side effects. Submit with act.write." } ] } }
```

```
{ "jsonrpc": "2.0", "id": 42, "method": "tools/call",
  "params": { "name": "act.write",
    "arguments": { "request": { "request_id": "rq_7f…" },
                   "idempotency_key": "acme-inv-2027-08-14-0012" } } }
```

```
{ "jsonrpc": "2.0", "id": 42, "result": {
    "structuredContent": { "outcome": "refused", "code": "GATE.PRECONDITION_FAILED",
      "rung": 3, "gate": "precondition", "seq": 4098, "digest": "…",
      "operator_action": "Fix the record state",
      "detail": "credit_exposure + amount exceeds credit_limit (300000)" },
```

```
    "content": [ { "type": "text",
      "text": "Refused at precondition. Do not retry. Credit limit exceeded." } ],
    "isError": false } }
```

The refusal is a successful tool result (`isError: false`), for the same reason it is HTTP 200: an error invites the agent's framework to retry [E02 §04, E05 §07]. The agent's obligations are the consumer's three outcomes [E05 §11 item 8]: on `committed`, report the `seq`; on `refused`, stop and surface the operator action; on `escalated`, stop and wait for `obligation.resolved`, then resubmit. In Attested advisory mode the agent then performs the write itself and reports the outcome; nothing but its own code stops it writing after a refusal [O03 §04].

rung1's ERPNext MCP servers are a different thing. They live in stack1's tool plane (`fusion.tools/v1`) and are the tools rung1's agent calls inside the sandbox [S01 §02 C7, RTM Ch.5 §5.9]. They are not the kernel's MCP surface, and nothing they do is a governed write until it reaches `assert`.

6.6 The HTTP API

Path	Class	Serves
`/v1/writes`	act	Submit, read, list governed writes
`/v1/obligations`	act	Escalations and approvals
`/v1/records`	ro	Governed records and versions
`/v1/evidence`	ro	Proof entries and chain continuity
`/v1/meter`	ro	Events, totals, reconciliation
`/v1/declarations`	ctl	Author, validate, simulate, certify, publish
`/v1/actors`	ctl	Actors, delegations, autonomy grants
`/v1/tenants`	ctl	Config, stop, residency, keys

Source: E05 §04. `GET /v1/versions` returns the four numbers a consumer pins [O06 §05].

Error taxonomy [E05 §07]:

Kind	HTTP	Proof entry	Meter	Retry?
Refusal	200, `outcome: refused`, code, rung, gate, every gate result, `seq`, digest	yes	zero	never
Escalation	200, `outcome: escalated`, `obligation_id`	yes	zero	never; wait for the outcome
Rejection — malformed	4xx; nothing entered the kernel	no	no	only after fixing
Throttle	429 with `Retry-After`	no	no	yes, with backoff
Platform failure	5xx, or a `PLATFORM.*` refusal if the kernel was reached	refusal only	no	yes, same key
Tenant stopped	200, `outcome: refused`, `POLICY.TENANT_STOPPED`	no — the only refusal without one	no	no

"Ten are PLATFORM class — we failed. They are not billable and each one pages" [E05 §07].

Rate limits. Per tenant token, returned on every response as `X-RateLimit-Limit`, `-Remaining` and `-Reset`; a breach is 429 with `Retry-After`, "a transport rejection, not a refusal" [E05 §06]. Sandbox keys get 10 writes a minute, 2 concurrent and 5,000 a month with a hard stop; production limits are per contract, and default production tiers are open because "no observed load exists to set them from" [E09 §02 item 3, E05 OQ2].

Webhooks. Signed with an HMAC over the body using a per-endpoint secret, in `X-Elan1-Signature` and `X-Elan1-Timestamp`; anything older than five minutes is rejected; delivery is at least once, so consumers are idempotent on `write_id` [E05 §09]. The events are `write.committed`, `write.refused`, `write.escalated`, `obligation.resolved`, `obligation.expired`, `declaration.certified` and `tenant.stopped`; every write payload carries `seq` and digest so a

consumer can quote a proof identifier and "detect a gap in its own delivery by watching `seq` contiguity per tenant" [E05 §09]. "`write.refused` fires as reliably as `write.committed`" [E05 §09]. Whether to publish elan1's own HMAC scheme or adopt an existing standard signature header is open, due November 2026 [E05 OQ1]; the product-folder stack recommends Standard Webhooks signatures with delivery from the outbox [ETS §08]. **(Build recommendation)** Adopt Standard Webhooks: it is the same HMAC construction, and a consumer can verify it with an existing library rather than elan1's.

Polling. `GET /v1/obligations?status=open` and `GET /v1/writes/{write_id}`, no faster than every 10 seconds; polls are `ro` and not metered. "`/v1/evidence` pages by `seq`, not by offset or cursor timestamp — contiguity is the pagination contract and the integrity property at the same time" [E05 §09].

Versioning. The HTTP surface is versioned in the path; additive changes ship without notice and consumers must ignore unknown fields; breaking changes get a new path version and the old one runs for 12 months with `Sunset` and `Deprecation` headers on every response [E05 §10, O06 §04]. Chapter 4 §4.7 gives the full policy.

6.7 Sandbox key, production key and the shadow sandbox

"Two different things share the word 'sandbox' and confusing them is expensive" [E05 §05].

	Sandbox key `SK_SBX_…`	Production key `SK_LIVE_…`
Tenant	An isolated sandbox tenant with synthetic fixtures	The customer's tenant
Target	A sandbox ERPNext site, never a live one	The live site, via the shadow sandbox on every write
Gates	All of them, identically	All of them
Proof chain	Its own chain, `mode: sandbox` in every entry	The tenant's chain
Meter	Events emitted, quantity zero, not billed	Billed
Rate limit	10 writes/min, 2 concurrent	Per contract
Issued	Self-serve on e-mail verification (from GA)	By elan1, after review; never self-serve

Sources: E05 §05, E09 §01–03. "The mode field is sealed into the proof entry, so a sandbox entry can never be presented as a production one" [E05 §05]. Resolving `"latest"` is available with sandbox keys only [E05 §10]. "Sandbox keys are self-serve from general availability; production keys never are, and the gap between the two is the entire product" [E09, in one line].

6.8 The sales rule — advisory is never called enforcement

> **The sentence a salesperson may not say in Attested advisory mode:** "The write cannot happen without elan1 approving it." It can. Also forbidden: "we block", "we prevent", "nothing gets through without us". Permitted: "every attempt is evaluated and recorded, including the ones we refused" [E05 §02].

Mode	Permitted	Forbidden
Enforced	"This write could not have occurred without passing the gates" [E05 §02]; "No write reaches the system of record without a gate decision and a proof record" [O03 §02]	Anything implying L2 at L1: "signed", "tamper-proof", "independently verifiable" [E02 C14]
Attested advisory	"Every attempt is evaluated and recorded, including the ones we refused" [E05 §02]; "Every write intent is evaluated against policy and recorded, and the record is exportable" [O03 §02]	"cannot", "prevented", "blocked", "we block", "we prevent", "nothing gets through without us" [E05 §02, O03 §02]; and the L1 words above

"Writing 'enforced' on a slide about an MCP integration is the one claim that would end a first enterprise deal. Check which mode the deployment is in before the slide is made" [O03 §02]. "A customer who thinks they bought Enforced and got Attested advisory has been mis-sold, whatever the contract says" [IM §4.6].

The rule is made mechanical in three places. The Studio shows, per channel, "the sentence the developer will be permitted to say about it" as soon as the mode is set [E09 §04 step 3]. The mode is rendered on the Assurance door and printed in every export [E05 §02]. Conformance test C14 runs as a string check over the consumer's rendered copy and fails the build on the L1 words [E02 §08]; after a promotion, C14 "now permits the Enforced sentence… for those write types and no others" [E09 §09 step 7]. **(Build recommendation)** C14 is extended to the advisory list: for every channel whose declared mode is `attested_advisory`, the check fails on "block", "prevent", "cannot" and "nothing gets through" in copy rendered for that channel, so both halves of the sales rule are build failures rather than one.

6.9 Third-party onboarding and conformance

6.9.1 What GA means

"General availability at L1 means an outside developer can sign up without talking to us, get a sandbox key in a minute, build against the reference tenant, and be issued a production key after a review we run and they pass. It does not mean self-serve production" [E09 §01]. GA does not change the Assurance Level; a consumer who tells its own customers the record is signed "has broken conformance test C14, which is a build failure in their CI, not a conversation" [E09 §01].

6.9.2 The readiness gate

GA opens only when all eight rows read "done, measured rather than asserted"; "a partial GA… is the arrangement under which the first external incident is handled by nobody" [E09 §01, decision 2].

#	Item	Done means (abridged)	Measured by
1	Documentation	E02, E03, E04, E05, E12, E13 at stable URLs; every refusal code has a public page	Link check in CI
2	Sandbox keys	Self-serve on e-mail verification; tenant with fixtures in under 60 s	Timed; p95 published
3	Quotas	Sandbox 10/min, 2 concurrent, 5,000/month; `X-RateLimit-*`; 429 on breach	Quarterly load test
4	Status page	Per component, incident history never edited; chain integrity its own component	A missed nightly job flips it
5	Support path	One address, a response-time promise per tier, a named human on a rota	First response published monthly
6	Incident process	Severity ladder, page targets, a note for every P0/P1 within five working days	A P0 without a note is a P1
7	Versioning policy	`/v1`, 12-month sunset, headers, codes never reused	Header assertion in contract tests
8	Conformance tests	C1–C14 runnable by a consumer with fixtures and a result file	Passes on reference, fails on a broken one

Source: E09 §02. "Items 4, 6 and 8 are the ones that get deferred under schedule pressure. They are also the three a serious integrator checks first" [E09 §02].

6.9.3 The path, from sign-up to first invoice

1. **Sandbox key.** Work e-mail, company name and a one-line description; verification link; isolated tenant with the reference fixtures; the key shown once [E09 §03]. A fixture failure issues no key, because a key against an unprovisioned tenant "would produce refusals the developer would reasonably read as our product being broken" [E09 §03].
2. **Declaration.** Seven blocks in the Studio; every field a value, `out_of_scope` with an ADR, or `not_measurable` with reasons; blast radius and exactly one of reversal plan or `irreversible: true` per write type, with the resolved meter basis shown; `channels[].mode` chosen; Validate; Simulate (a simulation with no refusals is returned with "Add a fixture that fails"); registration as a dual-control governed write; certification by a named human [E09 §04].
3. **Conformance.** C1 and C2 first, then C3–C13, then C14; a machine-readable result file [E09 §05].
4. **Production.** Request with the result file, declaration version, target system, residency domain and a named production owner [E09 §06]. Then wire `stage.write` before `act.write`, prove all three outcomes, implement idempotency and

prove a replayed key returns the original `seq`, register a webhook and prove seq-gap detection [E05 §11 items 7–10].

5. **First invoice.** Two-way reconciliation before any invoice; the refusal count at ₹0 on the face of the invoice; every line opens its records [E09 §07].

6.9.4 The production review, and who is refused

The reviewer "checks four things and only four": the enforcement mode is stated in writing and matches `channels[].mode`; every write type has a reversal plan or an explicit irreversible flag; the three outcomes were proved in sandbox, "including a real refusal and a real escalation resolved by a real human"; a named human certified the declaration [E09 §06]. A named notice contact is a further condition of a production key [O06 decision 7]. Two working days is the promise, flagged as having "no staffing behind it yet", with the question of who reviews at twenty requests a week open to 31 March 2027 [E09 OQ1, IM §6.9].

E09 names eight categories refused before a key exists: no declaration; no stated enforcement mode; all write types irreversible with no approval step; a per-customer meter basis; disabling a never-skippable gate; a signature, bundle or regulator claim at L1; a residency domain elan1 does not run; and no named production owner [E09 §08]. To the applicant who needs a signature the scripted answer is "You need L2… Keep talking to us and do not buy this yet" [E09 §08].

6.9.5 The promotion review

Run per write type, triggered by the consumer after at least 30 days of production traffic in Attested advisory [E09 §09]. The consumer names the mechanism (credential custody or token acceptance); the reviewer demands the bypass proof — C2 "run against the real target, not the reference tenant"; confirms no long-lived credential exists anywhere reachable, including CI, admin tooling and break-glass paths (a break-glass path is acceptable only as a declared dual-control write type with its own record); and checks the rollback drill on both paths. The mode change is a new certified declaration version under dual control, and the Assurance door shows the boundary date and the count of records on each side [E09 §09 steps 1–6]. A failed bypass proof leaves the consumer Attested advisory: "A consumer who cannot close it is not a failed customer; they are an honest Attested advisory customer" [E09 §09]. Whether the 30-day observation period "has any basis, or is it a number we picked" is open to 30 April 2027 [E09 OQ2].

6.10 Decisions and open questions

E05 records seven decisions (21 September 2026): `enforcement_mode` is declared, sealed and rendered and cannot be set in a tool call (1); refusals are HTTP 200 and throttling is 429 with no proof entry (2); breaking API changes get a new path version with a 12-month sunset (3); sandbox writes seal into a separate chain and are never billed (4); tool descriptions are hash-pinned and a description change is a certified version (5); production keys are issued by elan1 after a checklist, not self-serve, for all of Phase 1 (6); "sandbox key" and "shadow sandbox" are never abbreviated to "sandbox" in consumer-facing text (7) [E05 §12]. O03 fixes two modes only and restricts "Enforced" to kernel-held credentials or token acceptance [O03 decisions 1–3]. E09 keeps production keys human-reviewed at GA and makes promotion per write type with immediate reversion on an unrecorded write [E09 decisions 1, 6, 7].

#	Open question	Owner, by	Source
1	Publish elan1's HMAC webhook scheme or adopt a standard header?	Kernel engineer, Nov 2026	E05 OQ1
2	Default production rate limits with no observed load	elan1 lead, first design partner	E05 OQ2
3	Does token acceptance need an ERPNext reference implementation in Phase 1?	ERPNext engineer, Dec 2026	E05 OQ3; O03 OQ1 (31 Oct 2026)
4	Is 12 months the right sunset for consumers under a regulator's change control?	elan1 lead, Phase 2	E05 OQ4
5	Which of the 34 tools ship in Phase 1	Kernel engineer, Oct 2026	E05 OQ5
6	Does the shadow sandbox's second gate run need a different third-party latency promise?	Kernel engineer, Dec 2026	E05 OQ6

#	Open question	Owner, by	Source
7	Who reviews production requests at twenty a week?	Founder, 31 Mar 2027	E09 OQ1
8	Basis for the 30-day advisory observation period	elan1 lead, 30 Apr 2027	E09 OQ2
9	Publish the conformance suite's expected-failure fixtures, given they map the gate set?	elan1 lead, 28 Feb 2027	E09 OQ3
10	Is the caller-performed flag visible to the end customer or only in exports?	elan1 kernel lead, 30 Nov 2026	O03 OQ4

This chapter adds one for the rung1 lead: whether hosted Frappe can take a short-lived credential scheme or an egress gateway, so that rung 7's ten-minute single-use credential is real rather than a wrapper around a long-lived API key (§6.3.1).

7. The Declaration — the ontology, controls, regulations and policy as data

Chapter 5 described what the kernel does to one write. This chapter describes what the kernel does it *against*. Every gate the Right Path runs, every write type rung 2 resolves, every field rung 8 commits and every screen the console renders comes from one versioned object: the Declaration. It is the elan1 ontology, and it is the reason a regulatory pack is configuration rather than a software release [E03 §01; IM §4.5]. The chapter covers why policy is held as data, the seven blocks field by field, the closed enumerations, the validation and compilation pipeline, versioning and pinning, the two families that complete the ontology (Controls and Regulations) and the five-link chain that binds a regulation clause to a proof field. It closes with the Palantir comparison, the tests, and what is in Level 1 and what is not.

> **Designed is not shipped (DR-010).** As at 23 September 2026 no declaration has been certified for a customer, the Studio does not exist as running software, and the Controls and Regulations families are a Phase 4 specification. Everything below is specification or build recommendation [O01 DR-010; E21 §01].

7.1 Why policy is data, not code

A declaration says, in one machine-readable object, *this thing, shaped this way, may be changed by these actors through these write types, subject to these gates, proving these fields, over these channels* [E03 §01]. It is JSON with `additionalProperties: false`, it stays under 900 lines, and it is never a programming language: "if a declaration needs a loop, the design is wrong" [E03 §01; IM §6.6].

Holding policy as data is the load-bearing commercial claim of the architecture, not a style preference [IM §4.5]. Four consequences follow from it, and each would be lost if policy were written as code in the kernel or in a consumer.

Consequence	Mechanism	What breaks if policy were code	Source
A new regime needs no kernel release	The nine gates already exist as pure functions; a regime supplies parameters, a declaration, a verifier profile and an approver model	Every regime becomes a kernel change, every kernel change re-opens conformance, and the "kernel built once" economics disappear	[IM §4.5; IM §5.6]
The approver's screen can be rebuilt years later	Compilation is deterministic and content-addressed; every version is kept verbatim for ever	"What did the approver see in March 2027" becomes a question about which build was deployed that week	[E03 §06; E06 §01]
The proof names the rule that judged it	Records carry `declaration_id`, `declaration_version`, `policy_version` and `lexicon_hash` (F06–F08, F15)	A record would point at a code commit that an offline verifier cannot resolve	[E02 §03; E03 §07]
The UI cannot allow what the backend forbids	Screens, controls and refusal copy are compile outputs of the same object the kernel resolves	Two sources of truth for "allowed", which is the defect class DR-001 exists to prevent	[EBB EB-15 §3; O01 DR-001]

The division between code and data is sharp. Code: the nine rungs and their order (plane P3, "shut"), the nine gate functions G1–G9 as pure functions of `(context, declaration_version, record_state, as_of)`, and the offline verifier. Data: gate parameters, order and enablement; rung 4 rule sets at a `policy_version` the kernel resolves; declarations, Controls and Regulations; and verifier profiles, which the fixed verifier reads [E04 §03; E02 §03; E13 §02; O01 §03]. Each data artefact is authored by one named human and certified by another [E03 §06; E21 §10 D8].

> **A model may produce a write intent; it may never produce an allow.** The declaration governs what the kernel evaluates, but nothing in it can place a model in the decision path. Rung 4 `evaluate()` reads versioned rules over a projection of declared fields, and "there is no model in this path and no configuration that puts one there" [E02 I1; E04 §03].

The prohibition on loops is the other half of the same idea. A declaration is a set of facts about a write; anything procedural belongs in a gate function (kernel code, pure, clock-free) or in an adapter (consumer code, outside the kernel). Lint LN-01 warns past 900 lines, "because it has become a program" [E03 §05].

7.2 The ontology claim, stated precisely

The v3.0 corpus uses the word "ontology" deliberately, reversing earlier caution about entering a category an incumbent named, because in 2026 it is the word buyers use to sort serious enterprise AI from chatbots [E03 §01; GIP §2.1]. The claim is narrow. Palantir's ontology is *constructed* by forward-deployed engineers against a data estate. Yonyou's is *extracted* from structured and unstructured sources. Both are made first and reasoned over afterwards. elan1's is a *residue*: it accretes from governed writes that versioned rules typed and a human approved, and each entity, relation and constraint carries the transaction that attested it. The system is named after the write and not after the ontology, because without the write path there is no ontology to name it after [E03 §01; GIP §2.1]. Compressed: Palantir models the world; elan1 models the permission to change it [IM §6.8].

Two naming rules travel with the word. Block 7 is `channels`; the v2.0 name `doors` is retired from the declaration and survives only as the Four Doors UX grammar (Work · Records · Agents · Assurance) [O01 DR-012; E03 §01]. And the autonomy ladder is written A0–A5; "L" is reserved for Assurance Levels, and "the declaration schema" is called by its name rather than "L0 schema" [O01 DR-011; E04 §05].

7.3 The seven blocks

The seven blocks, read in order as one sentence, are `identity · shape · permissions · write_types · gates · proof_binding · channels` [E03 §01]. Figure 9 shows them and the references between them. The arrows are the semantic rules of §7.6: every reference from one block to another must resolve at validation, in CI, with no platform running.

Figure 9 — The Declaration — seven blocks and their relations

L1 — BEING BUILT

Seven blocks of one JSON object (under 900 lines); arrows are references checked at VALIDATE, in CI

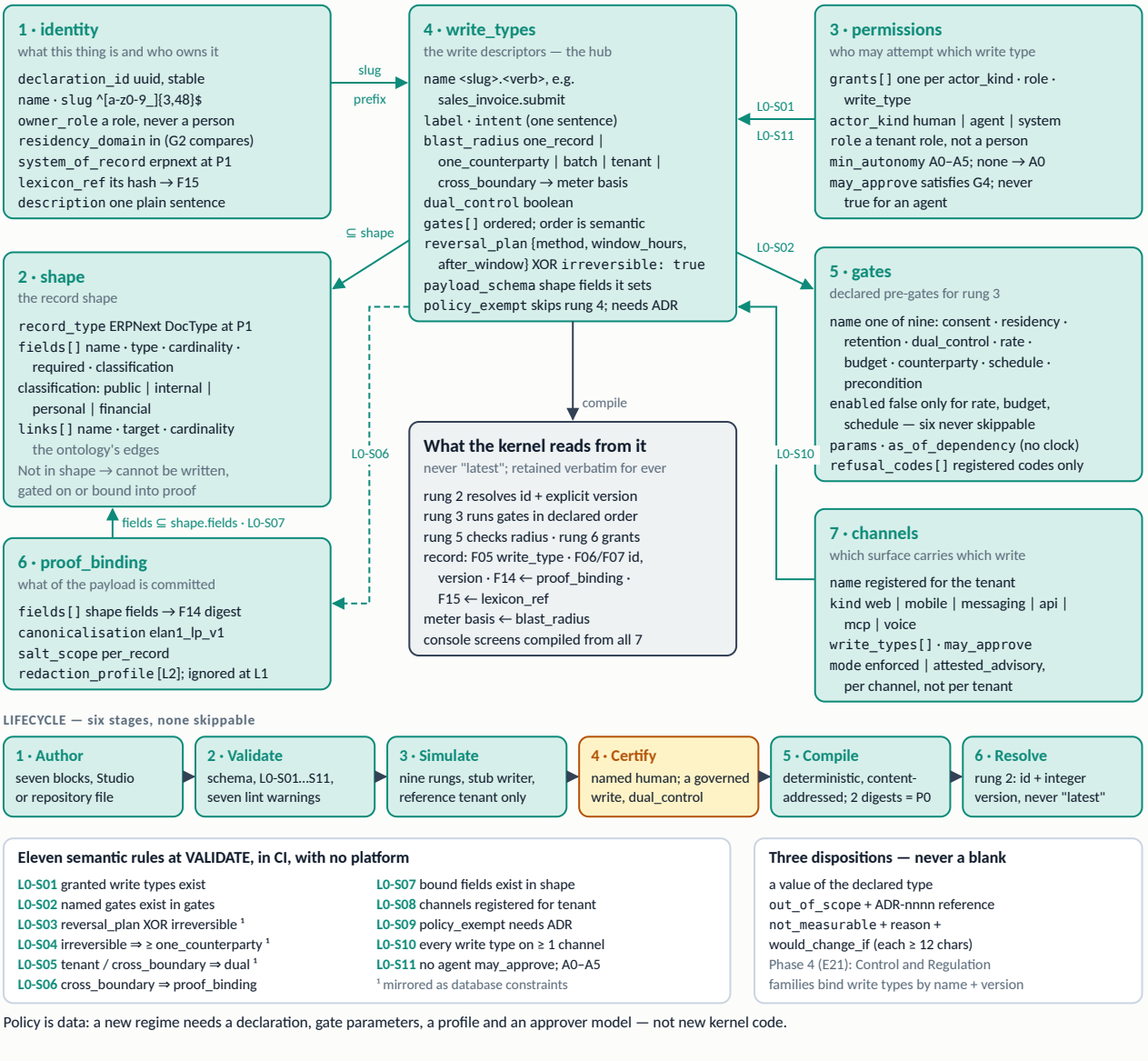


Figure 9 — The seven Declaration blocks, the references between them that validation resolves, and the Controls and Regulations families that bind to them by name and version.

The sketches in this section are shown in YAML for legibility. The stored, validated and compiled artefact is JSON under JSON Schema 2020-12 with `additionalProperties: false` [E03 §01; ETS §08]. Accepting YAML as an authoring front-end is a **(build recommendation)**, restricted to a strict subset (no anchors, aliases, tags or implicit typing) and converted to canonical JSON before validation, so that the digest is always computed over the JSON form and never over the YAML text.

7.3.1 Block 1 — identity

What the thing is called, who is accountable for it, where it lives and which system of record it binds to [E03 §02].

Field	Type	Req	Meaning
`declaration_id`	uuid	yes	Stable across every version
`name`	string	yes	Human name shown in the product
`slug`	string	yes	`^[a-z0-9_]{3,48}$`; the prefix of every write type name

Field	Type	Req	Meaning
`owner_role`	string	yes	The accountable role, never a person — "people leave"
`residency_domain`	enum	yes	`in` at Phase 1; gate G2 compares the record's domain to the tenant's
`system_of_record`	string	yes	`erpnext` at Phase 1; the adapter this declaration binds to
`lexicon_ref`	string	yes	The term set in force; its hash enters record field F15
`description`	string	yes	One sentence a non-engineer can read

The lexicon is the quiet part of the identity block. Every declaration references a term set whose hash is sealed into every record, so that "credit limit" means in 2033 what it meant on the day of the write [IM §6.7]. Whether the lexicon lives inside the declaration or beside it — beside lets several declarations share one term set, which a multi-module tenant will want — is open [E03 §11 Q5].

7.3.2 Block 2 — shape

The record shape: the target system's record type and every governed field [E03 §02].

Field	Type	Req	Meaning
`record_type`	string	yes	Target system's type name (an ERPNext DocType at Phase 1)
`fields[].name`	string	yes	Field name as the target system spells it
`fields[].type`	enum	yes	`string · integer · decimal · boolean · date · datetime · reference · enum`
`fields[].cardinality`	enum	yes	`one · many`
`fields[].required`	boolean	yes	Required for a valid payload
`fields[].classification`	enum	yes	`public · internal · personal · financial`
`fields[].references`	string	cond	Required when `type = reference`; names another `record_type`
`links[].name / .target / .cardinality`	string / string / enum	no	Named relations to other declarations' record types — the ontology's edges

Two properties make this block more than a schema. First, *absence is enforcement*: a field absent from `shape` cannot be written, cannot be gated on and cannot enter `proof_binding` [E03 §02]. Second, classification is part of the ontology rather than a tag applied later. It drives consent at L1, redaction at L2 and disclosure at L3 [E21 §02; IM §6.6]. Whether `links[]` should carry enough semantics to answer graph questions, which invites a comparison with Palantir on its own ground, is open [E03 §11 Q4].

7.3.3 Block 3 — permissions

Who may attempt which write type, at what minimum autonomy, and whether they may approve [E03 §02].

Field	Type	Req	Meaning
`grants[].actor_kind`	enum	yes	`human · agent · system`. Never inferred at runtime
`grants[].role`	string	yes	Role in the tenant, not a named individual
`grants[].write_type`	string	yes	Must exist in `write_types`
`grants[].min_autonomy`	enum	yes	A0–A5 the actor must hold; A0 cannot reach rung 7
`grants[].may_approve`	boolean	yes	Whether this role can satisfy gate G4; `true` is invalid where `actor_kind = agent`

Permissions are not a UI concern: "a control the actor may not use is not rendered at all" — not disabled, not rendered [E03 §02; EBB EB-15 §3]. A missing grant resolves to A0 at rung 6, never to a tenant default and never inherited from a delegation chain [E04 §03]. From Phase 4, `min_autonomy` becomes the floor that an earned-autonomy graduation may raise against, and never a value support staff may edit [E21 §02; E24].

7.3.4 Block 4 — write_types

The descriptors. This is where the declaration states what changes in the world and how the world is put back [E03 §02].

Field	Type	Req	Meaning
`name`	string	yes	`<slug>.<verb>`, e.g. `sales_invoice.submit`
`label`	string	yes	What the button says, in the user's words
`intent`	string	yes	One sentence: what changes in the world when this succeeds
`blast_radius`	enum	yes	`one_record · one_counterparty · batch · tenant · cross_boundary`; sets the meter basis
`dual_control`	boolean	yes	Forced `true` by `tenant` and `cross_boundary` radii
`gates[]`	array	yes	Gate names, in the order they run; the order is semantic
`reversal_plan`	object	cond	Exactly one of `reversal_plan` or `irreversible`
`reversal_plan.method`	enum	cond	`snapshot_restore · compensating_entry · target_native_undo`
`reversal_plan.window_hours`	integer	cond	How long the method remains available
`reversal_plan.after_window`	enum	cond	`compensating_entry · not_reversible`
`irreversible`	boolean	cond	`true` only; forces radius ≥ `one_counterparty` and bars auto-approval
`payload_schema`	object	yes	Which `shape` fields this write type may set, and their bounds
`policy_exempt`	boolean	no	Skips rung 4; requires `adr`; never valid on an irreversible write type
`adr`	string	cond	`^ADR-[0-9]{4}$`

The reversal rule — exactly one of a plan or an explicit `irreversible: true` — is the block's defining constraint. It is checked at validation (L0-S03) and mirrored as a database `CHECK`, "because a rule enforced only in an application layer is a rule a migration script will violate at two in the morning" [E03 §05; EBB EB-15 §6]. The sales invoice example shows why a single boolean would not do: the invoice is reversible by native cancellation for the 24-hour IRN cancellation window and thereafter only by a credit note, which is a different governed write with its own record [E03 §04].

7.3.5 Block 5 — gates

The declared pre-gates for rung 3 [E03 §02; E04 §03].

Field	Type	Req	Meaning
`name`	enum	yes	One of nine: `consent · residency · retention · dual_control · rate · budget · counterparty · schedule · precondition`
`enabled`	boolean	yes	Only `rate`, `budget` and `schedule` may be `false`
`params`	object	cond	The gate's declared parameters: a rate, a budget, a window, a record-state predicate
`as_of_dependency`	boolean	yes	Whether the gate evaluates against the intent's `as_of`; no gate reads a clock
`refusal_codes[]`	array	yes	Register codes this gate may raise; an unregistered code fails CI

The six never-skippable gates — consent, residency, retention, dual control, counterparty, precondition — are enforced twice: a declaration that omits one fails validation, and the kernel runs them at rung 3 regardless of what the declaration says, "belt and braces, deliberately" [E03 §02; E04 §03]. Disabling `rate`, `budget` or `schedule` is permitted but should be recorded with an ADR (lint LN-07) [E03 §05].

The precondition gate (G9) is where most domain logic lives, and it has to be kept honest. Its `params` hold a record-state predicate — "period not closed; no duplicate invoice number; PO and GRN cross-checked where a PO exists" — evaluated as a pure function over declared fields [E03 §04]. The predicate language is not specified in the corpus. This memorandum recommends a small, non-Turing-complete expression language over declared field paths and `as_of` only, with no function calls other than a fixed library of comparisons and aggregates over fields the declaration names **(build recommendation)**. CEL is the nearest existing candidate and is already listed as an alternative for the rung 4 evaluator [ETS §08].

7.3.6 Block 6 — proof_binding

What of the payload enters the digest, F14 `payload_hash` [E03 §02].

Field	Type	Req	Meaning
`fields[]`	array	yes	Field names from `shape` that enter F14
`canonicalisation`	enum	yes	`elan1_lp_v1` — length-prefixed, no JSON, no delimiters; `elan1_lp_sd_v1` added in Phase 4 for selective disclosure
`salt_scope`	enum	yes	`per_record` at Phase 1; `per_field` added in Phase 4
`redaction_profile`	string	no	Which fields an export reveals; ignored at L1, live from Phase 4

A `cross_boundary` write must have a non-empty binding: "a write that crosses an organisational boundary and commits to nothing about its payload is not a governed write" [E03 §02]. Leaving a field out is a decision, not an oversight. In the invoice example `customer_gstin` is deliberately excluded because it is personal, adds nothing the digest needs to commit to, and binding it would make a future erasure request harder than it needs to be; lint LN-06 fires when a financial or personal field is unbound without an ADR [E03 §04–§05]. The per-record salt is what lets L2 erase a payload without breaking the chain (Chapters 9 and 10).

7.3.7 Block 7 — channels

Which channels may carry which write types, and in which enforcement mode [E03 §02].

Field	Type	Req	Meaning
`channels[].name`	string	yes	A channel registered for the tenant
`channels[].kind`	enum	yes	`web · mobile · messaging · api · mcp · voice`
`channels[].write_types[]`	array	yes	Which write types this channel may carry
`channels[].may_approve`	boolean	yes	Whether an approval may be given through this channel
`channels[].mode`	enum	yes	`enforced · attested_advisory`, stated per channel, not per tenant

"A channel adapts; it never decides." The kernel re-checks every declared gate even where the channel has already checked one [E03 §02]. Mode is per channel so that a tenant may be enforced on one surface and attested advisory on another — and must be told so [E03 §10 D8]. Changing a channel's mode is a declaration change and therefore a new certified version [E09 §09]. Chapter 6 covers what each mode lets the company say; Chapter 8 covers the approval channels.

7.3.8 The seven blocks together — `sales_invoice.submit` on ERPNext

The worked example in E03 is the write a chartered accountant asks about first: submitting a Sales Invoice for rung1 on hosted ERPNext, which posts to the ledger and, above the GST threshold, requests an IRN [E03 §04]. Assembled, it reads as follows (values from E03 §04; the structural form is illustrative).

```
l0_version: 1
identity:
  declaration_id: 7b0e…            # stable across versions
  name: "rung1 — Sales invoicing (ERPNext)"
  slug: sales_invoice
  owner_role: "Accounts head"
  residency_domain: in
  system_of_record: erpnext
  lexicon_ref: rung1/erpnext/gst/2026-09
  description: "Submitting and cancelling sales invoices in the tenant's books."
shape:
  record_type: "Sales Invoice"
  fields:
    - {name: customer,       type: reference, references: Customer,
       cardinality: one,  required: true,  classification: internal}
    - {name: posting_date,   type: date,    cardinality: one,  required: true,
       classification: internal}
    - {name: items,          type: reference, references: "Sales Invoice Item",
```

```
       cardinality: many, required: true,  classification: financial}
    - {name: grand_total,    type: decimal, cardinality: one,  required: true,
       classification: financial}
    - {name: taxes,          type: reference, references: "Sales Taxes and Charges",
       cardinality: many, required: true,  classification: financial}
    - {name: customer_gstin, type: string,  cardinality: one,  required: false,
       classification: personal}
    - {name: irn,            type: string,  cardinality: one,  required: false,
       classification: financial}
  links:
    - {name: customer,            target: Customer,      cardinality: many_to_one}
    - {name: against_sales_order, target: "Sales Order", cardinality: many_to_one}
permissions:
  grants:
    - {actor_kind: human, role: "Accounts head",        write_type: sales_invoice.submit,
       min_autonomy: A3, may_approve: true}
    - {actor_kind: human, role: "Accounts executive",   write_type: sales_invoice.submit,
       min_autonomy: A2, may_approve: false}
    - {actor_kind: agent, role: "rung1 drafting agent", write_type: sales_invoice.submit,
       min_autonomy: A2, may_approve: false}
    - {actor_kind: human, role: "Owner",                write_type: sales_invoice.submit,
       min_autonomy: A4, may_approve: true}
write_types:
  - name: sales_invoice.submit
    label: "Submit invoice"
    intent: "Posts the invoice to the ledger and, above the e-invoicing threshold,
             requests an IRN"
    blast_radius: one_counterparty
    dual_control: true            # threshold carried as a precondition parameter
    gates: [consent, residency, retention, counterparty, precondition,
            dual_control, budget]
    reversal_plan:
      method: target_native_undo  # cancel the IRN and the invoice
      window_hours: 24            # the GST IRN cancellation window
      after_window: compensating_entry   # a credit note, never a silent delete
    payload_schema: {sets: [customer, posting_date, items, taxes], bounds: {…}}
gates:
  - {name: consent, enabled: true, as_of_dependency: true,
     params: {purpose: invoicing, applies_to: [customer_gstin]},
     refusal_codes: [GATE.CONSENT_MISSING, GATE.CONSENT_EXPIRED]}
  - {name: residency, enabled: true, as_of_dependency: false,
     refusal_codes: [GATE.RESIDENCY_MISMATCH]}
  - {name: retention, enabled: true, as_of_dependency: true,
     params: {window: open_gst_period}, refusal_codes: [GATE.RETENTION_WINDOW_CLOSED]}
  - {name: counterparty, enabled: true, as_of_dependency: true,
     params: {require: [registered, gstin_validated]},
     refusal_codes: [GATE.COUNTERPARTY_UNKNOWN, GATE.COUNTERPARTY_SANCTIONED]}
  - {name: precondition, enabled: true, as_of_dependency: true,
     params: {all: [period_open, no_duplicate_invoice_number, po_grn_match_if_po]},
     refusal_codes: [GATE.PRECONDITION_FAILED]}
  - {name: dual_control, enabled: true, as_of_dependency: false,
     params: {required_when: "grand_total > 200000"},
     refusal_codes: [GATE.DUAL_CONTROL_PENDING, GATE.DUAL_CONTROL_SAME_ACTOR]}
  - {name: budget, enabled: true, as_of_dependency: true,
     params: {per_period_agent_draft_budget: {…}}, refusal_codes: [GATE.BUDGET_EXHAUSTED]}
  - {name: rate,     enabled: false, disposition: out_of_scope, adr: ADR-0nnn}
  - {name: schedule, enabled: false, disposition: out_of_scope, adr: ADR-0nnn}
proof_binding:
  fields: [customer, posting_date, grand_total, items[].item_code, items[].qty,
           items[].rate]
  canonicalisation: elan1_lp_v1
  salt_scope: per_record
channels:
  - {name: web_console, kind: web,       write_types: [sales_invoice.submit],
     may_approve: true,  mode: enforced}
  - {name: mobile_pwa,  kind: mobile,    write_types: [], may_approve: true,
     mode: enforced}
  - {name: whatsapp,    kind: messaging, write_types: [], may_approve: true,
     mode: enforced}
  - {name: rung1_mcp,   kind: mcp,       write_types: [sales_invoice.submit],
     may_approve: false, mode: enforced}
```

Read against the Right Path, the example does exactly what the blocks promise. The drafting agent holds A2 Propose; it can stage and submit, and rung 6 stops it at approval. It cannot hold A3 on this write type in any tenant because the declaration does not grant it [E03 §04]. Approval is available on three channels, submission on two, and no channel lets the same actor approve what they submitted, because G4 catches that regardless of channel [E03 §04]. The `rate` and `schedule` gates are declared off with an ADR each: "invoicing happens when the business invoices; an operating window would be a policy nobody asked for" [E03 §04].

Two details of the example remain open and matter for the build. Whether the ₹2 lakh dual-control threshold is a declaration constant or a per-tenant parameter — a per-tenant parameter means a declaration version per tenant, "either the right model or an operational disaster" — is due for decision by 31 October 2026 [E03 §11 Q1]. And whether the 24-hour IRN window belongs in `reversal_plan.window_hours` or in the ERPNext adapter is open on the same date, because a statutory window in a declaration makes a rule change a declaration version for every tenant [E03 §11 Q2]. This memorandum recommends a tenant-scoped parameter overlay — a small, separately versioned object that may set only parameters the declaration marks `tenant_overridable`, whose version enters the Decision beside `declaration_version` — rather than a declaration fork per tenant **(build recommendation)**. That choice would need a new optional field and cannot be made without a recorded decision, since the proof record has no seventeenth field in which to carry an overlay version.

7.4 Identifiers and closed enumerations

Almost every value in a declaration is either an identifier with a fixed pattern or a member of a closed enumeration. Closedness is the point: an enumeration that could grow at runtime would let a declaration say something the kernel does not understand. Adding a value is a schema change under the version policy (§7.12), and adding a value to a disposition enum is breaking [O06 §03].

Identifiers.

Identifier	Pattern or form	Stable across versions?	Source
`declaration_id`	uuid	Yes	[E03 §02]
`declaration_version`	uint32, exactly last + 1, never `latest` in production	Each version immutable	[E03 §07; O06 §02]
`slug`	`^[a-z0-9_]{3,48}$`	Yes	[E03 §02]
Write type name	`<slug>.<verb>`, e.g. `sales_invoice.submit`	Yes; removal is a new version	[E03 §02]
`policy_version`	uint32, resolved by the kernel, never supplied by the caller	Each version immutable	[E02 §03; O06 §02]
ADR reference	`^ADR-[0-9]{4}$`	—	[E03 §02]
`control_id`	`^C-[A-Z]{2,6}-[0-9]{2}$`, e.g. `C-AT-01`	Yes	[E21 §04]
`control_version`, `regulation_version`, `profile_version`	uint32, last + 1	Each version immutable	[E21 §04–§05; E13 §02]
`regulation_id`, `profile_id`	`^[a-z]{2}-[a-z0-9-]{3,40}$`, jurisdiction-prefixed, e.g. `in-mca-audit-trail`	Yes	[E21 §05; E13 §02]
Control test id	`^T-[0-9]{2}$`, unique within a control	—	[E21 §04]
Profile check id	`^C[0-9]{2}$`, unique within a profile	—	[E13 §02]
`pack_id`	`^[a-z][a-z0-9]{2,15}1$`, e.g. `ledger1`	Yes	[E18 §04]

Note that a Control's identifier (`C-AT-01`) and a profile check identifier (`C01`) share a leading letter and mean different things. The patterns do not collide, but documentation and screens should always print the full form **(build recommendation)**.

Enumerations.

Enumeration	Values	Where used	Constraint	Source
`actor_kind`	`human · agent · system`	Permissions; F10	Set by the kernel at rung 1; never inferred, never caller-supplied	[E03 §02; E04 §03]
Autonomy	A0 Observe · A1 Draft · A2 Propose · A3 Act reversible · A4 Act bounded · A5 Act broad	`min_autonomy`; F11	Missing grant = A0; A3 requires a reversal plan; A5 requires dual control on the declaration	[E04 §03; E05 §11]
`blast_radius`	`one_record · one_counterparty · batch · tenant · cross_boundary`	Write types; rung 5; meter	`tenant` / `cross_boundary` force dual control; irreversible forces ≥ `one_counterparty`; basis is normative	[E03 §02; E10 §03]
Disposition	value · `out_of_scope` + ADR · `not_measurable` + reason + would_change_if	Every field	No blank	[E03 §03]
Decision `disposition`	`allow · refuse · escalate`	Rung outputs	"There is no fourth"	[E02 §03]

The remaining enumerations — reversal method, `after_window`, the nine gate names, field `type`, `classification`, `canonicalisation`, `salt_scope`, channel `kind` and `mode` — are listed with their blocks in §7.3.

The blast-radius enumeration deserves a second look because it does three jobs. It is a guardrail, checked at rung 5 against the measured radius of the resolved payload — "the rung that catches 'update one invoice' arriving as an unbounded filter" [E04 §03]. It forces controls: `tenant` and `cross_boundary` force dual control, and `cross_boundary` forces a non-empty proof binding [E03 §05]. And it is the meter basis, normatively and without negotiation: `one_record` bills per record, `one_counterparty` per counterparty, `batch` per batch, `tenant` per tenant and `cross_boundary` per crossing [E10 §03]. An author choosing a radius is therefore choosing, at the same moment, a control posture and a price line, and the Studio shows the resolved meter basis beside each write type before any integration code is written [E09 §04].

Refusal codes are the one enumeration a declaration references but does not own. The register is closed at 43 codes in three classes (policy 14, gate 19, platform 10), 28 active at L1, with wire spellings fixed in E02 [E02 §06]. Each gate lists in `refusal_codes[]` the codes it may raise, and a gate that raises an unregistered code fails CI [E03 §02]. Adding a code "takes a decision, not a commit" [E02 §06]. Chapter 4 carries the full register.

7.5 The three dispositions — never a blank

Every field in every block carries exactly one of three things [E03 §03]:

Disposition	Form	Rendered as
A value	The field's declared type	The value
`out_of_scope`	Plus `adr` matching `^ADR-[0-9]{4}$`	Hidden, with the decision reference available on hover
`not_measurable`	Plus `reason` and `would_change_if`, each ≥ 12 characters	"not measured", with both strings shown

The rule exists to defeat "n/a" and "tbd". The twelve-character floor is there because those two strings "are how an honest unknown becomes a shrug, and a shrug in a declaration becomes a hole in a proof that somebody discovers during an audit" [EBB EB-15 §5]. The Studio rejects them with the sentence "An honest unknown needs a sentence. What would change your answer?" [E06 §06]. Dispositions are not decoration for the compiler either: a `not_measurable` field renders differently from a value, so the declaration's honesty is visible to the person using the product [EBB EB-15 §5].

`not_measurable` is meant to be used: "a declaration full of `not_measurable` fields is a declaration telling the truth about an immature process, which is more useful than one full of invented values" [E03 §03]. The same three dispositions govern verifier profiles, Controls, Regulations and packs, and there is no fourth anywhere in the system — no blank and no "partially compliant" [E13 §02; E21 §09].

7.6 Validation — schema, eleven semantic rules and seven lint rules

Validation runs in three layers, in CI, with no platform running [E03 §05–§06].

1. **JSON Schema** catches shape: required fields, types, patterns, closed enums, `additionalProperties: false` [E03 §01; ETS §08].
2. **Semantic rules** catch meaning. They are failures, never warnings.
3. **Lint rules** catch smells. They are warnings.

Rule	Assertion	Mirrored in the database?
L0-S01	Every write type named in `permissions` exists in `write_types`	—
L0-S02	Every gate named in a write type exists in `gates`	—
L0-S03	Exactly one of `reversal_plan` or `irreversible: true` per write type	Yes, `CHECK`
L0-S04	`irreversible: true` forces `blast_radius` ≥ `one_counterparty`	Yes (IM)
L0-S05	`tenant` or `cross_boundary` radius forces `dual_control: true`	Yes (IM)
L0-S06	`cross_boundary` forces `proof_binding` present and non-empty	—
L0-S07	Every field in `proof_binding` exists in `shape`	—
L0-S08	Every channel in `channels` is registered for the tenant	—
L0-S09	`policy_exempt` requires an `adr` and is never set on an irreversible write type	—
L0-S10	Every write type in `channels` exists in `write_types`, and every write type appears on at least one channel	—
L0-S11	`may_approve` is `false` wherever `actor_kind = agent`; `min_autonomy` is one of A0–A5	—

Sources: [E03 §05]; database mirroring [E03 §05; IM §4.5, which states three rules are mirrored as constraints].

L0-S10 says "an unreachable write type is a defect, not a draft"; L0-S11 is the declaration-time half of the rule that an agent may never approve [E03 §05, §10 D7]. The six never-skippable gates are a schema-level requirement in addition: a declaration missing any of them "fails here and cannot be argued with" [E09 §04].

Lint	Fires when	Why it matters
LN-01	The declaration exceeds 900 lines	It has become a program; split it
LN-02	`would_change_if` or `reason` under twelve characters	"tbd" in a costume
LN-03	A `not_measurable` field unchanged across three versions	Nobody is working on it; fix it or scope it out with an ADR
LN-04	`gates[]` order differs from another write type with the same gate set	Gate order is semantic; a difference should be deliberate
LN-05	`intent` under forty characters or repeats `label`	The auditor's sentence has not been written
LN-06	A financial or personal field is absent from `proof_binding` with no ADR	Possibly correct, but it should be a decision
LN-07	`rate`, `budget` or `schedule` disabled without an ADR	Commercial gates may be off; turning them off should be recorded

Source: [E03 §05].

A failing semantic rule refuses at validation with the rule, block and field, in words: "L0-S03 — `sales_invoice.submit` declares neither a reversal plan nor `irreversible: true`. Every write type must say which it is" [E03 §06]. The Studio shows failures against the block that caused them rather than as a list of codes [E06 §06].

The validator is a pure library with no platform dependency, so the same code runs in the Studio, in the author's CI and in the registry on certification. This memorandum recommends a Python reference validator beside the JSON Schema and a TypeScript port for in-browser feedback, both run against one shared fixture corpus whose expected findings they must reproduce exactly **(build recommendation)**; the compiler language remains open [ETS §04].

7.6.1 The database mirrors

Three of the rules are enforced again by the registry's own schema, because an application-layer rule is a rule a migration script will eventually violate [EBB EB-15 §6; IM §4.5]. A sketch of the write-type registry table (column names are **(build recommendation)**; the constraints are from source):

```
CREATE TABLE write_type_registry (
  declaration_id      uuid        NOT NULL,
  declaration_version integer     NOT NULL CHECK (declaration_version >= 1),
  name                text        NOT NULL,
  blast_radius        text        NOT NULL CHECK (blast_radius IN
      ('one_record','one_counterparty','batch','tenant','cross_boundary')),
  dual_control        boolean     NOT NULL,
  irreversible        boolean     NOT NULL DEFAULT false,
  reversal_plan       jsonb,
  policy_exempt       boolean     NOT NULL DEFAULT false,
  compiled_digest     bytea       NOT NULL CHECK (octet_length(compiled_digest) = 32),
  PRIMARY KEY (declaration_id, declaration_version, name),
  -- L0-S03: exactly one of a plan or an explicit irreversible flag
  CHECK (irreversible <> (reversal_plan IS NOT NULL)),
  -- L0-S04: irreversible forces radius >= one_counterparty
  CHECK (NOT irreversible OR blast_radius <> 'one_record'),
  -- L0-S05: tenant and cross_boundary force dual control
  CHECK (blast_radius NOT IN ('tenant','cross_boundary') OR dual_control),
  -- L0-S09: policy_exempt never on an irreversible write type
  CHECK (NOT (policy_exempt AND irreversible))
);
-- Append-only: the registry role holds INSERT and SELECT only; no UPDATE, no DELETE.
```

E03 phrases the reversal check as "mirrored as a database `CHECK`" and the Build Book as `CHECK (irreversible OR reversal_plan IS NOT NULL)` [E03 §05; EBB EB-15 §6]. The Build Book form admits a row with both a plan and the flag; the exclusive form above matches the rule as E03 states it — "exactly one" — and is recommended.

7.7 Policy rules — what rung 4 evaluates

The declaration's gates are the fixed, named checks that run at rung 3. Rung 4 is different: `evaluate(request, declaration_version, policy_version)` runs versioned rules over a projection of declared fields and returns `allow`, `refuse` or `escalate`, "never a score" [E04 §03]. Two refusal codes belong to it: `POLICY.DENIED_BY_RULE`, which names the rule that denied, and `POLICY.NO_RULE_MATCHED`, whose operator action is "author a rule; never widen a default" [E02 §06].

Three properties are fixed by source. The policy service has no read path to records; its input is a projection of "the fields the declaration's `proof_binding` and the policy's rules reference, and nothing else", so a rule can depend only on what is declared, versioned and therefore inside the proof [EBB EB-03 §6]. The evaluator is deterministic and clock-free, so every refusal in the reference tenant replays identically at its `as_of` (conformance test C4; eval EV3) [E02 §08; EBB EB-12]. And `policy_version` is resolved by the kernel, recorded in every Decision and in F08, and never supplied by a caller [E02 §03; O06 §02].

The rule language is open. The stack document recommends "own rules-as-data evaluator" (`@elan1/policy`) with Cedar as the principal alternative and CEL and OPA/Rego (with `time.now_ns` and `http.send` disabled) as further alternatives, and requires an ADR to record the choice [ETS §01, §08, §13]. This memorandum recommends the own evaluator over a rule format that is itself declarative data, so that a rule set is authored, validated, simulated, certified and versioned exactly as a declaration is **(build recommendation)**:

```
# policy rule set — sketch (build recommendation)
policy_id: rung1-erpnext-sales
policy_version: 11
```

```
applies_to: {declaration_id: 7b0e…, versions: [7, 8]}
projection: [grand_total, customer.credit_limit, customer.exposure, posting_date]
rules:
  - id: R-SI-01
    when: {write_type: sales_invoice.submit}
    if:   "customer.exposure + grand_total > customer.credit_limit"
    then: refuse              # -> POLICY.DENIED_BY_RULE, rule R-SI-01 named
  - id: R-SI-02
    when: {write_type: sales_invoice.submit, actor_kind: agent}
    if:   "grand_total > 500000"
    then: escalate            # a human decides; rung 7 not reached
  - id: R-SI-99
    when: {write_type: sales_invoice.submit}
    then: allow
default: no_rule_matched      # never a default allow
```

The final line is the invariant expressed as data: a rule set whose evaluation falls through produces `POLICY.NO_RULE_MATCHED`, and the validator refuses any rule set that declares `default: allow` **(build recommendation, implementing E02 §06)**. A `policy_exempt` write type skips rung 4 entirely; it requires an ADR and is never valid on an irreversible write type [E03 §02].

7.8 Controls and Regulations — the two families that complete the ontology

A declaration in v1 is a *write-type* declaration: it governs changes to one record type in one system of record. Two further entity families complete the ontology: **Controls** and **Regulations** [E03 §09; E21 §01]. They are specified in full in E21 and are deliberately not built at Level 1.

> **Timing.** E03 dated both families "Phase 3, with L2". E14 then froze Phase 3 to four additions — signing, anchoring, executing profiles and evidence as a priced product — none of which is a family. E21 resolves the conflict: the families are **Phase 4**, and a document dating them to Phase 3 "is v3.0-early and means Phase 4" [E21 §01; E13 §08 D10]. The reasoning is E03's, unchanged: a control family that maps to a regime is a promise about what a regulator expects, and it becomes safe to make only after an outside auditor has accepted a bundle — which is L3's entry condition [E03 §09; E21 §01].

The families change the seven blocks by exactly one enumeration value (`canonicalisation: elan1_lp_sd_v1`) and one scope value (`salt_scope: per_field`), with `redaction_profile` becoming live. "A family that required the seven blocks to change would prove the seven blocks were wrong" [E21 §02, §10 D2]. Gate names become, in addition, the binding target of a control row [E21 §02].

7.8.1 The Controls family

A Control is a named control, the write types it covers, what it expects of the evidence, and its test. "One control, one assertion. A control that asserts two things is two controls" [E21 §04].

Group	Field	Type	Meaning
identity	`control_id`, `control_version`	string, uint32	`C-AT-01`; last + 1, never `latest`
	`name`, `statement`	string	Register name; one present-tense sentence naming what always holds
	`owner_role`	string	Accountable role, never a person
	`control_kind`	enum	`preventive · detective · corrective`
	`operating_mode`	enum	`continuous · per_event · periodic` (periodic declares its period)
	`status`	enum	`draft · certified · superseded · withdrawn`
coverage	`declaration_ref[]`	array	(`declaration_id`, version) pairs

Group	Field	Type	Meaning
	`write_types[]`	array	Covered names; `*` must be justified in `scope_note`
	`gates[]`	array	Gates that must be declared and enabled; required for preventive controls
	`actor_kinds[]`, `scope_note`	array, string	Which of human/agent/system; what is deliberately outside
evidence_expectation	`record_fields[]`	array	Which of F01–F16 must be present and resolvable
	`proof_binding_fields[]`	array	Payload fields that must be inside F14
	`gate_results_required`, `refusals_required`, `approval_record_required`	boolean	Whether gate rows, refusals and the approval record must be in scope
	`continuity_scope`	enum	`full_chain · declared_write_types · range`
test	`test_id`, `assertion`, `population`	string	The predicate, "written so a second implementer can write it without asking us", and its denominator
	`exception_handling`	enum	`none_permitted · exceptions_listed · exceptions_with_adr`
	`on_fail`	enum	`fail_integrity · fail_profile · incomplete`, never collapsed
	`operated_throughout`	boolean	Whether the test asserts operation across the whole period
certification	`author`, `certifier`, `certified_at`, `supersedes`	string, RFC 3339, uint32	Certifier is never the author

Source: [E21 §04].

`operated_throughout` is the field that makes the family worth having. "Control C7 operated throughout the period" is the sentence an assurance engagement needs; here it is a query over an unbroken chain "rather than a folder of screenshots" [E21 §04].

7.8.2 The Regulations family

A Regulation is a named regime, the clauses in it that constrain a write (not every clause in the instrument), the controls it expects, and the verifier profile a bundle must claim to satisfy it [E21 §05].

Group	Field	Meaning
regime_identity	`regulation_id`, `regulation_version`	`in-mca-audit-trail`; last + 1
	`regime_name`, `issuer`, `jurisdiction`, `instrument_ref`	As the issuer names and cites it; ISO 3166-1 alpha-2
	`rule_as_at`, `in_force_from`, `in_force_to`	Date of the regime text used; the obligation's own dates
clauses[]	`clause_ref`, `clause_text`	Citation as spelt by the issuer; the obligation quoted verbatim — paraphrase is a validation failure
	`obligation_kind`	`record_keeping · human_oversight · retention · disclosure · segregation · non_repudiation`
	`expects_controls[]`	(`control_id`, `control_version`) pairs
	`disposition`	`bound · out_of_scope + adr · not_measurable + reason + would_change_if`
	`severity`	`blocking · material · advisory`
profile_binding	`profile_id`, `profile_version`	The E13 verifier profile that executes this Regulation

Group	Field	Meaning
	`clause_to_check_map[]`	Each `clause_ref` to the check ids that serve it — generated, then reviewed, never hand-maintained
	`min_assurance_level`	2 for any regime asserting non-repudiation; 3 where a clause needs a party outside both organisations

Source: [E21 §05].

The relationship between a Regulation and a verifier profile is "one of expression, not duplication". The Regulation is the readable object a compliance officer reviews; the profile is the executable object the offline verifier runs. `clause_to_check_map[]` is compiled from the two, and a mismatch is a validation failure, so the pair cannot drift [E21 §05].

7.8.3 Semantic rules for the two families

Rule	Assertion
L0-C01	Every (`declaration_id` , version) in a control's coverage resolves to a certified declaration
L0-C02	Every gate in `coverage.gates[]` is declared and enabled for every covered write type
L0-C03	Every field in `evidence_expectation.proof_binding_fields[]` is in that declaration's `proof_binding.fields[]`
L0-C04	A preventive control names at least one gate; a detective control names at least one test with a stated population
L0-C05	`write_types: *` requires a `scope_note` of at least forty characters
L0-R01	Every clause carries a disposition; a blank is a validation failure, never a draft state
L0-R02	Every bound clause names at least one (`control_id` , version) that resolves to a certified control
L0-R03	Every certified control is named by at least one clause, or is marked standalone with a reason
L0-R04	`clause_to_check_map[]` resolves both ways: every clause has a check, every check serves a clause
L0-R05	`min_assurance_level` is not above the level of any tenant bound to the Regulation, or the binding is refused with the shortfall printed
L0-R06	A `blocking` clause may not be `not_measurable`

Source: [E21 §05].

L0-R06 carries the sentence the Studio prints when it fires: "A blocking obligation we cannot measure is not a pack. Bind it, scope it out with an ADR, or stop" [E21 §06]. L0-R03 prevents the opposite failure — "an orphan control is a control nobody asked for" [E21 §05].

7.9 The five-link chain, worked end to end

The ontology is finished when "a sentence a regulator would write can be followed, by name, to a field in a record" [E21 §03]. The chain has five links:

> regulation clause → control → gate → write type → proof field

Link	From → to	Resolved by	Fails when
1	`clause_ref` → `control_id`	Name and version	The clause expects no control — then `out_of_scope` with an ADR, or the Regulation is incomplete
2	`control_id` → `gates[].name` in a declaration	Name, within a named (`declaration_id` , version)	The control names a gate nobody declared — a validation failure, never a warning
3	gate → `write_types[].name`	The gate appears in that write type's ordered `gates[]`	The gate is declared but unused by the write types the control claims to cover
4	write type → record fields F01–F16	The write type produces records; the fields are structural	The control expects a field the record does not carry — a design error in the control

Link	From → to	Resolved by	Fails when
5	write type → `proof_binding.fields[]`	Names resolved against block 2	The control expects a payload field in F14 and it is not there — "the most common real failure", caught at authoring, not at audit

Source: [E21 §03; IM §6.7].

"A control that cannot name its gate is prose." Where an author cannot name the gate, the honest disposition is `not_measurable` with a reason and a `would_change_if` [E21 §03]. The five links are why a Regulation object is worth building at all: without it, "we comply with the MCA audit-trail rule" is a claim in a deck; with it, the claim is a query — resolve every clause, follow it to its checks, run the checks over the bundle, print the ones that are not bound [E21 §03].

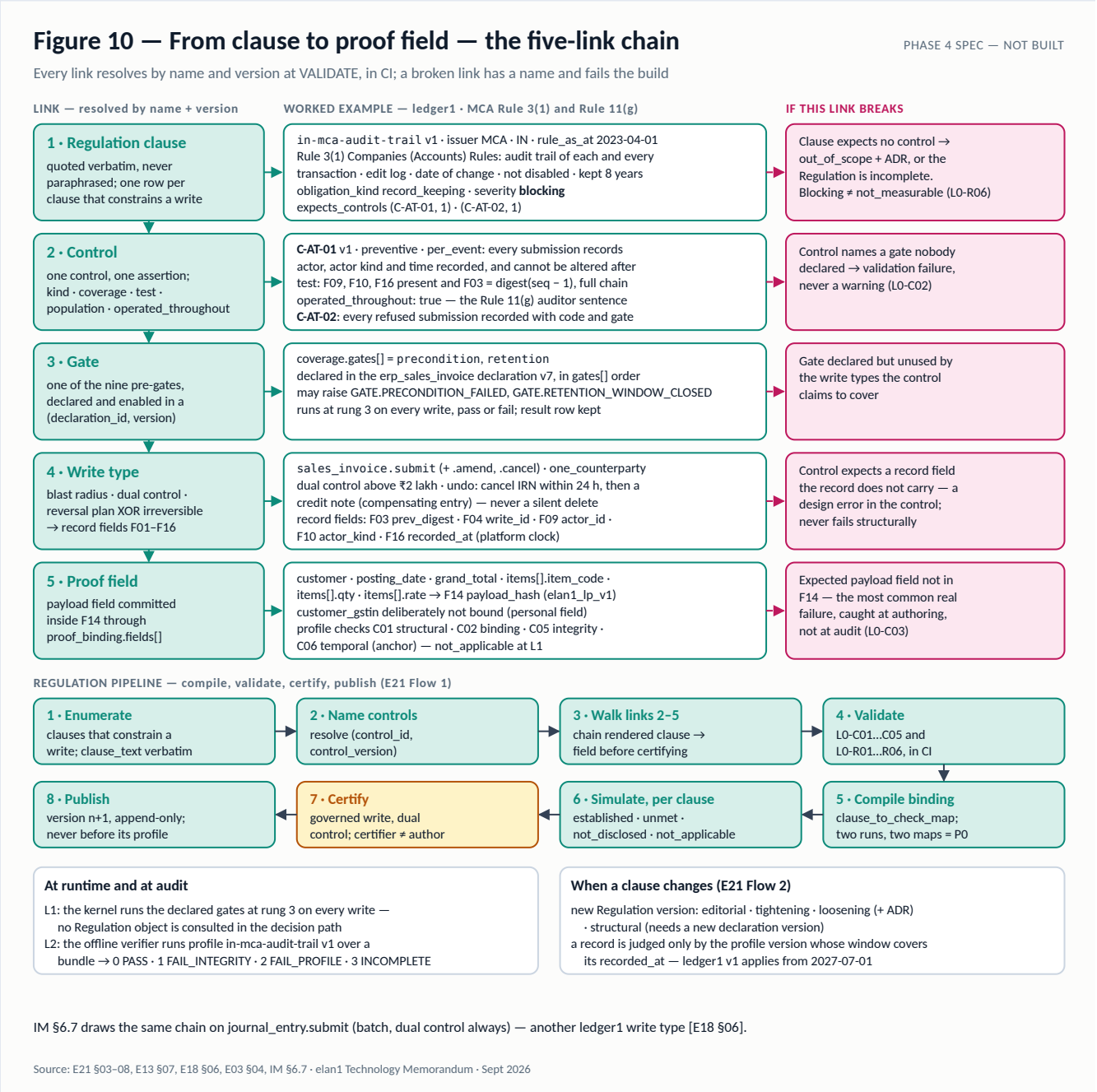


Figure 10 — The five-link chain from a regulation clause to a proof field, each link resolved by name and version in CI, worked for the MCA audit-trail rule down to a runtime gate and a hash-chained record.

7.9.1 The worked example — MCA Rule 3(1) to a runtime gate and a proof

The regime is the Indian audit-trail obligation for accounting software. The proviso to Rule 3(1) of the Companies (Accounts) Rules, 2014, in force since 1 April 2023, requires software that records an audit trail of each and every transaction, creates an edit log of each change made in the books of account with the date of the change, and cannot be disabled; the IM adds that the trail must be preserved for at least eight years. Rule 11(g) of the Companies (Audit and Auditors) Rules obliges the statutory auditor to report whether the trail existed, operated throughout the year, was not tampered with and was preserved [IM §3.6; IM §10; R03 §06]. R03 marks the precise preservation period and the wording of the auditor's reporting clause as needing counsel confirmation before launch, and E11 declines to assert the clause reference at all [R03 §06; E11 §08]. The example below follows E21 §08, which works one clause through one control, one gate, one write type and one field.

Link 0 — the Regulation object. Clause text is shown here as a paraphrase marker only; the certified object carries it verbatim, because paraphrase is a validation failure [E21 §05].

```
regulation_id: in-mca-audit-trail
regulation_version: 1
regime_name: "Companies (Accounts) Rules, 2014 — audit trail for accounting software"
issuer: "Ministry of Corporate Affairs"
jurisdiction: IN
instrument_ref: "Rule 3(1), proviso"
rule_as_at: 2023-04-01
in_force_from: 2023-04-01
clauses:
  - clause_ref: "Rule 3(1), proviso — audit trail of each and every transaction"
    clause_text: "<quoted verbatim from the instrument>"
    obligation_kind: record_keeping
    expects_controls: [[C-AT-01, 1], [C-AT-02, 1]]
    disposition: bound
    severity: blocking
  # … the edit-log, date-of-change and non-disablement clauses follow
profile_binding:
  profile_id: in-mca-audit-trail
  profile_version: 1
  clause_to_check_map: []        # generated at step 5 of Flow 1, then reviewed
  min_assurance_level: 2
```

Link 1 — clause to control. The clause expects two controls. C-AT-02 asserts that every refused submission is recorded with its refusal code and the gate that raised it, with `refusals_required: true` [E21 §08]. C-AT-01 is the preventive control:

```
control_id: C-AT-01
control_version: 1
name: "Audit trail — governed invoice changes"
statement: "Every submission of a governed sales invoice records the actor, the actor
            kind and the time it was recorded, and the record cannot be altered
            afterwards."
owner_role: "Finance controller"
control_kind: preventive
operating_mode: per_event
status: certified
coverage:
  declaration_ref: [[erp_sales_invoice, 7]]
  write_types: [sales_invoice.submit, sales_invoice.amend, sales_invoice.cancel]
  gates: [precondition, retention]
  actor_kinds: [human, agent, system]
  scope_note: "Changes made directly in ERPNext outside the elan1 write path are not
               covered; they are the accounting system's own audit trail."
evidence_expectation:
  record_fields: [F04, F09, F10, F16, F03]
  proof_binding_fields: []           # see 7.9.2 — this is where authors fail
  gate_results_required: true
  refusals_required: true
  approval_record_required: true
  continuity_scope: full_chain
test:
  - test_id: T-01
    assertion: "For every record in the population, F09, F10 and F16 are present and
                F03 equals the digest of seq - 1."
```

```
    population: "Every record whose F05 is one of the three covered write types, over
                 the full chain."
    exception_handling: none_permitted
    on_fail: fail_integrity
    operated_throughout: true
certification: {author: "<named human>", certifier: "<a different named human>"}
```

Values for `statement`, `control_kind`, `operating_mode`, coverage, `record_fields` and the test are from E21 §08; `owner_role`, `scope_note`, `on_fail` and the flags not stated there are illustrative.

Links 2 and 3 — control to gate to write type. C-AT-01 names `precondition` and `retention`. Validation (L0-C02) confirms both are declared and enabled in declaration `erp_sales_invoice` version 7, and that each appears in the ordered `gates[]` of every covered write type [E21 §08]. The IM's version of the same figure binds `dual_control`, `precondition` and `retention`, and uses `journal_entry.submit` as the write type [IM §6.7]; both are valid readings of one control covering the books of account, and the pack (Chapter 13) covers both write types.

At runtime these two gates are where the clause becomes enforcement. The retention gate refuses a change to a record outside its declared window (`GATE.RETENTION_WINDOW_CLOSED`) [E02 §06]. The precondition gate carries the audit-trail state predicate that rung1 specifies as its G8: the ERPNext audit-trail feature is enabled before and after the change, no changeset disables or narrows tracking, and no migration updates or deletes version rows [R03 §06]. In kernel terms that predicate raises `GATE.PRECONDITION_FAILED`, because the closed register has no audit-trail-specific code; rung1's `GATE_AUDIT_TRAIL_DISABLED` is a consumer-side name for the same outcome (see the conflicts block).

Link 4 — write type to record fields. `sales_invoice.submit` produces records, and F03 `prev_digest`, F04 `write_id`, F09 `actor_id`, F10 `actor_kind` and F16 `recorded_at` are structural in every one of them [E02 §03]. This link cannot fail structurally; it fails only if a control expects a field the frozen record does not have, and there is no seventeenth field at any level [E21 §03; IM §5.6].

Link 5 — write type to proof field. The profile requires "the fields that constitute the posted entry" inside F14 so that "what changed" is recoverable from the declaration plus F14 [E13 §07]. For `sales_invoice.submit` the declaration binds `customer`, `posting_date`, `grand_total`, `items[].item_code`, `items[].qty` and `items[].rate` [E03 §04]. If the control listed `taxes` in `proof_binding_fields[]`, L0-C03 would fail at authoring, naming the control, the declaration version and the field — the failure E21 calls the most common in practice.

7.9.2 The same chain at runtime

The five links resolve in CI. At runtime the chain is exercised by an ordinary governed action. A sketch of one, for an agent at A2 submitting a ₹2,40,000 invoice (sequence numbers illustrative, following E04 Flow 3):

```
rung 0b  STOP          tenant live
rung 1   IDENTITY      actor=ar-agent  actor_kind=agent (kernel-set)
rung 2   WRITE DOOR    erp_sales_invoice v7 · sales_invoice.submit · channel rung1_mcp
rung 3   PRE-GATES     consent pass · residency pass · retention pass (period open)
                       counterparty pass · precondition pass (audit trail enabled,
                       no duplicate number) · dual_control pending · budget pass
rung 4   evaluate()    allow @ policy_version 11
rung 5   GUARDRAIL     measured radius one_counterparty = declared
rung 6   AUTONOMY      A2 Propose -> escalate; obligation created, expiry 4 h
rung 8   PROOF         seq 4099  outcome escalated  (F12 zero-UUID, F13 0)
rung 9   METER         quantity 0
-- human approver resolves the obligation: a governed write, seq 4100 --
rung 1-8 resubmission  obligation_ref set; G4 finds a second named actor
rung 7   ACT           snapshot → sandbox → gates re-run → live via single-use
                       credential → gates re-run → commit + one outbox row
rung 8   PROOF         seq 4101  F05 sales_invoice.submit  F07 7  F08 11
                       F09 ar-agent  F10 agent  F14 payload_hash  F16 recorded_at
rung 9   METER         quantity 1, basis per counterparty
```

The record at 4,101 is the object the control's test reads. At L1 a check over it — contiguity, F09/F10/F16 present, F03 equal to the digest of 4,100 — can be run by the tenant against elan1's store, and E11's two-way reconciliation against ERPNext's own audit trail is what makes it meaningful to an outsider [E11 §07]. At L2 the same record is signed and anchored,

exported in an eight-member bundle claiming profile `in-mca-audit-trail` v1, and checked offline. The profile's clause map sends the transaction clause to checks C01 and C02, the edit-log clause to C03, the date clause to C04, and the non-disablement clause to C05 and C06 [E13 §07].

> **What L1 cannot answer.** C06 tests that the trail has not been silently truncated. A chain cut at the tail is internally consistent and looks complete; only an external time anchor makes the non-disablement clause checkable. On an L1 dry run C06 reads `not_applicable — assurance level 1` on the face of the report, never a silent PASS. It is the clearest single demonstration of what L2 buys [E13 §07; IM §5.4].

What the example does not establish is stated in the specification and must be stated by the pack: that the invoice was commercially genuine; that changes made directly in ERPNext outside the write path are covered; or that the MCA has reviewed this reading of the rule. "It is our reading, dated, with its author and certifier named" [E21 §08].

7.10 When a clause changes

A clause change must reach profiles and reports "without breaking a single existing record" [E21 §07]. The author pastes the amended text, sets `rule_as_at`, and the system creates Regulation version *n* + 1 and computes the blast radius of the change at each of the five links — clauses changed, controls expected, gates named, write types carrying them, tenants running them [E21 §07]. The change is then classified:

Class	Means	Effect downstream
Editorial	Citation or wording changed; obligation did not	New Regulation version; controls unchanged; profile version bumped with identical checks
Tightening	The obligation expects more	New Regulation, control and profile versions; `applies_to_records_from` set to the obligation's own effective date, never earlier
Loosening	The obligation expects less	As tightening, plus an ADR — "the change an auditor will ask about"
Structural	Requires evidence the declaration cannot produce — a field not in `shape`, a gate nobody declared	A new declaration version; the only class touching blocks 2 or 5; tenant-visible, with its own runbook

Source: [E21 §07].

> **The propagation rule.** A record is judged by the profile version whose `applies_to_records_from`/`applies_to_records_to` window covers its `recorded_at`, and by no other. A clause that tightened in March 2029 does not make a February 2029 record non-conforming. A verifier asked to judge a range spanning a boundary refuses the single-profile run and offers the two sub-ranges [E21 §07]. Retro-judging a record under a later rule "is the compliance equivalent of back-signing, and it is refused for the same reason" [IM §5.7; E13 §08 D6].

Setting `applies_to_records_from` earlier than the obligation's effective date refuses with "This would judge records by a rule that did not exist when they were written"; a Regulation may never be certified ahead of the profile that executes it; and a blast-radius computation that is unavailable creates nothing partial [E21 §07, §10 D7].

7.11 The pipeline — author, validate, simulate, certify, compile, resolve

The lifecycle has six stages that cannot be skipped [E03 §06; E09 §04; IM §6.6]. Mapped onto the conventional build vocabulary of lint, type-check, test, sign-and-version and publish:

Stage	Conventional name	What runs	Output	Refusal or failure
1 AUTHOR	—	Studio or a file in the repository; the compiled surface preview renders beside the source	A draft	"n/a"/"tbd" rejected at entry [E06 §06]
2 VALIDATE	lint + type-check	JSON Schema; L0-S01–S11 (and L0-C/L0-R for the families); LN-01–07	Valid, or the first failing rule with block and field, plus warnings	`POLICY.DECLARATION_INVALID` in the Studio flow [E09 §04]
3 SIMULATE	test	All nine rungs against the reference tenant's synthetic fixtures with a stub writer; never production	Per write type: decision, gate-result rows, reachable refusal codes, resolved radius and meter basis	"This simulation produced no refusals. Add a fixture that fails" [E09 §04]
4 CERTIFY	sign + version	A named human certifies; certification is a governed write with `dual_control: true`	Version N = last + 1; a record in the chain	`GATE.DUAL_CONTROL_SAME_ACTOR` if the certifier is the author
5 COMPILE	build	Deterministic, content-addressed compiler	Surface plan, write-type registry entries, meter basis per write type, [L2] verifier-profile stub, `compiled_digest`	Two runs, two digests → build fails as a P0
6 RESOLVE	publish / consume	Rung 2 resolves `(declaration_id, explicit integer version)`	The descriptor used by the Right Path	`POLICY.DECLARATION_UNRESOLVED` for an absent or uncertified version

Sources: [E03 §06; E06 §06; E09 §04; EBB EB-15 §2].

The simulation stage cannot touch production because the control plane has no read path to any record store — "not a restricted one, none, enforced at the network layer" [E05 §04; E06 §06]. A simulation "with no refusals in it is a simulation that has not been designed" [E06 §06].

Certification is the signature at L1. There is no signing key at Level 1 [O01 DR-006; E07 §02]. What makes a version authoritative is that a named human certified it through a governed write, which climbs the nine rungs under dual control and appends its own record, so the chain itself carries who certified which digest and when [E03 §06; E09 §04]. The `declaration.certified` webhook carries `declaration_id`, version and certifier [E05 §09]. Cryptographic signing of compiled artefacts is not specified in the corpus; this memorandum recommends that from L2 the registry publish, per certified version, the `compiled_digest` inside a signed registry manifest using a key separate from the record-sealing keys, under the custody rules of Chapter 12 **(build recommendation)**.

Determinism is a P0 property. The compiler must produce the same digest for the same declaration for ever, because "a non-deterministic compiler makes the 2035 reconstruction claim false" [E03 §06]. The Build Book lists what COMPILE emits: a surface plan (which components appear, in what order, bound to which fields, behind which write types), registry entries for rung 2, the meter basis, and for packs a verifier-profile stub [EBB EB-15 §2]. Idempotency of registration is keyed on `(declaration_id, version, compiled_digest)`, so a retry after `PLATFORM.REGISTRY_UNAVAILABLE` with the same content returns the same acceptance [E09 §04].

The recommended CI job for a declaration repository, with every step runnable offline **(build recommendation)**:

```
# .ci/declaration.yml — sketch
steps:
  - run: elan1-decl canonicalise decl/*.yaml --out build/   # strict YAML -> JSON
  - run: elan1-decl validate build/*.json --schema-major 1  # schema + L0-S01..S11
  - run: elan1-decl lint build/*.json --fail-on none         # LN-01..LN-07 as warnings
  - run: elan1-decl simulate build/*.json --tenant ref --fixtures fixtures/
         --require-refusal-per-gate                          # E18 T4 discipline
  - run: elan1-decl compile build/*.json --out dist/ && elan1-decl compile
         build/*.json --out dist2/ && diff -r dist dist2     # determinism: P0
  - run: elan1-decl links build/*.json --controls ctl/ --regulations reg/
         --profile prof/                                     # five-link resolution
# certification is NOT a CI step: it is a governed write by a named human
```

Tooling follows the stack document: JSON Schema 2020-12 with Python `jsonschema` recommended and Ajv as the TypeScript alternative; the semantic rules as code beside the schema; the surface compiler in Python or TypeScript, with the choice open, and a hard requirement that it be deterministic and content-addressed [ETS §04, §08].

7.12 Versioning and pinning of `declaration_version`

Every change produces a new integer version, exactly last + 1, and versions are immutable [E03 §07].

Change	Effect
Any change at all	New version; never edited in place
Adding a write type	New version; no effect on writes already recorded
Removing a write type	New version; the old version still resolves, so old writes still explain themselves
Tightening a gate	New version; takes effect at the next write, never retroactively
Loosening a gate	New version plus an ADR — "the artefact an auditor will ask for"
Changing `proof_binding`	New version; old digests stay valid under the old binding
Correcting a typo in `description`	Still a new version — "there is no such thing as a small change to a governed declaration"

Source: [E03 §07].

Every version is retained verbatim, for ever, append-only and monotonic, and there is no garbage collection [E03 §07, §10 D9]. The reason is a chain of dependencies, each link of which is load-bearing: a record carries the declaration version and the lexicon hash → a verifier must resolve both → resolving them by calling elan1's API would not be offline → so an evidence bundle carries them → so every version must still exist verbatim [E03 §07].

Pinning. Consumers pin an explicit integer version and write it into their own configuration. `latest` exists in sandbox only; in production it is a *rejection* (4xx, nothing entered the kernel), not a refusal [E05 §10; O06 §05]. A pinned version that does not exist or is uncertified refuses with `POLICY.DECLARATION_UNRESOLVED`, recorded and metered at zero [O06 §05]. `GET /v1/versions` returns the four numbers a consumer depends on — port major, declaration schema major, refusal register version and live policy version — and every kernel response echoes `declaration_version` so the consumer can fail its own build on a mismatch [O06 §05]. The pinned id and version appear on every intent and in F06 and F07; the resolved policy version in F08 [O06 §05].

Declaration versions do not expire. Declarations are forward-only and never migrated, so a Governed Action written under version 4 resolves against version 4 for ever [O06 §02]. Plane P4 is described as "forward only, never migrated" for the same reason [O01 §03].

The schema is versioned separately, by a major integer in the identity block with which `compiled_digest` moves. A new optional field, a new gate name or a relaxed rule is additive; a new required field, a renamed block or a rule tightened so that a certified declaration would no longer validate is breaking and takes a new major with dated notice. Renaming block 7 from `doors` to `channels` is breaking and must land before L1 GA, while no external party has yet authored a declaration [O06 §02–§03].

Tool descriptions are pinned by hash in the declaration, so changing one is a new certified version — "we pay a certification for a typo fix and consider that the correct price" — and the MCP tool surface is versioned as `declaration@version` [E05 §04; O06 §02].

7.13 How packs extend declarations without code

A pack is one declaration, one certified verifier profile and one regime-specific export mapping, aimed at one buyer with one auditor under one dated obligation. The kernel does not know what a pack is, and the declaration registry has no pack table [E18 §01]. Chapter 13 covers packs in full; this section states only what the declaration layer lets a pack do and forbids it to do.

A pack may	A pack may not	Source
Reference an existing declaration by (`declaration_id`, version) — `ledger1` uses rung1's ERPNext declaration and has "no new ontology"	Define its own ontology or its own record fields	[E18 §04, §06]
Author a new declaration where no consumer declaration covers the writes (as `collect1` would for calls)	Add a gate: "the nine gates already exist; a new regime parameterises them rather than adding to them"	[E27 §02; IM §5.6]
Set gate parameters, gate order and the enablement of `rate`, `budget`, `schedule`	Disable any of the six never-skippable gates	[E03 §02; E18 §04]
Add rung 4 rule sets at a new `policy_version`	Put a model in the decision path	[E04 §03]
Add Controls and a Regulation (Phase 4) and bind a verifier profile	Add a seventeenth record field, "at any level, ever"	[E21; IM §5.6]
Add refusal fixtures and tests	Add a refusal code without a register decision	[E02 §06; E18 §04]

"Six steps, no code. If a pack needs code, it needs a component" in the shared renderer layer [ESB Ch.11 §6]. The marginal cost of a pack is the reading and the binding — person-weeks of a regulatory engineer's time, not kernel releases [IM §5.6].

7.14 The Palantir comparison

Palantir is the reference because its design is the nearest to elan1's and because every technical buyer will ask [E03 §08]. The mapping is close on purpose; the right-hand column is the argument, and it is honest in both directions.

Palantir concept	elan1 equivalent	Where elan1 goes further — or is behind
Object types	`shape`	Field classification drives consent, the proof binding, redaction and disclosure; sensitivity is part of the ontology, not a later tag
Link types	`shape.links[]`	**Behind.** Palantir's link semantics are richer and its tooling is a decade old
Action types	`write_types`	Every action declares a blast radius and exactly one of a reversal plan or an explicit irreversible flag; Palantir models the action but does not force the author to state how the world is put back
Submission criteria	`gates`	Ordered, versioned, pure, clock-free, six of nine never skippable, each mapped to a typed refusal code with an operator action. "Submission criteria are validation; gates are a control surface"
Action log	The hash-chained record	Sixteen fields per write including refusals, chained per tenant; signed and anchored at L2, verifiable offline by someone who does not trust elan1. "An action log is a log; a record is evidence"
Interfaces	`identity.system_of_record` plus the adapter	Narrower and hashed: one declaration, one system of record, one lexicon, so a term's meaning is versioned with the write
Functions	Rung 4 `evaluate()`	Versioned rules over declared fields with no model in the decision path at any level; "Palantir's functions may call a model. This is the whole difference"
—	`permissions` and A0–A5	Autonomy is a kernel concern at rung 6; a missing grant is A0
—	`channels`	Which surface may carry which write, and whether it is enforced or attested advisory, is declared and recorded
—	`proof_binding`	The ontology says what the record commits to cryptographically, so the proof is designed at authoring time
—	The meter basis	The billable atom is the governed action, the same object the auditor reads
—	Controls family	A versioned control with a population, an `operated_throughout` assertion and a certifier who is not its author
—	Regulations family	A clause quoted verbatim, resolving through named objects to a field in a signed record, with a judging window
—	Five-link resolution and three dispositions	A binding that cannot resolve is a build failure, not an audit finding; there is no blank and no "partially compliant"

Sources: [E03 §08; E21 §09; IM §6.8].

The honest direction is stated in the corpus and should survive any diligence conversation: Palantir's ontology "is bigger, older, better tooled, and runs on data estates we will not see for years. It is also Foundry-only. Ours is narrower, it is a residue of writes rather than a model built in advance, and it is designed to sit in front of somebody else's system of record" [E03 §08]. With the two families the direction reverses on one axis only: "we model a write and prove it … a narrower claim than theirs and a harder one to fake" [E21 §09].

7.15 Tests and acceptance criteria

The declaration layer is tested as software, with every guard shown able to fail before it is trusted. The corpus fixes most of the tests; the fixture shape is a **(build recommendation)**.

Test	Passes when	Source
Schema and semantic rules	For each of L0-S01–S11, one positive fixture validates and at least one negative fixture fails with the named rule, block and field	[E03 §05] (fixture pairs: build recommendation)
Never-skippable gates	A declaration omitting any one of the six fails validation; a declaration forced past a validator bug still has the gate run at rung 3	[E03 §02; E04 §03]
Database mirrors	Direct `INSERT`s violating L0-S03/S04/S05 are rejected by the database, not by the application	[E03 §05; IM §4.5]
Compile determinism	Two compiles of every certified version in the registry produce identical digests; run nightly over the full history	[E03 §06]
Screen reconstruction	For a sampled record, the surface plan compiled from its F06/F07 renders the same approver screen as the stored plan digest	[E06 §01; E02 §03 `basis`]
Zero screen files	App three renders from its declaration with no component file (EV4); a CI glob fails on any screen file under `app-*`	[EBB EB-12; ETS §08]
Refusal replay	Every reference-tenant refusal replays identically at its `as_of` (C4 / EV3)	[E02 §08; EBB EB-12]
Missing grant	1,000 fuzzed missing or contradictory grants resolve to A0 (C6 / EV7)	[E02 §08; EBB EB-12]
Five-link resolution (Phase 4)	L0-C01–C05 and L0-R01–R06 each have a failing fixture; a clause map compiled twice is identical	[E21 §05–§06]
Propagation (Phase 4)	A record recorded before a tightening is judged by the earlier profile; a spanning range is refused as a single run	[E21 §07]

The reference tenant also carries an *adversarial declaration*, to which every new refusal code must add positive and negative fixtures; ownership of the tenant and that declaration — kernel team or a separate assurance function — is an open question due 31 October 2026 [E02 §06, §10 Q5].

Acceptance for L1. A declaration for rung1's first write-type family validates with no blank field, passes the six never-skippable gate check, simulates with at least one refusal per declared gate, is certified by a named human who is not its author through a governed write, compiles to the same digest twice, and is resolved by rung 2 at an explicit version in production — with the surface compiled from it rendering the Decide screen of Chapter 8 **(build recommendation, assembled from E03, E06, E09)**.

7.16 What is in Level 1, and what is not

At L1 the build delivers the seven blocks with the three dispositions, JSON Schema, L0-S01–S11 and LN-01–07; the Studio's author, validate, simulate and certify modes, with authoring by elan1 engineers (partner authoring from Phase 2 is open [E03 §11 Q3]); the deterministic compiler; rung 4 rules at `policy_version`; and `proof_binding` with `elan1_lp_v1` and a per-record salt. It does not deliver `redaction_profile` (ignored at L1), executing verifier profiles (L2, from the COMPILE stub), `elan1_lp_sd_v1` and `per_field` salts (Phase 4), the Controls and Regulations families (Phase 4), or signed registry manifests (no signing key exists at L1) [E03; E14; E21 §01–§02; O01 DR-006].

The seven blocks are the one part of the architecture that does not change as the assurance level rises. "A level is a property of the proof and the claim, never of the policy. Any design that would require re-declaring on upgrade is rejected on sight" [E07 §01].

8. The console and the human approval path

At Level 1 the kernel's strongest claim is not cryptographic. It is that a person approved the writes that needed one, and that the person saw what they were approving. "At L1, elan1 decides whether a write is permitted, forces a human to approve" where the declaration says so, and records the outcome [E01 §01]. This chapter covers the surface through which that happens: the console, its screens and who uses them; the approval as a data object and its state machine; dual control, delegation and step-up; what the approver sees; the measures against rubber-stamping; the notification and approval channels, including rung1 Voice as a consumer channel; accessibility; the audit of approvals; the APIs behind the console; and a front-end build recommendation.

The one-line test for every screen in this chapter is E06's: "An approver who presses Approve has already seen the diff, the gate results and the policy version, or the screen is wrong" [E06 cover].

> **Designed is not shipped (DR-010).** No console screen runs for a customer as at 23 September 2026. The 4-hour default expiry is "asserted, not observed" [E04 §06 Q3; E06 §11 Q3]. Every timing and behaviour below is specification or build recommendation [O01 DR-010].

8.1 What the console is

The console has no bespoke screens. Every screen is compiled from a declaration's seven blocks and rendered by a small set of shared components; no `app-*` package contains a screen file [E06 §01]. The Build Book puts the count at about eleven shared components rendering thirty compiled screens, with "115 endpoints, 30 screens, 0 screen files" [EBB EB-06; EBB EB-15 §1]. Because compilation is deterministic and content-addressed, "what did the approver actually see in March 2027" is answerable in 2035 from the declaration version the proof names [E06 §01; Chapter 7 §7.11].

Every module presents the Four Doors and nothing else [E06 §01]:

Door	Answers	Content shapes it admits
Work	What needs me now	Queues
Records	What is the state of things	Lists, forms, pinned fields
Agents	Who and what is acting here	Agents, models, autonomy grants
Assurance	Can I show someone this was done properly	Chain standing, refusals, coverage, meter, exports

The platform is itself a module, called Platform, and its doors carry the screens that are about no single business object: approvals and obligations behind Work; declarations, actors and write types behind Records; agents, autonomy grants and keys behind Agents; chain, refusals, meter and exports behind Assurance. "There is no fifth door and no 'Settings' tab floating outside the grammar" [E06 §01, §10 D1].

Phase 1 honesty is a rendering rule, not a footnote. The Assurance door at L1 shows chain standing and coverage; it does not show a signature, an anchor time or a verifier verdict, because there are none, and "a screen that renders an empty 'Signed by' row is a screen that teaches a customer to expect something we do not ship" [E06 §01]. The Proof screen must say in the interface: "This record is hash-chained. It is not signed, and it cannot yet be checked by someone who does not trust elan1. That arrives at Assurance Level 2" [E06 §05].

8.2 The screens, and who uses them

Door · screen	Purpose	Primary users	Source
Work · **Approvals**	Queue of open obligations, oldest deadline first; "the one a customer opens twenty times a day"	Approvers (accounts head, owner, finance controller)	[E06 §02]

Door · screen	Purpose	Primary users	Source
Work · **Decide**	"The single most consequential surface in the product": diff, gates, decision, Approve or Reject	Approvers	[E06 §03]
Work · **Not done**	Refusal explanation in the words of the attempted action, with the operator action	Requesters, operators, support	[E06 §04]
Records · **Record**	Current state, pinned fields, and a timeline of every governed write, refusal and escalation	Operators, CAs, auditors	[E06 §05]
Assurance · **Proof**	One entry: `seq`, digests, versions, actor kind, outcome, `mode`, bound fields; chain continuity three-valued	Auditors, CAs, security reviewers	[E06 §05]
Platform · Records · **Studio**	Author → Validate → Simulate → Certify; cannot skip forward	Declaration authors, certifiers	[E06 §06]
Platform · Agents · **Tenants**	Residency domain, status, stop switch (a governed write under dual control)	Tenant administrators, elan1 operations	[E06 §07]
Platform · Agents · **Keys**	API keys, last used, rotation window; production keys issued by elan1 only in Phase 1	Tenant administrators	[E06 §07]
Platform · Agents · **Agents**	Every registered agent, its model, tools and hash-pinned descriptions	Tenant administrators, security reviewers	[E06 §07]
Platform · Agents · **Autonomy**	Grant per actor per write type, A0–A5; a missing grant renders as "A0 Observe (no grant)"	Tenant administrators	[E06 §07]
Platform · Assurance · **Meter**	Governed writes by write type, refusals at quantity zero on the same table, cost to date	Finance, tenant administrators	[E06 §07]

Record and Proof are two screens in two doors "because they answer different questions to different people" [E06 §10 D2]. Record's timeline shows refusals and escalations alongside commits, because "a timeline of successes only is a timeline that lies by omission" [E06 §05]. Proof states chain continuity as `continuous`, `break at seq N` or `not checked over this range`, with scope and count printed next to the word, never as a green tick: a chain verified over three of twenty-three write types "is a verified chain over almost nothing, and it reads identically on screen unless the screen says so" [E06 §05, §10 D3].

8.3 The approval as data, not UI

The approval path is made of kernel objects. The console renders them; it does not own them. That is the design decision this chapter rests on, and it follows from DR-001: no product builds its own gates, approval, proof or meter [O01 DR-001].

The obligation. When rung 6 resolves an actor's autonomy below what the write type needs and an approval route exists, the kernel creates an obligation row "with the `write_id`, the level that triggered it, an expiry and the set of actors who may resolve it — and the ladder stops". Rung 7 is never reached for an escalated write [E04 §03]. Rung 4 may also return `escalate`, "the honest version of warn — a human must decide and the write does not proceed until they do" [E04 §03; E02 I6]. The caller receives HTTP 200 with `outcome: escalated` and the `obligation_id`, is metered at zero, and must not retry [E05 §07].

The approval. "An approval is itself a governed write. It is not a flag" [E02 §03].

Field	Type	Meaning
`approval_id`	uuid	Kernel-assigned
`write_id`	uuid	The write being approved
`approver_id`	uuid	Must differ from `actor_id` when gate G4 applies
`approver_kind`	enum	`human` only at L1; an agent may never approve
`decision`	enum	`approve · decline`
`basis`	string	What the approver saw: the surface plan version and the rendered payload digest
`at`	RFC 3339	Platform clock

Source: [E02 §03].

Declining is a governed outcome, not an error: the original write refuses with `POLICY.APPROVAL_DECLINED` [E02 §03]. The console labels the button "Reject"; the data value is `decline` [E06 §03; E02 §03].

Resubmission, never resumption. On approval the original write is re-submitted with `obligation_ref` set, climbs the ladder again from rung 0b, finds a second named actor at G4, and commits at rung 7 [E04 Flow 3; E02 §03]. There is no suspended-and-resumed state inside the kernel. One escalated business decision therefore produces separate proof entries for the escalation, the resolution and the resubmission — or for the lapse, if it expires — and "one extra row is the price of having no suspended-and-resumed state inside the kernel, and we take that trade every time" [E04 Flow 3, §05 D4].

A sketch of the obligation row as the kernel would hold it (field names beyond those in E02/E04/E05 are **(build recommendation)**):

```
CREATE TABLE obligation (
  obligation_id   uuid PRIMARY KEY,
  tenant_id       uuid NOT NULL,
  write_id        uuid NOT NULL,                -- the escalated intent
  raised_at_rung  smallint NOT NULL CHECK (raised_at_rung IN (4, 6)),
  autonomy_level  text NOT NULL,                -- A0..A5 that triggered it
  resolver_set    uuid[] NOT NULL,              -- human actors only (E06 D8)
  quorum          smallint NOT NULL DEFAULT 1,  -- build recommendation, 8.5
  expires_at      timestamptz NOT NULL,         -- declared expiry, default 4 h
  paused_seconds  integer NOT NULL DEFAULT 0,   -- outage pause accounting (E06 D6)
  status          text NOT NULL CHECK (status IN
                    ('open','approved','declined','expired')),
  resolved_by     uuid,                         -- approval_id of the resolving write
  escalation_seq  bigint NOT NULL               -- proof entry of the escalation
);
```

8.4 The state machine

Figure 11 shows the obligation's life. Four terminal states are fixed by source — approved, declined, expired and (for the resubmitted write) committed or refused. Two conditions modify the open state: a platform outage, which pauses expiry, and a step-up requirement, which holds an approval until the approver's identity is re-established.

Figure 11 — The approval path — obligation state machine

L1 — BEING BUILT

An escalated write stops; a named human resolves it; every outcome, including expiry, is its own record

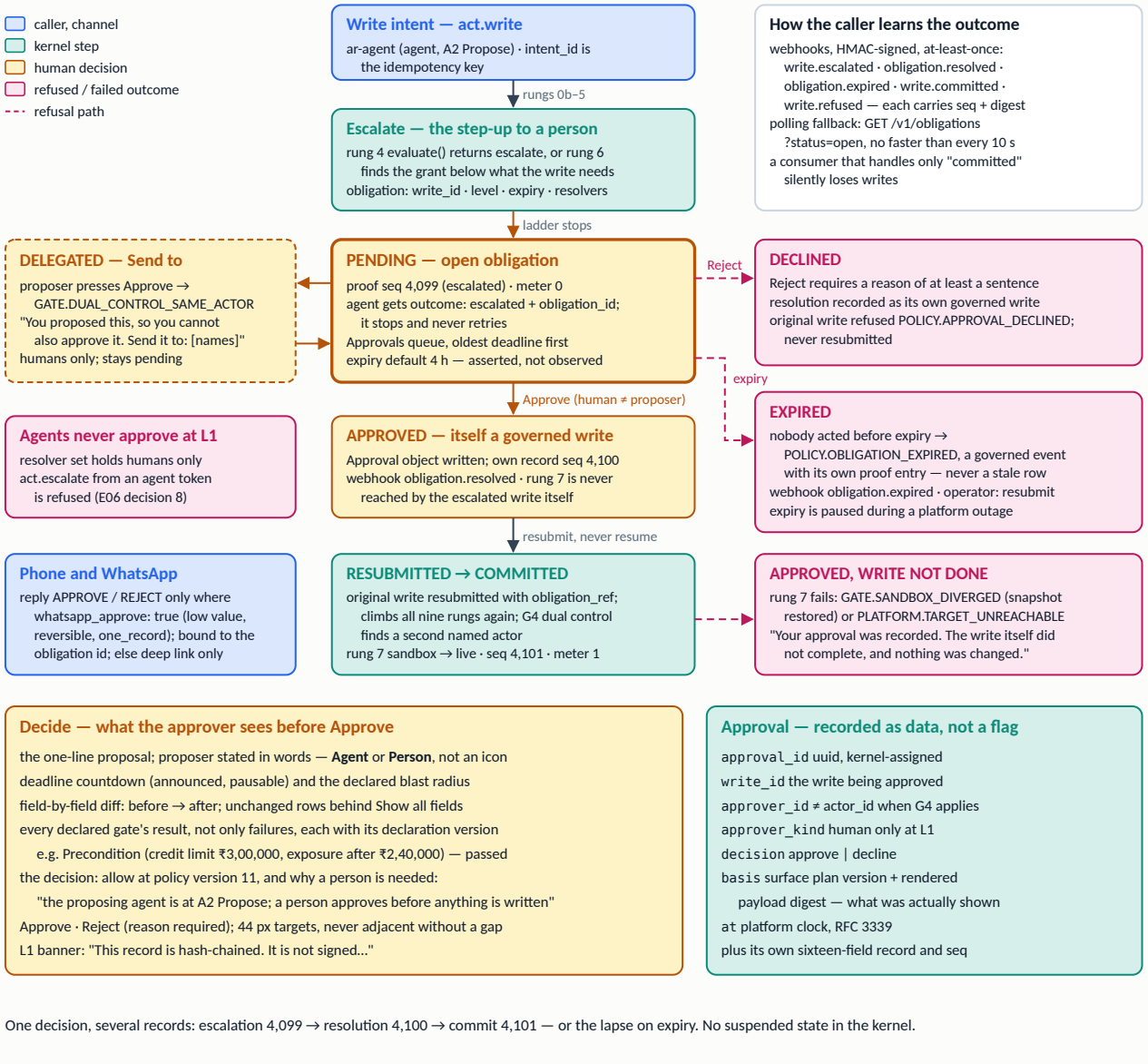


Figure 11 — The approval path: an escalation at rung 4 or 6 opens an obligation; resolution is itself a governed write; approval causes a resubmission through all nine rungs; decline and expiry are governed events with their own proof entries.

From	Event	To	Kernel effect	Record	Webhook
—	Rung 6 (or rung 4) returns `escalate`	**open**	Obligation row; ladder stops	Escalation entry, meter 0	`write.escalated`
open	Platform outage	open (**paused**)	Expiry clock stops; nothing is approved	—	—
open	Approver with required channel lacks fresh authentication	open (**step-up**)	Resolution held until step-up succeeds	Failed step-ups recorded	—
open	`act.escalate {decision: approve}` by a human in the resolver set	**approved**	Approval write climbs its own ladder	Resolution entry	`obligation.resolved`

From	Event	To	Kernel effect	Record	Webhook
open	`act.escalate {decision: decline, note}`	**declined**	Original write never resubmitted	Resolution entry; reason text	`obligation.resolved`; then `write.refused POLICY.APPROVAL_DECLINED`
open	`expires_at` passes	**expired**	Governed event	Lapse entry	`obligation.expired (POLICY.OBLIGATION_EXPIRED)`
open	Resolution by the proposer	open	Refused `GATE.DUAL_CONTROL_SAME_ACTOR`	Refusal entry	`write.refused`
open	Resolution by an agent token	open	Refused (agent may never approve)	Refusal entry	`write.refused`
approved	Resubmission commits at rung 7	**committed**	Sandbox → gates → live → gates → commit	Committed entry, meter 1	`write.committed`
approved	Resubmission refused (a gate changed, or `GATE.SANDBOX_DIVERGED`)	**refused**	Snapshot restored where applicable	Refusal entry	`write.refused`

Sources: [E02 §03, §06; E04 §03, Flow 3; E05 §07, §09; E06 §02–§03, §10 D6, D8]. The step-up state is specified for rung1 Voice [R19 §04]; applying it to other channels is a **(build recommendation)**, §8.5.

Expiry. The default is four hours, declared per write type, and "asserted, not derived from any observed approver behaviour"; both E04 and E06 leave the per-write-type default to be set with the first design partner [E04 §06 Q3; E06 §11 Q3]. Expiry is itself a governed event with its own entry, "not a row that quietly goes stale" [E06 §03]. Expiry is paused during a platform outage: the error state of the queue says "Nothing has been approved in the meantime and nothing expires while this is down" [E06 §02, §10 D6]. This memorandum recommends that pause accounting be kept as elapsed seconds on the obligation row and that the effective deadline be computed as `expires_at + paused_seconds`, so that the pause is itself auditable **(build recommendation)**.

A write the approver approved can still fail. If the resubmitted write fails at rung 7, or the live re-run disagrees with the sandbox (`GATE.SANDBOX_DIVERGED`, snapshot restored), the approver sees: "Your approval was recorded. The write itself did not complete, and nothing was changed. We have been paged." Both facts, in that order [E06 §03].

8.5 Dual control, quorum, delegation and step-up

Dual control (G4). G4 is never skippable and asserts that "a second named actor has approved this `write_id`" [E04 §03]. It is forced by `tenant` and `cross_boundary` blast radii and may be declared on any write type, including with a threshold parameter such as "required where `grand_total` > 200000" [E03 §02, §04]. The approver must differ from the requester (`GATE.DUAL_CONTROL_SAME_ACTOR`), and the resolver set on an obligation "contains human actors only, the Send to control offers no agents, and an `act.escalate` call from an agent token is refused" [E06 §03, §10 D8]. At declaration level, `may_approve: true` is invalid wherever `actor_kind = agent` (L0-S11) [E03 §05]. Irreversible write types can never be auto-approved, and auto-approve is off by default per tenant [E01 §02].

When the approver is the proposer, the refusal copy does not say "insufficient permissions". It says "You proposed this, so you cannot also approve it. Send it to: [names]", with a Send to control [E06 §03].

Quorum. The corpus specifies one second actor. rung1 Voice refers to changes that "need two approvers" where dual control is forced by a tenant or cross-boundary radius [R19 §03, §07]. Whether a tenant-radius write proposed by an agent needs one human approver or two is not decided. This memorandum recommends a `quorum` on the obligation, default 1, set to 2 by the declaration for `tenant` and `cross_boundary` radii, with distinct approvers and each approval a separate governed write **(build recommendation)**.

Delegation. The kernel's intent carries `on_behalf_of`, which "confers accountability, never autonomy"; an expired delegation refuses at rung 1 with `POLICY.DELEGATION_INVALID` [E02 §03, §06]. rung1 Voice permits no approval on behalf of anyone [R19 §05]. The corpus does not specify approver delegation (for example, cover during leave). The recommended

form is a time-boxed resolver-set membership granted by a governed write under dual control, naming a human delegate, the write types and the window; it never names an agent and never raises anyone's autonomy **(build recommendation)**.

Step-up. rung1 Voice requires a one-time code, sent to the registered mobile and read back, before an approval is posted, and treats calling-line identity as "a hint, never an identity" [R19 §04–§05]. For the console, this memorandum recommends step-up re-authentication (a platform authenticator or a one-time code) for any approval of an irreversible write type, a `tenant` or `cross_boundary` radius, or an amount above a declared ceiling, with the step-up method recorded in the approval write's payload **(build recommendation)**.

8.6 What the approver sees — the Decide screen

The Decide screen is specified line by line [E06 §03]:

1. **Above the fold**: the one-line proposal; the proposer, with *Agent* or *Person* stated in words, not an icon; the deadline; and the declared blast radius.
2. **The field-by-field diff**: before and after, changed rows highlighted, unchanged rows collapsed behind *Show all fields*. "Nothing is summarised away; a diff a human cannot read is not a control."
3. **Every declared gate's result**, not only the failures, because rung 3 runs them all even after the first failure. Each in plain words with its numbers — "Precondition (credit limit ₹3,00,000, exposure after this write ₹2,40,000) — passed" — and the declaration version it ran against [E06 §03, §10 D9].
4. **The decision and why it reached a person**: "allow at policy version 11 … the proposing agent is at A2 Propose; a person approves before anything is written."
5. **Approve or Reject**; Reject requires a reason of at least a sentence [E06 §10 D7].
6. After approval: "Approved · posted · proof #4,101", with *Open the record* and *Reverse this (30 days)*.

The queue row carries the minimum an approver must see before opening an item: human or agent proposer, the money or the radius, the gate summary ("5 of 5 passed") and the time left. Filters (waiting on me, expiring within an hour, raised by an agent, write type, counterparty) live in the URL so a filtered view can be pasted into a message [E06 §02]. The empty state explains itself: "Nothing is waiting for you. Agents are running at their declared autonomy; anything that needs a person will appear here", with a link listing the write types that escalate for this actor [E06 §02].

Binding what was seen to what is written. The approval's `basis` field records "the surface plan version and the rendered payload digest" [E02 §03]. That field is what turns E11's segregation test — confirm "that what they were shown matches the change that was applied" — from an interview into a check [E11 §07 test 5]. The corpus does not say how the rendered payload digest is produced. This memorandum recommends that the console compute it as SHA-256 over the canonical form of the payload fields it rendered (using the declaration's `elan1_lp_v1` encoding) and submit it with the resolution; the kernel recomputes it from the stored intent and the named surface plan, and refuses the resolution if the two differ, so an approval given on a stale or tampered view never resolves an obligation **(build recommendation)**.

```
{
  "tool": "act.escalate",
  "args": {
    "obligation_id": "0f3c…",
    "decision": "approve",
    "note": "Checked against PO 4471 and GRN 2210.",
    "basis": {
      "surface_plan": "sp:erp_sales_invoice@7#c41e…",
      "rendered_payload_digest": "9b2d…",
      "channel": "web_console",
      "step_up": "none"
    }
  }
}
```

The `act.escalate` signature `{obligation_id, decision, note}` is from E05 §04; the structure of `basis` is **(build recommendation)**.

8.7 Against rubber-stamping

The failure mode the product exists to prevent is "a stream of Approve presses" [E06 §02]. An approval given where the approver could not have understood the difference "is not an attestation — it silently writes a false fact into the ontology" [GIP §6.3]. The measures fall into three groups.

Measure	Status	Source
Queue row shows human/agent, amount or radius, gate summary, countdown	Specified	[E06 §02]
Full diff, never summarised; every gate result, not only failures	Specified	[E06 §03, D9]
The reason a person is needed is stated ("an agent proposed this; you are the decision")	Specified	[E04 Flow 3; E06 §03]
Reject needs a written reason; an approval's `basis` is recorded	Specified	[E06 D7; E02 §03]
Approve and Reject never adjacent without a gap; 44 px targets	Specified	[E06 §09]
WhatsApp reply-approval only for low-value, reversible, `one_record` write types; everything else deep-link only	Specified	[E06 §08, D5]
Voice: read-back names what cannot be undone; explicit "yes, approve"; band limits	Specified (rung1)	[R19 §03, §06]
Proposer cannot approve; agents never approve	Specified, kernel-enforced	[E06 D8; E02 §08 C11]
Bulk approve for identical low-value items	**Open** — "does that defeat its purpose?"	[E06 §11 Q2]
Basis digest checked by the kernel (stale-view refusal)	Build recommendation	§8.6
No default focus on Approve; a keyboard user lands on the diff	Build recommendation	—
Approver telemetry on the Assurance door: median time-to-decide per approver and write type, share of decisions taken without expanding a collapsed diff	Build recommendation	—
Detective control: a monthly sample of approvals re-read by a second approver, recorded as a governed write	Build recommendation	—

The telemetry recommendation has a second use that the corpus already anticipates. At Phase 4, the precedent score that lets an agent graduate to higher autonomy counts `approved_unamended` records — "a human repeatedly approving without changing anything is the closest thing to evidence that approval is ceremonial" — and uses a high ratio as evidence *for* graduation [E24 §04]. Ceremonial approval is therefore not only a control weakness; it is a signal the system is designed to act on, by moving the write out of the approval path under a governed, dual-controlled grant change rather than leaving a person to press a button they no longer read [E24 §05].

8.8 Notification and approval channels

Which channels may carry an approval is declared, per channel, in block 7 (`channels[].may_approve`), and "a channel adapts; it never decides" — the kernel re-checks every declared gate whatever the channel did [E03 §02]. The approval write records the channel it arrived on, because every intent carries `channel` [E02 §03].

Channel	Kind	Carries	May approve	Notes	Source
Web console	web	Submissions and approvals	Yes	The reference surface; full diff	[E03 §04; E06]
Mobile PWA	mobile	Approvals	Yes	Responsive to 360 px; diff collapsed to changed fields	[E03 §04; E06 §08]
WhatsApp	messaging	Notification; reply-approval only for declared low-stakes write types	Yes, bounded	Template pre-approval by the BSP is slow; 24-hour session windows; opt-out honoured in the consent ledger and checked by G1	[E06 §08]

Channel	Kind	Carries	May approve	Notes	Source
MCP / API	mcp, api	Submissions	No	Agents never approve	[E03 §04; E06 D8]
rung1 Voice	voice	Approvals only, inbound only	Yes, within a band	A rung1 consumer channel; §8.8.1	[R19]
Webhooks and polling	—	Outcome notification to consumers	—	`write.escalated`, `obligation.resolved`, `obligation.expired`	[E05 §09]

WhatsApp. A raised obligation sends a template message — what is proposed, who proposed it, the amount, "5 of 5 checks passed" and a deep link. Only for write types declared `whatsapp_approve: true` may the approver reply APPROVE or REJECT; the reply "is bound to the obligation id in the deep link, not to the message body, and it expires with the obligation". For everything else the only action is the deep link, because "a 'yes' in a chat thread is not informed consent to a write whose diff the person has not seen", and the template says so: "Open the console to see the change and decide" [E06 §08]. Which ERPNext write types qualify, if any, is for the rung1 lead by November 2026 [E06 §11 Q1].

8.8.1 rung1 Voice as a consumer approval channel

rung1 Voice is a rung1 product line, not elan1's; to the kernel it is one more channel in a consumer's declaration, of kind `voice`, carrying no write types, with `may_approve: true` and `mode: enforced` [R19 §03]. It shows how the approval path is meant to be extended: entirely in the consumer, with the kernel unchanged.

Property	Rule	Source
Direction	Inbound only; rung1 never places a call, and a CI test asserts no origination credentials and no origination verb	[R19 §01]
What it carries	The caller's own pending approvals, read back; approve; decline with a spoken reason; request a single-use link	[R19 §03]
What it never carries	New requests; destructive or irreversible items; `tenant` or `cross_boundary` radius; items above the tenant's voice ceiling (default ₹1,00,000, never above the approver's own band); anything the caller raised	[R19 §03]
Identity	Calling-line identity selects a candidate; a one-time code read back proves possession of the handset	[R19 §05]
Confirmation	A read-back composed from the same fields the review screen renders, never a model's paraphrase; "yes, approve" or the keypad equivalent; a decision word is never inferred	[R19 §06]
Kernel call	The approval is posted "to the elan1 layer against the existing decision id, with the channel recorded as `voice_approval`, the utterance id, and the step-up method"	[R19 §04]
Failure	Dropped call before posting = no approval; layer unreachable = "Nothing has been changed", no cached allow	[R19 §04]
Fallback	"Nothing is only approvable by voice"; every voice item is approvable on the console	[R19 §08]

The `VOICE_*` refusal phrases (`VOICE_VALUE_CEILING`, `VOICE_DUAL_CONTROL_REQUIRED`, `VOICE_APPROVER_IS_REQUESTER` and others) are rung1's channel-level codes and are not in the kernel's closed register [R19 §07; E02 §06]. The kernel still enforces the invariants that matter — approver ≠ requester and human-only approval — whether or not the channel checked them first, and "the approval row names a human, not a channel" [R19 §04]. In kernel terms the voice approval is an `act.escalate` resolution whose `basis` carries the utterance id and step-up method (§8.6).

8.9 Accessibility

E06 fixes eight requirements for the approval path [E06 §09]:

1. Keyboard-complete — queue, open, read the diff, approve, confirm — with a visible focus ring at 3:1 against its background.
2. Contrast 4.5:1 for body text and 3:1 for interface elements, in both themes.

3. Never colour alone: pass and fail carry a word and a shape as well as green and red.
4. Diffs are tables with real row and column headers, so a screen reader can say "Credit limit, before ₹3,00,000, after ₹2,40,000".
5. Live regions announce outcomes: "Approved. Posted. Proof 4101."
6. 44 px minimum touch targets; Approve and Reject never adjacent without a gap.
7. Timers are announced and pausable; an approver using assistive technology "must not lose an obligation to a countdown they could not perceive".
8. Language plain enough to survive translation to Tamil and Hindi, with numbers in Indian grouping.

Two items are open: whether a WCAG 2.2 AA audit is in Phase 1 scope or waits for third-party GA (elan1 lead, November 2026), and whether a Tamil interface ships in Phase 1 or with rung1 Voice later (rung1 lead, December 2026) [E06 §11 Q4–Q5]. Item 7 needs one design choice the corpus does not make: pausing a displayed countdown cannot move a kernel expiry. This memorandum recommends that an approver may request a single extension of a declared length, recorded as a governed event on the obligation, and that the pausable timer be the display of that mechanism **(build recommendation)**.

8.10 The audit of approvals

Everything in the approval path is recorded as governed data, so the audit is a query, not a reconstruction [E06 §03; E04 Flow 3].

Recorded	Where
The escalation: `write_id`, autonomy level, expiry, resolver set	Obligation row; escalation proof entry
The resolution: approver, approver kind, decision, basis, reason text on a decline, channel	Approval write and its proof entry
The resubmission and its outcome	A separate proof entry, committed or refused
An expiry	A governed lapse entry, `POLICY.OBLIGATION_EXPIRED`
A refused resolution (same actor, agent token)	A refusal entry
Registration and certification of declarations, stop switches, grant changes	Governed writes under dual control

Sources: [E02 §03; E04 Flow 3; E06 §03, §07; E09 §04].

E11's Level 1 checking procedure gives an auditor the test to run: take approvals where a second approver was required and confirm the approver is a different person from the requester, that the approver is a person rather than a software agent, and that what they were shown matches the change that was applied; "the platform refuses same-actor approval and refuses agent approval; test that it does" [E11 §07 test 5]. E02 carries the same property as conformance test C11 [E02 §08]. The console's Assurance door should offer that test as a saved query over a chosen period — approvals, approvers, same-actor refusals, agent-token refusals, declines with reasons and expiries — with the scope printed **(build recommendation)**.

Because approvals are data, they also feed the rest of the system: the meter (escalations meter at zero), E11's reconciliation, and at Phase 4 the precedent counters `approved_unamended`, `approved_amended`, `declined` and `expired` [E05 §07; E24 §04].

8.11 The APIs behind the console

The console is a consumer of the same public surface as any other. It uses a tenant token for the signed-in human — "one tenant, one actor, one autonomy level", the only class that can reach `/v1/writes` — and the Studio uses a control token that cannot reach any record store [E05 §05]. Tenant and actor come from the token, never from an argument [E05 §05].

Endpoint family	Class	Serves	Source
`/v1/obligations`	act	Escalations and approvals (seven endpoints in the Build Book inventory)	[E05 §04; EBB EB-06]
`/v1/writes`	act	Submit, read, list governed writes	[E05 §04]
`/v1/evidence`	ro	Proof entries and chain continuity; pages by `seq`	[E05 §04, §09]
`/v1/declarations`	ctl	Author, validate, simulate, certify, publish	[E05 §04]
`/v1/actors`	ctl	Actors, delegations, autonomy grants	[E05 §04]
`/v1/tenants`	ctl	Config, stop, residency, keys	[E05 §04]

The seven obligation endpoints are not enumerated in the corpus. A recommended set **(build recommendation)**:

```
GET  /v1/obligations?status=open&resolver=me     queue (E05 polling form)
GET  /v1/obligations/{id}                        row + write + gate results
GET  /v1/obligations/{id}/surface                compiled plan ref + diff model
POST /v1/obligations/{id}/resolve                = act.escalate; idempotency_key
POST /v1/obligations/{id}/send-to                re-route within resolver set
POST /v1/obligations/{id}/extend                 single declared extension (8.9)
GET  /v1/obligations/{id}/history                escalation, resolution, lapse
```

Every `act` call takes a client-supplied idempotency key retained for 30 days; a repeated key returns the original outcome, including `seq` and digest [E05 §08]. Webhooks are signed with an HMAC over the body (`X-Elan1-Signature`, `X-Elan1-Timestamp`, five-minute window), delivered at least once, and every payload that references a write carries `seq` and `digest`, so a consumer can detect a gap by watching `seq` contiguity [E05 §09]. "A consumer that only handles the committed case will silently lose writes" — `obligation.expired` and a declined approval must be handled as first-class outcomes [E05 §09].

8.12 Front-end build recommendation

The stack document fixes the direction and leaves the details open [ETS §04, §08]. The recommendation:

Concern	Choice	Status	Source
Framework	React 18 + Vite + TypeScript + Tailwind, with design tokens and the hash router carried over from the repository	Recommended in source	[ETS §04]
Primitives	A headless component library (Radix UI) for focus management, overlays and keyboard behaviour	Source lists as alternative; recommended here	[ETS §08; GIP §6.4]
Components	About eleven shared components in a closed renderer package owned by one team; no screen files in `app-*`, enforced by a CI glob	Source	[EBB EB-15 §1, §4; ETS §08]
Surface compiler	Deterministic, content-addressed; Python or TypeScript, open	Source (open)	[ETS §04]
Rendering contract	The console renders only from a compiled surface plan fetched by digest; the plan digest is sent back in `basis`	Build recommendation	§8.6
Testing	Playwright end-to-end over the approval path; Storybook for the eleven components; Vitest; axe-core accessibility checks in CI against the eight E06 requirements	Playwright and Storybook in source; axe-core recommended	[ETS §08, §09]
Mobile	Progressive web app at 360 px; no native app in Phase 1	Build recommendation from E06 §08	[E06 §08]
Internationalisation	ICU message catalogues from day one; Indian digit grouping; Tamil and Hindi catalogues when decided	Build recommendation	[E06 §09, §11 Q4]

The closed renderer seam is a throughput cost the Build Book accepts deliberately: "renderers are where deadline pressure lands", and the answer to a request for a bespoke screen is "we can add a component that all twenty apps get" [EBB EB-15 §4].

8.13 Tests and acceptance criteria

Test	Passes when	Source
Same-actor approval	Proposer resolving their own obligation is refused `GATE.DUAL_CONTROL_SAME_ACTOR`, recorded, and the screen offers Send to with human names only	[E06 §03; E02 C11]
Agent approval	`act.escalate` from an agent token is refused on every channel	[E06 D8; E02 C11]
Decline	Original write never resubmitted; caller receives `POLICY.APPROVAL_DECLINED`; reason text stored	[E04 Flow 3]
Expiry	At expiry a lapse entry exists and `obligation.expired` fires; during an injected outage the deadline moves by the outage length	[E04 Flow 3; E06 D6]
Four entries	One approved escalation yields escalation, resolution and resubmission entries with contiguous `seq` and correct outcomes	[E04 Flow 3]
Diverged sandbox	An injected divergence restores the snapshot and shows both sentences in order	[E06 §03]
Stale basis	A resolution whose rendered digest differs from the kernel's recomputation is refused	Build recommendation
WhatsApp reply	Reply-approval works only for declared write types and only while the obligation is open	[E06 §08]
Accessibility	The full approval path completes keyboard-only; axe-core reports no violations at the declared level	[E06 §09]
Phase 1 honesty	No screen renders a signature, anchor or verifier verdict at L1; the Proof screen shows the unsigned sentence	[E06 §01, §05]

Acceptance for L1. One real escalation, raised by an agent at A2 in a design partner's tenant, resolved on the Decide screen by a named human who is not the proposer, producing the escalation, resolution and committed entries — with the approval's `basis` resolving to the surface plan that rendered it — and one real decline and one real expiry, each with its own entry [E05 §11 step 8; E04 Flow 3].

9. Data architecture and the Level 1 proof record

This chapter describes where elan1 keeps its data, how tenants are kept apart inside one residency domain, what the Level 1 proof record is and how it is chained, how the database itself forces a refusal to meter at quantity zero, how long each class of data is kept, how it is backed up and restored, how an erasure request is reconciled with a log that is never rewritten, and what an auditor is given at Level 1. The byte-level format of the record (Seal Spec v6) is in Chapter 10; the bundle and the offline verifier are in Chapter 11; keys and timestamp authorities are in Chapter 12; the meter's commercial plumbing is in Chapter 19; the security baseline is in Chapter 18.

> **Status.** Designed, not shipped (DR-010) [O01]. elan1 has no external customer and no governed action has been committed or refused in a production tenant [IM §4.9]. What exists is a specification corpus and an implementation of the L1 kernel against a single system of record; an earlier working tree shows a proof engine under a superseded seal domain (`elan1.proof.seal.v2`, thirteen fields) with a per-tenant advisory lock on append [HFS; IM §4.9]. Every DDL block in this chapter is a **build recommendation** unless a constraint is quoted from a source, and every size, latency and cost figure is a target or a model, not a measurement.

9.1 What Level 1 proves, and what it does not

The data architecture exists to make one sentence true and to stop anyone from saying a stronger one. The frozen L1 sentence is: at L1, elan1 decides whether a write is permitted, forces a human to approve the writes that need it, executes it through a single writer, records the decision in a hash-chained append-only log, and meters it; the log is tamper-evident; it is not signed, not anchored to an external time source, and not verifiable by a third party who does not trust elan1 [E01 §01; IM §7.1]. The auditor-facing note compresses the limitation into one line: an elan1 record shows that a change to a system of record passed stated rules, with a named actor and a named approver, in an order that can be checked, "and at Level 1 it shows that only to someone who is willing to trust our infrastructure" [E11, in one line].

The claim the store supports is therefore narrow, and the design is organised around its edges.

An L1 record establishes	An L1 record does not establish
That a change of a stated type was attempted by a named actor of a stated kind (person, agent, scheduled system), at a stated position in an ordered per-tenant sequence [E11 §3]	That the rules were the right rules. "A well-recorded bad rule produces a clean chain" [E11 §4]
Which declaration version and policy version decided it, resolvable to the verbatim text as it stood that day; superseded versions are never garbage-collected [E11 §3; E03 §07]	That the underlying transaction was genuine: the payload digest binds what was written, not whether the supply, counterparty or approver's understanding was real [E11 §4]
Which gates ran, in which order, and what each returned, including the gates that passed [E11 §3; E02 §05]	The current state of the target system. A later direct edit in the target leaves the chain reading clean; the note calls this the most important limitation [E11 §4; IM §7.1]
Whether a person approved it, which person, and a digest of what they were shown [E11 §3; E02 §03]	Anything to a party who does not trust elan1's infrastructure. Whoever controls the store could in principle recompute the whole chain [E11 §4]
That the change was applied once, and that a repeated submission returned the original outcome [E11 §3; E05 §08]	That the tail is complete. A chain cut at record 4,000 of 4,200 is internally consistent and looks complete [E11 §4; IM §7.1]
That the refused cases are present, with the same shape as the permitted ones [E11 §1, §3]	Conformance with any statute, standard or framework; no such claim is made at any level [E11 §4, decision 4]
That nothing in the retained sequence has been altered or removed since written, subject to the rows above [E11 §3]	That an erased or expired payload can still be opened; such a record reads "chain intact, payload unbound", not as a failure [E11 §4]

> **Word discipline is a build requirement.** L1 may be described as tamper-evident to the tenant, never as tamper-proof, signed, immutable, court-ready or independently verifiable [E07 decision 4; E14 §05]. Conformance test C14 runs a string check over every consumer's rendered copy and fails the build on "signed", "tamper-proof" or "independently verifiable" while the tenant runs at L1 [E02 §08]. The data layer has a matching rule: no column, API field or export member may carry a value that reads as a signature at L1 (§9.7.6).

The truncated-tail property deserves a sentence of engineering consequence, because it shapes three later sections. A hash chain can prove that nothing between record 1 and record *n* was altered; it cannot prove that *n* is the last record. At L1 the only defence is reconciliation against the target system, which is why E11 test 3 (two-way reconciliation) is "the one that does the real work at Level 1, because it is the only one that does not depend on trusting our store" [E11 §6]. At L2 the defence becomes the external anchor and the `unanchored_head` field (Chapters 10–12) [E15 §02]. This chapter adds one internal control at L1, a head-checkpoint log in a separately administered WORM bucket (§9.13.3), and is explicit that it is not an anchor and may not be described as one.

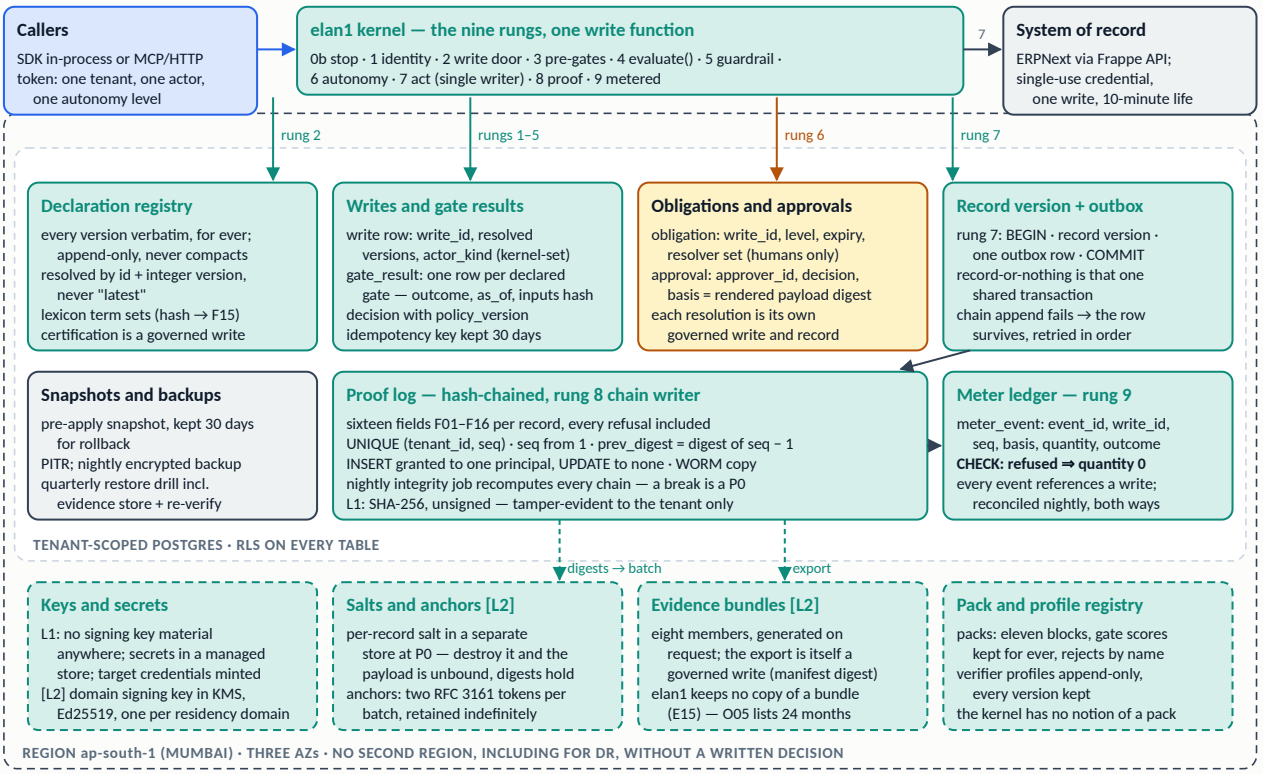
9.2 The stores

elan1 owns eight logical stores. Each has exactly one writing principal, and the principal that writes the proof log writes nothing else. Figure 12 shows them inside one residency domain.

Figure 12 — Data architecture — stores, flows, tenancy and retention

L1 — BEING BUILT · L2 DASHED

Every store sits in one Indian region; every tenant row carries tenant_id under row-level security



Retention by class

Proof records (the chain)	MCA Rule 3(1): ≥ 8 years · O05 table: 7 years WORM (conflict)
	E11 / E15: chains and anchors held indefinitely
Declarations, profiles, lexicons	verbatim, for ever — the 2027 screen rebuilt in 2035
Anchors [L2]	indefinitely, with TSA certificate path + revocation data
Evidence bundles [L2]	O05: 24 months WORM · E15: not retained by elan1
Snapshots · backups	30 days, encrypted, in India
Security and access logs	≥ 1 year; 180-day minimum in India (CERT-In)
Application logs	180 days, append-only, no PII (CI lint)
Idempotency keys	30 days

Tenancy and residency, checked in CI

RLS on every tenant table; a migration that disables RLS fails CI
cross-tenant read test runs in CI, expects failure
a call not attributable to a tenant is rejected
caches, indexes, vector stores partitioned by tenant, not filtered at query time
control plane: no network route to records
Postgres primary + synchronous standby (2nd AZ)
CI fails any resource outside the region list
residency and key custody at plane P0: no migration path

At L1 every store is checkable only by those who trust elan1; E11 test 3 — reconciliation to the target system — is the outside check.

Source: E02 §03, §05; E04 rungs 7–9; E11 §2; E07; E15 §04; E16; O05 §1–6; IM §6.10, §7.1 · elan1 Technology Memorandum · Sept 2026

Figure 12 — elan1 data stores in one residency domain: the declaration registry, the action store, approvals, the proof log, the meter ledger, the salt and key stores and the pack catalogue, each with its single writer, its tenancy boundary and its retention class.

Store	Holds	Plane	Writer (only)	Mutability	Retention (design)	Source
Declaration registry	Declarations at every version (seven blocks, verbatim), compiled digest, write-type descriptors, meter basis, lexicon term sets, policy versions and rules	P4	Kernel `define` path	Insert-only; a new version is a new row	Forever; never compacted	E02 §05; E03 §07; O01
Action store	Write intents, decisions, gate-result rows, obligations, idempotency keys, outbox	P3 (outbox on P1)	Kernel `assert` path	Insert-only; obligation resolution is a new event row	Life of the chain; idempotency keys 30 days	E02 §03, §05; E05 §08; EBB EB-02
Approvals	One governed record per approve or decline, with the rendered-payload digest	P3	Kernel (approval is a governed write)	Insert-only	As the chain	E02 §03; E11 §3

Store	Holds	Plane	Writer (only)	Mutability	Retention (design)	Source
Proof log	The sixteen-field records, digest, assurance level; at L2 signatures, key ids and anchor references in separate tables	P3 → evidence store	Chain writer (`seal`)	INSERT to one principal, UPDATE to none	Indefinite	E04 rung 8; E12 §05; E15 §04
Meter ledger	One meter event per governed action, basis, quantity, outcome	P3	Kernel at rung 9	Insert-only	As the chain; invoicing reads it	E02 §03; E04 rung 9; O07 §03
Keys and secrets	At L1: no signing key material anywhere; per-tenant target credentials in the credential broker. At L2: key register (ids, windows, status), keys in KMS	P0 / P1	Custodian (dual control); credential broker	Register insert-only	Register forever	E01 §07; E12 §06; O05 §3
Salt store	Per-record salts for the F14 payload commitment	P0	`e1-salt` only	Rows destroyed on erasure (the one sanctioned delete)	Until erasure or retention expiry	O01 planes; ETS §05; O05 Flow B
Pack catalogue	Pack definitions (eleven blocks), gate scores, profile bindings, pilot record; verifier profile registry	P8	Pack author + certifier (dual control)	Insert-only, versioned	Forever	E18 §01–§02; E13 §03

Four structural decisions sit under the table.

One PostgreSQL deployment per residency domain. The recommended engine is PostgreSQL 16 or later on a managed service in private subnets, chosen "for the strength of its grants, constraints and catalogue — not for scale features" [ETS §05]. A residency domain is a whole deployment with its own record store, evidence store, key custody and erasure salt store, and "a cross-domain query is not slow, it is impossible" [EBB EB-02 §7]. Phase 1 has one domain: ap-south-1 (Mumbai), three availability zones, with a synchronous standby in a second zone and no second region configured until a residency decision is written down and counsel-confirmed [O05 §1].

Three databases, not one. Inside the domain, the recommendation is three logical databases on the same managed cluster **(build recommendation)**: `e1_kernel` (registry, action store, approvals, meter), `e1_evidence` (proof log and, at L2, signatures and anchors), and `e1_salt` (the salt store, separately granted, reachable only by the `e1-salt` role) [ETS §05]. Separating the evidence database means an operator who can administer the kernel database cannot alter the proof log through the same credential, and it expresses boundary B2 — "no foreign key from an evidence table into a record table" — as a physical fact rather than a convention [EBB EB-02 §3].

The linkage is by value. A proof record names its target record by `record_id` value, unconstrained, "precisely so that the evidence store can be exported, archived and verified in a place where the record store does not exist" [EBB EB-02 §3]. The same holds between the evidence database and the kernel database: the proof log does not reference the action store by foreign key; it carries `write_id`, and the reconciliation job joins by value (§9.8.2).

The kernel does not know what a pack is. E18 is explicit: "the declaration registry does not have a pack table. A pack is a composition of things that already exist" [E18 §01]. The pack catalogue in the table above is therefore a P8 artefact store beside the kernel, not a kernel table; the kernel resolves only declarations and, at L2, verifier profiles by identifier and version (§9.6).

9.2.1 Principals

The single-writer invariant (I2) is about the target system [E02 §02]. The data layer applies the same idea to its own tables: one named principal per write path, no convenience principal. EBB puts the reason plainly: "A convenience principal is the second write path arriving through the database rather than through the application" [EBB EB-02 §8].

Role	Login	May write	May read	Notes
`e1_owner`	No (migrations only, via pipeline)	DDL	—	Owns objects; never used by a running service; migrations are forward-only [ETS §05]

Role	Login	May write	May read	Notes
`e1_kernel`	Yes	Registry, action store, approvals, meter (INSERT)	Same, under RLS	Runs rungs 0b–7 and 9
`e1_chain`	Yes	`evidence.proof_record` (INSERT only)	Outbox (SELECT), own chain head	The only writer of the proof log; the chain writer process
`e1_salt`	Yes	`salt.record_salt` (INSERT, DELETE on erasure)	Same	Separate database; dual-controlled erasure path [O05 Flow B]
`e1_verify`	Yes	Nothing	Registry, proof log	Serves the `verify` call; `ro` token scope [E02 §04]
`e1_integrity`	Yes	`ops.integrity_run` (INSERT)	Everything, one tenant at a time	Nightly job; no `BYPASSRLS` (§9.3)
`e1_reconcile`	Yes	`ops.reconcile_run` (INSERT)	Action store, proof log, meter	Nightly two-way meter reconciliation [O07 §09]

All roles are `NOSUPERUSER NOBYPASSRLS NOCREATEROLE`. The application "connects as a restricted role, never as owner or superuser", checked on every deploy [O05 §2; S12 §2].

9.3 Tenancy: row-level security or schema per tenant

Tenancy comes from the token, never from a call argument; a call that cannot be attributed to a tenant is rejected, not defaulted; a token for one tenant presented with another tenant's `tenant_id` in the body is "a rejection and an alert, not a refusal" [E02 §04]. The group baseline then fixes the storage mechanics for every brand: a `tenant_id` column on every tenant table, `NOT NULL`, foreign-keyed to `tenant`; row-level security enabled per table, with a migration that disables it failing CI; a cross-tenant read test in CI that expects failure; and caches, search indexes and vector stores partitioned by tenant rather than filtered at query time [O05 §2; IM §6.10].

The sources leave one choice open for elan1's own stores. stack1 runs "Postgres schema per tenant with row-level security on every table" for the runtime components it hosts [S12 §1], and states that elan1's own isolation "is elan1's to specify and is not covered here" [S12 §01]. EBB specifies that every governed table "carries tenant_id and is partitioned by it" [EBB EB-02 §3]. Three options exist: shared tables with `tenant_id` and RLS; a schema per tenant with RLS; or a database per tenant. At 10,000 tenants the second multiplies the catalogue, every migration and every grant-catalogue check by the tenant count, and a partially failed migration leaves tenants on mixed schemas; the third cannot live in one cluster at all.

Recommendation (build recommendation). elan1's kernel and evidence stores use shared tables with `tenant_id NOT NULL`, `FORCE ROW LEVEL SECURITY`, and hash partitioning on `tenant_id` for the three large tables (action store, proof log, meter). Schema-per-tenant buys little over RLS for tables elan1 designs itself, and it multiplies migration and catalogue-check work by the tenant count. A tenant that needs physical separation beyond RLS gets a dedicated residency domain, which is already the answer the key-custody design gives to a tenant that needs its own signing key [IM §7.7; E14 open question 5].

```
-- Tenancy scaffolding (build recommendation; rules from O05 §2, E02 §04)
CREATE TABLE core.tenant (
  tenant_id         uuid PRIMARY KEY,
  residency_domain  text NOT NULL CHECK (residency_domain ~ '^[a-z]{2}$'),  -- 'in'
  enforcement_mode  text NOT NULL CHECK (enforcement_mode IN ('enforced','advisory')),
  environment       text NOT NULL CHECK (environment IN ('live','sandbox','reference')),
  created_at        timestamptz NOT NULL DEFAULT now()
);
-- tenant is itself insert-only; the stop flag lives in core.stop_event (append-only),
-- so "stopped" is the latest event, never an UPDATE of a boolean.

-- Every tenant table follows this pattern.
ALTER TABLE kernel.write ENABLE ROW LEVEL SECURITY;
ALTER TABLE kernel.write FORCE ROW LEVEL SECURITY;          -- applies to the owner too
CREATE POLICY tenant_isolation ON kernel.write
  USING      (tenant_id = current_setting('e1.tenant_id', false)::uuid)
  WITH CHECK (tenant_id = current_setting('e1.tenant_id', false)::uuid);
```

```
-- The service sets the tenant once per transaction, from the verified token:
--   SET LOCAL e1.tenant_id = '<uuid from token>';
-- current_setting(..., false) raises if unset: an unattributed query fails, not defaults.
```

Three details make the pattern hold under pressure. First, `current_setting('e1.tenant_id', false)` raises an error when the setting is absent, so a code path that forgot to attribute a tenant fails closed rather than seeing zero rows and continuing [E02 §02 I3]. Second, `FORCE ROW LEVEL SECURITY` means even the table owner is filtered, which removes the temptation to run support tooling as owner. Third, jobs that must touch every tenant — the nightly integrity job, the reconciliation job — iterate over `core.tenant` and set `e1.tenant_id` per tenant in a fresh transaction; none holds `BYPASSRLS` (build recommendation), so a defect in the job cannot produce a cross-tenant join.

The CI and deploy checks are those of the group baseline and stack1's isolation note, applied to elan1's schemas [O05 §2; S12 §2]:

Test	Asserts	Cadence	Red means
RLS cross-tenant read	Authenticated as tenant B, a read of tenant A's rows in every tenant table is refused **by the database**, not filtered by the application	Every CI run	Build fails
RLS-enabled migration	No migration can `DISABLE` or drop a tenant policy; every table with a `tenant_id` column has RLS enabled and forced	Every migration	Migration rejected
Restricted-role check	No running service connects as owner, superuser or a `BYPASSRLS` role	Every deploy	Deploy blocked
Unattributed-call test	A request with no resolvable tenant is rejected, never defaulted	Every CI run	Build fails
Enumeration test	A list call under B's token returns B's own set; a response revealing a count of A's items fails, "because the count is the leak"	Nightly	Merge blocked

The enumeration rule is taken from stack1's note and applies unchanged to the console's list endpoints [S12 §2 Flow A step 3].

9.4 The declaration registry

The registry is where "policy as data" becomes rows. Every declaration version is "retained verbatim, for ever, append-only and monotonic", and the corpus gives the reason as a chain of dependencies: a record carries the declaration version and the lexicon hash; a verifier must resolve both; resolving them by calling elan1's API would not be offline; so an evidence bundle carries them; so every version must still exist verbatim [E03 §07]. The registry "grows monotonically and never compacts" [EBB EB-02 §6]. Registering a declaration is itself a governed write under `dual_control: true`, so it climbs the rungs and produces its own record [E02 §05].

```
-- Declaration registry (build recommendation; semantics from E02 §05 and E03 §07)
CREATE TABLE registry.declaration_version (
  tenant_id          uuid    NOT NULL,
  declaration_id     uuid    NOT NULL,
  version            bigint  NOT NULL CHECK (version BETWEEN 1 AND 4294967295),  -- uint32
  body               jsonb   NOT NULL,            -- the seven blocks, verbatim as certified
  body_bytes         bytea   NOT NULL,            -- exact bytes submitted; exported verbatim
  compiled_digest    bytea   NOT NULL CHECK (octet_length(compiled_digest) = 32),
  certifier_id       uuid    NOT NULL,
  supersedes         bigint,
  registered_seq     bigint  NOT NULL,            -- seq of the registering governed write
  PRIMARY KEY (tenant_id, declaration_id, version)
);
-- version must be exactly last + 1: enforced in the define path under a per-declaration
-- advisory lock, and checked by a constraint trigger (belt-and-braces).

CREATE TABLE registry.write_type (
  tenant_id            uuid   NOT NULL,
  declaration_id       uuid   NOT NULL,
  declaration_version  bigint NOT NULL,
  write_type           text   NOT NULL CHECK (write_type IS NFC NORMALIZED
                         AND octet_length(convert_to(write_type,'UTF8')) BETWEEN 5 AND 64),
  blast_radius         text   NOT NULL CHECK (blast_radius IN
```

```
                        ('one_record','one_counterparty','batch','tenant','cross_boundary')),
  meter_basis          text   NOT NULL,           -- from blast_radius; never configurable
  irreversible         boolean NOT NULL,
  reversal_plan        jsonb,
  CHECK (irreversible OR reversal_plan IS NOT NULL),                      -- EBB schema rule 3
  CHECK (meter_basis = blast_radius),                                     -- E05 §06 mapping
  PRIMARY KEY (tenant_id, declaration_id, declaration_version, write_type),
  FOREIGN KEY (tenant_id, declaration_id, declaration_version)
    REFERENCES registry.declaration_version (tenant_id, declaration_id, version)
);

-- registry.lexicon (lexicon_hash PK, term_set bytes) and registry.policy_version
-- (declaration_id, policy_version, rules, rules_digest) follow the same insert-only pattern.
```

The constraint `CHECK (irreversible OR reversal_plan IS NOT NULL)` is quoted from EBB's seven schema rules [EBB EB-02 §3]; the `meter_basis = blast_radius` check is this memorandum's expression of the rule that the basis is taken normatively from the declared blast radius and "is never configurable per customer" [E05 §06; E07 §04]. Idempotency of `define` is keyed on `(declaration_id, version, compiled_digest)`: the same digest returns the original acceptance, and a different digest at the same version is a rejection because versions are immutable [E02 §05].

Two points are easy to get wrong. The lexicon is content-addressed by its F15 digest, so identical term sets resolve to one stored copy; if a term set can carry tenant-private definitions it needs a `tenant_id` and RLS (build recommendation; E03 is silent on sharing). And `body_bytes` is stored alongside the parsed `jsonb` because `jsonb` normalises whitespace and key order; a bundle must carry the declaration "verbatim", and only the original bytes are verbatim [E15 §02].

9.5 The action store: intents, decisions, gate results and the outbox

The action store holds everything the kernel knows about a governed action before it becomes a proof record. EBB's `write` table groups the columns by purpose — identity, declaration, decision, authority, context, effect and evidence — and makes one sharp design point: `seq` is nullable "for exactly as long as it takes the seal worker to consume the outbox row, and a row where seq is null and received_at is older than the SLO is an incident (runbook RB1), not a retry" [EBB EB-02 §5]. Because the proof log is in a separate database in this design, the recommendation replaces that nullable column with a separate insert-only `kernel.write_sealed` row written by the chain writer's acknowledgement, so no row in the action store is ever updated **(build recommendation)**.

```
-- Action store (build recommendation; columns from EBB EB-02 §5 and E02 §03)
CREATE TABLE kernel.write (
  tenant_id            uuid   NOT NULL,
  write_id             uuid   NOT NULL,                -- kernel-assigned
  intent_id            uuid   NOT NULL,                -- client idempotency key
  actor_id             uuid   NOT NULL,
  actor_kind           text   NOT NULL CHECK (actor_kind IN ('human','agent','system')),
  on_behalf_of         uuid,
  declaration_id       uuid   NOT NULL,
  declaration_version  bigint NOT NULL,
  write_type           text   NOT NULL,
  channel              text   NOT NULL,
  as_of                timestamptz NOT NULL,           -- supplied, never read from a clock
  received_at          timestamptz NOT NULL DEFAULT now(),
  outcome              text   NOT NULL CHECK (outcome IN ('committed','refused','escalated')),
  refusal_code         text,
  refusing_rung        text,
  refusing_gate        text,
  policy_version       bigint,
  autonomy_level       smallint NOT NULL CHECK (autonomy_level BETWEEN 0 AND 5),
  blast_radius         text   NOT NULL,
  record_id            uuid   NOT NULL,                -- zero UUID unless committed
  record_version       bigint NOT NULL,                -- 0 unless committed
  payload_hash         bytea  NOT NULL CHECK (octet_length(payload_hash) = 32),
  PRIMARY KEY (tenant_id, write_id),
  UNIQUE (tenant_id, intent_id),
  CHECK ((outcome = 'refused') = (refusal_code IS NOT NULL)),
  CHECK ((outcome = 'committed') =
         (record_id <> '00000000-0000-0000-0000-000000000000' AND record_version >= 1)),
  CHECK (refusal_code IS NULL OR refusal_code ~ '^(POLICY|GATE|PLATFORM)\.[A-Z_]+$')
) PARTITION BY HASH (tenant_id);
```

```
CREATE TABLE kernel.gate_result (
  tenant_id     uuid     NOT NULL,
  write_id      uuid     NOT NULL,
  ordinal       smallint NOT NULL,                  -- declared order
  gate          text     NOT NULL,                  -- G1..G9 name as declared
  outcome       text     NOT NULL CHECK (outcome IN ('pass','fail')),
  inputs_hash   bytea    NOT NULL CHECK (octet_length(inputs_hash) = 32),
  inputs        jsonb    NOT NULL,                  -- what the gate saw, shown to auditors
  PRIMARY KEY (tenant_id, write_id, ordinal),
  FOREIGN KEY (tenant_id, write_id) REFERENCES kernel.write (tenant_id, write_id)
) PARTITION BY HASH (tenant_id);

CREATE TABLE kernel.outbox (
  tenant_id    uuid   NOT NULL,
  outbox_seq   bigint NOT NULL,                     -- per tenant, assigned under a lock
  write_id     uuid   NOT NULL,
  context      bytea  NOT NULL,                     -- the sixteen-field write context
  enqueued_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, outbox_seq),
  UNIQUE (tenant_id, write_id),                     -- one outbox row per governed action
  FOREIGN KEY (tenant_id, write_id) REFERENCES kernel.write (tenant_id, write_id)
);
```

Gate results are one row per declared gate, passed or failed, in declared order, because rung 3 runs every declared gate even after the first failure so that "an operator sees every problem at once" [E02 §05, decision 5]. They are deliberately outside the proof preimage — they "vary in length and structure, and a variable-shape member in a frozen preimage is a future incompatibility" — and are bound instead through the declaration version and exported beside each record [E12 §03; E15 §03]. That binding is only as good as the store: a gate-result row that could be edited after the fact would make the auditor's "which gates ran" question answerable only by trust. Hence `kernel.gate_result` is insert-only and on the append-only list in §9.11, which extends EBB's seven (build recommendation). The `inputs_hash` column gives the verifier something stable to cite; the `inputs` column gives the auditor the values the gate saw [E11 §2].

Ordering is established by the outbox, and the outbox must be ordered by commit, not by insert. The kernel guarantees that "within a tenant, records are in commit order, not arrival order" [E02 §05; E05 §08]. A naïve outbox with a global `bigserial` does not give that: two concurrent transactions for one tenant can take identifiers 10 and 11 and commit as 11 then 10. The recommended fix is to assign `outbox_seq` as `max + 1` for the tenant while holding a transaction-scoped advisory lock on the tenant, taken at the start of the transaction that inserts the outbox row and released only at commit. Rolled-back transactions leave no row and consume no number, so `outbox_seq` is gap-free and its order is the commit order **(build recommendation)**. The earlier working tree used the same serialisation point — `pg_advisory_xact_lock(hashtext(tenant_id))` with a read of the head before append on every branch, "including refusals" — and the corpus notes that "that lock is exactly the serialisation point a sequence number and an anchor would use" [HFS]. Refusals and escalations also enqueue an outbox row, because a refusal record is "produced by exactly this flow" [E12 §05].

The rung-7 transaction is where "record-or-nothing" lives: `BEGIN · record version · enqueue exactly one outbox row · COMMIT` [E04 rung 7]. If the chain append at rung 8 then fails, "the rung-7 transaction already committed, so the outbox row survives and the chain writer retries in order; no lost write, no gap in the sequence" [E02 §05]. The idempotency key (`intent_id`) is retained 30 days; a repeat returns the original decision including `seq` and `digest`, does not re-run the rungs and does not meter again [E05 §08; E02 §05].

9.6 Approvals and obligations

"An approval is itself a governed write. It is not a flag" [E02 §03]. It has its own record and its own sequence number in the tenant's chain, which is why the approver's identity is deliberately outside the approved write's preimage: "binding it twice would let the two copies disagree" [E12 §03]. The fields are fixed by the kernel contract: `approval_id`, `write_id`, `approver_id`, `approver_kind` (human only at L1), `decision` (approve or decline), `basis` (the surface plan version and the rendered payload digest) and `at` (platform clock) [E02 §03]. The console path is described in Chapter 8.

```
-- Approvals (build recommendation; fields from E02 §03, rules from E02 C11 and E11 §6 test 5)
CREATE TABLE kernel.approval (
```

```
  tenant_id           uuid  NOT NULL,
  approval_id         uuid  NOT NULL,
  write_id            uuid  NOT NULL,             -- the write being approved
  approval_write_id   uuid  NOT NULL,             -- the approval's own governed write
  approver_id         uuid  NOT NULL,
  approver_kind       text  NOT NULL CHECK (approver_kind = 'human'),   -- agents never approve
  requester_id        uuid  NOT NULL,             -- copied from the write at approval time
  decision            text  NOT NULL CHECK (decision IN ('approve','decline')),
  surface_plan_version bigint NOT NULL,
  rendered_digest     bytea NOT NULL CHECK (octet_length(rendered_digest) = 32),
  at                  timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, approval_id),
  CHECK (approver_id <> requester_id),            -- GATE.DUAL_CONTROL_SAME_ACTOR, in DDL
  FOREIGN KEY (tenant_id, write_id) REFERENCES kernel.write (tenant_id, write_id)
);

-- kernel.obligation (write_id, trigger_level, expires_at, resolvers[] with no agents)
-- Resolution is a new row in kernel.obligation_event, never an UPDATE of obligation.
```

The `CHECK (approver_id <> requester_id)` constraint is the database's copy of `GATE.DUAL_CONTROL_SAME_ACTOR`; the gate refuses first, and the constraint makes a defect in the gate unable to store a same-actor approval [E02 §06; E02 C11]. `approver_kind = 'human'` encodes "an agent may never be an approver at L1" [E02 decision 7]. The `rendered_digest` is what makes "what did the approver actually see" answerable rather than reconstructed [E11 §3]. Declining is a governed outcome: the original write refuses with `POLICY.APPROVAL_DECLINED`, and an expired obligation refuses with `POLICY.OBLIGATION_EXPIRED`, "itself a recorded governed event" [E02 §03, §06].

The verifier profile registry is the only P8 artefact the kernel reads. A profile version is written append-only with its digest, author, certifier and `rule_as_at` date, "verbatim and forever", under dual control where the certifier is never the author [E13 §03]. Profiles are keyed by `(profile_id, profile_version)` and are not tenant data; registration is idempotent on `(profile_id, profile_version, digest)` [E13 §03].

9.7 The proof log: the hash-chained Level 1 record

Every governed action produces exactly one record, "including the ones that did not happen": a refusal produces "a record of the same shape as one it allowed, with the target record identifier set to zeros" [E11 §1]. The record format is `elan1.proof.seal.v6`, sixteen fields fixed in number and order; "the layout does not vary by customer, by change type or by assurance level" [E11 §2; E12 §02]. At L1 the record is hash-chained and unsigned. Rung 8 "does" four things — consume the outbox in order, assign `seq` per tenant from 1, chain `prev_digest`, compute SHA-256 over the frozen sixteen-field preimage — and appends to the evidence store, which "grants INSERT to one principal and UPDATE to none" [E04 rung 8]. Figure 13 shows the structure.

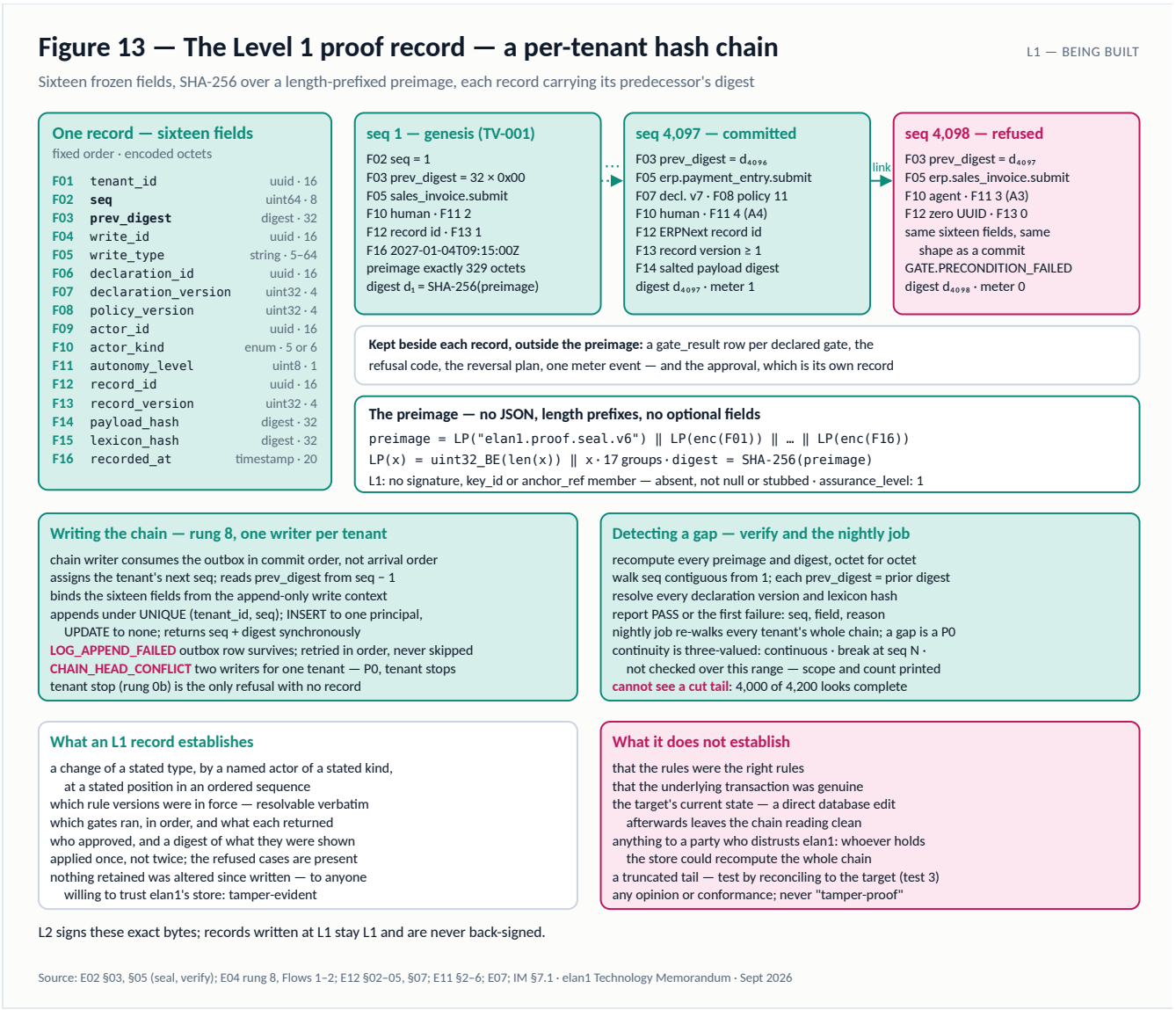


Figure 13 — The Level 1 proof chain: each record carries sixteen fields including the digest of its predecessor; genesis carries thirty-two zero octets; refusals carry the zero record identifier; the chain writer appends under a unique (tenant, seq) key and the nightly job recomputes every link.

9.7.1 What a record holds

The field definitions, encodings and the reason for each field are in Chapter 10 [E12 §02]. Three storage decisions belong here. Unsigned integers are stored as `bigint` with range checks (`uint32` fields bounded at 2^32 − 1), because PostgreSQL has no unsigned types and a silent sign flip would change the preimage. Text fields carry the constraints the encoder relies on — `write_type IS NFC NORMALIZED` and 5–64 octets — so a value the encoder would reject cannot be stored. And `recorded_at` is stored as the exact 20-character string that entered the preimage, not as a `timestamptz`, so that no driver, time-zone setting or precision conversion can change what a verifier recomputes (build recommendation). The platform clock is synchronised to NPL or NIC NTP servers under the CERT-In baseline [O05 §5]; the timestamp is "the platform's clock at the moment of recording, never the caller's" [E11 §2]. A refusal stores the zero UUID in F12 and 0 in F13 [E12 §02].

9.7.2 The chaining rule, genesis, and three unresolved values

The rule is short. For a tenant *t*, record *n* has `seq = n`; its `prev_digest` is the digest of record *n* − 1, or thirty-two zero octets when *n* = 1; its digest is SHA-256 over the length-prefixed preimage of the domain string and the sixteen encoded fields [E12 §02–§03]. Sequence numbers start at 1 "and never skip" [E11 §2]; a gap "is a P0, not a warning" [E02 §05]. Each tenant has its own chain; across tenants "there is no ordering guarantee and none is implied" [E02 §05].

Genesis is the first record the chain writer writes for the tenant, whatever it is. A tenant whose first record is a refusal has a refusal at genesis, and that case is one of the published positive vectors [E12 §07]. In practice the first governed writes in a tenant are the registration of its first declaration and its first grants, which are themselves governed writes under dual control [E02 §05]; genesis is therefore normally a registry write, not a business write.

The sources leave three values under-specified for refusals raised before the rung that would produce them. A refusal at rung 0b, 1 or 2 is raised before rung 4 resolves a policy version, and a refusal at rung 2 for an unresolvable declaration (`POLICY.DECLARATION_UNRESOLVED`) is raised before a lexicon can be resolved [E02 §05–§06]. Yet the record has "no optional fields" and a refusal "carries the zero UUID in F12 and 0 in F13; it does not omit them" [E12 §03]. The recommended treatment is to extend the same zero convention: `policy_version = 0` when rung 4 was not reached, and `lexicon_hash` = thirty-two zero octets when no declaration version resolved; F11 records the actor's resolved grant (missing resolves to A0) regardless of whether rung 6 ran **(build recommendation)**. These must become published vectors before any implementation freezes, because an implementation that fills them differently will produce a different digest for the same event. The gap is recorded in the conflicts block.

9.7.3 The chain writer

The chain writer is the only process that may assign a sequence number, and there is exactly one per tenant at a time. Two writers for one tenant is `PLATFORM.CHAIN_HEAD_CONFLICT`, "a P0 that stops the tenant" [E04 rung 8; E12 §05]. The flow is fixed by E12: consume the outbox in commit order; assign the next `seq`; read `prev_digest` from `seq − 1`; BIND the sixteen fields from the write context, which is append-only within a write so "every field is already present"; PREFIX; DIGEST; CHAIN under `UNIQUE (tenant_id, seq)`; return `seq` and digest to rung 8 synchronously [E12 §05]. "If BIND ever has to compute something, a stage upstream has not finished its job" [E12 §05]. The writer does not refuse; "a refusal is an input to it" [E12 §05].

```
# Chain writer loop for one tenant (build recommendation; flow from E12 §05, E02 §05)
ZERO32 = bytes(32)

def append_next(conn, tenant_id):
    with conn.transaction():
        conn.execute("SET LOCAL e1.tenant_id = %s", [tenant_id])
        # One writer per tenant: a session-independent, transaction-scoped lock.
        got = conn.scalar("SELECT pg_try_advisory_xact_lock(%s, %s)",
                          [CHAIN_LOCK_NS, tenant_lock_key(tenant_id)])
        if not got:
            raise ChainHeadConflict(tenant_id)          # PLATFORM.CHAIN_HEAD_CONFLICT -> P0
        head = conn.one("SELECT seq, digest FROM evidence.proof_record "
                        "WHERE tenant_id = %s ORDER BY seq DESC LIMIT 1", [tenant_id])
        next_seq, prev = (1, ZERO32) if head is None else (head.seq + 1, head.digest)
        row = conn.one("SELECT outbox_seq, write_id, context FROM kernel_fdw.outbox "
                       "WHERE tenant_id = %s AND outbox_seq = %s",
                       [tenant_id, sealed_outbox_head(conn, tenant_id) + 1])
        if row is None:
            return None                                  # nothing committed yet; wait
        fields = bind(row.context, seq=next_seq, prev_digest=prev,
                      recorded_at=platform_clock_rfc3339_seconds())
        d = sha256(preimage(fields))                     # Chapter 10, normative encoding
        conn.execute(INSERT_PROOF_RECORD, [*fields.values(), d, 1])  # assurance_level 1
        conn.execute(INSERT_SEALED_ACK, [tenant_id, row.outbox_seq, row.write_id, next_seq])
    return next_seq, d
```

Four properties of the sketch are deliberate. The advisory lock is transaction-scoped, so a crashed writer releases it and a second writer cannot interleave; the head is read with an index scan rather than `SELECT … FOR UPDATE`, because `FOR UPDATE` requires an UPDATE privilege that the chain role must not have. The outbox is consumed strictly in `outbox_seq` order with no skipping: if row *k* + 1 is not yet visible, the writer waits rather than taking *k* + 2, so a slow transaction can delay the chain but cannot reorder it [E02 §05]. Idempotency is keyed on `write_id`: a re-consumed outbox row "returns the existing `seq`" because `UNIQUE (tenant_id, write_id)` makes the second insert fail and the writer then reads the existing record [E02 §05]. And `PLATFORM.LOG_APPEND_FAILED` is handled by retrying the same row in order — "never skipped, never reordered, never assigned a later `seq`" [E12 §05].

The acknowledgement row lives in the evidence database and is read back by the kernel through a read-only foreign table, so the action store is never updated (build recommendation). The synchronous return means the caller sees `seq` and digest on every outcome, including refusals [E04 rung 8]; the kernel budget models rung 8 at about 12 ms p50 in one source and about 40 ms in another, both unmeasured and unreconciled [IM §6.10; E10 §07].

9.7.4 The table, with the chain enforced by the database

```
-- Proof log, L1 (build recommendation; field semantics E12 §02, rules E04 rung 8, E12 §05)
CREATE TABLE evidence.proof_record (
  tenant_id            uuid   NOT NULL,
  seq                  bigint NOT NULL CHECK (seq >= 1),
  prev_digest          bytea  NOT NULL CHECK (octet_length(prev_digest) = 32),
  write_id             uuid   NOT NULL,
  write_type           text   NOT NULL CHECK (write_type IS NFC NORMALIZED AND
                              octet_length(convert_to(write_type,'UTF8')) BETWEEN 5 AND 64),
  declaration_id       uuid   NOT NULL,
  declaration_version  bigint NOT NULL CHECK (declaration_version BETWEEN 1 AND 4294967295),
  policy_version       bigint NOT NULL CHECK (policy_version BETWEEN 0 AND 4294967295),
  actor_id             uuid   NOT NULL,
  actor_kind           text   NOT NULL CHECK (actor_kind IN ('human','agent','system')),
  autonomy_level       smallint NOT NULL CHECK (autonomy_level BETWEEN 0 AND 5),
  record_id            uuid   NOT NULL,
  record_version       bigint NOT NULL CHECK (record_version BETWEEN 0 AND 4294967295),
  payload_hash         bytea  NOT NULL CHECK (octet_length(payload_hash) = 32),
  lexicon_hash         bytea  NOT NULL CHECK (octet_length(lexicon_hash) = 32),
  recorded_at          text   NOT NULL CHECK (recorded_at ~
                              '^[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}Z$'),
  digest               bytea  NOT NULL CHECK (octet_length(digest) = 32),
  assurance_level      smallint NOT NULL CHECK (assurance_level IN (1, 2)),
  -- derived columns used only by constraints
  prev_seq             bigint GENERATED ALWAYS AS (NULLIF(seq - 1, 0)) STORED,
  is_null_record       boolean GENERATED ALWAYS AS
                         (record_id = '00000000-0000-0000-0000-000000000000') STORED,
  PRIMARY KEY (tenant_id, seq),
  UNIQUE (tenant_id, write_id),
  UNIQUE (tenant_id, seq, digest),                        -- target of the chain FK
  UNIQUE (tenant_id, seq, is_null_record),                -- target of the meter FK (§9.8)
  CHECK ((seq = 1) = (prev_digest = e1_zero32())),        -- genesis iff 32 zero octets
  CHECK (is_null_record = (record_version = 0)),          -- refusal shape, both or neither
  -- The chain itself: record n must name record n-1's digest, and n-1 must exist.
  FOREIGN KEY (tenant_id, prev_seq, prev_digest)
    REFERENCES evidence.proof_record (tenant_id, seq, digest)
) PARTITION BY HASH (tenant_id);

REVOKE ALL ON evidence.proof_record FROM PUBLIC;
GRANT INSERT, SELECT ON evidence.proof_record TO e1_chain;
GRANT SELECT ON evidence.proof_record TO e1_verify, e1_integrity, e1_reconcile;
-- No UPDATE, DELETE or TRUNCATE to anyone, including e1_chain and e1_owner at runtime.
```

The self-referencing foreign key is the most useful line in the table. Because `prev_seq` is `NULL` at genesis, `MATCH SIMPLE` semantics exempt record 1; for every other record, the database refuses the insert unless a record `seq − 1` exists for the same tenant with exactly the digest this record names. A gap, a fork and a mis-linked record are therefore impossible to write through any path, including a defect in the chain writer, and the nightly job (§9.7.7) becomes a second, independent check rather than the only one **(build recommendation)**. The digest itself is not recomputed by the database. Recomputation belongs to code that shares nothing with the writer — the integrity job and, at L2, the offline verifier — because "the whole value of recomputing the digest evaporates if the recomputation uses the function that computed it" [E12 §09].

`e1_zero32()` is an immutable SQL function returning `decode(repeat('00', 32), 'hex')` (build recommendation).

9.7.5 Gap detection and the verification queries

The `verify` call recomputes each preimage and digest over a caller-supplied range, walks the chain for `seq` contiguity and `prev_digest` linkage, resolves every referenced declaration version and lexicon hash, and returns PASS or the first failure "with sequence, field and reason" [E02 §05]. At L1 "verify requires us — state that limitation to the customer in those

words" [E02 §05]. It is never metered and "never … the only integrity check" [E02 §05]. The structural half of that walk can be written as queries; the digest half is done in the integrity worker's own code.

```
-- Gaps: every seq in [lo, hi] with no successor-by-one (build recommendation)
SELECT seq + 1 AS missing_from,
       next_seq - 1 AS missing_to
FROM (
  SELECT seq, lead(seq) OVER (ORDER BY seq) AS next_seq
  FROM evidence.proof_record
  WHERE tenant_id = current_setting('e1.tenant_id')::uuid AND seq BETWEEN $1 AND $2
) s
WHERE next_seq IS NOT NULL AND next_seq <> seq + 1;

-- Broken links: prev_digest not equal to the predecessor's stored digest
SELECT r.seq, 'prev_digest' AS field, p.digest AS expected, r.prev_digest AS found
FROM evidence.proof_record r
LEFT JOIN evidence.proof_record p
  ON p.tenant_id = r.tenant_id AND p.seq = r.seq - 1
WHERE r.tenant_id = current_setting('e1.tenant_id')::uuid
  AND r.seq BETWEEN $1 AND $2
  AND r.seq > 1
  AND (p.seq IS NULL OR p.digest <> r.prev_digest);

-- Range completeness: count must equal width; a short count is a gap even at the ends
SELECT count(*) = ($2 - $1 + 1) AS complete, min(seq) = $1 AS starts, max(seq) = $2 AS ends
FROM evidence.proof_record
WHERE tenant_id = current_setting('e1.tenant_id')::uuid AND seq BETWEEN $1 AND $2;
```

With the foreign key in place these queries should always return empty; they exist because the nightly job must not trust the schema it is checking (a dropped constraint is exactly the change it must detect), and because a restore from backup can reintroduce a state the constraint never saw (§9.13). The digest recomputation reads each row, encodes the sixteen fields under the Chapter 10 rules in the worker's own encoder, and compares the result with the stored `digest`; a mismatch names the `seq`, the stored value and the recomputed value, in the E12 style: "FAIL at seq 3,181: prev_digest mismatch, expected a376… got 0000…" [E12 §08].

9.7.6 No signature member at L1

"At L1 the signature field is absent, not stubbed. Not null, not an empty string, not thirty-two zero octets, not the word unsigned" [E12 §04]. A nullable `signature` column on `proof_record` would satisfy "absent" in the database only in the sense that `NULL` means absent; it would also invite an export path to serialise it as `"signature": null`, which E12 forbids and which a conforming verifier fails ("a verifier that finds one on an assurance_level: 1 record fails the record") [E12 §04, §07]. EBB's domain model lists `signature` as a column of `seal` [EBB EB-01 sheet E02]; this memorandum departs from it.

Recommendation (build recommendation). `proof_record` has no signature, key or anchor column. The L2 migration adds three insert-only tables — `evidence.record_signature (tenant_id, seq, key_id, signature)`, `evidence.anchor_batch` and `evidence.inclusion_path` — each keyed on `(tenant_id, seq)` or the batch, each with its own single writer. An L1 record is one with no row in `record_signature`; the export path emits a signature member only when a row exists and `assurance_level = 2`. At L1 the tables do not exist at all, which keeps E01's non-goal literal: "No key material anywhere. Not in a config file, not in a vault, not 'just for testing'" [E01 §07].

The assurance level is stored per record and never updated. Records written before a tenant's upgrade "stay L1 and are labelled so, permanently"; they are not signed retrospectively, because "back-signing a record produced before a key existed would be precisely the practice this system is built to make impossible" [E11 §5; E07 decision 2]. The boundary — the first `seq` at L2 — is itself recorded as the governed write that enabled signing [E19 §04].

9.7.7 The nightly integrity job and the continuity report

A job runs nightly "across every tenant's whole chain and re-computes it. A break is treated as a top-severity incident, not a warning" [E11 §2]. The same job underpins three L1 definition-of-done items: zero writes committed without a chained record, zero records mutated outside the kernel writer, and every refusal replayable [E01 §06]. The status page has chain

integrity as its own component, and "a missed nightly integrity job flips the component without anyone pressing anything" [E09 §02 item 4].

The job's output is the artefact the auditor asks for, so its shape matters. E11 requires a chain continuity report "stated three-valued — continuous / break at record N / not checked over this range — with the scope and the count printed", and warns that "a chain verified over three of twenty-three change types is a chain verified over almost nothing, and it reads identically to a full verification unless the report prints the scope" [E11 §6].

```
-- Integrity run output (build recommendation; shape from E11 §6 and E13 §04)
CREATE TABLE ops.integrity_run (
  run_id            uuid   PRIMARY KEY,
  tenant_id         uuid   NOT NULL,
  started_at        timestamptz NOT NULL,
  finished_at       timestamptz,
  seq_from          bigint NOT NULL,
  seq_to            bigint NOT NULL,
  record_count      bigint NOT NULL,
  scope             text   NOT NULL
                    CHECK (scope IN ('full_chain','declared_write_types','range')),
  write_types       text[] NOT NULL,                -- printed, never implied
  verdict           text   NOT NULL CHECK (verdict IN ('continuous','break','not_checked')),
  first_break_seq   bigint,
  first_break_field text,
  expected          bytea,
  found             bytea,
  worker_version    text   NOT NULL,                -- the encoder that recomputed
  CHECK ((verdict = 'break') = (first_break_seq IS NOT NULL))
);
```

Two design rules follow from E11 and E01. The job never reports `continuous` over a range it did not fully recompute; a run that dies part-way is recorded as `not_checked` with the range it reached, never as a partial pass (the same rule stack1 applies to its isolation runs, "a partial K2 is never reported as green") [S12 §2]. And a `break` pages at P0, stops new writes for that tenant through the rung 0b stop flag until the forensic question is answered, and is never auto-remediated: nothing in the platform may "repair" a chain, because a repaired chain is a rewritten one [E02 §05; O05 Flow A] (the stop on break is a build recommendation).

9.8 The meter ledger and the constraint that makes refusals free

Rung 9 emits exactly one meter event per governed action, with the basis "taken from the write type's declared blast radius, never configurable per customer", append-only, "reconciled nightly in both directions", and "every invoice line traces to a record" [E01 §02 item 9]. The meter event object is fixed by the kernel contract: `event_id`, `write_id` (always present), `seq`, `basis`, `quantity` ("1 for a committed write; 0 for every refusal, enforced by CHECK"), `outcome` (committed, refused, escalated) and `at` [E02 §03]. Escalations meter zero, and platform-class refusals are "not billable" [E05 §07; E02 §04].

9.8.1 The constraint

The group pricing model states the property in structural terms: "The meter event schema requires a quantity, a refusal's quantity is 0, and the database refuses to store anything else. A caller cannot forget to pass zero, because the caller does not pass it" [O07 §03]. EBB gives the constraint verbatim as schema rule 4: `CHECK (outcome <> 'refused' OR quantity = 0)` on `meter_event` [EBB EB-02 §3]. The recommended table tightens it in three ways **(build recommendation)**: quantity is exactly 1 for a commit and exactly 0 otherwise, so neither an escalation nor an over-count can be stored; the event must reference a proof record that exists; and the event's outcome must agree with the shape of that record.

```
-- Meter ledger (build recommendation; constraint of record: EBB EB-02 §3 rule 4, E02 §03)
CREATE TABLE evidence.meter_event (
  tenant_id       uuid   NOT NULL,
  event_id        uuid   NOT NULL,
  write_id        uuid   NOT NULL,
  seq             bigint NOT NULL,
  basis           text   NOT NULL CHECK (basis IN
```

```
                    ('one_record','one_counterparty','batch','tenant','cross_boundary')),
  outcome         text   NOT NULL CHECK (outcome IN ('committed','refused','escalated')),
  quantity        integer NOT NULL,
  at              timestamptz NOT NULL,
  is_null_record  boolean GENERATED ALWAYS AS (outcome <> 'committed') STORED,
  PRIMARY KEY (tenant_id, event_id),
  UNIQUE (tenant_id, write_id),                          -- exactly one event per action
  UNIQUE (tenant_id, seq),
  CHECK (outcome <> 'refused' OR quantity = 0),          -- the source constraint, verbatim
  CHECK ((outcome = 'committed' AND quantity = 1)
      OR (outcome <> 'committed' AND quantity = 0)),
  -- the event must point at a record, and a zero-quantity event at a zeroed record
  FOREIGN KEY (tenant_id, seq, is_null_record)
    REFERENCES evidence.proof_record (tenant_id, seq, is_null_record)
) PARTITION BY HASH (tenant_id);
```

The meter table sits in the evidence database beside the proof log in this design so that the foreign key can exist without crossing a database boundary; it is written by the kernel's rung-9 role, which holds INSERT on `meter_event` and nothing in `proof_record` (build recommendation). The composite foreign key means a meter row claiming `committed` cannot point at a refusal-shaped record, and a meter row claiming `refused` cannot point at a committed one — a mismatch between billing and evidence becomes a failed insert rather than a reconciliation finding. The constraint answers the question every CFO asks "within about ten minutes": whether the vendor is incentivised to make the system say no [IM §4.8]. The earlier proposal to bill refusals at 0.3× was overruled by the founder in writing, with a stated condition for revisiting (a customer demonstrating refusal-farming) [IM §4.8; E01 §09].

A throttle (HTTP 429) and a malformed request (4xx) never entered the kernel and produce neither a proof entry nor a meter event [E05 §06–§07]; sandbox tenants meter at zero through a billing view that excludes them (build recommendation) [E05 §05]. `UNIQUE (tenant_id, write_id)` makes conformance test C7 — a repeated `intent_id` "produces no second meter event" — a property of the table [E02 §08].

9.8.2 Nightly two-way reconciliation

`meter.reconcile()` runs nightly and compares proof records to billing lines [O07 §09]. The four assertions come from the L1 definition of done and conformance test C10: every committed write has one record; every record one write; every meter event a write that exists; every refusal quantity zero [E01 §06; E02 §08].

```
-- Direction 1: committed actions without a proof record (must be empty; non-empty is P0)
SELECT w.write_id FROM kernel_fdw.write w
LEFT JOIN evidence.proof_record r ON r.tenant_id = w.tenant_id AND r.write_id = w.write_id
WHERE w.tenant_id = current_setting('e1.tenant_id')::uuid AND r.write_id IS NULL
  AND w.received_at < now() - interval '5 minutes';          -- outbox SLO, see RB1

-- Direction 2: proof records without exactly one meter event (must be empty)
SELECT r.seq FROM evidence.proof_record r
LEFT JOIN evidence.meter_event m ON m.tenant_id = r.tenant_id AND m.seq = r.seq
WHERE r.tenant_id = current_setting('e1.tenant_id')::uuid AND m.seq IS NULL;

-- Direction 3: billable quantity per period, by write type, refusals shown at zero
SELECT r.write_type, m.outcome, count(*) AS events, sum(m.quantity) AS billable
FROM evidence.meter_event m
JOIN evidence.proof_record r USING (tenant_id, seq)
WHERE m.tenant_id = current_setting('e1.tenant_id')::uuid AND m.at >= $1 AND m.at < $2
GROUP BY r.write_type, m.outcome;
```

The five-minute threshold is a placeholder for the outbox SLO that EBB's runbook RB1 names but does not quantify [EBB EB-02 §5]. An invoice "is never issued over an unreconciled period"; a non-zero discrepancy "stops the invoice and pages us" [IM §4.8; E09 §07]. The refusal count appears on the invoice itself — "412 refused, Rs 0" — with a control that opens the underlying records by write identifier and sequence [IM §4.8]. When the kernel meter disagrees with rung1's credit ledger, "the kernel is right and the ledger is a display defect" [O07 §03]. The commercial side of the meter is in Chapter 19.

9.9 Keys, secrets and the salt store

At L1 the key store is empty by design: "No key material anywhere. Not in a config file, not in a vault, not 'just for testing'" [E01 §07; E12 §06]. What does exist at L1 are target-system credentials. The single writer mints a per-tenant, single-use credential at the moment of the write with a ten-minute life, scoped to one write, from a per-tenant credential broker; minted credentials expire in ten minutes because "a long-lived credential will end up in a log" [E01 §02 item 4; O05 §3]. All other secrets live in a managed secret store; a secret in a repository environment file fails a pre-commit hook and a CI scan; "one write credential per environment. A second one entering the codebase fails CI" [O05 §3].

From L2 the key register holds `key_id` (`k_<domain>_<YYYYMMDD>_<4 hex>`), `valid_from`, `valid_to`, `successor_id` and `status` (active, retired, revoked); windows "abut and never overlap"; the register is written append-only and registering a key is a governed write under dual control [E12 §06; IM §7.7]. Key material stays inside the custody module and never appears in any export; `keys.json` in a bundle "carries identifiers and validity windows and nothing else" [E12 §06]. Chapter 12 owns this design, including the disclosure that one signing key serves a whole residency domain rather than one tenant [IM §7.7].

The salt store holds the per-record secret behind F14's salted payload commitment. It sits at plane P0 with residency and key custody, in a separate database reachable only by `e1-salt` [O01 planes; ETS §05]. It is the one store where a row may be deleted, and deletion is the erasure mechanism itself (§9.12). Two source positions pull against each other here: E12, E02 and E11 describe F14 as "over the salted, canonicalised payload", salted per record, at L1 [E12 §02; E02 §03; E11 §2]; E01 lists "salted erasure" among L2 exclusions and "no erasure salt" among Phase 1 non-goals [E01 §03, §07]. Because F14 is written into every L1 record and the preimage cannot change at L2, the salt must exist from the first record or the L1 → L2 retrofit breaks. This memorandum therefore treats the salt store as an L1 component and records the conflict; the salting scheme itself is E12's open question 1, due 31 March 2027 [E12 open question 1].

```
-- Salt store, separate database e1_salt (build recommendation; ETS §05, O05 Flow B)
CREATE TABLE salt.record_salt (
  tenant_id   uuid  NOT NULL,
  write_id    uuid  NOT NULL,
  subject_ref bytea,                                  -- keyed digest of the data principal, for
                                                      -- erasure lookup; never the identifier
  secret      bytea NOT NULL CHECK (octet_length(secret) = 32),
  PRIMARY KEY (tenant_id, write_id)
);
-- INSERT by e1_salt at rung 7; DELETE only by the erasure job, itself a governed write
-- (irreversible: true, dual control). The deletion is recorded in salt.destruction_log.
```

9.10 Retention and residency

Retention is where the sources disagree most, and the disagreement changes what the storage tier must guarantee.

Class	O05 baseline	E-series	Statutory driver	Design in this memorandum
Proof chain (records)	7 years, WORM, India [O05 §5]	Indefinite; "the chain and its anchors are retained indefinitely" [E15 §04]; "we hold records indefinitely" [E11 §7]	MCA Rule 3(1): audit trail preserved at least eight years [IM §3.6, §10]; RBI draft MRM: ten-year retention, still a draft [IM §10]	Indefinite, with eight years (MCA) as the contractual floor; never shortened below the longest applicable statutory period
Declaration registry, lexicon, profiles	—	Forever, never compacted [E03 §07; E13 §05]	As above (a record must resolve its rules)	Forever
Gate results, approvals, obligations	—	Retained beside the record [E12 §03; E15 §03]	As above	As the chain
Payloads (in the target system)	—	Not in elan1's proof store; F14 commits without carrying them [E12 §03]	Target system's own regime	Not held by the evidence plane

Class	O05 baseline	E-series	Statutory driver	Design in this memorandum
Salts	—	Until erasure destroys them [E15 §03]	DPDP erasure	Until erasure or payload-retention expiry
Evidence bundles	24 months, WORM [O05 §5]	Not retained; manifest digest and export record kept in the chain [E15 §04, decision 4]	—	Not retained (E15)
Idempotency keys	—	30 days [E05 §08]	—	30 days
Sandbox snapshots	30 days (database snapshots) [O05 §5]	30-day rollback window [E01 §02 item 6]	—	30 days
Security and access logs	≥ 1 year, 180-day minimum in India [O05 §5]	—	CERT-In Directions 2022 [O05 §01]	As O05
Application logs	180 days, India [O05 §5]	—	CERT-In	As O05; no PII in logs [O05 §5]

The memorandum follows the later and more specific E-series positions for the chain and for bundles, reads O05's "7 years" as a floor that must be raised to eight for the MCA obligation, and records all three as conflicts to close. Per-regime retention profiles at L1 are an open question: E11 asks whether "indefinite retention [is] the only honest answer until Level 2", due 31 March 2027 [E11 open question 3]. SEBI's proposed five-year period and the EU AI Act's six-month minimum are both shorter than MCA's and do not bind the design [IM §13.7; E11 §7]. The storage cost of indefinite retention is modelled at about 4 KB per record including gate-result rows, ₹0.01 in the first year and ₹0.03 for a ten-year carry at Mumbai block-storage rates [E10 §07], all modelled.

Residency is a P0 property "which has no migration path — treat every choice here as permanent" [O05 §1]. Primary region ap-south-1, three availability zones; object store, queues, backups, evidence store and logs all in the same region; a CI check fails the build if a resource is declared outside the region allow-list; no second region, "including for disaster recovery, until a residency decision is written down and counsel-confirmed" [O05 §1]. A tenant lives in exactly one residency domain, and `cross_boundary` exists as a blast-radius value "precisely because leaving the boundary must be a declared, dual-controlled, proof-bound act rather than a join" [EBB EB-02 §7]. The residency gate G2 is one of the six gates that can never be skipped [E01 §02 item 1]. Sovereign and customer-VPC deployments, which multiply this footprint, are Chapter 17.

9.11 Append-only, enforced by grants

EBB fixes seven tables that "grant INSERT and nothing else — no UPDATE, no DELETE, to any principal including the worker that populates them": `seal`, `outbox`, `meter_event`, `declaration`, `certification`, `anchor_batch`, `refusal` [EBB EB-02 §3–§4]. The technology stack document names the mechanism: `GRANT INSERT` only, `REVOKE UPDATE, DELETE, TRUNCATE`, with `BEFORE UPDATE/DELETE` triggers that raise as belt-and-braces — "grants are the control; triggers are belt-and-braces" [ETS §05]. The check "reads the live catalogue rather than the migration files, because migrations describe intent and catalogues describe reality" [EBB EB-02 §4]. Evidence is additionally copied to object storage with Object Lock in compliance mode, per the O05 rule "PostgreSQL insert-only tables plus object storage with Object Lock (WORM) for evidence" [O05 §5; ETS §05].

This design maps EBB's `seal` to `proof_record` and extends the list to `gate_result`, `approval`, `obligation_event`, `integrity_run` and the L2 signature tables (build recommendation). The check is a query, run on every deploy and nightly:

```
-- Append-only invariant: must return zero rows (build recommendation; ETS §05, EBB EB-02 §4)
SELECT c.relname, g.grantee, g.privilege_type
FROM information_schema.role_table_grants g
JOIN pg_class c ON c.relname = g.table_name
WHERE g.table_schema IN ('evidence','registry','kernel')
  AND c.relname = ANY (current_setting('e1.append_only_tables')::text[])
  AND g.privilege_type IN ('UPDATE','DELETE','TRUNCATE');
```

EBB lists what the DDL "must never gain": a soft-delete column on an append-only table ("a DELETE with extra steps"), an `updated_at` on the seal ("a column that cannot be written is a column that invites someone to try"), a foreign key from evidence to records, and a "system" principal with broad grants [EBB EB-02 §8].

9.12 Deletion under DPDP against a log that is never rewritten

DPDP Rules were notified on 13 November 2025 with main obligations from 13 May 2027 [O05 §01; IM §10]. elan1 is a Data Processor for every tenant's data and a Data Fiduciary for its own users [O05 §7]. The erasure flow is fixed by the baseline: a request opens a case with a 72-hour clock; the system resolves every location of the subject's data — record, derived indexes, embeddings, caches — and shows the erasure plan before anything is deleted; erasure is itself a governed write type with `irreversible: true`, which forces dual control; on approval, records are erased, derived artefacts rebuilt or purged, "the subject's salt is destroyed in the erasure side store"; and "the proof records are retained with the subject's identifiers erased. The chain stays intact and verifiable; the person is no longer in it" [O05 Flow B]. A request that conflicts with a statutory retention obligation is refused in writing "with the obligation named and the date it expires" [O05 Flow B].

The data architecture makes that flow possible because the preimage never contained the payload. F14 commits to the payload without carrying it; "destroying the per-record salt makes the payload unrecoverable — including to us — while every digest, signature and anchor stays valid" [E12 §03]. Three consequences follow for the stores:

- **What is erased.** The salt row in `el_salt` (so the commitment can no longer be opened); any payload copy the evidence plane held (in this design, none — payloads live in the target system and in the tenant's sandbox snapshots, which expire at 30 days); personal fields in gate-result `inputs` and approval `basis` rendering (build recommendation: store these through the same salted-commitment scheme so erasure reaches them).
- **What survives.** Every proof record, its digest and `prev_digest`; every meter event; the erasure's own governed record. The actor and subject identifiers in the sixteen fields are opaque UUIDs, not personal data in themselves; whether they remain personal data once the mapping is erased is part of the counsel question below.
- **How it reads.** A verifier reports "chain intact, payload unbound", never FAIL; at L1 "the same wording applies to any record whose payload retention has expired" [E11 §4; E13 decision 4].

> **The legal question is open.** O05 asks counsel whether retaining a proof record with erased identifiers satisfies DPDP erasure, "given the hash chain cannot be rewritten", target 30 November 2026 [O05 open question 4]. The IM calls it "the sharpest unresolved legal tension in the corpus" and sets the deadline as before the first paying tenant and in no case later than 13 May 2027 [IM §10]. Three candidate answers exist and none is blessed: salted commitments with per-field disclosure; erasure of payload with retention of commitment under a statutory retention floor (eight years MCA, ten under the RBI draft); and tenant-held keys and salts [IM §10]. If the answer is unfavourable, the chain would be segmented so a per-subject sub-chain can be truncated without invalidating the parent — "engineering work of a quarter or two and a new suite identifier", not a rewrite [IM §10]. The data architecture above supports the first two answers without change; the third is L3 custody (Chapter 14).

9.13 Backups, restore, and why a restore is an integrity event

The baseline requires point-in-time recovery on the primary, a nightly encrypted backup retained 30 days, a quarterly restore drill "into a clean environment, timed, with the result recorded", a drill that includes "restoring the evidence store and re-running verify on a sample of proof records" and the conformance suite against the restored stack, and "no go-live without one successful drill on record"; RPO and RTO are "not yet set; set before go-live" [O05 §6; O05 open question 1]. The technology stack document adds that restore drills "must re-verify a bundle" [ETS §07].

9.13.1 What a restore can do to a chain

A point-in-time restore of an append-only chain is not neutral. Restoring to 10:00 when the chain head was at `seq` 4,200 at 10:30 silently removes records 4,101–4,200 — the exact truncated-tail failure a hash chain cannot detect [E11 §4]. Worse, the target system may have been restored to a different point, or not at all, so the two-way reconciliation of E11 test 3 will

then show committed writes in the target with no elan1 record: the finding E11 says "matters". A restore of the evidence database is therefore treated as an integrity event in its own right **(build recommendation)**:

Step	Action	Evidence produced
1	Declare the restore as an incident; set the rung 0b stop for affected tenants so no new writes chain onto a restored head [O05 Flow A]	Incident record; stop events
2	Restore evidence and kernel databases to the same recovery point; never one without the other	Restore log with both LSNs
3	Compare each tenant's restored head with the latest head checkpoint (§9.13.3)	Per-tenant list: head restored = checkpoint, or records lost from *a* to *b*
4	Re-drive the kernel outbox for committed writes whose proof records were lost; the chain writer re-chains them in `outbox_seq` order with new `recorded_at` values	Re-chained records; each carries a new platform timestamp, so the gap is visible, never closed by adjusting anything
5	Run the full integrity job and the two-way meter reconciliation over the affected range	`integrity_run` rows; reconcile rows
6	Notify affected tenants with the range, the cause and the re-chained counts; lift the stop	Governed record of the notice

Step 4 is only possible because the rung-7 transaction and the outbox row commit together in the kernel database; if the kernel database was also rolled back, the lost writes exist only in the target system, and the only honest output is the reconciliation list of target changes without an elan1 record. That list is shown to the tenant, not repaired.

9.13.2 Replicas and the single writer

The synchronous standby in a second availability zone is the normal protection against loss [O05 §1]. A failover promotes the standby; the chain writer's advisory lock is lost with the old primary and re-acquired on the new one, and the self-referencing foreign key ensures that any writer that resumed from a stale head fails on insert rather than forking the chain (build recommendation). Read replicas may serve `verify` and the console's Records door, never the chain writer.

9.13.3 Head checkpoints (an internal control, not an anchor)

To make step 3 possible, the integrity job writes, after each clean nightly run, one line per tenant — `tenant_id`, head `seq`, head `digest`, run id — to an object-store bucket with Object Lock in compliance mode, in a separate account administered by people who do not administer the database (build recommendation, using the WORM control O05 already requires [O05 §5]). A restore that lost records now has something to compare against, and an insider who truncated the chain in the database would also have to defeat the lock in a second account. This is not anchoring: the checkpoint is written by elan1 and held by elan1, it proves nothing to a party who distrusts elan1, and it may not be described externally as a timestamp or an anchor [E07 §02; E14 §05]. E07 already asks whether L1 needs "a named intermediate for tenants who want anchoring without signing" (open question 4, Q1 2027); the checkpoint is an internal precursor to that decision, not an answer to it [E07 open question 4].

9.14 The suppression attack, and why a single writer closes it

The standards literature on agent receipts names the failure mode no receipt format closes on its own: "a receipt proves a call happened; it cannot prove a call did not" [IM §3.4]. An agent that is compromised, misconfigured or routed around emits no receipt for the write it made out of band, and an observer holding a complete, valid, signed set of receipts sees nothing wrong. "Absence of evidence is not evidence of absence — unless the system producing the evidence is the only path to the act" [IM §3.4].

The data architecture answers it by coupling the store to the chokepoint rather than to the caller. Exactly one process mutates the target per tenant, holding the only credential and minting a single-use, ten-minute token scoped to one write; "then every write that landed went through the kernel, and a write in the target with no corresponding record is not a gap in the evidence but a finding" [IM §3.4; E02 §02 I2]. The stores contribute three things. The outbox shares a transaction with the target-side record version at rung 7, so there is no committed kernel write without a queued proof. The self-

referencing chain and the nightly job make a removed record detectable except at the tail. And E11 test 3 — every change in the target's own audit trail for the governed record types must have an elan1 record, and every elan1 committed record a change in the target — converts "absence" into a query with a result [E11 §6].

The attack is closed only in enforced mode. In attested advisory mode the customer's own software holds the credential and "could have written around the platform"; the record then supports "every write this product made through elan1 is recorded", never "every write to this system went through the gates" [E11 §6 test 6; E01 §05]. The mode is recorded per tenant (`core.tenant.enforcement_mode` above), shown in the product and disclosed to a reviewer on request without qualification [E11 decision 6]. It is deliberately outside the preimage and "rendered in every export" [E12 §03]. The surfaces and credentials behind the two modes are Chapter 6.

9.15 The audit note: what an auditor is given at Level 1

E11 is the artefact that "opens every auditor conversation and contains no marketing language"; its section on what a record does not establish "is never shortened, moved later or moved to an appendix in any derived version" [E11 decisions 1–2]. It opens with two facts: elan1 is pre-revenue with no customer in production, so nothing has been examined by an auditor in a live engagement; and the system runs at Level 1, so records are hash-chained but not signed [E11 §01].

What the auditor asks for. Five things, for the period under review: a sequence range; the chain continuity report for that range, three-valued, with scope and count; the declaration versions referenced, verbatim; the nightly integrity job's output for every night in the period; and the meter reconciliation for the period [E11 §6]. Each maps to a store in this chapter: `proof_record`, `integrity_run`, `registry.declaration_version`, `integrity_run` again, and the reconciliation output.

The six tests. [E11 §6]

#	Test	What the store must support
1	Contiguity: no gap from the start of the range; each `prev_digest` equals the preceding digest; re-computed in front of the auditor	The gap and link queries (§9.7.5) and an independent recomputation tool
2	Scope: read the continuity statement; insist the scope is printed	`integrity_run.scope` and `write_types`
3	Two-way reconciliation to the target system's own audit trail	`write_id` / `record_id` / `record_version` by value; an export of committed records for the period
4	Refusals: each carries a code, the raising stage, every gate's result and a named next action; sample and ask what happened next	`kernel.write` refusal columns, `gate_result`, the register's operator action
5	Segregation: second approver differs from requester, is a person, and saw what was applied	`kernel.approval` with `rendered_digest` compared with F14
6	Enforcement mode per change type	`core.tenant.enforcement_mode`, rendered

Test 3 is "the one that does the real work at Level 1" [E11 §6]; exceptions in the direction target-without-record "are changes that did not go through the gates". E15 asks whether a future bundle should carry a member for the target system's own audit trail so test 3 can be run offline (open question 5, 31 December 2027) [E15 open question 5].

The mapping table states help and limit, never satisfaction. For the MCA audit trail the record covers changes made through the platform; changes made directly in the accounting system are that system's trail and "the two must be reconciled", and the precise clause reference "should be confirmed with the entity's own CA; we do not assert it" [E11 §7]. At L1 "the record serves every one of these as an internal control and none of them as independent proof" [E11 §7; E07 §05].

What elan1 asks in a pilot, and what it will not ask. Half a day at the start for the three questions the auditor would ask; a written statement of what would be necessary to rely on a record of this kind (the input to the first verifier profile); running test 3 personally on real data; and corrections to the language. Not asked: an endorsement, a logo, a quotation, "or … anything we could present as your having accepted the artefact"; and "if you tell us Level 1 is not sufficient for your purposes, we will write that down and cite it" [E11 §8, decision 5]. The L2 pilot protocol that follows from this is in Chapter 11.

9.16 Acceptance criteria for the data layer

#	Criterion	Measured by	Source
D1	Zero committed writes without a chained record; zero records mutated outside the chain writer	Nightly reconciliation; non-zero is P0	E01 §06 items 3–4
D2	10,000 records verify; a flipped octet, a re-ordered pair, a duplicated `seq` and a gap each fail with `seq` and field named	Conformance test C9	E02 §08
D3	Every meter event references an existing write; every refusal meters zero; no second event on a repeated intent	Database CHECK and FK; C7, C10	E01 §06 item 7; E02 §08
D4	Cross-tenant read refused by the database for every tenant table	RLS test in CI	O05 §2
D5	Append-only grant check returns zero rows on the live catalogue	Every deploy and nightly	EBB EB-02 §4; ETS §05
D6	One restore drill into a clean environment, with evidence store restored and verify re-run; head-checkpoint comparison exercised	Quarterly; written up	O05 §6
D7	Erasure path tested end to end on a fixture tenant; verifier wording "chain intact, payload unbound"	Before first external user	O05 §12; E13 P7
D8	No signature, key or anchor column or member exists on an L1 record	Schema check; C14 string check	E12 §04; E02 C14

10. Seal Spec v6 — canonical fields, signatures and test vectors

This chapter specifies the proof record format `elan1.proof.seal.v6`: its sixteen fields, the canonical encoding of each, the length-prefixed preimage and SHA-256 digest, the Ed25519 signature and RFC 3161 anchoring that Level 2 adds, the transport object in which a record travels, the sixty-four test vectors, and the two-implementation rule that governs when any of it may be trusted. Chapter 9 describes how the record is stored and chained at L1; Chapter 11 describes the bundle and the verifier that read it; Chapter 12 owns keys and timestamp authorities.

> **Status.** The specification text (E12, v3.0, 21 September 2026) is written and marked "Draft for review" [E12 cover]. By its own rule it remains a draft, and no record produced under it may be shown outside the company, until two independent implementations have agreed byte for byte on all sixty-four vectors; that has not happened [E12 §01; IM §4.9]. The fields and encoding are used at L1 today in the sense that L1 writes all sixteen fields into the chain [E01 §03]; signing and anchoring are L2 and do not exist [IM §4.9]. The expected TV-001 digest is deliberately not written anywhere, including here [E12 decision 4]. Where this chapter proposes an implementation detail the sources leave open, it is marked **(build recommendation)**.

10.1 What the specification freezes, and why now

`elan1.proof.seal.v6` is "the one hard freeze in the platform: once a single record exists outside our building, the field set cannot change without invalidating it. There is no v6.1. A change is a new domain string and a new verifier, and the old records keep verifying under the old one" [E12 §01]. The strategy blueprint states the consequence plainly: seven of the programme's eight frozen interfaces can be versioned forward; this one cannot, because "a customer holding a bundle from 2027 must be able to verify it in 2035 with a verifier that knows only v6, and a verifier that knows only v6 cannot be taught about a seventeenth field" [ESB Ch.05 §1].

The order of work is deliberate. The specification is written in Phase 2 for a signature that ships in Phase 3, because E01 froze the sixteen fields at L1 "precisely so that L2 adds a signature over bytes that already exist, making the upgrade a retrofit rather than a migration. Writing the spec after the chain had millions of records in it would be writing it too late" [E12 §01; E01 §03]. That is the single most important architectural consequence for the L1 build: **every L1 record must already be encoded exactly as this chapter specifies**, because at L2 the signature is computed over the same 32-octet digest, and an L1 record whose digest was computed any other way can never join a verifiable chain [E14 §02; E19 §05].

10.2 The sixteen fields

Field names, order and encodings are normative; "there are no optional fields: every record carries exactly sixteen" [E12 §02]. The "source rung" column shows which rung of the Right Path (Chapter 5) produces each value into the append-only write context; it is this memorandum's mapping from E02 and E04, not a column in E12 **(build recommendation)**. The "why included" column is E12's reasoning.

#	Field	Type	Octets	Source rung	Why it is in the preimage
F01	`tenant_id`	uuid	16	Token, verified at 0b/1 [E02 §04]	Whose chain this is; a record cannot be replayed into another tenant's chain
F02	`seq`	uint64	8	8, chain writer only	Position in the chain is part of what is committed; "starts at 1 and never skips" [E11 §2]
F03	`prev_digest`	digest	32	8	Links to the predecessor; thirty-two zero octets at genesis
F04	`write_id`	uuid	16	Kernel, on intake; stable for the life of the intent [E02 §03]	The governed action this records

#	Field	Type	Octets	Source rung	Why it is in the preimage
F05	`write_type`	string	5–64	2 (write door)	`<slug>.<verb>` from the declaration; which declared kind of change
F06	`declaration_id`	uuid	16	2	"So the record commits to which rules decided, not merely that some rules did"
F07	`declaration_version`	uint32	4	2	Which version; "never 'latest'"
F08	`policy_version`	uint32	4	4 (evaluate)	Which rules evaluated it; "resolved by the kernel, never supplied"
F09	`actor_id`	uuid	16	1 (identity)	Who asked
F10	`actor_kind`	enum	5 or 6	1, from the token	`human` · `agent` · `system`, "never inferred"; "an agent did this at A3" becomes part of the sealed fact
F11	`autonomy_level`	uint8	1	6 (autonomy)	A0–A5 held at the moment, encoded 0–5
F12	`record_id`	uuid	16	7 (act)	Target-system record; the zero UUID on a refusal
F13	`record_version`	uint32	4	7	Version written; 0 on a refusal; orders two proofs against one record without a clock [EBB EB-02 §6]
F14	`payload_hash`	digest	32	7 (committed bytes); intake for refusals	"So the values are committed without the values being in the record"
F15	`lexicon_hash`	digest	32	2	"So the meaning of a declared term is bound to the record. 'Credit limit' in 2033 means what it meant on the day"
F16	`recorded_at`	timestamp	20	8, platform clock	"A caller-supplied timestamp is a caller-supplied lie waiting to happen"

Sources: E12 §02–§03; E02 §03; E11 §2. The earlier spellings `sealed_at` (F16) and "L0–L5" (F11) are retired; "a document reading sealed_at or 'autonomy L3' is v2.0 and means recorded_at and A3" [E12 §02, decision 1].

E12 is explicit about what BIND may not do: "every field is already present, because the context is append-only within a write and each stage wrote the fields it owns. If BIND ever has to compute something, a stage upstream has not finished its job" [E12 §05]. The source-rung column is therefore also an ownership table: a stage that fails to write its field is the defect, never the chain writer. Refusals raised before the rung that owns F08 or F15 have no specified value; Chapter 9 §9.7.2 recommends zero conventions and records the gap.

10.2.1 What is deliberately outside the preimage

Excluded	Why [E12 §03]
The payload itself	"A record containing payloads could not survive an erasure request." F14 commits without carrying it; destroying the per-record salt makes the payload unrecoverable, including to elan1, while every digest, signature and anchor stays valid
Gate results	Variable length and structure; "a variable-shape member in a frozen preimage is a future incompatibility". Bound through the declaration version and exported beside each record
The approver's identity	An approval is its own governed write with its own record and sequence number; "binding it twice would let the two copies disagree"
The enforcement mode	Declared, resolvable from F06/F07, rendered in every export; "not a sixteenth-and-a-half field"
Any vertical's extra field	"A seventeenth field does not exist. A vertical that needs to commit to more commits it through F14 as payload, or it does not ship"
A signature, at L1	Absent, not stubbed (§10.5)

10.3 Canonicalisation rules

The encoding table is normative and short [E12 §02, decision 2]:

Type	Encoding
uuid	16 octets, binary, big-endian as in RFC 9562 §4. Never the 36-character text form
uint64 · uint32 · uint8	8 · 4 · 1 octets, unsigned, big-endian
digest	32 octets, as produced
string	UTF-8, NFC normalised, no BOM, no terminator, no padding
enum	The lowercase ASCII token exactly as spelled in the specification
timestamp	ASCII RFC 3339, UTC, second precision, `Z` suffix: `YYYY-MM-DDTHH:MM:SSZ`, exactly 20 octets. No offset form, no fractional seconds

Each rule removes a place where two implementations would otherwise disagree. The implementation notes that follow are where competent engineers diverge in practice; each should become a sentence in the specification or a vector before the two teams start **(build recommendation)**.

- **UUID byte order.** RFC 9562 fixes network (big-endian) order for all fields. Platform APIs that expose a "GUID" in mixed-endian form (the first three groups little-endian) produce a different 16 octets for the same text; the encoder must parse the canonical text form and emit bytes in text order. The zero UUID is sixteen `0x00` octets.
- **NFC before length.** The 5–64 octet bound on F05 applies to the normalised UTF-8 bytes. PV-12 exists for exactly this case: "a multi-byte UTF-8 write_type requiring NFC normalisation" [E12 §07]. The encoder normalises; it does not reject a non-NFC input that normalises into range. The declaration registry should store the NFC form so the question never arises in production (Chapter 9 §9.4).
- **Enum tokens.** `human` and `agent` encode to 5 octets and `system` to 6, which is why E12 lists F10's length as "5 or 6" [E12 §02]. No capitalisation, no whitespace, no numeric codes.
- **`autonomy_level` is an integer, not a token.** A0–A5 encode as the single octet `0x00` – `0x05`, not as the text "A3" [E12 §02].
- **Timestamps.** Exactly 20 ASCII octets. `2027-01-04T09:15:00Z` is valid; `2027-01-04T09:15:00+00:00`, `2027-01-04T09:15:00.000Z` and a lower-case `z` are not. The value is truncated (not rounded) to the second from the platform clock at chain-write time (build recommendation; E12 says "second precision" without choosing).
- **Same-second ordering.** Second precision loses order within a second; `seq` carries it. E12 asks whether a same-second pair needs a vector, due 28 February 2027 [E12 open question 4]. The recommendation is yes: add a positive vector with two consecutive records carrying the same `recorded_at`, and a check that `recorded_at` is non-decreasing with `seq` (which the first profile's check C04 already requires) [E13 §07].
- **Length prefixes are always four octets.** `LP(x) = uint32_BE(octet_length(x)) ‖ x` applies to fixed-length fields too; a UUID is prefixed with `00 00 00 10`. An implementation that skips prefixes for fixed-width fields produces a shorter preimage and fails TV-001's octet count.

10.3.1 The payload commitment (F14)

F14 is "a one-way digest over the changed values, salted per record" [E11 §2], "over the salted, canonicalised payload" [E12 §02]. E12 does not specify the salting scheme or the payload canonicalisation, and lists the salting scheme as its first open question: "What is the exact salting scheme for F14, given that the erasure story only matters at L2 but a scheme chosen now must survive the retrofit?", due 31 March 2027 [E12 open question 1]. The IM describes the candidate the corpus leans towards: "the payload never travels, what is committed is a per-field salted digest with salts derived from one per-record secret, and destroying that secret makes every commitment unopenable — including to elan1" [IM §10].

The following construction implements that candidate without JSON anywhere in the digest path **(build recommendation)**:

```
s          = 32 random octets per record, held only in the salt store (Chapter 9 §9.9)
salt(f)    = HMAC-SHA-256(s, LP("elan1.payload.salt.v1") || LP(path(f)))
c(f)       = SHA-256( LP("elan1.payload.field.v1") || LP(salt(f)) || LP(path(f))
                      || LP(type(f)) || LP(value(f)) )
F14        = SHA-256( LP("elan1.payload.v1") || LP(uint32_BE(n))
```

```
                      || LP(c(f_1)) || ... || LP(c(f_n)) )      -- fields sorted by path octets

path(f)    = the declared field path, UTF-8 NFC, e.g. "items[0].amount"
type(f)    = the declared shape type token, e.g. "decimal", "string", "date", "uuid"
value(f)   = typed canonical bytes: decimal as a plain ASCII string with no exponent and no
             trailing zeros ("420000", "0.5"); date as YYYY-MM-DD; uuid as 16 octets;
             string as NFC UTF-8; boolean as 0x00/0x01; absent field omitted from the set
```

The construction gives three properties the corpus asks for. Destroying `s` makes every `c(f)` unopenable and leaves F14, the digest, the signature and the anchor valid [E12 §03]. Revealing `salt(f)` and `value(f)` for one field, with every other field present only as its opaque commitment `c(f)`, proves that field alone and discloses nothing about the others, which is the per-field disclosure a redaction profile or the L3 selective-disclosure work needs (Chapter 14) [E14 §03]. And the whole commitment is a function of declared field paths and types, so it depends only on the declaration version already bound by F06/F07. The alternative named in the stack document for export-level redaction is SD-JWT (RFC 9901) [ETS §02]; it can be carried in a bundle without changing F14, but it should not become the F14 construction, because it brings JSON and base64url into the digest path that E12 keeps free of both [E12 §03].

The decision has a hard deadline that is earlier than the one E12 gives. F14 is written into every L1 record; the first real tenant record makes the construction permanent for that chain. The scheme must therefore be frozen, with vectors, before rung1 makes its first governed write to a real customer's books — a Phase 1 event — not merely before L2 [E01 §06 item 10]. The conflict with E01's "no erasure salt" non-goal is recorded in Chapter 9.

10.4 The preimage and the digest

With `||` for concatenation [E12 §03]:

```
LP(x)     = uint32_BE(octet_length(x)) || x
preimage  = LP(DOMAIN) || LP(enc(F01)) || LP(enc(F02)) || ... || LP(enc(F16))
digest    = SHA-256(preimage)
DOMAIN    = "elan1.proof.seal.v6"        -- ASCII, 19 octets
```

"Seventeen length-prefixed groups, always, in that order" [E12 §03]. Figure 14 shows the pipeline from the sixteen fields to a sealed, anchored record, with the two-implementation gate that governs its release.

Figure 14 — Seal Spec v6 — sixteen fields to a signed, anchored seal

L2 — SPECIFIED, NOT BUILT

elan1.proof.seal.v6 is the one hard freeze: L1 chains these bytes now, L2 signs and anchors the same bytes

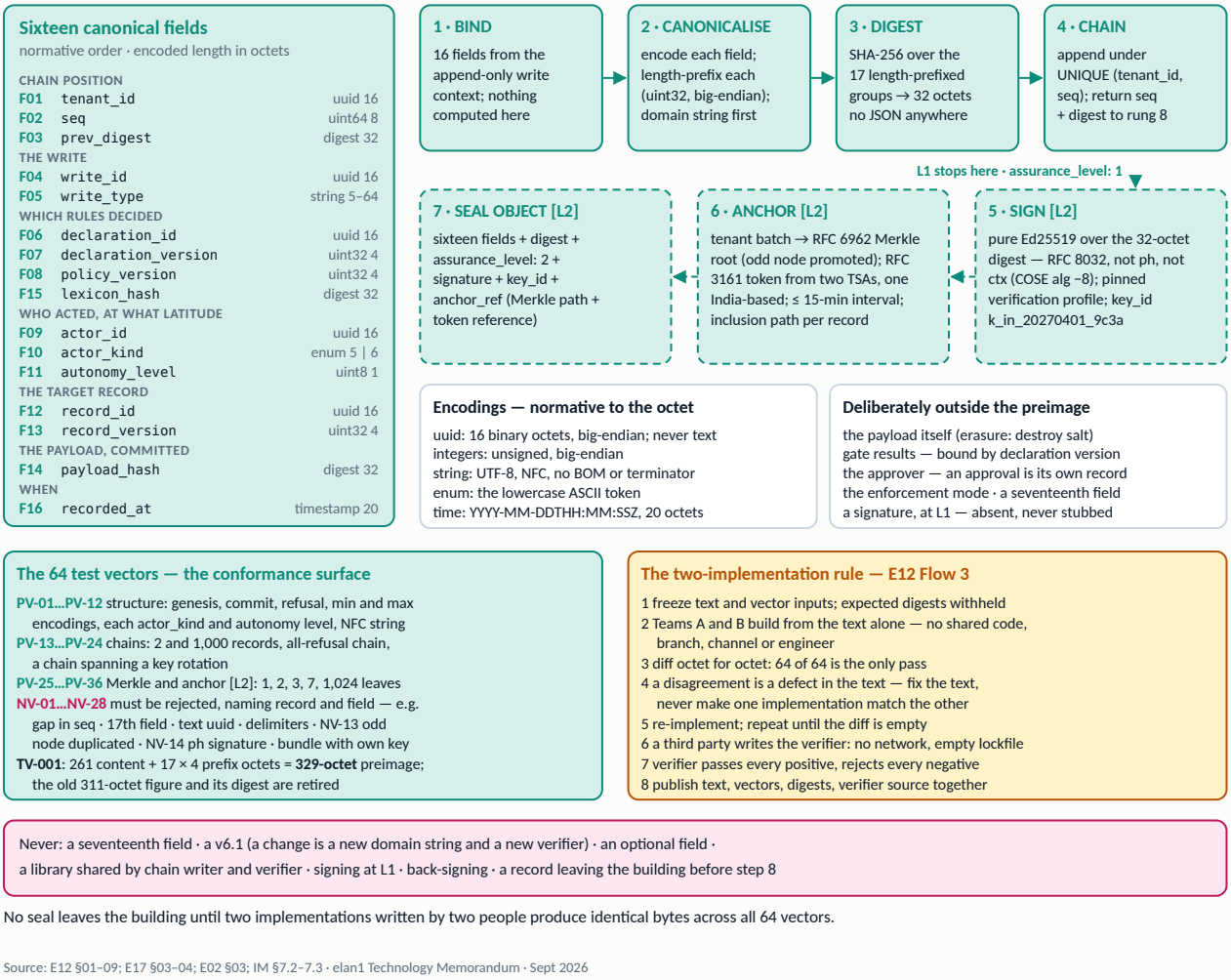


Figure 14 — Seal v6: sixteen fields are encoded and length-prefixed behind the domain string, hashed with SHA-256, and — at Level 2 only — signed with pure Ed25519 and anchored through a Merkle root at two RFC 3161 authorities; no record leaves the building until two independent implementations agree on all sixty-four vectors.

10.4.1 TV-001, the afternoon check

TV-001 is a genesis commit and "the check that costs an afternoon" [E12 §07]. Its octet count is normative:

Group	Field	Value in TV-001	Content octets
0	DOMAIN	`elan1.proof.seal.v6`	19
1	`tenant_id`	as published	16
2	`seq`	1	8
3	`prev_digest`	32 zero octets	32
4	`write_id`	as published	16
5	`write_type`	`sales_invoice.submit`	20
6	`declaration_id`	as published	16
7	`declaration_version`	1	4
8	`policy_version`	1	4
9	`actor_id`	as published	16

Group	Field	Value in TV-001	Content octets
10	`actor_kind`	`human`	5
11	`autonomy_level`	2	1
12	`record_id`	as published	16
13	`record_version`	1	4
14	`payload_hash`	as published	32
15	`lexicon_hash`	as published	32
16	`recorded_at`	`2027-01-04T09:15:00Z`	20
	Content		**261**
	Length prefixes	17 × 4	**68**
	Preimage		**329**

"An implementation that produces a 329-octet preimage for TV-001 has almost certainly got the length prefixes, the domain string and the binary uuid encoding right. One that does not has a bug it can find in an afternoon rather than in production" [E12 §07]. The earlier figure of 311 octets and the digest `a376103f…2b280d31` were computed under the v2.0 encoding table, which differed in the domain string and in F16; both are retired and "may not be quoted" [E12 §07, decision 3]. The Build Book still prints 311 on its seal sheet [EBB EB-01; EBB EB-05]; it is superseded.

10.4.2 Six choices and the failure each prevents

Choice [E12 §03]	Failure it prevents
No JSON anywhere in the digest	Number representation, key ordering, Unicode normalisation and escape forms each let two implementations disagree, and "a customer's proof fails for a reason no lawyer can follow"
Length prefixes, not delimiters	Adjacent delimited fields "can be made to collide by choosing content"
No optional fields	Optionality "is the mechanism by which two implementations produce different preimages for the same event"; a refusal carries zeros, it does not omit fields
A domain string in the first group	A digest computed under one specification "can never be mistaken for one computed under another"; it also stops a v6 preimage being replayed into any protocol with the same field order [ESB Ch.05 §2]
Platform clock for F16	A caller-supplied timestamp is attacker-controlled
Second precision for F16	Sub-second precision varies by platform and library, and "a field whose encoding depends on the host's clock resolution is a field two implementations will disagree about"

10.5 Level 1 and Level 2 records, and the Ed25519 signature

The field layout, the preimage and the digest are identical at both levels. What differs is what travels with the record [E12 §04]:

Member	L1 Guarded	L2 Attested
Sixteen fields, this encoding	yes	yes, unchanged
Preimage and digest	yes	yes, over identical bytes
Chain: `seq` contiguous, `prev_digest` linked	yes	yes
`assurance_level`	1	2
`signature`	**absent** from the record	`Ed25519(digest)`, pure — RFC 8032, not ph, not ctx
`key_id`	absent	present
`anchor_ref`	absent	Merkle inclusion path plus the RFC 3161 token reference

> **Absent, not stubbed.** At L1 the signature member is "not null, not an empty string, not thirty-two zero octets, not the word unsigned. A stub is a value, values get copied, and a copied stub eventually gets read as a signature by something that does not check" [E12 §04]. A verifier that finds a signature member on an `assurance_level: 1` record fails the record. A verifier reading a well-formed L1 record "reports UNSIGNED — assurance level 1 and exits PASS. It does not report FAIL. Unsigned is what L1 is" [E12 §04, decision 5].

What is signed. The signature covers the 32-octet digest, not the preimage: "pure Ed25519 over a 32-octet message" [E12 §04]. Pure Ed25519, Ed25519ph and Ed25519ctx "produce different signatures over the same message and are mutually incompatible. COSE EdDSA (alg −8) means pure. We say pure in the spec, in the code and in the vector" [E12 §04]. The signing key is the residency domain's key, invoked only by the chain writer at rung 8 [E12 §05 step 6; IM §7.7]. The recommended custody is AWS KMS with the Ed25519 key spec, pure mode, a key policy permitting signing to exactly one principal; Google Cloud KMS at HSM protection level is the named alternative (Chapter 12) [IM §7.7; ETS §06].

The verification profile is part of the specification. "Ed25519 strictness varies by library — cofactored versus cofactorless, small-order point handling, non-canonical scalars. Naming the algorithm is not enough" [E12 §04]. The stack document records the decision to reject non-canonical S and small-order points, proved by a published negative vector each library must reject [ETS §13 decision 3]. The recommended profile, stated so that a third implementer can write it without reading elan1 code **(build recommendation)**:

Check	Rule	Negative vector
Signature length	Exactly 64 octets (R ‖ S)	malformed-length (to add)
Scalar S	Reject unless S < L (the group order); no reduction	"a non-canonical scalar" [E12 §07]
Public key A	Reject non-canonical encodings and the eight small-order points, before any library call	"a small-order public key" [E12 §07]
R	Must decode to a curve point; no small-order check required	—
Equation	Cofactorless: `[S]B = R + [k]A`, k = SHA-512(R ‖ A ‖ M) mod L, M = the 32-octet digest	"a signature made with the ph variant", "… with ctx" [E12 §07]
Key window	`recorded_at` must fall within the `key_id`'s `[valid_from, valid_to)` window	"a record whose recorded_at falls outside its key_id's window" [E12 §07]

The explicit small-order and canonical-encoding checks are there because standard libraries differ on them; the verifier must not delegate the decision to whichever library it links. The `key_id` has the form `k_<domain>_<YYYYMMDD>_<4 hex>`, for example `k_in_20270401_9c3a`, "opaque to a verifier except for the domain segment" [E12 §06]. Key windows abut and never overlap; a revoked key's records still verify, and revocation limits what may be signed after the revocation instant, never what was signed before [E12 §06, decision 10]. Key material never appears in an export [E12 §06].

10.6 Anchoring at two authorities

At L2 the chain writer enqueues each digest into the tenant's current batch, and the anchor service closes the batch at the published interval (≤ 15 minutes), computes a Merkle root and obtains an RFC 3161 token over it [E12 §05 step 7]. The Merkle construction is fixed: RFC 6962 §2.1, `leaf(d) = SHA-256(0x00 ‖ d)`, `node(l, r) = SHA-256(0x01 ‖ l ‖ r)`, and "an odd final node is promoted, not duplicated" [E12 §05]. Each record's inclusion path is stored "so any single record proves alone"; a path is $\log_2(n)$ digests — ten digests or 320 octets for a 1,024-record batch [E12 §05; IM §7.6].

```
# RFC 6962 Merkle root with odd-node promotion (normative rule: E12 §05)
from hashlib import sha256

def leaf(d: bytes) -> bytes:
    return sha256(b"\x00" + d).digest()

def node(l: bytes, r: bytes) -> bytes:
    return sha256(b"\x01" + l + r).digest()
```

```
def merkle_root(digests: list[bytes]) -> bytes:
    level = [leaf(d) for d in digests]
    while len(level) > 1:
        nxt = [node(level[i], level[i + 1]) for i in range(0, len(level) - 1, 2)]
        if len(level) % 2:            # odd final node: promoted, never duplicated (NV-13)
            nxt.append(level[-1])
        level = nxt
    return level[0]
# Bottom-up promotion yields the same tree as RFC 6962's split at the largest power of two.
```

Every batch is anchored by two independent authorities, at least one India-based — eMudhra alongside one global commercial authority — and both tokens go into the bundle; "a bundle verifies if either verifies, and a bundle where the two disagree about the root is an integrity failure and a P0" [IM §7.6]. Anchoring never refuses a write; when one authority is unreachable the batch is anchored by the other and marked `single_anchor`; batches are never held open waiting and are never merged [IM §7.6]. One detail must be fixed in the text before the Merkle and anchor vectors (PV-25 … PV-36) are generated: whether the RFC 3161 `messageImprint` carries the 32-octet root itself under the SHA-256 algorithm identifier, or SHA-256 of the root. The recommendation is the root itself, since it is already a SHA-256 output **(build recommendation)**. Authority selection, cadence, failover and cost are Chapter 12.

10.7 The seal object: how a record travels

The digest path contains no JSON. A record must nevertheless travel — in the `assert` response, in the console, and one per line in a bundle's `seals.jsonl`, which carries "the sixteen fields, the digest, the assurance level, and at L2 the signature and key_id" [E15 §02]. E15 models one line at about 640 bytes at L2 [E15 §04]. The transport form is not specified in the sources; the following is recommended so that two exporters produce byte-identical lines, which E15 requires for re-export ("byte-identical except for exported_at") **(build recommendation)** [E15 decision 5].

```
{"format":"elan1.proof.seal.v6","assurance_level":1,"seq":"4098",
 "fields":{"tenant_id":"3f9a1c2e-7b44-4d1a-9e0f-5a6b7c8d9e01","seq":"4098",
  "prev_digest":"<64 lowercase hex>","write_id":"<uuid>",
  "write_type":"erp.sales_invoice.submit","declaration_id":"<uuid>",
  "declaration_version":7,"policy_version":11,"actor_id":"<uuid>","actor_kind":"agent",
  "autonomy_level":3,"record_id":"00000000-0000-0000-0000-000000000000",
  "record_version":0,"payload_hash":"<64 hex>","lexicon_hash":"<64 hex>",
  "recorded_at":"2027-09-14T10:02:41Z"},
 "digest":"<64 lowercase hex>"}
```

The rules behind the example: UUIDs in the transport object are the canonical lowercase hyphenated text form, converted to 16 binary octets by the encoder (the text form never enters the preimage, which is what negative vector "the 36-character text uuid encoding" tests); digests and signatures are lowercase hexadecimal; `seq` is a decimal string because uint64 exceeds the 2^53 integer range many JSON parsers use; `uint32` and `uint8` values are JSON integers; strings are the NFC values; keys appear in the fixed order shown, with no insignificant whitespace, one record per line terminated by a single `\n`. At L2 three members are appended in fixed order: `"key_id"`, `"signature"` (128 hex characters) and `"anchor_ref"` (batch identifier and leaf index; the path itself lives in `anchors/`) [E15 §02]. An L1 line has none of them — not even as `null` [E12 §04].

The action receipt standard (Chapter 15) names the construction as a registry identifier, `seal_suite: elan1-seal-v6`, and states that a receipt at conformance level 2 or above carries a pure Ed25519 signature over the digest [E25 §02]. The suite identifier and the domain string are different strings for different jobs: the domain string is inside the preimage; the suite identifier is a label outside it.

10.8 The sixty-four test vectors

"Sixty-four vectors, published with the specification text and with the verifier source. They are the conformance surface; the prose above is the explanation" [E12 §07]. They fall into four groups:

Group	IDs	Count	Covers [E12 §07]
Positive — structure	PV-01 … PV-12	12	Genesis; ordinary commit; refusal with zero UUID and `record_version` 0; every field at its minimum and at its maximum encoding; each of the three `actor_kind` tokens; each of the six `autonomy_level` values; `write_type` at 5 and at 64 octets; a multi-byte UTF-8 `write_type` requiring NFC
Positive — chain	PV-13 … PV-24	12	A 2-record chain; a 1,000-record chain; consecutive refusals; a chain of only refusals; a chain spanning a key rotation; a tenant whose first record is a refusal
Positive — Merkle and anchor [L2]	PV-25 … PV-36	12	One, two, three (odd node promoted), seven and 1,024 leaves; inclusion paths for first, middle and last leaf; an anchor token whose time precedes the check
Negative — must be rejected	NV-01 … NV-28	28	See §10.10

"An implementation that passes all the positives and fails to reject the negatives is not an implementation. Both directions are asserted, and the negative set is the one that matters" [E12 §07]. NV-13 (odd final node duplicated) and NV-14 (a ph signature) are named as the two places "where two competent engineers reading 'RFC 6962 Merkle' and 'Ed25519' most often build something self-consistent and mutually incompatible" [E12 §07].

10.8.1 How the vectors are organised (build recommendation)

The sources fix the groups and counts, not the file layout. The recommended layout keeps inputs, expected outputs and the disagreement log in one specification repository, owned by the lead, separate from every implementation:

```
seal-spec/                       # the only repository all three implementers may read
  spec/seal-v6.md                # normative text (E12), versioned with the vectors
  vectors/v6/
    inputs/PV-01.json … NV-28.json   # field values, chains, bundles; published at freeze
    expected/                         # EMPTY until two implementations converge (E12 §08)
      PV-01.expected.json             # preimage_len, preimage_hex, digest (after step 3)
      NV-01.expected.json             # verdict: reject, seq, field, class
  disagreements.md               # every disagreement, the sentence, the amendment (E12 §08)
  tampered/ref-bundle-tampered/  # the deliberately tampered bundle (E15 §07)
```

A positive input file carries the sixteen field values in the transport form of §10.7 and, for chain and Merkle vectors, the ordered list of records or digests. The expected file for a positive vector records the preimage length, the preimage in hexadecimal and the digest; for TV-001 the length is 329 and nothing else is written until both teams agree [E12 decision 4]. A negative input carries a record, chain or bundle that must be rejected; its expected file names the verdict, the `seq` and the field that must appear in the rejection, and the failure class (integrity, profile, incomplete) that Chapter 11's exit codes map to [E12 §08 step 7; E13 §04].

The vectors are used in five places: by implementation A in its unit tests; by implementation B in its unit tests; by the offline verifier's release gate, where every positive must PASS and every negative must be REJECTed "with record and field named" [E15 §07 step 3]; by every verifier profile's conformance test P4 [E13 §06]; and by the L1 → L2 upgrade precondition that the verifier "passes the published test vectors, including refusing the known-tampered bundle" [E07 §06 item 4; E19 §03].

10.9 The two-implementation rule, and how to run it in CI

> **The rule.** "Two implementations, written by two different people from this text alone, with no shared code and no shared channel, must produce identical bytes for every vector in the set. Until that has happened, this specification is a draft and no record produced under it may be presented to anyone outside the company. When the two disagree, the text is wrong and the text is fixed first. Making one implementation match the other is always the wrong repair" [E12 §01, decision 8].

The IM explains why this is a quality signal to a non-cryptographer: the promise of L2 fails "not when the mathematics is wrong … but when two honest programs reading the same document build slightly different things and one of them says a genuine record is fake" [IM §7.3]. The corpus has already shown the defect class on itself: an earlier working tree held three canonicalisations at once — a length-prefixed 13-field v2 seal in the backend, a six-field pipe-joined v1 mirror in a test script, and an eight-field newline-joined digest in the embedded verifier — and for the same seeded proof the backend and the mirror computed different hashes, a disagreement "currently invisible" because the test halted before reaching it [HFS evidence lane]. That is the failure the rule is designed to surface on day one rather than in an audit.

10.9.1 The validation run

E12 Flow 3 fixes eight steps [E12 §08]:

1. The lead freezes the text and the vector inputs and publishes both to the two teams; "the expected digests are not published, because a team given the answer is a team checking its arithmetic against ours rather than checking the text".
2. Team A and Team B implement from the text alone: "no shared code, no shared branch, no shared channel, no shared engineer".
3. The lead diffs the two outputs octet for octet; "a match on all sixty-four is the only pass".
4. Every disagreement is a defect in the text, recorded and fixed in the text first.
5. Both teams re-implement from the corrected text and return to step 3 until the diff is empty.
6. A third party, preferably neither team, writes the verifier from the same text; it shares no code with the chain writer, "imports nothing, reads no network, and ships with an empty lockfile".
7. The verifier must PASS every positive and REJECT every negative, naming the record and the field.
8. The lead publishes the text, the vector inputs, the agreed digests and the verifier source together, in one release; "only then may a record produced under this spec leave the building". Step 8 "does not move for a commercial deadline".

Convergence obtained after one team has read the other's code "is convergence that proves nothing about the text"; the run is void and restarts with a new pair of engineers [E12 §08]. The disagreement log — every disagreement, the sentence that caused it and the amendment — is "the most valuable document this process produces" and is published with the specification [E12 §08].

10.9.2 Languages and team separation

The stack document recommends three languages so that independence is structural: implementation A in Python with pyca/cryptography (it runs inside the `e1-seal` service and may have dependencies), implementation B in TypeScript on Node with `node:crypto`, and the verifier in Go with the standard library only [ETS §04, §13 decisions 1–2]. "A different language and team from impl A makes byte-level disagreements surface as spec defects" [ETS §01]. Who writes B — a second internal pair or a paid external team — is open until 30 April 2027; "internal is cheaper and shares our misreadings" [E12 open question 2].

"No shared channel" is enforced most reliably by access control rather than by instruction **(build recommendation)**: three repositories with disjoint write and read permissions (`seal-a`, `seal-b`, `verify`), one shared read-only repository (`seal-spec`), a separate chat channel per team, and a lead who is the only person with read access to all three. A dependency-review check fails any implementation that imports another's package, and the verifier's CI fails on a non-empty lockfile [E12 decision 9; ETS §09].

10.9.3 The CI pipeline

Each implementation repository emits one JSON output file per vector (`preimage_hex`, `digest`, or for negatives `verdict`, `seq`, `field`, `class`) as a build artefact. The spec repository runs the comparison. Before convergence the expected directory is empty and the job compares A with B; after convergence it compares both with the frozen expected files on every pull request to either implementation, so a regression in either is caught against the published answer.

```
# seal-spec/.github/workflows/two-implementations.yml (build recommendation)
name: seal-v6-two-implementations
on:
  workflow_dispatch:
  schedule: [{cron: "0 2 * * *"}]
  repository_dispatch: {types: [seal-a-built, seal-b-built, verify-built]}
jobs:
  compare:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - name: Fetch artefacts built from the same spec commit
        run: ./ci/fetch-artefacts.sh --spec-sha "$GITHUB_SHA" seal-a seal-b verify
      - name: Octet-for-octet diff of A and B over all 64 vectors
        run: ./ci/diff-outputs.py out/seal-a out/seal-b --require-count 64 --fail-on-any
      - name: Compare with frozen expected outputs (only after convergence)
        if: hashFiles('vectors/v6/expected/*.json') != ''
        run: ./ci/diff-outputs.py out/seal-a vectors/v6/expected --require-count 64
      - name: Verifier PASSes every positive and REJECTs every negative with seq and field
        run: ./ci/check-verifier.py out/verify --positives 36 --negatives 28 --named
      - name: Verifier lockfile is empty and the build is reproducible
        run: ./ci/check-verifier-hermetic.sh out/verify
```

Three controls in the pipeline carry the weight. `--require-count 64` fails a run that silently skipped vectors (a green diff over 60 files is a failure). The expected directory is written only by a protected job that runs once, after the lead has signed off an empty A-versus-B diff, which keeps step 1's rule that teams never see the answer first. And the job is triggered by any of the three repositories building, so the three stay in agreement for the life of the specification, not only at freeze.

After freeze, implementation B has a production job (build recommendation): it is the encoder used by the nightly integrity job (Chapter 9 §9.7.7), so the code that writes digests (A, in the chain writer) and the code that recomputes them every night (B) share nothing, which is the property E12 demands of the verifier applied inside the platform [E12 §09].

10.10 Negative cases and error handling

The negative set names what must be rejected [E12 §07]. The class column maps each to the failure class a verifier must report, which Chapter 11 turns into exit codes [E15 §06; E13 §04].

NV (order as listed in E12)	Must be rejected	Class
01–02	A flipped bit in `payload_hash`; a flipped bit in the digest	integrity
03–06	A re-ordered pair; a duplicated `seq`; a gap in `seq`; a `prev_digest` that does not match	integrity
07–09	A seventeenth field; a missing field; a null field	integrity
10–12	The 36-character text UUID encoding; delimiters in place of length prefixes; a wrong domain string	integrity (a wrong domain string may instead be *unsupported*; see below)
13	An odd final Merkle node duplicated rather than promoted	integrity
14–17	A ph signature; a ctx signature; a non-canonical scalar; a small-order public key	integrity
18	A bundle carrying its own verification key	integrity
19	An anchor whose timestamp postdates the check	integrity
20–21	A bundle missing a referenced declaration version; missing a referenced lexicon	incomplete
22	A record whose `recorded_at` falls outside its `key_id`'s window	integrity
23	An `assurance_level: 1` record carrying a signature member	integrity
24–28	Not individually listed in E12	To be published at freeze

Three points need settling in the text, and each is recorded as a conflict. First, E12 lists twenty-three negative cases for twenty-eight numbered vectors, so five are unnamed. Second, the listing order places the Ed25519 strictness cases at positions 16–17, while E12 elsewhere cites "Vector NV-06" as the one that catches a non-conforming Ed25519 library, and positive vector PV-07 is cited as the key-rotation boundary although rotation belongs to the chain group PV-13 … PV-24 [E12 §04, §06, §07]; the numbering must be fixed at freeze. Third, E13's profile test P4 expects each of the 28 negatives to exit `FAIL_INTEGRITY`, while E15 defines a missing declaration or lexicon member as `INCOMPLETE` (exit 3), which is what NV-20 and NV-21 describe [E13 §06; E15 §06]; this memorandum follows E15. A record under an unknown domain string should be `UNSUPPORTED` (exit 4) rather than integrity when the string is well-formed but unknown, and integrity when it is a malformed version of `elan1.proof.seal.v6` (build recommendation).

Error handling in the encoder. The encoder in A and B never repairs input. It raises a typed error naming the field and rule, and the chain writer treats any encoder error as `PLATFORM.LOG_APPEND_FAILED`: the rung-7 transaction has committed, the outbox row survives, and nothing is skipped [E12 §05]. The recommended error set **(build recommendation)**:

Error	Raised when	Example
`E_FIELD_MISSING`	Any of F01–F16 absent from the context (BIND found a gap)	Refusal context without F15
`E_FIELD_NULL`	A field present with no value	`record_id: null` on a refusal
`E_UUID_FORM`	Not a canonical UUID	Braced or upper-case GUID text in the context
`E_INT_RANGE`	Integer outside its unsigned range	`declaration_version = 0` or ≥ 2^32
`E_STRING_LENGTH`	F05 outside 5–64 octets after NFC	`write_type = "pay"`
`E_ENUM_TOKEN`	F10 not exactly `human`, `agent` or `system`	`Agent`
`E_TIMESTAMP_FORM`	F16 not exactly 20 octets of `YYYY-MM-DDTHH:MM:SSZ`	Fractional seconds
`E_DIGEST_LENGTH`	F03, F14 or F15 not 32 octets	Hex text passed instead of bytes

Error handling in the verifier. A verifier "does not refuse, it reports", and a report that says only FAIL "is useless in a dispute; one that says 'FAIL at seq 3,181: prev_digest mismatch, expected a376… got 0000…' turns a shouting match into a forensics exercise" [E15 §06; E12 §08].

10.11 Versioning: why v6, what changed, and what is frozen

The domain string is the version. There is no minor version of a domain string, no v6.1, and "a change is a new domain string and a new verifier"; old records keep verifying under the old one [E12 §01, §09]. The working brief for the specification asks it to say what a future v7 may change [WD Phase 2 01]; the strategy blueprint's answer is that a v7 "that can verify v6 is only possible if v7 is a strict superset and the domain string change is handled by the verifier reading both — which is a migration path we have explicitly said does not exist" [ESB Ch.17 R1]. The practical position is therefore: a v7 is a new suite with its own vectors and verifier, and a verifier release that supports v7 continues to support v6 unchanged; a chain never mixes suites, so a tenant moving to v7 would start a new chain whose genesis names the v6 chain's head digest in its payload (build recommendation). The risk this guards against is named as fatal: the sixteen fields turn out to be missing something essential — "a jurisdiction, a purpose limitation, a counterparty identifier" — after records are in the wild [ESB Ch.17 R1].

The lineage visible in the sources explains why the current text is strict about retiring figures:

Stage	What it was	Status	Source
v1	Six fields joined with `|` (previous hash, write type, diff JSON, actor, approver, time)	Found in a test-script mirror in the working tree; retired	HFS
`elan1.proof.seal.v2`	Thirteen fields, length-prefixed `len(name):name=VALUE` in sorted name order, domain with a trailing NUL octet; 164 backend tests passing	The proof engine in the September 2026 working tree; superseded	HFS

Stage	What it was	Status	Source
v6 under the v2.0 encoding table	Sixteen fields; F16 spelled `sealed_at`; autonomy L0–L5; TV-001 = 311 octets, digest `a376103f…2b280d31`	Retired; the figures "may not be quoted"	E12 §02, §07; EBB EB-05
v6 under the v3.0 encoding table	Sixteen fields as in §10.2; `recorded_at`; A0–A5; TV-001 = 329 octets; digest to be derived by two implementations	Current text; draft until E12 Flow 3 step 8	E12

The sources do not explain the step from v2 to v6 in the domain string; the v6 label is inherited from the Strategy Blueprint and Build Book lineage [ESB Ch.05; EBB EB-05]. Two different version numbers should not be confused: "v3.0" is the document version of E12; "v6" is the suite.

"Frozen" has a precise meaning here. The field set, encoding and domain string are frozen in the specification text as of 21 September 2026 [E12 decisions 1–3]. The specification's status is nevertheless "draft" until the two-implementation run completes, and the working brief sets the deadline as "signed before L2 build starts" [WD Phase 2 01]. Because L1 records use the same encoding, the practical freeze date is earlier: the encoding must be final, and implementation A must agree with B, before the first governed write to a real customer's books, so that no L1 record is ever written under an encoding later found ambiguous (build recommendation; see §10.3.1 on F14 for the same reasoning).

10.12 Code sketch: seal and verify

The sketch below is the reference shape for implementation A. It is illustrative, not the implementation; implementations A and B must be written from the specification text, and the verifier must not share this code [E12 §09].

```
# Seal v6: encode, preimage, digest; L1 chain verification (build recommendation)
import hashlib, re, unicodedata, uuid

DOMAIN = b"elan1.proof.seal.v6"                                # 19 octets
ZERO32 = bytes(32)
TS = re.compile(rb"^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$")
FIELDS = [("tenant_id","uuid"),("seq","u64"),("prev_digest","d32"),("write_id","uuid"),
          ("write_type","str"),("declaration_id","uuid"),("declaration_version","u32"),
          ("policy_version","u32"),("actor_id","uuid"),("actor_kind","enum"),
          ("autonomy_level","u8"),("record_id","uuid"),("record_version","u32"),
          ("payload_hash","d32"),("lexicon_hash","d32"),("recorded_at","ts")]

def LP(x: bytes) -> bytes:
    return len(x).to_bytes(4, "big") + x

def enc(name: str, kind: str, v) -> bytes:
    if v is None:
        raise EncodeError("E_FIELD_NULL", name)
    if kind == "uuid":
        if not re.fullmatch(r"[0-9a-f]{8}(-[0-9a-f]{4}){3}-[0-9a-f]{12}", v):
            raise EncodeError("E_UUID_FORM", name)
        return uuid.UUID(v).bytes                               # RFC 9562 network order
    if kind in ("u64", "u32", "u8"):
        n = {"u64": 8, "u32": 4, "u8": 1}[kind]
        if not (0 <= int(v) < 1 << (8 * n)):
            raise EncodeError("E_INT_RANGE", name)
        return int(v).to_bytes(n, "big")
    if kind == "d32":
        if len(v) != 32:
            raise EncodeError("E_DIGEST_LENGTH", name)
        return v
    if kind == "str":
        b = unicodedata.normalize("NFC", v).encode("utf-8")
        if not 5 <= len(b) <= 64:
            raise EncodeError("E_STRING_LENGTH", name)
        return b
    if kind == "enum":
        if v not in ("human", "agent", "system"):
            raise EncodeError("E_ENUM_TOKEN", name)
        return v.encode("ascii")
    if kind == "ts":
        b = v.encode("ascii")
        if len(b) != 20 or not TS.match(b):
```

```
            raise EncodeError("E_TIMESTAMP_FORM", name)
        return b
    raise AssertionError(kind)

def preimage(rec: dict) -> bytes:
    missing = [n for n, _ in FIELDS if n not in rec]
    if missing:
        raise EncodeError("E_FIELD_MISSING", missing[0])
    return LP(DOMAIN) + b"".join(LP(enc(n, k, rec[n])) for n, k in FIELDS)

def digest(rec: dict) -> bytes:
    return hashlib.sha256(preimage(rec)).digest()

def verify_chain(records: list[dict], stored: list[bytes]) -> tuple[str, int|None, str|None]:
    """Three-valued: continuous | break at (seq, field) | not_checked."""
    if not records:
        return ("not_checked", None, "empty range")
    prev_seq, prev_d = None, None
    for rec, d in zip(records, stored):
        if prev_seq is None and rec["seq"] == 1 and rec["prev_digest"] != ZERO32:
            return ("break", 1, "prev_digest")               # genesis must be 32 zeros
        if prev_seq is not None and rec["seq"] != prev_seq + 1:
            return ("break", rec["seq"], "seq")              # gap, duplicate or re-order
        if prev_d is not None and rec["prev_digest"] != prev_d:
            return ("break", rec["seq"], "prev_digest")
        if digest(rec) != d:
            return ("break", rec["seq"], "digest")
        if "signature" in rec and rec.get("assurance_level") == 1:
            return ("break", rec["seq"], "signature")        # absent, not stubbed
        prev_seq, prev_d = rec["seq"], d
    return ("continuous", None, None)
```

Implementation B's length prefix, written independently in TypeScript, looks different on purpose, which is the point of the rule:

```
// seal-b: length prefix (independent implementation; build recommendation)
export function lp(x: Uint8Array): Uint8Array {
  const out = new Uint8Array(4 + x.length);
  new DataView(out.buffer).setUint32(0, x.length, false);   // big-endian
  out.set(x, 4);
  return out;
}
```

10.13 Acceptance criteria

#	Criterion	Evidence	Source
S1	Two independent implementations agree octet for octet on all 64 vectors	Empty diff; disagreement log published with the spec	E12 §08; E14 §07 item 1
S2	TV-001 preimage is 329 octets in both; its digest is written down only after S1	Vector output; no document carries the digest before S1	E12 decisions 3–4
S3	The verifier PASSes all 36 positives and REJECTs all 28 negatives naming record and field; empty lockfile	Verifier CI	E12 §08 step 7; E14 §07 item 2
S4	No L1 record carries a signature, key or anchor member, in storage or in any transport form	Schema check; export test	E12 §04
S5	The F14 salting scheme and payload canonicalisation are specified with vectors before the first real-tenant record	Spec amendment; new vectors	E12 open question 1 (recommended earlier)
S6	Refusal values for F08 and F15 before their owning rungs are specified with vectors	Spec amendment	Chapter 9 §9.7.2
S7	The Ed25519 verification profile is written into the spec and each library in use is shown rejecting the strictness vectors	Vector run per library	E12 §04; ETS §13 decision 3
S8	Text, inputs, agreed digests and verifier source are published together in one release; no record leaves the building before	Release record	E12 §08 step 8

11. The evidence bundle, the offline verifier and verifier profiles

This chapter specifies how elan1 evidence leaves the platform and how a stranger checks it: the evidence bundle (its eight members, the manifest that binds them, what is deliberately left out), the export flow, the zero-dependency offline verifier (its algorithm, its six exit codes, how it is built, published and proved to be the one elan1 published), verifier profiles that turn a regulatory regime into machine-checkable requirements, the auditor pilot through which the whole construction meets its first real reviewer, and the targets and tests that define done. The record format the bundle carries is Chapter 10; keys and timestamp tokens are Chapter 12; packs, which pair a declaration with a certified profile, are Chapter 13.

> **Status.** Everything in this chapter is Level 2 and none of it exists [IM §4.9]. At L1 there is "no verifier, no bundle, no redaction profile" by explicit non-goal [E01 §07]. The bundle, verifier and profile specifications (E13, E15) are dated 21 September 2026 and marked "Draft for review". L2 general availability is modelled for April 2028 [IM §4.9]; E14 frames Phase 3 as Q3 2027 to Q1 2028 [E14 §01]. An earlier working tree contains a hand-written in-browser verifier that checks a bundle format nothing produces, and whose "key" check renders *passed* without reading any signature bytes, because the format has no signature field [HFS evidence lane]; it is a prototype of the reporting style, not a component of this design. All sizes and timings below are modelled or targets.

11.1 What a bundle is

"A bundle is a directory, or a .tar of one, containing everything required to verify a range of one tenant's chain without contacting elan1, without an elan1 account, and without any code we control running on the reader's machine. It is the artefact SKU 2 sells and the only thing that makes the L2 claim checkable" [E15 §01]. Three properties define it [E15 §01; IM §7.4]:

- **Self-contained.** Every declaration version, lexicon and anchor token a record references is inside it.
- **Inert.** No executable content, no scripts, no macros, no URL a verifier is expected to fetch.
- **Claim-bearing.** The manifest states the profile version the bundle claims, "so a verification has something to be true or false against".

Two things arrive separately, on purpose: the verifier, from a source the reader chooses, and the key fingerprint, from the reader's own out-of-band source. "Key material is never in a bundle … A receipt that ships its own verification key is worse than no signature at all: the party who produced the receipt also supplied the means of checking it" [E15 §01].

11.2 Layout

The format string is `elan1.evidence.bundle.v1`; "a change is a new string, not a minor version" [E15 §02, decision 1]. A bundle for one tenant range looks like this (file names inside `declarations/`, `lexicon/` and `anchors/` are a **build recommendation**; the member set is E15's):

```
bundle-3f9a1c2e-000001-012480/
  manifest.json                     # always — binds every other member by digest
  seals.jsonl                       # always — one record per line, in seq order
  chain.json                        # always — genesis, head, range, boundary rows
  anchors/                          # L2 — one file per closed batch
    batch-000412.json               #   root, leaf count, range, both tokens (DER, base64),
                                    #   TSA certification paths, revocation data,
                                    #   inclusion paths for each covered record
  keys.json                         # L2 — key ids, windows, status, successor, fingerprint
  declarations/                     # always — verbatim, one file per (id, version)
    9c1e…-v7.decl                   #   exact bytes as certified
  lexicon/                          # always — verbatim, one file per lexicon digest
```

```
    5b0d…e2.lex
  VERIFY.txt                          # always — plain text, for a reader new to elan1
```

Member	Contents [E15 §02]	Required
`manifest.json`	What is in the bundle, the range, the counts, the digests of every other member, the profile claimed	always
`seals.jsonl`	One record per line in `seq` order: the sixteen fields, the digest, the assurance level, and at L2 the signature and `key_id`	always
`chain.json`	Genesis digest, head digest, the `seq` range, and the boundary rows where the assurance level or the signing key changes	always
`anchors/`	One file per closed batch: RFC 3161 token, Merkle root, leaf count, each covered record's inclusion path; the TSA certification path and revocation data captured at anchor time	L2
`keys.json`	`key_id`, `valid_from`, `valid_to`, `status`, `successor_id`, public key fingerprint — never the key	L2
`declarations/`	Every referenced declaration version, verbatim, one file per `(declaration_id, version)`	always
`lexicon/`	Every term set referenced by F15, verbatim, one file per lexicon digest	always
`VERIFY.txt`	How to check this without elan1, what each failure looks like, what a PASS does not prove	always

`VERIFY.txt` "is the member most likely to decide whether the bundle is trusted". It names the verifier's published digest, the command, the exit codes, and the claimed profile's limits statement verbatim [E15 §02].

11.2.1 The manifest

Field	Type	Req	Meaning [E15 §02]
`bundle_format`	string	yes	`elan1.evidence.bundle.v1`
`tenant_id`	uuid	yes	Whose chain
`seq_from` · `seq_to`	uint64	yes	The range exported, inclusive
`record_count`	uint64	yes	Must equal `seq_to − seq_from + 1`
`refusal_count`	uint64	yes	How many were refusals
`assurance_levels[]`	array	yes	Which levels appear, with the count and `seq` range of each
`continuity_scope`	enum	yes	`full_chain` · `declared_write_types` · `range`, printed in every report
`profile_id` · `profile_version`	string · uint32	cond	The profile claimed; absent means no claim, and the verifier runs integrity checks only
`redaction_profile`	string	no	Which fields the export reveals; the rest are present as commitments
`members[]`	array	yes	Every other member with its path, byte length and SHA-256
`unanchored_head`	uint64	yes	Records in the range not yet covered by an anchor at export time; "0 is a value, not an absence"
`exported_at`	timestamp	yes	Platform clock, RFC 3339 UTC, second precision
`verifier_digest`	digest	yes	The published verifier release this bundle was tested against

```
{"bundle_format":"elan1.evidence.bundle.v1",
 "tenant_id":"3f9a1c2e-7b44-4d1a-9e0f-5a6b7c8d9e01",
 "seq_from":"1","seq_to":"12480","record_count":"12480","refusal_count":"611",
 "assurance_levels":[{"level":1,"count":"8117","seq_from":"1","seq_to":"8117"},
                     {"level":2,"count":"4363","seq_from":"8118","seq_to":"12480"}],
 "continuity_scope":"full_chain",
 "profile_id":"in-mca-audit-trail","profile_version":1,
 "members":[{"path":"seals.jsonl","bytes":7987200,"sha256":"<64 hex>"},
            {"path":"chain.json","bytes":1180,"sha256":"<64 hex>"}],
 "unanchored_head":"22","exported_at":"2028-05-02T11:40:09Z",
 "verifier_digest":"<64 hex>"}
```

The example follows the transport rules of Chapter 10 §10.7 (uint64 as decimal strings, fixed key order) and is abbreviated in `members[]`. It shows the case every upgraded tenant will produce: a full-chain export in which records before the upgrade are Level 1 and records after it are Level 2, under a profile whose `min_assurance_level` is 2 and whose `applies_to_records_from` is later than genesis. The sources do not say how such a bundle is judged; this memorandum recommends that profile checks, including the minimum assurance level, apply only to records inside the profile's applicability window, while records outside it are integrity-checked and reported in the scope section as "outside profile window — integrity only" **(build recommendation)**. Without that reading, `continuity_scope: full_chain` and `min_assurance_level: 2` cannot both be satisfied by any tenant that started at L1, which is every tenant (see the conflicts block). "The manifest is not signed as a whole. Each record carries its own signature, and the manifest's `members[]` digests bind the rest. A manifest signature would create a second trust root that could pass while the records failed, which is the failure mode a verifier exists to prevent" [E15 §02, decision 2].

11.2.2 How the records chain inside the bundle

Line *n* of `seals.jsonl` carries `seq`, the sixteen fields and `prev_digest`. The verifier recomputes each preimage under the Chapter 10 encoding, takes SHA-256, and checks that the recomputed digest matches the line, that `prev_digest` equals the previous line's digest, and that `seq` increments by exactly one [E15 §02]. `chain.json` carries the genesis and head digests so that a partial range can be positioned inside the tenant's whole history, "and a bundle whose `seq_from` is greater than 1 says so in its scope line rather than looking complete" [E15 §02]. For a range starting after genesis, the first line's `prev_digest` is checked against a `chain.json` entry for record `seq_from − 1` (build recommendation), so the range is anchored to its predecessor rather than floating.

> **A hash chain cannot detect a truncated tail.** "A chain cut at 4,000 of 4,200 is internally consistent. This is why `anchors/` exists and why `unanchored_head` is a required manifest field: the anchor is the only member that can testify about records the exporter chose not to include" [E15 §02]. Each anchor file carries the batch's leaf count and range, so a record that should have been in an anchored batch and is missing becomes visible [IM §7.5].

`chain.json` also carries the boundary rows: where the assurance level changes (the L1 → L2 upgrade) and where the signing key changes (rotation or revocation) [E15 §02]. "A customer cannot produce a bundle that hides the boundary, and neither can we" [E19 §05].

11.3 What is included, and what is deliberately not

Included	Why [E15 §03]
Refusals, in full	"The refused case is the record that matters in a disparate-impact or conduct question"; `refusals_required` in a profile makes their presence checkable
Every referenced declaration version and lexicon, verbatim	So "which rules decided" resolves inside the bundle in ten years
Gate results, beside each record	Retained and exported; bound by declaration version rather than by the preimage
The approval record	An approval is its own governed write with its own `seq`; it appears as a record, not as a field
The TSA certification path and revocation data at anchor time	So a token stays checkable after the authority's certificate expires

Excluded	Why [E15 §03]
Payloads	F14 commits to the payload without carrying it; "a bundle full of payloads could not survive an erasure request"
Salts	Held in a separate store; destroying a salt unbinds a payload permanently, including to elan1, while every digest and signature still verifies
Key material	See §11.1
Any executable	"A bundle is data. The verifier arrives separately, from a source the reader chooses"

Excluded	Why [E15 §03]
Target-system state	The bundle says a change was applied through the single writer; "it says nothing about what the accounting system looks like now"

Gate results are listed as included but no member holds them in E15's eight. The recommendation is to carry them inside each `seals.jsonl` line as a `gate_results` array outside the `fields` object — name, outcome and `inputs_hash`, in declared order — so they travel with their record without becoming a ninth member or entering the preimage **(build recommendation)** [E12 §03; E02 §03].

11.4 Exporting a bundle

Exporting is a governed write; it "climbs the nine rungs" [E15 §05; IM §7.4]. Preconditions: the tenant is at L2; a certified profile exists for the claimed regime; the requester holds an `ro` token [E15 §05].

Step	What happens [E15 §05]
1	Requester opens Assurance · Evidence · Export, names a `seq` range or a date range, picks a profile or *No profile claimed*; the screen shows the record count, refusal count and unanchored head before anything is generated
2	*Prepare export*: every declaration version and lexicon the range references is resolved; any that cannot be resolved stops the export and is named
3	Gate check: residency (the bundle may not be generated outside the tenant's domain), retention, consent where the redaction profile reveals personal fields, and precondition — the range must be covered by a completed nightly integrity run
4	The eight members are assembled; each member's digest is computed; `manifest.json` is written last; the export's own record is appended to the tenant's chain
5	The requester sees *Export ready — 12,480 records, 611 refusals, 22 unanchored, profile in-mca-audit-trail v1* with the bundle digest, and downloads it
6	The screen says: "Verify it yourself first. If it does not pass for you it will not pass for them."

Refusal paths. A residency violation refuses and offers the in-domain endpoint; `POLICY.DECLARATION_UNRESOLVED` names the missing `(declaration_id, version)`; a range with an incomplete nightly integrity run refuses at precondition, shows the last clean night and offers *Export the clean range instead*; a profile whose `applies_to_records_from` excludes part of the range refuses and offers the two sub-ranges with their profile versions [E15 §05].

Failure paths. Object store unavailable: the export fails, nothing partial is written, and retry is idempotent on `(tenant_id, seq_from, seq_to, profile_id, profile_version)`. An export that would exceed the size limit "is refused with the limit and the suggested split, never silently truncated" [E15 §05, decision 9]. The limit itself, and whether it is a product constant or a tier property, is open until 30 September 2027 [E15 open question 1].

What is recorded. The export as a governed write — who, which range, which profile, the bundle digest, the redaction profile, the unanchored head at that instant — metered on the `one_tenant` basis; "the bundle itself is priced ₹0" [E15 §05]. elan1 retains chains and anchors indefinitely and retains no bundles: "we keep the manifest digest and the export as a governed write in the tenant's own chain … and we keep nothing else. A customer who loses a bundle re-exports it" [E15 §04, decision 4].

11.4.1 Determinism: why re-export must be byte-identical

"A bundle re-exported over the same range with the same profile is byte-identical except for `exported_at`" [E15 §04, decision 5]. The property matters because it lets a reviewer who receives two copies compare them, and because an exporter that produces different bytes for the same range has non-determinism an auditor will eventually mistake for tampering. It constrains the implementation in ways the specification does not spell out **(build recommendation)**:

- Records are read by `seq`, never by an index whose order can vary; declarations and lexicons are written from `body_bytes`, not re-serialised.

- Every JSON member uses the fixed key order and number representation of Chapter 10 §10.7, UTF-8, `\n` line endings, no trailing whitespace.
- Anchor files embed DER tokens as captured, byte for byte; revocation data is the set captured at anchor time, never re-fetched at export.
- The `.tar` form uses sorted entry names, a fixed `mtime` equal to `exported_at`, uid/gid 0, fixed modes (`0644` files, `0755` directories) and the POSIX ustar format; no compression inside the specification (a reader may compress for transport).
- The one variable field, `exported_at`, appears only in the manifest; because the manifest is excluded from its own `members[]` list, every other member's digest is stable across re-exports.

11.4.2 Size

Figure	Model [E15 §04]	Basis
One record in `seals.jsonl`	~640 bytes at L2	Sixteen fields as JSON text plus digest, signature and key id
One anchor file	~6 KB plus ~1 KB per covered record's inclusion path	RFC 3161 token with certification path, plus $\log_2(n)$ sibling digests per record
A 100,000-record year	~150 MB	Dominated by inclusion paths and declaration copies, not by records
Declarations and lexicon	Flat	One copy per referenced version, however many records cite it

The evidence team's own open question asks whether ~640 bytes survives once inclusion paths are real, and whether the 100,000-record figure survives a measurement, dated 31 August 2027 [E15 open question 4]. One arithmetic point is worth adding: the ~1 KB per record for inclusion paths is a ceiling rather than a floor, since a path is $\log_2(n) \times 32$ octets — 320 octets for a 1,024-leaf batch — and hex encoding in JSON doubles it (build recommendation; path size from IM §7.6).

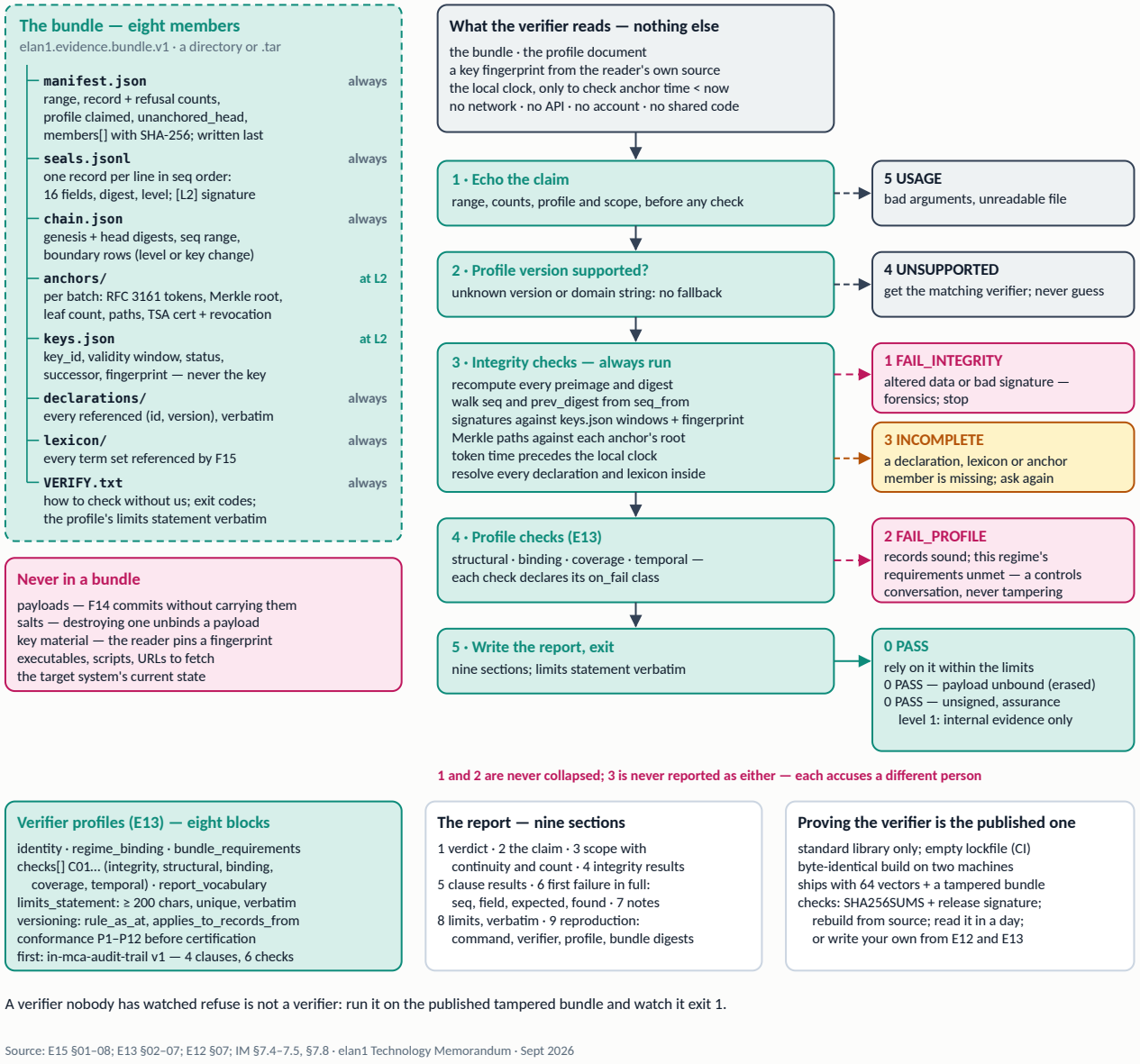
11.5 The offline verifier

The verifier "has zero dependencies, an empty lockfile enforced in CI, a reproducible build, and shares no code with the chain writer. It reads exactly four things: the bundle; the profile document …; the key fingerprint the reviewer supplies from their own source; and the local clock, used only to check that an anchor time precedes now. It reads no network, calls no API, writes nothing but its report, and records nothing" [IM §7.5; E15 §06; E13 §04]. "A verifier that phoned home would not be offline and would not be a verifier" [E15 §06]. Figure 15 shows the bundle, the two things that arrive separately, and the verification flow to one of six exit codes.

Figure 15 — The evidence bundle and the offline verifier

L2 — SPECIFIED, NOT BUILT

A bundle needs nothing from elan1; the verifier reads it on an air-gapped machine and answers with one of six codes



A verifier nobody has watched refuse is not a verifier: run it on the published tampered bundle and watch it exit 1.

Source: E15 §01–08; E13 §02–07; E12 §07; IM §7.4–7.5, §7.8 · elan1 Technology Memorandum · Sept 2026

Figure 15 — The evidence bundle and the offline verifier: eight inert members bound by the manifest; the verifier and the key fingerprint arrive from the reader's own sources; integrity checks always run, profile checks run against the claimed profile, and the run ends in one of six exit codes that never collapse integrity, profile and incompleteness into one.

11.5.1 The algorithm, step by step

The command is `elan1-verify --bundle <path> --profile <path> --key-fingerprint <hex>` [E15 §06]. The verifier echoes the bundle's claim — range, counts, profile, scope — before checking anything [E15 §06 step 2]. It then proceeds in four phases. The eight integrity steps are this memorandum's enumeration of E15 step 3 and E13 step 3 into the "eight integrity steps" the report section names [E13 §04 report item 4] **(build recommendation for the ordering)**.

Phase 0 — admission. (a) Arguments parse and every path is readable, else exit 5 `USAGE`. (b) `bundle_format` is `elan1.evidence.bundle.v1` and every record's domain is `elan1.proof.seal.v6`, else exit 4 `UNSUPPORTED`, with the unknown identifier printed. (c) If a profile is claimed, the verifier supports that `profile_id` and `profile_version`, else exit 4; "it does not guess, and it does not fall back to a neighbouring version" [E13 §04 step 2]. (d) A profile that has been withdrawn exits 4 with the withdrawal date printed [E13 §05].

Phase 1 — integrity (profile-independent, always run).

#	Integrity step	Fails as
I1	Every member listed in `members[]` is present with the stated length and SHA-256; nothing unlisted is present except `manifest.json`	incomplete (missing) or integrity (digest mismatch, extra executable)
I2	For every line: recompute the preimage from the sixteen fields under the Chapter 10 encoding; SHA-256; compare with the stored digest	integrity
I3	Walk the chain: `seq` increments by exactly one from `seq_from`; each `prev_digest` equals the previous digest; the first line links to `chain.json`; `record_count` equals the range width; `refusal_count` equals the zero-`record_id` count	integrity
I4	Assurance level: L1 lines carry no signature member; L2 lines carry one; boundary rows in `chain.json` match the lines	integrity
I5	Signature: pure Ed25519 over the 32-octet digest, under the strictness profile of Chapter 10 §10.5, with the key resolved from `keys.json` and its fingerprint equal to the one supplied on the command line	integrity
I6	Key window: `recorded_at` inside the key's window; a revoked key's records still PASS, with the revocation printed in Notes	integrity
I7	Anchors: each covered record's inclusion path reproduces its batch root (RFC 6962, odd node promoted); each RFC 3161 token verifies against its captured certification path; the token time precedes the local clock; when two tokens exist, they commit to the same root	integrity
I8	Resolution: every `(declaration_id, version)` in F06/F07 and every F15 lexicon digest resolves to a file inside the bundle whose bytes hash as expected	incomplete (absent) or integrity (present but wrong)

On an L1 range, I5, I6 and I7 each read `not_applicable — assurance level 1` rather than passing or failing [E13 §01; E13 P8].

Phase 2 — profile checks (only when a profile is claimed). Structural checks (required members present, minimum assurance level met, redaction profile applied as declared); binding checks (for every clause, the named write types appear, the named gates were declared and enabled at the declaration version each record names, the named fields are present and resolvable); coverage and temporal checks (refusals present where `refusals_required`, continuity scope as declared, `recorded_at` inside the profile's applicability window, retention satisfied) [E13 §04 steps 4–6]. Each check carries its own `on_fail` class: `fail_integrity`, `fail_profile` or `incomplete` [E13 §02 block 4].

Phase 3 — report and exit. The verifier writes the nine-section report (§11.5.3) to a file or standard output and exits with one code [E13 §04 step 7].

```
// cmd/elan1-verify: control flow (build recommendation; Go, standard library only)
func run(a args) int {
    b, err := openBundle(a.bundle)                 // reads files only; no net import
    if err != nil { return report(usage(err)) }    // 5 USAGE
    if !supported(b.Manifest, a.profile) {
        return report(unsupported(b.Manifest))      // 4 UNSUPPORTED
    }
    echoClaim(b.Manifest)                           // printed before any checking
    r := newReport(b, a)
    for _, step := range integritySteps {           // I1..I8, always
        r.add(step(b, a.fingerprint, time.Now().UTC()))
    }
    if a.profile != nil {
        for _, c := range a.profile.Checks {        // structural, binding, coverage, temporal
            r.add(c.eval(b))
        }
    }
    return r.write()                                // 0, 1, 2 or 3 by precedence (§11.5.2)
}
```

11.5.2 The six exit codes

"Six verifier exit codes": 0 through 5, where code 0 has three verdict variants [IM §7.4 figure; E15 §06].

Code	Name	Means	What the reader does [E15 §06; E13 §04]
0	`PASS`	Every integrity and profile check passed	Rely on it, within the limits statement
0	`PASS — payload unbound`	As above; one or more salts were destroyed under an erasure obligation. Chain intact, payload unbound	Note it. Never printed as a failure
0	`PASS — unsigned, assurance level 1`	Structure and chain verified; signature, key-window and anchor checks `not_applicable`	Internal control evidence only; not third-party-checkable
1	`FAIL_INTEGRITY`	A digest, chain link, signature, inclusion path or anchor did not verify	Stop. "This is forensic, not compliance"
2	`FAIL_PROFILE`	Integrity sound; a profile requirement unmet	"The records are trustworthy and the bundle does not meet this regime's requirements"
3	`INCOMPLETE`	A referenced declaration, lexicon or anchor member is missing	Ask for a complete bundle
4	`UNSUPPORTED`	Unknown profile version or domain string	Obtain the matching verifier. Never interpret
5	`USAGE`	Bad arguments, unreadable file	Operator error

> **Three failures a verifier must never confuse.** An integrity failure means the data was altered or the signature is wrong, and whoever held the records has a problem; reported as a profile failure, it "hides a P0". A profile failure means the data is sound and the regime's requirements are not met; reported as an integrity failure, it "accuses a customer of tampering". Incomplete means the bundle is missing a piece; reported as either, it "turns a packaging bug into an accusation". "1 and 2 are never collapsed, and 3 is never reported as either" [E15 §06; E13 decision 3].

A run can find more than one problem, and the sources do not fix precedence among codes 1, 2 and 3. The recommended order is **1 over 3 over 2**: an integrity failure anywhere dominates, because it is a P0 regardless of what else is wrong; an incomplete bundle dominates a profile failure, because a profile cannot be judged fairly on a bundle missing the rules it would read; and a profile failure is reported only over a bundle whose integrity is sound and complete **(build recommendation)**. Every finding is still listed in the report; only the exit code is singular. A mixed L1/L2 range that passes reports `0 PASS` with both levels and their counts in the scope section [E19 §04 step 19].

11.5.3 The report

The report has nine sections, "in this order, every time" [E13 §04]:

1. **Verdict** — the exit name in one line.
2. **The claim** — profile identifier, version, `rule_as_at`, and the bundle's own claim, printed even when it failed.
3. **Scope** — the sequence range, record count, write types covered, and the continuity scope with its count. "A chain verified over three of twenty-three write types is a chain verified over almost nothing, and it reads identically on a page unless the page says so."
4. **Integrity results** — each of the eight integrity steps: passed, failed or `not_applicable — assurance level 1`.
5. **Clause results** — one row per clause: citation, disposition, the checks that served it, the outcome.
6. **First failure, in full** — record sequence, field, expected, found.
7. **Notes** — erased payloads, key rotations spanned, unanchored head at export time.
8. **Limits** — the profile's `limits_statement`, verbatim.
9. **Reproduction** — the exact command, the verifier version, the profile digest, the bundle digest.

"Section 8 is what makes sections 1 to 7 credible" [E13 §04]. An abbreviated failing report looks like this (illustrative values):

```
VERDICT        1 FAIL_INTEGRITY
CLAIM          in-mca-audit-trail v1 (rule_as_at 2023-04-01); bundle claims the same
SCOPE          seq 1-12,480 · 12,480 records · 6 of 6 write types · full_chain
               L1: seq 1-8,117 (8,117) · L2: seq 8,118-12,480 (4,363)
INTEGRITY      I1 passed · I2 FAILED · I3 passed · I4 passed · I5 passed · I6 passed
               I7 passed · I8 passed
```

```
FIRST FAILURE  seq 3,181 · field payload_hash · digest expected 9e41…c2 found 0b7d…18
NOTES          22 records unanchored at export · 1 key rotation spanned (seq 10,002)
LIMITS         A PASS under in-mca-audit-trail v1 establishes that … (verbatim)
REPRODUCE      elan1-verify --bundle ./b --profile ./in-mca-audit-trail-v1.json
               --key-fingerprint 5f…a1 · verifier 1.0.0 sha256 7c…
```

11.6 Distribution, reproducible builds and no phone-home

The release flow is fixed by E15 Flow 3 [E15 §07]. The verifier builds from a tagged commit, "standard library only, empty lockfile, CI fails if the lockfile is not empty", producing static binaries for the published platforms and a source tarball. The build runs twice, on two different machines, "with a pinned toolchain digest and `SOURCE_DATE_EPOCH` fixed"; the outputs must be byte-identical, and "a non-reproducible build is not released". The full vector set runs: every positive PASSes and each of the 28 negatives is REJECTed with record and field named. Publishing a release is a governed write under `dual_control: true`; the release approver signs the artefacts with the release key; the release publishes binaries, source tarball, `SHA256SUMS`, the signature, build instructions, the vector set and "the deliberately tampered bundle", to at least two independent locations [E15 §07, decisions 6–7]. The release key is held on an offline hardware token by two named humans, "not in any cloud", rotated every 24 months [IM §7.7].

The stack document recommends Go with the standard library for the verifier because it is "the only mainstream option whose stdlib covers SHA-256, Ed25519, ASN.1/DER and X.509 — all a zero-dependency RFC 3161 check needs", with `-trimpath` reproducible builds and cosign-signed artefacts [ETS §01, §04, §09]. The zero-dependency check reads `go.mod` and `go.sum`, which must declare no requirements — "read the lockfile, not the manifest" [ETS §09].

```yaml
# verify/.github/workflows/release-gate.yml (build recommendation)
name: verifier-release-gate
on: {push: {tags: ["v*"]}}
jobs:
  hermetic:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - name: Zero dependencies (lockfile, not manifest)
        run: test ! -s go.sum && ! grep -q '^require' go.mod
      - name: No network-capable imports outside an allow-list
        run: ./ci/forbid-imports.sh net net/http net/url crypto/tls os/exec plugin
  build-a: {uses: ./.github/workflows/build.yml, with: {runner: ubuntu-24.04}}
  build-b: {uses: ./.github/workflows/build.yml, with: {runner: ubuntu-22.04-arm-cross}}
  reproducible:
    needs: [build-a, build-b]
    runs-on: ubuntu-24.04
    steps:
      - run: ./ci/compare-artefacts.sh build-a build-b     # byte-identical or fail
  vectors-air-gapped:
    needs: [reproducible]
    runs-on: ubuntu-24.04
    steps:
      - name: Run all 64 vectors and the tampered bundle with no network namespace
        run: sudo unshare --net -- ./ci/run-vectors.sh --positives 36 --negatives 28
```

`build.yml` pins the Go toolchain by digest and sets `CGO_ENABLED=0`, `-trimpath`, `-buildvcs=false`, `-ldflags=-buildid=` and `SOURCE_DATE_EPOCH` from the tag's commit time (build recommendation). The air-gapped job runs the verifier inside a network namespace with no interfaces, which turns E14's "air-gap test" into a CI assertion: any attempt to open a socket fails the run [E15 §08; E13 P12].

How a sceptic checks the verifier is the one elan1 published. Four routes, "in increasing order of paranoia, and all four must work" [E15 §07]:

#	Route	What it establishes
1	Compare the binary's SHA-256 against `SHA256SUMS`; check the release signature against the release-key fingerprint obtained out of band	It is the file elan1 published

#	Route	What it establishes
2	Rebuild from the source tarball with the documented toolchain and compare digests	The published binary corresponds to the published source
3	Read the source: standard library only, "small enough to read in a day"	There is no network call and no shared code with the chain writer
4	Ignore all of the above and write a verifier from E12 and E13, then run it on the bundle	The bundle is checkable without elan1's software at all

"Route 4 is the one that matters … A verifier we supply is a convenience; the format is the product" [E15 §07]. A defective release "is superseded, never withdrawn silently: the old release stays downloadable with a dated note saying what it got wrong, because a reviewer who ran it needs to know what their run was worth" [E15 §07, decision 8]. The verifier "is never priced. It ships as source, free, to anyone, at L2 and after" [E07 §04, decision 3]. Which second publication location — a Linux distribution package, a language package registry or a source-forge mirror — is open until 31 July 2027 [E15 open question 3]; the stack document's npm-style name `@elan1/verify` implies a package registry [ETS §02].

11.7 Verifier profiles

"A profile is a checkable claim — this bundle is conforming for this regime under this profile version — and the only way to make that sentence true is to bind every clause to a write type, a gate and a field, by name" [E13, in one line]. It answers two questions for one regime: what must a bundle contain, and what does a conforming report say. "It is not a legal opinion, not a certification, not a claim that a regulator has accepted anything, and not a control framework" [E13 §01]. Profiles are an "open-n seam: one per regime, independently authored, versioned separately, with no shared code" [E13 §01].

Profiles execute at L2, because signature and anchor checks are half of what they check; the format is frozen in Phase 2 "so that a profile authored in 2027 does not need the spec to change", and a profile "may be authored and dry-run against an L1 chain today", with the signature, key-window and anchor checks reporting `not_applicable — assurance level 1` [E13 §01, decision 1].

11.7.1 The eight blocks

Every field takes a value, `out_of_scope` with an ADR reference, or `not_measurable` with a reason and a `would_change_if`, each at least twelve characters — the same three dispositions as a declaration [E13 §02].

Block	Carries [E13 §02]
1 `profile_identity`	`profile_id` (`^[a-z]{2}-[a-z0-9-]{3,40}$`, jurisdiction-prefixed), `profile_version` (last + 1, never "latest"), `regime_name`, `issuer`, `jurisdiction` (ISO 3166-1 alpha-2), `author`, `certifier` ("never the author"), `status` (draft · certified · superseded · withdrawn)
2 `regime_binding`	One row per clause: `clause_ref`, `clause_text` (quoted, not paraphrased), `obligation_kind` (record_keeping · human_oversight · retention · disclosure · segregation · non_repudiation), `binds_write_types[]`, `binds_gates[]`, `binds_fields[]`, `retention_months`, `check_ids[]`, `disposition`
3 `bundle_requirements`	`required_members[]`, `min_assurance_level` (2 for any profile asserting non-repudiation), `proof_binding_fields[]`, `redaction_profile`, `refusals_required`, `continuity_scope`
4 `checks[]`	`check_id` (`^C[0-9]{2}$`), `kind` (integrity · structural · binding · coverage · temporal), `statement`, `inputs[]`, `pass_condition` ("stated so a second implementer can write it"), `on_fail` (fail_integrity · fail_profile · incomplete), `clause_refs[]`
5 `report_vocabulary`	The regime's own words for the report's terms
6 `limits_statement`	Required prose reproduced verbatim in every report: what a PASS does not establish
7 `versioning`	`supersedes`, `rule_as_at`, `applies_to_records_from` · `_to`
8 `conformance`	Which of P1–P12 this version passed, when, against which fixtures

> **A clause binds by name, or it does not bind.** "A profile row that says 'the system provides adequate audit trails' is prose, not a binding, and it is the failure mode every compliance mapping in this market has. If an author cannot name the gate, the honest disposition is `not_measurable` with a sentence saying what would change their answer" [E13 §02, decision 2].

11.7.2 The first profile: `in-mca-audit-trail` v1

The first profile binds the MCA audit-trail obligation for accounting software, in force since 1 April 2023, "because it is the obligation our first customers' auditors already carry" [E13 §07, decision 8]. Its binding is four clauses and six checks [E13 §07]. The YAML below is a transcription of E13's worked example into a machine-readable form; the serialisation is a **build recommendation**, the content is E13's.

```
profile_identity:
  profile_id: in-mca-audit-trail
  profile_version: 1
  regime_name: Companies (Accounts) Rules - audit trail for accounting software
  issuer: Ministry of Corporate Affairs
  jurisdiction: IN
  status: draft                    # certified only by a named certifier who is not the author
regime_binding:
  - {clause_ref: "audit trail of each and every transaction", obligation_kind: record_keeping,
     binds_write_types: ["*"], binds_fields: [F04, F05, F09, F10, F16], check_ids: [C01, C02]}
  - {clause_ref: "edit log of each change made in books of account",
     obligation_kind: record_keeping, binds_write_types: ["*"], binds_gates: [precondition],
     binds_fields: [F12, F13, F14, F07], check_ids: [C03]}
  - {clause_ref: "along with the date when such changes were made",
     obligation_kind: record_keeping, binds_write_types: ["*"], binds_fields: [F16],
     check_ids: [C04]}
  - {clause_ref: "the audit trail shall not be disabled", obligation_kind: non_repudiation,
     binds_write_types: ["*"], binds_fields: [F02, F03], check_ids: [C05, C06]}
bundle_requirements:
  required_members: [manifest.json, seals.jsonl, chain.json, anchors/, keys.json,
                     declarations/, lexicon/, VERIFY.txt]
  min_assurance_level: 2
  refusals_required: true
  continuity_scope: full_chain
checks:
  - {check_id: C01, kind: structural, on_fail: fail_profile,
     pass_condition: "every seals.jsonl line parses to sixteen fields; F05 resolves"}
  - {check_id: C02, kind: binding, on_fail: fail_profile,
     pass_condition: "F09 present and F10 in {human, agent, system} on every record"}
  - {check_id: C03, kind: binding, on_fail: fail_profile,
     pass_condition: "committed: F12 non-zero and F13 >= 1; refusal: F12 zero and F13 = 0"}
  - {check_id: C04, kind: temporal, on_fail: fail_profile,
     pass_condition: "F16 parses as RFC 3339 UTC and is non-decreasing with seq"}
  - {check_id: C05, kind: integrity, on_fail: fail_integrity,
     pass_condition: "seq contiguous from range start; every prev_digest matches"}
  - {check_id: C06, kind: temporal, on_fail: fail_integrity,
     pass_condition: "every record covered by an anchor whose token time precedes now"}
versioning:
  rule_as_at: 2023-04-01
  applies_to_records_from: 2027-07-01
```

C06 is "the interesting row". The non-disablement clause "is the one an audit-trail export cannot answer, because a chain cut at the tail is internally consistent and looks complete. C06 is the check that makes the clause checkable at all, and it is why `min_assurance_level` is 2 … On an L1 dry-run, C06 reads `not_applicable — assurance level 1`, and the report says so on its face rather than passing quietly. That single row is the clearest available demonstration of what L2 buys" [E13 §07]. The limits statement, reproduced in every report, says among other things that a PASS "does not establish that the books of account are complete: changes made directly in the accounting system, outside this write path, are recorded by that system's own audit trail and are not in this bundle", and that the profile "has not been reviewed or accepted by the Ministry of Corporate Affairs or by any regulator" [E13 §07].

11.7.3 Authoring, certification and versioning

An author enumerates every clause that constrains a write, quotes each verbatim, and names the write types, gates and fields that satisfy it; the system resolves each name against certified declarations, and "an unresolvable name is a validation failure, not a warning" [E13 §03]. Every clause needs at least one check and every check must serve a clause. The limits statement is rejected if shorter than 200 characters or textually identical to another profile's, "because a copied limits statement has not been thought about". The profile is dry-run against a known-good and a known-tampered bundle. Certification is a governed write under dual control; the certifier is never the author (`GATE.DUAL_CONTROL_SAME_ACTOR`), and the screen offers eligible names — no agents. The registry assigns `profile_version = last + 1`, computes the digest, and writes append-only [E13 §03].

When the regime text changes, "the profile does not change under it" [E13 §05]. A clarification gets a new version with checks unchanged and the old stays certified; a new or altered obligation gets a new version and the old becomes superseded but keeps verifying; a withdrawn regime or a profile found wrong becomes withdrawn, and a bundle claiming it exits 4 with the withdrawal date. The rule an auditor cares about most: "a bundle is judged under the profile version it claims, not the current one … Retro-judging a record under a later rule is the compliance equivalent of back-signing, and we refuse it for the same reason" [E13 §05, decision 6]. Commencement dates are handled through `applies_to_records_from` and `_to`; every profile version is retained verbatim forever [E13 §05].

11.7.4 Conformance tests for a profile

"A profile is software written by a person under deadline. It gets tested like software" [E13 §06]. Twelve tests; a profile that has not passed all twelve may not be certified [E13 decision 9].

#	Test	Passes when [E13 §06]
P1	Clause coverage	Every clause has a check; every check serves a clause
P2	Name resolution	Every write type, gate and field named resolves against a certified declaration version
P3	Known-good	The reference good bundle exits 0 PASS
P4	Known-tampered	The negative vectors are rejected, naming record and field (class per vector; see conflicts)
P5	Known-nonconforming	Sound integrity with a missing member, a disabled required gate and absent refusals exits 2 — never 1
P6	Incomplete	Missing a referenced declaration version exits 3 — never 2
P7	Erasure wording	A destroyed salt exits 0 and reads "chain intact, payload unbound"; the string FAIL appears nowhere
P8	L1 dry-run	Signature, key-window and anchor checks read `not_applicable — assurance level 1`; verdict `0 PASS — unsigned, assurance level 1`, never a silent PASS
P9	Scope printing	A partial-scope run prints write types covered and the count
P10	Limits statement	Present, over 200 characters, unique
P11	Second reader	A person who did not author the profile reads the known-good report and can state, unprompted, what it does not prove
P12	Offline	Completes on an air-gapped machine; any network call fails the test

P5 and P11 "are the two that fail most often in a first draft" [E13 §06]. P11 requires a second reader before elan1 has a compliance hire; who that is remains open until 31 March 2027 [E13 open question 4].

11.8 Who verifies what

The same bundle serves different readers asking different questions. The table maps each to what they verify, which profile they need and which outcomes matter to them; it synthesises E07, E14, E15 and E18 **(build recommendation for the grouping)**.

Reader	Question	Profile	Outcomes that matter	Source
Tenant's evidence owner (finance controller)	"Will this pass for them?" — runs the verifier before handing it on	The claimed regime's	Any non-zero exit is theirs to fix before sending	E15 §05 step 6
Statutory auditor or CA	Does the trail exist, operate throughout and resist tampering, for writes made through elan1? Plus E11 test 3 against the accounting system	`in-mca-audit-trail`	1 (forensic), 2 (control design), 0 with limits read	E13 §07; E18 §07
Lender's diligence team, customer compliance function	Can a third party check it without the vendor?	Per regime, or none (integrity only)	1 vs 2 distinction; scope line	E14 §04
Software vendor reselling assurance	Can the bundle ride with their product?	Their customer's regime	Free, reimplementable verifier	E14 §04
Regulator's or notified body's reviewer	Record-keeping and oversight evidence without provider cooperation	Regime profile (e.g. EU AI Act Art. 12, DPDP) — none certified yet	All, with the limits statement	E11 §7; E13 open question 1
Counterparty or disputant	Did this record exist in this form, signed and anchored, and has it been altered?	None required	`1; PASS — payload unbound` wording	E15 §06
Independent implementer	Is the format checkable without elan1 software?	Any	Agreement with published vectors (route 4)	E15 §07

No profile is certified yet, and which regime gets the first profile — MCA audit trail, DPDP or EU AI Act Article 12 — is itself an open question in two documents (30 June 2027 and Q2 2027), although E13 decides for `in-mca-audit-trail` [E11 open question 2; E07 open question 2; E13 decision 8]. Who may author a profile, elan1 only or a certified partner, is open until 30 June 2027; partner authoring is "the scale story and the largest risk surface" [E13 open question 2].

11.9 Auditor acceptance and the pilot

The working brief for the auditor acceptance document states its purpose — "how an audit firm or regulator-facing team tests our evidence, and what counts as acceptance" — and the decision it must end in: "a written acceptance or a written objection from a real auditor". Its status is *NOT STARTED* [WD Phase 2 07]. The protocol the memorandum can describe is therefore assembled from the audit note, the pack template and the IM, which agree with each other [E11 §8; E18 §07; IM §14.3].

Before the pilot opens. The pack passes seven engineering tests, stated in advance: the declaration validates with no blank fields; the profile is certified by someone who is not its author; every write type's reversal plan has been executed once in a dated drill; at least one real refusal exists per declared gate; a bundle exits PASS on an air-gapped machine run by someone outside the team; the published tampered bundle exits `FAIL_INTEGRITY` naming record and field; and a sound but non-conforming bundle exits `FAIL_PROFILE`, never `FAIL_INTEGRITY`. The eighth test is the auditor: they read a report unprompted and can state what it does not prove; "if they cannot, the limits statement is wrong" [IM §14.3; E18 §04].

The ninety days. One customer, one auditor [E18 §07]:

Day	Step [E18 §07; IM §14.3]
0	The auditor's half day, before anything is shown: three questions they would ask, and a written statement of what they would *need* to rely on a record. Where it names something unbound, the profile gets a `not_measurable` row, "not a quiet omission"
1	The tenant upgrades L1 → L2 under the runbook; the boundary date and counts on each side are shown to customer and auditor
2–75	Normal operation: no special handling, no curated data, no suppressed refusals
76	The customer exports the bundle themselves, without help. "If they need our help, that is the finding"
77	The auditor obtains the verifier from the published release or builds it, disconnects and runs it; elan1 "is not in the room"; then runs the tampered bundle and watches it fail

Day	Step [E18 §07; IM §14.3]
78	E11 test 3: two-way reconciliation between the elan1 chain and ERPNext's own audit trail
80	The written verdict, in the auditor's words, published internally verbatim "including if it is unflattering"
85	The founder asks the customer: would you pay for this on its own line? "The answer, whatever it is, closes the pilot"

What changes if the auditor says no. A `FAIL_INTEGRITY` during the pilot "is a P0 and stops the pack, not just the pilot"; a `FAIL_PROFILE` is a profile defect and goes back to the author; an `INCOMPLETE` is an export defect; if the auditor will not run the verifier themselves, "the pilot has failed its main purpose and is written up as such" [E18 §07]. If the day-80 verdict is that L2 is insufficient, the open question is whether they need L3 or "something elan1 has not built at all" [IM §14.3; E18 open question 6]. elan1 never asks for an endorsement, a logo or a quotation, and never describes an auditor conversation as validation [E11 §8, decision 5].

The first pilot is the ledger1 pilot, and the IM's gate calendar dates its dependencies: key custody with one rotation performed by 31 August 2027; two seal implementations agreeing on all 64 vectors ("L2 signing live") by 30 September 2027; two timestamp authorities contracted and ledger1 live with a named statutory auditor, pilot day 0 complete, by 31 October 2027; the anchor interval published and met over a measured month, and the verifier shipped as source with the tampered bundle, by 31 December 2027; Gate 1 — "a paying customer's write verifies on an air-gapped machine, no elan1 process running" — and the day-80 pilot verdict by 31 January 2028; L2 GA in April 2028 [IM §16.5; IM §4.9]. Two consequences follow. The pilot runs on a pre-GA L2 build for one tenant, which is consistent with its precondition that "the customer is at L2" [E18 §07]. But day 1's upgrade falls before the anchor SLO has been met over a measured month, which is one of the seven upgrade preconditions [E19 §03 item 3; E07 §06]. The recommendation is to hold day 1 until all seven E19 preconditions read true for the pilot tenant, and to let the calendar move rather than the precondition **(build recommendation)**; the tension is recorded in the conflicts block. The pilot is Gate G-D's test, and the evidence line's Gate 3 at the end of Q4 2028 is "the day somebody signs a contract with evidence on its own line", or "an honest statement that it did not happen" [E18 §07; E14 §04, §07 item 11].

11.10 Performance targets

Nothing here has been measured, and the corpus gives almost no verifier timings; the targets below are proposed so that the build has numbers to fail against **(build recommendation)**, anchored where a source exists.

Measure	Target	Basis
Verify the reference golden bundle (4,096 seals, one tampered variant) offline	< 2 s on a laptop	EBB's reference fixture "verifies offline, on a machine with no network, in under two seconds" [EBB EB-01]
Verify a 100,000-record year (~150 MB)	< 60 s, single core; memory bounded by streaming `seals.jsonl`	Size model [E15 §04]; SHA-256 and Ed25519 verification each cost microseconds per record
Verifier memory	Independent of record count except for the current anchor batch	Streaming design
Export a 100,000-record range	< 10 min end to end, excluding download	Build target; the precondition gate requires a completed nightly run, so export reads a verified range
Re-export determinism	Byte-identical except `exported_at`, 100% of runs	E15 decision 5
Unanchored head at export	Within the published anchor interval (≤ 15 min) under normal operation	E12 §05; E14 §07 item 4
Verifier release size	Small enough to read in a day (source); single static binary per OS	E15 §07 route 3

Two cautions. The per-record cost is dominated by Ed25519 verification and SHA-256 over small inputs, both fast in Go's standard library; the long pole is JSON parsing and inclusion-path hashing, so the verifier should parse `seals.jsonl` line by line and cache each batch root once. And any published number must be measured on stated hardware, in line with the group rule that no latency figure appears in a customer document until measured [IM §6.10].

11.11 Test plan

#	Test	Asserts	Cadence	Source
T1	Vector run	36 positives PASS; 28 negatives rejected with record, field and class named	Every verifier commit and release	E12 §08; E15 §07
T2	Tampered bundle	The published tampered bundle exits 1 naming the altered record and field	Every release; every pilot	E15 §06 step 6
T3	Class separation	P5 (exits 2, never 1) and P6 (exits 3, never 2) fixtures	Every commit	E13 §06
T4	Erasure wording	Destroyed-salt fixture exits 0; the string FAIL absent from the report	Every commit	E13 P7
T5	L1 range	L1-only bundle exits `0 PASS — unsigned, assurance level 1`; a signature member on an L1 line exits 1	Every commit	E12 §04; E13 P8
T6	Boundary-spanning range	Mixed L1/L2 bundle exits 0 with both levels and counts in scope	Every commit	E19 §04
T7	Key rotation and revocation	Records under a retired key PASS; under a revoked key PASS with the revocation in Notes; outside the window exit 1	Every commit	E12 §06; IM §7.7
T8	Two-authority anchors	Either token verifying suffices; tokens disagreeing on the root exit 1; `single_anchor` batch passes with a note	Every commit	IM §7.6
T9	Truncation	A bundle with the last anchored record removed exits 1 via leaf count; unanchored tail printed	Every commit	E15 §02; E13 C06
T10	Air gap	Full run inside a network namespace with no interfaces	Every release	E13 P12; E15 §08
T11	Reproducible build	Two builders, byte-identical outputs; an outsider reproduces the digest	Every release; once before GA by an outsider	E15 §07; E14 §07 item 10
T12	Zero dependencies	Empty lockfile; no network-capable imports	Every commit	E12 decision 9
T13	Parser fuzzing	Malformed JSON, oversize lines, truncated tar, path traversal in member names: exit 3 or 5, never a crash, never a read outside the bundle directory	Nightly (`go test -fuzz`)	ETS §09 (fuzzing tool); build recommendation
T14	Export determinism	Two exports of one range and profile differ only in `exported_at`	Every export-path commit	E15 decision 5
T15	Export refusals	Residency, unresolved declaration, incomplete nightly run and split-window cases refuse with the stated offers; oversize refuses with a split	Every export-path commit	E15 §05
T16	Witnessed air-gapped run	A person who has never seen the code exports and verifies on an air-gapped machine; written up with the witness named	Once before L2 GA	E14 §07 item 7

T9 deserves a note: removing a record from the *middle* fails at I3; removing the *last* anchored record fails at I7 because the batch's leaf count and root no longer match; removing unanchored tail records cannot be detected by the bundle at all and is disclosed through `unanchored_head` and Notes [E15 §02; IM §7.5]. The test asserts all three behaviours, including the third as an honest non-detection.

11.12 What exists at Level 1 in the meantime

At L1 there is no bundle and no offline verifier, and elan1 may not imply an auditor can check the chain without it [E01 §07; E07 §02]. What an L1 tenant has is the `verify` call inside the tenancy boundary, the nightly integrity job and its three-valued continuity report, the reconciliation outputs, and the declaration versions verbatim — the five things E11 tells an auditor to ask for (Chapter 9 §9.15) [E11 §6]. A profile can be dry-run against an L1 chain, and the dry-run is useful precisely because it fails to pass quietly: C06 reads `not_applicable — assurance level 1` on the face of the report [E13 §07]. When a Phase 1 customer is asked for the L2 sentence — "here is a file you can check yourself, on your own machine, without our vendor's help" — the instruction is to "say we are not there yet and give them the date" [E07 §07].

12. Key management, custody and timestamp authorities

Level 2 rests on two things elan1 does not fully control: a signing key held in a custody module, and a timestamp issued by an authority with no interest in the matter. The Technology Stack register calls these "the two load-bearing vendors" and says they are the only places in the Build Book where a vendor choice matters: both must be chosen per residency domain and contracted before any seal leaves the building [ETS §06]. This chapter specifies both. It covers the key inventory, how keys are generated and held, the register and its rotation windows, the compromise procedure, dual control and the audit of key use, how public keys are published, key ceremonies, the customer-held keys that arrive at L3, and then the timestamp authorities: why there are two, how an RFC 3161 token is obtained and checked, what happens during an outage, what it costs and what the resulting time evidence is worth in India.

The governing documents are the Key Management and Custody specification [E16] and the Timestamp Authority Memo [E17], both dated 21 September 2026 and both in "draft for review" status. The IM summarises them in §7.6 and §7.7 [IM §7.6, §7.7]. Nothing in this chapter exists in code or in a contract as at 23 September 2026.

> **Designed is not shipped (DR-010).** No elan1 signing key exists, no timestamp authority is under contract, and no anchor has been obtained. The Level 1 product being built now carries no key material at all. Every interval, cost and volume figure below is a target or a modelled number unless it is marked otherwise.

12.1 What exists at each level

12.1.1 Level 1: no signing key, by rule

At L1 there is no key material anywhere: "not in a config file, not in a vault, not 'just for testing'" [E16 §01; E12 §06]. The rule was cheap to keep because nothing at L1 needs a key. The L1 record is hash-chained (Chapter 9) and its claim boundary forbids the words "signed", "independently verifiable" and "tamper-proof"; the only permitted integrity claim is that the log is tamper-evident to the tenant, who holds the chain [E07]. An L1 dry run of a verifier profile therefore reports `0 PASS — unsigned, assurance level 1`, with the signature, key-window and anchor checks reading `not_applicable` [E13 §04; E15 §06].

L1 does have transport secrets — TLS certificates and per-call target credentials that expire in ten minutes — which are not signing keys and are not in the evidence path [E16 §02; O05 §04].

12.1.2 Level 2: signing and anchoring arrive together

L2 adds exactly four things: Ed25519 signing of the frozen sixteen-field record, RFC 3161 anchoring of a batched Merkle root, verifier profiles executing rather than dry-running, and evidence export as a priced product [E14 §01]. Two of the four depend on this chapter. The L2 frozen sentence names both: "elan1 signs each record with a key held in the tenant's residency domain, anchors a Merkle root of those records to two external timestamp authorities at a published interval" [E14 §01].

The group dependency map dates the inputs. Key custody per residency domain depends on stack1's residency-domain deploy topology and secrets handling reaching GA quality, needed by 31 August 2027. RFC 3161 anchoring depends on a signed authority contract by 31 October 2027 and anchoring live by 31 December 2027. Both sit on the critical path to Gate 1 on 31 January 2028 and the Phase 3 exit on 31 March 2028 [O09 §03, §04]. L2 general availability is targeted for April 2028 [IM §8.1]. The timestamp contract is described as "the single most likely cause of a Phase 3 miss" and is one of two Phase 3 items the founder may not delegate [O09 §04, §09].

12.1.3 Level 3: custody moves to the customer

L3 adds customer-held keys: "the customer's own HSM signing their own records, so that elan1 cannot have signed anything they did not" [E20 §03]. It is gated, like every L3 capability, on an outside auditor accepting an L2 bundle (Chapter 14). Customer-held keys are explicitly out of scope for Phase 3 [E14 decision 3].

Capability	L1 Guarded	L2 Attested	L3 Proven
Record signing key	None, anywhere	One per residency domain, managed KMS	Optionally customer-held, customer HSM
Anchor client key	None	KMS, same domain	Unchanged
Release key (verifier)	Not required (no verifier shipped)	Offline hardware token, two holders	Unchanged
Transport keys, minted credentials	Yes	Yes	Yes
Timestamp anchoring	None	Two authorities, ≤ 15 min interval	Unchanged; optional non-elan1 counter-signature
Key discovery by a reader	Not applicable	Out-of-band fingerprint	Adds issuer key document and trust-anchor routes

Sources: [E07; E14 §02; E16 §02; E17; E20 §03; E22 §03].

> **No back-signing, at any level.** A record written before a key existed is never signed afterwards. Records written before a tenant's upgrade stay Level 1 and are labelled so forever; the boundary date and the counts on each side are printed in every bundle [E07; E12; E14 §06; E19].

12.2 The design principle: a key is a capability to be accounted for

E16 states its principle in one line: "a key is not a secret we protect, it is a capability we account for" [E16 §01]. Protecting the key is table stakes. The product is being able to say afterwards exactly what the key signed and what could have asked it to. Four structural rules follow, and the rest of the chapter is their detail. **Custody:** key material is generated inside the custody module and never leaves it — never exported, backed up, printed, escrowed or copied between residency domains; "if a procedure in this document ever appears to require the private key in a file, the procedure is wrong" [E16 §01]. **One invoker:** only the chain writer may invoke the domain signing key, only for a record produced by rung 7, and there is no administrative "sign this" surface [E16 §08]. **No hierarchy:** no root key and no certificate chain; key discovery is a register problem, solved out of band [E16 §02]. **Three witnesses:** every use of a key is visible in the KMS audit log, the elan1 chain and the external anchors, which would all have to be corrupted together [E16 §08].

12.3 The key inventory

E16 defines four key classes. They do different jobs, live in different places, and a compromise of one does not imply a compromise of another [E16 §02].

Class	Purpose	Where it lives	Who may invoke	Rotation
Domain signing key	Signs the 32-octet record digest at rung 8	Managed KMS, asymmetric Ed25519, in the domain's region	The chain writer process, nothing else	12 months, or on compromise
Anchor client key	Authenticates elan1 to a timestamp authority; signs nothing a reader verifies	KMS in the same domain	The anchor service	Per authority policy, at least annually
Release key	Signs verifier binaries, source tarballs and published vector sets	Offline hardware token held by two named humans, not in any cloud	Release approver, at a release	24 months
Transport keys	TLS; the per-write single-use credential to the target system	Ordinary platform secret management	The single writer	Per platform policy

Source: [E16 §02; IM §7.7].

Figure 16 shows the hierarchy, where each class lives, who may invoke it, and the rotation and insider-limitation boundaries discussed below.

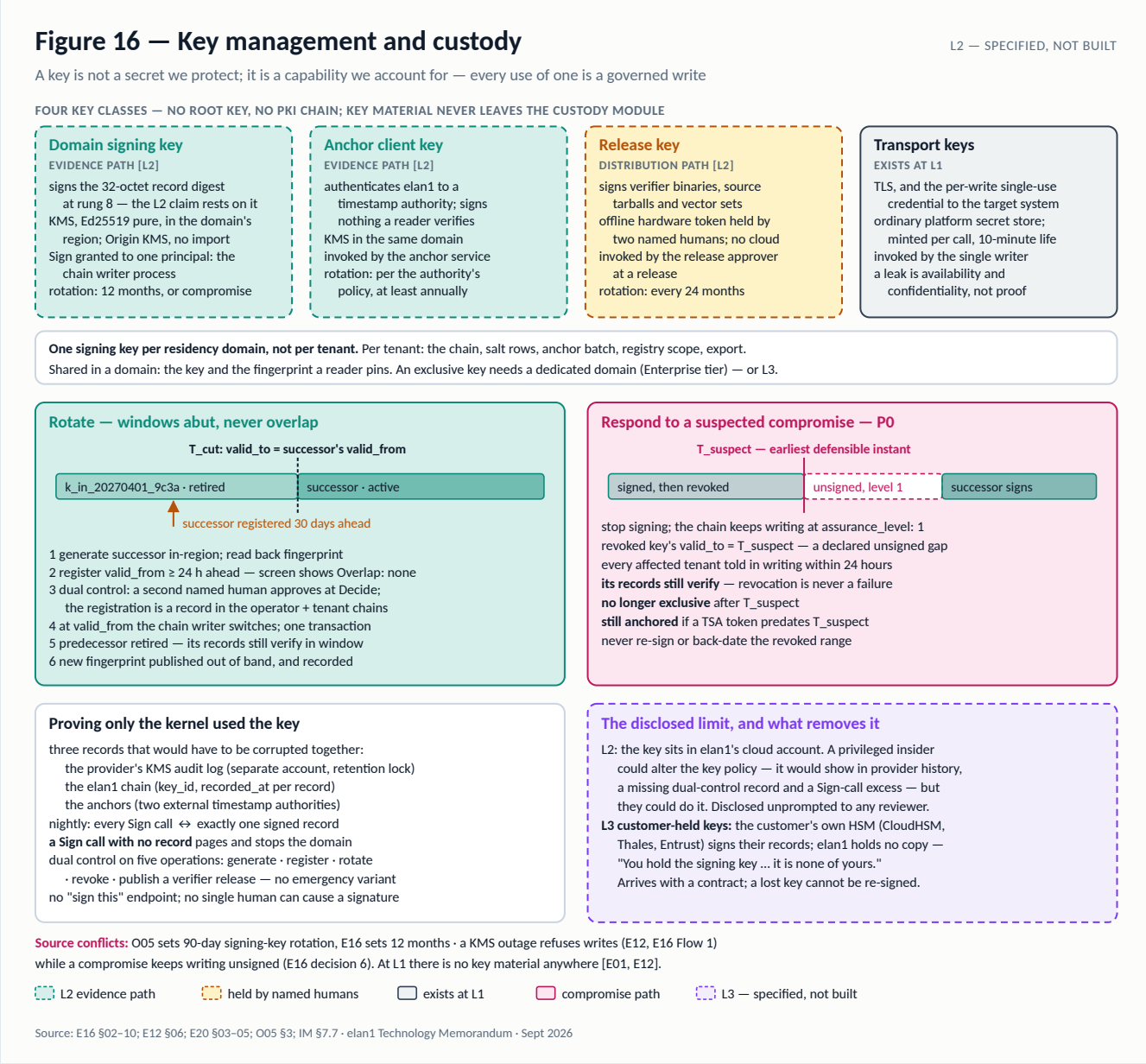


Figure 16 — Key classes by residency domain: the domain signing key and anchor client key in a managed KMS invoked only by the chain writer and anchor service, the offline release key, abutting rotation windows, three independent records of key use, and the insider limitation that customer-held keys remove at L3.

Only the first two are in the evidence path. The release key is in the distribution path: it is what lets a sceptic confirm that the verifier binary they downloaded is the one elan1 published [E15 §07]. Transport keys are listed so nobody mistakes them for a signing key during an incident [E16 §02].

12.3.1 The domain signing key

The domain signing key is the key "the whole L2 claim rests on" [E16 §02]. It signs the SHA-256 digest of the length-prefixed sixteen-field preimage defined in Seal Spec v6 (Chapter 10) using pure Ed25519 [E12; E14 §02]. The signature step is step 6 of the chain-write flow, after the record has been appended under `UNIQUE (tenant_id, seq)`; the writer records the `key_id` alongside the signature [E12 §05]. The signature member is present on `assurance_level: 2` records and absent — not stubbed — on L1 records [E14 §02].

12.3.2 The anchor client key

The anchor client key authenticates elan1 to the timestamp authorities. It signs nothing a reader ever checks. The token a reader checks is signed by the authority's key, not elan1's [E16 §02; E17 §02]. Its compromise would let an attacker consume elan1's timestamp quota or impersonate elan1 to the authority. It would not let anyone forge an anchor, because the anchor's value comes from the authority's signature.

12.3.3 The release key

The release key signs the offline verifier's binaries, the source tarball, `SHA256SUMS` and the published vector set. Publishing a verifier release is a governed write under dual control. The release approver signs the artefacts, and publication goes to at least two independent locations — elan1's own site and a public package or source registry [E15 §07]. The release key is held on an offline hardware token by two named humans and is never in a cloud [E16 §02]. Who the two holders are, and what happens when one leaves, is open until 31 May 2027 [E16 §12 Q4].

12.3.4 Development keys

Software keys in OpenBao, Vault transit or a file are forbidden at every stage. The one exception is a visibly non-production development key whose fingerprint is published as `dev-DO-NOT-TRUST`. Vectors and development bundles signed with it are recognisable by fingerprint alone, and a verifier given the production fingerprint rejects them [E16 §03]. The stack register adds that a YubiHSM 2 is acceptable as low-cost hardware custody for development and staging key ceremonies, never for production volume [ETS §06].

12.3.5 What is deliberately absent

Six things are absent by design: a root key or CA chain; a per-tenant key on a shared domain; a spare or fallback key ("a spare key in a drawer is the thing we are protecting against"); any key in a bundle or receipt ("strictly worse than no signature at all"); any backup or DR copy of key material, "including for disaster recovery"; and any administrative sign endpoint or break-glass signing [E16 §02, §04, §07, §08, §10; E12 §06; E22 §03].

12.4 Generation, storage and the custody module

12.4.1 Phase 3: managed KMS

The Phase 3 design is AWS KMS with the `ECC_NIST_EDWARDS25519` key spec, in the residency domain's region, using pure Ed25519 and never the pre-hash variant. The key is created with `Origin: KMS`, with no key-material import, and with a key policy that permits `Sign` to exactly one IAM principal [E16 §03]. AWS KMS has supported Ed25519 signing in all regions since November 2025 [ETS §06, §14]. Google Cloud KMS `EC_SIGN_ED25519` at HSM protection level is the named alternative; it runs in FIPS 140-2 Level 3 HSMs and is available in the Mumbai and Delhi regions [E16 §03; ETS §06]. The choice between them is open until 31 March 2027, and the deciding criterion is the attestation evidence each provider gives per domain [E16 §12 Q1; ETS §13 row 5].

The pre-hash exclusion is enforced by negative vector NV-14, a signature made with the `ph` variant, which every implementation must reject [E12 §07; ETS §06]. The seal specification also requires Ed25519 strictness: non-canonical `S` values and small-order public keys are rejected [ETS §13 row 3; E12 §07].

The key policy below illustrates the "exactly one principal" rule. It is a **build recommendation**: the sources fix the property, not the document.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ChainWriterSignOnly",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::<evidence-acct>:role/e1-chain-writer-in"},
      "Action": ["kms:Sign", "kms:GetPublicKey"],
      "Resource": "*"
    },
```

```
    {
      "Sid": "DenySignToEveryoneElse",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "kms:Sign",
      "Resource": "*",
      "Condition": {"ArnNotEquals": {"aws:PrincipalArn":
        "arn:aws:iam::<evidence-acct>:role/e1-chain-writer-in"}}
    }
  ]
}
```

The chain-writer role must be assumable only by the chain writer's compute identity, and no human principal may be listed. That is the first thing an auditor reads in Flow 3 [E16 §09]. The signing call must select the pure variant (for AWS, a raw-message Ed25519 call rather than the digest/pre-hash form); the exact API names are to be confirmed against current provider documentation when the ADR is written (**build recommendation**; [ETS §14 caveats]).

12.4.2 Later: single-tenant HSM and customer-held keys

Single-tenant HSM custody — AWS CloudHSM, or a customer's own Thales Luna or Entrust nShield device — is a stronger custody story and more operational work. It "arrives when a customer asks for it with a contract attached, not before" [E16 §03; E16 decision 3]. The stack register lists CloudHSM with a caveat to check Ed25519 mechanism support, and Luna and nShield with EdDSA support to be confirmed per firmware [ETS §06]. At what revenue single-tenant HSM custody stops being premature is an open founder question for Phase 4 [E16 §12 Q6]. Customer-held keys are treated in §12.11.

12.5 Residency domains and per-tenant isolation

A residency domain is the unit of key custody. Residency and key custody sit at plane P0, which has no migration path [O01; O05 §02], so choices made here are expensive to reverse. The group baseline fixes Phase 1 hosting at ap-south-1 Mumbai across three availability zones, with no second region without a written residency decision [O05 decision 2]. The first domain is therefore `in`, and the example key identifier `k_in_20270401_9c3a` carries it [E16 §05].

E16 makes a decision that the IM says "be volunteered rather than discovered": **one signing key per residency domain, not per tenant** [E16 §04; IM §7.7]. Ten thousand tenants on the `in` domain share a signing key and therefore share the fingerprint a reader pins.

Scope	What it covers
Isolated per tenant	The chain (F01, its own `seq` from 1); the salt store rows; the anchor batch; the declaration registry scope; the export
Shared within a domain	The signing key, and therefore the fingerprint a reader pins
Consequence	A tenant cannot prove its records were signed by a key no other tenant's records used
Who gets a dedicated key	A tenant on a dedicated residency domain (the Enterprise tier) — a key nothing else uses

Source: [E16 §04].

The per-tenant alternative was rejected because it multiplies rotation ceremonies, key-window bookkeeping and fingerprint distribution by the tenant count, and "the isolation it buys is mostly imaginary while we hold every key in the same account" [E16 §04]. A tenant that needs an exclusive key takes a dedicated domain now, or customer-held keys at L3 [E16 decision 2]. Whether a tenant on a shared domain needs its own signing key before L2 GA is still an open question, due 31 July 2027 [E14 §09 Q5].

> **Conflict with the group security baseline.** O05 requires per-tenant signing keys, ninety-day rotation and a published JWKS [O05 §04, §12, decision 4], and IM §10 repeats that wording. E16 fixes one key per residency domain, annual rotation and out-of-band fingerprints only. This memorandum builds to E16; the conflict is recorded at the end of the chapter.

Each tenant has a **key binding**: the record of which domain key signs its chain. It is set in stage 1 of the L1 → L2 upgrade, before signing is switched on, and the screen shows `Key bound · signing off` [E19 §04 stage 1]. Binding is fully reversible until signing is on. Unbinding leaves the key in the register, unused for that tenant [E19 §06].

12.6 The key register and rotation windows

12.6.1 Register fields

Field	Form	Note
`key_id`	`k_<domain>_<YYYYMMDD>_<4 hex>`, e.g. `k_in_20270401_9c3a`	Opaque to a verifier except for the domain segment
`valid_from`, `valid_to`	RFC 3339, second precision	The window during which this key may have signed
`successor_id`	`key_id`	Registered before the predecessor's window closes
`status`	`active` · `retired` · `revoked`	A revoked key's records still verify

Source: [E16 §05; E12 §06]. The same fields, plus the public key fingerprint and never the key, form `keys.json` in the evidence bundle [E15 §02].

> **Windows abut and never overlap.** The successor's `valid_from` is the instant the predecessor's `valid_to` is set to. "One second of overlap makes 'which key could have signed this record' unanswerable, and unanswerable is the state this entire platform exists to prevent." The registry refuses a `valid_from` earlier than the predecessor's last signature [E16 §05; E12 §06].

The database can enforce the non-overlap rule directly rather than trusting application code. The schema below is a **build recommendation**. It uses a PostgreSQL exclusion constraint over half-open ranges, so abutting windows are legal and any overlap fails at commit.

```
CREATE TABLE key_register (
  key_id        text PRIMARY KEY
                CHECK (key_id ~ '^k_[a-z]{2,8}_[0-9]{8}_[0-9a-f]{4}$'),
  domain        text NOT NULL,
  key_class     text NOT NULL CHECK (key_class IN ('domain_signing','anchor_client')),
  kms_key_ref   text NOT NULL,          -- provider ARN or resource name; never material
  fingerprint   text NOT NULL UNIQUE,   -- lower-case hex
  valid_from    timestamptz NOT NULL,
  valid_to      timestamptz,            -- NULL while active and open
  successor_id  text REFERENCES key_register(key_id),
  status        text NOT NULL CHECK (status IN ('active','retired','revoked')),
  revoked_at    timestamptz,            -- = T_suspect when status = 'revoked'
  revoke_reason text,
  registered_by_seq bigint NOT NULL,    -- operator-chain seq of the governed write
  CHECK (status <> 'revoked' OR (revoked_at IS NOT NULL AND revoke_reason IS NOT NULL)),
  EXCLUDE USING gist (
    domain WITH =, key_class WITH =,
    tstzrange(valid_from, coalesce(valid_to, 'infinity'), '[)') WITH &&
  )
);
-- Append-only: UPDATE permitted only to set valid_to, successor_id, status,
-- revoked_at, revoke_reason; enforced by trigger and by grants to the kernel role only.
```

The constraint does not stop a `valid_from` earlier than the predecessor's last signature; that check reads the chain and belongs in the governed write's precondition gate [E16 §05].

12.6.2 Cadence

Scheduled rotation is annual. The successor is generated and registered thirty days before the cut instant, with a `valid_from` at least 24 hours in the future [E16 §05, §06]. Rotation is "deliberately dull and deliberately practised: an organisation that has never rotated a key in production will not rotate one well during an incident" [E16 §05]. A rotation performed in production at least once is a hard precondition for L2 GA and for any tenant's L1 → L2 upgrade [E14 §07 item 3; E19 §03 row 2; IM §7.7]. Whether annual survives an auditor who asks for quarterly is open until 30 September 2027 [E16 §12 Q3].

12.6.3 Flow 1 — rotating a key

The actors are the key custodian and a second named human. The trigger is the annual schedule or the compromise procedure. The preconditions are a successor generated in the same residency domain and no open chain-integrity incident for the tenant [E16 §06].

Step	Actor	Action	System result
1	Custodian	Platform · Agents · Keys → Generate successor	KMS creates the key in-region; platform reads back the public key and fingerprint; nothing signs with it
2	Custodian	Registers the successor, `valid_from` ≥ 24 h ahead	Screen shows both windows and `Overlap: none`, or refuses
3	Second human	Approves at the Decide screen	Registration is a governed write with `dual_control: true`; it climbs the nine rungs and appends to the operator chain and each affected tenant's chain
4	Chain writer	At `valid_from`, signs with the successor	Predecessor `valid_to` set to the same instant; both windows written in one transaction
5	Custodian	Marks predecessor `retired`	It may no longer sign; its records still verify against its window in `keys.json`
6	Operator	Views both keys	Windows, rotation record's `seq`, record count under each; the next bundle carries both keys and a boundary row in `chain.json`
7	Custodian	Publishes the new fingerprint out of band	Publication recorded

Source: [E16 §06; E12 §06].

The refusal paths are three. `GATE.DUAL_CONTROL_SAME_ACTOR` fires when requester and approver are the same person; the screen offers eligible names and no agents. An overlapping `valid_from` refuses with both windows printed. A rotation attempted while a chain-integrity incident is open refuses at precondition, "because rotating during a break makes the forensics harder" [E16 §06]. A chain that spans a rotation is itself a published positive test vector, PV-07 [E12 §06, §07].

The failure path is the one that matters operationally. If the KMS is unavailable at the cut instant, the platform fails closed: every write in the domain refuses with a platform-class code (`PLATFORM.TOKEN_MINT_FAILED`-class in E12's wording), nothing is signed with a stale key, and a page goes out immediately. "A platform that stops sealing is recoverable; a platform that seals badly is not" [E16 §06; E12 §06].

> **An unresolved tension inside the sources.** E16 Flow 1, E12 and E19 make an unavailable KMS refuse every write in the domain; E16 Flow 2 and decision 6 keep writing unsigned at `assurance_level: 1` when signing must stop [E16 §06, §07; E12 §06; E19 §04]. This memorandum distinguishes an undeclared KMS failure on an active key (fail closed, page) from a declared unsigned interval opened by the incident commander as a dual-controlled governed write (writes resume at level 1, boundary rendered) (**build recommendation**; see the conflicts block).

12.7 Compromise, revocation and the unsigned interval

12.7.1 Triggers and the one number

Flow 2 starts on any of four triggers: the KMS audit log shows a `Sign` call the platform cannot account for; a credential with `Sign` permission is exposed; an insider event; or a vendor advisory. "Suspicion is enough to start" [E16 §07]. The incident commander declares a P0 key incident and records `T_suspect`, the earliest time the key may have been usable by someone else. "This is the number everything else hangs off, and it is always the earliest defensible time, never the most convenient one" [E16 §07].

12.7.2 Flow 2 — responding to a suspected compromise

Step	Action	Effect
1	Commander declares a P0 and records `T_suspect`	The anchor instant for every later statement
2	Commander stops signing in the affected domain	Chain writer keeps writing and chaining, unsigned, at `assurance_level: 1`; the Assurance door says so in those words
3	Custodian generates and registers a successor under Flow 1 dual control	Predecessor's `valid_to` is set to `T_suspect`, not to the successor's `valid_from`: a declared unsigned interval, not a fictitious continuous one
4	Custodian marks the predecessor `revoked` with reason and `T_suspect`	Written append-only; every later bundle carries the revocation, reason and instant in `keys.json`
5	Commander notifies every affected tenant in writing within 24 hours	Names the instant, the record range signed by the revoked key, what is still true and what is not; wording fixed in advance
6	Anchors used as the recovery instrument	Records signed before `T_suspect` and covered by an anchor issued before `T_suspect` have an independent witness
7	Commander re-enables signing under the successor	Unsigned interval closed; the boundary and counts on each side rendered permanently

Source: [E16 §07; IM §7.7].

12.7.3 What happens to records signed by a revoked key

Three statements are true at once, and the tenant notice must contain all three [E16 §07].

Statement	Meaning
They still verify	The signature is arithmetically valid and the verifier reports PASS. Revocation is not a verification failure and is never reported as one
They are no longer exclusive	After `T_suspect`, "only the kernel could have produced this signature" is no longer defensible. Before it, it is
They are still anchored	A record covered by an anchor issued before `T_suspect` has an independent witness to its existence and content at that time

> **Revocation limits what may be signed after the revocation instant. It never invalidates what was signed before.** Back-dating a revocation to cover an embarrassment, or re-signing a revoked range under the successor, is back-signing and is forbidden. A request to re-sign the revoked range is refused by the kernel, not by a policy document. A revocation with a `T_suspect` later than the moment suspicion was raised requires the founder's approval and a written reason retained with the incident [E16 §07].

In `keys.json` a revoked key looks like the entry below. The field set is from E15 and E16; the exact JSON shape is a **build recommendation**.

```
{
  "key_id": "k_in_20270401_9c3a",
  "fingerprint": "5f0c…e21a",
```

```
  "valid_from": "2027-04-01T00:00:00Z",
  "valid_to": "2027-11-14T06:12:00Z",
  "status": "revoked",
  "revoked_at": "2027-11-14T06:12:00Z",
  "revoke_reason": "KMS audit shows Sign call with no matching record (INC-0412)",
  "successor_id": "k_in_20271115_41be",
  "unsigned_interval": {"from_seq": 88213, "to_seq": 88790}
}
```

The verifier's treatment is fixed. For a bundle, a record inside a revoked key's window is still an integrity PASS, and the report prints the revocation [E16 decision 7]. For a cross-organisation receipt at L3, the verifier reports `REVOKED` with the revocation time and does not decide whether a record signed before it is trustworthy: "that is the receiver's judgement" [E22 §10].

If the successor cannot be generated — a regional KMS outage — the domain stays unsigned and keeps writing. "There is no fallback key and no spare in a drawer, because a spare key in a drawer is the thing we are protecting against" [E16 §07]. What gets recorded is the incident, `T_suspect`, the revocation as a governed write, the unsigned interval with its record count, the notifications with their timestamps and named recipients, and the reason — "verbatim, forever" [E16 §07].

12.8 Dual control and the audit of key use

12.8.1 The five dual-controlled operations

Five key operations require dual control: generating a key, registering a window, rotating, revoking, and publishing a verifier release. Each is a declared governed write with `dual_control: true`, so each climbs the nine rungs of the Right Path and appends a record [E16 §08]. None has an emergency variant. Break-glass is itself a declared write type with dual control, recorded in the operator's own chain and visible to the tenant [E16 §08]. Changes to the KMS key policy and IAM policy are also governed writes with dual control [E16 §09 step 2]. At L2, switching signing on for a tenant is a sixth dual-controlled write, the "irreversible instant" of the upgrade runbook [E19 §04 stage 2; E19 decision 5].

"No single human can cause a signature" [E16 §08]. The custodian can register a key, but only the chain writer can invoke it, and only for a record produced by rung 7. A self-approval is refused with `GATE.DUAL_CONTROL_SAME_ACTOR`, and agents never appear among eligible approvers [E16 §06]. The rule and the declaration validator reinforce each other: any write type with a high blast radius is forced to dual control by the declaration's semantic rules (Chapter 7) [E21; E22 §07].

12.8.2 Three independent records of key use

Source	What it shows	Who controls it
The cloud provider's KMS audit log	Every `Sign` call: principal, time, key, request count	The provider; written to a separate account with a retention lock
The elan1 chain	Every record, with `key_id` and `recorded_at`	The kernel, append-only
The anchors	An independent authority's timestamp over the Merkle root of those records	Two external timestamp authorities

Source: [E16 §08].

The group baseline adds that the key ring's own audit log is retained and reviewed, that security and access logs are kept at least one year with a 180-day minimum in India, and that evidence goes to WORM object storage in India [O05 §04, §06].

12.8.3 The nightly reconciliation

A nightly job compares the KMS log and the chain in both directions. Every `Sign` call must correspond to exactly one signed record, and every signed record to exactly one `Sign` call. "A Sign call with no record is the most serious alert the platform can raise — it means something invoked the key that was not the chain writer." It pages immediately and stops the domain [E16 §08; E16 decision 9]. An excess of records over calls is impossible and would indicate a fabricated record [E16 §09].

The query below is a **build recommendation**. It assumes the KMS log is ingested nightly into a read-only staging table in the evidence account, and that the chain writer stores the provider's request identifier with each signature so the join is exact rather than temporal.

```
-- Result is three-valued per domain and range: reconciled | break_at | not_checked
WITH calls AS (
  SELECT request_id, key_ref, event_time
  FROM kms_audit_ingest
  WHERE event_name = 'Sign' AND event_time >= :from AND event_time < :to
),
signed AS (
  SELECT r.kms_request_id AS request_id, r.tenant_id, r.seq, r.key_id, r.recorded_at
  FROM proof_record r
  WHERE r.assurance_level = 2 AND r.recorded_at >= :from AND r.recorded_at < :to
)
SELECT
  (SELECT count(*) FROM calls c
     WHERE NOT EXISTS (SELECT 1 FROM signed s WHERE s.request_id = c.request_id))
     AS sign_calls_without_record,      -- > 0 : P0, stop the domain
  (SELECT count(*) FROM signed s
     WHERE NOT EXISTS (SELECT 1 FROM calls c WHERE c.request_id = s.request_id))
     AS records_without_sign_call,      -- > 0 : fabricated record or log gap
  (SELECT count(*) FROM calls) AS sign_calls,
  (SELECT count(*) FROM signed) AS signed_records;
```

If the provider's log for a period has a gap, the reconciliation says so in three values: `reconciled`, `break at N`, or `not checked over this range`, with the scope and count printed. "We do not report an unchecked range as clean" [E16 §09]. Whether the KMS audit log should be streamed into each tenant's chain as system records, making the reconciliation checkable offline rather than on request, is open until 31 December 2027 [E16 §12 Q5].

12.9 Proving the key was used only by the kernel, and the insider limitation

12.9.1 Flow 3 — the auditor's question

The question arrives "in roughly these words: 'how do I know your engineers cannot sign whatever they like?'" [E16 §09]. The answer is a procedure the reviewer runs themselves, assuming no trust in elan1.

Step	Reviewer asks for	Reviewer checks
1	KMS key policy and IAM policy, verbatim	`Sign` granted to exactly one role, assumable only by the chain writer's compute identity; no human principal
2	Who can change that policy	Change-management record (a dual-controlled governed write) and the provider's configuration history service, which elan1 does not write to
3	KMS audit log for a period, from the retention-locked account	Counts `Sign` calls
4	The chain over the same period	Counts signed records
5	—	Reconciles the counts. Equality is the finding
6	A bundle from the period	Verifies offline, with a fingerprint elan1 did not give them; integrity passes independently of everything above
7	—	Reads the limitation, stated unprompted

Source: [E16 §09].

A request for the key material itself is refused, "and the refusal is the correct answer — a custody module that can export a key is not a custody module" [E16 §09]. The evidence request is recorded as a governed write in the tenant's chain, so the tenant can see that their auditor asked and what was supplied [E16 §09].

12.9.2 The disclosed insider limitation

> **The limitation, disclosed unprompted by founder decision.** At Phase 3 the signing key is held in elan1's own cloud account under a managed KMS. A sufficiently privileged insider in that account could alter the key policy. They could not do so without the provider's configuration history recording it, without the dual-control record being absent, and without the `Sign`-call reconciliation showing an excess — but they could do it. The custody model that removes this is customer-held keys, which is L3 [E16 §09; E16 decision 10; IM §7.7].

The IM describes this as "the strongest available argument that the rest of the document is true" [IM §7.7]. For a diligence team the practical reading is this. At L2, elan1's custody claim is detective, not preventive, against its own privileged staff. The controls make an insider signature detectable after the fact through three independent records; they do not make it impossible. Preventive separation needs a key elan1 never holds.

12.10 Publication of public keys and trust anchors

12.10.1 The out-of-band fingerprint

A reader pins the signing key's fingerprint, and gets it from their own source, never from the bundle [E13 §04; E15 §01]. `keys.json` in a bundle carries `key_id`, `valid_from`, `valid_to`, `status`, `successor_id` and the public-key fingerprint — never the key [E15 §02]. The verifier is invoked with `--key-fingerprint <hex>` supplied by the reviewer [E15 §06]. The fingerprint's construction is not fixed in the sources. This memorandum recommends SHA-256 over the 32-octet raw Ed25519 public key, rendered as 64 lower-case hex characters, with the `dev-DO-NOT-TRUST` label reserved for the development key (**build recommendation**). This should be written into the Seal Spec's vector set so that a second implementer computes it identically.

Three timing rules govern publication:

1. The fingerprint is published to the tenant's notice contact through the out-of-band channel **before any record is signed**, and the publication is recorded. "A reader who receives the fingerprint after the records is a reader who received it from the same party who produced them" [E19 §04 step 7; E19 decision 4].
2. After every rotation, the custodian publishes the new fingerprint through the channel every pinning reader was given at onboarding, and records that they did [E16 §06 step 7].
3. At the end of an upgrade, the operator verifies the first ten signed records using the fingerprint the tenant was given, "not one from our own registry" [E19 §04 step 11].

The channel itself is open until 30 April 2027: "a signed notice to the notice contact, a DNS record, a printed page at onboarding?" [E16 §12 Q2]. This memorandum recommends using two of the three, so that a reader who distrusts one channel has a second (**build recommendation**): a printed and countersigned onboarding page, plus a DNS TXT record under a domain elan1 controls, carrying `key_id` and fingerprint.

12.10.2 L3: issuer key documents and trust anchors

At L3, cross-organisation key discovery adds an issuer-published key document at `https://<domain>/.well-known/action-receipt-keys` and a specified-but-unoperated third-party trust-anchor route to bilateral pinning; a receipt's own key hint is never trusted, and elan1 never operates a trust anchor for its customers [E22 §03, decision 3]. Chapter 14 covers these routes. The key document is the only JWKS-like publication in the phase documents, and it is published by the tenant about its own keys, not by elan1 about a shared domain key.

12.10.3 Timestamp-authority roots and the release key

Trust in an anchor rests on the authority's certificate chain reaching a root the reader accepts; elan1 does not supply that root. The anchor service captures the full certification path and the OCSP or CRL response at anchor time into `anchors/`, so validity is assessed as at `genTime` [E17 §08; E15 §02]. The reference verifier should ship with no embedded root store and require the reader to name the authority roots they trust, as it requires the signing fingerprint (**build recommendation**).

The release key's fingerprint is also obtained out of band; it anchors the first of four routes by which a sceptic confirms a verifier binary, the others being a reproducible rebuild, reading the standard-library-only source, and writing an independent verifier from E12 and E13 [E15 §07].

12.11 Key ceremonies

The sources use the word "ceremony" in three places: the L3 definition of done requires a "key ceremony record" proving a tenant signs with a key elan1 has never held [E20 §07 item 7]; E16 notes that per-tenant keys would multiply "rotation ceremonies" [E16 §04]; and the stack register assigns YubiHSM 2 to development and staging ceremonies [ETS §06]. None of them specifies the ceremony itself. The protocol below is a **build recommendation**. It assembles the rules E16 does fix — dual control, generation inside the custody module, readback of the public key only, out-of-band publication — into a scripted, witnessed event that produces its own record.

Ceremony	When	Participants	Output
C1 Domain key generation (KMS)	New residency domain; each annual successor	Custodian, second named human	KMS key created in-region with `Origin: KMS`; fingerprint read back; register entry; governed-write record
C2 Release key generation	Before the first verifier release; every 24 months	The two named release-key holders, one witness	Key generated on the offline token; fingerprint published to two locations
C3 Customer-held key enrolment (L3)	Tenant moves to customer-held keys	Customer's key officers, elan1 custodian as witness only	Customer HSM generates key; elan1 receives public key only; ceremony record; elan1 register shows no private-key reference
C4 Dev/stage key	Environment creation	One engineer	`dev-DO-NOT-TRUST` fingerprint, or YubiHSM 2 in staging

A ceremony record should carry the ceremony kind, domain, participants and their roles, the custody module with its provider attestation reference, the public-key fingerprint, a digest of the verbatim key policy, the governed-write `seq` in the operator chain, the fingerprint publications with channel and time, and an explicit `private_material_exported: false` (**build recommendation**).

For C3, the proof E20 asks for has two halves: the customer's ceremony record showing the key was generated in their HSM, and "our own key register showing no entry" [E20 §07 item 7]. In practice the register will need an entry for a customer-held key, because a verifier resolves windows from `keys.json`. The entry must therefore hold the public fingerprint and window but no `kms_key_ref`, together with a marker that the custody module is external. This reading of "no entry" as "no private-key reference" is recorded as a conflict to close.

12.12 Customer-held keys at L3

Customer-held keys are the L3 answer to the question the regulated enterprise asks: "Can you prove you could not have signed this for me?" E20 calls it "a procurement blocker at large institutions, not a preference" [E20 §04]. The L3 claim is: "You hold the signing key. We can show you every record we signed, and it is none of yours" [E20 §05].

They change three things at once, which is why E14 kept them out of Phase 3 [E14 §03; E16 §03]:

What changes	At L2 (elan1 custody)	With customer-held keys
Custody model	elan1's managed KMS; insider limitation disclosed	Customer's HSM (CloudHSM, Thales, Entrust); elan1 cannot have signed
Compromise procedure	elan1 declares `T_suspect`, revokes, notifies	The customer's key officers hold the evidence for `T_suspect`; elan1 can stop invoking but cannot revoke what it does not hold (**build recommendation**: revocation becomes a customer-initiated governed write, co-approved)
Availability	Regional KMS	The customer's HSM and the network path to it become a dependency of every signed write
Meaning of a signature	"Only elan1's kernel could have produced this"	"Only the kernel, invoking the customer's key under the customer's policy, could have produced this"
Lost key	Not applicable (no export, no loss short of provider failure)	elan1 cannot re-sign; what is owed is open (founder with counsel, Q2 2028) [E20 §11 Q5]

What changes	At L2 (elan1 custody)	With customer-held keys
Salts	elan1 holds per-record salts	Who holds the salt store is open [E23 §12 Q4]; "we cannot see your data" becomes true only with customer-held keys *and* customer-held salts [E20 §05]

The IM places customer-held keys as the second of three sovereign topologies: a dedicated residency domain operated by elan1; customer-held keys inside an elan1-operated deployment; and the whole kernel and evidence plane inside the customer's boundary, operated by them [IM §8.5]. The third is gated on a defect-propagation process existing before any sale [E20 §08].

Architecturally, the chain writer should call a custody interface — `key_for(tenant, at)`, `sign(key, digest32)` returning signature and provider request id, `public_fingerprint(key)` — with no method that accepts or returns key material, so that a customer-HSM backend (for example via PKCS#11) can be added at L3 without touching rung 8 (**build recommendation**). What happens when a customer HSM is unreachable is not specified. E16's logic points to failing closed, since continuing unsigned would let elan1 write records into a customer-held-key tenant's chain that the customer's key did not cover; the customer could pre-authorise a declared unsigned interval as a dual-controlled governed write (**build recommendation**, open for the L3 specification).

12.13 Timestamp authorities: what an anchor is for

A hash chain is arithmetic over data held by the party who wrote it. It detects alteration in the middle. It cannot detect two things [E17 §01]:

- **A truncated tail.** "A chain cut at record 4,000 of 4,200 is internally consistent and looks complete." Every digest verifies and every link matches.
- **When anything happened.** F16 `recorded_at` is the platform's clock, and elan1 owns the platform. A chain recomputed wholesale with plausible timestamps is indistinguishable from an honest one to anyone without an outside reference.

An anchor fixes both. A timestamp token from an authority elan1 does not control, over a Merkle root covering a set of records, establishes that those records existed in that form no later than the token's time. The batch's leaf count and inclusion paths make a removed record visible [E17 §01]. This is why the MCA audit-trail profile's non-disablement check requires every record to be covered by an anchor whose token time precedes now, and why that profile's minimum assurance level is 2 [E13 §07; IM §7.8].

> **What an anchor proves.** "This digest existed, in this form, no later than this instant, according to a party who has no interest in the matter. Nothing more." It does not prove the record was created at that time, only that it existed by then. It does not prove the record set is complete; it detects removal after submission, not records never submitted. It does not prove elan1 is honest: "if we anchor fabricated records faithfully, the anchors are faithful evidence of fabricated records." And it has no legal effect that elan1 determines [E17 §01].

Figure 17 shows the anchoring path from chain write to bundle, and the degraded states described in §12.17.

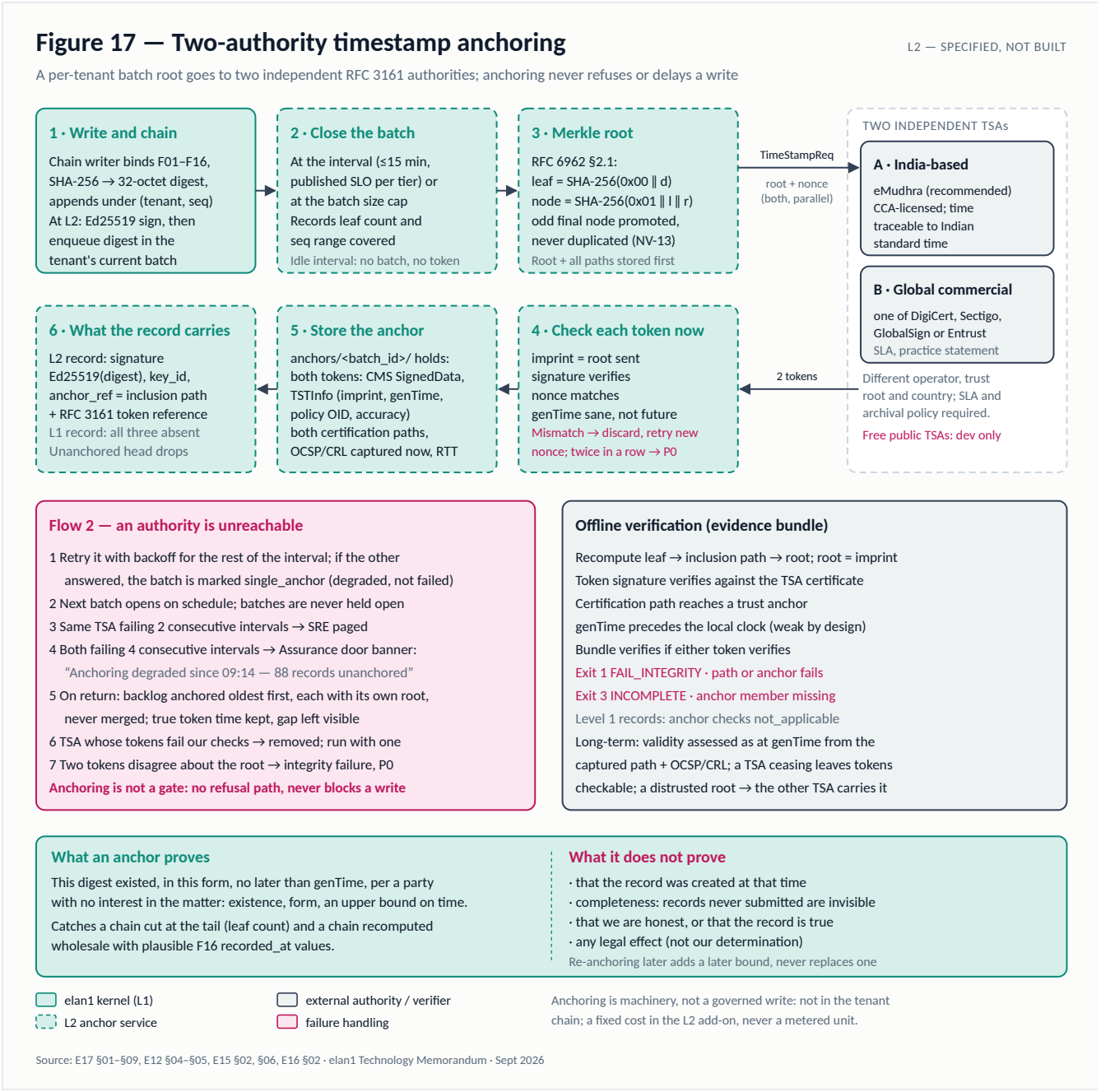


Figure 17 — Two-authority anchoring: per-tenant batch, RFC 6962 Merkle root with odd node promoted, parallel RFC 3161 requests to an India-based and a global authority, token checks, anchors/ contents, and the single_anchor, paging and banner states when an authority fails.

12.14 The RFC 3161 exchange

12.14.1 Four steps

E17 describes the protocol in four steps "with no cryptography needed to follow them" [E17 §02]:

1. elan1 computes a hash: a Merkle root over a batch of record digests.
2. elan1 sends the authority a request containing only that hash and a random nonce. "The authority never sees a record, a payload, a tenant name or anything else."
3. The authority attaches its own clock reading, signs the pair with its own key, and returns a token.
4. Anyone holding the token, the hash and the authority's certificate can check the signature and read the time, with no further contact with the authority. That is what makes the token work "offline, in a bundle, in ten years".

The token is a CMS `SignedData` wrapping a `TSTInfo` that carries the `messageImprint` (the hash and its algorithm), `genTime`, the authority's policy identifier and the accuracy [E17 §02]. The request structure below follows RFC 3161; the field choices are a **build recommendation**.

```
TimeStampReq ::= SEQUENCE {
  version         1,
  messageImprint  { hashAlgorithm: id-sha256, hashedMessage: <32-octet Merkle root> },
  reqPolicy       <authority's policy OID, pinned per contract>,     -- optional in RFC
  nonce           <64-bit+ random integer, fresh per request and per retry>,
  certReq         TRUE,        -- ask for the TSA certificate in the token
  extensions      absent }
```

12.14.2 What the anchor service checks on receipt

Before storing a token, the anchor service verifies it: the imprint matches the root it sent, the signature verifies, the nonce matches, and `genTime` is sane [E17 §05 step 3]. A token whose imprint does not match is discarded and the request retried with a new nonce; two mismatches in a row is a P0 against that authority. A token whose `genTime` is in the future is discarded as a defect at the authority [E17 §05].

12.14.3 What a reader's verifier checks

A verifier checks four things. The imprint equals the root it recomputed from the inclusion path. The token's signature verifies against the authority's certificate. The certification path reaches a trust anchor. And `genTime` precedes the verifier's own clock. That last check "is deliberately weak — it catches a token from the future, which is the mistake a test fixture makes, not an attacker" [E17 §02]. In the offline verifier this is step 6 of seven, after Merkle inclusion [E15 §06; ETA]. A failed anchor check is `1 FAIL_INTEGRITY`, and a missing anchor member is `3 INCOMPLETE` [E15 §06].

12.15 Selecting two authorities

12.15.1 Six criteria

All six must pass. The first three are disqualifying [E17 §03].

#	Criterion	Why
1	RFC 3161 compliant, with a published practice statement	A proprietary "timestamping service" cannot be checked by a standard-library verifier
2	A traceable time source, stated	"An authority that will not say where its time comes from is not an authority"
3	An SLA with an availability number and a response-time number	elan1 publishes an anchor interval; without an SLA that is "a hope"
4	Long certificate validity and an archival policy	A token outlives the batch
5	Priced per token or per subscription, quotable in writing	The anchor is the one unresolved cost line
6	Independent of the other authority: different operator, trust root and country of incorporation	"Two authorities that fail together are one authority"

12.15.2 The two-authority rule

Every batch is anchored by two authorities, and at least one is India-based. Both tokens go into `anchors/`. A bundle verifies if either token verifies. A bundle whose two tokens disagree about the root is an integrity failure and a P0 [E17 §03; E17 decisions 1–2]. The rule serves two purposes. The first is availability: two missed intervals in a row because a single authority was down is an avoidable gap. The second, "the one that pays for it", is survival: an authority that ceases operating, loses its licence or has its root distrusted takes its tokens' checkability with it [E17 §03].

12.15.3 Candidates

eMudhra is the recommended India-based authority. The pair is one global commercial authority from DigiCert, Sectigo, GlobalSign or Entrust [E17 §03; ETS §06]. Free public authorities such as FreeTSA have no SLA and are permitted only in development and reference environments, never in a production bundle [E17 decision 9; ETS §06]. The stack register records two global authorities as the alternative to the India-plus-global pair, to be settled in wave W2 against the SLA for the ≤ 15-minute interval [ETS §13 row 6].

Three questions remain open. Which global authority pairs with eMudhra, and does the pair survive a procurement review on independence (31 May 2027) [E17 §10 Q2]? Does an India-resident authority exist that meets both RFC 3161 and DPDP residency, "or do we anchor outside India and explain it?" (31 March 2027) [O09 §12 Q1]? And does any Indian auditor care that the token is India-based? E17 frames that last one as "ask before paying for it", with two chartered accountants, by 31 March 2027 [E17 §10 Q6; IM §7.6].

12.16 Cadence, batching and Merkle aggregation

The published interval is **≤ 15 minutes, per tier, as an SLO rather than a tuning knob**. "An auditor's question is not 'is it anchored?' but 'how long could a record have sat unanchored?'" [E17 §04]. The interval is never negotiated per customer: "the moment an interval is negotiable, the anchor stops being a property of the platform and becomes a property of the contract" [E17 §07; E17 decision 3]. Whether Enterprise may buy a shorter interval, and whether that contradicts decision 3, is open until 31 July 2027 [E17 §10 Q3]. L2 is done only when the interval has been met over a measured month, with the anchor-lag metric published rather than estimated [E14 §07 item 4].

Batching is one token per tenant per interval. The chain writer enqueues each record's 32-octet digest into the tenant's current batch. At the interval, or at the batch size cap, the anchor service closes the batch and builds a Merkle tree per RFC 6962 §2.1: `leaf(d) = SHA-256(0x00 ‖ d)`, `node(l,r) = SHA-256(0x01 ‖ l ‖ r)`, and an odd final node is promoted, not duplicated. It then requests tokens over the root from both authorities, and stores each record's inclusion path so that any single record proves alone [E17 §04; E12 §05]. A path is $\log_2(n)$ digests: for a 1,024-record batch, ten digests or 320 octets [E17 §04]. The path is why a bundle can cover one record without exposing a thousand others, and why a redacted export still anchors [E17 §04].

> **The odd-node rule.** RFC 6962 promotes the odd final node; several popular implementations duplicate it. "This is where two competent engineers build something self-consistent and mutually incompatible." Negative vector NV-13 exists so that a duplicating implementation fails loudly [E17 §04; E12 §07].

The reference computation below is a **build recommendation**. It follows RFC 6962's recursive definition, which splits at the largest power of two below n. That definition is equivalent to pairwise reduction with the odd node promoted.

```
import hashlib
def _h(b: bytes) -> bytes: return hashlib.sha256(b).digest()
def leaf(d: bytes) -> bytes: return _h(b"\x00" + d)
def node(l: bytes, r: bytes) -> bytes: return _h(b"\x01" + l + r)

def mth(ds: list[bytes]) -> bytes:
    """RFC 6962 §2.1 Merkle Tree Hash over record digests ds (each 32 octets)."""
    n = len(ds)
    if n == 1:
        return leaf(ds[0])
    k = 1 << ((n - 1).bit_length() - 1)      # largest power of two < n
    return node(mth(ds[:k]), mth(ds[k:]))     # odd tail is promoted, never duplicated
```

The sources do not fix a batch size cap. A cap of 4,096 leaves (twelve-digest paths) is proposed so that a single anchor file stays bounded for high-volume tenants (**build recommendation**). E15 estimates an anchor file at about 6 KB plus about 1 KB per covered record's inclusion path [modelled] [E15 §04].

12.17 Flow 1 — anchoring a batch, and Flow 2 — when an authority fails

12.17.1 The normal path

Step	Action	Why it matters
1	Close the batch; record leaf count and `seq` range	The count is evidence: it makes a missing leaf visible
2	Build the tree; store root and every inclusion path before contacting anyone	A failed request leaves a batch that can be retried unchanged
3	`TimeStampReq` to authority A with a fresh nonce; verify the token	Imprint, signature, nonce, `genTime`
4	Same to authority B, in parallel	Two independent witnesses
5	Write both tokens, both certification paths and the revocation data to `anchors/<batch_id>/` ; mark records anchored	The Assurance door's unanchored head drops
6	Operator sees `last counter-signature 3 minutes ago, 0 records unanchored`	The gap is shown, not logged

Source: [E17 §05]. Egress to both authorities is by hostname and is the evidence plane's only internet egress [E17 §05; ETS §06]. Anchoring is not a gate and never refuses or delays a write. A record is written and chained whether or not it has been anchored, because "coupling it to the write path would mean an authority's outage stopped a customer's business" [E17 §05; E17 decision 5]. The anchoring operation is not a governed write and does not appear in the tenant's chain. It is machinery, and putting it in the chain "would make the chain grow in proportion to its own anchoring" [E17 §05]. The dependency map adds one integration hazard: the batched Merkle job depends on stack1's `fusion.run/v1` contract guaranteeing that a replay does not re-run the anchor side effect, otherwise duplicate anchors result (needed by 30 November 2027) [O09 §03 item 4].

12.17.2 Degraded states

Condition	State	Response
One authority fails within an interval, the other answers	Batch marked `single_anchor`	Degraded, not a failure; retry the failed authority with backoff for the rest of the interval
Same authority fails two consecutive intervals	Still `single_anchor`	SRE paged
Both authorities fail four consecutive intervals	Unanchored head grows	Assurance door changes from a count to a banner: `Anchoring degraded since 09:14 — 88 records unanchored`
Authority returns	Outstanding batches anchored oldest first, each keeping its own root	Never merged, because merging loses interval boundaries an auditor is entitled to read
Late anchor	Batch carries its true token time	The gap is visible and never closed by adjusting anything
Authority returns tokens that fail elan1's own checks	Removed from the pair by operational decision	Remaining authority continues alone; replacement contracted; "running with a known-bad authority to preserve the pair is worse than running with one"

Source: [E17 §06; E17 decisions 6, 10]. Batches are never held open waiting for an authority. Holding them "makes one outage into a growing unanchored head and hides the problem in a single large batch later" [E17 §06]. SRE writes up any period in which both authorities failed for more than one interval, and the write-up is available to any tenant whose records sat in the gap [E17 §06]. The `single_anchor` marking travels into every bundle covering the batch. Whether it should block a profile that asserts non-repudiation, or only be printed in the report, is open until 30 September 2027 [E17 §10 Q5]. L2 is not done until a single-authority outage has been survived in a dated, written-up drill [E14 §07 item 5].

12.18 Long-term validity

Event	Effect on an issued token	What elan1 does
Authority certificate expires	None on the arithmetic; the verifier notes validation against an expired certificate	Capture full certification path and revocation data at anchor time; assess validity as at `genTime`
Certificate revoked later	Token remains evidence of what it was; a revocation after `genTime` does not retroactively unsign it	The captured OCSP or CRL response from anchor time is what the reader assesses
Authority ceases to exist	None; checking needs no contact	Why path and revocation data are captured, and why there are two
Authority's root distrusted	Path no longer reaches an accepted trust anchor	The second authority carries the batch
SHA-256 or Ed25519 weakens	Everything weakens together, including the chain	Out of scope for Phase 3; a change is a new domain string and a new verifier

Source: [E17 §08]. Re-anchoring an old root under a new authority "proves less than it looks": it establishes only that the root existed no later than today. It is a hedge before an authority winds down, adds a later upper bound, and never replaces the earlier one. A bundle carrying both prints both times [E17 §08]. Whether to re-anchor historical roots on a schedule is open until 31 December 2027 [E17 §10 Q4].

elan1 does not run a blockchain. "The property wanted here is a timestamp from a party with no interest in the matter, which RFC 3161 provides for a fee, with a practice statement, an SLA and a legal entity to complain to" [E17 §08]. Publishing to a transparency log under RFC 9943 (SCITT, with COSE Receipts as RFC 9942) is an optional extra publish step, and offline verification never depends on it [E17 §08; IM §7.6; ETS §06].

12.19 Cost, and how it enters the meter

The anchor is "the one number in the commercial model we still do not have". Pricing at 10^8 records a year is not publicly quoted [unverified], and at that volume "it is either trivial or fatal" [E17 §07]. Batching makes the exposure calculable, because cost scales with tokens per tenant per interval rather than with records.

Tenant behaviour	Tokens per month	Records per token
Writing continuously, 15-minute interval	~2,880 per authority, ~5,760 for the pair	Whatever accrued
300 governed actions a month	~300 for the pair [modelled]	An idle interval produces no batch and no token

Source: [E17 §07]. A small tenant's anchoring cost is proportional to its activity; a large tenant's is capped by the interval. The plan carries anchoring as a cost-of-revenue line at two authorities of Rs 0.07 crore in FY2028-29, Rs 0.15 crore in FY2029-30 and Rs 0.29 crore in FY2030-31 [modelled], "under two-tenths of a paisa" per governed action against 2,491 million governed actions in the final year. "That is the arithmetic the plan assumes, not a quotation received" [IM §7.6].

Anchoring never appears on the meter as a unit. There is no per-anchor and no per-bundle charge for anchoring. It is a fixed cost inside the L2 attestation add-on, alongside key custody, for three reasons: it is not proportional to governed actions; a per-anchor line would give elan1 an incentive to lengthen the interval; and a customer should never be able to buy a cheaper interval [E17 §07; E17 decision 8]. The meter basis stays the declared blast radius, unchanged from L1 [E17 §07]. The real price must be known by 30 June 2027, because it is "the only cost line that could invalidate the L2 price" [E17 §10 Q1; E14 §09 Q1].

12.20 Evidence of time, and its standing in India

The platform carries three kinds of time, and they must never be conflated.

Time	Source	What it establishes
F16 `recorded_at`	Platform clock at chain-write time, never the caller's; RFC 3339 UTC, second precision	When elan1's clock says the record was written. "A caller-supplied timestamp is a caller-supplied lie waiting to happen" [E12 §02, §03]

Time	Source	What it establishes
System clock discipline	NTP synchronised to NPL or NIC, verified by a monitoring check	That the platform clock tracks Indian national time, as a control [O05 §06]
Anchor `genTime`	The authority's clock, signed by the authority	An upper bound on when the record existed, from a party with no interest [E17 §01]

A reader can compare `recorded_at` against `genTime` for every anchored record. A record whose `recorded_at` falls after its anchor's `genTime` is impossible in an honest chain. The published negatives cover the adjacent cases — an anchor whose time postdates the check, and a `recorded_at` outside its key's window [E12 §07] — and this memorandum recommends adding `recorded_at` later than `genTime` as a further negative vector (**build recommendation**). A record whose `recorded_at` falls long before `genTime` shows an anchoring delay, which the bundle prints.

The legal position is recorded as sourced and no further. The Controller of Certifying Authorities publishes the Time Stamping Services Guidelines for Certifying Authorities, version 2.0 (September 2016). They require tokens to comply with RFC 3161, permit licensed Certifying Authorities to issue timestamping certificates to external entities, and require synchronisation to Indian Standard Time within ±1 second against a National Physical Laboratory caesium clock, all under the Information Technology Act 2000 [published] [IM §7.6; IM ref 43; ETS §06]. The IM draws the commercial conclusion that a token from a CCA-licensed authority "has its time attested under the IT Act", which is "a materially stronger evidentiary posture" in an Indian court, RBI inspection or MCA proceeding than a timestamp from a transparency log [IM §13.4].

The engineering documents are more guarded, and they govern what the product and its staff may say. E17 states that a token from a licensed Indian CA "has whatever standing Indian law gives it. That is not our determination and we do not opine on it" [E17 §01]. E14 bans "court-ready" and "legally admissible" at L2 because "admissibility is a matter for a court and for the Indian Evidence Act's treatment of electronic records. We supply an artefact; we do not opine on its reception" [E14 §05]. E20 keeps the ban at L3 [E20 §05].

> **Two points for counsel, outside the corpus.** First, the Indian Evidence Act 1872 was replaced by the Bharatiya Sakshya Adhiniyam 2023 with effect from 1 July 2024. E14's reference should be updated, and counsel should confirm how electronic-record certification under the new Act interacts with an RFC 3161 token. Second, the IM's sentence that the token "has its time attested under the IT Act" is stronger than E17's position. Product material should use E17's wording, and the IM's wording should be checked by counsel before external use.

12.21 Acceptance criteria

The L2 definition of done and the E16 and E17 decisions reduce to the checks below. None has been run.

#	Check	Evidence	Source
K1	Key register exists for `in`, with an active key, a registered successor and abutting windows	Register extract; exclusion-constraint test	[E14 §07 item 3]
K2	One production rotation performed before L2 GA and before any tenant upgrade	Rotation records in the operator and tenant chains; PV-07 regenerated	[E14 §07; E19 §03]
K3	Nightly two-way reconciliation runs, and an injected unaccounted `Sign` call pages and stops the domain	Drill record	[E16 decision 9]
K4	Self-approval of any of the five operations refused with `GATE.DUAL_CONTROL_SAME_ACTOR`	Negative fixture	[E16 §06, §08]
K5	Re-sign of a revoked range refused by the kernel	Negative fixture	[E16 §07]
K6	Verifier reports PASS for records under a retired or revoked key, and prints the revocation	Vector	[E16 decision 7]
K7	Flow 3 run by a reviewer outside the team, limitation stated unprompted	Written account	[E16 §09]

#	Check	Evidence	Source
T1	Two authorities contracted, one India-based	Contracts	[E14 §07 item 5]
T2	Single-authority outage survived in a drill	Dated write-up	[E14 §07 item 5]
T3	Interval SLO met over a measured month, anchor lag published	Metric	[E14 §07 item 4]
T4	Duplicating Merkle implementation fails NV-13	CI vector run	[E17 §04]
T5	Disagreeing tokens produce `1 FAIL_INTEGRITY` and a P0	Vector and runbook	[E17 decision 2]
T6	Anchoring outage never refuses a write	Fault-injection test	[E17 decision 5]

13. Packs — the pack factory, ledger1, invoice1 and collect1

The kernel is built once; the regulatory surface is built continuously. A pack is the unit in which that surface is built, sold and audited: "one declaration, one certified verifier profile and one regime-specific export mapping, aimed at one buyer with one auditor under one dated obligation" [E18 §01]. This chapter covers what a pack is and is not, the five admission gates and the eight candidates rejected by name, the pack definition template field by field, how packs are tested, the first pack (`ledger1`) in full, the second and third (`invoice1` and `collect1`), the pack factory and its cadence, the principle that a regime is a pack and a country a set of packs, and how packs are packaged, versioned, signed and distributed. Chapter 7 covered the declaration layer packs are built from; Chapters 10 and 11 cover the seal, bundle and verifier a pack's evidence depends on.

> **Designed is not shipped (DR-010).** No pack exists as running configuration on 23 September 2026. `ledger1` is planned live in October 2027, `invoice1` in April 2028 and `collect1` in October 2028, all MODELLED. `collect1` is a spec sketch whose rule citations "need counsel confirmation before anything ships" — the only document in the corpus carrying that label [E27; IM §5.5–§5.6; O01 DR-010].

13.1 What a pack is

A pack is not a feature and not a module. It is "the smallest thing a buyer can buy that produces an artefact their auditor will open" [E18 §01]. The kernel does not know what a pack is and the declaration registry has no pack table: a pack is a composition of things that already exist, "plus the two things that do not exist until somebody does the work: a certified profile and a buyer who can name their auditor" [E18 §01]. "Packs are the sale. Screens are the surface. Nobody buys an approvals screen" [E18 §01].

The IM describes the same object in five parts — "declarations, policy rules, regulatory bindings, an evaluation set and an approver model" [IM §1.4]. The template in E18 is written in eleven blocks. The two descriptions reconcile as follows:

IM part	Where it lives in the E18 template	Kernel artefact it resolves to	Source
Declarations	`declaration_ref`, `write_types[]`, `gates[]`	A certified declaration version (seven blocks); gate parameters and order	[E18 §04; E03]
Policy rules	`gates[]` parameters; rung 4 rule sets	Gate `params`; a `policy_version`	[E18 §04; E04 §03]
Regulatory bindings	`obligation`, `profile_binding`	A certified verifier profile (E13); from Phase 4 a Regulation and its Controls (E21)	[E18 §04; E13; E21]
Evaluation set	`tests[]`, the fixture set, the tampered bundle	T1–T8, profile tests P1–P12, reference-tenant fixtures with a refusal per gate	[E18 §04; E13 §06]
Approver model	Not a named block	`permissions` grants with `may_approve`, `dual_control` per write type, resolver sets on obligations	[E03 §02; E06 §03]

The approver model is the one IM part with no block of its own. This memorandum recommends that the pack manifest carry an explicit `approver_model` section — which roles may approve which write types, dual-control thresholds, quorum, voice or WhatsApp reply eligibility, and default expiry — compiled into the referenced declaration's `permissions` and `write_types` rather than held separately, so that there is still one source of truth **(build recommendation)**.

What a pack may change, and what it may never change, follows from policy being data (Chapter 7 §7.13). The marginal pack adds no rung, no gate, no record field — "and specifically no seventeenth field, at any level, ever" — no declaration schema change and no verifier change, "because a profile is data and the verifier reads it" [IM §5.6]. "Six steps, no code. If a

pack needs code, it needs a component, and the component belongs in [the shared layer] where all twenty apps get it" [ESB Ch.11 §6].

13.2 The five pack gates and the eight rejections

Every candidate is scored against five gates before any build work. All five must pass, and "a gate scored 'probably' is scored fail" [E18 §02–§03].

Gate	Test	Fails when
G-A A dated hook	A named regulation, standard or contract with a date on it	The hook is a principle, a direction of travel, or "regulators are increasingly focused on"
G-B An auditor	Somebody whose job is to ask for the evidence — internal audit, external assurance, an examination team, a certification body, a counterparty's procurement	Nobody is obliged to ask. "If nobody is obliged to ask, nobody asks"
G-C Reversible, or irreversible and declared so	The write types can declare which they are without lying	Everything important is irreversible and everyone pretends otherwise, so the reversal-plan discipline produces fiction
G-D Evidence sold separately	A buyer would pay for the bundle on its own line item	Evidence is a nice-to-have attached to something else
G-E We own the write	elan1 is in the write path, not reading somebody else's log	"We observe rather than govern … observation produces telemetry, and telemetry is not evidence"

Source: [E18 §02]. G-E "kills the most candidates and is the least negotiable"; a candidate whose answer is "we could integrate with their log" is refused on the spot [E18 §02–§03].

Figure 18 shows how a candidate passes the gates, enters the template and moves through the factory.

Figure 18 — The pack factory: five gates, eleven blocks, eight tests

DESIGNED — NOT YET BUILT

A pack is configuration, not code: a candidate must pass all five admission gates before any build work starts

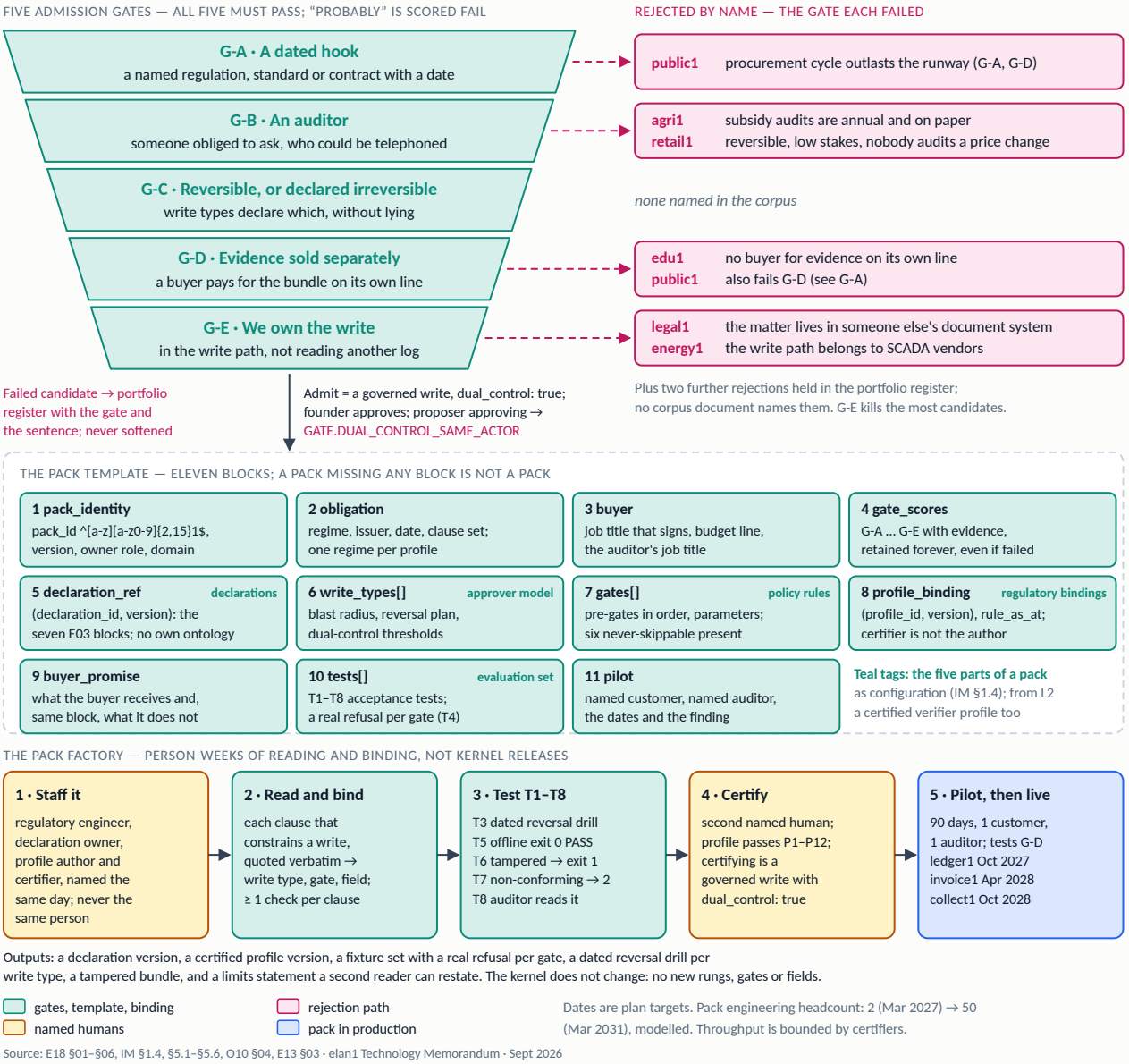


Figure 18 — The pack factory: a candidate is scored against the five gates G-A to G-E (eight rejected by name), admitted by a governed write under dual control, composed from the eleven-block template, tested against T1–T8, piloted with a named auditor and certified.

The eight rejections. E18 names six and says two more are "held in the portfolio register"; the IM reproduces the six and discloses the other two "as a gap rather than reconstructed" [E18 §02; IM §5.2]. The product-folder Strategy Blueprint and the Requirement-to-Deployment Checklist name all eight, and this memorandum uses them, at the lower weight those documents carry:

Candidate	Gate failed	The sentence	Source
agri1	G-B	Subsidy audits are annual and on paper	[E18 §02]
legal1	G-E	The matter lives in someone else's document system	[E18 §02]
energy1	G-E	The write path belongs to SCADA vendors	[E18 §02]
public1	G-A, G-D	The procurement cycle outlasts the runway	[E18 §02]
edu1	G-D	No buyer for evidence on its own line	[E18 §02]

Candidate	Gate failed	The sentence	Source
retail1	G-B	Reversible, low stakes, nobody audits a price change	[E18 §02]
hr1	G-A	Overlaps hire1 without a separable dated hook	[ESB Ch.11 §4; ERD §1.6]
"a general AI governance product"	G-E	Fails on every deal, because a general governance product is definitionally outside the write path	[ESB Ch.11 §4; ERD §1.6; EBA]

The last row is the most instructive: the company rejected, by name, the product most of its market sells [ESB Ch.11 §4]. Each rejection carries a revisit trigger — energy1 becomes viable "the moment a grid regulator requires per-action evidence and a path exists for us to be in the write path rather than beside it" [ESB Ch.11 §8].

Admission is a governed write. The author writes the candidate on one page (domain, dated hook, buyer's title, auditor's title, write types elan1 would own), scores the five gates with a sentence each, and names the auditor by role and, where possible, organisation — "G-B is not satisfied by a category; it is satisfied by somebody who could be telephoned". Admission to the build queue is a governed write with `dual_control: true`, approved by the founder at the Decide screen; a candidate admitted by its own proposer is refused `GATE.DUAL_CONTROL_SAME_ACTOR`. A rejected candidate goes into the portfolio register by name with its failed gate — "not softened, parked or renamed". An admitted one enters the queue with a declaration owner, a profile author and a separate profile certifier named on the same day. If the named auditor later turns out not to exist, the pack returns to the register "whatever has been built by then" [E18 §03].

13.3 The pack definition template, field by field

"Every pack declares the same eleven things. A pack missing any of them is not a pack" [E18 §04].

#	Block	Contents	Constraint
1	`pack_identity`	`pack_id` (`^[a-z][a-z0-9]{2,15}1$`), version, owner role, the domain in one sentence a non-engineer can read	Owner is a role
2	`obligation`	Regime, issuer, date, and the clause set served	One regime per profile; a pack may carry several profiles for several jurisdictions
3	`buyer`	The job title that signs, the budget line it comes from, the auditor's job title	—
4	`gate_scores`	G-A to G-E with their evidence	Retained for ever, including for packs that later fail
5	`declaration_ref`	(`declaration_id`, version)	"The pack does not have its own ontology"
6	`write_types[]`	Each with blast radius, reversal plan, reversible or declared irreversible	Must exist in the referenced declaration
7	`gates[]`	Which of the nine pre-gates, in order, with parameters	The six never-skippable gates present by construction
8	`profile_binding`	(`profile_id`, `profile_version`), `rule_as_at`	Profile authored under E13, certified by someone who is not its author
9	`buyer_promise`	What the buyer receives, in the buyer's words, and in the same block what it does not establish	Both halves required
10	`tests[]`	T1–T8	§13.4
11	`pilot`	Named customer, named auditor, dates, the finding	Filled during the pilot

Source: [E18 §04].

Blocks 6 and 7 restate, for the pack's scope, what the referenced declaration already declares. The recommended rule is that they are *views* — the pack validator checks them against the declaration version and fails on any difference — rather than a second source **(build recommendation)**.

What a pack promises. Three sentences, and a pack may not ship until all three are true of it [E18 §04]:

1. "Every governed change of these types went through stated rules, with a named actor and, where required, a named approver."
2. "Here is a file covering a period you choose. Your auditor can check it on their own machine, offline, with software they did not get from us."
3. "Here is what that file does not establish, in the profile's own words, printed in every report."

The second sentence is only true at L2, which is why every pack's commercial life is coupled to L2 (§13.5.7).

A pack manifest sketch for `ledger1` (values from E18 §06; structure **(build recommendation)**):

```
pack_identity:
  pack_id: ledger1
  pack_version: 1
  owner_role: "elan1 lead"
  domain: "Changes to the books of account, and the evidence that each one passed
           stated rules."
obligation:
  regime: "Companies (Accounts) Rules — audit trail (edit log) for accounting software"
  issuer: "Ministry of Corporate Affairs"
  in_force_from: 2023-04-01
  clauses: [each_and_every_transaction, edit_log_of_each_change,
            date_of_each_change, not_disabled]
buyer:
  signs: "Promoter or finance controller of a ₹5–50 crore firm, on the CA's advice"
  budget_line: "Audit and compliance"
  auditor: "Statutory auditor"
gate_scores:
  G-A: {result: pass, evidence: "In force 1 April 2023"}
  G-B: {result: pass, evidence: "Statutory auditor, named in the pilot"}
  G-C: {result: pass, evidence: "Native cancel or compensating instrument; both declared"}
  G-D: {result: pass_provisional, evidence: "The gate the pilot tests"}
  G-E: {result: pass, evidence: "rung1 hosted mode; the single writer is the only path"}
declaration_ref: {declaration_id: "<rung1 ERPNext declaration>", version: "<at pilot start>"}
profile_binding:
  profile: [in-mca-audit-trail, 1]
  rule_as_at: 2023-04-01
  applies_to_records_from: 2027-07-01
  min_assurance_level: 2
  refusals_required: true
  continuity_scope: full_chain
approver_model:                       # build recommendation, 13.1
  approvers: ["Accounts head", "Owner"]
  dual_control: {journal_entry.submit: always, payment_entry.submit: always,
                 sales_invoice.cancel: always, sales_invoice.submit: "> 200000",
                 purchase_invoice.submit: "> 200000", customer.update: on_gstin_change}
  default_expiry_hours: 4             # asserted, not observed (E04 Q3)
tests: [T1, T2, T3, T4, T5, T6, T7, T8]
pilot: {customer: null, auditor: null, window_days: 90, finding: null}
```

`G-D: pass_provisional` is not a fourth disposition; it records E18's own words — "G-D pass, provisionally — this is the gate the pilot actually tests" — and the recommended schema would carry it as `pass` with a `not_measurable` evidence field whose `would_change_if` is the day-85 answer [E18 §06].

13.4 Pack testing and the evaluation set

A pack passes eight acceptance tests before it ships [E18 §04].

#	Test	Passes when
T1	Declaration validates	Seven blocks validate; the six never-skippable gates are present; no field is blank
T2	Profile certified	The bound profile passes P1–P12 and carries a certifier who is not its author
T3	Reversal is honest	Every write type's reversal plan has been executed once, in a drill, and the drill is dated — "a plan nobody has run is a plan that does not exist"

#	Test	Passes when
T4	Refusals are real	The fixture set contains at least one refusal per declared gate, each with a code, the rung, every gate's result and a named operator action
T5	Bundle verifies offline	A bundle for the pack's write types exits `0 PASS` on an air-gapped machine, run by someone outside the team
T6	Tampered bundle refuses	The published tampered variant exits `1 FAIL_INTEGRITY`, naming record and field
T7	Non-conforming bundle is not accused	A sound bundle with a missing required member exits `2 FAIL_PROFILE`, never 1
T8	The auditor reads it	The named auditor reads a report unprompted and can state what it does not prove; if not, the limits statement is wrong

"T8 is the one that fails first drafts," for the same reason the profile's second-reader test P11 does [E18 §04; E13 §06]. T7 matters for a reason beyond correctness: reporting a profile failure as an integrity failure "accuses a customer of tampering", and the reverse "hides a serious incident" [IM §5.3].

The **evaluation set** of a pack is therefore concrete: (a) the reference-tenant fixture set, with at least one real refusal per declared gate and every refusal code the regime can produce; (b) a known-good bundle and a known-tampered bundle; (c) a known-nonconforming bundle and an incomplete one; (d) the dated reversal drill records; and (e) the profile's own P1–P12 results, including P8 (the L1 dry run reads `not_applicable — assurance level 1`, never a silent PASS) [E18 §04; E13 §06]. The IM's phrase "evaluation set" is not a model-evaluation corpus; there is no model in the decision path to evaluate [IM §1.4; E04 §03].

The recommended CI for a pack repository runs T1, T4, T6 and T7 on every change, T2 whenever the profile changes, and T5 nightly on a network-isolated runner; T3 and T8 are human acts whose dated records are inputs to certification rather than CI steps **(build recommendation)**. Because a refusal is a governed outcome, the refusal fixtures are also the regression suite for the kernel's determinism property: every fixture refusal must replay identically at its `as_of` (conformance C4, eval EV3) [E02 §08; EBB EB-12].

13.5 ledger1 in full

`ledger1` is the first pack: "changes to the books of account, and the evidence that each one passed stated rules" [E18 §06].

13.5.1 The obligation

The proviso to Rule 3(1) of the Companies (Accounts) Rules, 2014, in force since 1 April 2023, requires every company using accounting software to use software that records an audit trail of each and every transaction, creates an edit log of each change made in the books of account with the date of the change, and ensures the audit trail cannot be disabled [R03 §06; E18 §06]. The IM states that the trail must be preserved for at least eight years, that the rule reaches about 1.85 million active companies, and that Rule 11(g) of the Companies (Audit and Auditors) Rules obliges the statutory auditor to report affirmatively whether the trail existed, operated throughout the year, was not tampered with and was preserved, with penalties of ₹50,000 to ₹5,00,000 and up to a year's imprisonment for wilful violation [IM §3.6; IM §10; IM §11 PUBLISHED]. R03 marks "the precise preservation period and the exact wording of the auditor's reporting clause" as needing counsel confirmation before launch [R03 §06]. In the IM's own phrasing, elan1's product "is a Rule 3(1)-compliant audit trail for writes made by non-human actors, with Rule 11(g)-grade tamper evidence" [IM §10]; the pack itself never asserts compliance (§13.5.4).

The pack binds four clauses, one row each, in profile `in-mca-audit-trail` v1 [E13 §07; E18 §06]:

Clause	Obligation kind	Binds write types	Binds gates	Binds fields	Checks
Audit trail of each and every transaction	`record_keeping`	`*`	—	F04, F05, F09, F10, F16	C01, C02
Edit log of each change made in books of account	`record_keeping`	`*`	`precondition`	F12, F13, F14, F07	C03

Clause	Obligation kind	Binds write types	Binds gates	Binds fields	Checks
Along with the date when such changes were made	`record_keeping`	`*`	—	F16	C04
The audit trail shall not be disabled	`non_repudiation`	`*`	—	F02, F03	C05, C06

The eight-year preservation requirement is not one of the four bound clauses: the profile has no row of `obligation_kind: retention` and no `retention_months` [E13 §07]. Preservation is therefore met, if at all, by elan1's retention of its own records and by the customer's accounting system, not by a check a verifier runs. The build consequences are set out in §13.5.4.

13.5.2 Buyer, auditor and gate scores

The buyer is the promoter or finance controller of a ₹5–50 crore firm, "usually on their CA's advice", paying from the audit and compliance line, not software. The auditor is the statutory auditor, who must report on whether the audit-trail feature operated throughout the year [E18 §06]. The gate scores: G-A pass (1 April 2023); G-B pass (statutory auditor, named in the pilot); G-C pass (ledger writes reverse natively or through a compensating instrument, both declared); G-D pass provisionally — "the gate the pilot actually tests"; G-E pass (rung1 hosted mode, where the single writer is the only path) [E18 §06].

MCA beat GST e-invoicing on G-E alone. Both pass G-A, G-B and G-D. For the MCA rule, the write elan1 governs is the change to the books of account, and in hosted mode there is no other path to it. For e-invoicing, the decisive artefact — the IRN — is issued by the government Invoice Registration Portal and reaches elan1 as a response; a pack built on it "would be reading somebody else's log for the part that matters" [E18 §05]. Two further reasons follow: the profile already existed, certified as the first profile; and the buyer and auditor are the two people the group already talks to, because rung1 sells through CAs [E18 §05].

13.5.3 Write types and gates

`declaration_ref` is rung1's ERPNext declaration at the version current at pilot start; "no new ontology" [E18 §06].

Write type	Blast radius	Reversal	Dual control
`sales_invoice.submit`	`one_counterparty`	`target_native_undo` — cancel the IRN and the invoice	Above ₹2 lakh
`sales_invoice.cancel`	`one_counterparty`	Irreversible after the 24-hour IRN window, and declared as such	Always
`purchase_invoice.submit`	`one_counterparty`	`target_native_undo`	Above ₹2 lakh
`payment_entry.submit`	`one_counterparty`	Compensating instrument	Always
`journal_entry.submit`	`batch`	Compensating instrument	Always
`customer.update` (credit terms, GSTIN)	`one_record`	`target_native_undo`	On GSTIN change

Source: [E18 §06; IM §5.4].

Gates run in declared order — consent, residency, retention, counterparty, precondition, dual control, budget. The first six are never skippable; `budget` is declared on "because a runaway journal entry is the failure mode the buyer imagines first" [E18 §06]. Whether `customer.update` belongs in an audit-trail pack at all, or the obligation reaches only transactions, is open for the profile author by 31 May 2027 [E18 §09 Q3].

Two declaration-level details need fixing before T1 can pass. `compensating_instrument` is not a value of `reversal_plan.method`, whose enum is `snapshot_restore · compensating_entry · target_native_undo` [E03 §02]; the pack should declare `compensating_entry`. And `sales_invoice.cancel` is described as reversible for 24 hours and

irreversible thereafter, which in E03's grammar is `reversal_plan` with `window_hours: 24` and `after_window: not_reversible` rather than `irreversible: true` — a distinction that matters, because `irreversible: true` bars auto-approval and a plan does not [E03 §02]. Both are recorded in the conflicts block.

The audit-trail state predicate that rung1 specifies as its gate G8 — the ERPNext audit-trail setting is on before and after the change, no changeset disables or narrows tracking, no migration updates or deletes version rows, and the rung1 change ledger is unbroken — is the natural parameter of the `precondition` gate for every `ledger1` write type [R03 §06]. The profile binds `precondition` to the edit-log clause for this reason [E13 §07].

13.5.4 The profile, the evidence and the auditor's report

The profile runs six checks over an exported bundle [E13 §07]:

Check	Kind	Asserts	On fail
C01	structural	Every `seals.jsonl` line parses to sixteen fields and F05 resolves in `declarations/`	`fail_profile`
C02	binding	F09 present and F10 in {human, agent, system} on every record	`fail_profile`
C03	binding	F12 non-zero and F13 ≥ 1 on every committed record; zero-UUID and 0 on every refusal	`fail_profile`
C04	temporal	F16 parses as RFC 3339 UTC and is non-decreasing with `seq`	`fail_profile`
C05	integrity	`seq` contiguous from the range start; every `prev_digest` matches	`fail_integrity`
C06	temporal	Every record covered by an anchor whose token time precedes now; unanchored head printed	`fail_integrity`

C06 is the row that decides the pack's schedule. A chain cut at the tail is internally consistent and looks complete; only an external time anchor makes the non-disablement clause checkable, which is why `min_assurance_level` is 2. On an L1 dry run C06 reads `not_applicable — assurance level 1` on the face of the report [E13 §07].

What the buyer receives is a bundle covering any period, containing every governed change to the books in that period including every refusal, each with actor, actor kind, target record, version written and platform-clock time; an unbroken sequence anchored by two timestamp authorities; a verification report the auditor produces themselves, offline; and every declaration version and lexicon referenced, verbatim [E18 §06]. **What it does not establish**, in the same breath and reproduced verbatim in every report: that the books are complete — "changes made directly in the accounting system, outside this write path, are recorded by that system's own audit trail and are not in this bundle" — that transactions were genuine, that the rules were adequate, or that any obligation is satisfied; "that determination belongs to the reader. … it has not been reviewed or accepted by the Ministry of Corporate Affairs or by any regulator" [E13 §07].

Auditor reporting. The pack gives the statutory auditor evidence for the Rule 11(g) questions within the write path's scope; it does not write the auditor's sentence. The completeness gap is closed by E11's test 3, the two-way reconciliation between the elan1 chain and ERPNext's own audit trail for the period: every change in the target for the governed record types should have an elan1 record and every committed elan1 record a change in the target, and "exceptions in the first direction are the finding that matters — they are changes that did not go through the gates" [E11 §07]. Whether the bundle should carry a member for the target system's own audit trail so that test 3 can run inside the verifier is open [E15, open question 5].

Preservation. Because the profile binds no retention clause, eight-year preservation must be engineered rather than verified. The chain, its anchors and every declaration version are already retained indefinitely, because "a bundle must be checkable in ten years" [E15 §04], and the pricing model costs evidence storage as a ten-year carry [E10 §07]; what can lapse is the *payload*, whose expiry or erasure reads "chain intact, payload unbound" [E11 §05]. This memorandum recommends three rules for any tenant bound to `ledger1` **(build recommendation)**: (a) a payload-retention floor of at least 96 months from `recorded_at`, set as a tenant property the tenant cannot lower while the pack is bound; (b) no payload salt destroyed or payload retention expired before that floor except under an erasure obligation that overrides it, in which case the request is refused in writing with the MCA obligation named and its expiry date, the reasoning the security baseline already uses [IM §10]; and (c) a `retention` clause row added to the Regulation object in Phase 4 with

`retention_months: 96` and disposition `not_measurable` for the verifier — with reason "a bundle cannot prove its own future preservation" and `would_change_if` "an external archive attests custody" — rather than silently omitted. The period itself awaits counsel confirmation [R03 §06].

13.5.5 ledger1 at runtime on rung1 Enterprise

Figure 19 shows `ledger1` running under rung1 Enterprise in hosted mode, where rung1 owns the ERPNext site, holds the only credential and is the only writer [R10 §01].

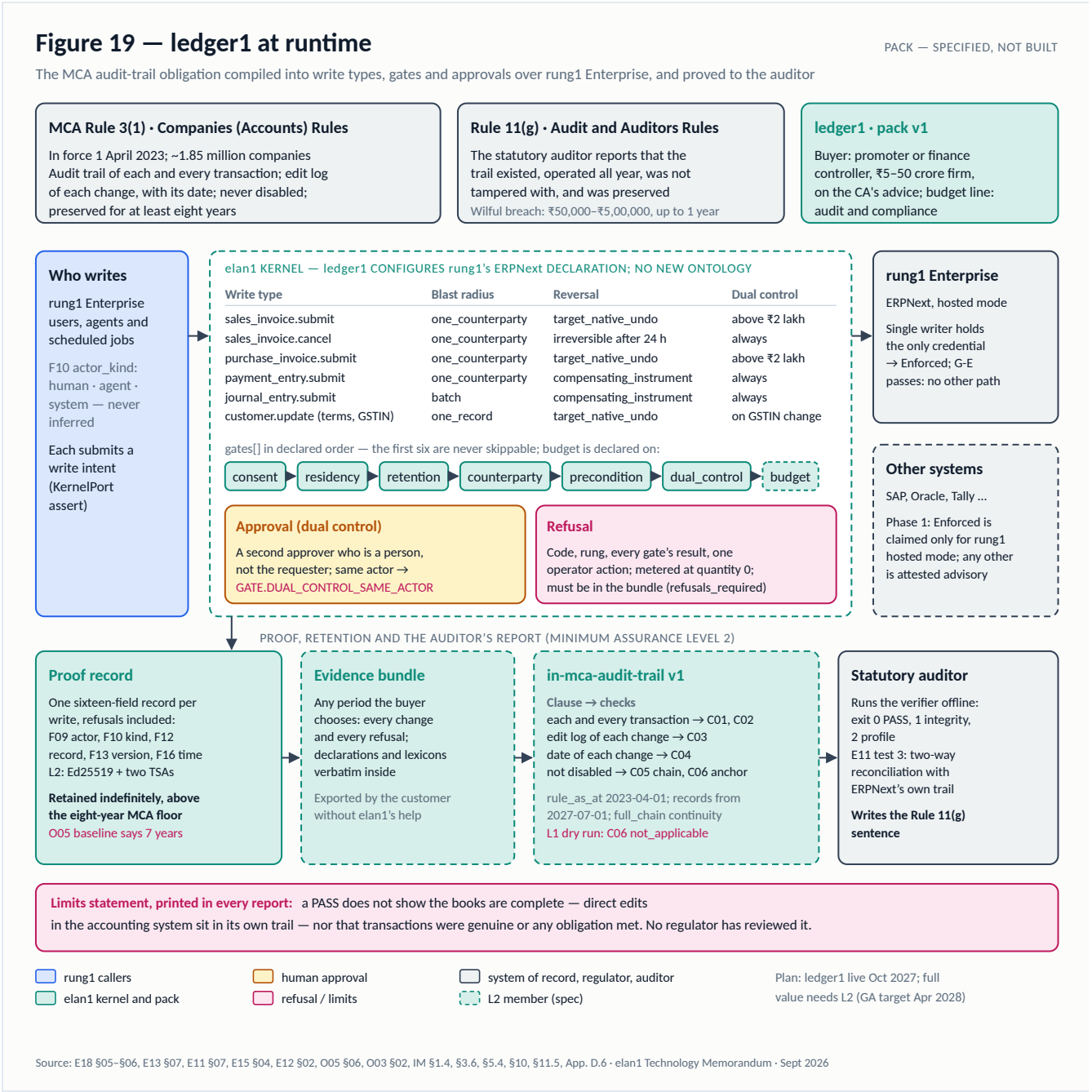


Figure 19 — ledger1 at runtime: a books-of-account write under rung1 Enterprise passes the nine rungs with the audit-trail precondition, is approved under dual control, is applied through Frappe's API by the single writer and hash-chained; at L2 the chain is signed, anchored and exported to the statutory auditor, who verifies offline and reconciles against ERPNext's own trail.

The sequence for a `journal_entry.submit` proposed by a rung1 agent:

1. rung1 compiles the request into a typed intent against `journal_entry.submit` at the pinned declaration version, with `actor_kind` set by the kernel, not rung1 [E04 §03; O06 §05].

2. Rungs 0b–2 resolve the tenant, the actor and the write type; rung 3 runs all seven declared gates, including the audit-trail `precondition` and `budget`, recording a gate-result row for each [E04 §03; E18 §06].
3. Rung 4 evaluates the rule set at a `policy_version`; rung 5 checks the measured radius is a declared `batch`; rung 6 escalates because dual control is "always" for journal entries and an agent at A2 cannot act alone [E04 §03; E18 §06].
4. A named human approves on the Decide screen; the approval is its own governed write; the intent is resubmitted (Chapter 8) [E04 Flow 3].
5. Rung 7 snapshots, applies to the tenant's sandbox, re-runs the gates, applies identical bytes to live through Frappe's own API — never the ORM — with a single-use credential scoped to one write and a ten-minute life, re-runs the gates on live, then commits the version and one outbox row in one transaction [E04 §03]. Every governed write that touches a customer's books runs inside a Temporal workflow, so a replay never re-runs a side effect [O01 DR-002].
6. Rung 8 appends the sixteen-field record; rung 9 meters one action on the `batch` basis; refusals at any rung meter at zero [E02 §03; E10 §03].

At L1 the tenant sees the Proof screen's unsigned sentence and a three-valued chain standing [E06 §05]. From the tenant's L1 → L2 upgrade, records are signed and anchored; records written before the boundary stay L1 and "are not signed retrospectively" [E11 §06; E19].

13.5.6 ledger1 on attached systems

In Attach mode rung1 governs a customer-run ERPNext through a connector using an API key the customer created; rung1 owns neither the machine nor the only credential, and "what changes in Attach mode is not who may allow; it is what an allow can guarantee about the system afterwards" [R10 §01–§02]. For `ledger1`, G-E passes for the writes rung1 makes and fails for everything else, because the customer's own users and integrations can still write to the books directly.

The pack remains saleable on an attached tenant only with three additions **(build recommendation, from R10 and E11)**: the report's scope line states that the tenant is attached and other writers exist; the two-way reconciliation of E11 test 3 runs on a schedule against the attached instance's version tables, and its exceptions are shown on the Assurance door rather than left to the auditor; and the limits statement's completeness sentence is printed first. Tally and Zoho Books are read-only migration sources for rung1 and cannot carry `ledger1` at all: elan1 is not in their write path [R10 §01; RTM Ch.1 §1.3].

13.5.7 L1 now, L2 for the pack, and the pilot

`ledger1` needs L2. Its profile's `min_assurance_level` is 2, `applies_to_records_from` is 2027-07-01 ("the earliest L2 record"), and T5 and T6 require a bundle and an offline verifier that do not exist at L1 [E13 §07; E18 §04]. The IM states the consequence plainly: the pack's full commercial life is coupled to L2 general availability in April 2028, even though the plan books first pack revenue from October 2027 [IM §5.4]. Before L2, `ledger1` can run as configuration on L1 tenants — gates, approvals, the chain and dry-run reports with C06 `not_applicable` — but cannot make its second promise.

The pilot is one customer, one auditor, ninety days, and its purpose is to find out whether G-D is real [E18 §07]:

Day	Step
0	The auditor's half day, before anything is shown: three questions they would ask, and a written statement of what they would *need* to rely on a record like this. Anything the profile cannot bind becomes a `not_measurable` row, never an omission
1	The tenant upgrades L1 → L2 under E19; the boundary date and counts are shown to customer and auditor
2–75	Normal operation — no curated data, no suppressed refusals
76	The customer exports the bundle without help; needing help is the finding
77	The auditor obtains the verifier from the published release or builds it from source, disconnects, runs it, then runs the tampered bundle and watches it fail. elan1 is not in the room
78	E11 test 3: two-way reconciliation against ERPNext's own trail
80	The auditor's written verdict, published internally verbatim even if unflattering

Day	Step
85	The G-D question: would you pay for this on its own line? The answer closes the pilot

Source: [E18 §07]. A `FAIL_INTEGRITY` during the pilot is a P0 and "stops the pack, not just the pilot" [E18 §07]. What is not asked for is recorded too: no endorsement, logo or quotation, and "a conversation with an auditor is never described, internally or externally, as validation" [E18 §07]. The pilot customer and auditor must both be named before Q3 2027 [E18 §09 Q1].

13.6 invoice1 — GST e-invoicing

GST e-invoicing "is not abandoned; it is second, and it is a different pack" [E18 §05]. The volume is not in doubt. Mandatory e-invoicing reaches firms above a ₹5 crore turnover threshold; firms above ₹10 crore must report to the portal within thirty days with no grace period; an invoice without a valid IRN has no standing under GST and the buyer cannot claim input tax credit; and 8,55,738 GSTINs generated e-invoices in FY2025-26, with 21.4 crore IRNs [IM §5.5 PUBLISHED; E18 §05]. "Each of those is a write with a statutory deadline, an irreversibility property and a financial consequence for a third party" [IM §5.5]. `invoice1` enters the plan in April 2028 [IM §5.5].

The pack's defining feature is what it says it does not cover. It "binds a profile over the write that requests an IRN and states plainly, in its limits statement, that the IRN, the portal response and the 24-hour cancellation window are the accounting system's evidence and not ours" [E18 §05; E18 §08 D6]. E11 draws the same line for any GST reader: the record shows "the write that produced the invoice was gated, approved where required", and "does not evidence the IRN, the portal response or the 24-hour cancellation window" [E11 §08]. G-E passes "partly": elan1 governs the request, not the clearance [E18 §05].

The corpus gives the pack's hook, boundary and limits but not its template. A sketch against the eleven blocks **(build recommendation throughout, except where cited)**:

Block	Proposed content
`obligation`	CGST e-invoicing mandate: IRN generation for in-scope B2B invoices; thirty-day reporting for firms above ₹10 crore; the 24-hour cancellation window [IM §5.5; E18 §05]
`buyer`	Finance controller of an in-scope firm; the auditor at year end and the GST officer at assessment [E18 §05]
`declaration_ref`	rung1's ERPNext declaration — the same one `ledger1` references; no new ontology
`write_types[]`	`sales_invoice.submit` as in `ledger1`; a separate `e_invoice.request_irn` of radius `cross_boundary`, because the payload leaves the tenant's domain for the IRP [R03 §07]; `e_invoice.cancel_irn` with a 24-hour window then `after_window: compensating_entry` (a credit note) [E03 §04]; `credit_note.submit`
`gates[]`	The six never-skippable gates; `precondition` carrying payload-schema validity against the IRP schema, HSN and place-of-supply checks (rung1's G4 evidence) and "IRN not already issued"; `schedule` on, parameterised by the thirty-day reporting deadline
`profile_binding`	A new profile, e.g. `in-gst-einvoice` v1, with `min_assurance_level: 2`; clauses bound to the request write, the cancellation write and the reporting deadline; the IRN itself bound as `not_measurable` with reason "issued and held by the IRP, outside the write path"
`buyer_promise`	Every IRN request and cancellation went through stated rules with a named actor and approver; the IRN and the portal's acceptance are not in the bundle and must be reconciled from the accounting system

Treating the IRP call as a separate `cross_boundary` write type follows rung1's gate specification, which lists "push an e-invoice payload to the IRP" as its example of a `cross_boundary` radius and notes that such writes require a proof binding and dual control, "which is why e-invoice submission stays a human action in Phase 1 rather than becoming a feature" [R03 §07]. E03's example instead folds the IRN request into `sales_invoice.submit` at `one_counterparty` [E03 §04]. The split is recommended because the two writes have different radii, different reversal windows and different evidence, and because the meter basis follows the radius [E10 §03]. The observed portal response enters the record only as a declared field of the request's outcome, never as something elan1 attests.

13.7 collect1 — RBI collections conduct

`collect1` is the third pack, planned for October 2028, and it shows what the template does to a hard domain [IM §5.5]. Its source is a spec sketch, and "the rule citations need counsel confirmation before anything ships" [E27].

13.7.1 Four obligation layers

Layer	Obligation
Conduct	RBI fair-practices rules on recovery: permitted calling hours, no harassment, no third-party disclosure, identified caller, grievance route
Telecom	DLT registration of the sender, approved headers and templates, preference-register scrub, correct number series, attempt limits
Data	DPDP lawful basis, purpose limitation, retention, erasure handling
Sector	The lender's own board-approved policy and outsourcing rules — evidence the lender can put in front of an inspection

Source: [E27 §01].

13.7.2 Write types

Write type	Reversibility in E27	Declaration form (recommended)
`call.attempt` — a dial is authorised	Irreversible: "a call cannot be un-made"	`irreversible: true`, radius `one_counterparty`
`call.record` — the sealed outcome record	Append-only	`irreversible: true` (L0-S03 admits no third value)
`promise.capture` — a promise-to-pay	Reversible, retractable with its own record	`reversal_plan: compensating_entry`
`disposition.set` — contactability, wrong number, dispute raised	Reversible	`target_native_undo`
`escalation.raise` — hand to a human agent	Reversible	`target_native_undo`
`preference.update` — borrower asks not to be called	"Irreversible by design — never undone by an agent"	`irreversible: true`; `permissions` grants no agent any write that reverses it

Source: [E27 §02]; the declaration column is **(build recommendation)** under E03's rules [E03 §02, §05].

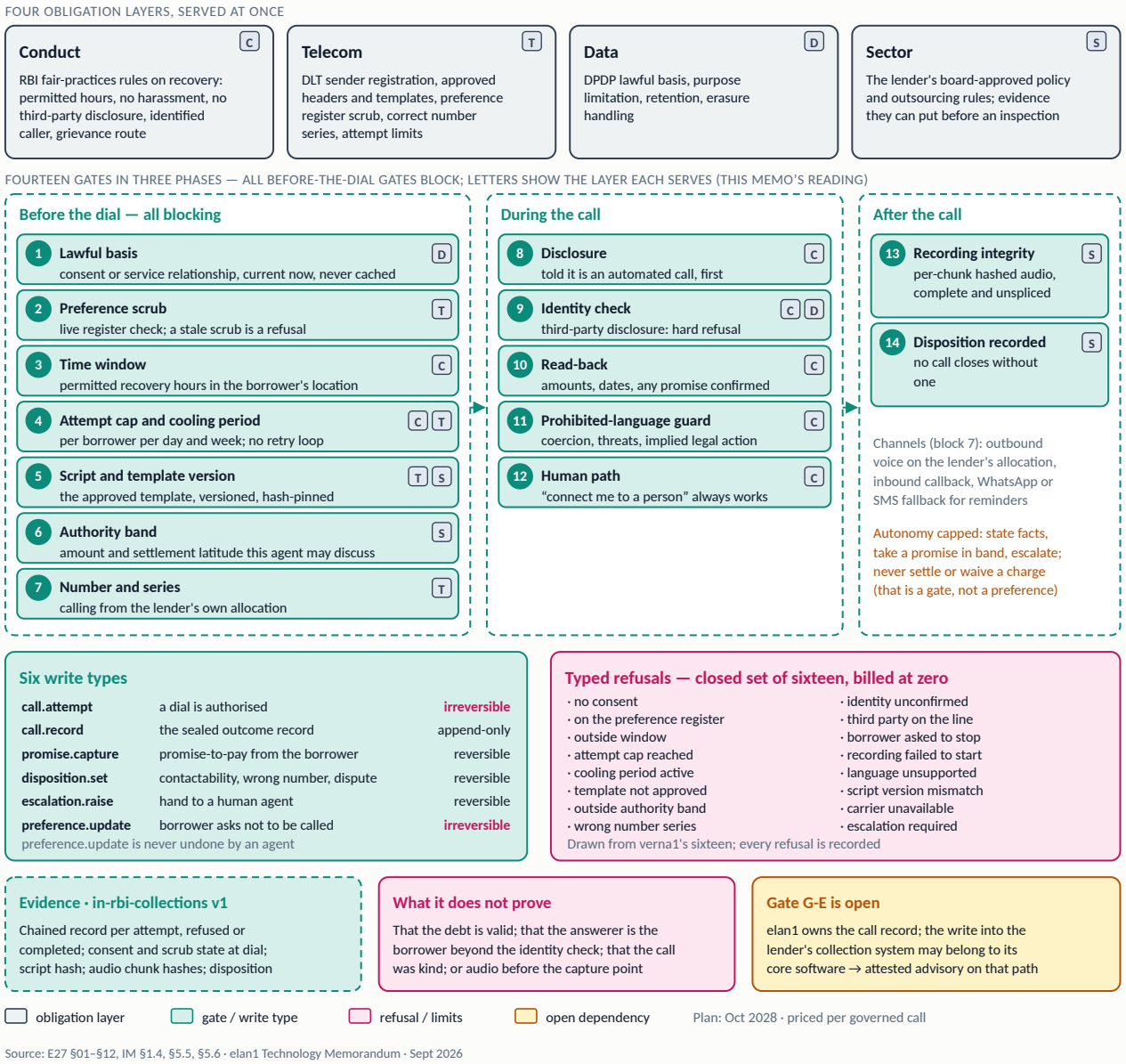
13.7.3 Fourteen gates on nine

E27 calls the gates "the part that is actually the product": fourteen, in three phases [E27 §03]. The kernel has nine pre-gates and the IM is explicit that a new regime "parameterises them rather than adding to them" [IM §5.6]. The fourteen therefore have to be expressed as gate parameters, rung 4 rules, write-type bounds and — for the during-call checks — preconditions on the writes that record the call's outcome. The mapping is not in the corpus; Figure 20 and the table below are this memorandum's proposal **(build recommendation)**.

Figure 20 — collect1: four obligation layers, fourteen gates

SPEC SKETCH — CITATIONS NEED COUNSEL

Every governed collections call passes seven gates before the dial, five during it and two after; a failure is a typed refusal



Source: E27 §01–§12, IM §1.4, §5.5, §5.6 · elan1 Technology Memorandum · Sept 2026

Figure 20 — collect1: four obligation layers (conduct, telecom, data, sector) and fourteen gates in three phases — seven before the dial, five during the call, two after — mapped onto the kernel's nine pre-gates, rung 4 rules and the preconditions of the writes that record the call.

#	collect1 gate (E27)	Phase	Kernel expression	Write type it guards
1	Lawful basis, current at this instant, not cached	Before	G1 `consent`, `as_of` = dial instant; consent ledger read by the adapter, evaluated by the gate	`call.attempt`
2	Preference scrub, live; a stale scrub is a refusal	Before	G9 `precondition` over declared fields `scrub_result`, `scrub_at`; freshness bound as a parameter	`call.attempt`
3	Permitted recovery hours in the borrower's location	Before	G8 `schedule`, enabled, window in borrower-local time	`call.attempt`
4	Attempt cap and cooling period, per day and per week	Before	G5 `rate`, per borrower, with a cooling parameter	`call.attempt`
5	Approved script and template, versioned, hash-pinned	Before	G9 `precondition`: template hash in the approved set at `as_of`	`call.attempt`
6	Authority band — amount and settlement latitude	Before	Rung 4 rule set plus `payload_schema` bounds; outside the band → `escalate`	`call.attempt`, `promise.capture`

#	collect1 gate (E27)	Phase	Kernel expression	Write type it guards
7	Number and series from the lender's own allocation	Before	G9 `precondition` over the calling number and its series	`call.attempt`
8	Disclosure that it is an automated call, first	During	Precondition on `call.record` and `promise.capture`: disclosure event present and first	`call.record`
9	Identity check; third-party disclosure a hard refusal	During	G9 `precondition` on `promise.capture` and `disposition.set`	`promise.capture`
10	Read-back of amounts, dates and any promise	During	G9 `precondition` on `promise.capture`: read-back confirmed	`promise.capture`
11	Prohibited-language guard	During	Precondition on `call.record`: guard result present and clean	`call.record`
12	Human path — "connect me to a person" always works	During	`escalation.raise` granted to the borrower-facing agent at all autonomy levels; never refused by `rate` or `budget`	`escalation.raise`
13	Recording integrity — per-chunk hashed audio, complete, unspliced	After	Chunk-hash list bound into `call.record` `proof_binding`; precondition: no gap	`call.record`
14	Disposition recorded — no call closes without one	After	Precondition on closing the call: a `disposition.set` exists for the attempt	`call.record`

G2 `residency`, G3 `retention`, G4 `dual_control` and G7 `counterparty` run on every write type in addition, because they cannot be skipped [E03 §02].

The mapping exposes a boundary the pack must state. The kernel is never in the audio path. Gates 8–12 are checks performed by the voice runtime during a live conversation; the kernel can only refuse the *writes that record their outcome* if the evidence is missing or bad. A call in which the disclosure was not played will be refused at `call.record` — and recorded as refused — but it will still have happened. That is the same distinction E27 draws when it says the pack proves the conversation "followed an approved script and contained no prohibited language", not that it "was kind" [E27 §08].

13.7.4 Refusals, autonomy and channels

E27's refusals are "drawn from verna1's existing sixteen": no consent, on the preference register, outside window, attempt cap reached, cooling period active, template not approved, outside authority band, wrong number series, identity unconfirmed, third party on the line, borrower asked to stop, recording failed to start, language unsupported, script version mismatch, carrier unavailable, escalation required [E27 §05]. The kernel's register is closed at 43 codes and has no such names [E02 §06]. Under the mapping above, several collapse into `GATE.PRECONDITION_FAILED`, which would defeat E27's own point that "the refusal rate [is] a compliance metric rather than a failure metric" [E27 §05]. The register permits new codes by decision — named protection, contract, rung, class, operator action, fixtures and a 30-day notice [E02 §06–§07]. This memorandum recommends a small set of gate-class codes for the collections conduct refusals the lender's inspector will count (preference register, attempt cap, template, third party) rather than one code per verna1 reason **(build recommendation)**. "Escalation required" is not a refusal in kernel terms; it is the `escalate` outcome [E02 §03]. "Carrier unavailable" is `PLATFORM.TARGET_UNREACHABLE`, which pages and is never billed [E02 §06].

Autonomy is capped by declaration: the agent "may state facts, take a promise inside a declared band and escalate. It may never negotiate a settlement, waive a charge, or agree anything outside the band — those escalate to a human, which is a gate, not a preference" [E27 §09]. Channels are declared in block 7: outbound voice through a carrier partner on the lender's allocation, inbound callback on the same number, and WhatsApp or SMS for reminders that never needed a call, "so a write type that may not travel by voice simply cannot" [E27 §04].

The irreversibility problem. `call.attempt` is irreversible, and under E03 `irreversible: true` "bars auto-approval"; under E05 "no reversal plan means no A3 or above"; and under E01 irreversible write types "can never be auto-approved" [E03 §02; E05 §11; E01 §02]. Read literally, every automated dial would need a human approval, which is incompatible with an automated collections agent. The corpus does not resolve this. This memorandum recommends a batch-radius, dual-controlled `dial_list.approve` write — reversible until its first dial — whose committed record enumerates the borrowers, window, template and band; each `call.attempt` then carries a precondition that it belongs to an approved, unexpired dial

list, so the human approval precedes every irreversible act while the dial itself runs without a per-call approval **(build recommendation)**. Whether that counts as "approval" for an irreversible write type is a founder decision that must be recorded before `collect1` is built (conflicts block).

13.7.5 Evidence, limits, pricing and pilot

The buyer receives a call evidence bundle, checkable offline: chained records for every attempt, refused and completed; consent state and scrub result "at the moment of dial, not reconstructed afterwards"; the window and cap check that authorised the attempt; the script and template version, hash-pinned; per-chunk audio hashes and the disposition; the verifier report and exit code; and a limits page [E27 §06]. The verifier profile `in-rbi-collections` v1 binds clause → control → gate → write type → proof, "so an inspector can walk from a rule to the record that satisfies it", and is versioned so a rule change creates a new profile rather than reinterpreting old records [E27 §07]. What it does not prove is written into the pack: that the debt is valid; that the person who answered is the borrower beyond the identity check performed; that the conversation was kind; and that the audio is authentic before elan1's capture point [E27 §08].

Pricing is per governed call, refusals at zero, evidence sold separately per regime — G-D, "what stops this becoming a voice-minutes business competing with a ₹2.80 resolved call" [E27 §10]. The pilot is one lender, one early-bucket book, one quarter, three languages, on the lender's own number allocation, "with an auditor login from day one and cost per governed call published weekly beside their human baseline" [E27 §11].

Two dependencies decide whether the pack can exist [E27 §12]. A carrier partner who holds the carriage and whose compliance plumbing elan1 sits on. And G-E: elan1 owns the call record, but "the write into the lender's collection system may belong to their core software. If it does, this pack is Attested advisory on that path, not Enforced — and the pack must say so rather than blur it." A third follows from the perimeter: outbound calling is verna1's regulated business, verna1 is parked until a paid NBFC pilot with a signed order and a named carrier partner, and rung1 Voice is inbound-only and shares no code path with it [O01 DR-007; R19 §01]. `collect1` is an elan1 pack over a voice runtime elan1 does not build; its October 2028 date assumes a runtime that is not yet funded.

13.8 The pack factory

"A pack is a repeatable industrial artefact and the plan treats it as one" [IM §5.6].

	Content	Source
Inputs	A regulatory engineer who can read an instrument clause by clause; a declaration owner; a profile author; and — named the same day, never the same person — a profile certifier	[IM §5.6; E18 §03]
Outputs	A declaration version; a certified profile version; a fixture set with a real refusal per gate; a dated reversal drill per write type; a tampered bundle; a limits statement a second reader can restate	[IM §5.6]
Steps	Name the hook and the auditor; write the declaration (`write_types` is the hard part); write the verifier profile; write the export mapping; simulate against the reference tenant; certify	[ESB Ch.11 §6]
Cost	The reading and the binding: enumerate the clauses, quote each verbatim, resolve every name against a certified declaration, write at least one executable check per clause, draft a limits statement "neither generic nor copied", dry-run good and tampered bundles, obtain a second human's certification — person-weeks, not kernel releases	[IM §5.6]

Staffing and money (all MODELLED). Pack and regulatory engineering rises from 2 people at March 2027 to 11, 23, 37 and 50 at successive financial year ends — by March 2031 the largest engineering group, above kernel engineering at 38. The regulatory pack revenue line runs ₹0.06, ₹0.83, ₹3.24 and ₹7.63 crore across FY2027-28 to FY2030-31, at a blended list price of ₹1,35,000 per tenant per year, with verifier profiles priced at ₹3,00,000 per regime per year. That line is small; packs pay for themselves by converting tenants from L1 to L2, by making the evidence SKU saleable at all (₹17.84 crore by FY2030-31), and by raising governed-action volume [IM §5.6].

The bottleneck is certification, not generation. The Strategy Blueprint caps concurrent declarations at about three "at any team size … because the binding constraint is review capacity, not generation": a declaration takes a competent reviewer about half a day to read, plus a simulation review and the certification act, over three to five iterations, and only two or three people are qualified to certify [ESB Ch.16 §3 (M)]. E18 asks the same question of profiles — how many packs can be

in review at once with one certifier who cannot be an author — and whether a partner may author a pack, "and if so what stops a bad pack carrying our name" [E18 §09 Q4–Q5]. "Those two decide whether the factory runs at two packs a year or ten, and neither is answered" [IM §5.6]. The cadence is deliberately slow for that reason: one pack in October 2027, a second in April 2028, a third in October 2028 [IM §5.6]. The Blueprint's sequencing rule adds a hard dependency between packs: "no pack ships before an auditor has verified a bundle from the previous pack" [ESB Ch.11 §7].

The recommended factory cadence is a quarterly intake review at which candidates are scored and admitted (each admission a governed write), with no more packs in build than the certifier count supports and the certifier bench grown by apprenticeship, since "it grows by apprenticeship rather than by hiring" [ESB Ch.16 §3] **(build recommendation)**. Once a pack is live, its maintenance is Flow 2 of E21: a clause change creates a new Regulation version, classifies the change, versions the affected controls and profile, and judges each record only by the profile version whose window covers its `recorded_at` [E21 §07; Chapter 7 §7.10].

13.9 A regime is a pack; a country is a set of packs

Verifier profiles are jurisdiction-prefixed by construction (`in-mca-audit-trail`, `in-rbi-collections`) and versioned against the date of the regime text [E13 §02; IM §5.7]. A pack is one domain and may carry several jurisdictional profiles under the same declaration: `ledger1`'s first shipped binding is India's MCA rule, and the US SEC Rule 17a-4 audit-trail alternative from earlier portfolio material "remains a valid second profile under the same pack" [E18 §06, §08 D7]. Entering a new country "is therefore not a re-platforming exercise and not a new codebase; it is a set of clause bindings against write types that already exist" [IM §5.7].

The regimes that could populate such sets are dated and public — among them EU AI Act Article 12 logging (standalone high-risk obligations from 2 December 2027), Colorado SB 26-189 (1 January 2027), the DPDP Rules (main obligations from 13 May 2027), RBI's FREE-AI framework and draft model-risk guidance, SEBI's AI/ML logging guidance and an IRDAI working group constituted 18 June 2026 [IM §5.7 PUBLISHED]. The discipline applied to that list is the part to underwrite: "a strategy that assumes a date is a strategy with a fuse in it", and two reserved pack names — telecommunications and government — are marked demand-led because no India-specific, agent-directed rule with a firm date was found [IM §5.7]. Expansion is carried by the pack factory and the partner channel rather than country subsidiaries, against sovereign and on-premises deployments from July 2029 (Chapter 17) [IM §5.7].

The earlier portfolio of ten packs — claim1, ledger1 (then against SEC 17a-4), carbon1, passport1, gxp1, trace1, hire1, close1, invoice1 and health1 — predates the India-first sequencing and is superseded where it conflicts with E18 [ESB Ch.11 §3; ERD §1.6; E18 §08 D5].

13.10 Packaging, versioning, signing and distribution

The corpus says what a pack is and how it is admitted and tested; it does not specify how a pack is packaged or delivered to a tenant. Everything in this section is **(build recommendation)** unless cited, constrained by three sourced facts: the kernel does not know what a pack is and the registry has no pack table [E18 §01]; every declaration, profile, control and Regulation version is immutable, integer-versioned and retained for ever [E03 §07; E13 §05; E21 §07]; and certification is a governed write under dual control by someone who is not the author [E21 §10 D8].

Packaging. A pack version is a manifest (the §13.3 sketch) whose members are referenced by identifier, version and digest, never embedded by copy: the declaration (`declaration_id`, version, `compiled_digest`), the policy rule set (`policy_version`), the profile (`profile_id`, version, digest), from Phase 4 the Regulation and Controls, the export mapping, the fixture set digest, the tampered and non-conforming bundle digests, the dated drill records, the T1–T8 results and the `gate_scores` record's `seq`. The manifest's own digest is computed over its canonical JSON form, so a pack version is content-addressed like everything it references.

Versioning. `pack_version` is an integer, last + 1, immutable once certified. Any change to any member's version is a new pack version; a pack version never floats to a member's latest. The pack's version is recorded by reference, not in the sixteen-field record: a record already names its declaration and policy versions (F06–F08), and the bundle names the profile

it claims, which is sufficient to reconstruct the pack's configuration at the time of the write without a seventeenth field [E02 §03; IM §5.6].

Signing. At L1 there is no signing key [O01 DR-006]; the certification governed write, hash-chained in elan1's own tenant, is the authority for a pack version. From L2, the recommendation is that elan1 sign each certified pack manifest with a release key separate from the record-sealing keys, under the custody rules of Chapter 12, and publish manifests alongside the verifier release so that a sovereign or air-gapped deployment can check a pack's integrity offline.

Distribution and binding. Packs are published through the declaration and profile registries (`/v1/declarations`, `/v1/bundles` profiles) rather than a separate channel [E05 §04; EBB EB-06]. A tenant *binds* a pack by a governed write under dual control that pins the declaration version, the policy version and the profile version; the binding is refused if the profile's `min_assurance_level` exceeds the tenant's level, with the shortfall printed and the upgrade runbook offered — L0-R05's behaviour applied at the binding [E21 §05, §07]. Unbinding and moving to a new pack version are governed writes of the same kind. A withdrawn pack version stops accepting new bindings; records written under it continue to be judged by the profile version whose window covers them, and are never re-judged [E21 §07].

Partners. Whether a partner may author a pack is open [E18 §09 Q5; E13 §09 Q2]. If partner authoring is allowed, the recommendation is that a partner may author but never certify, that certification stays with a named elan1 certifier, and that the pack manifest carry the author's organisation so that a report reader can see whose reading of the regime they are relying on.

13.11 What is in Level 1, and what comes later

Item	L1 (building now)	Later
Pack gates, admission flow, portfolio register	Yes — admission is a governed write	—
`ledger1` configuration: declaration, gates, approvals, chain	Yes, under rung1 Enterprise hosted	—
`ledger1` dry-run reports	C06 reads `not_applicable — assurance level 1`	Full profile at L2
Bundle export, offline verification, T5–T7	No	L2 (GA targeted April 2028)
`ledger1` pilot	Requires the tenant at L2	Pilot customer and auditor named before Q3 2027
`invoice1`	No	April 2028
`collect1`	No	October 2028; counsel confirmation, carrier partner and verna1 runtime required
Regulation and Controls objects for packs	No	Phase 4
Signed pack manifests	No signing key	From L2 (build recommendation)

Sources: [E13 §07; E18 §04, §07, §09; E27; IM §5.4–§5.6; E21 §01].

14. Level 3 — cross-organisation verification, selective disclosure and earned autonomy

Level 3 Proven is the top of the assurance ladder, and it is a frozen specification rather than a build programme. The scope freeze [E20] and three technical specifications — Cross-Organisation Verification [E22], Selective Disclosure [E23] and Precedent and Earned Autonomy [E24] — were all written on 21 September 2026, and all are in "draft for review" status. The IM summarises them in §8 [IM §8]. This chapter explains how each capability works, what it changes and what it leaves untouched in the Level 1 and Level 2 machinery, what the build depends on, and when the build may start. The rule that governs the chapter is the entry condition in §14.1.2: nothing described here may be staffed until an outside auditor has accepted an L2 evidence bundle in writing.

> **Designed is not shipped (DR-010).** No L3 capability exists in code. L2, on which all of L3 depends, is itself a frozen specification with general availability targeted for April 2028. The IM models L3 entry for October 2029 [IM §8.1]. Every threshold, interval and volume in this chapter is a specification value or a target, not a measurement.

14.1 Scope freeze and the entry gate

14.1.1 The frozen sentence

> **The frozen sentence.** "At L3 an elan1 action receipt is defined by a public specification, produced by two independent implementations that agree octet for octet, signed by a key the customer may hold themselves, anchored by parties that are not us, verifiable by an organisation that has no relationship with us, disclosable field by field without revealing the payload, and usable after this company has ceased to exist. It still does not prove the transaction was genuine, it still does not prove the rules were adequate, and no regulator has approved it" [E20 §01].

L3 ships six capabilities and the open standard, and nothing else [E20 decision 2].

#	Capability	What it is	Specification	Covered in
1	Cross-organisation verification	A protocol between two organisations: counterparty identity and key discovery, trust anchors neither party owns, a receipt exchanged and accepted, one chain's record bound to another's	[E22]	§14.2
2	Selective disclosure	Proving a fact about a payload without revealing the payload, from one bundle, to different recipients	[E23]	§14.3
3	Customer-held keys	The customer's own HSM signs their own records	[E23 custody notes; E16 successor]	Chapter 12
4	Sovereign deployment	Kernel and evidence plane inside a customer's or government's boundary, operated by them; a deployment model, not a proof capability	Deferred specification	Chapter 17
5	Precedent intelligence	Reading across a tenant's own chain: "this write resembles forty earlier writes, three of which were refused"	[E24]	§14.4
6	Earned autonomy	An action class graduating from A2 to A3 on recorded evidence, reviewable and always revocable	[E24]	§14.4
—	The Action Receipt standard	The public specification, the reference verifier's licence and the governance of the standard	[E25]	Chapter 15

Source: [E20 §03].

Three conditions carry forward from E07's L3 definition. They are conditions, not features: a second independent seal implementation kept in agreement with the first; published test vectors including a known-tampered bundle; and counter-signature by a party that is not elan1 [E20 §03; E07].

Two things do not change. **The sixteen fields do not change.** There is no seventeenth field, at L3 or ever. Cross-organisation binding and selective disclosure both go through F14 (the payload commitment), "which is why E22 and E23 were designable at all" [E20 §03; E20 decision 3]. **The decision path does not change.** No model becomes reachable at rungs 3, 4, 5, 6 or 8 [E20 §03].

14.1.2 The entry condition

> **Entry condition.** "No L3 capability is designed, staffed or promised until an auditor who does not work for us and is not paid by us has accepted an L2 evidence bundle as evidence in a real engagement, and has said so in writing we may keep" [E20 §02; E20 decision 1].

"Accepted" is measured. All four tests must be true [E20 §02]:

#	Test	Evidence
1	The auditor ran the offline verifier themselves, on their own machine	Their run, their exit code, dated
2	They relied on the bundle for a working paper or a finding, not merely looked at it	The reference in their file, or a written statement
3	They can state, unprompted, what a PASS does not establish	E18 test T8, applied to a person outside the design-partner relationship
4	They would ask for one again	In writing

If Gate 3 (the evidence-contract gate at the end of Q4 2028) closes without this, L3 is not started, and elan1 "remains an L2 business with a kernel, a pack and a smaller ambition. That is a real outcome and not a failure of nerve" [E20 §02; E20 decision 8]. The Series A carries the same logic as capital discipline. The third tranche, Rs 45.0 crore at Gate B in July 2029, is released on an external auditor's written acceptance of an L2 bundle, and L3 entry is modelled for October 2029, three months later. An investor's exposure to the L3 thesis is therefore the third tranche, not the round [IM §8.1].

> **The specifications already exist.** E20 forbids L3 work before the entry condition "including design work, including a slide" [E20 §08], and the working-draft briefs say each L3 specification is "only started once L2 evidence has been accepted outside the company" [WD]. E22, E23 and E24 were nevertheless written in September 2026, and their open questions are dated Q1–Q3 2028, before Gate 3 can close. This memorandum treats the three documents as frozen specifications written to fix the L2 design choices that L3 depends on — the sixteen fields, F14, the separation of verifier failure classes. It treats no further L3 design, staffing or roadmap commitment as permitted before the entry condition. The open-question dates need re-basing to follow entry (see the conflicts block).

14.1.3 Who buys L3, and what it adds commercially

Six buyers are named. The counterparty pair pays for cross-organisation verification. The regulated enterprise with its own key policy pays for customer-held keys, which E20 calls "a procurement blocker at large institutions, not a preference". The sovereign buyer pays for a deployment. The privacy-constrained discloser pays for selective disclosure. The operations owner who has outgrown approvals pays for earned autonomy, the only L3 capability that reduces daily labour rather than strengthening a claim. And the market gets the standard, which "nobody pays for" and which makes the other five sellable.

Source: [E20 §04]. L3 adds no new meter. Earned autonomy raises governed-write volume, which raises the platform licence. Customer-held keys and sovereign deployment are deployment SKUs sold by deal. Cross-organisation verification is the only new commercial object, and whether it is priced per receipt, per counterparty relationship or per year is open until Q2 2028 [E20 §04; E20 decision 5; E20 §11 Q2]. The IM's price book carries a counterparty verification seat at Rs 45,000 a year [IM §7.9].

14.1.4 The claim boundary

Becomes true at L3, not a day earlier	Remains false at L3
"The receipt format is published … a stranger has implemented it, and their implementation agrees with ours on every vector"	"Tamper-proof" — banned at every level, forever
"Two implementations written by two teams produce identical bytes for the same event"	"Court-ready" / "legally admissible"
"You hold the signing key. We can show you every record we signed, and it is none of yours"	"Approved by a regulator"
"A counterparty who has never heard of us can verify a receipt you gave them, against a trust anchor neither of us publishes"	"Certified" — unless a dated ISO/IEC 42001 or SOC 2 report exists, and then only for its scope
"You can disclose three fields to your auditor and five to your lender, from one bundle"	"The transaction was genuine" / "Your books are verified"
"This agent reached A3 on its own record of 900 approvals and 4 refusals … and one person can take it back in a minute"	"The agent is safe because it earned A3" / "We cannot see your data" / "Standardised"

Source: [E20 §05].

14.1.5 Definition of done

L3 is done when twelve measurements hold [E20 §07].

#	Done means	Evidence	Class
1	Entry condition met	Auditor's written statement, four tests	Blocks
2	A second seal implementation, by a team with no access to the first's source, agrees octet for octet on all 64 vectors and on a full production chain	Empty diff, disagreement log published	Blocks
3	Action Receipt specification published under terms that let a competitor implement it, with vectors and a tampered bundle	Artefacts and licence text	Blocks
4	One implementation by another organisation passes the conformance suite	Their name and result, or an honest statement that nobody did	Decides "open"
5	Two organisations exchange, accept and verify a receipt in production, each binding it into its own chain	Two chains, two records, two verifier runs	Decides "cross-organisation"
6	A redacted bundle verifies, and a withheld field is reported as not disclosed rather than missing	E23 flows, run by someone outside the team	—
7	One tenant signs with a key elan1 has never held	Key ceremony record; register shows no entry	—
8	A non-elan1 counter-signature is in production under a commercial arrangement	Contract and counter-signed roots	Decides "cross-organisation"
9	An action class has graduated A2 → A3 on recorded evidence, and a separate one has been revoked	Both governed writes	Protects model boundary
10	No model reachable from rungs 3, 4, 5, 6 or 8; the precedent score is replayable from the chain alone	Import-graph check in CI; replay producing identical score	Protects model boundary
11	A regulator-facing reviewer has checked a receipt without elan1's help	Reviewer's own account (E26)	Keeps claim honest
12	Every claim in L3 sales material maps to the claim boundary	Founder review	Keeps claim honest

14.2 Cross-organisation verification

14.2.1 Third-party verification is not cross-organisation verification

E14 split a phrase that had been doing two jobs [E14 decision 2; E22 §01]:

- **Third-party verification** is a bundle handed over and checked by its recipient. It is L2, works under E15, and needs no protocol: "the recipient gets a file, the verifier and a key fingerprint from their own source."
- **Cross-organisation verification** is a protocol between two organisations: counterparty identity, key discovery, trust anchors neither party owns, a receipt exchanged in the course of business, and one chain's record bound to another's. It is L3.

"The difference is not cryptographic. It is that in the first case a human carries the artefact and supplies the trust root by hand, and in the second a system does it repeatedly, at transaction speed, between parties who have never spoken to our sales team" [E22 §01].

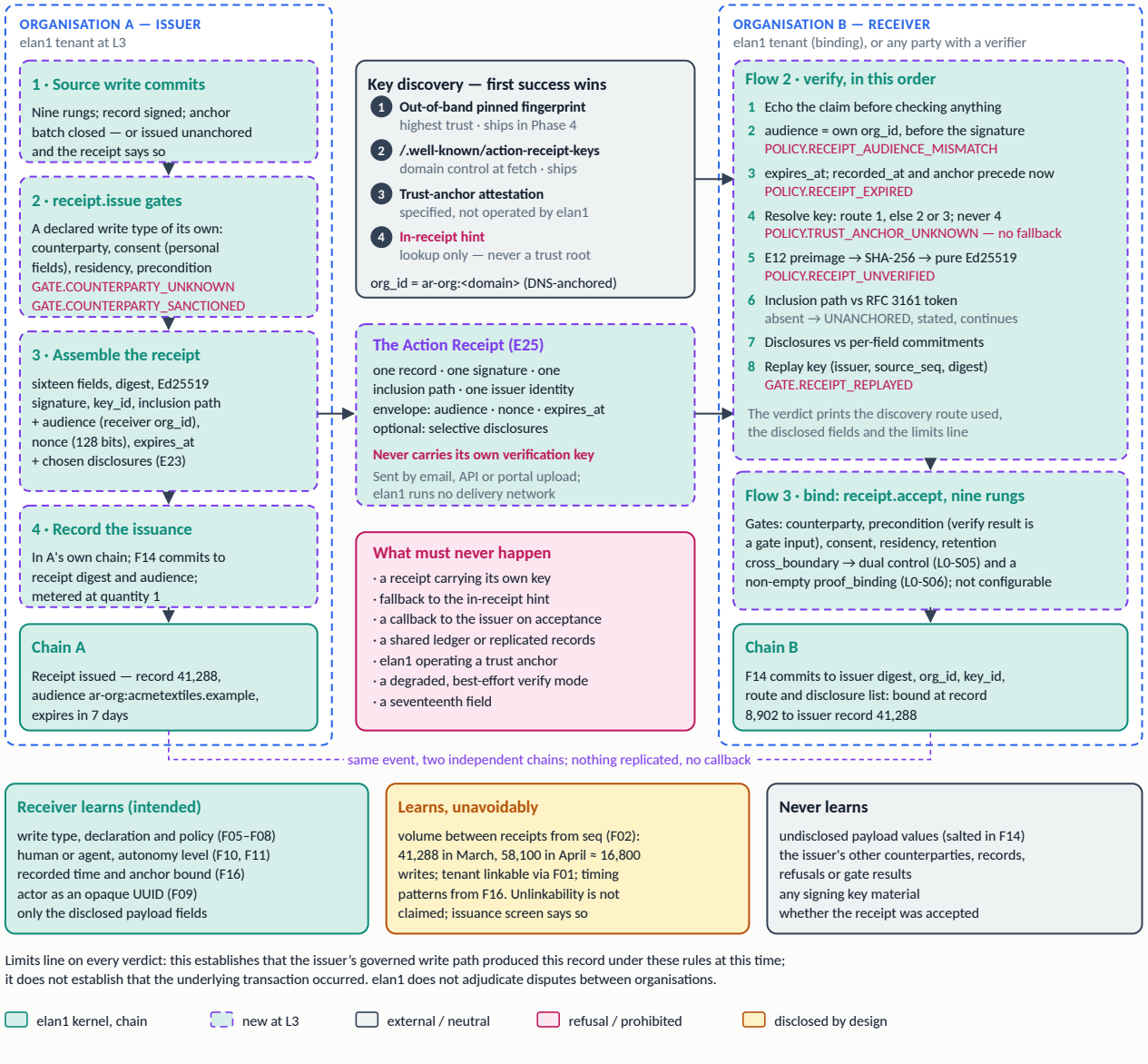
> **The receipt is the unit that crosses.** One record, one signature, one anchor inclusion path, one issuer identity, and, where selective disclosure applies, the disclosures the issuer chose. Every organisation keeps its own chain and nothing is replicated. "If the design ever requires a party to write into another party's chain, it is wrong." A shared ledger, a consortium chain and any network elan1 operates are out of scope [E22 §01].

Figure 21 shows the two organisations, their separate chains, the receipt crossing between them and the key-discovery routes.

Figure 21 — Cross-organisation verification

L3 — SPECIFIED, NOT BUILT

One receipt crosses the boundary, is checked against a key it never carries, and is bound into the receiver's chain via F14



Source: E22 §01–§12, E20 §01–§03, E12 §02, E25 · elan1 Technology Memorandum · Sept 2026

Figure 21 — Cross-organisation verification: the issuer's governed write and receipt.issue record, the receipt (sixteen fields, signature, inclusion path, audience, nonce, expiry) travelling by the business's own channel, key discovery by pinned fingerprint, issuer key document or trust anchor, offline verification, and the receiver's receipt.accept record binding the issuer's digest through F14.

14.2.2 The parties and objects

Object	Held by	Contents	Note
Receipt	Issuer, then receiver	Sixteen fields, digest, signature, `key_id`, anchor inclusion path, issuer identity, audience, nonce, expiry	Format defined by E25; E22 adds only the three envelope fields
Counterparty register entry	Each organisation, separately	`org_id`, display name, trust-anchor route, accepted `key_id` set with windows, first-seen, status	Exists at L1 as the counterparty gate's register; L3 adds key and anchor columns
Trust anchor	Neither party	"Organisation X's signing keys, as at time T, are these fingerprints", published by a disinterested party	"The load-bearing and least-solved part of this design"
Binding record	Receiver	A governed write of type `receipt.accept` committing to the receipt digest through F14	How one chain binds to another without a seventeenth field

Source: [E22 §02].

14.2.3 Identity and key discovery

`org_id` is DNS-anchored: `ar-org:<domain>`, for example `ar-org:acmetextiles.example`. Domains are already how organisations identify themselves commercially, resolution exists everywhere, and revocation has a natural home, "because an organisation that loses its domain has a bigger problem than a receipt" [E22 §03; E22 decision 1]. Discovery tries four routes in strict order, and the first success wins [E22 §03]:

#	Route	Mechanism	Trust it carries
1	Out-of-band fingerprint	Given by a person, in a contract or an existing relationship, and pinned	Highest; the L2 model and still the strongest
2	Issuer-published key document	`https://<domain>/.well-known/action-receipt-keys`: `key_id`, fingerprint, `valid_from`, `valid_to`, `status`, `successor_id`, over TLS	Moderate; proves control of the domain at fetch time
3	Trust-anchor attestation	A third party publishes a signed, timestamped statement binding `org_id` to fingerprints	Highest available at scale; only as good as the anchor
4	In-receipt hint	The receipt names issuer and `key_id`	None; used to look up, never to trust

> **A receipt never carries its own verification key.** A self-supplied key would let the issuer of a false receipt also supply the means of validating it. Route 4 is a lookup hint and the verifier must be incapable of treating it as a key [E22 §03]. If key discovery fails, there is no fallback to the hint "at any timeout", and no degraded mode [E22 §06, §12].

A key document is the issuer speaking about itself over a channel "an attacker with a certificate could occupy". It is acceptable where the loss from a bad receipt is bounded and the relationship is ongoing, and not for a first transaction of material size. The receiver's register records the route per receipt and renders it: "Accepted on a pinned fingerprint since March 2028" reads differently from "Accepted on a key document fetched four minutes ago" [E22 §03; E22 decision 4].

14.2.4 Trust anchors

Candidate	Assessment
elan1 publishes its customers' keys	Rejected. A vendor-operated trust root is what L3 exists to remove, and would make elan1 a single point of failure for transactions it is not party to
A transparency log operated by someone else	The strongest technical fit and closest to SCITT; depends on a log existing that is not elan1's
An existing PKI or sectoral registry	Realistic in one jurisdiction and sector; each is a negotiation
Bilateral pinning only	Works, scales badly; the honest starting state

Source: [E22 §04]. Phase 4 ships routes 1 and 2 and specifies route 3 so that an anchor can be added without a format change. elan1 does not operate a trust anchor, does not launch a registry, and does not gate the protocol on one existing [E22 §04; E22 decision 3]. Whether any usable anchor will exist in India by 2029 is open until Q2 2028 [E22 §14 Q1].

14.2.5 The receipt envelope

The receipt format is E25's (Chapter 15). E22 adds three envelope fields: `audience` (the receiver's `org_id`), `nonce` (128 random bits) and `expires_at` (a bounded window the issuer chooses) [E22 §02, §05]. The envelope is not in the record digest. It is signed separately, over `LP(digest) ‖ LP(audience) ‖ LP(nonce) ‖ LP(expires_at)`, as an envelope signature distinct from the record signature. The record's digest is therefore identical whether or not it was ever placed in a receipt, and one record can be issued to two audiences without two records existing [E25 §02; E25 decision 3]. The member names below are E25's; the JSON serialisation and the `F01…F16` key spelling are a **build recommendation**.

```
{
  "format": "action-receipt/1",
  "seal_suite": "elan1-seal-v6",
  "sealed_fields": {"F01": "<tenant uuid>", "F02": 41288, "F05": "sales_invoice.submit",
                    "F10": "agent", "F11": 3, "F14": "<payload commitment, 64 hex>",
```

```
                    "F16": "2028-03-14T10:02:11Z", "…": "…"},
  "digest": "<64 hex>",
  "signature": "<pure Ed25519, RFC 8032, 128 hex>",
  "key_id": "k_in_20280101_77d0",
  "anchor": {"suite": "rfc3161-6962", "root": "<hex>", "leaf_count": 82,
             "path": ["<hex>", "…"], "token": "<b64 DER>"},
  "issuer": "ar-org:issuer.example",
  "audience": "ar-org:acmetextiles.example",
  "nonce": "<32 hex = 128 bits>",
  "expires_at": "2028-03-21T10:02:11Z",
  "envelope_signature": "<Ed25519 over LP(digest)‖LP(audience)‖LP(nonce)‖LP(expiry)>",
  "disclosure_suite": "elan1-sd-v1",
  "disclosures": [{"name": "invoice_total", "value": "420000.00", "salt": "<hex>"}],
  "limits": "Establishes that the issuer's governed write path produced this record …"
}
```

14.2.6 Flow 1 — issuing a receipt

The actor is the issuing organisation's system, acting for a governed write that has already committed. The trigger is a write type whose declaration names a counterparty and whose `blast_radius` is `one_counterparty` or `cross_boundary`. The tenant must be at L3, and the record must be signed with its batch anchored, or the issuer must say that it is issuing ahead of anchoring [E22 §05].

1. The governed write completes the nine rungs and produces its record. Issuing is a separate governed write after the fact, "never a rung".
2. A receipt is requested for record `seq`. The counterparty is resolved from the register; an unregistered counterparty refuses with `GATE.COUNTERPARTY_UNKNOWN`, the existing L1 code.
3. `receipt.issue` climbs the rungs as a declared write type. It is checked for counterparty (registered, not sanctioned), consent where a disclosed field is `personal`, residency (a receipt may not be assembled outside the tenant's domain), and precondition (the source record exists, is signed and is not inside an open erasure request).
4. The receipt is assembled: sixteen fields, digest, signature, `key_id`, inclusion path if the batch has closed, and the envelope.
5. Chosen disclosures are attached (§14.3). The operator sees the revealed fields by name and classification before issue.
6. The issuance is appended to the issuer's chain, committing to the receipt digest and audience through F14: `Receipt issued — record 41,288, audience ar-org:acmetextiles.example, expires in 7 days`.
7. The receipt travels by the channel the business already uses: email, API, portal or the invoice's document flow. elan1 operates no delivery network.

Source: [E22 §05]. A sanctioned counterparty refuses with `GATE.COUNTERPARTY_SANCTIONED` and is never overridable. An unanchored source record issues with the path absent and the receipt states `unanchored`. An unavailable key produces no receipt and a `PLATFORM.TOKEN_MINT_FAILED`-class page. Issuance is metered at the source write type's basis, quantity 1, and is idempotent on `(source_seq, audience, disclosure_profile)` [E22 §05].

14.2.7 Flow 2 — receiving and verifying

The receiver may or may not be an elan1 customer. It needs a verifier — elan1's, or its own written from E25 — and a discovery route [E22 §06]. The order of checks is normative. The cheap, non-cryptographic refusals come first.

Step	Check	On failure
1	Echo the claim: issuer, audience, write type, disclosure list, expiry, anchored or not	—
2	`audience` equals the receiver's own `org_id`	Hard failure: `POLICY.RECEIPT_AUDIENCE_MISMATCH`
3	Expiry against own clock; `recorded_at` and anchor time precede now	`POLICY.RECEIPT_EXPIRED`
4	Resolve issuer key: route 1, else 2 or 3 as configured; never 4; route printed	`POLICY.TRUST_ANCHOR_UNKNOWN`

Step	Check	On failure
5	Recompute preimage (E12 encoding), SHA-256, compare digest; verify pure Ed25519 against the resolved fingerprint and the key window covering `recorded_at`	`POLICY.RECEIPT_UNVERIFIED`
6	Check the inclusion path against the RFC 3161 token	Absent: report `UNANCHORED` and continue
7	Check disclosures against per-field commitments (E23); print disclosed and withheld counts	`POLICY.RECEIPT_UNVERIFIED`
8	Replay: has `(issuer, source_seq, digest)` been accepted before?	`GATE.RECEIPT_REPLAYED` — reported, not silently deduplicated
9	Show verdict, route, disclosures and the limits line	—

Source: [E22 §06]. The limits line is fixed: "This establishes that the issuer's governed write path produced this record under these rules at this time. It does not establish that the underlying transaction occurred" [E22 §06].

The six codes in the table (with `GATE.COUNTERPARTY_UNKNOWN`) are added to the kernel's closed refusal register through E02's own procedure. For each one, the procedure names what it protects, writes its contract, registers it with rung, class and operator action, and adds positive and negative fixtures. "They are not new machinery; they are new rows" [E22 §06; E22 decision 10]. The verifier records nothing, "because a verifier that phoned home would not be a verifier" [E22 §06].

14.2.8 Flow 3 — binding a receipt into the receiver's chain

For an elan1 tenant, acceptance is a governed write of type `receipt.accept`. It climbs the nine rungs [E22 §07]. The gates are counterparty, precondition (the verification result is an input to the gate, not a claim in the payload), consent, residency and retention. At rung 5, `receipt.accept` declares `blast_radius: cross_boundary`. By the declaration's semantic rules L0-S05 and L0-S06, that forces `dual_control: true` and a non-empty `proof_binding`; neither is configurable [E22 §07; E03 §05]. At rung 7 the receiver records the acceptance in its own system of record. At rung 8 its record commits, through F14, to the issuer's receipt digest, `org_id`, `key_id`, discovery route and disclosure list.

> **How one chain binds to another, with no seventeenth field.** The receiver's `proof_binding.fields[]` names the receipt digest and the issuer identity as payload fields. They enter F14 like any other committed value. The receiver's chain commits to the exact bytes of the issuer's record without either chain changing shape and without a shared ledger [E22 §07].

A declaration fragment for the write type looks like this. The block and field names follow E03 and E22; the values are a **build recommendation**.

```
write_types:
  - id: receipt.accept
    blast_radius: cross_boundary   # L0-S05: dual_control; L0-S06: non-empty proof_binding
    dual_control: true
    irreversible: false
    reversal_plan: receipt.accept.reverse   # records a withdrawal of acceptance; never edits
    gates: [counterparty, precondition, consent, residency, retention]
    proof_binding:
      canonicalisation: elan1_lp_v1
      fields: [issuer_org_id, receipt_digest, issuer_key_id, discovery_route,
               disclosed_field_names, verifier_exit, verifier_version]
```

The receiver sees `Receipt accepted — bound at record 8,902 to issuer record 41,288`. Nothing new is written in the issuer's chain. There is no acceptance callback, because "a protocol in which acceptance phones the issuer would make every receipt a tracking beacon" [E22 §07; E22 decision 7]. For a receiver that is not an elan1 tenant, a standalone verifier can check a receipt but cannot bind one. What the receiving side looks like for them is open until Q1 2028 [E22 §14 Q2].

14.2.9 Replay, correlation and what a counterparty learns

Replay is stopped by three mechanisms, all required and none sufficient alone [E22 §08]:

Mechanism	Stops	Limit
`audience`, signed	Presentation to a party it was not issued to	Only if checked, hence step 2 before signatures
`nonce`, 128 bits, unique per issuance	Two identical receipts presented as two events	Receiver keeps a nonce set, bounded by expiry
Acceptance keyed on `(issuer, source_seq, digest)`	Binding the same event twice	Receiver's bookkeeping; the issuer cannot enforce it

Correlation is disclosed rather than denied. `tenant_id` (F01) is constant, so all receipts from one tenant are linkable. `seq` (F02) is monotonic, so "a receiver who gets receipt 41,288 in March and 58,100 in April knows the issuer made roughly 16,800 governed writes in between. This is a real leak and there is no mitigation that keeps the chain verifiable." Write type, actor kind and autonomy level (F05, F10, F11) disclose that an agent acted, and at what level. Timing patterns in F16 disclose working hours [E22 §08].

> **Receipt unlinkability is not claimed and will not be.** An issuer who cannot accept volume disclosure should not issue receipts to that counterparty, and the issuance screen says so in those words before the first receipt to a new audience [E22 §08; E22 decision 8].

A counterparty learns the write type, declaration and policy versions, actor kind and autonomy level, the recorded time and the anchor's upper bound, an opaque actor UUID (F09), the disclosed fields, and — unavoidably — volume between receipts. It never learns undisclosed values, the issuer's other counterparties, records, refusals or gate results, any key material, or whether a receipt was accepted [E22 §09]. Whether `seq` could be blinded without breaking verification is open until Q3 2028; the current answer is that it cannot [E22 §14 Q3].

14.2.10 Failure modes and disputes

Mode	Correct handling
Issuer key rotated between issuance and verification	Passes; abutting windows make the signing key answerable (Chapter 12)
Issuer key revoked after compromise	Verifier reports `REVOKED` with the time and does not judge earlier records; that is the receiver's judgement
Issuer stops existing	Pinned fingerprints still verify: "the receipt outlives the issuer as well as the vendor"
elan1 stops existing	No change; nothing in the flow calls elan1
Two receipts for one record with different disclosure sets	Both verify; the record digest is identical
Source record later reversed	The receipt stays true about what happened; the receiver asks for the reversal receipt; the issuance screen warns

Source: [E22 §10].

In a dispute (Flow 4), the disputing party captures the exact bytes received, and both parties run the same published verifier version independently. If the two runs disagree, the verifier build is at fault, and that is a P0 for elan1 whoever raised it. If both fail identically, the failure class says what happened. A digest mismatch means the bytes were altered after signing. An invalid signature means a wrong key or a forgery, with a route-2 resolution the first suspect. An invalid inclusion path means the anchor does not cover this record. An audience or expiry failure is procedural. The issuer then produces the source record from its own chain, which "either contains that record at that `seq`, or it does not, and that is the answer" [E22 §11]. elan1 does not adjudicate between two organisations and declines such requests in writing [E22 decision 9].

What must never happen: a receipt carrying its own key; a fallback to the hint; a callback on acceptance; a shared ledger or replication; elan1 operating a trust anchor for its customers; a degraded verification mode ("a receipt verifies or it does not"); a seventeenth field [E22 §12].

14.3 Selective disclosure

14.3.1 The problem L2 does not solve

At L2, `proof_binding` commits the whole payload as one digest in F14, and a `redaction_profile` decides which fields an export reveals, one profile per export [E23 §01; E19 §02]. That serves a bundle handed to an auditor. It does not serve two Phase 4 situations. The first is **one record with two recipients**: the statutory auditor needs the invoice amount and tax identifier, the lender needs the amount and the counterparty, and neither needs what the other got. The second is **a fact without a file**: "this write was under the declared credit limit", where the recipient needs the fact, not the limit or the balance [E23 §01].

> **The record does not change.** Sixteen fields, the same encoding, digest and signature at L1, L2 and L3. Everything happens inside F14 and in a new bundle member. "A design requiring a seventeenth field was considered and rejected in one line" [E23 §01].

14.3.2 What may be disclosed, and what never can

Category	Disclosable?	Why
Payload fields named in `proof_binding.fields[]`	Yes, field by field	The mechanism
Payload fields not in `proof_binding`	No	Never committed; a disclosure would be an assertion, not a proof
The sixteen fields F01–F16	Always fully disclosed	They are the record; hiding one breaks preimage, digest and chain
Gate results	As a set or not at all	Bound by declaration version, not by the preimage
The per-record salt	Never	Unbinds every field at once and destroys the erasure property
A per-field salt	Only for the fields disclosed	This is what a disclosure is
Signing key material	Never, in any artefact	[E15; E22]
A derived fact without its value ("under the limit")	Not in Phase 4	A predicate proof; machinery rejected

Source: [E23 §02].

The last row covers what most people imagine selective disclosure to be, and it is not available. "We cannot prove a predicate over a hidden value. We can prove a value, disclosed." The supported workaround is for the declaration to commit the predicate's outcome as its own field — `credit_limit_satisfied: true` — computed at rung 4 by versioned rules and committed in F14. Disclosing that field discloses exactly the fact, and the number stays hidden. "It is not zero-knowledge; it is careful declaration design" [E23 §02; E23 decision 8]. Whether this pattern needs kernel support as a declared derived field is open until Q1 2028 [E23 §12 Q5].

Figure 22 shows one record, its per-field commitments folded into F14, and two recipients each opening a different subset.

Figure 22 — Selective disclosure: revealed versus committed

L3 — SPECIFIED, NOT BUILT

Per-field salted commitments fold into F14, so one record reveals different fields to different recipients

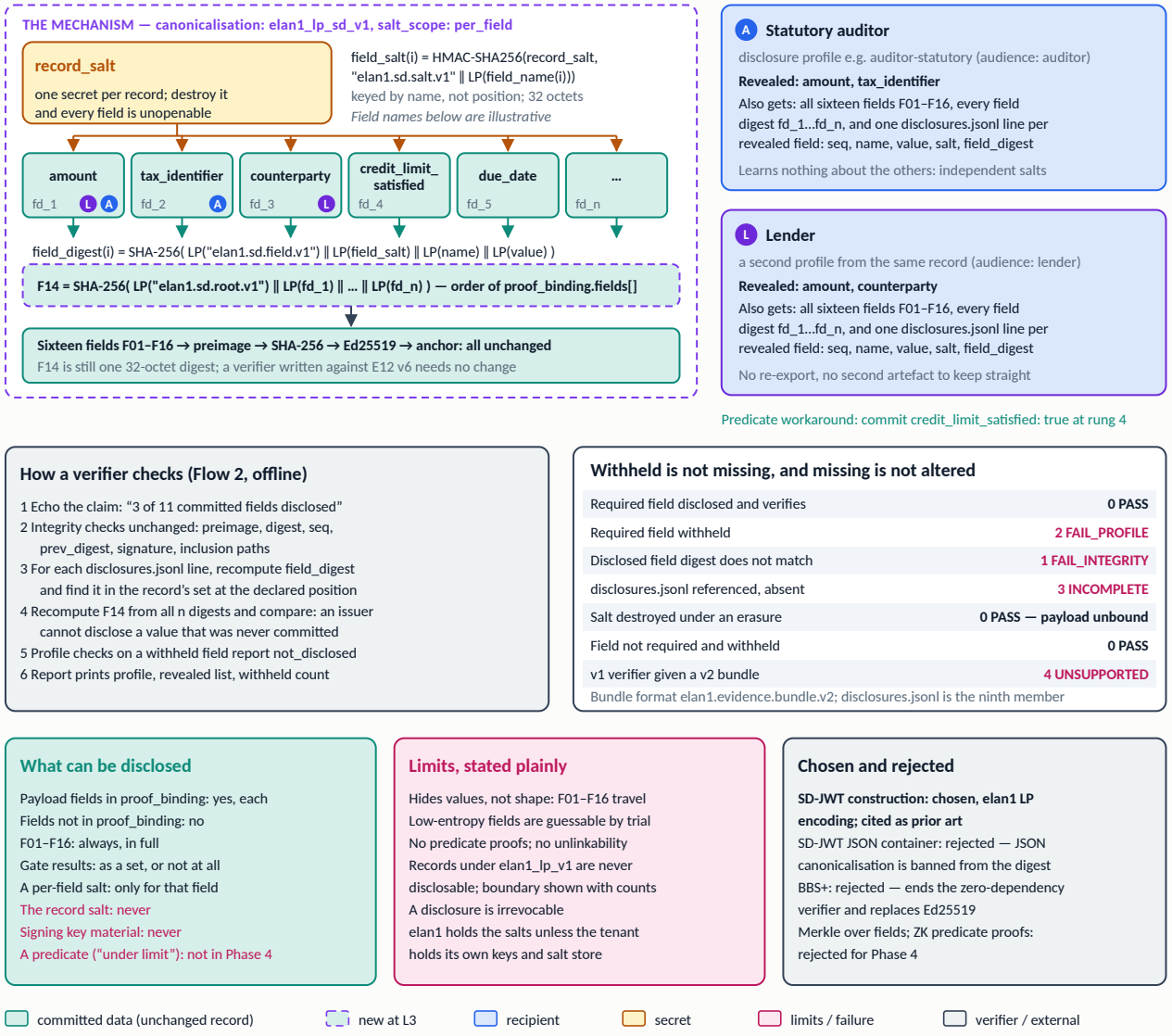


Figure 22 — Selective disclosure under elan1_lp_sd_v1: per-field salts derived from the record salt, per-field digests folded into F14, the unchanged sixteen fields and signature, and two recipients each receiving different openings in disclosures.jsonl while the verifier recomputes F14 from disclosed and carried digests.

14.3.3 The mechanism

At L3, `proof_binding.canonicalisation` gains one value, `elan1_lp_sd_v1`. Under it, F14 is computed in two stages [E23 §03]:

```
field_digest(i) = SHA-256( LP("elan1.sd.field.v1") ‖ LP(field_salt(i))
                         ‖ LP(field_name(i)) ‖ LP(field_value(i)) )
F14             = SHA-256( LP("elan1.sd.root.v1") ‖ LP(field_digest(1)) ‖ …
                         ‖ LP(field_digest(n)) )
field_salt(i)   = HMAC-SHA256( record_salt, "elan1.sd.salt.v1" ‖ LP(field_name(i)) )
                  -- truncated to 32 octets
LP(x)           = uint32_BE(octet_length(x)) ‖ x      -- as in E12
```

Field order is the order of `proof_binding.fields[]` in the declaration version the record names. That version is immutable, versioned and inside every bundle, so the order is never ambiguous [E23 §03]. Four properties follow [E23 §03]:

1. F14 is still one 32-octet digest. The preimage, signature, chain and anchor are untouched, and a verifier written against Seal Spec v6 verifies an L3 record without modification.
2. A field is revealed by publishing its name, value and salt. Anyone can recompute its digest and, with the other field digests, recompute F14.
3. Revealing one field reveals nothing about another, because each has an independent salt.
4. Destroying the per-record secret still unbinds everything, so erasure works exactly as at L2.

The salt design choices are deliberate. Salts are derived, not stored, so there is one secret per record and the erasure story is identical to L2. They are keyed by field *name*, not position, because a position-keyed derivation "would silently change meaning if a future version reordered". HMAC is used rather than a plain hash of a secret. And 32 octets makes a guessing attack on the salt infeasible [E23 §03; E23 decision 4].

The verifier-side check must live in the verifier's own source and share no code with the producer, because "recomputing a digest with the function that computed it proves nothing" [E14 §06; E25 §03].

14.3.4 Why SD-JWT's construction, and not its container

The construction is SD-JWT's disclosure construction: salted per-claim hashes, with disclosures carried beside the signed object. "It is the most reviewed design for this exact problem and there is no reason to invent one" [E23 §03]. The alternatives were assessed as follows.

Option	Verdict	Reasoning
SD-JWT as specified: JWT container, JSON claims, base64url disclosures	Container rejected	JSON canonicalisation is banned from the digest path (E12); adopting it would reintroduce at L3 the defect class the length-prefixed preimage removed
SD-JWT's construction in elan1's LP encoding	Chosen	A reviewer who knows SD-JWT recognises it in a minute; a second implementer has nothing to get wrong
BBS+ signatures	Rejected	Better (unlinkable, multi-show), but needs pairing-friendly curve arithmetic in the verifier, which ends the zero-dependency verifier, and it would replace Ed25519, breaking E12's freeze
Merkle tree over fields	Rejected	Costs $\log_2(n)$ siblings per disclosed field instead of n−1 digests; at typical field counts under twenty the flat form is smaller and has no tree-shape ambiguity
Zero-knowledge predicate proofs	Rejected for Phase 4	Right answer to the predicate problem, wrong answer to everything else: proving-system dependency, trusted-setup arguments, an unreadable verifier

Source: [E23 §03; E23 decisions 2–3]. elan1 cites SD-JWT as prior art and does not claim to implement it [E23 decision 2]. E23 marks SD-JWT's RFC status as unverified. The IM records it as published as RFC 9901 [E23 §03; IM §8.3]. The IM also recommends registering selective disclosure as a suite in Action Receipt v1 rather than retrofitting it in v2, because "a format designed for others to adopt cannot acquire redaction later without invalidating every implementation built against it" [IM §9]. The suite identifier is `elan1-sd-v1` [E25 §02].

14.3.5 Declaration and bundle changes

Block	Field	Change
6 `proof_binding`	`canonicalisation`	New value `elan1_lp_sd_v1`; `elan1_lp_v1` remains valid and the default
6	`salt_scope`	New value `per_field` (derived per field from the record salt)
6	`disclosure_profiles[]`	New optional array: `name`, `fields[]` ⊆ `proof_binding.fields[]`, `intended_audience_kind` (auditor · counterparty · lender · regulator · internal), `justification` ≥ 40 characters

Block	Field	Change
2 shape	`fields[].classification`	A `personal` field in a disclosure profile forces the consent gate on `receipt.issue` and on export

Source: [E23 §04]. Four semantic rules run at VALIDATE: L0-S12 (every disclosure field exists in `proof_binding`), L0-S13 (`elan1_lp_sd_v1` if and only if `salt_scope: per_field`), L0-S14 (a profile with a personal field requires the consent gate on every issuing write type), and L0-S15 (a lint warning where a profile is empty or discloses everything) [E23 §04].

> **The canonicalisation boundary.** A record written under `elan1_lp_v1` can never be selectively disclosed, because its F14 commits to the payload as a whole. Changing canonicalisation is a new declaration version, so the boundary is a `seq`, and the bundle renders it with counts on each side. Nothing is recomputed or re-signed [E23 §04; E23 decision 5].

This rule has a consequence for today's build (**build recommendation**). Every L2 record written under `elan1_lp_v1` is permanently non-disclosable. A pack whose regime is likely to need per-recipient disclosure should consider adopting `elan1_lp_sd_v1` from its first L2 declaration version once the canonicalisation value is frozen, rather than creating a large legacy range. That requires the L2 seal implementations and the verifier to support the two-stage F14 from L2 GA. E14's freeze forbids that, so the choice is recorded as a question for the L2 scope owner, not a recommendation to breach the freeze.

The bundle gains a ninth member, `disclosures.jsonl`, with one line per disclosed field (`seq`, `field_name`, `field_value`, `field_salt`, and the `field_digest` it opens). The format string becomes `elan1.evidence.bundle.v2`. The manifest requires `disclosure_profile` where disclosures are present and adds `disclosed_field_counts` per write type, where 0 is a value, not an absence. A v1 verifier reading a v2 bundle exits `4 UNSUPPORTED` and names the verifier to obtain. It never verifies "the parts it recognises" [E23 §05; E23 decision 6].

14.3.6 Producing and checking a redacted bundle

Producing (Flow 1). The requester picks a range, a verifier profile and a disclosure profile. The screen shows the record count, refusal count, unanchored head and the revealed and withheld field lists by name and classification: "You are revealing 3 of 11 committed fields" is "the sentence that prevents the mistake this feature exists to prevent." Export is a governed write. Selective disclosure adds a `consent` gate for personal fields and a `precondition` gate that fails if any record in the range was written under `elan1_lp_v1`; the range is never silently split. The system opens each disclosed field and checks it against the committed value before writing the line; a mismatch is a P0 chain-integrity finding. The export's own record lists the fields revealed and withheld, so "what did we show that lender in 2029" is answerable from the chain [E23 §06; E23 decision 9]. A field whose salt was destroyed under erasure is reported `payload unbound`, and the verifier exits `0 PASS — payload unbound`, never a failure [E23 §06].

Checking (Flow 2). The verifier echoes the claim, including "3 of 11 committed fields disclosed", and runs the integrity checks unchanged. A redacted bundle and a fully disclosed one produce identical integrity results, because the record is identical. For each disclosure line it recomputes the field digest, checks it at the declared position, recomputes F14 from all n digests and compares it with the record. "This is the step that makes disclosure sound: an issuer cannot disclose a value that was not committed" [E23 §07]. Profile checks whose input is an undisclosed field report `not_disclosed`.

Situation	Exit	Reasoning
Profile requires field X; X disclosed and verifies	0 PASS	Normal
Profile requires X; X not disclosed	2 FAIL_PROFILE	Records sound; the bundle does not meet the regime. A scope conversation
A disclosed field's digest does not match	1 FAIL_INTEGRITY, naming record and field	Somebody altered a value or salt

Situation	Exit	Reasoning
`disclosures.jsonl` referenced and absent	3 INCOMPLETE	Packaging failure
Salt destroyed under erasure	0 PASS — payload unbound	Never a failure
Profile does not require X; X withheld	0 PASS	Scope line only

Source: [E23 §07; E23 decision 7]. "Withheld is not missing, and missing is not altered." Because integrity and profile failures were already separated by E13 and E15, selective disclosure "required no new failure class", which is why it is a ninth bundle member and not a redesign [E23 §07, §08].

14.3.7 Limits, stated plainly

1. **Disclosure hides values, not shape.** F01–F16 always travel: tenant, sequence, write type, actor kind, autonomy level and time.
2. **A low-entropy field is guessable.** A recipient who knows `status` has four values can test all four against the commitment. "Commitment is not encryption." Where the value space is small and sensitive, commit a predicate outcome instead.
3. **Undisclosed does not mean unknowable.** The same field across many receipts gives many guesses.
4. **No predicate proofs.**
5. **No unlinkability.** Two receipts from one tenant are always linkable; BBS+ would fix this at the cost of the zero-dependency verifier.
6. **Pre-boundary records are not disclosable**, permanently.
7. **elan1 still holds the salts**, unless the tenant holds its own keys and salt store; until then "we cannot see your data" may not be said.
8. **A disclosure is irrevocable.** The export record exists so that at least the fact of the disclosure is not deniable.

Source: [E23 §09]. Three questions remain open: whether disclosure profiles belong in the declaration or beside it as a tenant-level object (Q1 2028); whether low-entropy fields should be padded or bucketed by default (Q2 2028); and — to be asked before the UI is built — whether any real auditor wants field-level disclosure or only counterparties do (Q4 2027) [E23 §12 Q1–Q3].

14.4 Precedent and earned autonomy

14.4.1 Three rules before any mechanism

E14 named precedent intelligence "the first thing on this platform that would sell itself" and also "the first thing that could put a statistical model near a decision", and deferred it on the second ground. Precedent intelligence waits until the model boundary is proven in production, not merely specified [E24 §01; E14 decision 7]. E24 therefore opens with three rules and only then describes a number.

> **Rule 1 — Nothing crosses a tenant boundary.** Precedent is computed over one tenant's own chain, for that tenant, and is visible only inside it. There is no cross-tenant benchmark and no "firms like yours" comparison: "a benchmark built from customers' records is a product our customers did not agree to be an input to."
>
> **Rule 2 — No model in the decision path, and no model in the graduation decision.** The precedent score is a versioned deterministic function over counts, replayable from the chain alone. A learned model may rank, summarise or explain for a human reader and "may never be an input to a score, a grant, a gate or an allow". The CI import-graph check covers the score function as it covers rungs 3 to 8.
>
> **Rule 3 — Autonomy is earned and always revocable.** It is raised only by a dual-control governed write against stated evidence, "and never by a support action, an admin toggle, a contract clause, a pricing tier or a salesperson. There is no API that sets an autonomy level" [E24 §01].

What may cross a tenant boundary is fixed. Records, payloads, salts, gate results, refusals, approvals, precedent counts, scores and graduation histories never cross; the existing isolation (RLS on every tenant table, a cross-tenant read test in CI) carries them. Cross-tenant benchmarks are "not built". Declaration and profile templates elan1 authors move freely. A defect learned from one tenant's chain travels only as a fix, never as data [E24 §02; O05 §03]. A future cross-tenant statistic would need a per-tenant opt-in that is itself a governed write, a k-anonymity floor, counts only, and a published methodology. None of that is in Phase 4 [E24 §02; E24 decision 1].

Surface	Model permitted?
Rung 4 `evaluate()`	No, at any level, ever (E01 invariant 2)
Rungs 3, 5, 6, 8	No; import-graph check in CI
The precedent score function	No; same check extended
Graduation eligibility	No; deterministic thresholds only
Proposing a write intent	Yes — "a model may produce a write intent. It may never produce an allow"
Summarising precedent on screen	Yes, with the underlying counts beside it, so a reader can check the sentence against the numbers
Drafting a graduation justification	Yes; a human signs it, and the signature is the accountable act

Source: [E24 §03; E24 decision 3].

14.4.2 The autonomy ladder

Rung 6 resolves the acting actor's grant for the write type from `autonomy_grant`. A missing grant resolves to A0 — "not to tenant default, not inherited from the delegation chain" [E04 rung 6; E02 I3]. The level held is sealed on every record as F11, encoded 0–5 [E12 §02].

Level	What rung 6 permits	How it is reached (Phase 4)
A0 Observe	Reads only; cannot reach rung 7 by any path	Default; missing grant
A1 Draft	Produces a write intent a human must submit	Declaration grant
A2 Propose	Submits; a human approves before rung 7	Declaration grant
A3 Act reversible	Acts alone where a reversal plan exists	**Graduation only**
A4 Act bounded	Acts alone inside a declared blast radius and budget	Declaration decision with an ADR; not in Phase 4
A5 Act broad	Acts alone at tenant scope; dual control on the declaration itself	Declaration decision with an ADR; not in Phase 4

Source: [E04 rung 6; E24 §05]. Graduation never applies to a write type with `irreversible: true`, because A3 is "act reversible" and an irreversible write type has no A3 to reach. It never applies to `blast_radius` of `tenant` or `cross_boundary`, which force dual control by L0-S05. Humans are granted, not graduated [E24 §05; E24 decision 6].

Figure 23 shows the ladder, the one graduated step, the evidence it needs and the one-person revocation path.

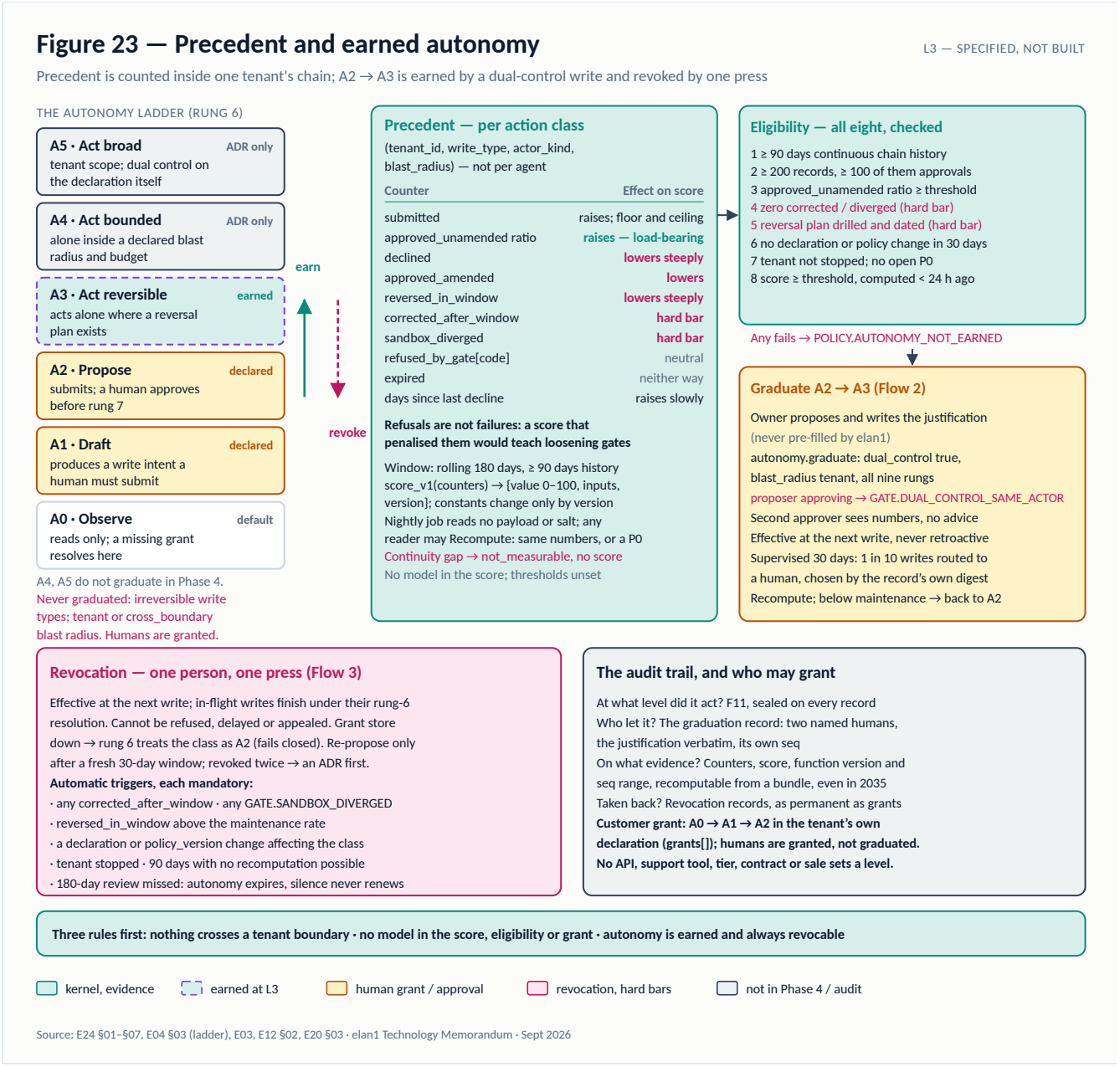


Figure 23 — Earned autonomy: the A0–A5 ladder resolved at rung 6, precedent counters projected from the tenant's own chain into a deterministic versioned score, the eight eligibility conditions, dual-control graduation from A2 to A3 with a 30-day supervised period, and one-person revocation with its automatic triggers.

14.4.3 How precedent is recorded

Precedent is kept per **action class**, `(tenant_id, write_type, actor_kind, blast_radius)` — not per agent. Autonomy attaches to what may be done, and a class survives an agent being replaced [E24 §04; E24 decision 4]. The counters are a projection of records that already exist, not new data:

Counter	Source	Meaning
`submitted`	Records in the class	Every attempt, including refusals
`refused_by_gate[code]`, `refused_by_policy[rule]`	Refusal records	Which gate or rule, at which `policy_version`
`escalated`	Obligation rows	Reached a human
`approved_unamended`	Approvals where applied payload = proposed payload	The load-bearing counter

Counter	Source	Meaning
`approved_amended`	Approvals where it differs	A human corrected the agent; evidence against
`declined`	`POLICY.APPROVAL_DECLINED`	A human said no
`expired`	`POLICY.OBLIGATION_EXPIRED`	Nobody answered; neither for nor against
`reversed_in_window`	Reversal records	Undone inside the window
`corrected_after_window`	Compensating entries	"The worst signal in the set"
`sandbox_diverged`	`GATE.SANDBOX_DIVERGED`	The write path disagreed with itself

Source: [E24 §04]. Counters run over a rolling 180-day window with at least 90 days of history. They are always replayable: "given the chain and the score function version, anyone — including a regulator with a bundle — recomputes the same numbers" [E24 §04].

14.4.4 The score

`precedent_score` is a versioned function `score_v1(counters) → {value, inputs, version}` producing 0–100. E24 fixes its shape, not its constants, which change only by a new function version [E24 §04; E24 decision 2].

Component	Direction
Volume of `submitted`	Raises, between a floor and a ceiling
`approved_unamended / (approved_unamended + approved_amended + declined)`	Raises
`declined` rate	Lowers, steeply
`approved_amended` rate	Lowers
`reversed_in_window` rate	Lowers, steeply
`corrected_after_window`, `sandbox_diverged`	Hard bars: any occurrence disqualifies
Gate refusal rate	**Neutral**
Days since the last decline or in-window reversal	Raises slowly

> **Refusals are not failures.** A high refusal rate means the gates are working and does not lower the score. What lowers it is a human intervening on a write that passed. "A score that penalised refusals would teach tenants to loosen gates to earn autonomy, which is the worst incentive this platform could create" [E24 §04; E24 decision 5].

The thresholds do not exist yet. They "must be set against a real tenant's data, not invented here", by Q2 2028 [E24 §09 Q1; IM §8.4]. The sketch below fixes the properties E24 requires — pure, integer-deterministic, versioned, returning its inputs — with placeholder constants. It is a **build recommendation**, and every number in it is illustrative.

```
SCORE_VERSION = "score_v1"
C = dict(vol_floor=200, vol_ceiling=2000, w_ratio=60, w_vol=15, w_recency=10,
         pen_declined=400, pen_amended=150, pen_reversed=400, recency_cap_days=90)

def score_v1(k: dict[str, int], days_since_last_bad: int) -> dict:
    """Pure function of counters; no clock, no network, no model. Integers only."""
    if k["corrected_after_window"] or k["sandbox_diverged"]:
        return {"value": 0, "bar": "hard", "inputs": k, "version": SCORE_VERSION}
    human = k["approved_unamended"] + k["approved_amended"] + k["declined"]
    if k["submitted"] < C["vol_floor"] or human == 0:
        return {"value": 0, "bar": "insufficient_evidence", "inputs": k,
                "version": SCORE_VERSION}
    ratio = (100 * k["approved_unamended"]) // human
    vol = min(k["submitted"], C["vol_ceiling"]) * 100 // C["vol_ceiling"]
    rec = min(days_since_last_bad, C["recency_cap_days"]) * 100 // C["recency_cap_days"]
    pen = (C["pen_declined"] * k["declined"] + C["pen_amended"] * k["approved_amended"]
           + C["pen_reversed"] * k["reversed_in_window"]) // max(k["submitted"], 1)
```

```
    raw = (C["w_ratio"] * ratio + C["w_vol"] * vol + C["w_recency"] * rec) // 85 - pen
    return {"value": max(0, min(100, raw)), "inputs": k, "version": SCORE_VERSION}
    # refusal counters are deliberately absent from the arithmetic
```

The nightly precedent job (Flow 1) reads the chain over the window. It reads no payload, salt or target-system state. It projects the counters, computes the score, and writes the score with its inputs, function version and `seq` range. The console renders the class, full counters, score, window and eligibility verdict. A **Recompute** control reruns from the chain: "the same numbers, or a P0". A chain with a continuity gap produces `not_measurable`, never a score. A stale score is shown with its date and may not be used in a graduation. The computation is not a governed write; only its use in a graduation is [E24 §04].

14.4.5 Graduation from A2 to A3

All eight eligibility conditions must hold, each checked rather than asserted [E24 §05]:

#	Condition
1	≥ 90 days of continuous chain history for the class, continuity verified over the window, not sampled
2	≥ 200 records in the class, of which ≥ 100 are approvals
3	`approved_unamended` ratio at or above the threshold in the current score version
4	Zero `corrected_after_window` and zero `sandbox_diverged` in the window — **hard bar**
5	The write type has a `reversal_plan`, executed once in a dated drill (E18 T3) — **hard bar**
6	No declaration or `policy_version` change affecting the class in the last 30 days
7	Tenant not stopped, no open P0
8	`precedent_score` at or above the graduation threshold, computed within 24 hours

Flow 2 runs as follows [E24 §05; E24 decision 7]. The tenant's owner opens Agents · Autonomy and sees `Eligible for A3` with all eight conditions ticked, or the first that fails and why. The owner presses **Propose A3**. The proposal shows the class, current and proposed level, evidence (counters, score, function version, `seq` range), the reversal plan and its drill date, and a justification the owner writes in their own words; elan1 does not pre-fill it. `autonomy.graduate` is a declared write type with `dual_control: true` and `blast_radius: tenant`, and it climbs the nine rungs. The proposer may never approve it (`GATE.DUAL_CONTROL_SAME_ACTOR`). The second approver sees the numbers, "not a recommendation: there is no 'we suggest approving'." On approval the grant changes at the next write, never retroactively.

The class then enters a **30-day supervised period**. It acts at A3, but a sampled 1 in 10 writes is still routed to a human, chosen by the record's own digest "so the sample cannot be steered". At the end, the score is recomputed, and below the maintenance threshold the class auto-reverts to A2. Reversion is not escalated to elan1 [E24 §05]. The digest-keyed sample can be implemented without any clock or random source (**build recommendation**):

```
def supervised_sample(write_intent_digest: bytes, rate_denominator: int = 10) -> bool:
    """True -> route to a human at rung 6 during the supervised period."""
    return int.from_bytes(write_intent_digest[:8], "big") % rate_denominator == 0
```

E24 speaks of "the record's own digest", but the routing decision is taken at rung 6, before the record exists. The sample must therefore key on a digest available at rung 6, such as the digest of the canonical write intent. This memorandum recommends that, and records the wording as a point to close. Refusals use `POLICY.AUTONOMY_NOT_EARNED` (a new code registered through E02's procedure), `GATE.DUAL_CONTROL_SAME_ACTOR`, `POLICY.APPROVAL_DECLINED` and `GATE.PRECONDITION_FAILED`. A stale score at the moment of approval refuses [E24 §05]. Whether 1 in 10 is right and should taper is open until Q1 2028 [E24 §09 Q2].

14.4.6 Revocation and the asymmetry rule

> **The asymmetry rule.** Raising autonomy needs two named humans, stated evidence and a supervised period. Lowering it needs one person and one press, takes effect at the next write, and needs no justification at the time. "Any design that makes revocation as hard as graduation is rejected" [E24 §05; E24 decision 8].

Any single holder of the owner role, the system, or elan1 in an incident may revoke. In-flight writes complete under the rung-6 resolution they already had. The class may be proposed again only after a fresh 30-day window under A2, and a class revoked twice needs an ADR before a third proposal. Revocation "cannot be refused, blocked, delayed or appealed through the product". If the grant store is unavailable, rung 6 treats the class as A2 until the revocation can be applied — "failing closed, which is the only acceptable direction" [E24 §05].

Eight automatic triggers are mandatory: any `corrected_after_window`; `reversed_in_window` above the maintenance rate; any `GATE.SANDBOX_DIVERGED`; a declaration version change affecting the class; a `policy_version` change affecting it; the tenant entering a stopped state; 90 days with no recomputation possible; and the 180-day review falling due without a review. "Autonomy expires by default. It is not renewed by silence" [E24 §05; E24 decision 9].

14.4.7 The audit trail, and why the model never grants itself autonomy

Question	Answered by
At what autonomy did this agent act?	F11 on the record itself — a sealed field, not a lookup
Who let it act at that level?	The graduation record: two named humans, justification, date, its own `seq`
On what evidence?	Counters and score in the graduation record, recomputable from the chain by anyone holding the bundle
Who could have taken it back, and was it?	The owner role in the declaration; revocation records, as permanent as grants
Did it act at a level it had not earned?	Impossible by construction: rung 6 resolves the grant at the moment of the write and F11 seals what it resolved

Source: [E24 §06]. A bundle therefore carries the autonomy history with no extra member, because graduation and revocation records are records like any other [E24 §06].

The model cannot grant itself autonomy for four structural reasons, each enforced in a different place.

1. **No write path to the grant.** The only way to change `autonomy_grant` is the `autonomy.graduate` write type (or a revocation). There is no API, support tool, admin console, migration script, tier or contract clause that sets a level [E24 §07; E20 decision 7; E24 decision 10].
2. **No model in the arithmetic.** The score and eligibility are deterministic functions under the same CI import-graph check that keeps models out of rungs 3–8, and L3 done-item 10 requires a replay to produce the identical score [E24 §01; E20 §07].
3. **Agents cannot approve.** `grants[].may_approve` is false wherever `actor_kind` is `agent`, by declaration rule, and graduation needs two named humans [E03 §05; E24 §05].
4. **Rung 6 reads the grant at write time.** An agent's intent cannot carry its own autonomy; F11 records what rung 6 resolved, not what the caller claimed [E04; E24 §06].

This is invariant I1 applied to autonomy: a model may produce a write intent, including a draft justification, but never an allow, and a grant is a kind of allow [E02; E20 §05].

Two ownership questions remain open. Does precedent belong to elan1 or to rung1's product surface, given that the Agents door is rung1's screen and the evidence is elan1's (Q1 2028)? And is earned autonomy an L1-era capability deferred for the wrong reason, since it needs "no proof machinery, only evidence discipline" (Q1 2028) [E24 §09 Q4, Q6; E20 §11 Q6]?

14.5 Customer-held keys and sovereign deployment

These are the two L3 capabilities specified elsewhere. Customer-held keys are covered in Chapter 12 (§12.11–§12.12). Their L3 done-item is a tenant signing with a key elan1 has never held, evidenced by a key ceremony record [E20 §07 item 7]. They also change the salt question: "we cannot see your data" becomes true only for a tenant with customer-held keys *and* customer-held salts, and is "a deployment fact about one customer, never a product claim" [E20 §05; E23 §09]. Sovereign deployment is a deployment model, covered in Chapter 17. It may not be sold before a defect-propagation process exists [E20 §08; IM §8.5].

14.6 Build sequencing and dependencies on L2

14.6.1 What each L3 capability consumes

L3 capability	Depends on (L2)	Depends on (L1)	New at L3
Second seal implementation	Seal Spec v6, 64 vectors, E12 Flow 3 two-implementation procedure	—	A team with no access to the first's source [E20 §07 item 2; E20 Q3]
Action Receipt standard	Bundle format, offline verifier, release key, vector set	—	Suite registry, conformance levels, governance (Chapter 15)
Cross-organisation verification	Signing, `keys.json` windows, anchoring, out-of-band fingerprint, E25 format	Counterparty register and gate; E02 refusal-register procedure; `blast_radius` and L0-S05/S06	Envelope, `receipt.issue` / `receipt.accept`, six refusal codes, key document
Selective disclosure	`proof_binding`, per-record salt store, erasure, E13/E15 failure-class separation, bundle v1	Consent gate; field classification	`elan1_lp_sd_v1`, `disclosures.jsonl`, bundle v2, L0-S12…S15
Customer-held keys	Key register, windows, compromise procedure (E16)	—	Custody interface, customer HSM backend, ceremony record
Precedent and earned autonomy	Replayable, continuous chain; bundle carrying graduation records	Rung 6 grants, approval and refusal records, obligations, reversal plans and drills (E18 T3), nightly integrity run, import-graph check	Counter projection, `score_v1`, `autonomy.graduate`, supervised sampling, triggers

Sources: [E20; E22; E23; E24; E12; E15; E16].

Two dependencies run in a fixed order. **Cross-organisation verification needs the published format**: "E22 is impossible without E25", because two organisations cannot check each other's receipts against a proprietary format [E25 §11]. **Cross-organisation receipts with disclosures need the disclosure suite**, and the IM recommends that the suite ship in Action Receipt v1 [IM §9].

14.6.2 A proposed sequence

The sources fix the entry event and the done-items, not an order inside Phase 4. The sequence below is a **build recommendation**. It follows the dependency graph and assumes the "about four teams" of the overall plan are re-deployed rather than enlarged.

Step	Work	Gate to proceed
0	Entry condition met (four tests, in writing)	E20 §02
1	Commission the independent second seal implementation; freeze Action Receipt v1 structure with `elan1-seal-v6`, `rfc3161-6962` and `elan1-sd-v1` suites	Second implementation agrees on 64 vectors and a full production chain
2	Selective disclosure in kernel, bundle v2 and verifier	Redacted bundle verified by an outsider (done-item 6)
3	Action Receipt v1 comment period (60 days) and a second external implementer	E25 Flow 1 refusal rules pass
4	Cross-organisation verification: envelope, issue/accept write types, codes, key document	Two organisations in production (done-item 5)

Step	Work	Gate to proceed
5	Earned autonomy, in parallel with steps 1–4 on the kernel team, since it consumes only L1/L2 data	A2 → A3 graduation and a separate revocation (done-item 9)
6	Customer-held keys, only with a contract attached	Ceremony record (done-item 7)

Step 5 can run in parallel because earned autonomy needs "no proof machinery, only evidence discipline" [E20 §11 Q6]. If that open question resolves downward, it could move before L3 entry altogether. That is a scope decision for the founder, not a scheduling convenience.

14.6.3 Tests that must exist before any L3 claim

Test	Asserts	Source
Receipt audience check precedes key resolution	Misaddressed receipts never reach cryptography	[E22 decision 5; E25 §03]
Replay of `(issuer, source_seq, digest)` refuses with `GATE.RECEIPT_REPLAYED`	Single binding	[E22 §07]
`receipt.accept` without dual control fails declaration validation	L0-S05 enforced	[E22 §07]
Withheld field required by profile exits 2, altered disclosure exits 1, missing member exits 3	Failure classes never collapse	[E23 §07]
Range spanning `elan1_lp_v1`/`elan1_lp_sd_v1` boundary refuses at precondition	No silent split	[E23 §06]
Score replay from bundle equals stored score	Determinism	[E24 §04; E20 §07 item 10]
Import graph: no model package reachable from rungs 3–6, 8 or `score_v1`	Model boundary	[E24 §01]
Refusal-heavy class scores equal to refusal-free class with the same human counters	Refusals neutral	[E24 §04]
No endpoint writes `autonomy_grant` except the graduation and revocation write types	No back door	[E24 §07]

14.7 What is still not true at L3

E20 ends with seven sentences "an honest engineer should say in a meeting", and they apply unedited [E20 §06; IM §8.6]:

1. elan1 is still the only writer. A receipt proves what went through the write path; anything that did not is invisible to it, and reconciling the chain with the target system's audit trail is the customer's job.
2. An anchor is an upper bound on time and nothing else.
3. A counter-signature by a non-elan1 party proves that party saw a digest.
4. Selective disclosure hides values, not shape.
5. Precedent is not prediction, and earned autonomy is not a safety argument: "it is a graduated permission with an evidence trail and a revocation path."
6. An open specification is not adoption.
7. Sovereign deployment multiplies every operational surface, and is "the capability most likely to be sold before it is ready".

15. The Action Receipt standard, certification and regulator engagement

This chapter covers three things that sit outside the write path but decide whether its output is believed. The first is the **Action Receipt**, a public specification of the proof format that anyone may implement [E25]. The second is **certification** of elan1's own management system against ISO/IEC 42001 or SOC 2 [E26 Part A]. The third is **regulator engagement**, which is governed by one rule about what may be claimed [E26 Part B]. The IM covers the same ground in §9 and §10 [IM §9, §10]. The chapter closes with the overall standard and certification plan and the partner and verifier programme, for which the group working drafts set the brief [WD].

> **Designed is not shipped (DR-010).** The Action Receipt specification is a draft (E25, 21 September 2026). No version has been published, no external party has implemented it, and elan1 holds no certification of any kind — "no SOC 2, no ISO/IEC 27001, no ISO/IEC 42001, no assessment against NIST AI RMF" [E26 §02]. No regulator has reviewed anything elan1 has produced. All dates below are plan dates, and all costs are modelled or unverified.

15.1 Publish the format, keep the write path

The Action Receipt is "one record, one signature, one anchor, one procedure a stranger can implement from the document alone — published so that a receipt remains checkable after this company stops existing" [E25]. The IM states the principle in one line: "A proof nobody else can read is not a proof" [IM §9].

The format is given away because it was never the asset. "A sixteen-field preimage, a digest, an Ed25519 signature and a Merkle inclusion path are a weekend's reading for a competent engineer" [IM §9; E25 §11]. What takes years is the write path, the packs, the certified verifier profiles and the Regulation objects that bind a clause to a gate. The stronger argument concerns the **suppression attack**. A receipt can attest only to a call that was made. An agent that skips the call, or an integration that writes around the governed path, leaves no receipt, and in a receiver-attested design the absence of a receipt looks the same as the absence of activity. "No format solves this. No signature solves it. No transparency log solves it" [IM §9]. A single writer solves it by construction. If every consequential change to the system of record passes through one enforcement point, a missing receipt is itself evidence: either nothing happened, or something bypassed a control, and the second is a detectable, alertable condition [IM §9]. That is why elan1 can publish the format without publishing the product. The artefact can be checked by anyone; the guarantee that the set of artefacts is complete is a property of the architecture.

Given away, permanently	Sold
Specification text (CC BY 4.0)	The kernel: policy gates as data, forced approval, the single writer
Reference verifier, source and binaries (Apache-2.0)	Regulatory packs and certified verifier profiles
Test vectors and the tampered bundle (CC0)	The metered governed action and the assurance levels
Verification procedure and suite registry	Residency, key custody, anchoring, retention and the evidence SKU
Errata, disagreements and defect resolutions	Cross-organisation verification and counterparty seats

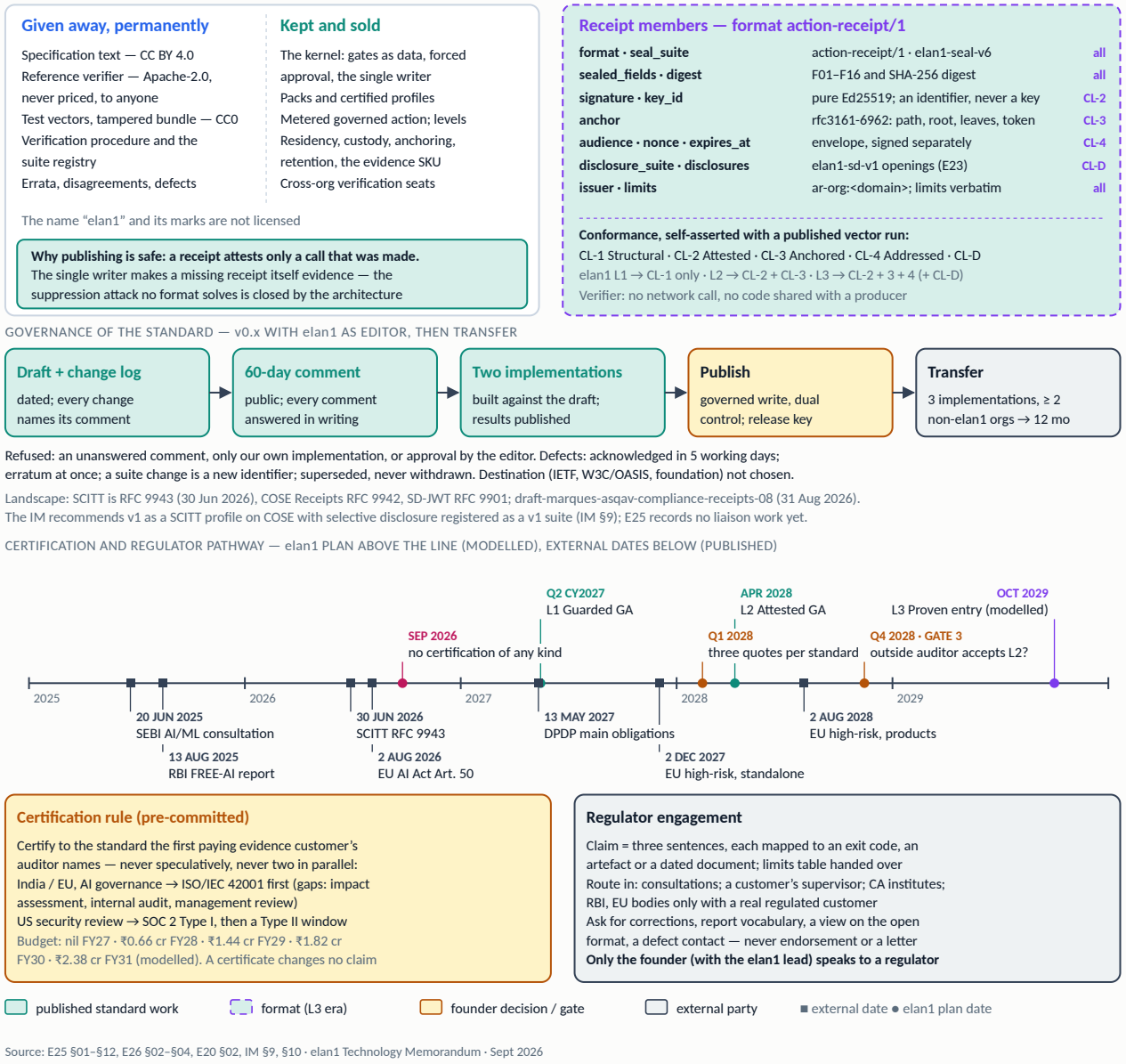
Source: [IM §9; E25 §05].

Figure 24 places the standard beside the certification and regulator pathways.

Figure 24 — Action Receipt standard, certification and regulators

PHASE 4 — SPECIFIED, NOT BUILT

Publish the format and a free verifier, keep the write path; certify only when a paying customer's auditor names a standard



Source: E25 §01–§12, E26 §02–§04, E20 §02, IM §9, §10 · elan1 Technology Memorandum · Sept 2026

Figure 24 — The Action Receipt boundary: specification, reference verifier and vectors published under open licences while the governed write path, packs and profiles stay commercial; conformance levels CL-1 to CL-D mapped to assurance levels; and the separate certification (ISO/IEC 42001 or SOC 2) and regulator-engagement pathways, each bounded by what a verifier can demonstrate.

15.2 Scope of the standard

The standard specifies four things and nothing else: the structure of a receipt, the verification procedure a conforming verifier performs, the conformance levels a producer or consumer may claim, and a registry of seal, disclosure and anchor suites "so the format can outlive any one cryptographic choice" [E25 §01; E25 decision 1].

Not in scope	Why
How a write is decided	The nine rungs are elan1's design; a conforming producer may govern writes however it likes
Transport	A receipt is a file (E22)
Identity and trust anchors	Naming and discovery are defined; no registry is operated or appointed
Legal effect	No opinion, in any jurisdiction

Not in scope	Why
Payload semantics	Committed to, not defined; the standard does not define what an invoice is
Certification of conformance	No certification mark in v0

Source: [E25 §01].

> **What a receipt establishes.** "A producer's write path recorded this event, in this form, under these named rules, with this actor kind, no later than this instant, and the record has not been altered since. Nothing more." It does not establish that the transaction was genuine, that the rules were adequate, or that the producer is honest. "A producer that records fabricated events faithfully produces faithful receipts of fabricated events" [E25 §01].

The standard leaves out the property that matters most commercially — whether the write path is the only path. A conforming producer that is not a single writer can emit valid receipts while writes bypass it. The standard is silent on this by design, and elan1's competitive position rests on that silence.

15.3 Receipt structure and its relationship to Seal v6 and the bundle

15.3.1 The thirteen members

Member	Required	Contents
`format`	Yes	`action-receipt/1`; a change is a new string, never a minor version
`seal_suite`	Yes	Registry identifier, e.g. `elan1-seal-v6` (E12)
`sealed_fields`	Yes	The suite's field set in order and encoding; for `elan1-seal-v6`, F01–F16
`digest`	Yes	The suite's digest over the suite's preimage
`signature`	From CL-2	For `elan1-seal-v6`: pure Ed25519 over the 32-octet digest, RFC 8032, not `ph`, not `ctx`
`key_id`	With `signature`	An identifier, never a key
`anchor`	From CL-3	Anchor suite id, inclusion path, root, leaf count, token; for `rfc3161-6962`, RFC 3161 over an RFC 6962 root
`issuer`	Yes	`org_id`, `ar-org:<domain>` (E22)
`audience`	From CL-4	`org_id` the receipt was issued to
`nonce`	From CL-4	128 random bits, unique per issuance
`expires_at`	From CL-4	RFC 3339 UTC, second precision
`disclosure_suite`, `disclosures`	Optional	e.g. `elan1-sd-v1` (E23); per-field openings: name, value, salt
`limits`	Yes	The producer's limits statement, reproduced verbatim in every report

Source: [E25 §02]. Carrying `limits` inside the receipt makes the discipline of E26 structural rather than editorial [IM §9].

Four structural rules each defend against a named failure [E25 §02]:

1. **A receipt never carries a verification key**, only `key_id` and `issuer`.
2. **No optional field inside the sealed set.** "Optionality in a digest specification is how two implementations produce different preimages for the same event"; a refusal carries zeroes, not omissions.
3. **A domain string in the suite's preimage**, so a digest under one suite cannot be mistaken for one under another.
4. **The envelope is not in the digest.** Audience, nonce and expiry are signed separately over `LP(digest) || LP(audience) || LP(nonce) || LP(expires_at)`, so one record can be issued to two audiences without two records existing [E25 decision 3].

E25's member table has no member that carries the envelope signature, although rule 4 and verification step 9 require one [E25 §02, §03]. This memorandum adds `envelope_signature` as a fourteenth member, required from CL-4 (**build recommendation**). The gap should be closed in the specification text before the comment period, because a second implementer cannot otherwise place the bytes.

15.3.2 Receipt, record and bundle

The receipt adds nothing to the record. `sealed_fields`, `digest`, `signature` and `key_id` are the Seal v6 record exactly as it sits in `seals.jsonl`. `anchor` is the record's inclusion path and token from `anchors/` [E12; E15 §02]. A receipt is a single-record projection of a bundle, plus an envelope and optional disclosures.

Aspect	Evidence bundle (E15, E23)	Action Receipt (E25)
Unit	A range of records, one tenant	One record
Purpose	Hand a period to an auditor (third-party verification, L2)	Cross an organisational boundary in the course of business (L3)
Format string	`elan1.evidence.bundle.v1`; `v2` with disclosures	`action-receipt/1`
Declarations and lexicon	Carried verbatim	Not carried; named by version in F-fields
Key	`keys.json` windows and fingerprints, never the key	`key_id` only; key resolved by a discovery route
Addressing	None	`audience`, `nonce`, `expires_at`, envelope signature
Profile claim	Manifest names the verifier profile	None; profiles apply to bundles
Continuity	Chain walk over the range	None beyond the single record; volume leaks via `seq`

Source: [E15; E22; E23; E25]. The bundle row "declarations not carried" means a receipt verifier cannot answer "which rules decided" from the receipt alone. It establishes that the named declaration version was in force, not what it said. A receiver who needs the rules asks for a bundle (Flow 4 of E22) [E22 §11]. This is an inference from the member list and should be stated in the specification's limits (**build recommendation**).

15.4 The suite registry and versioning

Suite kind	First entry	Defines
Seal suite	`elan1-seal-v6`	Field set, encodings, preimage, digest, signature algorithm
Anchor suite	`rfc3161-6962`	Inclusion path and time-token construction and checks
Disclosure suite	`elan1-sd-v1`	Per-field commitment and opening (E23)

Source: [E25 §02]. Another implementer may register another suite. elan1's suites are the first entries and carry its name openly: "they are ours, published, not neutral by pretence" [E25 §02]. Versioning follows three rules: a change to a suite's preimage, encodings or algorithms is a **new suite identifier, never a revision**; a change to the receipt structure is a **new format string**; and there are no silent minor versions [E25 §06; E25 decision 2]. A verifier that does not know a suite or format exits `UNSUPPORTED` and never interprets [E25 §02, §03]. The same rule already governs bundles, where a v1 verifier meeting a v2 bundle exits `4 UNSUPPORTED` [E23 §05; E15], and Seal Spec v6, where any change to the record format is a new domain string and a new verifier [E14 decision 4].

15.5 The verification procedure

The procedure is written so a second implementer can build a verifier without asking elan1. Its order is normative, and cheap non-cryptographic refusals come first [E25 §03]:

#	Step	On failure
1	Parse `format`	`UNSUPPORTED`

#	Step	On failure
2	Resolve `seal_suite`, and anchor and disclosure suites where present	`UNSUPPORTED`
3	Check `audience` against the verifier's own `org_id`	`AUDIENCE_MISMATCH`
4	Check `expires_at` against the local clock	`EXPIRED`
5	Reconstruct the preimage from `sealed_fields`, compute and compare the digest	`FAIL_INTEGRITY`, naming the field
6	Resolve the issuer's key for `key_id` by a configured route, never from the receipt	`TRUST_ANCHOR_UNKNOWN`
7	Check the key's validity window covers the recorded time	`FAIL_INTEGRITY`, naming the window
8	Verify the record signature	`FAIL_INTEGRITY`
9	Verify the envelope signature	`FAIL_INTEGRITY`
10	Verify the anchor: path to root, root covered by token, token time before local clock	`FAIL_INTEGRITY`; absent anchor is `UNANCHORED`, reported not failed
11	Recompute each disclosed commitment and fold into the committed root	`FAIL_INTEGRITY`, naming the field
12	Check the nonce against a seen-set	`REPLAYED`, reported
13	Emit the report: echoed claim, verdict, discovery route, disclosed and withheld counts, `limits` verbatim	—

Three rules bind any conforming verifier. It makes no network call during verification; key discovery is a separate, configured, reported step before step 5. It shares no code with any producer. And it never collapses failure classes: "altered is not unmet requirement is not missing piece" [E25 §03].

The receipt verdicts need a defined mapping onto the bundle verifier's six exit codes, so that one binary can serve both. The sources do not give one. The mapping below is a **build recommendation**: `UNSUPPORTED` → 4; `FAIL_INTEGRITY` → 1; `TRUST_ANCHOR_UNKNOWN` → 3 (INCOMPLETE, because an input the reader must supply is missing); `AUDIENCE_MISMATCH`, `EXPIRED` and `REPLAYED` → 2 (FAIL_PROFILE-class: sound bytes, unmet requirement); `UNANCHORED` → 0 with a note. This keeps the E15 rule that altered, unmet and missing are never confused [E15 §06]. E22's receiver-side refusal codes (`POLICY.RECEIPT_AUDIENCE_MISMATCH` and the rest) are the kernel's names for the same outcomes when the receiver is an elan1 tenant [E22 §06].

15.6 Conformance for other implementers

15.6.1 Levels

Conformance is claimed separately by producers and consumers, and a claim names both the level and the suites [E25 §04; E25 decision 4].

Level	Producer	Consumer
CL-1 Structural	Well-formed receipt, sealed field set, digest, limits; no signature	Parses and recomputes the digest; refuses to treat an unsigned receipt as attested
CL-2 Attested	Signs the digest; publishes key ids with windows; never ships a key	Resolves keys by a route other than the receipt; enforces windows
CL-3 Anchored	Anchors to an independent time source; carries inclusion paths	Verifies inclusion and token time; reports `UNANCHORED` without failing
CL-4 Addressed	Audience, nonce, expiry, envelope signature	Checks audience before keys; enforces expiry; keeps a nonce seen-set
CL-D Disclosing	Supports a disclosure suite	Verifies openings; reports withheld as withheld, never missing

The levels are deliberately not elan1's assurance levels. A table maps them: L1 Guarded is a CL-1 producer and "nothing above may be claimed"; L2 Attested is CL-2 plus CL-3; L3 Proven is CL-2, CL-3 and CL-4, with CL-D where a disclosure suite is used [E25 §04]. Keeping them apart matters commercially, because "a buyer cannot then be sold an assurance level on the strength of a conformance claim" [IM §9]. The IM adds that at L1 a string check over the consumer's own rendered copy fails the build if anything above CL-1 is claimed [IM §9 figure]. That check belongs in CI with the claims lint described in §15.13.

15.6.2 What an implementer does

A third party reads the specification and writes a producer or verifier. It does not contact elan1, "and the specification is defective if they must". It runs the vector set: every positive must pass, and every negative must be rejected with the record and field named. It runs the published tampered bundle and watches it refuse. It publishes its result naming the version, suites and levels. It may state conformance "without our permission, our review or our mark. We may link to it; we may not certify it". If its implementation disagrees with elan1's on any vector, the disagreement is published and investigated as a P0 in the specification until the specification says which is right [E25 §08; E25 decision 9]. "A claim without a published vector run is not a claim" [E25 §04].

The sources name the conformance artefacts — the specification, the vector set and the tampered bundle — but not a packaged kit. The kit below is a **build recommendation**. It is the smallest set that lets a stranger complete E25 Flow 2 without contacting elan1.

```
action-receipt-conformance-1.0/
  SPEC.md                    # E25 text, CC BY 4.0
  suites/                    # elan1-seal-v6.md, rfc3161-6962.md, elan1-sd-v1.md
  vectors/positive/*.json    # receipts + expected verdict + expected report fields  (CC0)
  vectors/negative/*.json    # each names the record and field that must be reported (CC0)
  tampered/bundle/           # known-tampered bundle; expected exit 1, record and field
  keys/test-fingerprints.txt # dev-DO-NOT-TRUST and vector keys only
  RESULT-TEMPLATE.json       # version, suites, levels, role, per-vector verdicts
  SHA256SUMS, SHA256SUMS.sig # release-key signature (E16); fingerprint out of band
```

15.7 Licensing and the reference verifier

Artefact	Licence	Reasoning
Specification text	CC BY 4.0	"A specification nobody may copy is not a standard"
Reference verifier source	Apache-2.0	Permissive with a patent grant; a copyleft verifier would be excluded from proprietary audit tooling, "the tooling we most want it inside"
Test vectors and tampered bundle	CC0	No attribution friction, "including by a competitor proving our implementation wrong"
The name elan1 and its marks	Not licensed	Anyone may implement; nobody may imply elan1 endorsed their implementation

Source: [E25 §05; E25 decision 5]. The reference verifier keeps E15's properties, which "are now part of the standard's credibility rather than ours alone": zero dependencies, an empty lockfile, a reproducible build, published digests, standard library only, no network call, no shared code with a producer, and a tampered bundle beside it [E25 §05; E15 §07]. The open-source licence register records the group's licence positions (Chapter 18) [O04].

> **The verifier is never priced, at any level, to anyone, including non-customers.** "A verifier you must buy from the vendor whose records it checks is not a verifier." The standard makes this a public commitment [E25 §05; E07 decision 3]. The group narrative adds that no investment term may allow a change of control to close the standard, revoke its grant or make the verifier chargeable [O12 §10; O12 decision 9].

One question is open and carries the moat argument: do the Regulation objects and verifier profiles go out under the same licences, or stay closed? It is scheduled for Q4 2028 [E25 §13 Q6; IM §9].

15.8 Governance of the standard

15.8.1 Now: elan1 as editor

While the standard is at v0.x, the named editor changes it on a published draft. A 60-day public comment period runs, every comment is answered in writing, and a change ships only when two independent implementations have built against it. Disputes are decided by the editor in public, with the reasoning recorded and disagreement published beside the decision. A defective version is superseded, never withdrawn [E25 §06; E25 decision 7].

Publishing a version (Flow 1) is a governed write with `dual_control: true`, approved by a second named human. The release approver signs the artefacts with the release key (Chapter 12) and publishes to at least two independent locations [E25 §07]. The refusal paths are fixed. A version with an unanswered comment is refused. A version with only elan1's own implementation behind it is refused. A version approved by its own editor is refused with `GATE.DUAL_CONTROL_SAME_ACTOR`. If no second implementer can be found, the version does not ship and the reason is published: "we could not find a second implementer" is "a true and useful sentence about a standard's health" [E25 §07].

Defect reports (Flow 3) come from anyone, with no account, contract or NDA. They are acknowledged within five working days and classified editorial, substantive or structural. Substantive and structural defects get an erratum immediately, ahead of any new version. A fix that changes a preimage, encoding or algorithm is a new suite identifier, and receipts under the old suite remain verifiable for ever. A report is never declined for being commercially inconvenient, and a competitor's report is handled identically [E25 §09].

15.8.2 Later: the transfer condition

> **Transfer condition.** "When three independent implementations exist from at least two organisations that are not elan1, editorship moves out of this company within twelve months. Until that day, the honest word is published, not open" [E25 §06; E25 decision 8; E20 decision 6].

Destination	Fit	Cost
IETF, individual draft into a working group	Best fit for a wire format; SCITT's natural neighbour	Slow; sustained participation elan1 may not be able to fund
W3C or OASIS community group	Lighter, quicker, weaker signal	Needs three participating organisations — the transfer condition itself
A small foundation with a charter	Fastest, controllable	Reads as a vendor foundation unless others join

Source: [E25 §06]. None is chosen: "choosing a destination before anyone else implements is a way of looking like a standards body while being one company" [E25 §06]. The choice is due Q4 2028 [E25 §13 Q3].

15.9 Alignment with existing work

Work	Relationship	Honest position
SCITT (RFC 9943, published 30 June 2026; COSE Receipts RFC 9942)	Closest neighbour; its transparency service is the trust anchor elan1 will not operate. Natural interop: register a receipt digest and carry the inclusion receipt as an anchor suite	No liaison work done
C2PA	Same claim shape, different subject (provenance of a thing, not of a change); its manifest-stripping and trust-list lessons apply	No relationship
RFC 3161	Used directly as the `rfc3161-6962` anchor suite	"The most concrete alignment we have"
SD-JWT (RFC 9901) / W3C VCs	Construction reused, container rejected (E23)	Cited as prior art; not claimed as implemented
ISO/IEC 42001, NIST AI RMF	A receipt is evidence for an AIMS control, not an implementation of one	E26 governs what may be said

Source: [E25 §10; IM §9].

Two further points come from the IM. First, elan1 is not first: `draft-marques-asqav-compliance-receipts-08`, an individual Internet-Draft published 31 August 2026 and expiring 4 March 2027, is "the most complete public articulation of what elan1 proposes", and a survey counts eight competing receipt protocols [IM §9]. Second, the IM recommends "in its own voice" that Action Receipt v1 be specified as a SCITT profile on COSE, with selective disclosure registered as a suite in v1 rather than added in v2. The Asqav draft states no SCITT dependency, which leaves that position open [IM §9]. E25 as drafted does not specify COSE. Section 15.3's structure is serialisation-neutral, and a COSE-profiled seal suite would be "a registry entry, not a redesign" [IM §9]. This memorandum treats the COSE recommendation as a proposal for the v1 comment period, not a decision (see conflicts).

The IM also proposes carrying OpenTelemetry GenAI trace and span identifiers "as first-class receipt fields", so that adoption becomes a configuration change [IM §9; IM §13.8]. Under E12 and E25 they cannot enter the sealed set, since that would be a seventeenth field. They can be committed through F14 as declared `proof_binding` fields, and so be signed, anchored and selectively disclosable, without changing the record (**build recommendation**).

15.10 Timing, and the case against publishing

"Timing follows the assurance ladder rather than leading it." L2 GA is April 2028 and L3 opens in October 2029 [modelled]. "Publication belongs to Phase 4, after an outside auditor has attached value to the artefact" [IM §9]. The Series A's third tranche (Gate B, measured at 30 June 2029) requires "Action Receipt v1 published under terms a competitor can implement, with the sixty-four vectors and the known-tampered bundle, and at least two named external implementers working against the specification, of whom at least one has produced a receipt our verifier accepts" [IM §16.5.2]. E25's own rules require a 60-day comment period and two implementations before any version ships [E25 §07], and E20 forbids L3 work before the entry condition [E20 §08]. The v1 track must therefore start as soon as the entry condition is met, which is itself Gate B item 1. On the IM's dates that leaves at most two quarters between an auditor's written acceptance and a published v1 with external implementers. The conflicts block records this compression.

E25 records the case against publishing "at full strength". elan1 is pre-revenue, publishing commoditises the only artefact it can show, standards work consumes founder time, and a standard someone else extends leaves elan1 maintaining a format it no longer controls [E25 §11]. The answer has four parts. Publication is gated on an outside auditor having valued the artefact. The format was never defensible. Standards work is budgeted: a standards, policy and regulator-affairs function of 1 person at March 2027 rising to 13 by March 2031, and Rs 2.95 crore across the plan [modelled]. And loss of control is chosen on a stated trigger [IM §9]. The falsification condition is measurable: "if, by the end of Phase 4, three or more competitors are emitting Action Receipts and none of them needed elan1's domain work", publication was the wrong call [E25 §11; IM §9; E25 decision 10].

15.11 The certification plan

15.11.1 Where elan1 stands

elan1 holds no certification. E26 says so first, "so nothing below reads as progress we have made", and the stack1 documents it cites already forbid anyone improving on that answer in a meeting [E26 §02]. The commercial pressure is dated: Crator has held SOC 2 since April 2026 [unverified — from their public material] [E26 §02; IM §10].

15.11.2 ISO/IEC 42001 mapped to what the system already produces

ISO/IEC 42001 certifies an AI management system, not a product. That limit should be stated before anyone asks, "because a certificate implies more than it delivers and the implication will be made for us if we do not make the limit ourselves" [E26 §02].

Requirement	What elan1 already produces	Gap
Scope and AI policy	Dated, versioned scope freezes (E01, E14, E20)	Management-system artefacts

Requirement	What elan1 already produces	Gap
Roles, responsibilities, competence	`owner_role` in every declaration; named author and certifier on every declaration, profile, control, Regulation and pack	—
AI risk assessment and treatment	Refusal register (class, rung, operator action); blast radius and reversal plan per write type; five pack gates	—
AI system impact assessment	Nothing	**Real gap**
Data management and provenance	Declarations name system of record, lexicon and hash; field classification	—
Human oversight	Rung 6, obligations, dual control, F10/F11 sealed on every record — "strongest area by a distance"	—
Logging and traceability	Hash-chained record, signed at L2; offline verifier	—
Supplier management	O04 licence register; stack1 substitution protocol; TSA contracts (E17)	—
Change management	Immutable, append-only, dual-control-certified versions	—
Internal audit programme	Nothing	**Real gap**
Management review	Nothing	**Real gap**
Incident response and corrective action	Runbooks; E16 compromise procedure; supersede-never-withdraw	—

Source: [E26 §02; E26 decision 3]. "Our evidence is unusually strong and our management system barely exists. ISO 42001 certifies the second." The three gaps are process work "a small company can do in a quarter with help" [E26 §02].

15.11.3 SOC 2 mapped to what the system already produces

SOC 2 is an attestation by a CPA firm against the Trust Services Criteria. Security is mandatory; Type II (operating effectiveness over 3–12 months) "is the one buyers mean" [E26 §02].

Criterion area	Existing evidence
Access control	Per-write short-lived credentials; the single writer; dual control on key use (E16)
Change management	Every governed change is a record; declaration certification is a governed write
Monitoring	Nightly integrity run; anchor-lag SLO; paging platform-class refusal codes
Vendor management	O04, E17, substitution protocol
Logical security	E16 key hierarchy; O05 residency baseline; RLS with cross-tenant read test in CI
Processing integrity (elective)	"The criterion our product is made of" — chain, gate results, verifier
Confidentiality (elective)	Field classification; salted payload commitments; payloads absent from bundles

Source: [E26 §02]. SOC 2 fits elan1 better than most early companies "because the evidence is generated by the system rather than collected about it", but it still needs a control environment, an observation period and fees [E26 §02].

15.11.4 The sequencing rule, dates and cost

> **Certify to the standard the first paying evidence customer's auditor names, and not before.** If that customer is Indian or European and the conversation is about AI governance, ISO/IEC 42001 goes first. If it is a US software buyer running a security review, SOC 2 Type I goes first with a Type II window opened behind it. The two are not pursued in parallel on a pre-revenue balance sheet, and neither is started speculatively: "a certificate purchased before a customer asked for it is a purchase made to feel credible rather than to be credible" [E26 §02; E26 decision 2].

The sources give no certification date, only triggers and deadlines. The table collects them.

Item	Date or trigger	Source
Three quotes per standard, before budgeting	Q1 2028 (founder)	[E26 §05 Q1; IM §10]
Does a management-system certificate mislead a product buyer?	Q2 2028	[E26 §05 Q2]
EU legal entity needed for the EU route?	Q1 2028	[E26 §05 Q6]
First paying evidence customer (Gate 3) — names the standard	By 31 December 2028	[E14 §07; IM §16.5.2]
Certification path started: engaged auditor, dated Stage 1 or readiness report	By 30 June 2029 (Gate B item 7)	[IM §16.5.2]
ISO/IEC 42001 elapsed time	6–12 months including Stage 1 and 2 [unverified]	[E26 §02]
SOC 2 Type II observation window	3–12 months after readiness [unverified]	[E26 §02]

On these triggers, the earliest realistic certificate is 2029 and the likely one is 2030. This is a reading of the dates, not a source statement. The plan carries certification and audit as its own expense line: nil in FY2026-27, Rs 0.66 crore in FY2027-28, Rs 1.44 crore in FY2028-29, Rs 1.82 crore in FY2029-30 and Rs 2.38 crore in FY2030-31, Rs 6.30 crore in total [modelled]. A security, compliance and certification function grows from 2 people at March 2027 to 21 by March 2031 [IM §10]. The IM notes that the FY2027-28 spend (US$73,000) is below the published floor for a single ISO/IEC 42001 certification (US$85,000–150,000 for a 50–200 person organisation). The sequencing rule and in-house readiness reconcile the two, and "the risk is that year one is thin" [IM §10].

> **A certificate changes no product claim.** It changes nothing in E01, E14 or E20. When one exists, elan1 may say "we hold X, covering Y, issued on Z", and nothing else. "A deck that places a certification logo beside an assurance-level claim is wrong" [E26 §02; E26 decision 4].

The RBI draft guidance cuts the same way from the buyer's side. A regulated entity remains accountable for third-party models, with independent validation "regardless of vendor certifications", so "no certificate elan1 holds will ever discharge a bank's own validation duty" [IM §10].

15.12 Regulator engagement

15.12.1 The governing rule and the three-sentence claim

> **Rule 1.** "Nothing is claimed to a regulator or a certification body that a verifier cannot demonstrate. Every sentence in any pack we hand over maps to a verifier exit code, a named artefact, or a dated document. A sentence that maps to none of those is cut before the meeting, not defended in it" [E26 §01; E26 decision 1].

The whole regulator-facing claim is three sentences [E26 §03; E26 decision 5]:

1. "Consequential changes made by an AI agent to a system of record pass through a single governed write path, where versioned rules and declared gates decide, a human approves what must be approved, and the refusals are recorded as carefully as the successes."
2. "Every such decision produces a record in an append-only, hash-chained log; at Assurance Level 2 the record is signed with a key held in the customer's residency domain and anchored to independent timestamp authorities."
3. "A reviewer can verify those records themselves, offline, on their own machine, using software they did not get from us — and from Phase 4, against a published specification anyone may implement."

Sentence 2's second clause is true only from L2 GA, and sentence 3's first clause only from L2. Before April 2028 the pack must stop at sentence 1 and the first clause of sentence 2. This follows from the L1 claim boundary [E07], and the pack generator should enforce it (§15.13).

A reviewer — at the RBI, SEBI, an EU authority, a notified body or a certification body — can check the claim without an account, contract or meeting, by six routes. They can check the verifier's digest and release signature; rebuild it from source; read the source; run it air-gapped with a fingerprint from a source other than elan1; run it on the published tampered bundle and watch it exit `1 FAIL_INTEGRITY` naming record and field; or write their own verifier from E25 and run that. "Route 6 is the one that matters … the strongest thing you can do with this is not use our software" [E26 §03].

15.12.2 What elan1 asks for, and never asks for

Four asks, none of them approval: tell elan1 where a clause binding is wrong, and it will publish the correction; tell it what a supervisory reader wants in a report; say whether an open receipt format is useful to supervisory tooling; and name a contact for defect reports [E26 §03; E26 decision 6]. Never asked for: "an endorsement, a no-objection letter, a listing, a pilot they must justify, a waiver, or any statement we could quote in a sales document. A regulator's time is not a marketing asset" [E26 §03].

15.12.3 The engagement plan, by regulator

Audience	Instrument and date	Route in	When	What elan1 brings
Public consultations (SEBI, MeitY, RBI, EU)	As published	Written response on the record	Whenever one is open	Answers to the questions asked, with limits; never a pitch
Customer's own supervisor	—	As the customer's supplier, if asked	When a customer is examined	Verifier, a bundle, the dossier
MCA, via industry bodies and CA institutes	Rule 3(1) and Rule 11(g), in force 1 April 2023	The channel rung1 already sells through	Phase 4 onward	A working session on the MCA audit-trail reading, asking for corrections
RBI	FREE-AI (13 Aug 2025); draft Model Risk Management guidance (24 Jun 2026, still draft)	Through a regulated NBFC or bank customer, or a published response; never cold	Only with a real customer	Human-in-the-loop evidence: rungs 5 and 6, obligations, recorded refusals
SEBI	AI/ML consultation (20 Jun 2025; final circular unconfirmed)	Consultation response first	On a final circular	Actor-kind separation: agent actions apart from human ones
EU authorities, notified bodies	AI Act Art. 50 from 2 Aug 2026; Annex III high-risk 2 Dec 2027; Annex I 2 Aug 2028	Through an EU customer or its notified body; not directly	Only with an EU customer	Art. 12 record-keeping and Art. 14 oversight evidence, and the binding profile
Data Protection Board (DPDP)	Rules notified 13 Nov 2025; main obligations 13 May 2027	Through a customer, or a consultation	2027 onward	Consent and residency as never-skippable gates; erasure by salt destruction — presented as a question, not an answer

Source: [E26 §03; IM §10].

Three further regulators appear in the brief for this chapter, and each has a narrower role. **CERT-In** is an operating obligation, not an engagement target. The CERT-In Directions 2022 require six-hour incident reporting, 180-day log retention in India, and clocks synchronised to NPL or NIC. No brand lets an outsider log in until a named CERT-In responder exists and the six-hour path has been tested once [O05 §06, §10, §12; IM §10]. **MeitY** is reached through consultations. Its India AI Governance Guidelines (November 2025) confirm there is no standalone AI Act and prefer "voluntary compliance, self-certification and sandboxes", and its recommended enterprise action plan names ISO/IEC 42001 adoption as step three of four [IM §10]. A **sandbox** route is an open question: "Is there an Indian route — an institute, a sandbox, an industry body — that gives a reading of the MCA audit-trail binding without a formal submission?" (founder, Q2 2027) [E26 §05 Q4]. No sandbox application is planned in the sources.

> **The dates are a reason the buyer already has, not a reason they will suddenly acquire.** Every line of the engagement plan is opportunistic, none is a launch, and "nothing in the business plan depends on a regulator acting by a particular date. Dates slip, circulars stay unconfirmed, and a strategy that assumes a date has a fuse in it" [E26 §03; E26 decision 9].

15.12.4 Who speaks, and how interactions are recorded

Only the founder speaks to a regulator or supervisor, with the elan1 lead for technical detail — "not sales, not a partner, not an SI, not an investor". Consultation responses are drafted by the elan1 lead and signed by the founder. In a customer's examination the customer leads, and converting the room into a sales conversation is forbidden. A defect report from a regulator is acknowledged within five working days and owned by the founder [E26 §03; E26 decision 7]. The first regulator conversation, and whether it comes through a customer or a consultation, is due to be decided by Q4 2027 [E26 §05 Q3].

Every interaction is recorded as a governed write in elan1's own operator chain: "If we cannot run our own engagement log through our own product, we should not be selling it" [E26 §03; E26 decision 8]. The declaration below is a **build recommendation** for that write type. It uses only mechanisms that exist at L1, so the log can start before L2.

```
write_types:
  - id: operator.regulator_interaction.record
    tenant: elan1-operator
    blast_radius: one_counterparty
    dual_control: true      # founder records; a second named human approves
    irreversible: false
    reversal_plan: operator.regulator_interaction.correct  # correcting record, never an edit
    gates: [identity, counterparty, precondition, retention]
    proof_binding:
      canonicalisation: elan1_lp_v1
      fields: [regulator, kind, occurred_at, attendees, questions_asked, statements_made,
               commitments_made, artefacts_supplied, claim_sentence_refs, follow_up_due]
```

15.13 What elan1 can lawfully claim, and what it cannot

The claims each level permits come from the three scope freezes. The banned words come from E14, E20 and E26 together.

Level	May say (examples)	Source
L1	Every write was checked against versioned rules before it happened; a person approved the ones that needed one; refusals are recorded as carefully as successes; any write can be reversed for thirty days; the log is tamper-evident to you	[E07]
L2	Signed with a key in your residency domain; verifiable air-gapped by a party holding a fingerprint obtained elsewhere; two independent timestamp authorities, one India-based; this bundle claims profile `in-mca-audit-trail` v1 and states what a PASS does not establish; pre-upgrade records are Level 1	[E14 §05]
L3	Published format with an agreeing external implementation; you hold the signing key; a counterparty can verify against an anchor neither party publishes; field-level disclosure to different recipients; A3 earned on a stated record and revocable in a minute	[E20 §05]

Never said, at any level	Why
Tamper-proof; immutable	The correct term is tamper-evident
Court-ready; legally admissible	Admissibility is for a court
Certified, accredited or approved	Only "we hold X, covering Y, issued on Z", once a dated certificate exists
A regulator has accepted, approved, endorsed or reviewed this, including "we are in conversation with"	No regulator has reviewed a profile; a profile is elan1's reading of an obligation
The transaction was genuine; your books are verified	A receipt covers the write path, not commercial reality or direct changes in the target system
An anchor proves when something happened	An anchor is an upper bound: existence, form, no later than

Never said, at any level	Why
We cannot see your data	True only for a customer holding its own keys and salts, and then only of that deployment
The agent is safe because it earned autonomy	Precedent is a record, not a prediction
Standardised; open	"Published" until another organisation's implementation passes the conformance suite

Source: [E14 §05; E20 §05; E26 §03; IM §10]. Three things must never happen: a regulator named as having accepted anything; a claim made in a room that is not in the dossier; a certification logo or regulator's name used as a proxy for the product claim [E26 §03].

These rules are cheap to enforce mechanically, and the corpus already enforces one of them in CI, the CL-1 string check [IM §9]. This memorandum recommends a **claims lint** (**build recommendation**). It runs over every generated artefact: console copy, bundle reports, receipt `limits` statements, the regulator pack template and sales templates in the repository. It fails the build on a banned term, and on any L2 or L3 claim sentence where the tenant's or product's assurance level is lower. Each rule maps to its source row, so a finding can be traced to the decision it enforces.

```
claims_lint:
  banned_terms:            # E14 decision 5; E20 §05; E26 §03
    - {pattern: "(?i)tamper[- ]proof",           source: E14 §05}
    - {pattern: "(?i)\\bimmutable\\b",           source: E14 §05}
    - {pattern: "(?i)court[- ]ready|legally admissible", source: E14 §05}
    - {pattern: "(?i)(approved|endorsed|accepted|reviewed) by (the )?(RBI|SEBI|MCA|regulator)",
       source: E26 §03}
    - {pattern: "(?i)in conversation with (the )?(RBI|SEBI|MCA|regulator)",
       source: IM §10}
    - {pattern: "(?i)we cannot see your data", source: E20 §05,
       allow_if: customer_held_keys_and_salts}
    - {pattern: "(?i)open standard", source: E20 decision 6,
       allow_if: external_conformance_passed}
  level_gated_sentences:
    - {pattern: "(?i)\\bsigned\\b|verif(y|iable) .*air[- ]gapped", min_level: 2,
       source: E07; E14 §05}
    - {pattern: "(?i)you hold the signing key",  min_level: 3, source: E20 §05}
  receipt_conformance:
    - {rule: "L1 artefacts claim no conformance level above CL-1", source: E25 §04; IM §9}
```

15.14 The overall standard and certification plan, and the partner and verifier programme

15.14.1 A dated plan

The working-draft brief for the group's "Standard and Certification Plan" asks for "a dated plan for publishing the Action Receipt spec and holding ISO/IEC 42001" and a costed certification gap list [WD overall Phase 3 01]. No such document exists yet. The table below assembles the dated items the sources do contain into one plan (**build recommendation** for the sequence; each date is sourced).

When	Standard track	Certification track	Source
Q2 2027	—	Indian reading of the MCA binding without formal submission: route decided	[E26 Q4]
Q4 2027	—	First regulator conversation: route decided	[E26 Q3]
Q1 2028	Second v1 implementer and how paid (re-base after entry)	Three quotes per standard; EU entity question	[E25 Q2; E26 Q1, Q6]
Apr 2028	L2 GA: vectors, tampered bundle, free verifier published (the L2 artefacts v1 reuses)	—	[IM §8.1]
Q3–Q4 2028	SCITT anchor usability; editorship destination; profiles' licence decided	—	[E25 Q1, Q3, Q6]

When	Standard track	Certification track	Source
By 31 Dec 2028	Gate 3: evidence on its own contract line	First evidence customer's auditor names the standard	[E14 §07; IM §16.5.2]
On entry condition	v1 draft published; 60-day comment; two external implementations	Readiness work on the three ISO 42001 gaps, or SOC 2 controls	[E20 §02; E25 §07; E26 §02]
By 30 Jun 2029 (Gate B)	v1 published with two named external implementers, one producing a receipt elan1's verifier accepts	Certification path started: engaged auditor, dated Stage 1 or readiness report	[IM §16.5.2]
Oct 2029	L3 entry (modelled)	—	[IM §8.1]
Transfer trigger	Three implementations from two non-elan1 organisations → editorship out within 12 months	—	[E25 decision 8]
End of Phase 4	Falsification check: ≥ 3 competitors emitting receipts without needing elan1's domain work	—	[E25 §11]

15.14.2 The partner and verifier programme

The working draft for the "Partner and Verifier Programme" asks how audit firms, CAs and system integrators are trained and certified to verify and resell. It names partner types (CA firms, audit firms, SIs, GPU operators, platform OEMs), certification levels and what each may claim, revenue share and customer ownership, and support escalation [WD overall Phase 3 02]. The IM supplies the commercial parameters. There are 6 certified partner firms at March 2028, 28 at March 2029 (a Gate B condition), 70 at March 2030 and 140 by March 2031, at Rs 1,50,000 per firm per year. Partner-sourced revenue rises to 45% of external revenue by March 2031 at 18% of list, shown as contra-revenue [modelled]. The partner takes first-line delivery and elan1 second [IM §9; IM §11.6; IM §16.4; IM §16.5.2]. The IM's invitation order is auditors and CA institutes first, then systems integrators and partner firms, then partner platforms and orchestration frameworks [IM §9].

The programme has to reconcile two different meanings of "certified". E25 forbids a conformance certification body or mark in v0: an implementer's conformance is self-asserted with a published vector run, and "we may link to it; we may not certify it" [E25 §08; E25 decision 6]. The IM's "certified partner firms" are a commercial accreditation of firms that sell and deliver elan1's products. The structure below keeps the two apart (**build recommendation**).

Tier	Who	What it means	What the firm may claim	What it may never claim
Verifier (open)	Anyone	Runs the free verifier; no relationship	Nothing about elan1	Endorsement by elan1
Implementer (open)	Anyone building a producer or consumer	Self-asserted conformance with a published vector run	"Conforms to action-receipt/1 at CL-n, suites …, result published at …"	"Certified by elan1"; use of elan1 marks [E25 §05]
Accredited partner — verification	CA and audit firms	Trained on bundle verification, profile limits and the E20 T8 test; annual fee	"Accredited elan1 verification partner", with the date	That any regulator accepted a profile; any banned term
Accredited partner — delivery	SIs, CA firms selling packs	Trained on declarations, packs, onboarding; first-line support; 18% of list	As above, for delivery	Representing elan1's claim to a regulator [E26 decision 7]
Platform partner	Platform OEMs, orchestration frameworks, GPU operators	Emits or consumes receipts; OTel IDs committed via F14	Conformance, as an implementer	Assurance levels, which only elan1's kernel produces

A partner may not speak for elan1 to a regulator in any tier [E26 decision 7]. That rule is written into the partner agreement, not left to training. The terms "certified partner" in the IM and "certification levels" in the working draft should be renamed "accredited" before any external material is issued, so that a buyer cannot read partner accreditation as product or conformance certification (see conflicts).

15.15 Acceptance criteria

#	Check	Evidence	Source
S1	v1 published under CC BY 4.0 / Apache-2.0 / CC0 with vectors and the tampered bundle	Published artefacts, licence text	[E20 §07 item 3; E25 §05]
S2	Every comment answered; two implementations' results published before release	Comment log, results	[E25 §07]
S3	An implementation by another organisation passes the conformance suite	Name, result, permission to cite	[E20 §07 item 4]
S4	A disagreement on any vector is raised as a P0 and resolved in the specification	Defect register	[E25 decision 9]
C1	Certification path started against the standard the first evidence customer's auditor named	Engaged auditor; dated Stage 1 or readiness report	[IM §16.5.2; E26 §02]
R1	A regulator-facing reviewer checks a receipt without elan1's help	The reviewer's own account	[E20 §07 item 11]
R2	Every regulator interaction is present as a governed write in the operator chain	Chain extract	[E26 decision 8]
L1	Claims lint green on all generated artefacts; no banned term; no level-gated sentence below its level	CI run	[E14 §05; E20 §05; E26 §03] (lint: build recommendation)

16. Assurance levels and the upgrade path — L1 GA, L1→L2 and L2→L3

elan1 has three assurance levels: L1 Guarded, L2 Attested and L3 Proven. The build plan funds only the first. L2 and L3 are frozen specifications with dated entry gates [E01; E14; E20; IM §4.7]. This chapter defines each level as a set of engineering facts and as a set of sentences the company may say. It covers the L1 scope freeze and the L1 general-availability (GA) gate, including third-party onboarding. It then covers the L2 scope freeze, the L1→L2 upgrade runbook step by step, the L2→L3 entry condition, the dated gates that tie each level to a funding tranche, and a consolidated table of acceptance tests per level.

The mechanisms behind the higher levels are specified elsewhere in this memorandum: the proof record in Chapter 9, the seal in Chapter 10, the bundle and offline verifier in Chapter 11, key custody and timestamp authorities in Chapter 12, and the Level 3 capabilities in Chapter 14. This chapter treats those mechanisms as components. Its subject is the order in which they are switched on, the evidence required before each switch, and the rule that nothing already written changes when they are.

> **Status, stated first.** As at 23 September 2026 elan1 has no external customer and no governed action has been committed or refused in a production tenant [IM §4.9]. L1 is being built against a single system of record. L2 and L3 do not exist in code. Every date in this chapter is a plan date (MODELLED in the IM) and every duration or latency is a target [IM §4.9; DR-010 in O01].

16.1 Why levels exist, and the rule that makes them honest

Levels exist for two reasons [E07 §01]. The first is structural honesty: Phase 1 ships an unsigned hash chain, and one undifferentiated product would create steady pressure to describe it as the signed chain will eventually work. A named level gives a salesperson, a deck and a contract a word for what is running, and a customer who wants the stronger claim has to buy it rather than infer it. The second is that the buyers differ: the person who wants an agent stopped is not the person who needs a document an examiner will accept.

One rule governs all three levels and is the reason the upgrade path can be a runbook rather than a migration:

> **A level is a property of the proof and the claim, never of the policy.** The same declaration carries across L1, L2 and L3. A tenant that declared `erp.sales_invoice.submit` at L1 does not re-declare it at L2. The seven blocks, the gates, the blast radius, the write types and the proof binding stay the same. What changes is what happens at rung 8 and what may be said about the result. Any design that requires re-declaration on upgrade is rejected on sight [E07 §01, decision 1; IM §4.7].

Three frozen properties make that rule mechanically true [IM §4.7; E01 decision 2; E12]:

1. **The sixteen-field record layout was frozen at L1 although it is unsigned.** L1 writes all sixteen fields into the chained record. L2 adds a signature over the same preimage. Same fields, same encoding, same digest. The Chapter 10 field table and the 64 test vectors are the contract [E01 §03; E12].
2. **At L1 the signature member is absent, not stubbed.** It is not null, not an empty string, not thirty-two zero octets and not the word "unsigned". A stub is a value, values get copied, and a copied stub is eventually read as a signature by something that does not check [IM §4.7; E14 §02 item 1].
3. **A chain written at L1 cannot be retroactively sealed.** Signatures start at the sequence number where signing starts. Records written earlier stay L1 for ever and are labelled so [E01 decision 3; E07 decision 2].

The `seal` call already exists in KernelPort at L1. It returns a hash-chain receipt, so the call shape does not change between Phase 1 and Phase 3; only its contents do [DR-013 in O01; O03 §09].

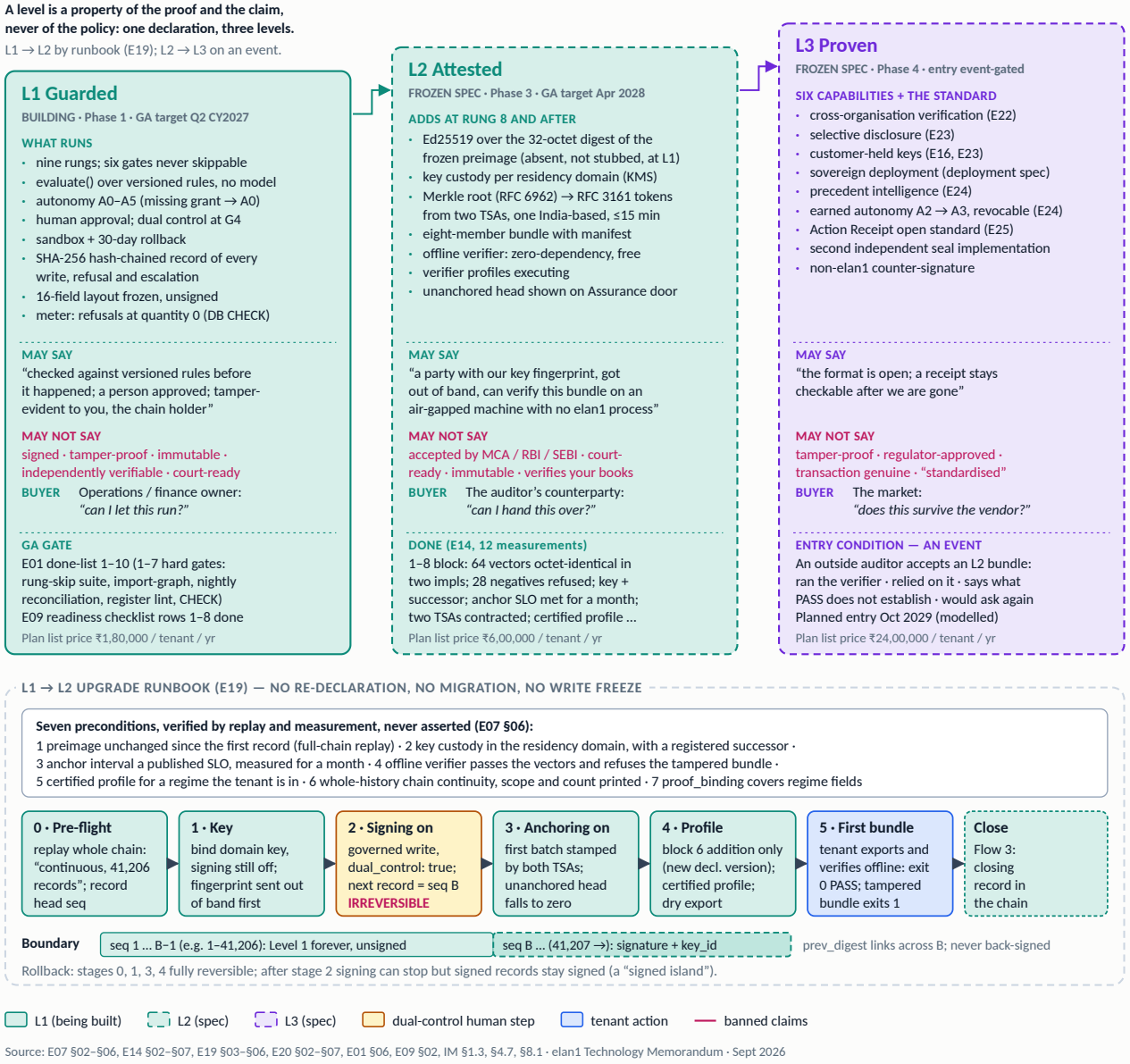
16.2 The three levels and what each may truthfully claim

Figure 25 — The assurance ladder and the upgrade path

L1 BEING BUILT · L2, L3 SPECIFIED, NOT BUILT

Same declaration at every level; what changes is rung 8 onward and what may be said. Dates are targets.

A level is a property of the proof and the claim, never of the policy: one declaration, three levels.
L1 → L2 by runbook (E19); L2 → L3 on an event.



Source: E07 §02–§06, E14 §02–§07, E19 §03–§06, E20 §02–§07, E01 §06, E09 §02, IM §1.3, §4.7, §8.1 · elan1 Technology Memorandum · Sept 2026

Figure 25 — The assurance ladder: what each level adds at rung 8 and after, the claim it permits, the entry gate to the next level, the five upgrade stages from L1 to L2, and the plan dates (all modelled).

Figure 25 shows the ladder. The table below states the engineering content of each level, taken from the definition document and the two scope freezes.

Property	L1 Guarded	L2 Attested	L3 Proven
Nine rungs, six unskippable gates	Yes	Yes	Yes
Human approval, autonomy ladder A0–A5	Yes	Yes	Yes

Property	L1 Guarded	L2 Attested	L3 Proven
Shadow sandbox, 30-day rollback	Yes	Yes	Yes
Record of every write, refusal, escalation	SHA-256 hash chain	Same chain	Same chain
Signature	None (member absent)	Ed25519 over the 32-octet digest of the frozen preimage	Same, plus agreement with an independently written implementation on a production chain
Key custody	No key material anywhere	Managed KMS per residency domain	Adds customer-held keys
Time anchor	None	RFC 3161, Merkle-batched, two TSAs (one India-based), interval ≤15 min as SLO	Adds a non-elan1 counter-signature
Exportable bundle and offline verifier	No	Eight-member bundle; verifier free, as source, zero dependencies	Adds published vectors under an open specification
Verifier profiles	No	Executing, per regime	Against an open standard
Checkable by someone who distrusts elan1	No	Yes, with an out-of-band key fingerprint	Yes, without elan1 existing
Buyer	Operations or finance owner	The auditor's counterparty	The market
Question answered	Can I let this run?	Can I hand this over?	Does this survive the vendor?

Sources: [E07 §02–03; E14 §02; E20 §03; IM §4.7].

16.2.1 Claim boundaries

Every level has sentences that become true on the day it ships and not one day earlier. Conformance test C14 turns the L1 boundary into a build failure in the consumer's CI rather than a style note [E02 C14; E09 §01, §05].

Level	May say (examples)	May not say
L1	"Every write was checked against versioned rules before it happened, a person approved the ones that needed one, refusals are recorded as carefully as successes, and any write can be reversed for thirty days." "Tamper-evident to you."	"Tamper-proof", "signed", "independently verifiable", "court-ready", "immutable"; anything implying an auditor can check without elan1; "this satisfies the MCA rule"
L2	"Signed with a key held in your residency domain, over the same sixteen fields." "Verifiable on an air-gapped machine by a party holding our fingerprint obtained out of band." "Records before your upgrade date are Level 1; the boundary and counts are in the bundle."	All L1 bans except "signed"; "legally admissible"; "accepted by" any regulator; "certified to" ISO/IEC 42001 or NIST AI RMF; "your auditor can verify your books"; "we cannot see your data"
L3	"A stranger's implementation agrees with ours on every vector." "You hold the key; none of your records was signed by us." "A counterparty can verify against a trust anchor neither of us publishes."	All L2 bans; "approved by a regulator"; "certified" beyond a dated certificate's scope; "the transaction was genuine"; "standardised" or "open" before another organisation implements

Sources: [E01 §08; E07 §02, §07; E14 §05; E19 §05; E20 §05–06].

What each level lets a customer say to its own regulator is the clearest statement of the ladder [E07 §07]:

- **L1:** "Our agents cannot write to our books without passing rules we set, a person approves anything material, and we have an unbroken internal record of every attempt including the refusals."
- **L2:** "…and here is a file you can check yourself, on your own machine, without our vendor's help."
- **L3:** "…and the format is open, so you can check it in ten years whether or not that vendor exists."

> **Sell the level the buyer needs, or do not sell.** If a Phase 1 customer is asked for the L2 sentence, the company says it is not there yet and gives the date. Selling L1 to a buyer who needs L2 loses the account at the audit, which is the worst moment to lose it [E07 §07; E01 §04].

16.2.2 Levels and enforcement modes are orthogonal

The assurance level describes the proof; the enforcement mode describes who holds the target credential (Chapter 6). **Enforced** means the kernel holds it or the target accepts only kernel-issued tokens; **Attested advisory** means the caller holds it and can ignore a refusal [E05 §02; O03 §02]. The two combine freely. An L2 signature on an Attested advisory record proves that a decision was made and recorded, not that no other path existed. Both properties are sealed into every record, and records keep the level and mode they were written under, for ever [E09 §09 rule; E19 §05].

16.2.3 The regulatory hooks, and what L1 serves

E07 maps each dated hook (the MCA audit-trail rule, GST e-invoicing, RBI FREE-AI, the DPDP Rules, the EU AI Act) to what each level serves. Its own summary is the point to carry: **L1 serves every hook as an internal control and none of them as independent proof**, and "a strategy that assumes a date is a strategy with a fuse in it" [E07 §05 and rule]. Chapter 18 covers the India obligations in detail.

16.3 L1 Guarded — the scope freeze

The L1 scope freeze exists "so nobody on the build side quietly adds a signature and nobody on the sales side quietly promises one" [E01 §01]. Its frozen sentence is the specification of L1:

> **The L1 frozen sentence.** At L1, elan1 decides whether a write is permitted, forces a human to approve the writes that need it, executes the write through a single writer, records the decision in a hash-chained append-only log, and meters it. The log is tamper-evident. It is not yet signed, not yet anchored to an external time source, and not yet verifiable by a third party who does not trust elan1 [E01 §01].

16.3.1 What L1 contains

L1 is not a cut-down kernel. It is the full kernel with the cryptographic tail removed [E01 §02]. The ten in-scope items are: declaration-driven gates, with the six never-skippable gates (consent, residency, retention, dual control, counterparty, precondition) run in declared order at rung 3; model-free decisions at rung 4; human approval (dual control at G4, approval at A0–A2, irreversible write types never auto-approved); a single writer holding a per-tenant, single-use, ten-minute credential; the shadow sandbox; 30-day rollback by fast or forward path; the hash-chained, insert-only log with a nightly integrity verifier; typed refusals from a closed register, metered at zero by a database CHECK; the meter, with its basis taken from the declared blast radius; and all nine rungs on every write [E01 §02]. Chapters 5, 8, 9 and 19 specify each.

16.3.2 What L1 excludes, and the non-goals

Excluded at L1	Arrives	Why it waits
Ed25519 seal	L2	Key custody is its own security boundary; "building it badly is worse than not building it"
RFC 3161 anchoring	L2	A timestamp over an unsigned root proves nothing anyone cares about
Verifier profiles	L2	A profile is a promise about a regulator's expectations
Evidence export (bundle, verifier, redaction, salted erasure)	L2	Selling evidence before it verifies offline is the one unrecoverable reputational error
Third-party verification	L2	The point of L2
Cross-organisation verification	L3	Needs an open format and a registry that is not elan1's

Sources: [E01 §03; E14 §03 rule and decision 2]. The E01 row that placed "cross-organisation verification" at "L2 onwards" is read, from v3.0, as third-party verification. The protocol between organisations is L3 [E14 decision 2; E20 §09].

The non-goals are as binding as the scope [E01 §07]:

- **No second kernel.** rung1 builds no gates, approval queue, log or meter of its own. A missing behaviour is a kernel change or a declaration change [E01 §07; DR-001].
- **No fast path, batch variant, admin mode or decision cache.** "A cached allow is an allow at an unknown rule version."
- **No warn.** Escalate is the honest version of warn: a human decides and the write waits.
- **No key material anywhere.** Not in a config file, not in a vault, not "just for testing" [E01 §07; E16 §01].
- **No verifier, bundle, redaction profile or erasure salt.** These promise something the chain cannot yet support. (This line conflicts with the O05 erasure flow; Chapter 18 §18.9.3 resolves it and records the conflict.)
- **No regulator-mapping claims.** Dated regimes may be cited as context, not as conformance.

16.3.3 The L1 definition of done

L1 is done when ten measurements hold. Items 1–7 are hard gates; items 8–10 make the claim honest [E01 §06].

#	Done means	Measured by	Status (Sept 2026)
1	Every rung runs on every governed write	Rung-skip test suite; a skipped stage fails CI	Not yet measured
2	No model reachable from rungs 3, 4, 5, 6 or 8	Import-graph check in CI	Not yet measured
3	Zero writes committed without a chained record	Nightly reconciliation; non-zero is a P0	Not yet measured
4	Zero records mutated outside the kernel writer	Same reconciliation	Not yet measured
5	Every refusal replayable	Nightly replay of every reference-tenant refusal; a stage that acquires a clock, random value or undeclared read fails that night	Not yet measured
6	Every refusal code maps to one rung, one class, one operator action	Register lint	Not yet measured
7	Every meter event references a write that exists; every refusal meters zero	Database CHECK plus nightly reconciliation	Not yet measured
8	Rollback restores a sandbox and a live tenant inside 30 days	Quarterly drill, written up and dated	Not yet run
9	Write latency within the published budget	Measured on real hardware; `[unverified]` until then	Modelled only (Chapter 17)
10	rung1 makes a real governed write to a real ERPNext site through the kernel, and nobody can make one around it	Demonstration with the bypass attempt shown failing	Plan date 31 Dec 2026

Sources: [E01 §06; IM §16.5]. The gate calendar dates item 10 at 31 December 2026 ("first real governed write … in a design partner's own site, with the bypass attempt shown failing") and makes the meter reconciliation a gate of its own on 31 January 2027 [IM §16.5]. The record shape must be frozen by 30 November 2026. If it slips, "L1→L2 becomes a migration of customer evidence, not a retrofit" [IM §16.5]. That date is the single most important engineering date for the whole upgrade path in this chapter.

16.4 L1 general availability and third-party onboarding

16.4.1 What GA means

At L1, GA means an outside developer can sign up without talking to elan1, get a sandbox key in a minute, build against the reference tenant, and be issued a production key after a review that elan1 runs and they pass. **GA does not mean self-serve production.** A production key is a credential to somebody's books, and "there is no honest way to issue one without a human reading what the applicant declared" [E09 §01; E05 decision 6]. GA does not change the assurance level. Everything runs at L1 [E09 §01].

The plan dates L1 GA at 30 June 2027 (Q2 CY2027), with the first external paying tenant in July 2027 [IM §16.5, §16.3.1]. The plan says plainly that tenants at 30 June 2027 are zero [IM §16.3.1]. Two kill criteria are read on the same day. rung1 terminates if fewer than ten design partners have run a governed write against their own ERPNext site in production. elan1 terminates if no third party, inside or outside the group, has integrated against KernelPort under a signed agreement (rung R3) [IM §16.3.1, §16.5].

> **The GA rule.** Nothing is available until every row of the readiness checklist reads done, measured rather than asserted. A partial GA — docs shipped, status page not yet — "is the arrangement under which the first external incident is handled by nobody" [E09 §01 rule, decision 2].

16.4.2 The readiness checklist

#	Item	Done means	Measured by
1	Documentation	E02–E05, E12, E13 at a stable versioned URL; a public page per refusal code	CI spider; a 404 fails the build
2	Sandbox keys	Self-serve on e-mail verification; isolated tenant with fixtures in < 60 s	Timed per issuance; p95 published
3	Quotas	Sandbox 10 writes/min, 2 concurrent, 5,000/month; `X-RateLimit-*`; 429 with `Retry-After`	Quarterly load test
4	Status page	Per component; history never edited; chain integrity its own component	A missed nightly integrity job flips it unaided
5	Support path	One address, a response promise per tier, a named human on a rota	First response published monthly
6	Incident process	Severity ladder; a note to affected tenants within five working days for every P0 and P1	A P0 without a note is itself a P1
7	Versioning policy	`/v1`; 12-month sunset with `Deprecation` and `Sunset` headers; codes never reused	Header assertion in contract tests
8	Conformance published	C1–C14 with fixtures and a machine-readable result file	Passes on reference, fails on a deliberately broken build

Source: [E09 §02]. The source names the rows most likely to be deferred under schedule pressure: 4, 6 and 8. They are also the three a serious integrator checks first [E09 §02]. The plan works the checklist through in February 2027, the month of lowest cash in the whole plan (₹4.4 crore closing, against ₹96.59 lakh monthly burn) [IM §16.2.5].

Two build notes (**build recommendation**). The chain-integrity status component should be driven by a dead-man signal, so that a verifier that did not run flips it as surely as one that found a break; this answers E09 open question 5. And the broken reference implementation in row 8 should be a repository fixture with one known defect per test, so each of C1–C14 has a recorded red.

16.4.3 The onboarding flows, as the kernel sees them

E09 specifies five flows from sign-up to first invoice; screen text is in Chapter 8. The kernel-relevant facts are few. **Sign-up** issues `sk_sbx_…` into an isolated tenant whose chain is marked `mode: sandbox` and metered at quantity zero; if fixture provisioning fails, no key is issued, and key material is never recorded anywhere, including by elan1. **Authoring** runs nine semantic rules; a declaration missing a never-skippable gate fails, a simulation with no refusals is returned with "Add a fixture that fails", and registration is itself a governed write under `dual_control: true`, idempotent on `(declaration_id, version, compiled_digest)`. **Conformance** C8 needs a real refusal returned as HTTP 200 with a register code, a rung, a sixteen-field record with zeroed record id and a meter event of quantity 0; a 429 is a transport rejection with no proof entry and no meter event. **Production** is a four-check human review: the enforcement mode is stated in writing and matches `channels[].mode`; every write type has a reversal plan or an `irreversible` flag; commit, refusal and a human-resolved escalation were proved in sandbox; a named human certified the declaration. The resulting `sk_live_…` is scoped to the tenant, the declared write types and the residency domain. **The first invoice** is never issued over an unreconciled period, and it shows the refusal count at ₹0 on its face [E09 §03–07, decisions 4–5].

The first production record is where the claim boundary becomes visible to the customer. Entry `seq 1` carries `prev_digest` of thirty-two zero octets, the declaration and policy versions, the actor kind and `mode: live`, and the Proof screen prints the Phase 1 honesty line: "This record is hash-chained. It is not signed, and it cannot yet be checked by someone who does not trust elan1. That arrives at Assurance Level 2" [E09 §06 step 7].

The conformance result file is the evidence the production decision rests on [E09 decision 3]. The source requires it to be machine-readable but does not fix its shape. A minimal shape (**build recommendation**):

```
{
  "suite": "elan1-conformance", "suite_version": "1.0.0",
  "kernelport": "v1", "assurance_level": 1,
  "tenant": "sbx_7f3c…", "declaration": "decl_2b91…@7",
  "run_at": "2027-05-14T10:22:31Z", "result": "pass", "passed": 14, "of": 14,
  "tests": [
    {"id": "C2", "name": "bypass_attempt", "result": "pass", "attempts": 6,
     "attempts_recorded_seq": [412, 417]},
    {"id": "C8", "name": "refusal_shape", "result": "pass", "http": 200,
     "code": "GATE.PRECONDITION_UNMET", "rung": 3, "record_id_zeroed": true,
     "meter_quantity": 0},
    {"id": "C14", "name": "claim_boundary", "result": "pass",
     "banned_terms_found": []}
  ],
  "digest": "sha256:9d0e…"
}
```

The `digest` is computed over the canonical JSON without the `digest` member and is what elan1 retains against the account [E09 §05 "what gets recorded"].

16.4.4 Who is refused onboarding

Refusing at onboarding is cheap; the same refusal after a production key exists is not. Eight categories are refused by founder decision: an applicant with no declaration; one who will not name the enforcement mode in writing; one whose write types are all irreversible with no approval step; one who needs a per-customer meter basis ("a per-customer kernel arriving through the finance department"); one who asks to disable a never-skippable gate; one who asks for a signature, bundle or regulator claim at L1 ("You need L2 … do not buy this yet"); one in a residency domain elan1 does not run; and one who will not name a production owner for incidents [E09 §08, decision 8]. The L1 refusal sentence in the source carries a date ("It is Q3 2027") that the investment plan has since moved (§16.8).

16.4.5 The promotion review — Attested advisory to Enforced

Promotion is the only review that changes what a consumer may say. It is run **per write type**, never per tenant, after at least 30 days of Attested advisory traffic (a figure the source admits may be "a number we picked", E09 open question 2) [E09 §09, decision 6]. It requires a named mechanism (credential custody or token acceptance); **the bypass proof**, which is C2 run against the real target with every attempt recorded in the chain; no long-lived target credential reachable by a consumer, agent or operator, including CI, admin tooling and break-glass; a rollback drill by both paths; and a new certified declaration version under dual control [E09 §09 steps 1–5; E05 §02]. A promoted write type that later shows an unrecorded write to the target reverts to Attested advisory immediately and the tenant is told [E09 decision 7]. Records keep the mode they were written under, for ever [E09 §09 rule].

16.5 L2 Attested — the scope freeze

E01 froze L1 so nobody added a signature early. E14 freezes L2 "so nobody quietly adds the rest of the roadmap to it" [E14 §01]. Phase 3 ships four things and only four: Ed25519 signing of the frozen sixteen-field record, RFC 3161 anchoring of a batched Merkle root, verifier profiles that execute rather than dry-run, and evidence export as a priced product [E14 §01, decision 1].

> **The L2 frozen sentence.** At L2, elan1 signs each record with a key held in the tenant's residency domain, anchors a Merkle root of those records to two external timestamp authorities at a published interval, and exports a self-contained bundle that a party who does not trust elan1 can verify offline, with no network, no elan1 account and no elan1 code in the verification path. It does not prove the transaction was genuine, it does not prove the rules were adequate, and it has not been accepted by any regulator [E14 §01].

Nothing in the nine rungs changes. Nothing in the declaration changes except an addition to block 6. Every addition sits at rung 8 or after it [E14 §02]: the Ed25519 signature member on `assurance_level: 2` records (absent on L1 records); KMS custody per residency domain with abutting windows; per-tenant Merkle batches anchored by two authorities, one India-based, at a published interval of at most 15 minutes; the eight-member bundle, exported free and without limit; the zero-dependency offline verifier, never priced; verifier profiles whose signature, key-window and anchor checks stop reading `not_applicable`; the unanchored head rendered on the Assurance door; and evidence as its own invoice line (SKU 2) [E14 §02]. Chapters 10–12 and 19 specify each. Cross-organisation verification, selective disclosure, customer-held keys, sovereign deployment and precedent intelligence are named out of scope, as L3 or later [E14 §03, decision 3].

The prohibitions during Phase 3 are engineering constraints, not style [E14 §06]:

- **No back-signing.** A record written before a key existed is never signed afterwards.
- **No seventeenth field.** A vertical that needs to commit to more does it through F14 (`payload_hash`) or does not ship.
- **No shared code between the chain writer and the verifier.** "Recomputing a digest with the function that computed it proves nothing."
- **No bundle shown to a customer before two independent seal implementations agree byte for byte across all 64 vectors**, "under any commercial pressure whatsoever" [E14 §06; E12].
- No priced verifier; no profile certified by its own author; no regulator named as having accepted anything.

16.5.1 The L2 definition of done

L2 is done when twelve measurements hold. Items 1–8 are hard gates and block the level; items 9–10 make the claim honest; items 11–12 decide whether L2 is a business or a capability [E14 §07, decision 6].

#	Done means	How known
1	Two independent seal implementations agree octet for octet on all 64 vectors	Empty diff; disagreement log published with the spec
2	Offline verifier passes every positive vector and refuses all 28 negatives, naming record and field	Vector run in CI with the empty-lockfile check
3	Key custody in the `in` domain, registered successor, abutting windows	Key register plus one production rotation before GA
4	Anchor interval published as an SLO per tier, met over a measured month	Anchor-lag metric, published
5	Two TSAs contracted, one India-based; single-authority outage survived in a drill	Dated drill write-up
6	One verifier profile certified by a certifier who is not its author, passing P1–P12	Profile registry entry with both names
7	A bundle verifies on an air-gapped machine, run by a person who has never seen elan1 code	Witnessed, written up, witness named
8	Chain continuity over a tenant's whole history reads continuous, scope and count printed	`verify` over the full range, not a sample
9	An L1 → L2 upgrade performed on a real tenant, boundary rendered with counts	E19 executed, tenant sign-off
10	A sceptic reproduces the verifier binary from source and matches its digest	Reproducible-build check by an outsider
11	First India evidence pack sold (one customer, one auditor)	Signed contract with evidence on its own line, or an honest statement it did not happen

#	Done means	How known
12	Every L2 sales claim maps to a row in the claim boundary	Founder review before any L2 material issues

Source: [E14 §07].

16.6 The L1 → L2 upgrade runbook

16.6.1 What the runbook proves

The claim that a level is a property of the proof has been in the documents since Phase 1 and has never been executed. E19 is where it becomes true or false. If an upgrade turns out to require a new declaration, a data migration or a write freeze, "the tier design was decoration and we should say so and redesign" [E19 §01]. **The success criterion for the first upgrade is not that it worked, but that it was boring** [E19 §01; IM §4.7].

> **The upgrade frozen sentence.** An L1 → L2 upgrade provisions a key, switches signing on at a named sequence number, switches anchoring on, installs a certified profile, and exports a bundle that verifies offline. The declaration is unchanged except for one addition to block 6. Writes never stop. Records written before the boundary stay Level 1, are labelled Level 1, and are never signed afterwards [E19 §01].

16.6.2 Data migration, and why there is none

Changes	Stays exactly as it was
Records from the boundary carry a `signature` member and a `key_id`	The seven declaration blocks, except one addition to block 6
Records from the boundary acquire an `anchor_ref` once their batch closes	The sixteen fields, their order, encoding, preimage and digest
`assurance_level` on new records reads 2	The nine rungs, their order, the never-skippable gates
Assurance door gains the unanchored-head line and the boundary row	Policy versions, rules, refusal codes, autonomy grants
Bundle export available with a claimed profile; SKU 2 invoice lines	The meter basis (the declared blast radius)
A per-record salt begins to be retained for erasure	`seq` continues from where it was; the chain does not restart. Everything already written, in every respect

Source: [E19 §02]. The absence of a migration follows from three facts established earlier. The record format does not change at L2; a change would be a new domain string and a new verifier [E14 decision 4]. The signature and anchor reference are members stored beside the preimage, not inside it (Chapters 9 and 10). And `seq` is per tenant and monotonic, so the boundary is simply a number. The only schema work is additive columns that are null on L1 rows. The sketch below shows the shape (**build recommendation**; the authoritative L1 table is in Chapter 9):

```
-- additive at L2: nullable members; L1 rows keep NULL for ever
ALTER TABLE proof.record
  ADD COLUMN assurance_level smallint NOT NULL DEFAULT 1
    CHECK (assurance_level IN (1, 2)),
  ADD COLUMN key_id     text  NULL,   -- k_<domain>_<YYYYMMDD>_<4hex>
  ADD COLUMN signature  bytea NULL,   -- 64 octets, pure Ed25519
  ADD COLUMN anchor_ref uuid  NULL,   -- set when the batch closes
  ADD CONSTRAINT sig_iff_l2 CHECK (
    (assurance_level = 1 AND signature IS NULL AND key_id IS NULL) OR
    (assurance_level = 2 AND signature IS NOT NULL AND key_id IS NOT NULL
       AND octet_length(signature) = 64));
-- insert-only: no UPDATE grant exists, so no L1 row can acquire a signature
REVOKE UPDATE, DELETE ON proof.record FROM kernel_chain_writer;
```

`anchor_ref` is set after insert, when the batch closes, so it cannot live on an insert-only row. In the recommended design it lives in a separate insert-only `proof.anchor_membership (tenant_id, seq, batch_id)` table. That keeps `proof.record` free of any UPDATE grant. The `DEFAULT 1` exists only for the migration. From L2 onward the chain writer sets `assurance_level` explicitly on every insert.

The one declaration change. `proof_binding` (block 6) gains, where the bound profile requires it, a `redaction_profile` and any additional `fields[]` the regime needs inside F14. It is published as a new declaration version, rides the release train, and applies to writes from that version onward. "If a proposed upgrade needs a second place, the upgrade is wrong" [E19 §02, decision 2].

16.6.3 Preconditions — the seven, verified not asserted

#	Precondition	Evidence
1	Proof preimage frozen and unchanged since the tenant's first record	Whole chain replayed, every digest recomputed
2	Key custody in the residency domain, rotation window, registered successor	Key register; one production rotation already performed
3	Anchor interval is a published SLO for the tier	Published SLO and a measured month
4	Offline verifier passes the vectors, refusing the tampered bundle	Vector run and reproducible-build check
5	A certified profile for a regime the tenant is in	Registry entry with author and a different certifier
6	Chain continuity over the whole history, scope and count printed	`verify` over the full range
7	`proof_binding` covers every field the regime's bundle needs	New declaration version, validated

Source: [E19 §03; E07 §06, decision 6]. A failure on 1 or 6 "stops everything and becomes a forensic question". Items 2–5 are platform problems and the tenant is not told a date until they are fixed. Item 7 is the only one the customer participates in [E19 §03].

Precondition 1 carries an open scale risk: how long a full-chain replay takes on a million-record tenant (E19 open question 4). The mitigation (**build recommendation**) is for the nightly verifier to persist a checkpoint `(tenant_id, seq, digest, verified_at)`, so the genesis replay runs offline before the window and only the tail from the last checkpoint is verified inside it.

16.6.4 The upgrade, stage by stage

Actors are an upgrade operator, a second named human and the tenant's declaration owner. The trigger is the seven preconditions reading true and the customer having confirmed a window. A further precondition is that the previous night's integrity run completed clean [E19 §04].

Stage	Steps	Reversible?
0 Pre-flight	(1) Full-chain replay prints "continuous, 41,206 records, seq 1–41,206, all 6 write types" or stops. (2) Key register shows an active domain key with a successor. (3) Profile registry entry, author and certifier confirmed. (4) Tenant's current head `seq` recorded; every later step refers to it	Fully: nothing changed
1 Key provisioning	(5) Custodian confirms the domain key is active, or registers one under E16 Flow 1 with dual control. (6) Tenant bound to the domain key: "Key bound · signing off". (7) **Fingerprint published to the tenant's notice contact out of band, before any record is signed**, and the publication recorded	Fully: unbind; no record signed
2 Signing on — the irreversible instant	(8) Enabling signing is a governed write with `dual_control: true`; it climbs the nine rungs and a second named human approves. (9) The chain writer signs the next record; its `seq` is the boundary **B**. Records 1…B−1 are `assurance_level: 1` with no signature member. (10) "Signing on at seq 41,207 · 41,206 records at Level 1 · 0 at Level 2", shown to operator and tenant. (11) First ten signed records verified with the fingerprint the tenant was given, not one from elan1's registry	Partially: signing can be switched off; signed records stay signed
3 Anchoring on	(12) Anchor service enabled; first batch opens, closes at the interval, is stamped by both authorities. (13) Unanchored head watched rising then falling to zero; if not within two intervals, stop and go to E17 Flow 2. (14) `anchors/` for the first batch holds two tokens, two certification paths and revocation data captured at anchor time	Fully: batches already stamped remain valid

Stage	Steps	Reversible?
4 Profile installed	(15) Declaration owner publishes the `proof_binding` version under normal dual-control certification. (16) Certified profile installed; export offers it by name. (17) Dry export over a recent range and verify; a `2 FAIL_PROFILE` here is "expected and useful"	Fully: profile uninstalled; declaration version stays published
5 First bundle	(18) **The tenant exports the bundle themselves**, without elan1's involvement; if they cannot, the upgrade is not finished. (19) Tenant verifies offline: `0 PASS` over the L2 range, and `0 PASS` with both levels and counts over a range spanning the boundary. (20) Tenant runs the published tampered bundle and watches it exit `1`. (21) Operator closes with Flow 3	Not applicable: a bundle handed over cannot be recalled and must not be described as invalid

Sources: [E19 §04, §06; E15 exit codes]. Refusals inside the runbook are kernel refusals, not procedural ones. Enabling signing while the previous night's integrity run is incomplete refuses at `precondition` and shows the last clean night. An enablement approved by its requester refuses with `GATE.DUAL_CONTROL_SAME_ACTOR`. A profile whose `applies_to_records_from` is later than the boundary refuses installation and prints both dates [E19 §04].

Failure behaviour is fail-closed in the right direction. If KMS is unavailable during stage 2, the domain fails closed, writes refuse with a platform-class code, nothing is signed with a stale key, and a page goes out. If both anchor authorities are unavailable during stage 3, writes and signing continue, the unanchored head grows, and the upgrade waits. A `1 FAIL_INTEGRITY` at any stage is a P0 that stops the upgrade [E19 §04; E16 §06].

Every stage is itself a governed write in the tenant's own chain: the key binding, the fingerprint publication, the signing enablement with both named humans, the anchoring enablement, the declaration version, the profile installation and the first export. The customer can read the whole upgrade in their own chain afterwards [E19 §04].

The runbook should exist as data, so that the upgrade console and the audit trail read the same definition (**build recommendation**). Each stage becomes a declared write type in the operator declaration:

```
# operator declaration excerpt — upgrade stages as governed write types
write_types:
  - id: assurance.key.bind          # stage 1
    dual_control: false
    reversal_plan: assurance.key.unbind
    preconditions: [integrity_last_night_clean, domain_key_active_with_successor]
  - id: assurance.fingerprint.publish
    irreversible: true              # a notice sent cannot be unsent
    preconditions: [key_bound, notice_contact_named]
  - id: assurance.signing.enable    # stage 2 — the irreversible instant
    irreversible: true
    dual_control: true              # GATE.DUAL_CONTROL_SAME_ACTOR if requester approves
    preconditions: [fingerprint_published_before_first_signature,
                    integrity_last_night_clean, preconditions_1_to_7_evidenced]
  - id: assurance.anchoring.enable  # stage 3
    reversal_plan: assurance.anchoring.disable
  - id: assurance.profile.install   # stage 4
    reversal_plan: assurance.profile.uninstall
    preconditions: [profile_applies_from_lte_boundary]
  - id: assurance.upgrade.close     # Flow 3
    irreversible: true
    preconditions: [full_chain_verify_continuous, customer_bundle_verified,
                    kms_sign_reconciliation_clean_first_night, unanchored_head_zeroed_once]
```

16.6.5 Re-sealing policy for historical records

The re-sealing policy is one sentence: **there is no re-sealing.** Records 1…B−1 are hash-chained, append-only and verifiable. Their `prev_digest` links them to the signed records after B, so the chain is continuous across the boundary. A verifier reports them as `PASS — unsigned, assurance level 1` and exits 0. They carry the same sixteen fields, so nothing about them is second-class in structure. They are never claimed to be signed, verifiable by a third party without elan1, anchored, eligible for a profile whose `min_assurance_level` is 2, or promoted to Level 2 on the upgrade date [E19 §05].

> **The banned sentence.** Never say, in any document, deck, contract or conversation: "the tenant's records are signed." After an upgrade the true sentence is "records from 12 August 2027 are signed; the 41,206 records before that date are Level 1 and are labelled so." Back-signing a record produced before a key existed "is exactly the fraud this product is built to make impossible" [E19 §05 rule, decision 7].

The bundle enforces the rule mechanically. `chain.json` carries the boundary row, `manifest.json` carries `assurance_levels[]` with the count and `seq` range of each, and the verifier's report prints both. "A customer cannot produce a bundle that hides the boundary, and neither can we" [E19 §05]. A request to back-sign is refused by the kernel. It is not refused by a policy document, because no code path exists that signs a record other than the one rung 8 is currently writing [E19 §06; E16 §08].

16.6.6 Rollback, and the signed island

A rollback before stage 2 leaves no trace beyond its own governed records. A rollback after stage 2 leaves a **signed island**: 1…B−1 at Level 1, B…C at Level 2, C+1… at Level 1 again. "This is ugly and it is honest." The chain is continuous, every record verifies, and `chain.json` carries both boundaries with counts [E19 §06, decision 6]. The rollback procedure is:

1. Switch signing off. The next record carries no signature member, and the Assurance door shows all three segments with counts.
2. Write the reason into the tenant's chain as a governed write ("We stopped signing on 14 August because the anchor pair could not be contracted").
3. Notify the customer in writing within 24 hours, naming the three ranges and what is true of each.
4. Never sign the trailing segment later to "tidy it up".

If signing cannot be switched off because the control plane is unavailable, the tenant keeps signing. That is "a degraded-but-safe state and is not an emergency" [E19 §06]. The source leaves one commercial question open: whether a customer may ask for signing to be switched off permanently to reduce cost, and what the resulting three-segment chain does to their profile claims (E19 open question 5).

16.6.7 Customer communication

Three messages are written before the upgrade, not during it [E19 §07]:

When	Message	Content
≥ 14 days before	Notice to the named contact	Window; writes do not stop; one additive declaration version; the fingerprint and its channel; "records written before the upgrade will remain Level 1 permanently and will not be signed retrospectively"
On the day	Boundary confirmation	Boundary `seq`, timestamp, counts on each side
After	First bundle plus a one-page auditor note	How to obtain the verifier from a source other than elan1; what a PASS means and does not. "The note does not sell anything"

Two wording rules apply. Marketing may say "upgraded to Level 2 Attested" about the tenant, never about the tenant's history. And no upgrade is described as making anything "compliant"; the profile's limits statement is the only sentence about a regime anyone may repeat [E19 §07, decision 10].

16.6.8 Closing the upgrade

The upgrade is closed by a governed record in the tenant's own chain, "and by nothing else — not by a ticket, not by an email, not by an operator's opinion" [E19 §08, decision 9]. Before that record is written, seven checks must pass: `verify` over `seq` 1 to head reads continuous with both segments printed; the customer verifies a boundary-spanning bundle and a wholly-L2 bundle against the installed profile; the nightly reconciliation between KMS `Sign` calls and signed records is clean for the first full night; the unanchored head has returned to zero within the interval at least once; and the boundary row is visible on the Assurance door without asking. Any failure reopens the upgrade. "There is no partial close and no 'closed

with follow-ups'" [E19 §08]. The closing record carries the boundary `seq` and timestamp, the counts, the profile, the digest of the bundle the customer verified, and both named humans. It is the first thing an auditor should be shown when they ask when the tenant went to Level 2 [E19 §08].

The plan schedules the first execution on a real tenant, with sign-off, in January–March 2028, ahead of L2 GA on 30 April 2028, because E14 done-item 9 requires it [IM §16.3; E14 §07]. The source leaves two staffing questions open that the build team inherits. Who is the second named human for stage 2 "in a two-person company on the day of the first upgrade"? (E19 open question 2, due 30 April 2027.) And is fourteen days' notice right for a change that breaks nothing? (Open question 1.)

16.7 L2 → L3 — the entry gate

L3 engineering does not begin on a date. It begins on an event [E20 §02].

> **The L3 entry condition.** No L3 capability is designed, staffed or promised until an auditor who does not work for elan1 and is not paid by elan1 has accepted an L2 evidence bundle as evidence in a real engagement, and has said so in writing the company may keep. That includes design work, and it includes a slide [E20 §02, §08, decision 1].

"Accepted" is measured by four tests, all of which must hold [E20 §02]:

#	Test	Evidence
1	The auditor ran the offline verifier themselves, on their own machine	Their run and exit code, dated
2	They relied on the bundle for a working paper or a finding	The reference in their file, or a written statement
3	They can state, unprompted, what a PASS does not establish	E18 test T8 applied to a person outside the design-partner relationship
4	They would ask for one again	In writing

If Gate 3 (a signed contract with evidence on its own line, end Q4 2028) closes without this, L3 is not started. The consequence is written down in advance: elan1 "remains an L2 business with a kernel, a pack and a smaller ambition. That is a real outcome and not a failure of nerve" [E20 §02, decision 8; IM §4.9].

Once the condition is met, L3 adds six capabilities plus the Action Receipt standard: cross-organisation verification, selective disclosure, customer-held keys, sovereign deployment (a deployment specification, not a proof capability), precedent intelligence and earned autonomy [E20 §03]. The sixteen fields do not change and no model becomes reachable at rungs 3, 4, 5, 6 or 8 [E20 §03, decision 3]. Chapter 14 specifies the capabilities. Of the twelve L3 done-items, items 1–3 block the level: the entry condition; a second seal implementation, by a team with no access to the first's source, agreeing on all 64 vectors and a full production chain; and the specification published under terms a competitor can implement [E20 §07].

The upgrade from L2 to L3 is not a per-tenant runbook in the way L1 → L2 is. The L3 capabilities are platform capabilities that a tenant adopts one at a time. Customer-held keys is the only one that changes a tenant's records. It is introduced by the same boundary mechanism as E19: a new `key_id` window begins at a named `seq`, earlier records keep their elan1-held signatures, and the Assurance door renders the custody boundary with counts (**build recommendation**, following E19 §05 and E16 §05). No L2 bundle is ever silently re-issued as an L3 receipt [E20 §08].

16.8 Dates and gates, tied to the funding tranches

The Series A is ₹180 crore in three tranches, each released against observable gates with no partial tranche [IM §19.1, §16.5]. The table shows the assurance-level gates and their tranche consequence. Every date is a plan date (MODELLED).

Date	Gate	Observable condition	If missed
30 Nov 2026	Record shape frozen	Sixteen-field preimage frozen; Declaration schema major published	L1→L2 becomes a migration of customer evidence

Date	Gate	Observable condition	If missed
31 Dec 2026	First real governed write	End-to-end write in a design partner's site, bypass shown failing	rung1 kill criterion 5 fires
31 Mar 2027	**Tranche 1 — ₹63.0 cr**	Funds received	The plan stops
30 Jun 2027	L1 GA; R3	All eight E09 rows done and measured; a third party integrated against KernelPort under a signed agreement	First external tenant slips; elan1 kill criterion 5 fires
31 Aug 2027	Key custody	Domain signing key, successor, one rotation	Signing has nowhere defensible to live
30 Sep 2027	L2 signing live	Two seal implementations agree on 64 vectors; diff empty	Gate 1 moves a quarter
31 Jan 2028	Gate 1	A paying customer's write verifies air-gapped, witness named, fingerprint out of band	Gate 1 moves to Phase 4
30 Apr 2028	L2 GA	E14 done-items 1–8 closed and dated	L2 cannot be quoted; SKU 2 cannot be invoiced
31 Oct 2028	**Tranche 2 — ₹72.0 cr (Gate A)**	Eight conditions incl. L2 GA since 30 Apr 2028 and cost per action measured ≤ ₹0.338	Hiring freeze; packs cut to two
31 Dec 2028	Gate 3	A signed contract with evidence on its own line	L3 is not started
31 Jul 2029	**Tranche 3 — ₹45.0 cr (Gate B)**	Eight conditions incl. the L3 entry condition met by 30 Jun 2029 and Action Receipt v1 published	L3 is not staffed
31 Oct 2029	L3 entry	Four-test entry condition met in writing	elan1 remains an L2 business

Sources: [IM §16.5, §16.5.1, §16.5.2, §16.3, §19.2]. Tranche 1 buys L1 GA and the L2 build, including the two independent seal implementations. Tranche 2 "converts a control product into an assurance product" through L2 GA and the evidence SKU. Tranche 3 buys L3 entry "once the external acceptance condition is met" [IM §19.2].

> **Date conflict, resolved for this memorandum.** E01, E07, E09 and E11 describe L2 as "Q3 2027", and E09 gives that date to refused applicants. The IM gate calendar has L2 signing live on 30 September 2027 and **L2 general availability on 30 April 2028**. This memorandum reads Q3 2027 as the start of the L2 build and signing on internal tenants, and 30 April 2028 as the date a customer may be told. Customer-facing copy must use April 2028 (see the conflicts block).

16.9 Acceptance tests per level

Each level has a small number of tests that decide it. The table consolidates them. Details are in the chapters cited.

Level	Blocking acceptance tests	Chapter
L1 kernel	E01 done-items 1–7 (rung-skip, import graph, zero unrecorded writes, zero out-of-writer mutations, refusal replay, register lint, refusal-meters-zero)	21, 22
L1 consumer	C1–C14 against the reference tenant; result file attached to the production request	6
L1 GA	E09 readiness rows 1–8, measured	this chapter
Enforced	C2 against the real target; no reachable long-lived credential; rollback drill by both paths	6
L2	E14 done-items 1–8	10–12
L1 → L2 per tenant	E19 seven preconditions; Flow 3 closing checks; tampered bundle exits 1 on the tenant's machine	this chapter
L3 entry / L3	E20 four-test acceptance; done-items 1–3 blocking, 10 guarding the model boundary	14, 15

Sources: [E01 §06; E02 C1–C14; E09 §02, §09; E14 §07; E19 §03, §08; E20 §02, §07]. One mutation rule applies to every row (**build recommendation**): each blocking test must have been seen red against a deliberately broken input before its green is admitted as evidence. The conformance suite's "fails on a deliberately broken one" clause [E09 §02 row 8] and the vector set's 28 negatives [E14 §07] already require this for two rows. Chapter 21 extends it to the rest.

16.10 Open questions carried by this chapter

#	Question	Owner	By	Source
1	Does an L1 record without a signature satisfy what a CA expects from the MCA audit-trail rule, or does the CA channel need L2 sooner?	Founder, with two CAs	31 Oct 2026 (E01) / 31 Dec 2026 (E11)	E01 OQ1; E11 OQ1
2	Does an external L1 SKU exist before L2, or is elan1 held internal and sold only through rung1?	Founder	31 Jan 2027	E01 OQ5
3	Is there an L1.5 (anchoring without signing), and does the runbook make it easy enough to be asked for?	elan1 lead	31 Jul 2027	E07 OQ4; E19 OQ3
4	Measured duration of a million-record full-chain replay	Kernel engineer	30 Sep 2027	E19 OQ4
5	Does a tenant on a shared residency domain need its own signing key before L2 GA?	elan1 lead	31 Jul 2027	E14 OQ5
6	Who is the air-gapped witness for L2 done-item 7?	Founder	30 Apr 2027	E14 OQ3

17. Runtime and deployment — stack1, hosting profiles and sovereign deployment

The elan1 kernel is not infrastructure. It runs on infrastructure it does not own. stack1, a separate venture under common founder control, supplies that runtime at arm's length through versioned `fusion.*/v1` contracts [IM §2.2; S02 §01; S03 §2]. This chapter covers what the kernel takes from stack1 and what it never takes; the kernel's own deployable units and where each runs; durable execution, secrets and observability; the hosting profiles the kernel is offered in; sovereign deployment, which here is a deployment profile and nothing more; tenancy and isolation; the capacity and cost model; scale and multiple regions; the substitution protocol as it touches the kernel; environments, infrastructure as code, release and rollback; the latency budget on the write path; and what happens when a dependency fails.

Status. Designed, not built (DR-010 in O01). stack1 has no external customer and one region [S17 §01]. No kernel latency, cost or capacity figure in this chapter has been measured under a real tenant's load. Each is a target or a modelled value and is labelled as such [E04 §02; S16 §01; O03 §08].

The boundary in one line. "The elan1 layer is not part of the stack1 runtime." stack1 owns the model gateway, observability, durable execution, the sandbox pool, the deploy pipeline, rollback, secrets and metering plumbing. The kernel runs on that runtime as a consumer. It is never imported into it, never linked into it and never deployed as part of it. "Infrastructure gets swapped; the write path must not" [S03 §2, decision 1].

17.1 What elan1 takes from stack1

17.1.1 The five contracts

In Phase 1 stack1 ships five of its ten contracts to exactly two consumers, rung1 and elan1, and sells to nobody [S01; S02 §1]. The same five later become the reference profile **P4 — elan1 Layer Host**. "The profile was not designed; it was observed" [S15 §02, decision 2].

Contract	stack1 guarantees	What the kernel uses it for	Kernel must not
`fusion.run/v1` (C8, Temporal)	Durable, resumable, idempotent execution; a namespace per tenant; drain, cancel and terminate as distinct operations	The governed-write workflow; approval waits as durable signals; nightly integrity, replay and reconciliation jobs; at L2 the anchor batch job	Run any write to a customer's books on anything else [S04 §01; DR-002]
`fusion.trace/v1` (C6)	OTel/OpenInference span ingest, 90-day default retention	Per-rung spans; the trace a proof record binds to; incident review	Depend on Langfuse's own UI routes or schema
`fusion.tools/v1` (C7)	MCP server registry, per-call authorisation, isolation for community-trust servers, call ledger	Registering the kernel's own MCP server (Mode B surface)	Register a tool outside the registry
`fusion.guard/v1` (C10)	PII detection and redaction, provenance tagging, injection defence	Provenance (`tenant_ingested` , `third_party`) as an *input* to the gate check	Treat a guard verdict as an approval [S03 decision 4]
`fusion.gateway/v1` (C2)	OpenAI-shaped model access, per-tenant keys and budgets, failover	None on the decision path. The kernel calls no model at rungs 3, 4, 5, 6 or 8. Consumers' agents use the gateway upstream of the kernel	Place any gateway call on the decision path [E01 §06 item 2]

Sources: [S02 §1; S03 §1; S15 §02–03; E01 §02]. Beyond these five, the kernel relies on three stack1 capabilities that are deliberately **not substitutable**, because they are the tenancy and attribution invariants themselves: secrets (OpenBao, one path per tenant, sealed references only), metering plumbing (the attribution triple, the transactional outbox, nightly

reconciliation), and the deploy pipeline (the Promoter, which alone holds apply credentials) [S03 §1]. It also uses the sandbox pool, which S03 lists as an operational service rather than a `fusion.*` surface in Phase 1 [S03 §1]. The kernel's own shadow sandbox, the per-write apply before the live apply, is an elan1 mechanism, not the stack1 pool [E05 §05; E01 §02 item 5]. The build must keep the two apart.

17.1.2 Terms the kernel receives

The internal platform contract is written "so that rung1 and elan1 could sign it as written" [S02 §01]. The terms that bind the kernel's own operation are these:

Term	Commitment	Source
Availability	99.5% monthly per contract, measured at `fusion-edge` by per-minute probe and real error rate; provider outages not excluded	S02 §3
Contract SLOs	Contract call p95 < 40 ms (excluding component work); budget reservation p99 < 15 ms; meter lag p99 < 30 s; blocking conformance 100% with no error budget	S02 §3
Notice	Minor version 2 working days; major 15; substitution 10 plus a side-by-side proposal; `/v1`→`/v2` one quarter's notice and two quarters concurrent	S02 §4
Change windows	Tue and Thu 21:00–23:00 IST; frozen two working days either side of rung1-nominated statutory filing dates and the last three working days of each month	S02 §5
Capacity	Consumers declare a quarterly forecast including "elan1's approval bursts"; stack1 provisions to peak plus 40%; beyond that the reservation refuses	S02 §9
Not covered	**The elan1 layer**: gates, approval, refusal semantics, proof records and rollback of a business write	S02 §8, decision 7

The last row matters most. stack1 "does not make them correct and is not liable when they are not" [S02 §8]. The kernel's own correctness, availability and incident obligations sit in elan1's GA readiness list (Chapter 16) and operations (Chapter 22), not in the runtime contract.

17.1.3 Boundary rules that bind the kernel's build

S03's eight boundary rules are CI and deploy-time artefacts, not conventions [S03 §3]. Four bind the kernel directly:

- **B1/B2 — one-way dependency.** No `stack1/` package imports an `elan1/` package, and no elan1 package name appears in a stack1 manifest, image layer or Helm chart. The kernel depends on stack1; never the reverse [S03 §3, decision 2].
- **B3 — reserved words.** `approve`, `allow`, `gate`, `refuse`, `proof` and `seal` are reserved in stack1 symbol and endpoint names. S03 asks whether `verify` and `scope` should join once KernelPort is public (S03 open question 4). The boundary "erodes through naming before it erodes through code" [S03 §3].
- **B5 — plane direction.** The stack1 control plane never calls the tenant plane. The tenant plane writes to the evidence plane only through the transactional outbox [S03 §3].
- **B8 — absent grants.** No stack1 service may UPDATE or DELETE evidence rows. The permission does not exist to be misused [S03 §3].

The kernel adds its own mirror rule, the KernelPort import-graph check: nothing above the kernel boundary imports a kernel type, and no module reachable from rungs 3–8 imports a model client (conformance test C1; E01 done-item 2) [E02 C1; O01 plane table]. Chapter 21 specifies both checks as CI jobs.

17.2 The kernel's deployable units

The sources fix the kernel's behaviour and its boundary with stack1. They do not fix a process layout. The layout below is a **build recommendation**. It keeps the invariants of Chapters 4, 5 and 9 true by construction, and it maps each unit to an architectural plane [O01 planes P0–P8].

Unit	Responsibility	Plane	Runs as	Level
`kernel-api`	KernelPort v1 over HTTP; rungs 0b–6 and 8–9 orchestration; refusal register	P3	Stateless service, N replicas per residency domain	L1
`kernel-mcp`	Mode B MCP server (`ro`, `stage`, `act` tools), registered in `fusion.tools/v1`	P3 surface	Stateless service	L1
`single-writer`	Rung 7: mint the ten-minute credential, shadow apply, gate re-run, live apply, inverse capture	P3	Temporal activity worker, **one lease-holder per tenant**	L1
`chain-writer`	Rung 8: append the sixteen-field record from the outbox in `seq` order	P3	One lease-holder per tenant; at L2 the only principal allowed to call KMS `Sign`	L1 (L2 signs)
`registry`	Declarations, versions, compiled digests, lexicon; append-only	P4	Postgres schema plus service	L1
`approvals`	Obligations, dual control, step-up; signals the workflow	P3/P5	Service; approval arrives as a Temporal signal [S03 decision 6]	L1
`meter`	One event per governed action; refusal at quantity 0 by CHECK; nightly two-way reconciliation	P3	Outbox consumer plus nightly workflow	L1
`console`	Studio and the four doors	P5	Web app, control token only	L1
`integrity`	Nightly chain verify, refusal replay, dead-man signal to the status page	P3	Scheduled Temporal workflow	L1
`salt-store`	Per-record salts for F14 (see Chapter 18)	P0	Separate database and grants	L1 (see conflicts)
`anchor`, `bundle`	Merkle batching and RFC 3161 tokens; bundle export	P3	Workflow and service	L2
Offline verifier	Zero-dependency binary, shipped as source	—	**Never deployed by elan1**; runs on the reader's machine	L2

Three placement rules follow from the sources and should be written down as deployment tests.

1. **One writer per tenant is a lease, not a hope.** "Exactly one process mutates the target per tenant" [IM §6.10]. Kubernetes replicas do not give that. The single writer and the chain writer each take a per-tenant lease, for example a Postgres advisory lock keyed on `tenant_id` held by a fenced worker, and a writer that loses its lease must stop before its next side effect. A lost writer lock restores the snapshot and pages [IM §6.10]. The lease is fenced by a monotonic token checked by the target adapter and the chain insert, so a paused process that wakes after losing the lease cannot write (**build recommendation**).
2. **The credential never enters a consumer process.** In Enforced mode the target credential sits in the per-tenant credential broker and is minted per call [O03 §06; O05 §04]. Only `single-writer` can ask the broker to mint. The broker's material lives in the tenant's OpenBao path, and consumers receive sealed references only [S12 §3].
3. **The control plane has no route to the record store.** Control-class paths (`/v1/declarations`, `/v1/actors`, `/v1/tenants`) have no read path to any record store, "enforced at the network layer" [E05 §04; O05 §03]. `console` and `registry` therefore run in a network segment with no route to `proof.*` or to target systems.

17.2.1 Mode A and Mode B, as deployments

Mode A is the SDK in-process; Mode B is MCP or HTTP [O03 §02; E05 §01]. E05 describes Mode A as consumers "linking `@elan1/kernel` behind KernelPort" [E05 §01]. Read literally, that would put rung execution inside the consumer's process and the consumer's trust boundary. It would also let several consumer processes act as writers, which breaks the single-writer invariant. This memorandum recommends that **the Mode A SDK be an in-process client library**. It carries the six call signatures, the typed objects and local validation. Every rung executes in `kernel-api` and `single-writer`, co-located in the same cell as the consumer so that the extra hop stays inside the O03 Mode A allowance of ≤ 15 ms (**build recommendation**) [O03 §02]. What makes Mode A Enforced is where the credential lives, not where the code runs [E05 §02]. Mode B is the same service reached over MCP or HTTPS. It is Attested advisory unless the target accepts only kernel-issued tokens [O03 §02].

S03 forbids stack1 from importing any elan1 package (B1). E05 names stack1 as an in-process Mode A consumer. This memorandum treats stack1 tooling as a Mode B consumer so that B1 holds. The conflict is recorded at the end of the chapter.

17.3 Durable execution — the governed write as a workflow

Every governed write runs inside a Temporal workflow [DR-002 in O01; S04 §01]. The reason is specific. A replay-style runner re-runs a step whose effect has already landed. If the step was "POST a purchase invoice to ERPNext" and the response was lost, the retry posts it again, and "the AI paid a supplier twice" [S04 §02]. Three things must hold to prevent it, and only the first is about the engine: the engine separates decision from effect, so recovery replays the decision and never the effect; every write carries an idempotency key the target honours; and the approval is durable state inside the run, not a callback [S04 §02].

The workflow mirrors the nine rungs. Deterministic rungs run in workflow code or in pure activities. Every side effect is an activity with an idempotency key (**build recommendation**; names illustrative):

```
@workflow.defn
class GovernedWrite:
    @workflow.run
    async def run(self, intent: Intent) -> Outcome:
        # rungs 0b-6: pure, clock-free, versioned; replay yields the same decision
        d = await workflow.execute_activity(evaluate_rungs_0b_to_6, intent,
                                            start_to_close_timeout=timedelta(seconds=2))
        if d.kind == "refuse":
            return await self._prove_and_meter(intent, d)        # rung 8, 9; qty 0
        if d.kind == "escalate":
            ok = await workflow.wait_condition(lambda: self.decision is not None,
                                               timeout=d.approval_window)
            if not ok or self.decision.declined:
                return await self._prove_and_meter(intent, d.expired_or_declined())
        # rung 7: one activity, one idempotency key, fenced single-writer lease
        act = await workflow.execute_activity(single_writer_apply,
            ActArgs(intent, key=intent.idempotency_key, fence=d.writer_fence),
            retry_policy=RetryPolicy(maximum_attempts=5,
                                     non_retryable_error_types=["SandboxDiverged"]))
        return await self._prove_and_meter(intent, act)

    @workflow.signal
    def approval(self, decision: ApprovalDecision) -> None:
        self.decision = decision          # durable: part of the run's history
```

Four properties of this shape are load-bearing. The decision in `evaluate_rungs_0b_to_6` is resolved at a pinned declaration and policy version and acquires no clock or random value, so the nightly replay reproduces it (E01 done-item 5). The approval is a signal, so a worker restart during a three-day wait loses nothing [S03 decision 6; S15 §08 P4 K5 on C8]. `single_writer_apply` is the only activity that touches the target, and it is idempotent on `intent.idempotency_key`. Whether ERPNext honours such a key natively or rung1's adapter must de-duplicate is an open question (S04 open question 5). And the proof append is driven by the outbox row committed in the act transaction, so a failed append retries in order with "no lost write and no gap in the sequence" [IM §6.10].

S04 records the costs plainly. Temporal is the heaviest item stack1 runs, and every air-gapped install inherits it. The mitigation is Postgres visibility instead of Elasticsearch and a namespace per tenant instead of a cluster per tenant [S04 §05]. `fusion.run/v1` cannot be shadowed during a substitution, so it is replaced by a 200-run replay corpus and always requires the consumer's own approval [S10 §04, decision 3]. For the kernel that means a Temporal-to-Prefect substitution is the most dangerous change stack1 can make beneath it. The kernel's H1 approver should require the governed-write replay corpus, including approval waits across restarts, to pass before approving (**build recommendation**).

17.4 Secrets, credentials and observability

Secrets. Every secret lives in OpenBao, one path per tenant. No secret sits in an environment file, and a pre-commit hook and a CI scan enforce it. A consumer never receives a secret value, only a sealed reference [S12 §3; O05 §04]. The kernel's target credentials are minted per call with a ten-minute life, because "a long-lived credential ends up in a log" [S12 §3]. At L1 there is **no signing key anywhere**, not even in OpenBao [E01 §07; E16 §01]. From L2 the domain signing key lives in a managed KMS with `Sign` granted to exactly one principal, the chain writer's compute identity, and never in OpenBao or any software store [E16 §03]. Chapter 18 covers keys and the conflict over per-tenant versus per-domain signing keys.

Observability. The kernel emits one OTel span per rung, with the rung, refusal code, declaration version and policy version as attributes, through `fusion.trace/v1` (**build recommendation**). Refusals are always logged in full with code, policy id, version and `as_of`: "refusals are evidence, not noise" [S12 §5; O05 §06]. Two traps need design attention. First, trace retention is 90 days by default, while proof records are retained for years. S02 asks whether the kernel needs its own longer-lived evidence store (S02 open question 4). This memorandum answers that the proof record must be self-sufficient: it must never require a trace span to be readable, and the binding to a trace is a convenience for incident review, not part of the evidence (**build recommendation**). Second, the Langfuse-to-Opik worked example shows the kernel's proof-binding query silently returning fewer rows because Opik renamed a span attribute [S10 §07 step 06]. The binding should be keyed on the span and trace identifiers stored in the kernel's own tables, not on attribute names an implementation may change. A `fusion.trace/v1` substitution is an evidence change and always needs the kernel's fresh approval [S10 §08].

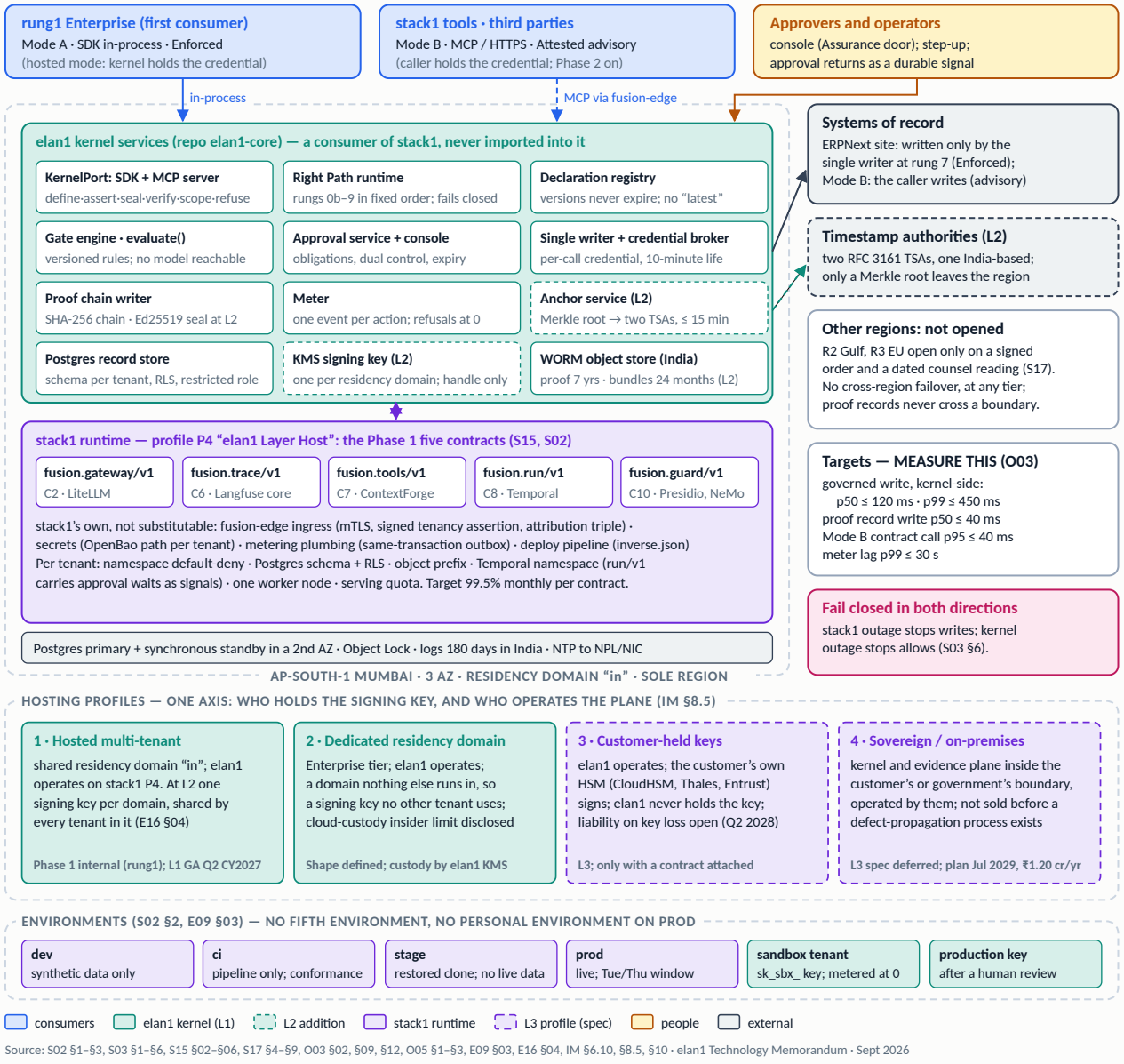
17.5 Hosting profiles for the kernel

S15's four profiles (P1 Private Assistant, P2 Document Intelligence, P3 Agent Automation, P4 elan1 Layer Host) describe what stack1 runs. For the kernel only P4 matters. It is the Phase 1 five contracts, 3–6 worker nodes, zero GPUs, and "the smallest profile and the one whose failure is least tolerable" [S15 §02, §04–05]. The question this section answers is different: **where the kernel and its evidence plane run, and who operates and holds keys for them.** The sources give three topologies on one axis, who holds the signing key and who operates the plane [IM §8.5], and a deployment guide outline naming multi-tenant, customer VPC and on-premises [WD elan1 Phase 3 04]. Combining them gives five hosting profiles.

Figure 26 — Runtime and deployment topology

L1 BEING BUILT · TARGETS, NOT MEASURED

The kernel runs on stack1 as a consumer of five fusion.*/v1 contracts, in one India region; four hosting profiles.



Source: S02 §1–§3, S03 §1–§6, S15 §02–§06, S17 §4–§9, O03 §02, §09, §12, O05 §1–§3, E09 §03, E16 §04, IM §6.10, §8.5, §10 · elan1 Technology Memorandum · Sept 2026

Figure 26 — Runtime and deployment: the kernel's units on stack1's P4 contract set inside the ap-south-1 residency boundary, the three planes and their absent routes, and the five hosting profiles from shared hosted to air-gapped, with who operates each and who holds the key.

#	Profile	Operated by	Where	Signing key (L2+)	Levels offered	Plan status
H1	Hosted, shared residency domain	elan1 on stack1 (P4)	ap-south-1, three AZs; the `in` domain	One domain key shared by every tenant in the domain	L1 now; L2 from GA	The default; the only profile at L1 GA [E16 §04; O05 §02]
H2	Dedicated residency domain	elan1	A domain nothing else runs in, in the tenant's region	A key nothing else's records used; custody in elan1's managed KMS	L1, L2	"Exists today as the Enterprise shape" in the design [IM §8.5]
H3	Customer-held keys, elan1-operated	elan1	As H1 or H2	Customer's HSM (CloudHSM, or their own Thales or Entrust device)	L3	"When a customer asks for it with a contract attached, not before" [IM §8.5; E16 §03]

#	Profile	Operated by	Where	Signing key (L2+)	Levels offered	Plan status
H4	Customer-hosted (customer VPC or on-premises) — sovereign	The customer or a government	Inside their boundary	Theirs; salts, chain, anchors and registry inside their perimeter	L1, L2 (L3 features by deal)	Deferred deployment specification, Phase 4; whether to build it is open [E20 §03, OQ4]
H5	Air-gapped (a variant of H4)	The customer	No network path to elan1	Theirs	L1; L2 only with a reachable timestamp path (below)	Not planned as its own row; constrains H4's packaging

Sources: [IM §8.5; E14 §03; E16 §03–04; E20 §03; WD elan1 Phase 3 04; S15]. Four properties hold in every profile, because they are properties of the kernel and not of the hosting: the nine rungs in order, the six never-skippable gates, the insert-only chain, and refusals metered at zero [E01 §02; E20 §03].

H1, the shared domain, carries an isolation limit that must be stated unprompted. There is one signing key per residency domain, not per tenant, so ten thousand tenants in `in` share a key and the fingerprint a reader pins. "A tenant cannot prove its records were signed by a key no other tenant's records used." The remedy is H2 now or H3 at L3 [E16 §04; IM §10]. Per tenant, the chain (its own `seq` from 1), the salt rows, the anchor batch, the declaration registry scope and the export all remain isolated [E16 §04].

H2's cost is carried by one tenant. E10 models the L1 fixed cost of a residency domain at ₹1,88,000 a month and the L2 fixed cost at ₹3,43,000 plus anchoring (Chapter 19). A domain costs the same "whether it holds one tenant or fifty" [E10 §07]. A dedicated domain is therefore priced as a deployment, not as a tier.

H5 and anchoring. An air-gapped deployment cannot reach an RFC 3161 authority, and L2 requires every batch to be stamped by two, one India-based [E17; E14 §02 item 3]. An air-gapped H5 is therefore L1 unless the customer provides a controlled one-way path to the authorities. That path could be an egress allow-list naming only the TSAs, which is the pattern ETS already recommends for the evidence zone (Z4) [ETS §07], or a CCA-licensed TSA appliance inside the perimeter. The second option needs its own E17 assessment, because an authority operated by the same party that produces the records fails E17's independence test (**build recommendation**). The offline verifier works air-gapped in every profile, since that is what it was built for [E15].

Two stack1 decisions constrain H1 and should be settled before the first external tenant (see the conflicts block). S16 decides that from GA stack1's Standard tier and above are sold customer-VPC or operator-hosted, and that stack1 Cloud is a Starter and pilot substrate "capped at 25 tenants" [S16 §02, decisions 2–3]. S15 sizes P4 at "1–50 consumer tenants" [S15 §04]. The IM plan has 148 paying elan1 tenants at March 2029 and 545 at March 2031 [IM §16.4]. Those numbers only reconcile if an elan1 tenant is not a stack1 tenant. The recommended mapping is in §17.7.

17.6 Sovereign deployment — a deployment profile, not a model strategy

17.6.1 What it is, and what the founder does not pursue

In this memorandum "sovereign deployment" means one thing: **the kernel and its evidence plane running inside a customer's or a government's own boundary, operated by them** [E14 §03; E20 §03 item 4]. It is "a deployment model, not a proof capability" [E14 §03; IM §8.5]. It is not a sovereign-AI strategy, and the corpus says so three ways.

- The founder rejected "Sovereign Intelligence Platform" as the platform name because "sovereign AI" is another vendor's frame: "taking their word puts you in their frame" [GIP §2.2].
- The IM states the technical position directly. "Sovereign compute is not achievable. Sovereign evidence is, today, through a CCA-licensed RFC 3161 anchor and a per-tenant transparency service pinned to a jurisdiction. The architecture follows: the policy kernel runs anywhere; the transparency service and timestamp anchor are pinned per tenant, per jurisdiction" [IM §13.8, §8.5].
- The company's own objective test rules "sovereign-cloud hosting" out as a business [MMS].

The kernel needs no model to decide, since no model is reachable from rungs 3–8 at any level [E01 §02; E20 §03]. A sovereign deployment therefore carries no model, no GPU and no inference obligation. What a sovereign buyer receives is custody: the chain, the salts, the keys, the anchors and the registry sit inside their perimeter. That is the whole offer. elan1 does not sell sovereign models, sovereign compute or a sovereign cloud.

17.6.2 Where it sits in the plan, and the conditions on selling it

Fact	Source
Out of scope for L2; named L3 or later	E14 §03, decision 3
One of the six L3 capabilities, "deferred to a deployment specification in Phase 4"	E20 §03
"No sovereign deployment sold before the defect-propagation process exists — how a fix reaches a deployment we do not operate, and how we know it arrived"	E20 §08
"Sovereign deployment multiplies every operational surface. A defect must be fixed in every deployment by people we do not employ. This is the capability most likely to be sold before it is ready"	E20 §06 item 7
Whether it is built at all is open: "or is it declined on the grounds that we cannot support deployments we do not operate?"	E20 open question 4, Q3 2028
Guide outline: topologies (multi-tenant, customer VPC, on-premises); key custody in each; operational limits and support model; price. "Only started once L2 evidence has been accepted outside the company"	WD elan1 Phase 3 04
Priced as a deployment: ₹1.20 crore per deployment per year; governed actions at ₹0.55 (against ₹1.60 at L1) because the customer carries operating cost	IM §8.5 (modelled)
First deployment July 2029; 4 by March 2030; 11 by March 2031; ₹9.46 crore in FY2030-31, 4.1% of that year's revenue	IM §8.5 (modelled)
Gate B (tranche 3) requires "one signed sovereign or on-premises deployment order" by 30 June 2029	IM §16.5.2 condition 6

The last two rows sit awkwardly against E20. Gate B asks for a signed order in the same month that the L3 entry condition must first be met, while E20 forbids promising any L3 capability before the condition is met and forbids selling sovereign deployment before the defect-propagation process exists. That process has no plan date. The conflicts block records this. The IM itself notes that declining the topology would cost under five per cent of final-year revenue and "remove the operational surface the corpus is most worried about", which makes it "a reversible commercial decision" [IM §8.5].

17.6.3 How it would be built

stack1 already designs the packaging for customer VPC and air-gapped sites. The same artefact ships in three modes: stack1 Cloud with a rolling weekly canary; an operator's Kubernetes via a Helm release on their cadence; and customer VPC or air-gapped via a **signed offline bundle, quarterly**. That mode constrains the design from day one: no managed-service dependency, no runtime registry call, and vectors checkable offline [STS §07]. The bundle is a signed OCI artefact verified offline with cosign, with Zarf as the alternative [STS §07]. The kernel inherits all of this. It adds three things of its own (**build recommendation**, answering the E20 §08 precondition):

1. **A release is a signed kernel bundle plus the conformance suite.** The customer runs C1–C14 against their own deployment on every release and returns the machine-readable result file (the same shape as §16.4.3). A release is not "arrived" until that file is returned.
2. **A fleet attestation.** Each sovereign deployment emits a signed statement of its running kernel release digest, declaration registry head and chain heads (counts, not contents), exported one way on a fixed cadence. elan1 tracks every deployment's last attestation. That answers "how we know it arrived" without any inbound route.
3. **A security-fix clock per deployment.** A kernel defect of P0 or P1 severity gets a stated time to a signed fixed release, and the deployment's attestation shows when it was applied. The clock is a support term, and the deployment's own operators apply the fix. elan1 never holds a route into the deployment. That follows S17's "the home region publishes; it never reads" [S17 §5].

S16 sets a staffing precondition that applies equally to the kernel. stack1's Sovereign tier "is not sold until operated tenants per engineer is above 8", weights each sovereign tenant at 2.5 operated-tenant slots, and at low scale "is a consultancy with a login page" [S16 §06–07].

17.7 Tenancy and isolation in the runtime

stack1 separates its consumers at seven boundaries. Each is labelled **physical** (the permission, route or key does not exist) or **policy** (it exists and is not used). "We are not going to describe a policy control as a physical one" [S12 §01, decision 1].

Layer	Mechanism	Physical or policy
Network	Namespace per tenant, default-deny NetworkPolicy; control plane has no route to tenant data; evidence plane cannot read back into a tenant	Physical, applied at deploy time
Compute	One worker per tenant; one process per contract per tenant; nothing linked into a shared binary	Physical for the process; **policy for co-residency on shared hardware**
Storage	Postgres schema per tenant with RLS on every table; restricted role; object-store prefix per tenant	Physical
Model calls	Virtual key per tenant; budget reserved before the first token	Physical for attribution; policy for what a hosted provider does
Traces	Tenant on every span; queries through the RLS-backed role	Physical at storage; policy for what a stack1 engineer may read
Secrets	OpenBao path per tenant; mTLS workload identity; sealed references only	Physical
Egress	Deny by default; allow-list in the tenant's stack spec; the composition agent may never widen it	Physical for the deny; policy for the list

Source: [S12 §1]. Two admissions travel with the table. There is no hardware isolation: two tenants' workers may share a physical host, and a CPU side channel is outside the design. And stack1 staff can reach tenant data through a dual-approved, time-boxed, alerting break-glass path that appears in the tenant's next attestation [S12 §1]. The K2 isolation class attacks every boundary nightly and blocks merge, release and substitution. A response that reveals a count of another tenant's items is a K2 failure [S12 §2, decisions 3–4].

Mapping elan1 tenants onto stack1 tenants. S12 explicitly leaves the kernel's own isolation to elan1: "how the write path keeps declarations, gate decisions and proof records apart — is elan1's to specify" [S12 §01]. The sources do not state the mapping. The recommended one (**build recommendation**) is that **each elan1 residency domain is one stack1 consumer tenant running the P4 profile.** Inside it, elan1 tenants are separated by elan1's own controls: `tenant_id` not null on every kernel table with RLS forced and the application connecting as a restricted role [O05 §03; IM §6.10]; per-tenant chains with their own `seq`; per-tenant single-writer and chain-writer leases; per-tenant OpenBao paths for target credentials; caches and indexes partitioned by tenant, not filtered at query time [IM §6.10]; and a Temporal task queue per elan1 tenant inside the domain's namespace. An H2 dedicated domain is then simply a P4 deployment with one elan1 tenant in it. The mapping reconciles the S16 25-tenant cap and the S15 "1–50 consumer tenants" band with the IM's tenant counts. It also makes K2 attack elan1 tenants as well as stack1 tenants, by adding elan1's cross-tenant read test (tenant B reading tenant A's proof rows, declarations, obligations and meter rows, expecting a database refusal) to the same nightly matrix [S12 §2; O05 §03]. Chapter 18 gives the SQL.

17.8 Capacity and cost model

Every figure in this section is **modelled**. S16 opens with a table of what is measured and what is not, and all per-node, per-contract and margin figures sit in the second column. "Treating a model as a measurement is the fastest way to price a business into a hole" [S16 §01].

What the kernel's load looks like. P4's load "is not tokens; it is approval waits held open as durable signals, trace writes the proof record binds to, and evidence-plane writes" [S15 §05]. Capacity planning for the kernel therefore counts four things: governed actions per second at peak per residency domain; concurrent open approval waits; chain appends per tenant per

second, bounded by the single chain-writer lease; and shadow-sandbox restores in flight, the dominant marginal cost. S02 asks each consumer to forecast these quarterly, including "elan1's approval bursts", and provisions to peak plus 40% [S02 §9].

Quantity	Target or modelled value	Label	Source
P4 sizing	1–50 consumer tenants on 3–6 worker nodes (8 vCPU / 32 GiB), zero GPUs; ₹11.4–16.8 lakh a year at stack1 list	Modelled	S15 §04
Node utilisation	65% sustained target; scale at 70%; alert at 85% for 30 minutes	Target	S16 §05
Headroom	Declared peak plus 40%; above it the reservation refuses rather than degrading another consumer	Policy	S02 §9
Marginal cost per governed action, L1	₹0.20: compute for rungs 0b–9 ₹0.002; shadow sandbox apply and re-run ₹0.15; evidence storage ₹0.01 first year plus ₹0.03 retained; meter ₹0.01	Modelled	E10 §07
Fixed cost per residency domain	L1 ₹1,88,000 a month; L2 ₹3,43,000 a month plus anchoring (anchor cost "not publicly quoted")	Modelled	E10 §07
Tenant density per shared domain	Ten tenants per domain assumed; "an assumption with nothing behind it"	Assumption	E10 OQ4
Fully loaded cost per governed action	₹3.039 falling to ₹0.129 across the plan	Modelled	IM §4.8
Gate A condition	Cost per governed action **measured** under production load, published weekly for 26 weeks, ≤ ₹0.338 for the six months to 30 Sep 2028	Tranche gate	IM §16.5.1

The table carries two warnings. First, the shadow sandbox dominates the marginal cost, and E10 asks whether ₹0.15 might be ten times that "on a real ERPNext site with a real dataset" (E10 open question 2, due 31 March 2027). If it is, the marginal cost moves from ₹0.20 to about ₹1.55, close to the modelled L1 overage rate of ₹1.60 [IM §4.8]. The sandbox restore must therefore be the first thing load-tested, not the last. Second, the anchor is "the open number in the whole model": at 10^8 records a year "it is either trivial or fatal and we do not know which". That is why the anchoring interval is a published SLO and batches are per tenant per interval, not per record [E10 §07; E17].

S16's rule about modelled numbers applies to the kernel's rows unchanged: "a number modelled for two consecutive quarters gets a date by which it is measured, or it is removed from the model" [S16 §10, decision 9]. The measurement plan is in §17.12.

17.9 Scale and multiple regions

S17 is a plan for 2028 onward, written by a runtime with one region and two internal consumers [S17 §01]. Four of its decisions bind the kernel wherever it runs.

1. **The cell is the unit of blast radius.** A cell is one control plane, one evidence plane, one sandbox pool and one conformance runner, with at most 40 operated-tenant slots and no route between cells. A tenant never moves between cells or regions live: it is "a custody export, a new tenant id and the tenant's written agreement" [S17 §3, decisions 3–4]. For the kernel, moving a tenant therefore means a new chain. The old chain is closed with a governed record naming the successor tenant id, and the new chain's first record names the predecessor (**build recommendation**).
2. **No synchronous cross-region call is ever on a write path.** Mumbai to the Gulf is roughly 35–45 ms and to Frankfurt 110–130 ms (modelled, never measured). One hop spends the p50 budget. Every region must be self-sufficient for the whole write path, "including the elan1 gate check and the approval wait" [S17 §5 rule 1].
3. **What never crosses a region boundary**, for any reason including support, includes "elan1 proof records. Not ours to move, and the claim depends on where they were written" [S17 §5, decision 6]. The kernel's declaration versions flow outward, signed and by version, and are cached in region. If the in-region cache cannot resolve a version, the gate check refuses ("We cannot check this change right now. Nothing was changed"). It never falls back to the home region [S17 §6].

4. **There is no cross-region failover, at any tier.** A region loss is restored in place from in-region backups [S17 §9, decision 7; O05 §02; S03 decision 7].

A new region is also a new **residency domain** for the kernel, and therefore a new signing key, because a key is never shared across domains ("a domain is a boundary or it is decoration") [E16 §10]. Regions open "on a signature, never on a forecast": India is open, the Gulf not before 1 April 2028, the EU not before 1 April 2029, and no United States region before ₹80 crore ARR [S17 §11, decision 10]. S17 leaves open whether the kernel opens a region because stack1 did or decides separately (S17 open question 7). This memorandum recommends that the kernel decide separately, against its own conditions: a counsel reading for the jurisdiction, a signing-key domain with a practised rotation, a TSA pair acceptable to that jurisdiction's auditors, and a verifier profile for at least one regime there (**build recommendation**).

17.10 The substitution protocol, as it touches the kernel

stack1 replaces a component under a live consumer in nine steps: trigger, scope, plan, sandbox, prove (Gate 1), shadow, propose-and-decide (Gate 2, with the consumer's approver H1 and a named stack1 decider H2), promote and record. "A substitution without a completed ledger row did not happen" [S10 §02, §06]. The kernel meets the protocol in three roles.

As a consumer whose evidence depends on what is swapped. Some substitutions are "technically a substitution and commercially something else" and always need a fresh H1 from every affected consumer. Three rows bind the kernel directly: any change to the residency domain of a contract's data; "a change to which implementation serves a contract the elan1 layer binds proof records to", because the proof record must remain resolvable; and any change to the attribution triple or meter units [S10 §08]. The kernel's H1 approver must be a named elan1 engineer on a roster, not a mailbox. The Langfuse-to-Opik worked example has the elan1 lead as H1 [S10 §07 step 07].

As the gate check inside Gate 2. At step 07 stack1 calls the elan1 layer with the proposed action, tenant, contract and state class. The kernel returns allow, allow with approval required, or refuse. If the kernel is unreachable "the gate fails closed and the proposal stays undecided. It does not proceed on a timeout" [S10 §05]. A substitution is therefore itself a governed write in the operator's chain.

As the thing that is never substituted. The kernel is not a `fusion.*` implementation, so no substitution can alter refusal semantics, approval semantics or proof content. That is the reason for the boundary [S03 §2]. A kernel release is a separate process (§17.11), governed by the version policy and an ADR, not by S10.

The substitution clocks matter to kernel operations planning. Scheduled substitutions run on the upgrade train with ten working days' notice. Forced substitutions (a licence diff, an archive notice, a CVE with no patch) run on the tier clock of 72 hours at Standard or 24 hours at Sovereign, and "cannot wait for the train" [S02 amended by S17 §2]. Any forced substitution under the kernel's contracts is also a kernel incident until its post-cutover blocking classes read green.

17.11 Environments, infrastructure as code, release and rollback

17.11.1 Environments

stack1 runs four environments and no fifth: `dev` (synthetic), `ci` (synthetic; conformance runs and the golden substitution trace), `stage` (a restored clone plus a representative slice, never live customer data without a written exception) and `prod` (ap-south-1) [S02 §2, decision 2]. The kernel adopts the same four. Two kernel concepts that share the word "sandbox" must not be confused with an environment [E05 §05]:

Concept	What it is	Where it lives
Sandbox tenant (`sk_sbx_…`)	A developer's isolated tenant with synthetic fixtures; its own chain marked `mode: sandbox`; metered at zero	`prod`, as a tenant class (**build recommendation**), so GA quotas and status apply to it
Reference tenant	The fixture tenant the conformance suite runs against	`prod`, sandbox class; also cloned into `ci`
Shadow sandbox	The per-write, per-tenant snapshot restore that every production governed write passes through before live	`prod`, beside each tenant's target

17.11.2 Infrastructure as code

The recommended toolchain is Terraform or OpenTofu, with plans "reviewed in PRs in front of witnesses", and Helm charts per service applied by Argo CD [ETS §07; STS]. GitOps makes the separation visible between the Promoter, which holds apply credentials, and everything else, including any agent [STS; S03 B6]. Three checks run on every plan, and each fails the build rather than warning:

1. **Region allow-list.** Any resource declared outside ap-south-1 fails, and a resource with no resolvable region fails too [O05 §02].
2. **Absent routes.** The plan is read, not probed, to assert that no route exists from control to record plane or from evidence plane back to a tenant. The source's phrase is "checked by reading the Terraform plan in CI, not by probing for timeouts" [ETS §07; S03 B5].
3. **Grants.** The schema migration is diffed to assert that no kernel role holds UPDATE or DELETE on `proof.*` or `meter.*`, and that RLS stays forced on every tenant table [O05 §03; S03 B8].

```
# .github/workflows/infra.yml (excerpt) — build recommendation
jobs:
  plan-guards:
    steps:
      - run: tofu plan -out plan.bin && tofu show -json plan.bin > plan.json
      - run: python ci/region_allowlist.py plan.json          # O05 §02
      - run: python ci/absent_routes.py plan.json \
               --deny control->record --deny evidence->tenant  # S03 B5
      - run: python ci/grants_diff.py migrations/ \
               --no-update proof.record,meter.event --rls-forced  # O05 §03
```

17.11.3 Release and rollback

The version policy governs what a kernel release may change. Additive changes need no notice. A new refusal code needs 60 days' external notice with the operator action published first. A new KernelPort major needs 12 months and two releases in parallel. A change to a rung's order or presence needs 12 months and a published argument. An emergency security change goes at deploy, with notice inside 30 minutes and a written note within 24 hours [O06; IM §6.11]. A kernel change needs an ADR signed by the founder and may not special-case a product [IM §6.11]. Two things are not versions: the sixteen-field record, which is frozen, and a declaration version, which never expires. Nothing resolves to "latest" in production [IM §6.11; O03 §09].

A kernel release is promoted only if four things hold (**build recommendation**, assembling rules from E01 and E02):

- C1–C14 are green against the reference tenant on the release candidate.
- The **nightly refusal replay passes on the candidate**, so every reference-tenant refusal reproduces the identical decision. A release that changes a historical decision has changed the kernel's semantics without a declaration version [E01 done-item 5; E02 C4].
- The chain verifier, built separately from the chain writer, reads the candidate's appended records as continuous.
- Migrations are additive only.

Rollback is by redeploying the previous release. Because every migration is additive, the previous release can run against the newer schema. Because chain appends go through the fenced lease, the cutover in either direction cannot produce two writers. A rollback never removes a record, a declaration version or a meter event written by the rolled-back release. Those records stay and are attributable to the release by version. The kernel adopts stack1's change windows and freezes, including the freezes around rung1's nominated filing dates and month end [S02 §5], because those are the moments when a customer's books are being written most.

17.12 The latency budget on the write path

Every number below is a **target**. The rung budgets are "p50 design targets, not measurements — every one of them is [unverified] until Gate 1" [E04 §02].

Segment	Target	Source
0b STOP · 1 IDENTITY · 2 WRITE DOOR	4 ms p50	E04 §02
3 PRE-GATES	8 ms p50	E04 §02
4 `evaluate()`	12 ms p50	E04 §02
5 GUARDRAIL · 6 AUTONOMY	8 ms p50	E04 §02
7 ACT (kernel transaction only)	18 ms p50	E04 §02
8 PROOF (hash chain, unsigned)	12 ms p50	E04 §02
9 METERED	2 ms p50	E04 §02
Kernel total	**≈ 64 ms p50**	E04 §02
Shadow apply, gate re-run, live apply, live re-run (rung1 hosted ERPNext)	≤ 1,200 ms p95, outside the kernel figure	E04 §02
Governed write end to end, excluding human approval time	≤ 2 s p95	E04 §02
Governed write, kernel-side envelope	p50 ≤ 120 ms, p99 ≤ 450 ms	O03 §08; S17 §9
Proof record write	p50 ≤ 40 ms	O03 §08; S17 §9
Mode A overhead / Mode B contract call	≤ 15 ms over envelope / ≤ 40 ms p95 excluding the target	O03 §02, §08
Cross-AZ round trip inside ap-south-1	1–2 ms	S17 §9 (unverified)

The sources disagree about the proof rung. E04 budgets 12 ms. E10's cost model assumes about 91 ms p50 in total, of which about 40 ms is rung 8. O03 and S17 allow the proof write 40 ms p50 [E10 §07; IM §6.10]. This memorandum treats E04's rung table as the engineering budget to design against, and the O03 envelope as the ceiling that must not be crossed. At L2 the signature adds a KMS `Sign` round trip to rung 8, and that is the most likely reason for E10's larger figure. The L2 budget for rung 8 should be set from a measured KMS call before signing goes live, not inherited from either number (**build recommendation**). The approval wait is human time and is excluded from every figure.

Measurement plan. The first p50 measurement of a full nine-rung write on the Mumbai hosting is due by 30 November 2026 (E01 open question 2). Per-rung p50 on the reference tenant and p95 for the ERPNext adapter are due at Gate 1 (E04 open questions 1–2). The recommended instrument (**build recommendation**) is one OTel span per rung, with p50/p95/p99 published per write type from the same trace rows a consumer can query. The rule for publishing is O03's: "none of them appears in a customer document until it has been measured under a real tenant's load" [O03 §08].

17.13 Failure domains and kernel-unreachable behaviour

The kernel and the runtime fail closed in different directions. "A runtime outage stops writes; an elan1 outage stops allows — and stopping allows is the safe direction" [S03 §6].

Failure	Kernel behaviour	Source
Declaration registry unavailable	Refuse at the write door (platform class)	IM §6.10
Policy evaluation unavailable	Refuse at rung 4 and page; never a default allow	IM §6.10
Target unreachable after shadow apply	`PLATFORM.TARGET_UNREACHABLE`; nothing reached live; retry safe with the same `intent_id`	E09 §06
Live re-run disagrees with shadow	`GATE.SANDBOX_DIVERGED`; snapshot restored; "Nothing was changed" said first	E09 §06
Writer lock lost	Snapshot restored; page	IM §6.10
Chain append fails after the act committed	Outbox row survives; chain writer retries in order; no gap in `seq`	IM §6.10
KMS unavailable (L2)	Signing stops in the domain; writes refuse with a platform-class code; nothing signed with a stale key	IM §6.10; E16 §06

Failure	Kernel behaviour	Source
Both TSAs unavailable (L2)	Writes and signing continue; unanchored head grows and is shown	E19 §04; E17
stack1 evidence plane down	Outbox buffers; if it cannot buffer, admissions stop	S17 §6
`fusion-edge` down	Nothing new admitted — "the correct failure"	S03 §6
Region lost	No failover; restore in place	S03 decision 7

In Mode B, "the write does not happen if the caller honours the refusal; nothing physically stops it otherwise" [O03 §02]. An Attested advisory consumer whose kernel call times out must treat the timeout as a refusal. The conformance suite's fail-closed matrix (C5) checks the kernel's side, but nothing in the kernel can check the consumer's [E02 C5]. Chapter 22 carries the operational runbooks.

17.14 Open questions

#	Question	Owner	By	Source
1	Where does the elan1 proof record physically live: the evidence plane stack1 runs, or storage elan1 owns outright?	elan1 lead, stack1 lead	31 Oct 2026	S03 OQ2
2	Is there a stack1 failure mode that could make an elan1 proof record and the actual ERP state disagree, and what detects it?	stack1 lead, elan1 lead	30 Nov 2026	S03 OQ5
3	What is the approval-signal timeout default, and who sets it per write type?	elan1 lead	15 Nov 2026	S03 OQ3
4	Does ERPNext honour an idempotency key on submission natively, or must the adapter de-duplicate?	rung1 lead	15 Oct 2026	S04 OQ5
5	Measured p50 of a full nine-rung write on the Mumbai hosting	elan1 lead	30 Nov 2026	E01 OQ2
6	Is the ₹0.15 shadow-sandbox cost right on a real ERPNext site?	Kernel engineer	31 Mar 2027	E10 OQ2
7	Does the kernel open a region because stack1 did, or decide separately?	elan1 lead, founder	31 Dec 2027	S17 OQ7
8	Is sovereign deployment built at all?	Founder	Q3 2028	E20 OQ4

18. Security, residency, data protection and open-source licences

A governance kernel is an unusual thing to secure. Most of what it protects is not confidential data. It protects three properties: that no write reaches a customer's books without passing the kernel, that the record of what passed is complete and unaltered, and that nobody, including elan1, can quietly change the rules or the record. This chapter sets out the threat model for those properties and the control that answers each threat. It then covers identity and token classes, secrets, keys and encryption, tenant isolation, residency, logging, the DPDP Act and the conflict between erasure and an append-only chain, CERT-In incident reporting, the RBI's draft model-risk guidance, the open-source licence register and its CI enforcement, and the security-testing plan.

The baseline is shared. One security, residency and DPDP standard is written once for rung1, elan1 and stack1, because "writing it three times produces three baselines and the weakest one is the real one" [O05 §01, decision 1; IM §10]. This chapter applies it to the kernel. Chapter 17 covers the runtime isolation stack1 supplies.

> **Fail closed.** Every control is written so that when it is unavailable, the write does not happen. "A control that degrades to 'allow' is not a control." A box that cannot be evidenced with a command or a screenshot is not ticked [O05 §01].

> **Status.** The baseline is specified, not evidenced. As at 23 September 2026 no row of the O05 go-live bar is ticked for elan1, no control has been tested against a real tenant, and elan1 holds no certification of any kind: no SOC 2, no ISO/IEC 27001, no ISO/IEC 42001, no NIST AI RMF assessment [IM §4.9, §10; DR-010].

18.1 What is being protected

Asset	Why it matters	Where it lives	Plane
Target-system credentials	Whoever holds one can write around the kernel	Per-tenant credential broker; minted per call	P1 (secrets), used at P3
The proof chain	The record of every write, refusal and escalation; the product at every level	`proof.*`, insert-only; WORM object copy	P3
Declarations and policy versions	Decide what is allowed; loosening one silently is the most valuable attack on the kernel	Registry, append-only	P4
Approvals and obligations	The human half of the decision	Kernel tables, bound to a write	P3
The meter	What the customer pays; refusals must stay at zero	`meter.*`, insert-only	P3
Per-record salts	Make F14 payload commitments unopenable once destroyed	Separate salt store	P0
Signing keys (L2+)	The whole L2 claim rests on the domain key	Managed KMS, in region	P0
Payloads and personal data	Customer data subject to DPDP	Target system; intent payloads; sandbox snapshots	P2, P3

Sources: [O01 planes; O05 §03–05; E12 F14; E16 §02]. The planes matter because P0, which holds residency, tenancy, key custody and the erasure salt store, "has no migration path". Every choice made there is permanent [O01; O05 §02].

18.2 Threat model for a governance kernel

Figure 27 — Security, residency and DPDP controls by layer

L1 BEING BUILT · DESIGNED, NOT AUDITED

Each threat mapped to its control and to the test that proves it; physical means the grant or route does not exist.

LAYER · OWNER	THREAT	CONTROL	KIND · VERIFIED BY
Identity and tokens elan1 · E05, O03	Caller asserts its own tenant or autonomy level; call with no tenant; shared logins	Three token classes: tenant (one tenant, one actor, one autonomy level; the only class that reaches /v1/writes) · control (no route to any record store) · ops or service (never reaches KernelPort). Tenant and actor come from the token, never from a tool argument; an unattributed call is rejected. SSO and MFA for staff.	PHYSICAL + POLICY ctl paths: no network read path; no token can set its own rung 6
Credential broker elan1 · O05 §3, IM §6.10	A long-lived key to a customer's books leaks into a log; an agent writes around the kernel	Per-tenant credential broker; target credential minted per call, scoped to one write, 10-minute life; exactly one process mutates the target per tenant. One write credential per environment. Break-glass sealed, alerts within one minute. Mode B: the caller holds the credential, so the claim is attested advisory only.	PHYSICAL a 2nd write credential fails CI; bypass attempt shown failing
Tenant isolation stack1 + O05 §2, S12 §1	Tenant A reads, lists or counts tenant B's rows, spans, files or cached results	tenant_id NOT NULL on every table; RLS on every table under a restricted role; Postgres schema, namespace (default-deny), object prefix, OpenBao path and Temporal namespace per tenant; caches and indexes partitioned, not filtered. Admitted: no hardware isolation; one L2 signing key per residency domain.	PHYSICAL CI cross-tenant read test expects failure; K2 attacks nightly
Encryption and keys elan1 · E16, O05 §3	Key theft or a forged seal; an insider signs for a tenant; data in clear at rest or in transit	TLS and mTLS workload identity between planes; backups, snapshots encrypted at rest. L2 domain key: Ed25519 in managed KMS, Origin KMS, Sign for one IAM principal; the service holds a handle only; register ACTIVE / RETIRED / REVOKED; release key on an offline hardware token held by two named humans.	PHYSICAL nightly KMS Sign ↔ records (L2); rotation: 90 d (O05) v 12 mo (E16)
Supply chain, licences group · O04, S12 §6	Upstream relicensing (AGPL, SSPL, BSL) or a tampered component in the paid service	Pin to an exact commit SHA, never a tag; LICENSE archived verbatim at that SHA; permissive only (MIT, BSD, Apache-2.0) inside the paid service; copyleft only as a separate network service behind a fusion.*/v1 contract; ee/ and enterprise/ excluded at build; register re-read quarterly and every row confirmed by counsel.	POLICY (CI-ENFORCED) K7 nightly LICENSE byte diff (P1); no signed provenance yet
Residency group · O05 §1, S17 §5	Data, logs or keys leave India — a support copy, a DR replica, a hosted model provider	ap-south-1 Mumbai, three AZ: Postgres, object store, queues, backups, evidence and logs in region; no second region, including for DR, until counsel confirms. Every processor on the published sub-processor list with its region; proof records never cross a boundary; at L2 only a Merkle root reaches a TSA.	POLICY (CI-ENFORCED) build fails on a resource outside the region allow-list
DPDP group · O05 §7, Flow B	Processing without consent; erasure impossible in an append-only hash chain	Data Processor for tenant data, Fiduciary for own users; DPA signed before the first real record; consent ledger (purpose, time, lawful basis, notice version). Erasure is a governed write, irreversible: true, dual control, 72 hours: the subject's salt is destroyed; the chain stays intact with identifiers erased.	POLICY erasure tested end to end on a fixture tenant; counsel Q open
Logging, CERT-In group · O05 §5, Flow A	Incident missed or reported late; logs lost or held outside India	180-day logs in India, append-only, Object Lock for evidence; clocks on NPL or NIC; no PII in logs; every refusal logged with its code and policy version. Six hours from noticing: contain (tenant stop at rung 0b), the named CERT-In responder files; if unclear, file. Tenant update 72 h; postmortem 5 working days.	POLICY six-hour path tested once before the first outside user

Governing rule: fail closed
Every control is written so that when it is unavailable, the write does not happen. A control that degrades to "allow" is not a control (O05).

Minimum bar before any outside user logs in
13 rows (O05 §12); all 13 are required of elan1, including signing keys in KMS and a proof record for every governed write, verified on a sample.

elan1 kernel control · stack1 runtime / group · group baseline (O05) · threat · control / test

Source: O05 §1–§8, Flows A–B, §12 · S12 §1–§6 · O04 §02–§03 · E05 §05 · O03 §06 · E16 §02–§04 · S17 §5 · IM §6.10, §10 · elan1 Technology Memorandum · Sept 2026

Figure 27 — Security, residency and DPDP controls by layer: identity and token classes at the edge, the model-free decision path, the single writer and credential broker, the insert-only chain and salt store, KMS custody at L2, and the ap-south-1 residency boundary that contains all of them.

Figure 27 places each control on its layer. The table names the threats the corpus treats, the controls that answer each one, and what is left over. A control marked L2 does not exist at L1, and the residual risk at L1 is stated with that in mind.

Threat	How it arrives	Controls	Residual risk
Agent prompt injection	A document, e-mail or upstream agent instructs the agent to write something harmful	Invariant I1: a model may produce a write intent, never an allow. Rungs 3–8 are model-free and deterministic. `fusion.guard/v1` provenance (`tenant_ingested`, `third_party`) is an input, never a decision, and such content cannot trigger an action above read without a human. Autonomy is a property of the actor, not the tool. Irreversible types are never auto-approved. MCP tool descriptions are pinned by hash in the declaration	An injected intent that falls inside the declared permissions and passes every gate will execute. The gates and the approver are the defence, and approvals can become rubber stamps
Suppression (writing around the kernel)	An agent or integrator holds its own credential and writes directly	Enforced mode only: the single writer holds the only credential, or the target accepts only kernel tokens. C2 bypass proof on the real target before promotion. Nightly reconciliation against the target's own trail. Automatic revert to Attested advisory on an unrecorded write	In Attested advisory mode nothing stops it; the record shows only that the caller was told no. Direct edits in the accounting system are that system's trail

Threat	How it arrives	Controls	Residual risk
Credential theft	A credential leaks into a log, a laptop or CI	Target credentials minted per call, single-use, one write, ten-minute life. No long-lived credential reachable by a consumer, agent or operator. Tenant tokens short-lived. Secrets in OpenBao; pre-commit and CI secret scans. Sealed break-glass credential that alerts within one minute	A minted credential is usable for at most ten minutes, for one write in one scope
Insider (elan1 or stack1 staff)	An engineer edits a record, loosens a gate or signs something	No UPDATE or DELETE grant on proof or meter tables. Production writes by named individuals over recorded sessions. Dual control on declarations, keys and break-glass. Control plane has no route to record stores. At L2, KMS `Sign` granted to the chain writer alone, with nightly reconciliation of `Sign` calls against signed records	A privileged insider in the cloud account could alter a key policy. That would be visible in the provider's configuration history and the reconciliation, but it is possible. Customer-held keys at L3 remove it. At L1 a DB superuser could rewrite a chain consistently (see below)
Insider (tenant side)	An approver waves through their own request; an admin weakens a rule	Author ≠ certifier; requester ≠ approver (`GATE.DUAL_CONTROL_SAME_ACTOR`); agents may never approve; step-up authentication; loosening a gate needs a new certified version plus an ADR; "eligible but did not sign" recorded	Collusion between two named humans
Record tampering	Edit, delete, reorder or truncate proof records	Insert-only tables; SHA-256 chain; nightly integrity verifier (a broken link is a P0); WORM copy with Object Lock. At L2, Ed25519 signature and two-TSA anchors. Truncation is detected by reconciliation against the target at L1 and by anchors at L2	At L1 the chain is tamper-evident to the tenant, not to a stranger. A consistent rewrite by someone holding database owner rights is detectable only against an externally held head
Replay	Resubmit an intent, reuse an approval, reuse a token	Idempotency: a repeated `intent_id` returns the original decision, `seq` and digest (C7). Minted credential single-use. Short-lived tenant tokens. Approval bound to one obligation and one write	Approval binding is a build recommendation (§18.3)
Supply chain	A dependency is compromised or relicensed	Pin to exact commit SHA; LICENSE archived at that SHA; K7 nightly byte check; `ee/` exclusion; every implementation a separate network service. Verifier: zero dependencies, empty lockfile, reproducible build, release key on an offline token held by two named humans	No signed provenance attestations or reproducible builds for upstream components (stated in S12)
Fail-open pressure	An outage tempts an operator to "just let writes through"	Fail closed everywhere; no warn, no decision cache, no admin mode, no fast path	Business pressure during an outage is a people risk
Cross-tenant leakage	A bug or an enumeration reads another tenant's data	RLS forced on every table; restricted role; unattributed calls rejected; K2 nightly attack including enumeration counts	Co-resident workloads on shared hardware; no side-channel defence claimed

Sources: [E01 §02, §07; E02 C2, C7; E05 §02, §04; E09 §09; E11; E16 §08–09; IM §3.4, §6.10, §10; O05 §03–05; S03 §3–4; S12 §1–7; GAM risk register rows 4, 7].

Three points in the table need explanation.

Suppression is the threat that decides the architecture. A receipt proves a call happened. It cannot prove that a call did not. An agent that routes around the kernel leaves no receipt, and an observer holding a perfect set of receipts sees nothing wrong. "Absence of evidence is not evidence of absence — unless the system producing the evidence is the only path to the act" [IM §3.4]. The single writer is therefore a security control first and an architecture choice second. It is also why the product refuses to describe Attested advisory as enforcement [E05 §02; O03 §02].

The L1 insider gap. At L1 the chain is "tamper-evident to you — you hold the chain and an edit is detectable" [E07 §02]. That sentence is true only if the tenant actually holds something elan1 cannot rewrite. The recommendation (**build recommendation**) is a daily chain-head receipt, `(tenant_id, seq, digest, recorded_at)`, delivered to the tenant's named notice contact and exposed through `read.evidence`. A later consistent rewrite of the chain by anyone with database owner rights then fails to match a head the tenant already holds. This adds no signature and no anchor, so it stays inside the L1 claim boundary. It gives the "you hold the chain" sentence a mechanism.

Prompt injection is answered by structure, not by detection. The kernel does not try to detect injected text. It makes injected text unable to authorise anything. The guard's injection verdict is attached to the intent as evidence, and the rungs decide [S03 §4 step 5; S03 decision 4]. Rubber-stamp approval, where an approver's dwell time collapses towards zero, is named as a risk in the corpus and should be monitored as a first-class metric, with suspiciously fast approvals sampled for audit [GAM risk 4].

18.3 Identity and token classes

18.3.1 Machine identity

Every token carries one tenant, one actor and one autonomy level. Tenant and actor come from the token, "never from a tool argument". A call that cannot be attributed to a tenant is rejected, not defaulted. A token that could assert its own autonomy level "would make rung 6 decorative, so it cannot" [E05 §05; O03 §06; O05 §03].

Class	May	May never	Source
Tenant token	Reach `/v1/writes` and the `ro`, `stage` and `act` tools for one tenant	Act for another tenant; assert its own autonomy level	E05 §05; O03 §06
Control token	Studio and the declaration registry	Reach any record store (no network route, not merely no permission)	E05 §04–05; O05 §03
Service token (O03, O05)	stack1 components calling each other	Reach KernelPort	O03 §06; O05 §04
Ops token (E05)	Read-only operations, no tenant data	Write; break-glass is a governed write with dual control	E05 §05

O03 and O05 say "three token classes and no fourth": tenant, control, service. E05 names tenant, control and ops. The two lists describe different principals: stack1 components that must never reach the kernel, and elan1 operators who need read-only operational access. This memorandum recommends four classes, each with its own audience claim, and amending the "no fourth" rule to name all four (**build recommendation**; recorded as a conflict). The kernel sets `actor_kind` (human, agent, system) itself, and no tool can claim it [E12 F10; E05 §01].

A tenant token's claims might look like this (**build recommendation**):

```
{ "iss": "https://id.in.elan1.example", "aud": "kernelport:v1", "cls": "tenant",
  "ten": "t_4c1e…", "sub": "act_9a02…", "akd": "agent", "aut": 2,
  "spn": "usr_77b1…",
  "scp": ["ro", "stage", "act"], "iat": 1790000000, "exp": 1790000300,
  "jti": "5f0c…", "rdom": "in" }
```

`spn` names the human sponsor of an agent and is absent on a human's own token. The lifetime is short (five minutes in the sketch). The residency domain (`rdom`) must match the edge that received the call, and a mismatch is a residency event, not an auth failure [S17 §6]. An agent token with no sponsor is refused ("This action needs a person behind it") [S03 §4 step 4].

18.3.2 Human identity and approvers

Internal access uses SSO for everyone, with no shared logins or browser profiles. MFA is mandatory for anyone with production access. Production database write access is limited to named individuals reviewed monthly, and every production action goes through a logged path. Access is removed within 24 hours of departure, and access reviews are quarterly and signed [O05 §05]. Keycloak (Apache-2.0) is the register's clean choice for SSO and realms [O04 §03].

Approvers are the human half of the decision, so their identity carries more weight than a login. Every approval is made by a named human, never an agent, with step-up authentication at the Decide screen. It is bound to exactly one obligation and one `write_id`, and it is valid once (**build recommendation** for the binding and single use; the rest is from [E09 §04; E16 §06; S10 §05]). An approval that arrives after the obligation's window has expired resolves nothing, and the write refuses

with the expiry recorded [S03 §4 failure paths]. Customer-side approvers authenticate against the customer's identity provider through federation in Keycloak. The approver's identity recorded beside the chain is the federated subject plus the tenant, not an e-mail address (**build recommendation**, keeping personal data out of the chain; see §18.9).

18.4 Secrets and target credentials

Control	Detail	Source
Secret store	All secrets in a managed store, one OpenBao path per tenant; nothing in an environment file; pre-commit hook and CI scan	O05 §04; S12 §3
Target credentials	Held by the per-tenant credential broker; minted per call; single use; one write; ten-minute life	O05 §04; E01 §02 item 4; E05 §02
Who can mint	Only the single writer at rung 7	E01 §02; E16 §02 transport keys
One write credential per environment	A second one entering the codebase fails CI	O05 §04
Consumers	Receive sealed references only; the export format cannot represent a secret value	S12 §3, decision 5
Break-glass	Sealed credential; use alerts within one minute; tested monthly. For the kernel it is a declared write type with `dual_control: true`, recorded in the operator chain and visible to the tenant	O05 §04; S12 §2; E16 §08; E09 §09
No long-lived target credential	Anywhere a consumer, agent or human operator can reach, including CI and admin tooling	E09 §09 step 3; E05 §02

The ten-minute life is chosen on purpose: "Short is deliberate: a long-lived credential will end up in a log" [O05 §04]. The broker is the one component whose compromise defeats Enforced mode, so it runs in the single writer's trust zone with its own workload identity. It is never co-located with a consumer process (Chapter 17 §17.2).

18.5 Keys and encryption

At L1 there is no signing key anywhere. "Not in a config file, not in a vault, not 'just for testing'" [E01 §07; E16 §01]. The only keys at L1 are transport keys, meaning TLS and the per-write credential, which "matter for availability and confidentiality, not proof" [E16 §02].

From L2 there are four key classes [E16 §02]:

Class	Purpose	Custody	Rotation
Domain signing key	Signs the 32-octet record digest at rung 8	Managed KMS, Ed25519 (`ECC_NIST_EDWARDS25519`, pure, never pre-hash), in region; `Sign` granted to one principal	Every 12 months, or on compromise
Anchor client key	Authenticates elan1 to a TSA	KMS in the same domain	Per the authority, at least annually
Release key	Signs verifier binaries, source tarballs, vector sets	Offline hardware token held by two named humans, in no cloud	Every 24 months
Transport keys	TLS; per-write target credential	Platform secret management	Per platform policy

Key material is generated inside the custody module and never exported, backed up, printed, escrowed or copied between residency domains [E16 §01 rule]. Windows abut and never overlap, and five operations need dual control: generate, register a window, rotate, revoke and publish a verifier release [E16 §05, §08]. Chapter 12 specifies custody in full. Two conflicts belong here. O05 and S12 call signing keys "per-tenant"; E16 and the IM say one per residency domain. And O05, S12 and the IM rotate signing keys every ninety days, while E16 rotates the domain key every twelve months. This memorandum follows E16 for elan1 domain keys, because E16 is the elan1-specific custody design and states why a per-tenant key "is mostly imaginary while we hold every key in the same account" [E16 §04]. Both conflicts are recorded.

Encryption. The sources require encrypted database snapshots and encrypted nightly backups [O05 §06–07] and mTLS workload identity between planes [S12 §1; S03 §4]. They are otherwise silent on encryption at rest and in transit. The recommended defaults (**build recommendation**) are: TLS 1.3 at the edge (CloudFront and WAF hold nothing and decide nothing [ETS §07]); mTLS between every kernel unit and every `fusion.*` endpoint; KMS-managed envelope encryption for Postgres storage, object storage and backups, with a data key per residency domain; and column-level encryption, under a key separate from the storage key, for intent payload tables and sandbox snapshot references, because those are where personal data sits. None of this is a claim of confidentiality from elan1 itself. "We cannot see your data" is banned at L1 and L2, and it is true at L3 only for a tenant on customer-held keys with salts held by them [E14 §05; E20 §05].

18.6 Tenant isolation inside the kernel

stack1 separates its consumers (Chapter 17 §17.7). The kernel separates its own tenants, and S12 leaves that to elan1 [S12 §01]. The controls are those of the baseline: `tenant_id` not null and foreign-keyed on every tenant table; RLS enabled and forced, with a migration that disables it failing CI; the application connecting as a restricted role, never owner or superuser; the cross-tenant read test running in CI and expecting a database refusal; unattributed calls rejected; caches, search indexes and vector stores partitioned rather than filtered [O05 §03; IM §6.10]. The kernel's tables add insert-only grants and the refusal CHECK (**build recommendation**; authoritative schemas in Chapters 9 and 19):

```
CREATE TABLE proof.record (
  tenant_id  uuid   NOT NULL REFERENCES core.tenant(id),
  seq        bigint NOT NULL CHECK (seq >= 1),
  prev_digest bytea NOT NULL CHECK (octet_length(prev_digest) = 32),
  digest     bytea  NOT NULL CHECK (octet_length(digest) = 32),
  -- ... the sixteen fields (Chapter 9)
  PRIMARY KEY (tenant_id, seq));
ALTER TABLE proof.record ENABLE ROW LEVEL SECURITY;
ALTER TABLE proof.record FORCE ROW LEVEL SECURITY;
CREATE POLICY t ON proof.record
  USING (tenant_id = current_setting('app.tenant_id', false)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', false)::uuid);
GRANT SELECT, INSERT ON proof.record TO kernel_chain_writer;   -- no UPDATE, no DELETE
GRANT SELECT ON proof.record TO kernel_api;
ALTER TABLE meter.event ADD CONSTRAINT refusal_is_free
  CHECK (outcome <> 'refused' OR quantity = 0);                  -- IM §4.8
```

`current_setting(..., false)` raises an error when the tenant is unset, so an unattributed session fails rather than reading everything [O05 §03]. The nightly isolation attack runs as tenant B against tenant A's proof rows, declarations, obligations, meter rows and salts. It counts a response that reveals how many of A's items exist as a failure [S12 §2, decision 4].

18.7 Residency

Requirement	Detail	Source
Region	ap-south-1 (Mumbai), three AZs; no exceptions in Phase 1	O05 §02
Database	Postgres primary with a synchronous standby in a second AZ; point-in-time recovery	O05 §02, §07
Everything else	Object store, queues, backups, evidence store and logs in the same region	O05 §02
Second region	None, including for DR, until a residency decision is written and counsel-confirmed	O05 §02, decision 2
Model inference	Self-hosted open-weight models on vLLM in Mumbai; a hosted provider only if on the sub-processor list with its region. The kernel's decision path uses no model	O05 §02; E01 §02
Enforcement	CI fails the build on any resource outside the region allow-list	O05 §02
What may leave	A receipt may cross a boundary; a payload may not. Bundles carry salted per-field commitments, never payloads	IM §10
What never crosses a region	Tenant records, traces, logs, secrets, key material, evidence rows and elan1 proof records	S17 §5, decision 6

Residency is also a never-skippable gate (G2) at rung 3, so the kernel refuses a write whose declared residency does not match where it would be executed or recorded [E01 §02 item 1; E07 §05]. At L2 the TSA pair includes an India-based authority, for residency and audience reasons rather than cryptographic ones. An Indian auditor is better served by a token traceable to Indian standard time from a CCA-licensed provider [E17].

18.8 Logging, time and retention

Logs are retained for 180 days inside India under CERT-In: "not 180 days somewhere and a copy in India — the retained logs are in India." Clocks are synchronised to NPL or NIC NTP and checked by monitoring. Logs are append-only. No PII goes into application logs; a CI lint fails on known field names and a monthly sampling review runs. Every refusal is logged with its code, gate, policy version and `as_of` [O05 §06; S12 §5]. Before a line is written, prompt and completion bodies, document text, tool arguments and results (a hash is kept instead), secrets, e-mail addresses and phone numbers are redacted [S12 §5].

Class	Retention	Storage	Source
Security and access logs	≥ 1 year; 180-day minimum in India	Append-only, India	O05 §06
Audit and proof records	7 years (see conflict)	WORM, India	O05 §06
Evidence bundles (L2)	24 months	WORM, India	O05 §06
Database snapshots	30 days	Encrypted, India	O05 §06
Application logs	180 days	Append-only, India	O05 §06
stack1 traces	90 days default	India	S12 §5
Shadow-sandbox snapshots	30 days (the rollback window)	India	E01 §02 item 6

The proof-record retention figure is wrong for the kernel's first pack. MCA Rule 3(1) requires the audit trail to be preserved for eight years, and the RBI draft asks for ten-year retention [IM §10]. E10's cost model already carries storage for a ten-year retention, and E11 says the platform holds records "indefinitely" pending per-regime retention profiles [E10 §07; E11 §06, OQ3]. A seven-year default would delete ledger1 evidence a year before the statute allows. This memorandum recommends that proof-record retention be set per tenant as the longest period any bound pack or regime requires, never below eight years for a ledger1 tenant, and enforced by the declared retention gate (**build recommendation**; recorded as a conflict).

18.9 The DPDP Act, personal data in proof records, and erasure

18.9.1 Roles, dates and the consent ledger

The DPDP Rules were notified on 13 November 2025. The main obligations apply from **13 May 2027**, when the Data Protection Board gains full adjudicatory powers, with penalties of up to ₹250 crore per major violation [IM §10; O05 §01]. That date falls between L1 GA (June 2027) and the first external paying tenant (July 2027), so the kernel's first external tenant arrives under the full regime. Roles are fixed per relationship: elan1 is **Data Processor** for every tenant's data and **Data Fiduciary** for its own users, staff and prospects [O05 §08, decision 7]. The baseline requires: a signed Data Processing Agreement before a tenant's first real record; a published privacy notice for elan1's own users; consent captured "with purpose, timestamp, lawful basis and version of the notice consented to — a consent ledger, not a boolean column"; a consent object designed to interoperate with a registered Consent Manager (framework expected from 13 November 2026, marked unverified); purpose limitation enforced at query time; no Significant Data Fiduciary obligations assumed, re-checked quarterly; and a staffed, tested data-subject request path [O05 §08].

In the kernel, consent (G1) and residency (G2) are two of the six never-skippable gates. A write type that processes personal data declares its lawful basis and purpose, and G1 refuses the write if the consent ledger cannot show a matching, unexpired consent at `as_of` [E07 §05; E01 §02 item 1]. The consent ledger belongs to the tenant as Fiduciary. The kernel reads it; it does not decide it.

18.9.2 What personal data a proof record holds

The sixteen-field record was designed so that it can survive an erasure request. It does not carry the payload [E12].

Field or companion	Personal data?	Design
F14 `payload_hash`	Commitment only	Digest over the salted, canonicalised payload; destroying the per-record salt makes the payload unrecoverable "including to us" while every digest, signature and anchor stays valid [E12; E15]
F09 `actor_id`	Pseudonymous identifier of a human or agent	Random UUID; the mapping to a person lives in the actor registry, outside the chain (**build recommendation**)
F12 `record_id`	Identifier of the target record	Points into the target system; carries no attributes
F10, F11	Actor kind and autonomy level	Not personal data
Gate results, obligations, approver identities	May contain personal data	Retained beside the record, not in the preimage [E12]; approver held as federated subject, not e-mail (§18.3.2)
Intent payloads, sandbox snapshots	Yes	Outside the chain; retention by declared class; erased with the salt

Sources: [E12; E11 §05; E15]. The principle is that personal data lives in erasable companion stores and the chain holds only pseudonymous identifiers and commitments.

18.9.3 Erasure against an append-only chain

Erasure is itself a governed write type, marked `irreversible: true`. That forces a blast radius of at least one counterparty and therefore dual control: two named approvers on a 72-hour clock [O05 §11 Flow B, decision 5]. The system resolves every location of the subject's data (the record, every derived index, embedding and cache) and produces an erasure plan the operator sees before anything is deleted. On approval the job erases records, rebuilds or purges derived artefacts, and destroys the subject's salt in the erasure side store. The proof records are "retained with the subject's identifiers erased. The chain stays intact and verifiable; the person is no longer in it." A failed job keeps the case open: "Dropping an index is not erasure, and a rebuild that reintroduces the data is a defect." A request that conflicts with a statutory retention obligation is refused in writing, naming the obligation and the date it expires [O05 §11].

The same baseline then asks whether that satisfies DPDP, "given the hash chain cannot be rewritten" (O05 open question 4, counsel, 30 November 2026). **Counsel has not answered.** The IM calls it "the sharpest unresolved legal tension in the corpus" and sets a hard deadline: settled before the first paying tenant, and in no case later than 13 May 2027 [IM §10]. Three candidate answers exist, and none has been approved [IM §10]:

1. **Salted commitments with per-field disclosure.** Salts derive from one per-record secret. Destroying it makes every commitment unopenable, including to elan1, so a commitment to a destroyed value can no longer identify anyone.
2. **Erase the payload, retain the commitment.** The identifiers go. The 32-octet digest stays, because removing it breaks the preimage, the signature and the chain. What remains is a record of a process held under a statutory retention floor (eight years under MCA, ten under the RBI draft).
3. **Tenant-held keys and salts.** elan1 cannot open the commitments at all, and the decision sits with the Fiduciary. This is real but narrow: it is a fact about one deployment (H3 or H4 in Chapter 17), not a product claim.

If the answer is unfavourable, and a retained commitment is itself personal data that erasure must reach, the consequence is bounded. The chain would be segmented so that a per-subject sub-chain can be truncated without invalidating the parent: "engineering work of a quarter or two and a new suite identifier", not a rewrite. The evidence SKU would slip [IM §10]. What would not survive is any claim that erasure is solved today, and no such claim is made.

A build decision cannot wait for counsel. F14 is frozen at L1 and defined over the *salted* payload [E12], so the salt question is a record-format question, due with the record shape on 30 November 2026 [IM §16.5]. E01 lists "no erasure salt" among L1's non-goals, and E19 says a per-record salt "begins to be retained" at L2 [E01 §07; E19 §02]. O05, however, requires the salt-destroying erasure path to be tested end to end before any brand's first external user [O05 §12]. The recommendation

(**build recommendation**) is to generate a per-record salt from the first L1 record and hold it in the P0 salt store, with separate grants, from day one. The E01 non-goal would then be read narrowly: no salt-based erasure *claim* at L1. Without that, every L1 record written before L2 carries a commitment that can never be unbound, which is the opposite of what O05 promises the data principal. E11 already anticipates the verifier wording for such records: "chain intact, payload unbound", never a failure [E11 §05; E15 exit code 0 variant]. The conflict is recorded.

18.10 CERT-In incident reporting

The CERT-In Directions of 2022 are in force: six-hour incident reporting, 180-day logs retained in India, clocks on NPL or NIC [O05 §01; IM §10]. The group's incident flow applies to the kernel unchanged [O05 §10]:

1. The on-call engineer declares the incident. The timestamp is recorded automatically and the six-hour deadline is shown in IST. **The clock starts at noticing, not at confirming.**
2. Containment follows the runbook: revoke credentials and **set the tenant stop at rung 0b**, so the kernel refuses every write for the affected tenant ("Writes are paused for this organisation").
3. The stop is verified, not assumed, by attempting a governed write, including from an automated workflow, and expecting a refusal.
4. The named CERT-In responder assesses reportability and files within six hours. If unclear, file: "A filed report that turns out to be unnecessary costs an afternoon."
5. In parallel, the DPDP assessment runs, and tenants are notified through the DPA contact.
6. A written update goes to the tenant within 72 hours, and a blameless postmortem is published internally within five working days.

If the responder is unreachable, escalation goes to the founder immediately, and the six hours do not pause [O05 §10]. stack1's notification table adds the consumer-facing clocks: every consumer inside 30 minutes of a P1, suspected cross-tenant access to every consumer "immediately, at any confidence", and a filing notified to every consumer the same day [S12 §7, decision 7]. The kernel's own GA readiness adds a customer-facing rule: a written note to affected tenants for every P0 and P1 within five working days, and a P0 without a note is itself a P1 [E09 §02 row 6]. **The named CERT-In responder and deputy were due on 30 September 2026 and are still unfilled** (O05 open question 2; S12 open question 5). The go-live bar requires the six-hour path to have been tested once [O05 §12].

18.11 RBI draft model-risk guidance and FREE-AI

The RBI's Draft Guidance on Model Risk Management was released on 24 June 2026, with comments closed on 24 July 2026, and is still a draft with no effective date [IM §10]. It reaches all eleven categories of regulated entity and every model, internal or third-party, "whether or not the entity recognises them as models" [IM §10]. The FREE-AI committee report of 13 August 2025 is not binding [IM §10]. What the draft asks for maps onto kernel mechanisms directly, and the limits of that mapping need stating as clearly as the mapping itself.

Draft requirement	Kernel mechanism	Limit
Kill switches	Rung 0b STOP per tenant, itself a governed write with a record	Stops writes through the kernel only
Human-in-the-loop by competent staff	Approval at A0–A2, dual control, step-up; irreversible writes never auto-approved	Competence of the approver is the bank's matter
Traceability and auditability	The chain; at L2 the bundle and offline verifier	At L1 checkable only with elan1's cooperation
Reproducibility	Rungs 3–8 deterministic at a pinned declaration and policy version; nightly replay of every refusal	Reproduces the decision, not the upstream model's intent
Ten-year retention	Per-tenant retention by the longest bound regime (§18.8)	Conflicts with O05's seven-year row
Accountability for third-party outcomes	Evidence for the bank's own validation	"No certificate elan1 holds will ever discharge a bank's own validation duty"

Source: [IM §10; E01 §02; E07 §05]. The kernel has no model in its decision path, but a bank may still classify the rule engine as a "model" under the draft's wide definition. The recommendation (**build recommendation**) is to ship, with collect1 and any bank-facing pack, a model-documentation set generated from the system itself: the declaration version, the policy version history, the determinism replay results and the refusal register. A bank's validator then starts from evidence rather than a questionnaire.

18.12 The open-source licence register and its CI enforcement

18.12.1 The three-box test

The register sorts every component by where it runs [O04 §02]:

- **Box 1, the customer's site.** Software the customer runs themselves, such as ERPNext on their own instance. Copyleft is fine.
- **Box 2, internal.** Software only elan1 logs into. Copyleft is fine, since there is no distribution.
- **Box 3, the paid service.** Anything a tenant reaches over the network: "the kernel, the gateway, the meter, the evidence store". This is the dangerous box. AGPL's network clause, SSPL, the Elastic Licence and BSL each make hosting the thing either an obligation to publish or a prohibition.

> **The standing rule.** Permissive licences only (MIT, BSD, Apache-2.0) for anything embedded in the paid service. Copyleft is acceptable on a customer's own site and for anything only the company logs into. The escape hatch for a necessary copyleft component is a separate network service behind a `fusion.*/v1` contract, "no shared process and no linked code" [O04 §02, decisions 1–2].

The whole kernel is box 3. Every library linked into a kernel process must therefore be MIT, BSD or Apache-2.0. The rows of the register that touch the kernel are:

Component	Licence	Verdict	Relevance to the kernel
Temporal	MIT	CLEAN	The workflow engine for every governed write (DR-002)
Keycloak	Apache-2.0	CLEAN	SSO and realms
vLLM	Apache-2.0	CLEAN	Self-hosted inference, never on the decision path
Superset	Apache-2.0	CLEAN	Preferred BI for anything multi-tenant hosted
LiteLLM, Langfuse	MIT core plus `ee/`	CONDITIONAL	Behind `fusion.gateway/v1` and `fusion.trace/v1`; build excludes `ee/`; Langfuse ownership changed in the last 24 months
Metabase	AGPL-3.0 (OSS)	CONDITIONAL	Internal only; never in a tenant-facing screen
Documenso	AGPL-3.0	CLEAN (tenant-installed only)	"E-signature near the evidence plane is exactly where an AGPL process must not sit"
Windmill · n8n · Inngest	AGPL + restriction · Sustainable Use · SSPL	BLOCKED	Workflow engines whose licences restrict running them as a service (DR-003, DR-004)
LiveKit turn-detector weights	Bespoke model licence	BLOCKED	"No bespoke-licence model sits in a critical path"
ERPNext · Frappe	GPL-3.0 · MIT (unverified at pinned SHA)	CLEAN (customer site)	The system of record; adapters call Frappe's API, "nothing of ours runs inside a GPL process"

Source: [O04 §03]. The register has 25 rows, and **none reads "Confirmed" by counsel** in the issued version. Each reads Sent or Open [O04 §03]. The go-live bar requires every CONDITIONAL row to be confirmed before any brand's first external user [O05 §12]. Three counsel questions touch the kernel directly: whether Frappe is MIT at the pinned SHA; where exactly the line falls between "separate network service" and "shared process" for an AGPL component under Indian law; and whether the Windmill and n8n questions are closed permanently now that Temporal is in place [O04 §07, OQ1, OQ4, OQ5].

The register does not yet list the kernel's own language-level dependencies: the web framework, database driver, JSON canonicaliser, and at L2 the Ed25519 and ASN.1 code in seal implementation A. It must, under the same rules (**build recommendation**). The offline verifier is the easy case, because it is written in Go against the standard library only and ships with an empty lockfile [ETS; E14 §02 item 5].

18.12.2 CI enforcement

Four controls are specified. Every component is pinned to an exact commit SHA, "never a tag, never latest", and its LICENSE is re-read at that SHA and archived verbatim [O04 §01 rule, decision 5; S12 §6]. One check excludes `ee/` and `enterprise/` paths across all five core-plus-EE components [O04 decision 4]. A BLOCKED verdict cannot be inserted into the register at all, and a CONDITIONAL one cannot bind into a tenant whose posture forbids it [S12 §6; S04 decision 5]. The K7 class runs nightly and on every commit, asserting that the deployed artefact contains no excluded directory, carries every required attribution, and that the LICENSE at the pinned SHA still matches the archive byte for byte. A diff on a pinned component is a P1, because "it is either an upstream relicensing or a supply-chain event" [S12 §6, decision 8]. The register lives in the repository and is generated into the knowledge vault; it is never hand-edited in a wiki [O04 §05].

For the kernel's own build, the equivalent job runs on every pull request (**build recommendation**):

```
# .github/workflows/licence.yml (excerpt)
jobs:
  licence-gate:
    steps:
      - run: syft dir:. -o cyclonedx-json > sbom.json            # SBOM per build
      - run: python ci/licence_gate.py sbom.json \
               --allow MIT,BSD-2-Clause,BSD-3-Clause,Apache-2.0,ISC \
               --register licence/register.yaml \
               --fail-on unknown,blocked,conditional-unconfirmed
      - run: python ci/pin_check.py --no-tags --no-latest          # O04 rule
      - run: python ci/ee_exclusion.py --paths ee/,enterprise/      # O04 decision 4
      - run: python ci/verifier_lockfile_empty.py verifier/go.sum   # E14 item 5
```

ISC is added to the allow-list as a permissive licence functionally equivalent to MIT. That addition is itself a register decision for counsel to confirm (**build recommendation**). The gate fails on any licence it cannot identify, because an unknown licence in box 3 is a blocked licence until someone has read it.

18.13 Security testing, penetration testing and bug bounty

The sources specify a large set of adversarial tests as part of conformance. They do not specify penetration testing or a bug bounty. The table separates what is specified from what this memorandum recommends.

Test	What it proves	Cadence	Status
C2 bypass attempt	No path to the target other than the single writer	Every consumer CI run; against the real target at promotion	Specified [E02; E09]
C5 fail-closed matrix	Registry, rule engine, autonomy table and target each unavailable in turn; every case refuses	CI	Specified [E02]
C6, C7, C9, C11	Missing grant → A0 in 1,000 fuzzed cases; idempotency; chain tamper (flipped octet, reorder, duplicate, gap); approver ≠ requester, agent approver refused	CI	Specified [E02]
C14 claim boundary	No "signed", "tamper-proof" or "independently verifiable" at L1	Consumer CI	Specified [E02]
K2 isolation	No cross-tenant read, list, enumeration, timing inference or side channel	Nightly, every PR	Specified for stack1 [S12 §2]; extended to kernel tables (§18.6)
RLS, region allow-list, secret scan, break-glass alert	As §18.4–18.7	CI; monthly for break-glass	Specified [O05; S12]

Test	What it proves	Cadence	Status
Negative vectors	28 negatives refused, naming record and field (L2)	CI	Specified [E14 §07]
Red-team library	Tamper scripts, kept outside the implementation, against the text and the verifier	Daily red-team hour during build waves	Practice in the build kits [ETR; ERD]
External penetration test	Kernel API, MCP server, console, tenancy, credential broker and bypass, by an independent tester	Before L1 GA; before L2 GA (adding key custody, anchor client and bundle export); then annually	**Build recommendation**
Private bug bounty	Invited researchers against sandbox tenants and the reference tenant	From L1 GA	**Build recommendation**
Public verifier challenge	The published tampered bundle and negative vectors as a standing invitation to find a verifier that accepts a tampered record	From L2 GA	**Build recommendation**

The penetration test should be scoped around the threats in §18.2 rather than a generic web checklist. The specific questions are: can any principal other than the single writer obtain a target credential; can a tenant token reach another tenant's rows; can an agent token approve or raise its own autonomy; can a control token read a record store; can any path produce a chain row out of `seq` order. The OWASP Top 10 for Agentic Applications (ASI01–ASI10) is already the build team's red-team vocabulary and is the natural checklist for the agent-facing surfaces [ETR]. The IM budgets certification and audit at nil in FY2026-27 and ₹0.66 crore in FY2027-28 [IM §10]. The pre-GA penetration test therefore falls in the first quarter of FY2027-28 or must be brought forward into the tranche-1 budget; the conflict between a pre-GA test and a nil FY2026-27 line should be settled when the tranche-1 budget is fixed. Findings from all three programmes become conformance vectors or K2 vectors, so each one remains a regression test (**build recommendation**, following S03's rule that a missed failure "is a missing conformance vector — the fix is the vector" [S03 §5]).

18.14 Certification posture

elan1 holds no certification, and "nobody is permitted to improve on that answer in a meeting" [IM §10]. The sequencing rule is a pre-commitment. elan1 certifies to the standard the first paying evidence customer's auditor names, and not before: ISO/IEC 42001 first for an Indian or European AI-governance conversation; SOC 2 Type I, then Type II, for a US security review. The two are never pursued in parallel on a pre-revenue balance sheet [IM §10]. The gate calendar names the target by 31 March 2029 and requires a certificate or Type II report by 31 March 2030 [IM §16.5]. The gaps against ISO/IEC 42001 are process gaps (AI system impact assessment, internal audit programme, management review), not evidence gaps [IM §10]. One further rule closes the subject: "a certificate changes no product claim." A deck that places a certification logo beside an assurance-level claim is wrong [IM §10].

18.15 The go-live bar, elan1 column

No brand lets a person outside the company log in until every row in its column is ticked and evidenced [O05 §12]. For elan1, whose first outsider is the L1 GA developer (Chapter 16), the bar reads:

Control	Required for elan1	Status (Sept 2026)
ap-south-1, 3 AZ, no data outside India	Yes	Specified
RLS on every tenant table; cross-tenant read test green in CI	Yes	Specified
Credentials minted per call, ten-minute life	Yes	Specified
Signing keys in managed KMS, registered with expiry	Yes per O05 — **but L1 has no signing key**	Not applicable at L1 (conflict)
180-day logs in India; NTP to NPL/NIC	Yes	Specified
Proof record for every governed write, verified on a sample	Yes	Specified
One successful restore drill on record	Yes	Not run; RPO and RTO not set (O05 OQ1)
DPA signed; privacy notice published; consent ledger live	Yes	Not started

Control	Required for elan1	Status (Sept 2026)
Sub-processor list published	Yes	Not published
Named CERT-In responder; six-hour path tested once	Yes	Responder unfilled
Erasure path tested end to end on a fixture tenant	Yes	Not run; depends on §18.9.3
Licence register CONDITIONAL rows counsel-confirmed	Yes	None confirmed
Break-glass tested and alerting	Yes	Not run

Source: [O05 §12, OQ1–2; O04 §03; E01 §07]. The restore drill must re-verify a sample of proof records and run the conformance suite against the restored stack, because "a backup that has never been restored is not a backup" [O05 §07]. Chapter 22 carries the drill.

18.16 Open security, residency and data-protection items

#	Item	Owner	By	Source
1	Does retaining a proof record with erased identifiers satisfy DPDP erasure?	Counsel	30 Nov 2026; hard stop 13 May 2027	O05 OQ4; IM §10
2	Named CERT-In responder and deputy	Founder	30 Sep 2026 (overdue)	O05 OQ2
3	Agreed RPO and RTO	stack1 lead	15 Oct 2026	O05 OQ1
4	Is the Consent Manager framework operative from 13 Nov 2026, and must an early tenant interoperate before 13 May 2027?	Counsel	31 Oct 2026	O05 OQ3
5	Which hosted model providers, if any, are acceptable on the sub-processor list?	Founder, counsel	31 Oct 2026	O05 OQ5
6	The exact F14 salting scheme	elan1 lead	31 Mar 2027	E12 OQ1
7	Where is the line between a separate network service and a shared process for AGPL under Indian law?	Counsel	30 Nov 2026	O04 OQ4
8	When do signed provenance attestations and reproducible builds become commitments?	stack1 lead	30 Jun 2027	S12 OQ3

19. Metering, pricing plumbing and billing

elan1 has never sent an invoice. It has no external price list, no external customer and no metered production traffic. Its only planned consumer, rung1, is itself pre-revenue [O07 §01; E10 §01; IM Appendix A]. This chapter therefore describes machinery, not a revenue history. It covers how one governed action becomes one meter event at rung 9, how that event is stored so that a refusal cannot be billed, how events are reconciled against the hash-chained proof log, how they are aggregated, rated and invoiced to an external tenant, how the same events produce the internal transfer charge to rung1, and how the cost of a governed action is measured against them. Price levels are taken from the pricing model [E10] and the Information Memorandum's price book [IM Appendix A; IM §14.5]. Where the two disagree, and they do, the disagreement is set out in §19.10 and recorded in the conflicts block.

The unit is the same object at all four points. A governed action is one write to a system of record that was permitted, executed, recorded and billed: "One row in the chain, one meter event, one line on the invoice, one thing an auditor can open. The invoice line and the evidence record are the same object — that is the whole design, and it is also the pricing argument" [E10 §02]. Every meter event carries a `write_id` and the record's `seq`; a line with no `write_id` cannot be invoiced, because such a line cannot exist in the meter's schema [O07 §04].

Designed is not shipped (DR-010). Every latency, lag, cost and price figure in this chapter is a target or a model. The meter lag target (p99 ≤ 30 s) [O03 §08; S02], the rung 9 budget (2 ms p50) [E04 §02], the modelled marginal cost per governed action (₹0.20) [E10 §07], the internal transfer price (₹0.40) [E10 §04; O07 decision 3] and every tier price [E10 §03; IM Appendix A] are unmeasured. The IM itself calls cost per governed action "the single most load-bearing modelled number in the memorandum, and nothing behind it has been measured" [IM §17.5].

19.1 The Governed Action as the billable unit

19.1.1 Definition and consequences

The Governed Action is defined once for the whole group: the unit that is "permitted, executed, recorded and billed — the same unit at all four points" [O03 §07; O07 §02]. It is deliberately a different noun from the units competitors bill. Microsoft Agent 365 bills a seat, Salesforce Agentforce bills an agent step, and IBM watsonx.governance bills a compute abstraction [E10 §02]. elan1 bills "one committed write to a system of record" [E10 §02]. The choice constrains the meter three ways. The count cannot inflate with model chattiness, so there is one event per governed write and never one per model step, tool call or retry: "two steps or forty, one invoice submitted is one governed action" [E10 §02]. The event is emitted by the kernel after rung 8 and references a proof record, never by the caller [E02 §03; E04 rung 9]. And the cost that scales is the never-compacting proof storage and the shadow-sandbox apply, not compute [E10 §07].

19.1.2 Three brands, three units, one owner per number

The group sells three different units and forbids dividing one into another without saying so: rung1 sells a band-month plus credits, counted in rung1's credit ledger; elan1 sells a committed Governed Action, counted by the kernel meter at rung 9; stack1 sells a stack-node-year, with runtime units (`token_in`, `token_out`, `gpu_second`, `byte_stored`, `page_ingested`, `tool_call`) counted at its contract edge. Refusals cost no credits, meter at quantity zero, and are recorded at ₹0 respectively [O07 §02–§03; S02]. The meter design keeps the three apart at the schema level, not only in slideware. **A credit is not a Governed Action**: one rung1 customisation is modelled at 600–1,800 credits and may contain one governed write or none [O07 §02]. The kernel meter is normative. rung1's credit ledger "is a rendering, not a source", and when it disagrees with the kernel "the kernel is right and the ledger is a display defect" [O07 §03; R13 §1].

19.2 Where the meter lives: rung 9

The meter is a kernel component, not a product feature and not a runtime service. O01 places it inside P3, the kernel plane, beside the one write function, the nine rungs, `evaluate()` and proof [O01 §02]. O03 lists the meter in `elan1-core`, which "nobody imports above P3" [O03 §12], and states that rung1 may never change "the meter" [O03 §10]. DR-001 forbids any product from building its own meter [O01 §05]. E01 lists "the meter" as one of the ten things L1 must contain at full strength: "one meter event per governed write, basis taken from the write type's declared blast radius, never configurable per customer. Append-only, reconciled nightly in both directions. Every invoice line traces to a record" [E01 §02 item 9].

Rung 9 METERED is the last rung of the Right Path. It "checks the meter basis from the declared blast radius. Does: emit exactly one meter event per governed write. Refusals meter at zero, enforced by a database constraint … Caller sees nothing. Recorded: one `meter_event`" [E04 rung 9]. Its design budget is 2 ms p50, and it is marked non-deterministic because it performs I/O [E04 §02]. The earlier product architecture names the same component `S12 meter`, deployed as the service `e1-meter` in zone Z3, the record plane [EPA; EBB EB-02; ETA].

The meter also has a relationship with stack1 that must not be confused with this one. stack1 runs its own runtime meter at the contract edge and states that "secrets, metering and the deploy pipeline are stack1's own code and are not substitutable" [S03 decision 5]. That meter counts runtime units. It never counts governed actions, and the elan1 kernel meter never counts tokens [O07 §03].

19.3 What produces a meter event, and what does not

The outcome of every KernelPort interaction falls into one of the classes below. The table is the specification the rung 9 emitter and the reconciliation job are both tested against.

Outcome	HTTP	Proof record	Meter event	Billable	Source
Committed governed write	200	Yes	Yes, quantity 1 in the declared basis	Yes	[E02 §03; E05 §07]
Refusal, `POLICY.*` or `GATE.*`	200	Yes, record identifier zeroed	Yes, quantity 0	Never	[E02 C8; E05 §07]
Escalation (awaiting a human)	200	Yes	Yes, quantity 0	No	[E05 §07]
Declared G5 rate or G6 budget gate failure (`GATE.RATE_EXCEEDED`, `GATE.BUDGET_EXHAUSTED`)	200	Yes	Yes, quantity 0	No	[E05 §06]
Platform failure (`PLATFORM.*`) after the kernel was reached	5xx, or 200 with a platform code	Yes, platform class	Quantity 0 with the platform code (recommended; see note); pages	No	[E02 §04; E05 §07]
Tenant stopped at rung 0b (`POLICY.TENANT_STOPPED`)	200	**No** — the only refusal without one	No	—	[E05 §07; E04 rung 0b]
Rejection: malformed request, nothing entered the kernel	4xx	No	No	—	[E02 §04]
Transport throttle	429	No	No	—	[E05 §06]
Idempotent repeat of an `intent_id` / `idempotency_key`	200, original outcome	No new record	No second event	—	[E02 C7; E05 §08]
`verify`, `scope`, reads, polls of `/v1/writes/{id}`	200	No	Never	—	[E02 §05; E05 §09]
Sandbox-key write (developer tenant)	200	Yes, `mode: sandbox`	Yes, quantity 0	Never	[E05 §05]
Offline verification by anyone	—	—	Never; the verifier is free and verification "is never metered"	—	[EPA; E07 §04]

Three classes need an explicit build decision because the sources describe them as governed writes without saying how they meter:

1. **An approval.** "An approval is itself a governed write. It is not a flag" [E02 §03]. rung1 states that the elan1 approval step must never consume credits, because "charging for the control undermines the control" [R13 §1].
2. **Registering a declaration version.** `define` "is itself a governed write under `dual_control: true`, so it climbs the rungs like any other and produces its own record" [E02 §05].
3. **An evidence export at L2.** A bundle export is recorded "as a governed write … Metered at the `one_tenant` basis; the bundle itself is priced ₹0" [E15 §05].

> **Does a platform refusal emit a meter event?** E02's table says a platform failure writes a full record and is "not billable" [E02 §04], and C8 requires "a meter event of quantity 0" on every refusal [E02 C8]. E05's error taxonomy instead marks the platform-failure meter column "no" [E05 §07]. If platform refusals emit nothing, the reconciliation rule "every record one meter event" [E02 C10] fails for every incident. We recommend emitting a quantity-0 event carrying the `PLATFORM.*` code, which keeps C10 exact, keeps the event out of every bill, and gives the incident cost line in §19.13 a join key (**build recommendation**). The conflict is recorded.

We recommend that all three emit meter events with a `meter_class` other than `governed_action` and a quantity of 0 at L1 (**build recommendation**). The reconciliation invariant "every record one meter event" then still holds, the count of control writes stays visible to the operator, and the approval step can never appear as a billable line. The evidence export case is taken up again in §19.9.

19.4 The meter event schema

19.4.1 The seven normative fields

E02 freezes seven fields for the meter event at KernelPort v1 [E02 §03]:

Field	Type	Meaning
`event_id`	uuid	Kernel-assigned
`write_id`	uuid	Always present; every invoice line traces to a record
`seq`	uint64	The proof record this event refers to
`basis`	enum	From the write type's declared blast radius, never configurable per customer
`quantity`	uint32	1 for a committed write; 0 for every refusal, enforced by CHECK
`outcome`	enum	`committed` · `refused` · `escalated`
`at`	RFC 3339	Platform clock, never the caller's

The basis mapping is normative and not configurable. "If the basis were a pricing parameter, every negotiation would reopen it and we would accumulate per-customer metering logic" [E05 §06]. E09 refuses to onboard an applicant who needs a per-customer meter basis, with the sentence "The basis is not a pricing parameter. Here is the mapping" [E09 §08].

Declared `blast_radius`	Meter basis
`one_record`	per record
`one_counterparty`	per counterparty
`batch`	per batch
`tenant`	per tenant
`cross_boundary`	per crossing

Source: [E05 §06; E10 §03]. The basis is resolved once, at `define` time, and returned in the `write_types` array of the `define` response ("registered names with their resolved meter basis and blast radius") [E02 §05]. The developer sees it in the Studio "before they have written a line of integration code" [E09 §04 step 2]. Rung 9 reads the resolved basis from the compiled registry entry at the pinned declaration version. It never recomputes it from the payload.

19.4.2 Join keys the seven fields do not carry

The seven fields are enough to bill. They are not enough to cost, attribute to a pack, or report by mode. Four further needs are stated in the sources:

- The cost method needs "the run id, the tenant, the write type, the blast radius and the outcome" on every event [R13 §6].
- O06 requires every decision to carry the pinned `declaration_version` and the resolved `policy_version` [O06 §05].
- Enforced and attested-advisory deployments make different claims and are priced differently [E01 §05; IM §4.6], so they must never be blended in a report.
- A sandbox-key event must be distinguishable from a production one, as it is in the proof record [E05 §05].

The sketch below keeps E02's seven fields as normative and adds the join keys as `x-` extensions. The extension names are a **build recommendation**.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://schemas.elan1.internal/meter-event/v1.json",
  "title": "elan1 kernel meter event v1",
  "type": "object",
  "additionalProperties": false,
  "required": ["event_id","write_id","seq","basis","quantity","outcome","at",
               "tenant_id","meter_class","write_type","declaration_id",
               "declaration_version","policy_version","mode","key_class"],
  "properties": {
    "event_id":  {"type": "string", "format": "uuid"},
    "write_id":  {"type": "string", "format": "uuid"},
    "seq":       {"type": "integer", "minimum": 1},
    "basis":     {"enum": ["one_record","one_counterparty","batch","tenant",
                           "cross_boundary"]},
    "quantity":  {"type": "integer", "minimum": 0, "maximum": 1},
    "outcome":   {"enum": ["committed","refused","escalated"]},
    "at":        {"type": "string", "format": "date-time"},
    "tenant_id": {"type": "string", "format": "uuid", "x-source": "E02 §04"},
    "meter_class": {"enum": ["governed_action","approval","define",
                             "evidence_export"], "x-source": "build rec."},
    "run_id":    {"type": ["string","null"], "x-source": "R13 §6"},
    "write_type":{"type": "string", "examples": ["erp.sales_invoice.submit"]},
    "declaration_id":      {"type": "string", "format": "uuid"},
    "declaration_version": {"type": "integer", "minimum": 1, "x-source": "O06 §05"},
    "policy_version":      {"type": "integer", "minimum": 0, "x-source": "O06 §05"},
    "refusal_code": {"type": ["string","null"],
                     "pattern": "^(POLICY|GATE|PLATFORM)\\.[A-Z_]+$"},
    "mode":      {"enum": ["enforced","attested_advisory"], "x-source": "E01 §05"},
    "key_class": {"enum": ["production","sandbox"], "x-source": "E05 §05"},
    "pack_id":   {"type": ["string","null"], "x-source": "build rec., §19.9"}
  },
  "allOf": [
    {"if": {"properties": {"outcome": {"enum": ["refused","escalated"]}}},
     "then": {"properties": {"quantity": {"const": 0}}}},
    {"if": {"properties": {"key_class": {"const": "sandbox"}}},
     "then": {"properties": {"quantity": {"const": 0}}}},
    {"if": {"properties": {"meter_class": {"not": {"const": "governed_action"}}}},
     "then": {"properties": {"quantity": {"const": 0}}}}
  ]
}
```

Two schema choices need a note. `quantity` is capped at 1 here although E02 types it as `uint32`. E02's own text says "1 for a committed write", and the basis names what that one unit is, so at L1 no committed write produces a quantity above one [E02 §03]. The wire type stays `uint32` so that a later declaration version could meter a multi-crossing write without a port

change (**build recommendation**). Second, `run_id` is nullable. It is rung1's cost join key and a third-party consumer may not have one. The cost method then falls back to joining "on tenant and period" [R13 §6].

19.5 The quantity-zero rule, enforced by the database

19.5.1 Why it is a constraint and not a policy

"Refusals are free and the database enforces it. A meter row whose write outcome is a refusal and whose quantity is non-zero cannot be inserted. When a CFO asks whether we are incentivised to make the system say no — and they ask within ten minutes — the answer is a constraint, not a policy" [E10 §02]. O07 adds the structural half: "The meter event schema requires a quantity, a refusal's quantity is 0, and the database refuses to store anything else. A caller cannot forget to pass zero, because the caller does not pass it" [O07 §03]. The rule survived an internal argument. The earlier commerce1 material billed refusals at 0.3× on the reasoning that "a free refusal is an incentive to refuse", and E01 overrules it with a stated condition for revisiting: "Revisit only if a customer shows us refusal-farming" [E01 §09, decision 4; IM §4.8].

The quantity-zero rule therefore has three layers of enforcement, and each is independently tested:

1. **The caller never supplies a quantity.** Neither `assert` nor `act.write` accepts one. Rung 9 derives it from the outcome, the key class and the meter class [O07 §03; E02 §05].
2. **The database refuses the row.** A CHECK constraint makes a non-zero refusal physically unstorable [E10 §02; E04 rung 9; E05 §06].
3. **Reconciliation asserts it anyway.** The nightly job checks that every refusal carries quantity 0, even though the CHECK makes a violation impossible. This is conformance class C10 and E01 done-item 7 [E02 C10; E01 §06].

19.5.2 Constraint sketch

The DDL below is a **build recommendation** that implements [E02 §03; E05 §06; E01 §06 item 7; EBB EB-11 §7]. It targets PostgreSQL 16+, the recorded choice for the meter tables [ETS §05].

```
CREATE TYPE meter_basis   AS ENUM ('one_record','one_counterparty','batch',
                                   'tenant','cross_boundary');
CREATE TYPE meter_outcome AS ENUM ('committed','refused','escalated');
CREATE TYPE meter_class   AS ENUM ('governed_action','approval','define',
                                   'evidence_export');

CREATE TABLE meter_event (
  event_id            uuid          PRIMARY KEY,
  tenant_id           uuid          NOT NULL,
  write_id            uuid          NOT NULL,         -- no write_id, no row
  seq                 bigint        NOT NULL CHECK (seq >= 1),
  basis               meter_basis   NOT NULL,
  quantity            integer       NOT NULL CHECK (quantity BETWEEN 0 AND 1),
  outcome             meter_outcome NOT NULL,
  meter_class         meter_class   NOT NULL,
  key_class           text          NOT NULL CHECK (key_class IN
                                              ('production','sandbox')),
  mode                text          NOT NULL CHECK (mode IN
                                              ('enforced','attested_advisory')),
  write_type          text          NOT NULL,
  declaration_id      uuid          NOT NULL,
  declaration_version integer       NOT NULL CHECK (declaration_version >= 1),
  policy_version      bigint        NOT NULL,
  refusal_code        text,
  run_id              text,
  pack_id             text,
  at                  timestamptz   NOT NULL DEFAULT clock_timestamp(),
  -- The rule. Only a committed, production, governed_action row may be non-zero.
  CONSTRAINT refusal_meters_zero CHECK (
    quantity = 0 OR (outcome = 'committed'
                     AND key_class = 'production'
                     AND meter_class = 'governed_action')),
  CONSTRAINT refusal_has_code CHECK (outcome <> 'refused' OR refusal_code IS NOT NULL),
  CONSTRAINT one_event_per_record UNIQUE (tenant_id, seq),
  CONSTRAINT one_event_per_write  UNIQUE (tenant_id, write_id, meter_class)
```

```
) PARTITION BY LIST (tenant_id);

ALTER TABLE meter_event ENABLE ROW LEVEL SECURITY;
ALTER TABLE meter_event FORCE  ROW LEVEL SECURITY;

-- Insert-only. The rung 9 writer may INSERT; nobody may UPDATE, DELETE or TRUNCATE.
REVOKE ALL    ON meter_event FROM PUBLIC;
GRANT  INSERT ON meter_event TO e1_meter_writer;
GRANT  SELECT ON meter_event TO e1_meter_reader, e1_reconciler, e1_billing;
```

The Build Book's constraint is weaker than the rule. The earlier product material writes the constraint as `CHECK (outcome <> 'refused' OR quantity = 0)` [EBB EB-02; EPA]. That form lets an `escalated` row carry a non-zero quantity. E05 states that an escalation meters zero [E05 §07], and E02 says quantity is 1 only "for a committed write" [E02 §03]. The constraint above is written positively ("non-zero only if committed"), so any future outcome value added to the enum defaults to unbillable. The conflict is recorded at the end of this chapter.

Append-only is enforced as an absence of grants, not as a trigger: "Grants are the control; triggers are belt-and-braces" [ETS §05]. The grant-checker that runs in CI and nightly reads the live catalogue and fails if any role holds `UPDATE`, `DELETE` or `TRUNCATE` on `meter_event` [ETS §09]. The Build Book's operations rule applies: operations never "edits an append-only table — there is no grant, so this is not a policy" [EBB EB-11 §9].

Correcting a wrong meter row therefore never updates it. A billing correction is a new row in a separate `meter_adjustment` table that references the original `event_id`, carries a reason code and a named approver, and is itself subject to dual control (**build recommendation**). Rating reads events and adjustments together. The proof log is never adjusted, because it records what happened. The meter records what is billable, and only the billable view may be corrected.

19.6 Emission at rung 9

19.6.1 Transaction boundaries

Rung 7 ACT commits the record version and exactly one outbox row in one transaction. Rung 8 PROOF consumes the outbox in commit order, assigns the tenant's next `seq` and appends the record. Rung 9 METERED then emits one meter event and "returns the decision" [E02 §05 assert steps 8–10; E04 rungs 7–9]. The meter event references a `seq`, so it cannot be written before rung 8 has assigned one. That ordering is why the proof record, not the meter, is the source of truth for reconciliation.

We recommend that rung 9 writes the meter event in the same database transaction that appends the proof record at rung 8, on the same Postgres primary (**build recommendation**). This makes "every record has exactly one meter event" true by construction for every outcome that produces a record. It also removes the one failure mode reconciliation otherwise has to chase, which is a record appended with its meter event lost. It costs nothing in the latency budget, because rung 8 (12 ms) and rung 9 (2 ms) are adjacent I/O on the same store [E04 §02]. It also matches stack1's load-bearing rule for its own meter: "The meter row and the contract call must commit in one transaction. No broker gives that" [STS §01].

19.6.2 Synchronous or asynchronous

E02 leaves one question open: "Does `assert` return synchronously through rung 9, or does the consumer get the decision at rung 8 with metering asynchronous? Synchronous is simpler to reason about and costs latency" (elan1 lead, 31 October 2026) [E02 Q1]. The recommendation above answers it for the write path. The meter row commits with the proof row, and `assert` returns after both. The *transport* from the kernel's meter table to the billing store is asynchronous. It runs through an outbox relay, under the group's meter lag target of p99 ≤ 30 s [O03 §08]. Billing never reads the `assert` response, so the choice made for E02 Q1 does not change the billing pipeline.

19.6.3 Failure modes at rung 9

Failure	Behaviour	Why
Meter insert violates `refusal_meters_zero`	The rung 8/9 transaction aborts; the chain writer retries with a quantity recomputed from the outcome; a violation that persists pages as a P1 kernel defect	A caller cannot cause this; only a kernel defect can
Meter store unavailable	The proof append and meter insert share a transaction, so both wait; the outbox row from rung 7 survives and the chain writer retries in order	"No lost write, no gap in the sequence" [E02 §05 failure paths]
Relay to billing store lags past 30 s p99	Alert only; billing reads per closed period, so lag delays a period close and never mis-bills	[O03 §08]

19.7 Idempotency across the pipeline

Idempotency is specified at the kernel, required of consumers and repeated at every stage downstream.

- **Kernel.** `assert` is "keyed on `intent_id`. A repeat returns the original decision including `seq` and `digest`; it does not re-run the rungs and does not meter again" [E02 §05]. Conformance class C7 tests exactly this: "A repeated `intent_id` returns the original decision, `seq` and `digest`, and produces no second meter event" [E02 C7].
- **Surface.** Every `act` call takes a client-supplied `idempotency_key`, scoped to the tenant and retained 30 days. Clients retry 5xx and 429 with backoff, at most five attempts, "always with the same key", and never retry a refusal [E05 §08].
- **Workflow.** Anything that touches a customer's books runs inside a Temporal workflow [O01 DR-002; S04 §01]. Temporal replays the decision and never the effect, and the approval arrives as a durable signal, not a callback [S04 §02; S03 decision 6]. A consumer's run that parks while the kernel is unreachable therefore resumes at the same `assert` with the same key and cannot produce a second event.
- **Billing store.** The relay upserts on `event_id`. Aggregation reads distinct `(tenant_id, seq)` only (**build recommendation**).

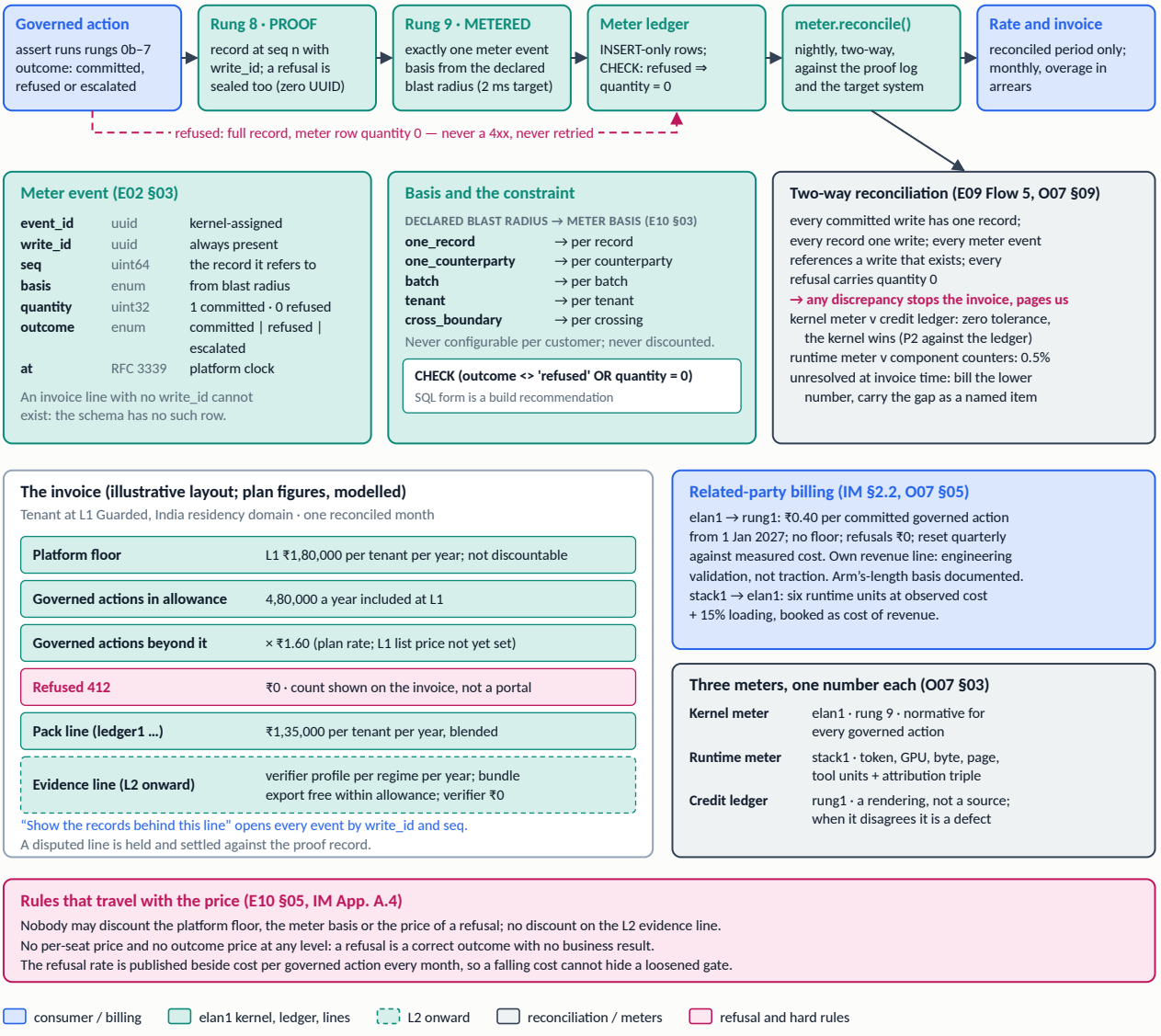
19.8 Reconciliation between the proof log and the meter ledger

Figure 28 shows the pipeline end to end: emission at rung 9, the two-way reconciliation against the proof log, and the path to an invoice and to the rung1 transfer statement.

Figure 28 — Metering pipeline: rung 9 to the invoice

DESIGNED — PRICES MODELLED, NOT QUOTED

One governed action is one record, one meter event and one invoice line; refusals are quantity 0 by database constraint.



Source: E02 §03–§04, E04 rung 9, E09 Flow 5, E10 §02–§06, O07 §02–§09, IM §2.2, §4.8, App. A.1–A.4 · elan1 Technology Memorandum · Sept 2026

Figure 28 — The metering pipeline: one meter event per governed action at rung 9, committed with the proof record; refusals, escalations and sandbox writes at quantity zero under a database CHECK; nightly two-way reconciliation against the hash-chained proof log; evidence and pack lines at rating; external invoices and the internal rung1 transfer statement produced from the same events.

19.8.1 The invariant

Conformance class C10 states it: "Every committed write has one record, every record one write, every meter event a write that exists, every refusal quantity 0" [E02 C10]. The Build Book's nightly reconciliation lists five checks in both directions [EBB EB-11 §8]:

1. every committed write has exactly one seal (at L1, one chained record);
2. every seal has exactly one write;
3. every write has exactly one `meter_event`;
4. every `meter_event` references a write that exists;
5. every refusal's `meter_event` has quantity = 0.

"Any non-zero on the first two is control C04 firing and it is a P0. Divergence on the meter lines is a billing incident and a ticket" [EBB EB-11 §8]. E01 names the same pair as L1 done-items 3 and 7 [E01 §06].

19.8.2 The queries

The reconciliation job is exposed as `meter.reconcile()` [O03 §07; O07 §09] and publishes its result on `/v1/meter` (reconciliation) and `/v1/ops` [E05 §04; EBB EB-11 §8]. Each check is a query that must return zero rows (**build recommendation**; invariants from [E02 C10; EBB EB-11 §8]):

```
-- M1: meter events pointing at no proof record
SELECT m.event_id FROM meter_event m
LEFT JOIN proof_record p USING (tenant_id, seq)
WHERE p.seq IS NULL;

-- M2: proof records with no meter event (tenant-stop refusals write neither)
SELECT p.tenant_id, p.seq FROM proof_record p
LEFT JOIN meter_event m USING (tenant_id, seq)
WHERE m.event_id IS NULL;

-- M3: outcome disagreement between record and meter
SELECT m.event_id FROM meter_event m JOIN proof_record p USING (tenant_id, seq)
WHERE (p.record_id = '00000000-0000-0000-0000-000000000000') <> (m.outcome <> 'committed');

-- M4: non-zero quantity on anything but a committed production action
--     (impossible under the CHECK; asserted anyway)
SELECT event_id FROM meter_event
WHERE quantity <> 0 AND NOT (outcome = 'committed' AND key_class = 'production'
                             AND meter_class = 'governed_action');

-- M5: committed writes in the target system with no record  (E11 test 3)
--     run per write type against the target's own audit trail
```

M3 uses the record shape from E02. A refusal is "a full record with zeroed record identifier" and `record_version` 0 [E02 §03 F12–F13; E02 C8]. M5 is the reconciliation to the target system that E11 names test 3: "Every change in the target system's … audit trail … must be reconciled" [E11 §07]. It is the one check that can detect a write made around the kernel in attested-advisory mode. That is why the claim boundary for that mode says only "every write this product made through elan1 is recorded" [E01 §05].

19.8.3 When two meters disagree

Disagreement	Tolerance	Action	Source
Kernel meter against rung1's credit ledger	Zero	Kernel wins; P2 defect against the ledger with a named owner	[O07 §09]
Invoice line with no traceable record	Zero	Line removed and credited; "it should have been impossible to create"	[O07 §09]
Proof ↔ meter (M1–M4)	Zero	Billing incident and ticket; the period cannot close	[EBB EB-11 §8; E09 decision 4]
Unresolved at invoice time	—	Invoice the lower number; carry the difference as a named open item with owner and date	[O07 decision 11]

19.8.4 Period close

"An invoice is never issued over a period whose meter reconciliation has not completed cleanly" [E09 decision 4; E10 §03]. E09's Flow 5 sets the sequence. The system closes the period and runs the two-way reconciliation, and "a non-zero discrepancy stops the invoice and pages us". If reconciliation cannot complete, "no invoice is issued, the consumer is told the period is open and why, and it pages us" [E09 §07]. Period close is therefore a state machine with an explicit blocked state (**build recommendation**):

```
OPEN --(period end + 24 h)--> CLOSING --every night clean--> RECONCILED --> RATED --> INVOICED
                                |                                ^
                                +--any M1-M4 non-zero--> BLOCKED --fixed--+
                                                                 (pages; tenant told "period open")
Late events for a closed period never enter it; they land in the next period, flagged.
```

The "lower number" rule in O07 applies only to disagreements *between meters*, such as the kernel against the credit ledger. It never licenses invoicing over a period whose proof-to-meter reconciliation is failing. That case is BLOCKED, as E09 requires.

19.9 Evidence line and pack line metering

19.9.1 What exists at L1

At L1 there is no evidence line to meter. "At L1 there is no bundle, no signature, no anchor and no verifier profile, so there is nothing to sell and selling it anyway is the one unrecoverable reputational error available to us" [E10 §06]. E01 excludes "evidence export as a product" from L1 [E01 §03]. The L1 meter therefore carries only `governed_action` events plus the quantity-zero control classes of §19.3.

19.9.2 Evidence at L2 and L3

The Build Book records that the meter can express whichever evidence shape wins, "because the underlying facts are already recorded: which bundles were built, under which profile, for which regime, containing which seals" [EBB EB-17 §9]. The sources name three shapes and do not agree on prices (see §19.10.3). For the plumbing, what matters is which fact each shape counts:

Evidence line	Unit counted	Fact it is rated from	Metered as	Source
Bundle export	One bundle	Export recorded as a governed write	`meter_class = evidence_export`, basis `tenant`, quantity 0; counted at rating against the allowance	[E15 §05; IM Appendix A.3]
Verifier profile	Per regime per year	Profile activation for a tenant, a dated entitlement	Entitlement row, not a meter event	[E10 §06; E07 §04]
The offline verifier	—	—	**Never metered, never priced**	[E07 §04; E10 §06; IM Appendix A.3]

E15 says the export is metered at "the `one_tenant` basis". The E05 enum has no `one_tenant` value; its value is `tenant` [E05 §06]. We read E15 as meaning `tenant` and record the spelling as a minor conflict. Two further rules constrain the evidence line. There is no discount on it, ever [O07 §08]. And the meter "must not … start charging for verification" [EBB EB-17 §9]. A reviewer who verifies a bundle offline leaves no trace in any elan1 system, by design [E15 §06].

19.9.3 Packs

A pack is configuration: a declaration, policy rules, regulatory bindings, an eval set and an approver model [E18 §01; IM §5.1]. Its commercial line in the price book is a subscription, "Regulatory pack, per tenant per year (blended)", at ₹1,35,000 [IM §14.5; IM Appendix A.3]. collect1's sketch prices "per governed call, refusals at zero, evidence sold separately per regime" [E27 §10]. Two meter requirements follow:

1. **Pack attribution on every event.** A governed action taken under a pack's write types must be attributable to the pack, so that the pack's own economics (cost per governed call, refusal rate, evidence uptake) can be reported per pack. The declaration a pack binds to is known at `define` time [E18 §06 `declaration_ref`]. We recommend that `define` records `pack_id` against each write type in the compiled registry entry, and that rung 9 copies it onto the meter event (**build recommendation**). No per-pack meter logic exists. The basis still comes only from the declared blast radius.
2. **Pack lines are entitlements, not meter events.** The per-tenant-per-year pack fee is an entitlement row with a start date, an end date and a price-book reference. Rating joins it to the period. The governed actions under the pack are billed through the ordinary platform line, not twice.

19.10 Price book mechanics as the rating engine sees them

19.10.1 The structure the system must support

Every source agrees on the shape: a **platform floor plus a rate per governed action**, with refusals at ₹0 [E10 §03; E07 §04; IM §14.5]. The floor exists because "the marginal cost of one governed action is low and the fixed cost of running an evidence plane is not" [IM Appendix A.1; E07 §04]. The rules that travel with the prices are constraints the rating engine must encode, not guidance [IM Appendix A.4]:

Rule	Encoded as	Source
Refusals are ₹0	The refusal line is priced by constant, not looked up	[E10 §08; IM Appendix A.4]
The floor is not discountable, by anybody	`discount.target <> 'floor'` enforced in the contract table	[E10 decision 2; IM Appendix A.4]
Overage discount: elan1 lead ≤ 20% for ≤ 12 months; founder unlimited	Discount rows carry `granted_by`, `pct`, `end_date`; CHECK on the lead's rows	[E10 §05]
Every discount has an end date	`end_date NOT NULL`	[O07 §08]
The meter basis and the refusal price are never negotiable	No contract field for either exists	[E10 decision 3]
No per-seat and no outcome price anywhere	No seat or outcome dimension exists in the rating schema	[O07 decisions 7–8]
Pilots free, 90 days, 2,000 actions, hard stop, at most five concurrent	Entitlement with a quantity cap and an end date	[E10 §03, decision 6]

19.10.2 The pilot hard stop

"Hard stop at the limit rather than overage, because a free pilot that generates an invoice is a support ticket and an apology" [E10 §03]. The sources do not say where in the path the stop is enforced. A declared G6 budget gate would record every over-limit attempt as a refusal [E05 §06], but the budget gate is the tenant's own commercial control and should not be overloaded with elan1's entitlement. We recommend that the entitlement service, reading the aggregate, sets the tenant's stop flag with reason class `entitlement_exhausted` when the pilot allowance is consumed (**build recommendation**). Writes then refuse at rung 0b with `POLICY.TENANT_STOPPED`, reads and verification continue, and the `tenant.stopped` webhook fires with its reason class [E05 §09]. At day 90 the same mechanism freezes the tenant read-only for 30 days, after which it is exported and deleted, with nothing billed automatically and nothing deleted silently [E10 §03].

19.10.3 Two price books

The sources carry two incompatible price books for the same SKU. Rating can be built for either, and the tables are effective-dated so that one can replace the other. The build must not hard-code either.

Element	E10 Pricing Model v1 (21 Sep 2026)	IM price book (22 Sep 2026)
Floor structure	Six tiers, monthly: Sandbox 0; Pilot 0; Starter ₹40,000; Growth ₹1,25,000; Scale ₹4,00,000; Enterprise ₹10,00,000+	By assurance level, per tenant per year: L1 ₹1,80,000; L2 ₹6,00,000; L3 ₹24,00,000; sovereign ₹1,20,00,000
Included actions	5,000 / 25,000 / 1,00,000 / 5,00,000 per month	4,80,000 / 15,00,000 / 42,00,000 per year
Overage per action	₹6.00 / 4.00 / 2.50 / 1.50 by tier	₹1.60 / 1.20 / 0.90 by level; sovereign ₹0.55
Verifier profile	₹6,00,000 per regime per year, indicative	₹3,00,000 per regime per year
Bundle export	₹0; "rationing the export teaches the customer not to use it"	₹2,500 each beyond allowance; free within allowance
L2 attestation add-on	₹2,00,000 per month, indicative	Folded into the L2 floor
Internal transfer to rung1	₹0.40 per committed action	₹0.40 per committed action

Sources: [E10 §03, §06; IM §14.5; IM Appendix A.1–A.3]. The IM states that the L1 overage rate it uses "is not a price the company has committed to externally", and that elan1's external list price at L1 GA is an open question owned by the founder, due before L1 GA in Q2 2027. That price is bounded by the ₹0.40 internal price and the ~₹8.80 Agentforce anchor [IM Appendix A.7; O07 Q2]. Both books agree on everything the *meter* needs: the unit, the basis, quantity-zero refusals, a floor plus per-action overage, a free verifier and a ₹0.40 transfer. They disagree only on rating parameters. The price book is therefore a data table in the billing store, keyed by `sku`, `effective_from` and `effective_to`, never a constant in code (**build recommendation**).

19.11 Aggregation, rating and invoicing

19.11.1 Aggregation

Aggregates are rebuildable from `meter_event`, `meter_adjustment` and the entitlement tables alone. Nothing in billing is a source of truth that cannot be recomputed (**build recommendation**). The grain is tenant × period × `meter_class` × outcome × `mode` × `key_class` × `write_type` × `pack_id`, with the `seq` range for each cell, held in a materialised view `meter_period_agg`. Hosted and attached rung1 deployments, and enforced and advisory channels, are always kept apart [R13 §6 rule 6; E01 §05].

Periods are cut in IST, because the invoice and the GST return are Indian documents (**build recommendation**).

19.11.2 Rating

Rating is deterministic and reads no model, in keeping with the rest of the kernel's economics (**build recommendation**). The sketch works for either price book in §19.10.3.

```
def rate(tenant_id: UUID, period: Period) -> list[Line]:
    if reconciliation_state(tenant_id, period) != "RECONCILED":
        raise PeriodOpen(tenant_id, period)          # E09 decision 4
    c   = contract_at(tenant_id, period.start)       # sku, level/tier, modifiers
    pb  = price_book_at(c.sku, period.start)         # effective-dated rows
    agg = aggregate(tenant_id, period, key_class="production")

    lines = [Line("platform_floor", pb.floor_for(c), trace=None)]   # never discounted
    used  = agg.billable_units(meter_class="governed_action")
    over  = max(0, used - pb.included_for(c, period))
    incl  = min(used, pb.included_for(c, period))
    lines.append(Line("governed_actions_included", 0, qty=incl,
                      trace=agg.seq_ranges(outcome="committed")))
    lines.append(Line("governed_actions_overage", pb.overage_rate(c), qty=over,
                      trace=agg.seq_ranges(outcome="committed")))
    lines.append(Line("refused", 0, qty=agg.events(outcome="refused"),  # always shown
                      trace=agg.seq_ranges(outcome="refused")))
    lines += entitlement_lines(tenant_id, period, pb)  # packs, profiles, seats (L2+)
    for d in c.discounts_active(period):
        assert d.end_date is not None and d.target not in {"floor", "evidence", "refusal"}
        lines.append(d.apply(lines))
    return lines
```

19.11.3 The invoice a tenant sees

E09 fixes the external invoice's content. It shows "the platform floor for the tier, the included volume, the overage at the tier rate, and the refusal line showing the count at ₹0". The finance contact "sees the refusal count on the invoice itself, not buried in a portal". A control, "Show the records behind this line", opens every event with its `write_id` and `seq` [E09 §07 steps 2–3; E09 decision 5]. A disputed line "opens a dispute against the named `seq` range, the line is held, and the rest of the invoice proceeds" [E09 §07 step 4]. The dispute is settled against the proof record. If the record supports the line it stands. If not, "the line is credited in full and a defect is opened against the meter" [O07 §04].

The IM calls this invoice "the best product demonstration the company has": "412 refused, Rs 0, on the same table as the billed lines" [IM §4.8]. The engineering consequence is that the invoice renderer must be able to page through every `seq` behind a line in one tenant's chain. `/v1/evidence` "pages by `seq`, not by offset or cursor timestamp", so the invoice link reuses that endpoint instead of a billing-specific one [E05 §09].

Tax treatment is not specified anywhere in the elan1 sources. Every price is quoted ex-GST [E10 §03]. We recommend that elan1's invoices are GST tax invoices produced by elan1's own accounting system, which carries the SAC, place of supply and the CA-confirmed rate. The e-invoice IRN is generated once elan1's turnover crosses the threshold that applies to it, with the rate and threshold confirmed by the company's CA and never hard-coded (**build recommendation**).

19.12 Internal transfer billing to rung1

19.12.1 Terms

rung1 is elan1's first production consumer and a related party. What rung1 pays elan1 is booked as an internal transfer, not revenue [E10 §04; IM §2.2].

Term	Value	Source
Rate per committed Governed Action	₹0.40 [modelled], reset quarterly against measured cost	[E10 §04; O07 decision 3]
Refusals, escalations	₹0	[E10 §04; O07 §05]
Platform floor	None for rung1	[E10 §04]
Charged, not stated	From 1 January 2027	[O07 decision 2]
Price floor rule	No price below measured unit cost except by the founder, in writing, with an end date	[O07 decision 5]
External use of the number	Never quoted externally; never presented as elan1 traction	[E10 §04 rules 1–2, decision 5]
stack1 → elan1 (the other direction of cost)	Observed unit cost plus a flat 15% platform loading; dedicated nodes at stack-node-year internal cost	[O07 §05, decision 4]

The review cadence in the sources differs. E10 says the transfer price "is reviewed once a year" and holds ₹0.40 "through Phase 2, reviewed at the start of Phase 3" [E10 §04 rule 3, decision 4]. O07 says it is "reset quarterly against measured cost" [O07 §05, decision 3]. The billing system can support either, because the transfer price is an effective-dated price-book row like any other. The difference matters for the related-party file, since a quarterly reset produces four arm's-length memos a year. It is recorded as a conflict.

For scale, a rung1 Growth tenant makes roughly 300–1,500 governed actions a month [modelled], which is ₹120–600 at ₹0.40, and the whole transfer at rung1's Phase 2 target of 25 tenants is about ₹9,000 a month [E10 §04]. The IM's model carries internal transfer revenue of ₹2.18 crore in FY2028-29, rising to ₹11.20 crore in FY2030-31 [IM Appendix A.5].

19.12.2 The mechanism

Nothing is built specially for rung1. The monthly transfer statement is the same `meter_period_agg` rows that an external invoice is rated from, filtered to `tenant.consumer = 'rung1'`, `outcome = 'committed'`, `key_class = 'production'` and `meter_class = 'governed_action'`, and priced at the transfer row of the price book. rung1 reads its own events and totals through `read.meter` and `GET /v1/meter` [E05 §04]. Both brands therefore read one count, and "two brands quoting different governed-action counts" has a tolerance of zero [O07 §09].

rung1 converts kernel meter events into credits for its own customers. It never recomputes the governed-action count, because "the meter event comes from the elan1 layer, not from rung1's arithmetic" [R13 §1]. The transfer is priced per committed action, and the credit that rung1 bills its customer is a different unit. A reconciliation between rung1's credit draws and the kernel meter therefore checks only that every credit draw attributed to a governed write references a committed meter event. It never compares money.

19.12.3 The related-party file

Whether the three brands are one legal entity or three decides whether the transfer is a cost allocation or "a related-party transaction needing arm's-length documentation" [O07 Q1]. O11 closes that question with four entities: a holding company, elan1, holding the kernel team and plane P3 and below, and operating companies for rung1 and stack1. On the

split, "O07's transfer prices stop being internal cost allocation and become related-party transactions between Indian entities", the prices unchanged and "the kernel meter is the only source for governed-action counts on any intercompany invoice" [O11 §02, §05]. The entity names are unverified and none is incorporated [O11 §02]. The IM describes the entity being raised as elan1 Private Limited, holding the kernel and the packs, with two related-party relationships disclosed, rung1 as first consumer and stack1 as runtime supplier [IM §2.1–§2.2]. On that basis the billing system should be built as though the transfer is arm's length from its first charge on 1 January 2027 (**build recommendation**):

1. A monthly tax invoice from elan1 to rung1 from the same pipeline as an external invoice, with the refusal line at ₹0 on its face, and dispute handling identical to an external tenant's, so that rung1 "will complain, which is the point" [O07 §05].
2. A quarterly transfer-pricing memo generated from the measured cost-per-action series in §19.13, comparing ₹0.40 with measured unit cost.
3. No netting. The elan1 → rung1 transfer and the stack1 → elan1 runtime charge are separate invoices on the same period boundaries, so that elan1's margin on the transfer (modelled at 50% [E10 §07]) is computed from two metered numbers.

19.13 Cost per governed action and its denominator

19.13.1 Why it is instrumented from the first write

O07 calls cost per governed action "the number that decides whether elan1 is a business". It is measured monthly, published internally, and set beside the refusal rate [O07 §06]. The IM commits the company to publishing the measured figure weekly from the first production write, "whether or not it flatters", and to repricing any tier that sits below 40% gross margin for two consecutive months [IM §17.5]. The meter is the spine of that measurement. "Every cost source is joined to it on `run_id` or, failing that, on tenant and period. No cost line enters the model without a join key; a cost we cannot join is an unallocated line by definition" [R13 §6].

19.13.2 Numerator: what a governed action costs elan1

E10 gives the modelled marginal cost at L1 [E10 §07]:

Line	₹ [modelled]	Assumption
Compute, rungs 0b–9	0.002	~91 ms p50, of which ~40 ms is rung 8, on Mumbai general-purpose compute at 15% utilisation
Shadow sandbox apply and re-run	0.15	"The dominant marginal cost." A snapshot restore and a second gate pass per write
Evidence storage, first year	0.01	~4 KB per record including gate-result rows, never compacted
Evidence storage, retained forever	0.03	Ten-year carry at Mumbai block-storage rates
Meter and reconciliation	0.01	Nightly two-way job amortised
Marginal total	**0.20**	

Fixed cost per residency domain is modelled at ₹1,88,000 a month at L1: ₹1,20,000 of residency footprint, ₹8,000 of registry storage and ₹60,000 of on-call and monitoring. It rises to ₹3,43,000 plus anchoring at L2 [E10 §07]. Two things about the marginal table matter for instrumentation. First, E10's compute assumption (~91 ms kernel p50) differs from E04's rung budget (≈ 64 ms kernel total) [E04 §02]; both are unmeasured and Gate 1 replaces them [E04 §02]. Second, the dominant line, the shadow-sandbox apply, is incurred in the consumer's target system (for rung1, a per-tenant ERPNext sandbox). E10 asks whether it is "10× that on a real ERPNext site with a real dataset" (kernel engineer, 31 March 2027) [E10 Q2].

O07 defines the group-level cost per governed action from five components: "Inference at rung 4 `evaluate()` — the only rung that calls a model"; gate execution and the rest of the path's compute; the proof write and its storage; the meter pipeline; and a share of the residency footprint [O07 §06]. **The first component conflicts with the kernel contract.** E02 says `assert` must never "call a model", that rung 4 is "versioned rules over a projection of the declared fields, no record read, no model", and conformance C1 fails the build if any module reachable from rungs 3–8 imports a model client [E02 §05, C1;

E01 §02 item 2]. At L1 the kernel's inference cost is therefore zero by construction. Model spend upstream of the kernel, which produces the write intent, belongs to the consumer: rung1's credits [R13 §1] and stack1's runtime meter [S02]. We treat O07's first component as the consumer's upstream inference, attributed through `run_id`, and never as a kernel cost line.

19.13.3 Denominator: which actions count

The sources give two definitions, and they produce different numbers:

Definition	Numerator includes refusals?	Denominator	Source
Group (O07)	Yes: refusals "consume the whole path except the act"	**Committed Governed Actions only**	[O07 §06]
rung1 (R13)	Yes	**All governed actions including refusals**; `PLATFORM_*` refusals excluded and booked to incidents	[R13 §6]

Both definitions push the same way. A high refusal rate raises cost per governed action, and O07 names that incentive as pointing the wrong way on purpose: "Nobody's target is a lower refusal rate. The refusal rate is published beside cost per governed action every month" [O07 §06, decision 10]. For elan1 we recommend computing and publishing both from the same meter rows, each labelled with its definition, and using O07's committed-only denominator as the headline because O07 governs the group and the external price is per committed action (**build recommendation**). A figure that does not name its denominator is not published.

19.13.4 Cost sources joined to the meter

Each cost bucket needs a join key onto `meter_event` (**build recommendation**). Kernel compute joins on tenant and period from stack1's cost-plus-15% statement, split by rung using OpenTelemetry spans. The shadow-sandbox apply joins on `run_id` to `write_id` and is reported separately, because it is incurred in the consumer's target. Proof and meter storage and the residency footprint are allocated per domain. At L2, anchoring joins per anchor batch. `PLATFORM.*` events carry incident cost on their own line and never enter a tenant's figure. The inputs come from [E10 §07; R13 §6; O07 §05–§06; IM §17.5]. Two targets bound the result. The first measured cost per governed action is a dated milestone, and L2 GA's gate criteria include "cost per governed action measured under production load — not modelled" [IM §16.5]. Ahead of that, O07 asks whether ₹0.40 survives measured cost against the modelled ₹0.20 (elan1 kernel lead, 31 March 2027) [O07 Q5].

19.14 The billing stack: build recommendation

The sources fix the meter's behaviour but name no billing product. The choices below are this memorandum's recommendation, tested against the group's licence register [O04] and the one-meter-per-number rule [O07 §03].

Layer	Recommendation	Alternatives considered	Why
Meter store	`meter_event` in the kernel's Postgres, same primary as the proof log	A separate metering product	The meter is kernel code in P3 [O01 §02; O03 §12]; a second meter would own a number the kernel owns
Transport to billing	Postgres outbox relay (`FOR UPDATE SKIP LOCKED`)	Kafka, Redpanda, NATS JetStream; Debezium CDC	A broker adds "a second place where ordering and exactly-once can fail" [ETS §05]; Redpanda is BSL [STS §05]
Aggregation and rating	Small in-house service (Python, with the kernel toolchain), effective-dated price-book tables	Lago (AGPL-3.0); OpenMeter (Apache-2.0); Kill Bill (Apache-2.0)	Rating is a few hundred lines of deterministic code; Lago is AGPL inside a paid service [O04 §02]; OpenMeter and Lago are meters in their own right and would contest the kernel's count; Kill Bill is licence-clean but heavy for ~40 tenants by March 2028 [IM §17.6]
Books, GST, e-invoice	ERPNext with India Compliance, run by elan1 for its own accounts (box 2, internal)	Tally; Zoho Books; Chargebee or Stripe Billing	GPL is clean in box 2 [O04 §02]; India GST and IRN are native; hosted SaaS billing places invoice data outside the residency domain and adds a second source

> **Rejected: any billing component that counts governed actions itself.** "There are three meters and each owns exactly one kind of number. Nobody owns two" [O07 §03]. A commercial usage-billing product that ingests raw events and computes its own totals becomes a fourth meter. Whatever sits downstream of the kernel meter may only price aggregates it did not compute.

19.15 Tests and acceptance criteria

The meter carries more tests than any other component relative to its size, because it is where a defect turns straight into a wrong number on a customer's invoice. The table maps each property to its test and its source.

#	Property	Test	Type	Source
MT-1	A refusal row with quantity > 0 cannot be inserted	Direct `INSERT` as `el_meter_writer` with `outcome='refused', quantity=1` must raise `23514` (check_violation); same for `escalated`, `sandbox`, `approval`	Unit (pgTAP)	[E10 §02; E05 §06]
MT-2	The caller cannot set quantity	Fuzz `assert` and `act.write` payloads with `quantity`, `basis`, `meter_*` fields; all rejected 4xx or ignored; emitted quantity always derived	Property (Hypothesis)	[O07 §03]
MT-3	Every refusal has a record and a zero meter event	C8 over the reference tenant's refusal fixtures	Conformance	[E02 C8]
MT-4	Two-way reconciliation is clean	C10 plus M1–M5 nightly; dead-man ping on success	Nightly	[E02 C10; EBB EB-11 §8]
MT-5	No `UPDATE`/`DELETE`/`TRUNCATE` grant exists on `meter_event`	Grant-checker against the live catalogue	CI + nightly	[ETS §09]
MT-6	Basis is resolved only from the declaration	Change a tenant's contract; basis on new events unchanged; a declaration with an unknown `blast_radius` value fails `define`	Unit	[E05 §06; E09 §08]
MT-7	An unreconciled period cannot be invoiced	Inject one orphan meter row; `rate()` raises `PeriodOpen`; tenant sees "period open"; page fires	Integration	[E09 decision 4]
MT-8	Floor, basis, refusal and evidence lines are undiscountable	Attempt each as `elan1_lead` and as `founder`; all refused	Unit	[E10 §05; O07 §08]
MT-9	Verification is never metered	Run the verifier and `verify` 10,000 times; zero meter rows	Integration	[E07 §04; EPA]
MT-10	Transfer statement equals kernel count	rung1 statement vs `meter_period_agg`, tolerance zero	Nightly	[O07 §09]
MT-11	Meter lag p99 ≤ 30 s	Synthetic writes timestamped at rung 9 vs arrival in billing store	SLO, **MEASURE THIS**	[O03 §08]

The mutation discipline in Chapter 21 applies here. Each of MT-1, MT-4, MT-7 and MT-10 must first be shown failing against a deliberately broken build before its green result counts as evidence.

19.16 What is in L1, what waits, and what is open

At L1 the build delivers the rung 9 emitter, the `meter_event` table with its CHECK and insert-only grants, nightly two-way reconciliation under C7, C8 and C10, rating of the internal transfer from 1 January 2027, the external invoice pipeline ready for L1 GA (Q2 CY2027), and cost-per-governed-action instrumentation from the first production write. Evidence metering, profile and attestation entitlements, and anchor-batch cost attribution wait for L2. Counterparty seats and metering inside a customer-operated sovereign domain wait for L3. The first external evidence line on an invoice is Gate 3's test: a signed contract with evidence on its own line by the end of Q4 2028 [E10 §06].

Open items that the billing build depends on:

#	Question	Owner	Due	Source
1	Does `assert` return through rung 9 synchronously? (Recommended: yes, with meter and proof in one transaction)	elan1 lead	31 Oct 2026	[E02 Q1]
2	Incorporation timing of the four entities in O11, and the CA's arm's-length basis for ₹0.40	Founder, CA	31 Dec 2026	[O07 Q1; O11 §02, §05]
3	Does an external L1 SKU exist in Phase 2 at all?	Founder	31 Jan 2027	[E01 Q5]
4	Is ₹0.15 for the shadow-sandbox apply right, or 10× on a real ERPNext site?	Kernel engineer	31 Mar 2027	[E10 Q2]
5	Does ₹0.40 survive measured cost?	elan1 kernel lead	31 Mar 2027	[O07 Q5]
6	External list price per governed action at L1 GA; which price book (E10 tiers or IM levels) is authoritative	Founder	Before L1 GA, Q2 2027	[O07 Q2; IM Appendix A.7]
7	What does RFC 3161 anchoring cost at 10^8 records a year?	elan1 lead	30 Jun 2027	[E10 Q1]

20. The technology stack — choices, alternatives and what was rejected

This chapter sets out the components elan1 is built from, layer by layer. For each layer it gives the component chosen, the alternatives considered, the reason, the licence, and the assurance level at which the component first matters. It also reconciles the earlier product-folder stack documents with the E and O series, records what was rejected and why, and states the build-versus-buy line and the supply-chain rules that bind every choice.

The sources say plainly how much of this is decided. The Build Book "deliberately contains no vendor choices except where load-bearing — an HSM for key custody and an RFC 3161 TSA for anchoring" [ETS §02]. The elan1 stack document is "a decision aid. Every OPEN item needs an ADR before its wave starts" [ETS cover]. Its epigraph is the rule this chapter follows: "Everything else is replaceable and naming it here would make it look decided" [ETS cover, quoting the Build Book]. What the sources *do* fix is standards: SHA-256, pure Ed25519, length-prefixed preimages, RFC 6962 Merkle trees, RFC 3161, JSON Schema 2020-12 and MCP [ETS §02; E12]. Separately, the stack1 series fixes the substrate that elan1 inherits as a consumer of the stack1 runtime [STS §02; S03 §02].

> **Designed is not shipped (DR-010).** None of the components below runs for a customer as at 23 September 2026. The elan1-platform repository exists with a toolchain (Python 3.12, FastAPI, uv, React with Vite and TypeScript, Postgres, Docker, GitHub Actions and a Claude Code harness), and its hosting decision, OD-28, is still open [ETS §01, §07, §14]. Where this chapter says "chosen", it means chosen in a source document or recommended here. It never means in production.

20.1 How the choices were made

20.1.1 The rule used throughout

Tools that change the answer to an invariant are architecture. Everything else is preference [ETS §00]. The elan1 stack document states eight selection tests [ETS §03], and stack1's states nine, adding licence resellability and attribution accuracy [STS §03]. Merged, and applied to the elan1 kernel, they read as follows:

#	Test	What it rules in or out for elan1	Source
1	Fails loudly	A missing grant, route or API beats application code that must remember to check; no "warn" modes, soft deletes or silent retries	[ETS §03 t1; STS §03 t8]
2	Enforceable by a check	Every property the tool guarantees is asserted in CI or nightly: a catalogue query, a lockfile read, a plan diff	[ETS §03 t2]
3	Deterministic where the kernel needs it	Anything used at rungs 3, 4, 6 or 8 gives the same output for the same inputs and `as_of`, forever	[ETS §03 t3; E02 C4]
4	Independent where the spec needs it	The two seal implementations and the verifier share no code and no library that could make them agree for the wrong reason	[ETS §03 t4; E12 §09]
5	Resident	Runs in, and keeps data in, the tenant's residency domain; India regions for Indian tenants	[ETS §03 t5; O05]
6	Resellable in box 3	Permissive licence for anything inside the paid service; copyleft only on a customer's site, internally, or as a separate network service	[O04 §02; STS §03 t1]
7	Replaceable behind a port or standard	SQL, OpenTelemetry, OCI, RFC 3161, MCP; the kernel and seal are the only shut seams	[ETS §03 t6; EBB EB-10 §9]
8	Cheap to review	"Small, boring and well-documented beats powerful"; review, not generation, is the binding constraint	[ETS §03 t7; EBB EB-09 §8]
9	Already in the repo	When two options tie, the one the repository, its CI and its Claude Code skills already use wins	[ETS §03 t8]

20.1.2 The two stacks elan1 sits between

elan1 is not a standalone stack. S03 fixes the boundary: "The elan1 layer is not part of the stack1 runtime. stack1 owns the model gateway, observability, durable execution, the sandbox pool, the deploy pipeline, rollback, secrets and metering. The elan1 layer … runs on this runtime as a consumer of it. It is never imported into it, never linked into it, and never deployed as part of it" [S03 §02]. The dependency is one-way: "elan1 depends on stack1, never the reverse", enforced by an import-graph check (B1) and a manifest scan (B2) [S03 §03, decision 2]. Three stack1 capabilities, "secrets, metering and the deploy pipeline", are stack1's own code and not substitutable [S03 decision 5].

Three consequences follow for this chapter:

1. **Runtime substrate is inherited, not chosen.** Compute orchestration, secrets, the durable-execution engine and the deployment pipeline come from stack1 under `fusion.*/v1` contracts [O03 §09; S03 §02].
2. **The kernel's own components are chosen by elan1.** These are the governed-write service, the chain writer, the policy evaluator, the meter, the declaration registry, the approval console, the SDK and MCP surfaces and, at L2, the seal, the key register, the anchor client and the verifier [O03 §12; E02; E05].
3. **Where the earlier elan1 stack document chose a runtime component, the stack1 choice now governs**, and the difference is recorded as a conflict (§20.4).

Figure 29 shows the resulting map: the layers elan1 owns, the layers it consumes from stack1, the chosen component in each, and its principal alternative.

Figure 29 — Technology stack by layer

MOSTLY RECOMMENDATIONS · ADR BEFORE EACH WAVE

Chosen component, credible alternatives and what is rejected; where E/O documents override the product folder, they win.

LAYER	CHOSEN	ALTERNATIVES	REJECTED / AVOID	STATUS
Kernel language	Python 3.12 · FastAPI · uv (in repo); write API with generated OpenAPI 3.1	Go (static binary) Kotlin / Java 21 · TypeScript	—	RECOMMENDED revisit if p50 misses
Seal A and B	A: Python + pyca/cryptography B: TypeScript + node:crypto, a separate team	PyNaCl · Rust ed25519-dalek (verify_strict) · Java 21 EdDSA	one shared library for A and B	RULE FIXED E12; langs REC
Offline verifier	Go, standard library only (SHA-256, Ed25519, ASN.1/DER, X.509); empty lockfile	TypeScript on Node with no npm dependencies	Python, Rust, C/C++ (need dependencies)	ZERO-DEP FIXED E12, E15
Database	PostgreSQL 16+: role grants, FORCE RLS, CHECK; INSERT-only evidence tables	Cloud SQL / AlloyDB · Azure Postgres · self-managed Patroni	QLDB (retired) · any blockchain · Supabase prod	FIXED O05, E02
Outbox and queue	Postgres outbox, FOR UPDATE SKIP LOCKED, per-tenant commit order	Debezium CDC · LISTEN/NOTIFY as a wake-up hint	a broker: Kafka, Redpanda, NATS	LOAD-BEARING STS; ETS REC
Durable workflow	Temporal behind fusion.run/v1, a namespace per tenant; approvals arrive as durable signals	Prefect · Dagster (substitutes behind the same contract)	Windmill · n8n · Inngest (licences)	FIXED S04, DR-002
Policy engine	Own rules-as-data evaluator at rung 4: reads the projection and as_of only; no clock, no model	Cedar (fallback) · CEL · OPA/ Rego with time and http off	any model in the allow decision	RECOMMENDED ADR at W3
Key custody (L2)	AWS KMS, ECC_NIST_EDWARDS25519, pure Ed25519, Origin KMS, one key per residency domain	Google Cloud KMS (HSM level) · CloudHSM · Thales · Entrust (L3)	OpenBao / Vault transit (software keys)	NAMED E16 §03
Timestamp (L2)	eMudhra (India, CCA-licensed) plus one global TSA: DigiCert, Sectigo, GlobalSign or Entrust	the global partner is still to be chosen	FreeTSA and free TSAs (no SLA)	OPEN to 31 May 2027
MCP surface	Official MCP SDK in the API language; stateless 2026-07-28 revision; descriptions pinned by hash	hand-rolled JSON-RPC	—	RECOMMENDED E05, ETS
Console front end	React 18 · Vite · TypeScript · Tailwind; screens compiled from declarations, no screen files	Svelte / SvelteKit · Vue 3 · Radix primitives	hand-written screen files under app-*	IN REPO ETS §04
Secrets, identity	OpenBao path per tenant; Keycloak realm per tenant; mTLS workload identity (stack1)	AWS Secrets Manager · Doppler (ETS recommendation)	HashiCorp Vault (BUSL, blocked)	VIA STACK1 S03 overrides ETS
Observability	OpenTelemetry spans per rung → fusion.trace/v1; Grafana stack · Sentry · healthchecks.io	Datadog · Honeycomb · Opik / MLflow behind trace/v1	vendor agents (lock-in)	REC · PAGING vendor OPEN
CI/CD, supply chain	GitHub Actions + merge queue; Claude Code harness; cosign + SLSA; import-linter / depguard; CodeQL	GitLab CI · Buildkite · Snyk · Renovate or Dependabot	pinning by tag; ee/ and enterprise/ in builds	IN REPO O04, ETS §09
Hosting and IaC	stack1 managed Kubernetes in ap-south-1 only; Terraform or OpenTofu; Argo CD delivery	ECS Fargate (ETS) · Pulumi · GCP asia-south1	multi-cloud · a 2nd region before counsel	OPEN OD-28 vendor
Evidence store	Object Lock (WORM) in India: S3 Object Lock (ETS) or SeaweedFS / Ceph / Garage (STS)	GCS Bucket Lock · Azure immutable blob storage	MinIO (AGPL under the evidence plane)	OPEN ETS v STS

elan1 choice · inherited via stack1 contract · external vendor · rejected or avoided

Source: ETS §01–§13, STS §01–§02, S03 §1, S04 §01, O04 §03, O05 §1–§3, E05, E12, E15, E16 §03, E17, IM §6.10 · elan1 Technology Memorandum · Sept 2026

Figure 29 — The elan1 technology stack by layer: kernel-owned components (governed-write service, policy evaluator, chain writer, meter, registry, surfaces, and at L2 the seal, key register, anchor client and offline verifier) above the stack1 runtime substrate consumed through fusion.*/v1 contracts, with the chosen component, its principal alternative and the assurance level at which each first matters.

20.2 The stack, layer by layer

The master table below is the reference. It carries a status mark: **FIXED** where a source names the component or a standard that forces it; **INHERITED** where stack1's substrate governs; **REC** where the component is recommended in ETS or here and awaits an ADR; **OPEN** where a named decision is outstanding. The level column gives the assurance level at which the component is first on the production path.

Layer	Chosen	Principal alternatives	Why	Licence	Level	Status
Kernel language (write API, rungs, workers)	Python 3.12, FastAPI, uv	Go; Kotlin/Java 21; TypeScript	Budget dominated by I/O, not interpreter; already in repo	PSF; MIT	L1	REC [ETS §04]
Declaration schema	JSON Schema 2020-12, `additionalProperties: false`	CUE; Protobuf	Named by source; runnable in CI without a platform	Spec (open)	L1	FIXED [ETS §02]
Schema validator	Python `jsonschema`; Ajv in TypeScript	—	Two ecosystems, same schema	MIT	L1	REC [ETS §08]
Policy engine (rung 4)	Own rules-as-data evaluator (`@elan1/policy`)	Cedar; CEL; OPA/Rego with time and HTTP disabled	Must be deterministic, clock-free, network-free, versioned	Own	L1	REC, ADR by W3 [ETS §13 #8]
Record, registry, proof, meter store	PostgreSQL 16+	Cloud SQL/AlloyDB; Azure; Patroni self-managed	Grants, FORCE RLS, CHECK, partitioning, queryable catalogue	PostgreSQL (permissive)	L1	FIXED in substance [ETS §05; STS §02]
Append-only enforcement	`GRANT INSERT` only; `REVOKE UPDATE, DELETE, TRUNCATE`	BEFORE triggers as belt-and-braces	"Grants are the control"	—	L1	FIXED [ETS §05]
Outbox relay / queue	Postgres outbox, `FOR UPDATE SKIP LOCKED`	Debezium CDC; LISTEN/NOTIFY wake-up	No broker: "a second place where ordering and exactly-once can fail"	—	L1	REC [ETS §05]; LOAD-BEARING in stack1 [STS §02]
Durable execution (consumer runs, approval waits, anchor job)	Temporal via `fusion.run/v1`	Prefect (substitute A); Dagster (B)	Replays the decision, never the effect; approval as a durable signal	MIT	L1	FIXED [O01 DR-002; S04]
Migrations	Alembic or Atlas, forward-only	Sqitch; Flyway; golang-migrate	Structure only; declarations and records never migrated	MIT; Apache-2.0	L1	OPEN [ETS §05]
Secrets broker (target credentials)	stack1 secrets broker on OpenBao, per-tenant paths	Cloud secret manager in stack1 Cloud only; Infisical	Credential minted per write, ten-minute life	MPL-2.0	L1	INHERITED [STS §06; E05 §02]
Identity (humans)	Keycloak, realm per tenant	Zitadel; Authentik	Projected identity; hostable air-gapped	Apache-2.0	L1	INHERITED [STS §06]
Kernel token issuer	Own issuer: tenant, control and ops tokens	Keycloak token exchange	Tenancy from the token, never an argument	Own	L1	FIXED in behaviour [E05 §05; E02 §04]
MCP server	Official MCP SDK, stateless 2026-07-28 revision	Hand-rolled JSON-RPC	34 tools; descriptions pinned by hash	MIT (SDK)	L1	REC [ETS §08; E05 §04]

Layer	Chosen	Principal alternatives	Why	Licence	Level	Status
SDK (Mode A, in-process)	Python first (rung1 on Frappe), TypeScript second; generated from OpenAPI 3.1	Hand-written clients	One source of truth for call shapes	Own	L1	REC (build recommendation)
Webhooks	Own delivery from the outbox, Standard Webhooks signatures	Svix	Payloads carry `seq` and `digest` for gap detection	Own	L1	REC [ETS §08; E05 §09]
Console and approval UI	React 18, Vite, TypeScript, Tailwind, design tokens; compiled surfaces	SvelteKit; Vue 3	Already in repo; no screen files	MIT	L1	REC [ETS §04]
Compute orchestration	stack1 managed Kubernetes, `ap-south-1`	ECS Fargate (ETS); Cloud Run	Same artefact across hosted, OEM and air-gapped modes	Apache-2.0	L1	INHERITED [STS §07]
Network policy between zones	Namespaces per zone, default-deny NetworkPolicy; Linkerd with SPIFFE	Cilium; Istio ambient	Absent routes asserted from reviewed IaC	Apache-2.0	L1	INHERITED [STS §07]
Egress control	Deny-all-except, allow-list compiled from the declaration registry	Cilium FQDN; forward proxy	Z3 to declared connectors only; Z4 to the TSAs only	—	L1 (Z3), L2 (Z4)	FIXED in behaviour [ETS §07; STS §07]
IaC and delivery	OpenTofu (Terraform-compatible); Helm via Argo CD	Terraform (BUSL, internal use only); Pulumi; Flux	Plans reviewed as code; infra-diff check	MPL-2.0; Apache-2.0	L1	REC [ETS §07; STS §07]; OpenTofu preference is a build rec.
Regions	Mumbai `ap-south-1` primary; a second Indian region for DR	GCP `asia-south1/2`; Azure Central/South India	India residency; no cross-region failover in Phase 1	—	L1	REC [ETS §07]; [S03 decision 7]
Observability	OpenTelemetry, a span per rung; Prometheus/Mimir, Loki, Tempo, Grafana	Datadog; Honeycomb	Measures the rung budgets; in-region for tenant-adjacent data	Apache-2.0; AGPL (Grafana, box 2 or separate service)	L1	REC [ETS §10]; runtime INHERITED [S03 §02]
Error tracking	Sentry, with payload scrubbing	Rollbar	Already connected; no tenant payloads leave India	Sentry licence (FSL)	L1	REC [ETS §10]
Dead-man switch	healthchecks.io on the nightly integrity and reconciliation joins	Cronitor	Silence pages	SaaS	L1	REC [ETS §10]
Paging	incident.io or PagerDuty	Better Stack; GitHub Mobile alerts	Two-person rotation	SaaS	L1	OPEN [ETS §13 #10]
Source and CI	GitHub, Actions, merge queue	GitLab CI; Buildkite	Already in repo	SaaS	L1	REC [ETS §09]
Agentic engineering harness	Claude Code: CLAUDE.md files, subagents, commands, hooks, skills	—	The Build Book specifies the harness shape	Commercial	L1	FIXED [EBB EB-09; ETS §09]
Import-graph checks	import-linter (Python), dependency-cruiser (TS), depguard (Go)	Custom AST walker	C1 and the P3 boundary as a failing build	BSD; MIT	L1	REC [ETS §09]
Tests	pytest with Hypothesis; go test fuzz; Vitest; Playwright; pgTAP; Testcontainers	—	Property, fuzz and catalogue tests	MIT; BSD; Apache	L1	REC [ETS §09]

Layer	Chosen	Principal alternatives	Why	Licence	Level	Status
Supply chain	cosign and SLSA provenance; Syft SBOM; Grype, Trivy or osv-scanner; Renovate	Dependabot; Snyk	Signed, enumerable artefacts	Apache-2.0	L1	REC [ETS §09]
Security scanning	CodeQL, Semgrep, gitleaks	Snyk	Secrets never in the repo	Mixed	L1	REC [ETS §09]
Chain digest (L1)	SHA-256 over the LP preimage	—	Frozen at L1 so L2 is a retrofit	Standard	L1	FIXED [E02 §03; E12]
Seal implementation A	Python with pyca/cryptography	PyNaCl (libsodium)	May have dependencies; only the verifier may not	Apache-2.0/BSD	L2	REC [ETS §04]
Seal implementation B	TypeScript with `node:crypto`	Rust with ed25519-dalek; Java 21 EdDSA	Different language forces different parsing choices	MIT (Node)	L2	REC [ETS §04, §13 #2]
Offline verifier	Go, standard library only; empty lockfile	TypeScript on Node without npm deps	Only mainstream stdlib with SHA-256, Ed25519, ASN.1/DER and X.509	BSD-3 (Go)	L2	REC [ETS §04; E15 §07]
Key custody	AWS KMS `ECC_NIST_EDWARDS25519`, pure mode, per residency domain	Google Cloud KMS `EC_SIGN_ED25519` at HSM level	Managed KMS at Phase 3; HSM and customer-held at L3	Service	L2	FIXED [E16 §03, decision 3]
Timestamp authorities	eMudhra (India) plus one global commercial TSA	DigiCert; Sectigo; GlobalSign; Entrust	Two operators, two trust roots; interval SLO ≤ 15 min	Service	L2	FIXED as recommendation [E17]
Evidence object store	S3 Object Lock (compliance mode) in stack1 Cloud; Garage, Ceph or SeaweedFS in sovereign and air-gapped modes	GCS Bucket Lock; Azure immutable blob	WORM is a second belt; the chain is the signed manifest	Service; AGPL/LGPL/Apache (run as separate service)	L2	REC; reconciled (§20.4)
Redaction	RFC 9901 SD-JWT library, pinned	—	Selective disclosure profiles	Varies	L2	REC [ETS §06]
Transparency publish (optional)	SCITT per RFC 9943/9942	Sigstore Rekor	Optional; offline verification never depends on it	Varies	L2+	REC [ETS §06; EBB EB-17 §8]
Customer-held keys, HSM custody	CloudHSM; Thales Luna; Entrust nShield	YubiHSM 2 for dev ceremonies	Stronger custody story, more operational work	Commercial	L3	FIXED as direction [E16 §03]

The rest of this section explains the rows where the reasoning is not obvious from the table.

20.2.1 Languages: three, assigned by failure mode

The seal release protocol requires two independent implementations and a verifier written from the same text. "Choosing three different languages makes independence structural rather than a promise" [ETS §04]. The assignment is:

- **Kernel and seal A in Python.** The kernel's latency budget is dominated by the proof write and I/O, not by interpreter speed. The ETS note says to revisit the language only if Gate 1 measurements miss the ≤ 120 ms p50 SLO [ETS §04]. E04's rung budgets add up to about 64 ms kernel p50, all unmeasured [E04 §02]. The repository, its CI and its skills are Python-first [ETS §04].
- **Seal B in TypeScript.** It must be "built from the text by people who did not talk to team A", and a different language "forces different parsing choices" [ETS §04]. This is E12's two-implementation rule made concrete: no seal ships until two independently written implementations produce identical bytes on every published vector [E12; IM §7.3].
- **The verifier in Go, standard library only.** The verifier must recompute SHA-256, verify pure Ed25519, walk RFC 6962 paths and parse RFC 3161 tokens, which are DER-encoded CMS signed by an X.509 certificate. Go is the only mainstream language whose standard library covers all four. Python lacks stdlib Ed25519 and DER, and Rust's standard library has no cryptography at all [ETS §04 feasibility table; ETR §12]. E15 makes the zero-dependency property a release gate: "standard library only, empty lockfile, CI fails if the lockfile is not empty", with two reproducible builds on two machines [E15 §07].

stack1 reaches the same three languages by the same reasoning: "Go where a failure is a latency budget or an auditor's machine, Python where a failure is a bad plan a human catches, TypeScript where a failure is a screen" [STS §04]. The groups converge, and hiring and tooling can be shared.

> **No shared library between the chain writer and the verifier.** "Sharing code makes the verifier a second copy of our assumptions rather than an independent check of the text, and the whole value of recomputing the digest evaporates if the recomputation uses the function that computed it" [E12 §09; E15 §08]. The CI rule that enforces this is in Chapter 21.

At L1 there is no verifier to ship, and E01 forbids one as a product: "No verifier, no bundle, no redaction profile, no erasure salt" [E01 §07]. L1 does have "a nightly integrity verifier" over the hash chain [E01 §02 item 7]. We recommend writing that nightly checker in Go, from the E12 text, by someone other than the chain writer's author (**build recommendation**). It then becomes the kernel of the L2 offline verifier, and L1 exercises the independence discipline a year before a signature exists.

20.2.2 The database and why it is Postgres

The data layer is where three of the eight invariants live: one write path, seal-or-nothing and append-only. "The database is chosen for the strength of its grants, constraints and catalogue — not for scale features" [ETS §05]. PostgreSQL 16+ gives role grants that make append-only an absence of permission, `FORCE ROW LEVEL SECURITY` for tenancy, CHECK constraints such as the quantity-zero rule in Chapter 19, declarative partitioning by `tenant_id`, and `information_schema` for the grant-checker [ETS §05]. stack1 reaches the same conclusion and fixes PostgreSQL 16+ with a synchronous standby in region [STS §02].

Two alternatives were rejected explicitly. **Distributed SQL** (CockroachDB, YugabyteDB) is "not needed at Gate 1": residency is already solved by one deployment per domain, and both add operational cost without adding a governance property [ETS §05; STS §05]. **Ledger databases and blockchains** are avoided: "The chain is the seal itself; the Blueprint refuses a blockchain — the anchor is RFC 3161 plus an optional transparency log", and Amazon QLDB reached end of support on 31 July 2025 [ETS §05, §14].

Supabase Postgres is in the repository for development and the reference tenant, but "Supabase is a public endpoint, so it cannot satisfy C05 in production" [ETS §05]. The same applies to the repository's current application host: "reached over public networks; C05 and C06 require private networks. Keep them for dev and ref only" [ETS §13].

20.2.3 No broker

The outbox relay is a Postgres table polled with `FOR UPDATE SKIP LOCKED`, keeping per-tenant commit order [ETS §05]. Kafka, Redpanda and NATS JetStream are listed as alternatives and not chosen, because "a broker adds a second place where ordering and exactly-once can fail" [ETS §05]. stack1 makes the same choice load-bearing for its own meter: "The meter row and the contract call must commit in one transaction. No broker gives that" [STS §01]. It also marks Redpanda

BLOCKED under BSL 1.1 [STS §05]. The Build Book's rule for when to buy one is equally plain: "Do not stand up production infrastructure, a message broker or a paging vendor before W0's two documents and verifier are done" [ETS §12, citing EBB EB-10 §10]. The ADR that settles the relay is measured, not argued: "Outbox-to-seal p99 ≤ 2 s under load" [ETS §13 #7].

20.2.4 Temporal, and where it sits relative to the kernel

DR-002 is closed: "Temporal owns anything touching a customer's books. Every governed write runs inside a Temporal workflow" [O01 DR-002]. S04 makes Temporal the bound implementation of `fusion.run/v1`, with Prefect as substitute A and Dagster as substitute B, chosen "on licence distance" [S04 §04, decisions 1–2]. The reason is the one failure a governed write path exists to prevent. A replay-style runner that re-runs a step after a lost response posts the same ₹4,80,000 purchase invoice twice, whereas "Temporal re-runs your workflow function but does not re-execute the effects" [S04 §02].

The earlier elan1 stack document never mentions Temporal. Its write path is a synchronous kernel transaction feeding an outbox relay [ETS §01, §05, §12]. The two fit together as follows (**build recommendation**, consistent with [O01 DR-002; S03 decision 6; S04 §02; E02 §05]):

Concern	Runs on	Why
The consumer's run (rung1's plan, sandbox lease, changeset), up to `assert`	Temporal workflow in the consumer's namespace	Durable; recovery replays the decision
The `assert` call and rungs 0b–9	Synchronous kernel code, invoked from a Temporal activity with the `intent_id` as idempotency key	Rungs must not be split across workflow steps; the kernel transaction is the unit
Waiting for a human approval (rung 6 escalation)	Temporal signal to the waiting workflow; the kernel records the obligation row	"Human approval reaches a run as a durable workflow signal, not a callback" [S03 decision 6]
Chain writer consuming the outbox (rung 8)	Kernel worker on the Postgres relay	Commit order by construction; no engine in the ordering path
L2 batched Merkle anchor job	Temporal on a blocking-green `fusion.run/v1`	"Replay must not re-run the side effect: duplicate anchors on replay" [O09 item 4; IM §16.6]

The kernel therefore depends on Temporal only indirectly, through its consumers and the anchor job. That keeps the kernel testable without a Temporal cluster and keeps S03's direction rule intact: the kernel is never deployed inside the runtime.

20.2.5 The policy engine

Rung 4 `evaluate()` is "versioned rules over a projection of the declared fields, no record read, no model", and it must never "read a clock in any gate" [E02 §05; E01 §02 item 2]. `policy_version` is sealed field F08 [E02 §03]. The recommended engine is an in-house rules-as-data evaluator, with Cedar as the fallback [ETS §01, §08]. OPA/Rego is an alternative only with `time.now_ns` and `http.send` disabled, and stack1 marks OPA with those built-ins enabled as AVOID for its own guard evaluator [ETS §08; STS §04]. The ADR is settled by "EV3 replay at `as_of`; no clock, no network" [ETS §13 #8].

We endorse the in-house choice for two reasons that go beyond the sources (**build recommendation**). First, the evaluator's input language is the Declaration's own rule syntax [E03; E21], so an external policy language would add a translation layer whose correctness would itself need a conformance suite. Second, C4 requires every reference-tenant refusal to replay to the identical decision [E02 C4]. That is simplest to guarantee in a small evaluator whose every built-in is known. If a customer or regulator later asks for an analysable standard language, Cedar is the substitute, because its analysis tooling lets a verifier profile make claims about the rule set.

20.2.6 Surfaces: MCP, SDK and console

The MCP tool surface is the 34 tools of E05 (18 `ro`, 10 `stage`, 6 `act`), with tool descriptions pinned by hash in the declaration [E05 §04]. ETS recommends the official MCP SDK in the API language and confirming that it supports the stateless 2026-07-28 revision [ETS §02, §08]. stack1 fixes the official SDK for its own server too, and avoids depending on

the official MCP Registry, which is in preview with its API frozen at v0.1 [STS §08]. elan1 follows the same rule: its tool catalogue is served from its own declaration registry and never from a public registry.

Mode A, the in-process SDK, has "six calls, identical signatures in every product, and one CI rule: nothing above the kernel boundary imports a kernel type" [E05 §03]. The sources do not fix SDK languages. rung1 Enterprise runs on Frappe and ERPNext, which are Python [R05]. We recommend a Python SDK first and a TypeScript SDK second, both generated from the OpenAPI 3.1 description of `/v1`, so that "a hand-written client is a second source of truth" cannot arise [STS §04] (**build recommendation**).

The console and approval screens (E06) use React 18, Vite, TypeScript and Tailwind with design tokens, already in the repository [ETS §04, §08]. The Build Book's structural rule is that screens are compiled from declarations and "a CI glob fails on any component, template or style file under `app-*`" [ETS §08; EBB EB-09 §1]. Under the E/O perimeter, elan1 owns the approval console and the shared renderers it needs. Product screens belong to rung1 [O03 §10, §12].

20.2.7 Cryptography, keys and time (L2 and L3)

No key material exists at L1: "Not in a config file, not in a vault, not 'just for testing'" [E01 §07]. The components in this group are therefore specified now and built for L2.

- **Key custody.** E16 fixes the Phase 3 answer: "managed KMS. AWS KMS with the `ECC_NIST_EDWARDS25519` key spec, in the residency domain", created with `Origin: KMS` and no import, and Google Cloud KMS `EC_SIGN_ED25519` at HSM protection level as the named alternative. HSM and customer-held keys arrive at L3 [E16 §03, decision 3]. ETS adds the test that settles it: prove the pure variant is used with the published pre-hash negative vector [ETS §06]. E12's NV-06 and NV-14 exist because "strictness varies by library — cofactored versus cofactorless, small-order point handling, non-canonical scalars" [E12 §04, §07].
- **Timestamp authorities.** "eMudhra is the recommended India-based authority; one global commercial authority — DigiCert, Sectigo, GlobalSign or Entrust — is the pair." Free public TSAs are refused for production [E17 §03; ETS §06]. The pair is a contract, not a library, and the contract is on the critical path to Gate 1 [O09 item 3].
- **Software keys are rejected for production.** OpenBao or Vault transit keys are "not HSM custody; acceptable only for dev with the visibly non-production `dev-DO-NOT-TRUST` key" [ETS §06; E16 §03]. HashiCorp Vault is also BLOCKED on licence grounds under BUSL [STS §06].

20.2.8 Hosting, and the OD-28 question

OD-28, the hosting vendor, is open in the repository [ETS §07, §13 #4]. ETS recommends AWS in Mumbai and Hyderabad, because that combines KMS Ed25519, S3 Object Lock and domain-list egress firewalls. It recommends ECS Fargate to "keep ops small for a two-person rotation" [ETS §07]. stack1 fixes managed Kubernetes per region, Mumbai first, because "the operator OEM mode is somebody else's Kubernetes, so the artefact has to be Kubernetes-shaped anyway" [STS §07]. Its phase B also uses Hetzner for bulk CI [STS §01].

Because elan1 runs on stack1 [S03 §02] and O09 makes key custody per residency domain depend on "stack1 residency-domain deploy topology and secrets handling at GA quality" [O09 item 2], elan1 production runs on stack1's Kubernetes footprint in `ap-south-1` and inherits its cloud choice. The ETS recommendation of AWS survives, because stack1's recommended in-region footprint is also AWS `ap-south-1`, itself marked DECISION OPEN [STS §07]. The recommendation of ECS Fargate does not. The repository's current host and Supabase remain development and reference environments only [ETS §13]. The Railway deployment path named in the repository's tooling is conditional on OD-28 closing, and it cannot satisfy the absent-route controls C05 and C06 in production for the same reason Supabase cannot [ETS §07, §13].

A second Indian region for disaster recovery is recommended [ETS §07]. S03 accepts "no cross-region failover in Phase 1", stated as a risk [S03 decision 7]. Both are true at different times: DR in a second Indian region is a Phase 2 build item and is not a Phase 1 promise (**build recommendation**; see Chapter 22).

20.3 Licences

Every component in §20.2 passes the group's three-box test [O04 §02]. The standing rule is "permissive licences only — MIT, BSD, Apache-2.0 — for anything embedded in our paid service. Copyleft is acceptable for anything installed on a customer's own site, and for anything only we log into" [O04 §02]. The escape hatch is structural: a copyleft component that is the best tool runs "as a separate network service behind a contract, with no shared process and no linked code", which is exactly what `fusion.*/v1` is for [O04 §02, decision 2].

Component class in elan1's path	Licence	Box	Verdict and control	Source
Temporal	MIT	3	CLEAN; bound to `fusion.run/v1`	[O04 §03; S04 §03]
PostgreSQL	PostgreSQL licence	3	Permissive; counsel row not listed in O04 (add it)	build rec.
Keycloak	Apache-2.0	3	CLEAN	[O04 §03]
OpenBao	MPL-2.0	3	File-level copyleft; publish changes to MPL files	[STS §06]
Grafana, Loki, Tempo, Mimir	AGPL-3.0	2, or 3 as separate service	Internal dashboards only; never embedded in a tenant-facing screen	[O04 §02] (build rec. for the row)
Superset	Apache-2.0	3	Preferred BI for anything multi-tenant hosted	[O04 §03]
Metabase	AGPL-3.0 OSS	2	CONDITIONAL; internal only	[O04 §03]
Garage / Ceph / SeaweedFS	AGPL-3.0 / LGPL / Apache-2.0	3 as separate service	Behind the object-store contract; never linked	[STS §05] (build rec. for elan1 row)
LiteLLM, Langfuse (consumer side)	MIT core + `ee/`	3	CONDITIONAL; CI fails if `ee/` enters the artefact	[O04 §03, decision 4]
ERPNext (rung1's system of record)	GPL-3.0	1	CLEAN on the tenant's own site; nothing of ours runs inside a GPL process	[O04 §03]
Go standard library (verifier)	BSD-3	Shipped to anyone	Source release is the product	[E15 §07]
Sentry	Functional Source Licence	2 (hosted SaaS)	Tooling only; tenant payloads scrubbed	build rec.

Two register rules bind the build. "Pin every component to an exact commit SHA — never a tag, never `latest`. Re-read the primary LICENSE file at that SHA when the decision is written down" [O04 §01, decision 5]. And "no CONDITIONAL row reaches a paying customer without a counsel answer" [O04 decision 6]. The register "lives in the repository and is generated into the knowledge vault; it is never hand-edited in a wiki" [O04 §05]. Chapter 21 makes the licence check a CI stage.

> **The register does not yet list most of elan1's own stack.** O04's 25 rows cover the group's catalogue and runtime choices: ERPNext, Frappe, Temporal, Keycloak, LiteLLM and the voice stack. They do not cover PostgreSQL, the Go toolchain, pyca/cryptography, the MCP SDK, React, the Grafana stack or the SD-JWT library. Every component in §20.2 needs a row, read at its pinned SHA, before it enters a build that reaches a paying customer [O04 §05 "on adoption"] (**build recommendation**).

20.4 Reconciling the product-folder stack with the E and O series

The product-folder documents, ETS, EPA, ETA, EBB, ERD and EAP, were written between 16 and 19 September 2026 against the "Governed Intelligence Platform" design. That design was a platform with twenty apps and ten packs, signing at rung 8 from the first write, and a 48-person build [EBB EB-10, EB-14; ETA]. The E and O series of 21 September narrow it: L1 first with no signing, a kernel-plus-packs perimeter, and stack1 as the runtime [E01; O01 §04; O03]. Where the two disagree, the E/O series prevail. The table records every difference that changes what is built.

Topic	Product folder says	E/O series says	What is built
Signing at L1	Seal synchronous at rung 8, Ed25519 on every write; "no seal, no commit (C15)" [ETA; ERD §7.4]	L1 is hash-chained and unsigned; "Signing at L1" must never happen [E01 §03; E12 §09]	L1 chain with SHA-256 over the frozen LP preimage; `seal` returns a hash-chain receipt [O01 DR-013]
Key material and HSM "before first seal"	Production keys in HSM custody before first seal [ERD §7.1]	No key material at L1; managed KMS at L2; HSM at L3 [E01 §07; E16]	No KMS on the L1 path
Verifier and bundle at W0	Verifier and 64 vectors are the first epics [EBB EB-10]	"No verifier, no bundle" at L1 [E01 §07]; E12 specified but "not implemented" in November 2026 [IM §16.2.2]	Specification and vectors drafted; nightly integrity checker at L1; offline verifier at L2
Hosting and compute	AWS, ECS Fargate, Secrets Manager [ETS §07; ETA]	elan1 runs on stack1; stack1 fixes managed Kubernetes and OpenBao [S03 §02; STS §02]	stack1 Kubernetes in `ap-south-1`; stack1 secrets broker
Object store for evidence	S3 Object Lock [ETS §05]	stack1: SeaweedFS, Ceph or Garage, never MinIO; S3 only in stack1 Cloud [STS §05]	S3 Object Lock in hosted mode; Garage/Ceph/SeaweedFS in sovereign and air-gapped
Workflow engine	Not mentioned [ETS]	Temporal for anything touching the books [DR-002; S04]	Temporal for consumer runs, approval waits and the anchor job (§20.2.4)
Autonomy scale	`L0–L5` [ETA]	A0–A5 [E01 §09; O01 §03]	A0–A5
Declaration block 7	`doors` [ETA]	`channels` [O01 DR-012]	`channels`
F16 field name	`sealed_at` [ETA]	`recorded_at` [E02 §03; E12 decision 1]	`recorded_at`
Voice	verna1 via CarrierPort [ETS §08; EBB EB-10 E13]	verna1 parked [DR-007]; inbound voice is rung1 Voice, a channel on P6 needing a `voice` kind in block 7 (`channels`) [O09 items 9–10]	Kernel supplies the channel kind; rung1 Voice builds the channel
Apps and packs in the monorepo	`/apps/app-*` (20) and `/packs/pack-*` (10) in one repo [EBB EB-09 §2]	Kernel repo owns P3; apps belong to consumers; packs are elan1 configuration [O03 §12; E18]	`elan1-core`, `elan1-declarations`, packs as configuration repositories; no product apps in elan1's repo
TV-001 as the first target	"TV-001 at 311 octets with the known digest" [EAP §05; ERD §4.7]	311 octets and its digest are retired; TV-001 is 329 octets under the v3.0 table and its digest is not written down until two implementations agree [E12 §07, decisions 3–4]	Any artefact built against 311 octets is discarded
Headcount shape	3–4 → 48 people across four waves [EBB EB-14]	11 in October 2026 → 27 at March 2027 → 305 at March 2031, ten functions [IM §15.4]	IM plan (Chapter 21)

20.5 What was rejected, and why

Rejections carry as much of the design as choices do. The list below collects every component or approach refused by name in the sources, with the reason and the source.

20.5.1 Rejected on licence

Rejected	Licence	Reason	Source
Windmill as the workflow engine	AGPL-3.0 backend plus a managed-service restriction	"Conditional, never clean"; permitted only as a tenant-installed catalogue app	[O01 DR-003; S04 §03]
n8n	Sustainable Use Licence	Reselling inside a paid platform is the prohibited case	[O01 DR-004; S04 §03]
Inngest	SSPL v1.0	§13 would reach the control plane offered alongside it	[O01 DR-004; S04 §03]
MinIO	AGPL, console removed from CE	"An AGPL dependency with a hostile roadmap sitting under the evidence plane is the exact failure the licence register exists to prevent"	[STS §05; O04 §02]
Redis ≥ 7.4	RSAL/SSPL	Replaced by Valkey (BSD-3) wherever a cache is needed	[STS §01, §05]
HashiCorp Vault	BUSL	Replaced by OpenBao (MPL-2.0)	[STS §06]

Rejected	Licence	Reason	Source
Redpanda	BSL 1.1	Not a queuing service we may host	[STS §05]
LiveKit turn-detector weights	Bespoke licence	"No bespoke-licence model sits in a critical path" (rung1 Voice side)	[O04 §03]

20.5.2 Rejected on architecture

Rejected	Reason	Source
Any model client reachable from rungs 3–8	Invariant I1: "a model may produce a write intent; it may never produce an allow." C1 fails the build	[E02 C1; E01 §02]
A decision cache, fast path, batch variant or admin mode	"A cached allow is an allow at an unknown rule version"	[E01 §07]
A message broker in the write path	Second place for ordering and exactly-once to fail	[ETS §05; STS §01]
Blockchain or ledger database	The chain is the record; anchoring is RFC 3161	[ETS §05]
Distributed SQL	Residency is one deployment per domain; no governance property gained	[ETS §05]
Multi-cloud	"Doubles the surface of every control you must later prove to an auditor"	[STS §07]
JSON or canonical JSON inside the digest	"No JSON in the digest; length prefixes, not delimiters; no optional fields"	[E02 §03; EAP §05]
A shared library between chain writer and verifier	Makes the verifier a second copy of the same assumptions	[E12 §09; E15 §08]
A priced or networked verifier	"A verifier behind a paywall is a verifier a sceptical auditor will not run"	[E10 §06; E15]
Python or Rust for the verifier	Stdlib lacks Ed25519/DER (Python) or all cryptography (Rust); breaks zero-dependency	[ETS §04]
Public endpoints for record stores (Supabase, current host) in production	Cannot satisfy the absent-route controls C05 and C06	[ETS §05, §13]
Software-held signing keys in production	Not HSM custody	[ETS §06; E16]
Free public TSAs in production	No SLA	[ETS §06; E17]
A workflow DSL in front of Temporal	"A year of work that produces a worse debugger"	[S04 decision 7]
Auth0, Okta or Entra as the identity anchor	Not hostable air-gapped	[STS §06]
Elasticsearch for Temporal visibility	Adds a cluster to every air-gapped install; Postgres visibility instead	[STS §07; S04 §05]
Buildpacks	Opaque contents under the artefact scan	[STS §07]
Depending on the official MCP Registry	Preview, API frozen at v0.1	[STS §08]
A hand-written SDK client	A second source of truth that will drift	[STS §04]

20.5.3 Rejected on sequencing

The Build Book's list of things deliberately not built early is a stack decision as much as a plan decision: "Do not stand up production infrastructure, a message broker or a paging vendor before W0's two documents and verifier are done", and "Anything before E01", the Build Book's seal-specification epic, is "the single most likely way the programme goes wrong" [ETS §12; EBB EB-10 §10]. Under the E/O series, W0's "two documents" are the frozen record layout and the Declaration schema major, closing on 30 November 2026 [IM §16.2.2; IM §16.5].

20.6 Build versus buy

The line is drawn by the seam type. "Shut" seams have one implementation and no parallel work. "Closed" seams have one team. Open seams can be partnered or bought [EBB EB-10 §9].

Build (elan1 code)	Adopt open source (pinned, registered)	Consume from stack1 (`fusion.*/v1`)	Buy as a service or contract
Governed-write service and the nine rungs	PostgreSQL	Kubernetes compute, deploy pipeline, rollback	Timestamp authorities (two)
Chain writer, preimage encoder, nightly integrity checker	JSON Schema validators	Secrets broker (OpenBao) and per-call credential minting	Managed KMS per residency domain (L2)
Rules-as-data policy evaluator	React, Vite, Tailwind	Durable execution (Temporal)	Source hosting and CI (GitHub)
Kernel meter, reconciliation, rating (Chapter 19)	OpenTelemetry SDKs	Observability pipeline	Paging vendor
Declaration registry, compiler, semantic rules	pytest, Hypothesis, pgTAP, Playwright	Sandbox pool (for consumers' shadow applies)	Error tracking
MCP server and SDKs (generated)	Official MCP SDK	Object store contract (L2)	Claude Code and the model API for the build harness
Approval console and shared renderers	cosign, Syft, Grype	Identity (Keycloak)	External penetration test and certification audit
Seal A and seal B, anchor client, key register, bundle builder (L2)	pyca/cryptography; `node:crypto`		
Offline verifier (L2), stdlib only	Go toolchain		
Pack declarations, verifier profiles, eval sets			Regulatory counsel for pack clause bindings

Two items in the "build" column would normally be bought, and the reasons are specific. **The policy evaluator** is built because rung 4 determinism is a conformance class (C4), and because the Declaration is itself the policy language (§20.2.5). **The meter and rating** are built because a bought usage-billing product would become a second counter of governed actions, which O07 forbids (Chapter 19, §19.14). Everything in the "consume from stack1" column is a deliberate non-build. "stack1 does not build a lightweight version [of governance] for internal use" [S03 §02], and elan1 does not build a lightweight runtime for its own use.

20.7 Supply-chain constraints

The kernel is sold on a claim that a stranger can check its evidence without trusting the vendor [E07; E15]. The supply chain is therefore part of the product claim at L2, and the discipline starts at L1 so that nothing needs retrofitting. The constraints below are drawn from O04, E15, ETS and STS. Their CI enforcement is in Chapter 21.

#	Constraint	Enforced by	Level	Source
SC-1	Every dependency pinned to an exact commit SHA or lockfile hash; no tags, no `latest`	Lockfile check; Renovate PRs reviewed	L1	[O04 decision 5]
SC-2	Licence re-read at the pinned SHA; row in the in-repo register; verdict before merge	Licence stage in CI; quarterly review	L1	[O04 §05]
SC-3	No `ee/` or `enterprise/` directory enters any artefact	Path check over the built image	L1	[O04 decision 4]
SC-4	Nothing proprietary runs inside an AGPL, SSPL, BSL or ELv2 process	Architecture review; separate network services only	L1	[O04 decision 2]
SC-5	The verifier's lockfile is empty	CI reads the lockfile, not the manifest	L2 (checker from L1)	[E15 §07; EBB EB-09 §1]
SC-6	No shared code between chain writer, seal A, seal B and verifier	Import graph and worktree isolation	L1–L2	[E12 §09]

#	Constraint	Enforced by	Level	Source
SC-7	Reproducible builds for the verifier: two machines, pinned toolchain digest, fixed `SOURCE_DATE_EPOCH`, byte-identical	Release pipeline	L2	[E15 §07]
SC-8	Release artefacts signed; SBOM per image; SLSA provenance	cosign, Syft, GitHub attestations	L1	[ETS §09; STS §06]
SC-9	Verifier published to two independent locations, with source, `SHA256SUMS`, signature, vectors and the tampered bundle	Release protocol	L2	[E15 §07]
SC-10	Publishing a verifier release is itself a governed write with dual control	Kernel	L2	[E15 §07 step 4]
SC-11	Air-gapped installs verify with no transparency-log lookup; one signed OCI bundle	cosign offline verification	L3 (sovereign)	[STS §06–§07]
SC-12	Tool descriptions pinned by hash; a description change is a new certified declaration version	Registry	L1	[E05 §04]
SC-13	Tenant-adjacent telemetry stays in-region; no tenant payloads in logs or error reports	Scrubbers; residency check in CI	L1	[ETS §10; O05]
SC-14	Model weights are not covered by a repository's LICENSE; no bespoke-licence weights in a critical path	Register row per model file	L1 (consumer side)	[O04 §04]

The failure rate that motivates these rules is recorded: "over the last twenty-four months there was one favourable licence move in seven, and five candidate components died in six months — Langfuse and Portkey acquired, Open WebUI relicensed, and others" [O04 §01]. stack1 lists further deaths: Flowise, HF TGI and LLM Guard archived, Zep CE discontinued [STS §02]. The kernel's own dependency surface is deliberately small (Postgres, a web framework, a schema validator and, at L2, one cryptography library per seal implementation and nothing at all in the verifier), so that a relicensing event touches the kernel through stack1's substitution protocol rather than through elan1's code [S04 §06; O04 §05].

20.8 Open stack decisions

#	Decision	Recommendation	Settled by	Owner	By	Source
1	Hosting vendor (OD-28)	stack1's AWS `ap-south-1` footprint	Key custody and deploy topology at GA quality	elan1 lead with stack1 lead	Before stage	[ETS §13 #4; O09 item 2]
2	Migrations tool	Alembic (in repo) for Python services; Atlas if schema diffing across languages is needed	Forward-only governed tables; RLS asserted in a test	Kernel lead	W1	[ETS §05]
3	Policy engine	Own rules-as-data evaluator	EV3 replay at `as_of`; C4	Kernel lead	Before rung 4 build	[ETS §13 #8]
4	Outbox relay	`SKIP LOCKED` polling	Outbox-to-chain p99 ≤ 2 s under load	Kernel lead	Before rung 8 build	[ETS §13 #7]
5	Seal B language and team	TypeScript, separate team	Byte-identical on 64 vectors	Evidence and cryptography lead	Before L2 build	[ETS §13 #2]
6	Verifier language	Go, stdlib only	Zero-dependency RFC 3161 parse on the tampered vectors	Evidence and cryptography lead	Before L2 build	[ETS §13 #1; ETR §12]
7	Ed25519 strictness profile	Reject non-canonical S and small-order points	NV-06 and NV-14 in CI against each library	Evidence and cryptography lead	With E12 v3.0 vectors	[ETS §13 #3; E12]
8	Key custody	AWS KMS Ed25519 per domain	Attestation evidence per domain	elan1 lead	31 Mar 2027	[E16 Q1]
9	TSA pair	eMudhra plus one global TSA	SLA against the ≤ 15 min interval; procurement review	Founder, not delegated	Contract 31 Oct 2027	[E17 Q2; O09 item 3]

#	Decision	Recommendation	Settled by	Owner	By	Source
10	Paging vendor	incident.io or PagerDuty	Two-person rotation fit and cost	SRE lead	Before first external tenant	[ETS §13 #10]
11	Where the proof record physically lives: stack1's evidence plane, or storage elan1 owns outright	Storage elan1 owns: a Postgres instance with its own credential set inside the stack1 footprint	O11's rule that P3 and below never travel with an opco	elan1 lead, stack1 lead	31 Oct 2026	[S03 Q2; O11 §02]
12	Whether ERPNext hosted mode can accept only kernel-issued tokens (Mode B enforced)	Test in October; decides Mode B's claim	Bypass proof	elan1 lead, rung1 lead	31 Oct 2026	[IM §16.2.1; O02 elan1 KC-4]

On decision 11 we recommend that the proof and meter tables live in a Postgres instance whose credentials and backups elan1 controls, deployed inside the stack1 footprint but not in stack1's evidence-plane database (**build recommendation**). S03 leaves the question open [S03 Q2]. O11 puts "the kernel team and plane P3 and below" in the holding company and requires that the kernel and the runtime never sit in the same entity [O11 §02]. A proof record held in the runtime supplier's database would make a stack1 spin-out a question about evidence custody. That is the case O11 exists to exclude.

21. Engineering plan — teams, phases, build sequence and quality gates

This chapter turns the specifications of Chapters 4 to 19 into a plan a team can execute from October 2026. It covers who builds, in what order, against which gates, and how the build proves each step before the next begins. It reconciles three kinds of source. The first is the E and O series, which fix scope, gates and dates. The second is the Information Memorandum's operating and people plan, which fixes months, headcount and tranches [IM §15–§16, §19]. The third is the earlier product-folder build material, which supplies the engineering method: the Build Book's backlog and harness, the Agenthon playbook, the requirement-to-deployment checklist and the Claude Build Dossier [EBB; EAP; ERD; ECB]. Where these disagree, the E and O series prevail and the difference is recorded.

> **Designed is not shipped (DR-010).** As at 23 September 2026 no governed write has been made for a customer, no gate has been attempted, and "governed actions committed and refused stand at zero" [IM §16.1]. The elan1 Agenthon ran from 17 to 21 September 2026 against a five-day plan [EAP §07; ERD §4.7]. The corpus records its plan and not its outcome, so this chapter treats nothing it produced as delivered. Week-level sequencing below is a **build recommendation** anchored on the IM's month-level commitments. Every latency figure is a target to be measured.

21.1 The plan on one page

elan1 builds **Level 1 Guarded only** in its first two phases [E01 decision 1; O01 DR-006]. L2 Attested and L3 Proven are frozen specifications until their entry conditions are met. L2's implementation begins after L1 general availability, and L3 is not started until an outside auditor has accepted an L2 bundle in a real engagement [IM §16.3–§16.4; E20]. The group runs about four teams in parallel. The elan1 kernel team is one of them and is shared by every consumer: "three brands, one kernel team, one founder" [O09 §01].

Phase	Window	elan1 builds	Gate that ends it	Tranche
Phase 1	Oct–Dec 2026	Six coding-blocker documents signed; record shape frozen; kernel rungs 0b–9 at L1; first real governed write in a design partner's ERPNext site with the bypass attempt failing	31 Dec 2026: first real governed write (rung1 kill criterion 5); stack1 real as runtime (stack1 kill criterion 5)	Founder-and-angel bridge ₹8.0 cr, Oct 2026
Phase 2	Jan–Jun 2027	Meter live and reconciled; O05 baseline; E09 readiness rows; conformance C1–C14 published; L1 GA; first third party on KernelPort	30 Jun 2027: L1 GA; R3 (a third party integrated under a signed agreement; elan1 kill criterion 5)	Tranche 1, ₹63.0 cr, Mar 2027
Phase 3	Jul 2027–Mar 2028	L2 build: two seal implementations, key custody, TSA contracts, anchor job, bundle, offline verifier; ledger1; upgrade runbook on a real tenant	31 Jan 2028: Gate 1, a paying customer's write verifies offline; 31 Mar 2028: Gate 2	—
Phase 4a	Apr–Dec 2028	L2 GA; invoice1 and collect1; evidence line; certification path	30 Apr 2028: L2 GA; 30 Sep 2028: Gate A; 31 Dec 2028: Gate 3	Tranche 2, ₹72.0 cr, Oct 2028
Phase 4b	2029–Mar 2031	L3 entry on an event; cross-org verification, selective disclosure, customer-held keys, sovereign deployment; Action Receipt v1	30 Jun 2029: Gate B; 31 Oct 2029: L3 entry; 31 Mar 2031: EBITDA positive	Tranche 3, ₹45.0 cr, Jul 2029

Sources: [P1-00; P2-00; P3-00; P4-00; IM §16.2–§16.5; IM §19.1; O02 §06; O09 §02–§07]. The phase windows follow the phase indexes: Phase 2 is "Q1–Q2 2027", Phase 3 "Q3 2027 to Q1 2028" and Phase 4 "2028 onward" [P2-00; P3-00; P4-00]. Phase 4 is split here only for readability.

Figure 30 lays the same plan on a calendar, with the teams, the tranche dates and the critical path through Phase 3.

Figure 30 — Build roadmap, October 2026 to March 2031

PLAN — TARGETS, NOT MEASURED

Builds, gates, tranches and teams on one calendar. Every date is a plan target; revenue and tenant counts are modelled.

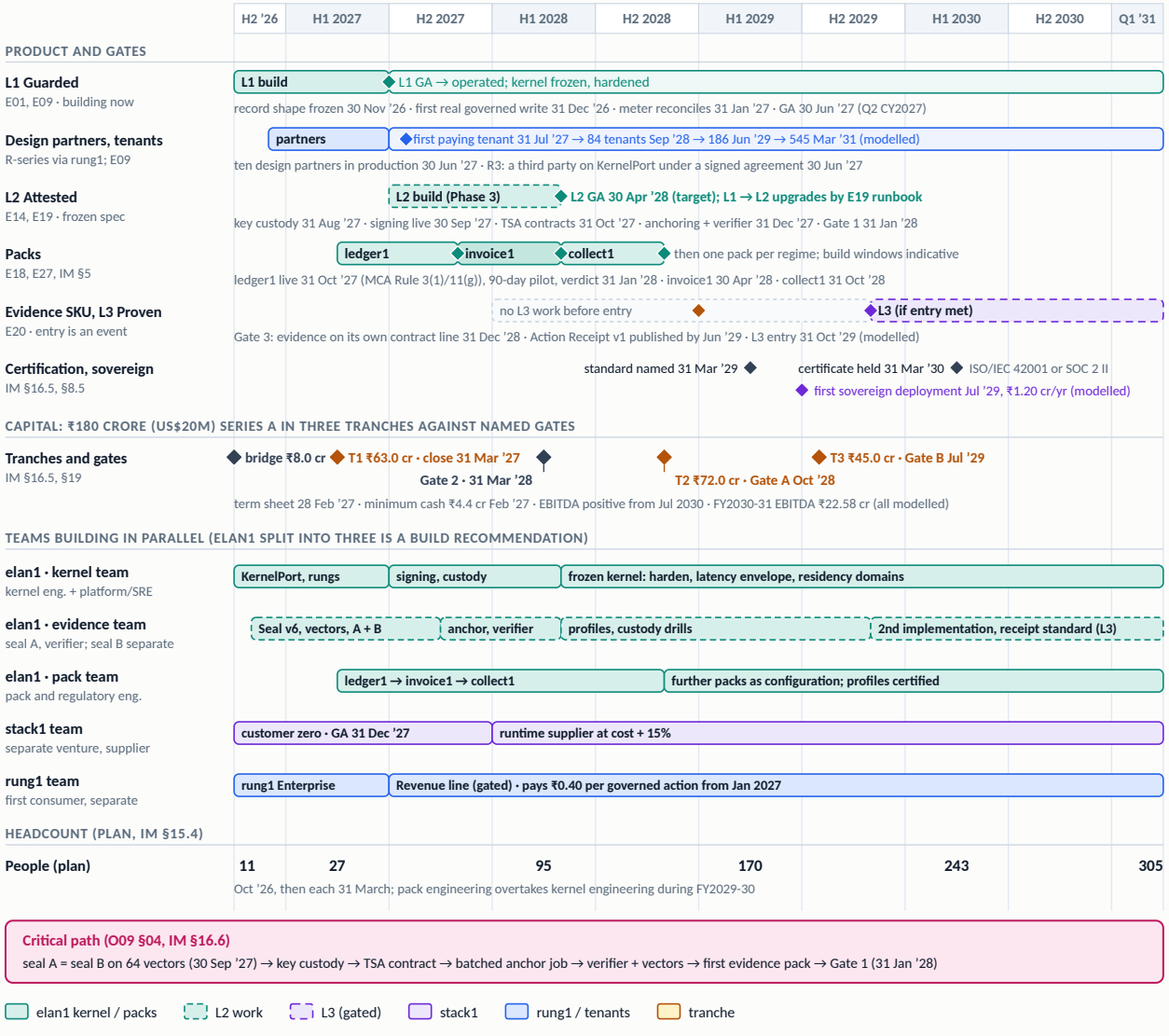


Figure 30 — Build roadmap, October 2026 to March 2031: the four phases, the elan1 kernel and evidence teams beside the stack1 and rung1 teams, the dated gates (first real governed write, L1 GA, L2 signing, anchoring, Gate 1, Gate 2, L2 GA, Gate 3, L3 entry) and the three tranches released against Gate A and Gate B; L1 is built now and L2 and L3 remain frozen specifications until their entry conditions.

> **Nothing is re-scoped to catch up.** "If Gate 1 is missed, it is reported as missed: nothing is re-scoped to catch up" [IM §16.3.4]. Scope added mid-phase "must displace something of comparable size", is limited to two additions per brand per phase, and never lands on the critical path [O09 §08]. A gate review compares evidence with a failure statement written in advance and returns pass, fail or re-enter. "There is no partial tranche, because the partial tranche is how gates decay into reviews" [IM §16.5; O11 §07].

21.2 Team topology

21.2.1 Four teams, one kernel

The sources describe one kernel team serving three brands, a stack1 build team of six seats, and rung1's product team building rung1 Enterprise on Frappe and ERPNext first [O09 §01; STS cover; EAP cover; R01]. Inside the elan1 entity, the people plan separates kernel engineering from evidence and cryptography and from pack and regulatory engineering. They

are three functions with different measures [IM §15.3]. We organise the group's parallel work as four teams (**build recommendation**; team boundaries follow [O03 §12; IM §15.3; S03 §02]):

Team	Entity	Owns (repositories and planes)	Must never own	Phase 1–2 focus
K — elan1 kernel	elan1	`elan1-core`: KernelPort, the nine rungs, gates, policy evaluator, chain writer, meter, approval service, `/v1` and MCP surfaces, SDKs; P3	A second write path; a special case for any product [IM §15.3]	L1 kernel end to end; conformance C1–C14; L1 GA
E — elan1 evidence, declarations and packs	elan1	`elan1-declarations`: schema, lexicon, registry and compiler (P4); E12 text and vectors; nightly integrity checker; pack configuration; at L2 seal B, key register, anchor client, bundle builder, verifier	The chain writer and the verifier sharing a library [IM §15.3]	Schema major; record shape frozen; digest vectors; reference and adversarial declarations; ledger1 declaration
S — stack1 runtime	stack1	The ten `fusion.*/v1` contracts, Kubernetes footprint, secrets broker, Temporal, sandbox pool, observability, deploy pipeline	Anything the kernel decides; reading a tenant record from the control plane [O03 §10]	Runtime real for rung1 and elan1 by 31 Dec 2026; first substitution by 31 Mar 2027
R — rung1 Enterprise	rung1	ERPNext adapters, rung1's declarations, screens within the Four Doors, the design-partner programme	Gate logic, approval logic, the proof record shape, the meter, the autonomy ladder [O03 §10]	First real governed write by 31 Dec 2026; ten design partners by 30 Jun 2027

Teams K and E are funded by this round. Teams S and R are separate ventures at arm's length, and their dates matter here only as dependencies (§21.5). The seam types in the Build Book decide how each team works. The kernel and seal seam is "shut: one implementation and no parallel work at all"; renderers are "closed: one team"; declarations, connectors and verifier profiles are open seams with many authors [EBB EB-10 §9]. That is why team K is deliberately not split into sub-teams by rung. A second kernel team is one of the things the Build Book says to hire late, "because the seam is shut" [EBB EB-14 §4].

21.2.2 The elan1 team today and to March 2027

The people plan gives the elan1 entity eleven people at October 2026 and 27 at March 2027, when tranche 1 closes [IM §15.2, §15.4]:

Function	Oct 26	Nov	Dec	Jan 27	Feb	Mar	Team
Kernel engineering	4	5	5	6	6	7	K
Evidence and cryptography	1	1	2	2	3	3	E
Pack and regulatory engineering	1	1	1	2	2	2	E
Platform and SRE	2	2	2	3	3	3	K (with S)
Security, compliance, certification	0	0	1	1	2	2	cross-team
Product and design	1	1	1	2	2	2	K (console)
Solutions and field engineering	0	0	1	1	2	2	with R
Go to market and partners	1	1	2	2	3	3	—
Standards, policy, regulator affairs	0	0	0	1	1	1	E
Finance, legal and operations	1	1	1	2	2	2	—
Total	**11**	**12**	**16**	**22**	**26**	**27**	

Sources: [IM §16.2.1–§16.2.6] for the monthly steps; [IM §15.4] for March 2027. The team column is our allocation (**build recommendation**).

Three hires are named as the ones that "must be filled before tranche 1 is spent": the evidence and cryptography lead, the pack and regulatory engineering lead, and the go-to-market and partners lead [IM §15.2]. The evidence and cryptography function "grows from one person in October 2026 to three by March 2027 and nine by March 2028". The reason is

structural: the two-implementation rule "means this hire cannot be a single person for long" [IM §15.2]. The security and certification owner is "already blocking, because the compliance owner is the named owner of the regulatory watch list and there is nobody in the seat" [IM §15.2]. The plan fills that seat in December 2026 [IM §16.2.3].

21.2.3 The founding roles inside team K and team E

The Build Book's founding four are a spec author, a kernel engineer, a verifier engineer and a second implementer, "hired on day one, implements the seal from the text, does not talk to the kernel engineer about how they did it" [EBB EB-14 §2]. The Agenthon compressed them into six seats: a lead (spec author and orchestrator), a co-lead (verifier engineer, reporting to the advisor and outside delivery), a first implementer, a second implementer who also owns the CI invariants, an advisor (second reader, red team, certifier-in-training) and the founder as decision owner [EAP §06; ETR §01]. The Build Book designed these roles for a seal-first programme. Under the E-series order the same roles map onto the L1 build (**build recommendation**). The spec owner in team E owns the meaning of the E02, E03 and E12 text and never implements the seal. The first implementer in team K writes the chain writer and preimage encoder in Python, and becomes seal implementation A in Phase 3. The second implementer in team E writes the nightly integrity checker in Go from the E12 text alone, and becomes seal implementation B in TypeScript. Neither reads the other's code [E12 §08]. The verifier engineer owns the CI invariants and the checker's empty lockfile at L1, and writes the offline verifier at L2. The second reader reviews every `/spec` change and apprentices as a certifier, and never certifies a declaration they authored [EBB EB-13 §9].

The certifier-in-training is the other hire that "looks premature on a normal schedule". The Build Book puts the ceiling at "~3 concurrent declarations at any team size", and "the harness does not move this number; only apprenticeship does" [EBB EB-13 §5, §7; EBB EB-14 §3]. The ceiling binds team E from the moment ledger1 and third-party declarations compete for certification in Phase 2 and Phase 3.

21.2.4 The shape to March 2031

Headcount grows to 95 by March 2028, 170 by March 2029, 243 by March 2030 and 305 by March 2031 [IM §15.4]. Kernel engineering grows 7 → 17 → 25 → 32 → 38. Pack and regulatory engineering grows 2 → 11 → 23 → 37 → 50 and overtakes kernel engineering in July 2029 [IM §15.4]. The reason is an engineering fact. "The kernel is frozen by design … A frozen surface needs a team that maintains and hardens it, not one that grows with revenue." Packs are "intended churn, one per regime". "If those two lines ever cross back the other way, it means the kernel is being reopened" [IM §15.4]. That crossover is a metric in §21.11.

21.3 Phases and milestones, October 2026 to March 2031

21.3.1 The gate calendar the build is judged against

The gates below are the engineering subset of the IM's gate calendar. Each has an observable condition and a consequence stated before the date [IM §16.5].

Date	Gate	Observable condition	Owner	If missed
31 Oct 2026	Six coding blockers	E02, E03, R03, R01, S01, O01 all SIGNED	Founder	Coding starts against a moving contract
30 Nov 2026	Record shape frozen	Sixteen-field preimage frozen; Declaration schema major (`doors` → `channels`) published	Kernel engineering	L1 → L2 becomes a migration of customer evidence
31 Dec 2026	First real governed write	One end-to-end write, gate check, approval and proof record in a design partner's own site, bypass attempt failing	rung1 lead	rung1 kill criterion 5 fires
31 Jan 2027	Meter reconciles	Two-way reconciliation clean for a full month; first internal invoice on a reconciled period	Kernel engineering	Billing discipline unproved before GA
30 Jun 2027	L1 GA	All eight E09 readiness rows done and measured	elan1 lead	First external tenant slips out of July
30 Jun 2027	R3	A third party integrated against KernelPort under a signed agreement	Founder	elan1 kill criterion 5 fires

Date	Gate	Observable condition	Owner	If missed
30 Sep 2027	L2 signing live	Two seal implementations agree on all 64 vectors; diff empty; disagreement log published	Kernel engineering	Gate 1 moves a quarter
31 Jan 2028	**Gate 1**	A paying customer's write verifies on an air-gapped machine, no elan1 process running, witness named	Kernel engineering	Gate 1 moves to Phase 4
31 Mar 2028	**Gate 2**	App three renders its four doors with zero screen code, built by a team that built neither of the first two	Product and design	The platform thesis is decoration
30 Apr 2028	L2 GA	E14 done-measurements 1–8 closed and dated	elan1 lead	L2 cannot be quoted
31 Oct 2029	L3 entry	The four-test external-auditor acceptance, in writing	Founder	L3 is not started

Sources: [IM §16.5; O09 §03–§07; O02 §06; E14]. The kill criteria for the kernel are engineering tests with dates [O02 §06]. Three of the five fall inside Phases 1 and 2. The path must hold "p50 ≤ 120 ms, p99 ≤ 450 ms — at ten design partners' load, measured, not estimated" (KC1). Enforcement outside the system of record must be possible, meaning some ERPNext mode lets the kernel hold the credential or the site accept only kernel tokens (KC4). A third party must be integrated against KernelPort by 30 June 2027 (KC5).

21.3.2 Tranches as engineering gates

Tranche releases are gated on engineering facts an outsider can check [IM §19.1]. Gate A (tranche 2, ₹72.0 crore, October 2028) requires, among eight conditions, L2 GA since 30 April 2028 with "two independent seal implementations, one written by a team with no access to the other's source, agree octet for octet on all sixty-four published vectors with an empty diff and the disagreement log published". It also requires the verifier to refuse all 28 negative vectors "with the empty-lockfile check green in CI", a survived single-TSA outage drill, and "cost per governed action measured under production load — not modelled — published weekly for at least twenty-six consecutive weeks" [IM §16.5.1]. Gate B (tranche 3, ₹45.0 crore, July 2029) requires the L3 entry condition, Action Receipt v1 "with at least two named external implementers … of whom at least one has produced a receipt our verifier accepts", and a certification path started [IM §16.5.2].

These conditions are written into the CI and release design below. The two-implementation diff, the empty lockfile and the published disagreement log are CI artefacts, not documents assembled for a board meeting (§21.7). The 26-week cost series is a scheduled job over the meter (Chapter 19, §19.13).

21.3.3 The Phase 3 critical path

Phase 3 is "one chain of six items ending in a real customer's write verifying offline" [O09 cover]: key custody (31 August 2027), L2 signing (30 September), the TSA contract (31 October, live by 31 December), the batched anchor job (30 November), the offline verifier and vectors (31 December), and the first India evidence pack (31 January 2028). "The chain carries roughly one quarter of float end to end. Spend it once" [O09 §04; IM §16.6]. The longest-lead item is the TSA contract, "a negotiation with an entity we have not yet named, on someone else's calendar, that no amount of engineering shortens" [O09 §04]. Engineering cannot shorten it, but it can avoid lengthening it. The anchor client is built against two dev-only TSAs from Q2 2027, so that the only thing left to do on contract signature is configuration (**build recommendation**; free TSAs are dev-only [E17 §03]).

21.4 The L1 build sequence

21.4.1 Ordering principles

Four rules decide the order. Each comes from a source, and each has cost someone before.

1. **Contracts before code.** "Six documents block coding" and "each moves from DRAFT to SIGNED" in October, or "coding has either not started or has started against a moving contract" [IM §16.2.1]. The build does not begin in October. The first weeks produce signed text, a harness and fixtures.

2. **The record shape before the rung that writes it.** "Rung 8 cannot be built before E01 is done, because rung 8 is the seal." "A kernel with a placeholder seal is a kernel whose most expensive rung has never been exercised" (E01 here is the Build Book's first epic, the seal specification, not the scope-freeze document) [EBB EB-10 §5]. Under the E-series the frozen object at L1 is the sixteen-field preimage and its SHA-256 digest, not a signature [E02 §03; E01 §03]. It freezes by 30 November 2026 [IM §16.2.2].
3. **Refusal paths before success paths.** The first governed write that matters is the one that is refused and the bypass that fails. "The demonstration without the bypass attempt is a demonstration of a feature, not of a boundary" [IM §16.2.3].
4. **Specification of L2 in parallel, implementation of L2 later.** "E12 Seal Spec v6 and its sixty-four test vectors are drafted this month. They are not implemented … If the spend ever buys implementation rather than specification, the defence has failed" [IM §16.2.2].

The Build Book's order is different. It puts the seal specification and the offline verifier first, as the first 75 days, and says "anything before E01" (its seal-specification epic) is "the single most likely way this programme goes wrong" [EBB EB-10 §2, §10; EB-14 §5]. That order was written for a design that signed from the first write. E01 removes signing from L1, so the sequencing reason behind the Build Book's rule, that a kernel built against a draft seal gets rewritten, now applies to the *preimage*, not the signature. The plan below keeps the rule's substance by freezing the preimage and its digest vectors before rung 8 is written, and drops the verifier-first step. The conflict is recorded.

21.4.2 The ordered L1 backlog

#	Item	Team	Depends on	Done when	Source
L1-01	Sign E02, E03 (with R01, R03, S01, O01)	K, E, founder	—	Status SIGNED on all six	[IM §16.2.1]
L1-02	Close E02 Q1 (synchronous rung 9), Q3 (`GATE.SANDBOX_DIVERGED` class), Q5 (reference tenant owner); R04 Q1 (`seq` visible)	K	L1-01	ADR per answer	[E02 Q1, Q3, Q5; R04 Q1]
L1-03	Harness, repository map, CI invariants C1 (import graph), C14 (claim-boundary strings), grants-checker, lockfile check	K, E	—	Repo refuses an upward import, a model client in rungs 3–8, a verifier dependency	[E02 C1, C14; EBB EB-09 §1]
L1-04	Reference tenant and its adversarial declaration; synthetic fixtures	E	L1-02	Simulation produces refusals on every declared gate	[E02 Q5; E09 §04 step 5]
L1-05	Record store DDL: records, outbox, proof, gate results, obligations, meter; grants, FORCE RLS, partitions	K	L1-01	Grants-checker green; cross-tenant read test in CI	[ETS §05; O05]
L1-06	Preimage encoder and chain writer (rung 8), Python	K	L1-05; E12 text	PV-01–PV-24 structure and chain vectors pass; L1 negatives rejected	[E02 §03; E12 §07]
L1-07	Nightly integrity checker, Go, from the E12 text by a second implementer	E	E12 text	Byte-identical digests with L1-06 on PV-01–PV-24; C9 over 10,000 records	[E01 §02 item 7; E12 §08]
L1-08	Declaration registry, `define`, schema validator and nine semantic rules; `define` itself governed under dual control	E	L1-05	Version-not-sequential, not-certified and invalid declarations rejected	[E02 §05; E03]
L1-09	Rungs 0b–2: stop, identity from token, write door at explicit version	K	L1-05, L1-08	C13 version pinning; tenant from token only	[E02 §05; E04]
L1-10	Rung 3 gates: six never-skippable plus rate, budget, schedule; a gate-result row per declared gate, all gates run	K	L1-09	C3 rung completeness over 100 sampled intents	[E04; E02 decision 5]
L1-11	Rung 4 `evaluate()`: rules-as-data evaluator, `policy_version`, no clock, no model	K	L1-08	C4 replay of every reference refusal	[E02 §05, C4; ETS §13 #8]
L1-12	Rungs 5–6: blast-radius guardrail; autonomy A0–A5, missing → A0; irreversible → escalate	K	L1-10	C6 over 1,000 fuzzed cases	[E02 C6]

#	Item	Team	Depends on	Done when	Source
L1-13	Approval service and obligations; approver ≠ requester; no agent approver; durable signal to the consumer's Temporal run	K	L1-12; stack1 `fusion.run/v1`	C11; expiry and decline are governed outcomes	[E02 C11; S03 decision 6]
L1-14	Rung 7 single writer: per-write credential via stack1 secrets broker (ten-minute life), sandbox apply, gate re-run, identical bytes to live, snapshot and 30-day undo	K with R	L1-12; ERPNext adapter; stack1 sandbox pool	C2 bypass attempts fail and are recorded; C12 rollback drill	[E01 §02 items 4–6; E02 C2, C12]
L1-15	Rung 9 meter and nightly reconciliation	K	L1-06	C7, C8, C10; MT-1 to MT-4 (Chapter 19)	[E02 C7, C8, C10]
L1-16	Surfaces: `/v1` HTTP, 34 MCP tools, Python SDK, signed webhooks	K	L1-09 to L1-15	Error taxonomy matches E05 §07 exactly	[E05]
L1-17	Console and approval screens: diff, gate results, policy version before Approve	K (product)	L1-13	Approver sees what E06 requires	[E06; IM §16.2.3]
L1-18	`verify` at L1 (chain continuity, three-valued) and the ops report	E	L1-07	Flipped octet, re-order, duplicate and gap each named	[E02 C9]
L1-19	Security, residency and DPDP baseline, 13 rows	SRE, security	L1-05, L1-14	Every row evidenced	[O05; IM §16.2.4]
L1-20	Latency measured per rung on production-shaped hosting	K, SRE	L1-16	p50/p99 published against 120/450 ms	[E01 Q2; O02 KC1]
L1-21	GA readiness: docs, sandbox keys, quotas, status page, support rota, incident notes, version policy, C1–C14 published runnable	K, E, SRE	L1-16 to L1-20	All eight E09 rows done and measured	[E09; IM §16.2.5]

21.4.3 Week-level plan, October 2026 to March 2027

The IM commits to one "must be true" condition per month [IM §16.2]. The week-level plan below schedules the backlog against those conditions. It is a **build recommendation**. Weeks start on Monday, and W1 is 5 October 2026.

Weeks	Dates	Team K	Team E	Dependency / exit
W1–W2	5–16 Oct	L1-03 harness and CI invariants; repository map with one writer per file; environments dev and ref on the current host	E02/E03 final text; reference-tenant fixture plan; E12 text drafting starts	Agenthon handover triaged: harness and invariant scripts kept, 311-octet artefacts discarded (§21.10.3)
W3	19 Oct	L1-02 ADRs: synchronous rung 9, `SANDBOX_DIVERGED` class, `seq` visibility (R04 Q1 due 15 Oct)	Adversarial declaration draft (L1-04)	ERPNext token-acceptance test with team R (decides whether Mode B can be enforced) [IM §16.2.1]
W4	26 Oct	L1-05 DDL, grants, RLS, outbox; grants-checker live	Declaration schema major draft (`channels`)	**31 Oct: six blockers SIGNED**
W5–W6	2–13 Nov	L1-09 rungs 0b–2; token classes	L1-08 registry and `define` ; semantic rules	Counsel: AGPL "separate service" line (30 Nov) [IM §16.2.2]
W7	16 Nov	L1-06 preimage encoder and chain writer against the draft E12 field table	L1-07 checker from the same text, no shared channel	First two-implementation diff on PV-01–PV-12
W8–W9	23 Nov–4 Dec	L1-10 rung 3 gates; L1-15 meter table and CHECK	Diff, fix text, re-implement; E02 Q2 salting scheme closed	**30 Nov: preimage frozen; schema major published**
W10	7 Dec	L1-11 `evaluate()` ; L1-12 guardrail and autonomy	Reference tenant runs rungs 0b–6	stack1 runtime hosting kernel services in stage
W11	14 Dec	L1-14 rung 7 single writer with team R on the design partner's site; L1-13 approval service	Nightly checker running on the ref chain	stack1 secrets broker minting per-write credentials
W12	21 Dec	L1-17 minimal approval screen; first end-to-end write; bypass attempt scripted	C2, C8 fixtures	Rehearsal on sandbox site
W13	28 Dec	Freeze; run the demonstration in the design partner's own ERPNext site	Witness notes	**31 Dec: first real governed write, bypass failing; stack1 real as runtime**

Weeks	Dates	Team K	Team E	Dependency / exit
W14–W15	4–15 Jan	Meter live on rung1 traffic; reconciliation M1–M4 nightly; internal rating	L1-18 `verify`; C9 at 10,000 records	O05 baseline rows closing [IM §16.2.4]
W16–W17	18–29 Jan	L1-16 surfaces: `/v1`, MCP tools, SDK; webhooks	C1–C14 harness packaged for consumers	**31 Jan: meter reconciles a full month; first internal invoice**
W18–W19	1–12 Feb	L1-20 latency per rung on stack1 hosting; L1-14 hardening; C12 rollback drill	Declaration Studio flows (author, validate, simulate, certify)	Restore drill with `verify` on restored sample [IM §16.2.4]
W20–W21	15–26 Feb	L1-21 readiness rows 1–4: docs, sandbox keys < 60 s, quotas, status page with chain integrity	Readiness rows 5–8: support, incident notes, version headers, C1–C14 published	**28 Feb: E09 rows worked through; term sheet**
W22–W23	1–12 Mar	Bug burn-down against design-partner traffic; C5 fail-closed matrix	ledger1 declaration draft; E12 vectors complete (unpublished)	stack1 first substitution (31 Mar)
W24–W26	15 Mar–2 Apr	Third-party onboarding flows (E09 Flows 1–4) in sandbox	Third-party reference integration	**31 Mar: tranche 1 received**

Two features of this schedule are deliberate. The first governed write in December uses the minimum surface: the SDK path rung1 uses, one approval screen and one design partner. The MCP surface, the full console and third-party onboarding come in Phase 2, because nothing on the 31 December condition needs them. And the second implementer's work on the nightly checker starts in the same week as the chain writer, so the first two-implementation diff happens before the preimage freezes, not after [E12 §08 step 3].

21.4.4 Phase 2 and Phase 3 in outline

From April to June 2027 the backlog is L1 GA and R3: third-party onboarding [E09 Flows 1–5], the four-check production review [IM §16.3.1], and a named third party integrated under a signed agreement [O02 KC5]. Team E's L2 build begins in the same quarter and follows the O09 chain in §21.3.3, ending with the L1 → L2 upgrade runbook [E19] executed on a real tenant in Q1 2028 [IM §16.3.4].

21.5 Dependencies on stack1 and rung1

The dependency direction is one-way in code: "elan1 depends on stack1, never the reverse", and nothing above P3 imports a kernel type [S03 decision 2; O01 §02]. In the calendar it runs both ways. elan1 needs stack1's runtime, and rung1 needs elan1's kernel. O09's rule governs slippage: "stack1 slippage reaches elan1 and never reaches rung1" [O09 §06; IM §16.6].

#	elan1 needs	From	By	If late	Source
D1	Runtime rung1 and elan1 actually run on: compute, deploy pipeline, observability	stack1	31 Dec 2026	stack1 kill criterion 5; elan1 has no production host	[O02 stack1 KC5]
D2	Secrets broker minting per-write target credentials	stack1	W11 (Dec 2026)	Rung 7 cannot be enforced; Mode A falls to advisory	[E05 §02; S03 decision 5]
D3	`fusion.run/v1` (Temporal) with approval as a durable signal	stack1	W11 (Dec 2026)	Escalations lost on restart	[S03 decision 6; S04 §02]
D4	Sandbox pool for shadow applies	stack1 / rung1	W11 (Dec 2026)	Rung 7 has nothing to apply to first	[E01 §02 item 5; S03 Q1]
D5	Residency-domain deploy topology and secrets at GA quality (for key custody)	stack1	31 Aug 2027	L2 signing slips; Gate 1 moves	[O09 item 2]
D6	`fusion.run/v1` blocking-green (for the anchor job)	stack1	30 Nov 2027	Duplicate anchors on replay	[O09 item 4]
D7	ERPNext adapter and rung1's declaration; idempotency key `sha256(changeset_id ‖ base_schema_hash)`	rung1	W11 (Dec 2026)	No real write by 31 Dec	[R04 decisions 1, 5]
D8	A design partner's own site for the first write	rung1	31 Dec 2026	rung1 kill criterion 5	[IM §16.2.3]

#	elan1 needs	From	By	If late	Source
D9	Ten design partners writing in production	rung1	30 Jun 2027	KC1 latency cannot be measured at load	[O02 rung1 KC1; elan1 KC1]
D10	A design partner who will hand a bundle to their own auditor	rung1	30 Sep 2027	Gate 1 fails on the customer side	[O09 Q3; IM §16.3.2]

#	rung1 or stack1 needs from elan1	By	Source
U1	KernelPort v1 signed and frozen	31 Oct 2026	[IM §16.2.1]
U2	Declaration schema major with `channels`	30 Nov 2026	[IM §16.2.2; O06 §03]
U3	Whether the L1 `seal` returns a user-visible `seq`	15 Oct 2026	[R04 Q1]
U4	A `voice` kind in Declaration block 7 for rung1 Voice, riding a schema major	30 Sep 2027	[O09 item 9]

O09's founder-attention rule is also a dependency. Seven counterparty classes in Phase 3 need the founder personally, and "there is no version of this plan in which the founder is the constraint and the portfolio absorbs it" [O09 §09–§10]. For the engineering plan this means that items needing a founder decision are batched to a fixed weekly slot, with a written default that applies if the slot is missed (**build recommendation**). The TSA contract is the exception, and is never defaulted.

21.6 Definition of done

21.6.1 L1 is done when ten measurements hold

E01 defines L1 done as ten measurements, "not an opinion". Items 1–7 are hard gates, and items 8–10 "make the claim honest" [E01 §06]:

#	Done means	How we know
1	Every rung runs on every governed write	Rung-skip suite; a stage that skips fails CI
2	No model reachable from rungs 3, 4, 5, 6 or 8	Import-graph check in CI
3	Zero writes committed without a chained record	Nightly reconciliation; non-zero is a P0
4	Zero records mutated outside the kernel writer	Same reconciliation
5	Every refusal replayable	Nightly replay of every reference-tenant refusal
6	Every refusal code maps to one rung, one class, one operator action	Register lint
7	Every meter event references a write that exists; every refusal meters zero	Database CHECK plus nightly reconciliation
8	Rollback restores a sandbox and a live tenant inside 30 days	Quarterly drill, written up, dated
9	Write latency within the published budget	Measured, not estimated
10	rung1 makes a real governed write to a real ERPNext site through the kernel, and nobody can make one around it	Demonstration with the bypass attempt shown failing

L1 GA adds the eight E09 readiness rows. "GA does not open until all eight readiness rows read done, measured; a partial GA is not a GA" [E09 decision 2].

21.6.2 A change is done when

For a single change we recommend the following definition, assembled from the sources (**build recommendation**; each clause sourced):

1. It answers the Build Book's three merge questions. Does it change what is recorded? That is a frozen-layout change, and the answer is no. Does it change what a declaration can express? Then it is a schema version with semantic rules, exercised by the adversarial declaration. Does it add a way to write? Then it is a second write path, and the answer is no [EBB EB-12 §6].

2. Its tests have been shown to fail. A test is not trusted green until it has failed against a deliberately broken build ("mutation proof for every gate test") [IM §15.6; WD].
3. The conformance classes it touches are green on the reference tenant, and the invariants in §21.7 are green.
4. Every count or claim in its documentation is derived from the source of counts, never typed [EBB EB-09 §4 `doc-crosschecker`].
5. If it touches the kernel for a product's sake, it passed O03's change-request test first: "can this be expressed as a declaration or an adapter? If yes, the request is closed with the expression and no kernel change happens". Otherwise an ADR is signed by the founder and "a CI invariant added that would fail if the old behaviour returned" [O03 §11; IM §15.5].
6. It carries a runbook step if it adds an operational behaviour, and every runbook step ends in a verification command [ETS §10].

21.7 The CI pipeline and quality gates

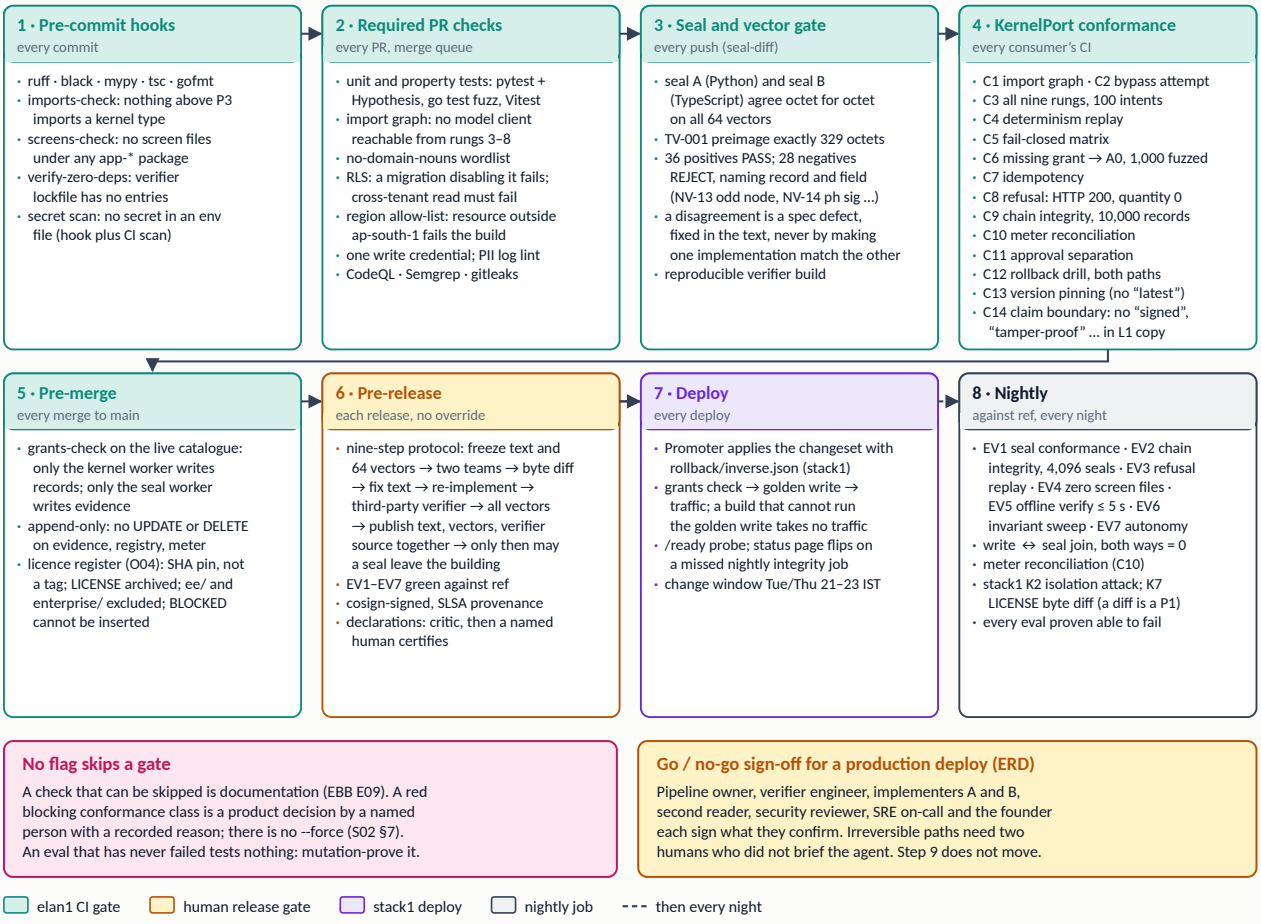
21.7.1 What the pipeline enforces

"A claim without a check is a claim that expires quietly. None of the eight can be skipped with a flag; a check that can be skipped is documentation" [EBB EB-09 §1]. The pipeline exists to turn the invariants of Chapters 4, 9, 10 and 19 into build failures. Figure 31 shows where each check runs, from the developer's machine to production traffic.

Figure 31 — CI pipeline and quality gates

DESIGNED — PIPELINE NOT YET RUNNING

From commit to traffic: eight gate stages, each a failing check rather than a review; none can be skipped with a flag.



Source: E02 §08, E12 §07–§08, EBB E09 and EB-05, ERD 6.11, 7.3, sign-off, O04 §01–§03, O05 §1–§2, S02 §7, S12 §6, IM §6.5 · elan1 Technology Memorandum · Sept 2026

Figure 31 — CI and quality gates: pre-commit (import graph and P3 boundary, lockfile, secrets), pull request (unit and property tests, register lint, schema lint, licence and ee/ path check, claim-boundary strings), push (digest vectors across two independent implementations), merge (grants check against stage, conformance C1–C14 on the reference tenant), nightly (replay, chain integrity, reconciliation, fail-closed matrix, evals), release (the seal release protocol from L2) and deploy (grants check, golden write, then traffic).

Stage	Check	Enforces	Level	Source
Pre-commit	Import graph: no consumer imports a kernel type; nothing reachable from rungs 3–8 imports a model client; stack1 never imports elan1	C1; I1; P3 boundary; S03 B1	L1	[E02 C1; O01 §02; S03 §03]
Pre-commit	Lockfile check on the integrity checker (L1) and verifier (L2): empty	Zero dependencies	L1	[E15 §07; EBB EB-09 §1]
Pull request	Refusal register lint: every code has one rung, one class, one operator action; wire spellings unchanged	E01 done-item 6	L1	[E01 §06; E02 decision 6]
Pull request	Declaration critic and schema lint when a declaration changes	Nine semantic rules; dispositions	L1	[EBB EB-09 §9]
Pull request	Licence stage: every new or bumped dependency has a register row at its pinned SHA; no `ee/` or `enterprise/` path in the artefact	O04 rules	L1	[O04 decisions 4–5]
Pull request	C14 claim boundary: consumer-rendered copy contains no "signed", "tamper-proof" or "independently verifiable" at L1	Honest claim	L1	[E02 C14]
Push	Digest vectors PV-01–PV-24 and the L1 negatives through both implementations; byte diff must be empty	Two-implementation rule at L1	L1	[E12 §07–§08]

Stage	Check	Enforces	Level	Source
Push (L2)	All 64 vectors through seal A, seal B and the verifier; 28 negatives rejected with record and field named	Two-implementation rule; verifier independence	L2	[E12 §08; IM §16.5.1]
Merge to main	Grants-checker against stage: only the kernel writes records, only the chain writer writes the proof log, no UPDATE/DELETE on append-only tables	One write path; append-only	L1	[EBB EB-09 §1; ETS §09]
Merge to main	Conformance C1–C14 against the reference tenant	KernelPort conformance	L1	[E02 §08; E09]
Merge to main	Infra-diff: the absent routes are absent in the reviewed plan	Network boundaries	L1	[ETS §07; STS §07]
Nightly	C4 refusal replay; C9 chain integrity; C10 meter reconciliation; C5 fail-closed matrix; C6 fuzz; write-to-record join both directions	E01 done-items 3–7	L1	[E02 C4–C10; EBB EB-11 §8]
Nightly	Evals: chain integrity (removed middle; cut tail at L2), refusal replay, autonomy default, invariant sweep	Specification conformance	L1–L2	[EBB EB-12 §4]
Pre-release (L2)	Nine-step seal release protocol; no override past step 9	"Only then may a seal leave the building"	L2	[EBB EB-05 §8; E12 §08]
Deploy	Grants check, then the golden write, then traffic	"A deploy that cannot run the golden write successfully does not take traffic"	L1	[EBB EB-09 §9]

The Build Book's eight invariants map onto this table with two adjustments for L1. First, `vectors-agree` runs over the digest-only vectors, because there is no signature to compare at L1. Second, `no-screen-files` applies to consumer apps, not to the kernel repository [EBB EB-09 §1]. `seal-or-nothing` becomes "every committed write has one chained record", which is E01 done-item 3 [E01 §06].

21.7.2 The two-implementation rule as a CI mechanism

E12's Flow 3 is a process with a hard failure rule: "If the two teams converge only after one team has read the other's code, the run is void and restarts with a new pair of engineers" [E12 §08]. A CI job can compare outputs. It cannot prove that two people did not talk. We recommend making isolation a repository fact rather than a behavioural promise (**build recommendation**):

- The chain writer or seal A, the integrity checker or seal B, and the verifier live in **three repositories** with disjoint write permissions and disjoint read permissions for their implementers. The Agenthon enforced the same thing with worktrees that "cannot check out" the other branch [EAP §06].
- A fourth, **vectors** repository holds the E12 text, the vector inputs and, only after convergence, the agreed digests. Its CI checks out all three implementations at pinned SHAs, runs every vector, and publishes the octet diff as an artefact.
- Every disagreement opens a defect against the **text**, recorded with the sentence that caused it and the amendment that fixed it. That log "is the most valuable document this process produces" and is published at L2 [E12 §08; IM §16.5.1].
- Agent sessions in one implementation's repository cannot read the others'. `spec-reader` answers only from the text: "If the text does not say, the answer is 'the text does not say'" [EBB EB-09 §4; EAP §05 practice 7].

21.7.3 A pipeline sketch

The workflow below shows the shape of the gates for the kernel repository (**build recommendation**; job names are illustrative, and the checks are those of §21.7.1).

```
name: kernel-ci                    # .github/workflows/kernel.yml (abridged)
on: [pull_request, push, merge_group, schedule]
jobs:
  invariants:                      # pre-commit checks, re-run in CI
    steps:
      - run: lint-imports --config ci/p3-boundary.ini    # C1, I1, nothing above P3
      - run: ci/no_model_in_rungs.py                     # rungs 3-8 import no model client
```

```
      - run: ci/lockfile_empty.sh checker/go.sum        # zero-dependency checker
      - run: gitleaks detect --no-git
  unit_property:
    steps: [ {run: "uv run pytest -q --hypothesis-profile ci"} ]
  registers:
    steps:
      - run: ci/refusal_register_lint.py                # E01 done-item 6
      - run: ci/licence_register_check.py --pins uv.lock # O04 rows at pinned SHA
      - run: ci/no_ee_paths.sh dist/                    # O04 decision 4
  vectors_agree:
    if: github.event_name == 'push'
    steps:
      - run: vectors/run.sh --impl a=chain-writer@${A_SHA} --impl b=checker@${B_SHA}
      - run: vectors/diff.sh --must-be-empty --set PV-01..PV-24,NV-L1
  conformance:
    if: github.event_name == 'merge_group'
    steps:
      - run: ci/grants_check.py --env stage             # live catalogue, not migrations
      - run: conformance/run.sh --tenant ref --classes C1..C14 --out results.json
      - run: infra/plan_diff.sh --assert-absent z2-z3,z4-z3
  nightly:
    if: github.event_name == 'schedule'
    steps:
      - run: conformance/run.sh --tenant ref --classes C4,C5,C6,C9,C10
      - run: ops/reconcile.sh --both-directions && curl -fsS "$HC_PING_URL"
```

The dead-man ping on the last line follows the Build Book: the nightly join "pings on success; silence pages" [ETS §10].

21.8 Test strategy

The Build Book separates two kinds of test: "unit tests are the team's business. The seven evals and the sixty-four vectors are not — those are specification conformance and they do not negotiate" [EBB EB-09 §10]. The strategy below keeps that split and adds the classes the kernel needs.

Class	What it covers in the kernel	Tools	Gate	Source
Unit	Each rung stage as a function; encoders; rating	pytest; go test; Vitest	PR	[ETS §09]
Property	Preimage encoder round trips and length prefixes; autonomy resolution (C6, 1,000 fuzzed cases); caller cannot set meter quantity; blast-radius arithmetic	Hypothesis; go test fuzz	PR	[E02 C6; EBB EB-12 §5]
Golden	Digest vectors (PV/NV); golden write end to end; refusal replay (C4); rating of closed periods	Vector runner; recorded fixtures	Push, nightly	[E12 §07; E02 C4]
Conformance	C1–C14 against the reference tenant; published so consumers can run them	Conformance harness	Merge, release	[E02 §08; E09]
Adversarial	The reference tenant's adversarial declaration; bypass attempts against the target (C2); injected instructions upstream of rung 0b that must not yield an allow; gate-set probing (E02 Q4)	Red-team fixtures; scripted bypasses	Merge, nightly	[E02 C2, Q4, Q5; S03 §02]
Chaos	Fail-closed matrix: registry, rule engine, autonomy table, target unavailable in turn (C5); chain writer killed mid-append; writer lock lost; secrets broker down; at L2 KMS and one TSA unavailable	Toxiproxy; fault injection in stage	Nightly, quarterly drill	[E02 C5, §05 failure paths; STS §01; IM §16.5.1]
Database	Grants, RLS, CHECK constraints asserted against the live catalogue	pgTAP; grants-checker	Merge	[ETS §09]
Performance	Per-rung spans; p50/p99 of the governed write against 120/450 ms	OpenTelemetry; load generator	Weekly from W18	[O02 KC1; O03 §08]
Drills	Rollback (C12), restore with `verify` on the restored sample, break-glass	Runbooks	Quarterly	[E01 §06; O05]

Three rules apply across all classes:

- **Mutation proof.** "A test is not trusted green until it has been shown to fail" [IM §15.6]. Every gate test, conformance class and reconciliation query is run once against a deliberately broken build, and the failing run is kept as evidence. The Build Book states the same thing for evals: "An eval that has never failed is an eval that is not testing anything" [EBB EB-12 §4].

- **Negative fixtures carry the weight.** EV2's cut-tail case and EV7's malformed grants exist "for exactly that reason" [EBB EB-12 §4]. E12's negative vectors are "the ones that matter" [E12 §07].
- **Determinism is tested, not assumed.** A stage that acquires "a clock, a random value or an undeclared read fails the same night" [E02 C4].

21.9 Release process

21.9.1 Versions, notice and deployment

Releases follow O06 [O06 §02–§04]. Additive changes ride the train with a release note. A new refusal code needs 30 days' internal notice and 60 days' external, with the operator action published first. A new KernelPort major needs 90 days internally with two releases in parallel, and 12 months externally. A change to a rung's order or presence needs 180 days internally, 12 months externally, and "a written argument for why the old order was wrong". An emergency security change ships at deploy, with notification inside 30 minutes. "A deprecated version keeps answering, unchanged, for the whole window" [O06 §04]. The `doors` → `channels` rename is breaking and "must land before L1 GA" [O06 §03]. That is why the schema major is a 30 November 2026 gate.

Deployment per residency domain runs in order: grants check against the target environment, golden write, then traffic. A failed golden write rolls back automatically [EBB EB-09 §9]. We recommend a canary on the reference tenant and one internal tenant before rung1's design-partner tenants, with automatic rollback on any non-zero in the nightly integrity join or any platform-refusal rate above baseline (**build recommendation**; paging triggers from [ETS §10]).

21.9.2 The seal release protocol (L2)

From Phase 3 a second, stricter release path governs anything that changes what a record carries. It is the nine-step protocol. Freeze the text and vectors. Implement twice from the text with no shared channel. Diff byte for byte. Fix every disagreement in the text first. Re-implement. Write the verifier from the same text. Run all positives and negatives. Publish the text, vectors and verifier source together. "Only then may a seal leave the building" [EBB EB-05 §8; E12 §08]. Publishing a verifier release is itself a governed write with dual control [E15 §07 step 4]. "Step 9 is the one a commercial deadline will attack. It does not move" [EBB EB-05 §8].

21.10 Agent-assisted development

21.10.1 The operating principle

"Generation is free and review is not" [EBB EB-13 §1]. The group's build material is written for a team that uses Claude Code heavily, and it is candid about the cost: "Assistance widens the gap between generation and review rather than closing it. A team that generates four times faster and reviews at the same rate has a larger queue, not a faster programme" [EBB EB-13 §4]. stack1's stack document quantifies the pattern from 2026 telemetry. Median review time rose 442% against a 34% rise in task throughput, and only a third of AI-assisted changes merged within 30 days [STS §03]. The harness is therefore built "for review throughput rather than for generation speed" [EBB EB-09 §8; ECB ch.01].

21.10.2 How the kernel team uses agents

Practice	Rule	Source
Zones	"Free vibe" for scoreboards, docs and throwaway spikes; "guided vibe" for implementations, vector tooling, CI scripts and DDL grant tests; "no vibe" for `/spec`, published vectors, the verifier, `.claude/` and every CLAUDE.md — two humans, no AI-only merge	[EAP §05]
Where agents multiply	Connectors, fixtures, vectors, the second implementation from text, export mappings, runbooks: high. Verifier profiles and declaration drafting: medium	[EBB EB-13 §2]
Where they do not	Deciding what a term means; whether an action is in fact irreversible; certifying a declaration; anything a named person signs	[EBB EB-13 §3, §9]

Practice	Rule	Source
Harness	Seven CLAUDE.md files with a "what never to propose" list; nine subagents (`spec-reader`, `vector-differ`, `import-auditor`, `declaration-critic`, `refusal-tracer`, `bundle-verifier`, `grant-checker`, `eval-runner`, `doc-crosschecker`); sixteen commands; six hooks; twelve skills	[EBB EB-09 §3–§7]
Hooks over sentences	"If a rule matters, it exits with code 2 or fails a CI job — it is not a sentence"	[ECB cover; EAP §05 practice 5]
Unit size	One unit, one session; diffs near 200 lines; anything larger than one context window or one 15-minute review is split	[EAP §04–§05; ECB ch.11]
Night shift	Headless diffs across all vectors, verifier runs and doc cross-checks; draft PRs only; hard token cap; "never merges; never edits `/spec`"	[ERD §4.8; EAP §07]
Isolation	The second implementer's agent "must never open the first implementation — not even 'to check'"	[EAP §05 practice 7]

The Build Book's list of what the harness must never do bounds the whole approach. It never certifies, never decides a specification's meaning, never makes a gate decision, never writes the judgement in a `not_measurable`, and never decides to ship a seal [EBB EB-13 §9]. These are E-series invariants in another form. A model may produce a write intent but never an allow [E02 I1], and the same holds for the build: an agent may produce a change but never a certification.

21.10.3 What to keep from the Agenthon

The elan1 Agenthon (17–21 September 2026) was planned to produce two seal implementations that "agree byte for byte on every drafted vector" and a verifier that passes a 4,096-seal bundle on an outside guest's offline laptop [ETR cover; EAP §07]. Three facts decide what carries into the L1 build:

1. **Its scope was L2.** Seal signing, Merkle anchoring and the offline verifier are excluded from L1 [E01 §03, §07], and the IM limits E12 work in November 2026 to specification [IM §16.2.2].
2. **Its first target is retired.** Day 1's exit was "A reproduces TV-001 at 311 octets with the known digest" [ERD §4.7; EAP §05]. E12 retires that figure: "Under the v3.0 table above, TV-001 is 329 octets and the v2.0 digest does not apply to it … Neither may be quoted" [E12 §07, decision 3].
3. **Its method is the right method.** Isolation, text-first resolution, a night shift that never merges, and daily harvest are exactly what E12 Flow 3 and the L1 CI in §21.7 need.

We recommend keeping the harness, the invariant scripts, the hooks, the unit-spec template and the working practices. Any preimage, digest or vector computed against the 311-octet table should be archived and not reused. Any seal or verifier code should be kept as reference material for Phase 3, and its authors are disqualified from being the "second implementer" of the implementation they have already read (**build recommendation**, applying [E12 §08 failure paths]).

21.11 Engineering metrics

Metrics are computed from data and never typed [ETS §10; ECB ch.11]. They fall into three groups.

Group	Metric	Target	Source
The zeros	Writes committed without a chained record	0	[E01 §06 item 3; EBB EB-12 §7]
	Records mutated outside the kernel writer	0	[E01 §06 item 4]
	Dependencies in the integrity checker (L1) and verifier (L2)	0	[EBB EB-12 §7]
	Meter events without a record; refusals with non-zero quantity	0	[E02 C10]
Kernel health	Governed write p50 / p99, per rung	≤ 120 ms / ≤ 450 ms (**MEASURE THIS**)	[O02 KC1; O03 §08]
	Outbox-to-chain lag p99	≤ 2 s	[ETS §13 #7]
	Meter lag p99	≤ 30 s	[O03 §08]

Group	Metric	Target	Source
	Refusal rate, beside cost per governed action	Published, never targeted downward	[O07 §06]
	Platform-refusal rate	Pages above baseline	[ETS §10]
	Days from a recorded write to offline verification by someone who does not trust us (L2)	Shrinking	[EBB EB-12 §7]
Build system	Median review wait; p75 PR size	< 30 min; ≈ 200 lines	[ECB ch.11]
	Open specification questions at day end	≤ 8, or generation pauses	[ERD §4.8]
	Declarations in certification concurrently; certifiers-in-training	≤ 3; rising	[EBB EB-13 §5, §7]
	Harness changes per day; repeat failures of the same kind	≥ 1; falling	[ECB ch.11]
	Cost per solved task	Published weekly	[ECB ch.11]
	Mutation-proved share of gate tests	100%	[IM §15.6]
	Kernel engineering headcount versus pack engineering headcount	Pack overtakes kernel during FY2029-30	[IM §15.4]

The last row is a structural check, not a productivity measure. If kernel engineering is still growing faster than pack engineering in 2029, the kernel is being reopened, which the architecture exists to prevent [IM §15.4].

21.12 Open questions the plan depends on

#	Question	Owner	Due	Source
1	Synchronous metering through rung 9 (recommended: yes)	elan1 lead	31 Oct 2026	[E02 Q1]
2	Who owns the reference tenant and its adversarial declaration	Founder	31 Oct 2026	[E02 Q5]
3	Can ERPNext hosted mode accept only kernel-issued tokens	elan1 lead, rung1 lead	31 Oct 2026	[IM §16.2.1; O02 elan1 KC4]
4	Where the proof record physically lives (recommended: elan1-owned Postgres)	elan1 lead, stack1 lead	31 Oct 2026	[S03 Q2]
5	F14 salting scheme that survives the L2 retrofit	elan1 lead	30 Nov 2026	[E02 Q2]
6	Which team builds app three for Gate 2, and which entity owns the surface compiler	Founder	30 Jun 2027	[O09 Q4]
7	Which design partner hands a bundle to their own auditor for Gate 1	rung1 lead	30 Sep 2027	[O09 Q3]
8	Which TSA pair, and does an Indian-resident RFC 3161 authority meeting DPDP residency exist	Founder with kernel lead	31 Mar 2027	[O09 Q1; IM §16.2.6]

22. Operations, reliability and incident response

This chapter covers how the elan1 kernel is run once somebody outside the kernel team depends on it: what the service levels are and how they are measured, what a consumer sees when the kernel cannot be reached, what is observed and what must never be, who is woken and for what, which incident classes have runbooks, how a security incident reaches CERT-In inside six hours, what is backed up and what is deliberately never backed up, how capacity is managed, and how design partners and third parties are supported. The primary elan1 sources are E09 (L1 GA readiness and third-party onboarding), E02 and E04 (the fail-closed contract and the refusal register), E16 and E17 (key and timestamp-authority incidents) and the Build Book's operations sheet [EBB EB-11]. O05 supplies the group security baseline, O11 the delegation of incident command, and the stack1 runbook set and GA checklist [S09; S13] supply the runtime underneath. The rung1 support runbook [R12] shows the first consumer's side of each failure.

> **Nothing here has run.** As at 23 September 2026 elan1 has no external customer, no production tenant and no governed action committed or refused [IM §4.9; IM App. E.1]. Every service level in this chapter is a **target**, and those marked MEASURE THIS were produced by decomposing the architecture rather than by measuring it [EBB EB-11 §1; O03 §08]. The recovery point and recovery time objectives have not been set [O05 §6; O05 OQ1]. The named CERT-In responder has not been appointed [O05 OQ2]. Where this chapter proposes a number, a ladder or a schema that the sources do not fix, it is marked **(build recommendation)**.

22.1 Operating principles

Six rules shape everything below. Each is stated in a source, and each is stricter than the default in most platforms.

> **Fail closed.** Every stage's default is refuse. An unavailable dependency is a platform-class refusal, never a silent allow [E02 I3]. Every control in the group baseline is written so that when it is unavailable the write does not happen: "a control that degrades to 'allow' is not a control" [O05 §01].

1. **Evidence integrity pages; latency does not.** "A seal worker that is behind is an incident. A write API at p99 is a ticket." Latency recovers; a gap in a chain does not [EBB EB-11 §2]. This inverts the usual ordering, where availability is the top-line concern.
2. **Every refusal the platform causes says so.** Ten of the forty-three registered refusal codes are PLATFORM class. They are not billable and each one pages. A governance layer that reports its own failure as a policy refusal tells the customer their controls worked when the infrastructure fell over [E02 §06; E05 §07].
3. **There is no administrative path.** No fast path, no batch variant, no admin mode, no decision cache [E02 I5]. Break-glass is a declared write type with `dual_control: true`, recorded in the operator's own chain and visible to the tenant [E02 decision 9; EBB RB12]. No operational procedure in this chapter may require a step that bypasses the nine rungs.
4. **Every runbook ends in a verification.** The last numbered step is a command with an expected output, and an incident is closed only when it returns that output: "an incident cannot be closed on a theory" [S09 §01, §11; R12 decision 7].
5. **Rollback is never the wrong first move.** The rung1 on-call may roll back without asking [R12 decision 6]; the stack1 on-call decides on rollback within 30 minutes of a P1 without consumer sign-off [S02 §5]. For the kernel, the equivalent first move is the tenant stop at rung 0b, which halts writes without losing any (§22.3).
6. **Nothing is closed by relabelling.** A record written under one mode, key or level keeps it forever. A revoked key's records still verify [E16 §07]; a promoted write type's earlier records stay Attested advisory [E09 §09]; a missing record is documented by a gap record, never back-filled [EBB RB3].

Figure 32 — Operations: SLOs, failure behaviour and incidents

DESIGNED — SLOs ARE TARGETS

Evidence integrity pages; latency does not. Every dependency failure refuses rather than allows; every incident has a runbook.

Nine SLOs (EBB E11, O03)

governed write p50	≤ 120 ms	MEASURE THIS
governed write p99	≤ 450 ms	MEASURE THIS
seal, rung 8, p50	≤ 40 ms	MEASURE THIS
outbox → seal lag p99	≤ 2 s	estimate
anchor interval (L2)	≤ 15 min	published SLO
bundle build, 10k seals	≤ 60 s	estimate
offline verify, 10k seals	≤ 5 s	estimate
writes without a seal	0	invariant
records mutated outside	0	invariant

Also: stack1 prod availability 99.5% a month per contract; meter lag p99 ≤ 30 s (S02 §3). None is quoted to a customer until measured under load.

Observability and paging

OpenTelemetry span per rung → fusion.trace/v1
status page per component: submission, gates, writer, chain, meter, console; a missed nightly integrity job flips chain itself

PAGES
outbox → seal lag over SLO · any non-zero invariant · TSA down two intervals · platform-class refusals above baseline · an eval failure

TICKETS
write latency · bundle build · Studio · connectors. One rotation, two people.

Kernel unreachable: fail closed

consumer: no degraded mode, no cached allow, no local gate (R12 RB-4, O03)
runs parked durably with the same idempotency key; SEAL_PENDING first
user: "We cannot check this change right now. Nothing was changed."

INSIDE THE KERNEL (IM §6.10, E04, E16, E17)
registry down → refuse at write door
policy down → refuse at rung 4, page
target down → refuse after sandbox
chain append fails → outbox retries, no gap
KMS down (L2) → domain refuses, no stale key
TSA down (L2) → writes go on, single_anchor

INCIDENT CLASS	DETECTED BY	RUNBOOK AND FIRST ACTIONS	SEVERITY
Proof-chain gap	nightly write ↔ seal join; CHAIN_HEAD_CONFLICT	RB3: never backfill a seal; write a sealed gap record stating the write_id, time and reason. Two chain writers stop the tenant.	P0
Seal worker behind	outbox → seal lag p99 over 2 s	RB1: confirm the lag, then decide whether to stop intake; stopping intake is the default, because a gap does not recover.	page
Meter / proof mismatch	nightly two-way reconciliation (C10)	The invoice for the period is stopped and we are paged; the kernel count wins over the credit ledger; a disputed line is held.	P2 → page
Key compromise (L2)	unaccounted KMS Sign call; exposed credential	RB6 / E16 Flow 2: fix T_suspect; stop signing, keep writing at L1; revoke; tenants told in writing within 24 h; anchors as witness.	P0
TSA outage (L2)	token timeout or malformed token	RB2 / E17 Flow 2: single_anchor; page at 2 intervals; banner at 4; late batches anchored oldest first, never merged. No write refused.	degraded
Declaration mis-publish	certifier or tenant report	RB10: a declaration version must be revoked; RB7 stops a tenant. Loosening a gate needs a new version plus an ADR (IM §6.11).	not stated
Isolation breach	K2 nightly attack; RLS test; customer report	O05 Flow A: tenant stop at rung 0b, proved by a refused attempt; revoke credentials; CERT-In responder assesses; DPDP check.	P0
Bundle fails at customer	customer or auditor report (L2)	RB9: reproduce air-gapped with our signed verifier first; outdated verifier, inconsistent bundle, or a spec defect disclosed to all.	P0 if real

Severity, CERT-In and notification

P0: compromise, data exposure, tamper evidence or fleet outage: response 15 min 24×7, contained in 2 h (R12)
CERT-In: the clock starts on noticing; the named responder files within 6 h; if reportability is unclear, file
Tenant update within 72 h; written note for every P0 and P1 to affected tenants within 5 working days (E09)

Backup, restore and DR (O05 §6, S09 RB-07, S17 §9)

Point-in-time recovery; nightly encrypted backups kept 30 days
Quarterly restore drill into a clean environment, timed; it restores the evidence store and re-verifies a sample
No second region, so no cross-region failover: a region loss is a restore in place from in-region backups
RPO and RTO not yet set (open, due 15 Oct 2026)

kernel / evidence · runtime / isolation · external dependency · customer-raised · P0 / fail-closed · people and process

Source: EBB E11 · S09 RB-01–RB-09 · R12 §3, RB-4 · E04 rungs 7–8 · E09 §02 · E16 Flow 2 · E17 Flow 2 · O05 §6, Flow A · O03 §03 · IM §6.10 · elan1 Technology Memorandum · Sept 2026

Figure 32 — Operations for the elan1 kernel: the service levels and invariants on the operations surface, the nightly integrity and reconciliation jobs, what pages and what raises a ticket, the incident flow from notice to verified close, fail-closed behaviour when the kernel or a dependency is unreachable (Mode A stops, Mode B relies on the caller), and single-region backup with quarterly restore drills that re-verify the proof chain.

22.2 Service level objectives and indicators

The sources give three kinds of number, and they must not be confused on a dashboard or in a contract: **MEASURE THIS** targets, which are design estimates to be replaced by measurement at Gate 1; **published SLOs**, which are promises the customer prices risk against; and **invariants**, which are correctness claims shown as a count that must read zero [EBB EB-11 §1; O03 §08]. A correctness claim has no error budget, "because a correctness claim with a budget is one we have already decided to break" [S02 §3].

22.2.1 The kernel SLO table

#	Indicator (SLI)	Target	Class	Level	Source
1	Governed write, kernel-side, p50	≤ 120 ms	MEASURE THIS	L1	O03 §08; EBB EB-11
2	Governed write, kernel-side, p99	≤ 450 ms	MEASURE THIS	L1	O03 §08; IM §18.2
3	Kernel rungs 0b–9, sum of rung budgets, p50	≈ 64 ms	design target	L1	E04 §02

#	Indicator (SLI)	Target	Class	Level	Source
4	Sandbox + live applies (rung1 hosted ERPNext), p95	≤ 1,200 ms	design target	L1	E04 §02
5	Governed write end to end incl. target, p95	≤ 2 s	MEASURE THIS	L1	E04 §02
6	Proof record write (rung 8), p50	≤ 40 ms	MEASURE THIS	L1	O03 §08; EBB EB-11
7	Outbox-to-record lag, p99	≤ 2 s	estimate [E]	L1	EBB EB-11 §1
8	Mode B contract call, p95, excluding target	≤ 40 ms	MEASURE THIS	L1	O03 §08
9	Meter lag, p99	≤ 30 s	MEASURE THIS	L1	O03 §07–08
10	Sandbox key issuance to usable tenant	< 60 s; p95 published	readiness	L1 GA	E09 §02 row 2
11	Committed writes without a chained record	0	invariant	L1	E01 §06 #3; EBB EB-11
12	Records mutated outside the kernel writer	0	invariant	L1	E01 §06 #4
13	Meter events without a write; refusals with quantity ≠ 0	0	invariant	L1	E01 §06 #7; E02 C10
14	Anchor interval (unanchored age of a record)	≤ 15 min, per tier	published SLO	L2	E17 §04; E14 §07 #4
15	Bundle build, 10,000 records	≤ 60 s	estimate [E]	L2	EBB EB-11 §1
16	Offline verification, 10,000 records	≤ 5 s	estimate [E]	L2	EBB EB-11 §1
17	KMS `Sign` calls without a record, and the reverse	0	invariant	L2	E16 §08

The latency model is not reconciled. The flow book puts the proof rung at 12 ms of an ≈ 64 ms kernel total [E04 §02]; the cost model assumes ≈ 91 ms, of which ≈ 40 ms is rung 8 [E10 §07; EBB EB-11 §6]. Neither is measured [IM §6.10]. The envelope in rows 1–2 is the one that matters, because it is a kill criterion at ten design partners' load, measured [IM §18.2]. The target system's own time sits outside it: the 18 ms rung-7 budget is the kernel transaction only [E04 §02; E04 decision 3], and whether third parties need a different promise from rung1's is open [E05 OQ6].

There is no kernel availability target in any source. stack1 commits to 99.5% monthly availability per `fusion.*` contract, measured by a per-minute synthetic probe from inside the region and from one external vantage point, with a minute counted unavailable if the probe fails or real-traffic errors exceed 5% [S02 §3]. rung1 targets 99.9% console availability [R12 §8]. The kernel sits between them and has no number of its own. **(Build recommendation)** Measure kernel availability exactly as stack1 does — per residency domain, per minute, by a synthetic governed write against the reference tenant that must return a sealed refusal with the expected code — and publish it as an indicator without a promise until the first quarter of measured data exists. A probe that checks only `/ready` is not acceptable: the stack register requires the external check to exercise "the golden write, not just a 200" [ETS §10].

22.2.2 How the indicators are produced

The telemetry standard is OpenTelemetry, with each rung instrumented as a span so that the rung budgets in E04 can be replaced with readings at Gate 1; metrics, logs and traces go to a Grafana stack (Prometheus/Mimir, Loki, Tempo) that must run in-region because the telemetry is tenant-adjacent [ETS §10]. SLOs are tracked as Grafana SLO or Sloth-generated Prometheus rules [ETS §10]. Invariants 11–13 are not computed from spans at all. They are computed by the nightly two-way reconciliation from the database, and the report is published on the operations surface as "the evidence that seal-or-nothing holds in production rather than merely in the grant catalogue" [EBB EB-11 §8].

A sketch of the rule set **(build recommendation)**, in Sloth's format, for rows 1, 2 and 11:

```
version: prometheus/v1
service: elan1-kernel
labels: { owner: kernel, domain: in }
slos:
  - name: governed-write-p99
    objective: 99.0                # 99% of writes under 450 ms; MEASURE THIS until Gate 1
    sli:
      events:
        error_query: sum(rate(elan1_assert_duration_seconds_bucket{le="+Inf"}[{{.window}}]))
          - sum(rate(elan1_assert_duration_seconds_bucket{le="0.45"}[{{.window}}]))
        total_query: sum(rate(elan1_assert_duration_seconds_count[{{.window}}]))
```

```
    alerting: { page_alert: { disable: true }, ticket_alert: { labels: { route: ticket } } }
  # Invariants are not SLOs: no objective, no budget. Any non-zero is a page.
rules_extra:
  - alert: WritesWithoutRecord
    expr: max(elan1_reconcile_writes_without_record) > 0
    labels: { severity: P0, route: page }
```

The latency SLO routes to a ticket and the invariant routes to a page. That is rule 1 of §22.1 written as configuration.

22.2.3 Error budgets and what spending one means

For the MEASURE THIS rows the error budget exists only after Gate 1 replaces the estimate with a measurement. Until then a breach is a data point, not a failure. From Gate 1, **(build recommendation)** apply the rung1 rule that an SLO missed for three consecutive weeks stops being an operations item and becomes a release blocker, with a named owner before the meeting ends [R12 §9]. The invariant rows have no budget and cannot acquire one. The anchor interval (row 14) is different again: it is published per tier, customers "price their own risk off it", and it is never a per-customer negotiation [EBB EB-11 §1; E17 decision 3]. The anchor lag metric must be published from measurement, not estimated, before L2 is done [E14 §07 #4].

22.3 What "kernel unreachable" means for a consumer

The kernel's failure behaviour is defined twice: once inside the kernel, as the platform-class codes each rung returns when a dependency is down, and once at the consumer boundary, where Mode A and Mode B behave differently because only one of them holds the credential [E02 §06; O03 §02].

22.3.1 Inside the kernel: the fail-closed matrix

Dependency down	Rung	Code	Behaviour	Retry	Pages
Tenant stopped (deliberate)	0b	`POLICY.TENANT_STOPPED`	Refuse; no chain entry — the only refusal without one	No	No
Declaration registry	2	`PLATFORM.REGISTRY_UNAVAILABLE`	Refuse at the write door	Same `intent_id`	Yes
Rule engine	4	`PLATFORM.POLICY_UNAVAILABLE`	Refuse; never a default allow	Same `intent_id`	Yes
Target system, after sandbox apply	7	`PLATFORM.TARGET_UNREACHABLE`	Nothing reaches live; sandbox discarded	Safe	Yes
Credential broker	7	`PLATFORM.TOKEN_MINT_FAILED`	Refuse	Same key	Yes
Single-writer lock lost mid-apply	7	`PLATFORM.WRITER_LOCK_LOST`	Snapshot restored	Same key	Yes
Chain append	8	`PLATFORM.LOG_APPEND_FAILED`	Rung-7 transaction already committed; outbox row survives; chain writer retries in order	Automatic	Yes
Two chain writers for one tenant	8	`PLATFORM.CHAIN_HEAD_CONFLICT`	**P0; tenant stopped**	No	Yes
KMS at a key cut instant (L2)	8	platform class	Every write in the domain refuses; nothing signed with a stale key	Yes	Yes
One or both TSAs (L2)	—	none	**Never refuses a write**; batch marked `single_anchor` or left unanchored	Automatic	At 2 intervals

Sources: E02 §06 register; E04 rungs 0b, 7, 8; E02 decision 5; E16 §06 failure paths; E17 §05–06.

Conformance test C5 makes the registry, rule engine, autonomy table and target unavailable in turn and requires every case to refuse [E02 C5]. Anchoring is deliberately **not** on the write path, or "an authority's outage stopped a customer's business" [E17 §05]. An unavailable KMS at L2 stops writes, because "a platform that stops sealing is recoverable; a platform that seals badly is not" [E16 §06; IM §6.10]. The same source requires the opposite behaviour after a suspected key compromise — the domain stops signing and keeps writing unsigned at `assurance_level: 1` [E16 §07 step 2, decision

6]. The two are reconcilable (an outage is involuntary and unexplained; a compromise response is a deliberate, recorded downgrade with a declared unsigned interval) but the sources do not reconcile them, and the chain writer's behaviour must be specified for both before L2 code is written. It is recorded in the conflicts block.

22.3.2 At the consumer boundary: Mode A against Mode B

	Mode A — SDK in-process (Enforced)	Mode B — MCP/HTTP (Attested advisory)
Who holds the target credential	The kernel, through the per-tenant credential broker	The caller, unless the target accepts only kernel-issued tokens
Kernel unreachable	The write does not happen. No write is attempted	The write does not happen **only if the caller honours the refusal**; nothing physically stops it
`assert` timeout	Treated as refuse	Treated as refuse by contract; "we cannot make it"
Outcome never reported	n/a — the kernel acted	Reconciled as `acted_unattested` and flagged
What the user is told	"We cannot check this change right now. Nothing was changed."	Consumer's own copy; must not say "blocked" or "prevented"

Sources: O03 §02–04; E01 §05.

In Mode A the kernel's unavailability is a property of the architecture: the credential is behind it, so an outage cannot produce an ungoverned write. In Mode B it is a property of the caller's code, and elan1 can do three things about it and no more: specify the behaviour, test it in the published conformance suite, and detect the violation afterwards. A caller that proceeds after a refusal or an outage produces "an unmatched write: an act with no allow behind it", which reconciliation catches after the fact — "exactly the limit of advisory mode" [O03 §04].

A reference Mode B client wrapper that fails closed **(build recommendation)**, following the E05 retry rules (retry 5xx and 429 with exponential backoff and jitter, at most five attempts, always with the same idempotency key; never retry a refusal) [E05 §08]:

```
class KernelUnavailable(Exception):
    """Raised when no decision was obtained. The caller MUST NOT write."""

def governed_write(client, intent, *, deadline_s=2.0):
    key = intent["intent_id"]                       # idempotency key, fixed before the call
    for attempt in range(5):
        try:
            r = client.post("/v1/act.write", json=intent, timeout=deadline_s,
                            headers={"Idempotency-Key": key})
        except (TimeoutError, ConnectionError):
            backoff_with_jitter(attempt); continue   # timeout on assert == refuse
        if r.status_code == 200:
            out = r.json()["outcome"]                # committed | refused | escalated
            return out                               # refused/escalated: never retried
        if r.status_code in (429, 500, 502, 503, 504):
            backoff_with_jitter(attempt, r.headers.get("Retry-After")); continue
        raise ValueError(f"rejected {r.status_code}")  # 4xx: fix the request, do not retry
    raise KernelUnavailable(key)                     # fail closed: no write, no local gate
```

The wrapper has no cached allow, no local fallback gate and no "degraded mode" flag, because each of those is a second definition of "allowed" [E02 I5; O01 DR-001]. The conformance suite should carry a Mode B test that kills the kernel mid-run and asserts that the caller's target received no write **(build recommendation)**; C5 tests the kernel's side of the same property [E02 C5].

22.3.3 The first consumer's view

rung1's runbook for an unreachable kernel is the model for any consumer's [R12 RB-4]. It is a P1, and a P0 if any run proceeded without a decision. rung1 confirms it has failed closed ("no degraded mode, no cached allow and no local gate standing in"); clears first any run already applied to a live site whose record is pending (`SEAL_PENDING`); confirms parked Temporal workflows hold the same idempotency key so that they resume at the same `assert` without re-planning or re-

charging; diagnoses "with the elan1 on-call, not around them"; and tells tenants "We cannot apply changes right now. Your site has not been touched." It verifies with `verify` over the affected chains (no sequence gap), zero runs pending, and promotions in the window equal to proof records in the window [R12 RB-4].

> **Three names for one state.** A write that reached the target but has no chained record yet is `SEAL_PENDING` in rung1 [R12 RB-4], `acted_unproven` in the group integration contract, which alarms and is "never silently completed" [O03 §03], and simply an unconsumed outbox row in the kernel contract, where the rung-7 transaction has committed and the chain writer retries in order [E02 §05]. The kernel design (outbox in the same transaction as the record version) makes the state transient rather than terminal, which is why E02 can say "no lost write, no gap". Where an `acted_unproven` action finally goes — retry queue or manual reconciliation — is open [O03 OQ3, due 15 Oct 2026]. This memorandum uses the kernel's term and treats a pending record older than the outbox-to-record SLO (row 7) as the paging condition.

22.3.4 The tenant stop as the kernel's first containment move

The stop set at rung 0b is the kernel's equivalent of rollback-first. It covers the tenant kill switch, revocation, blacklist and regulator directive [O01 §02]. Setting a stop is itself a governed write with a record; the refusals it then produces are the only refusals with no chain entry, because a stopped tenant writes nothing [E04 rung 0b; E04 decision 5]. Every security runbook below uses it as its containment step, and the baseline requires the stop to be **verified, not assumed**, by attempting a write and expecting `POLICY.TENANT_STOPPED` [O05 Flow A step 3]. Users see "Writes are paused for this organisation" [O05 Flow A; O03 §03]. The stop holds writes, not reads: `verify` and exports continue, so forensics can proceed while the tenant is frozen.

22.4 Observability without leaking tenant data

The kernel sits in the path of writes to other people's books. Its telemetry is therefore the most likely place for tenant data to leak by accident — into a log line, a span attribute, a metric label or an error tracker — and the rules below treat observability as part of the data-protection boundary rather than beside it.

22.4.1 The rules

- **No PII in application logs.** A CI lint fails on known field names; a sampling review runs monthly [O05 §5].
- **No tenant payloads in logs or traces.** The telemetry stack runs in-region because it is tenant-adjacent [ETS §10]. The Mumbai residency rule covers "object store, queues, backups, evidence store and logs" alike, with a CI check failing any resource declared outside the region allow-list [O05 §1].
- **Refusals are evidence, not noise.** Every refusal is logged with its code, gate, policy version and `as_of` [O05 §5].
- **180-day log retention inside India**, on append-only storage, with system clocks synchronised to NPL or NIC NTP and verified by a monitoring check [O05 §5; CERT-In Directions 2022 as cited in O05 §01].
- **Anything that can see tenant data is a sub-processor.** Every observability vendor is on the published sub-processor list with its region, and a component that cannot be named on the list does not enter the build [O05 §8].
- **The operations surface carries no tenant data.** `/v1/ops` is eight endpoints: health per service and zone; the SLOs current and trailing; the three numbers that must read zero; outbox-to-record lag per residency domain; anchor batch status and last successful TSA round trip; the nightly two-way reconciliation report; eval run history; and the grant-catalogue diff against the invariants [EBB EB-07 §8].

22.4.2 What a span may carry

The sources fix the principle but not the attribute list. **(Build recommendation)** Enforce an allow-list at the OpenTelemetry Collector, not in application code, so that a developer adding an attribute cannot widen what leaves the process:

```
processors:
  attributes/elan1_allowlist:
    actions:
```

```
      # Identifiers that are already in the proof record and carry no payload
      - { key: elan1.tenant_ref,  action: hash }    # salted per domain; raw uuid never leaves
      - { key: elan1.rung,        action: upsert }     # 0b..9
      - { key: elan1.write_type,  action: upsert }     # slug.verb from the declaration
      - { key: elan1.decl_version, action: upsert }
      - { key: elan1.policy_version, action: upsert }
      - { key: elan1.outcome,     action: upsert }     # committed | refused | escalated
      - { key: elan1.refusal_code, action: upsert }    # closed register, safe by construction
      - { key: elan1.actor_kind,  action: upsert }     # human | agent | system
  filter/drop_everything_else:
    spans:
      exclude:
        match_type: regexp
        attributes: [ { key: "^(?!elan1\\.|service\\.|http\\.status).*" } ]
# Never exported: payload, subject, record_id, actor_id, on_behalf_of, counterparty,
# approver comments, target-system responses, idempotency keys, token material.
```

The refusal code, rung, write type and versions are safe because they are enumerations or registry identifiers; they are also exactly the dimensions an operator needs to see which gate is refusing and at which policy version. Record and actor identifiers stay in the proof record, where access is governed, and never enter the observability pipeline. Metric labels follow the same list, with `tenant_ref` omitted from high-cardinality series and present only on per-tenant probe and reconciliation metrics.

22.4.3 The nightly jobs, and what watches them

Job	Checks	A failure means	Source
Chain integrity	`seq` contiguous from 1; each `prev_digest` matches; every digest recomputes	P0 — a broken link	E01 §02 #7; E02 §4 verify
Two-way reconciliation	Every committed write has one record; every record one write; every write one meter event; every meter event a write; every refusal quantity 0	First two: P0 (control C04). Meter lines: billing incident	EBB EB-11 §8; E01 §06 #3–4, #7
Refusal replay	Every reference-tenant refusal replays to the identical decision	A stage acquired a clock, random value or undeclared read (I8)	E01 §06 #5; E02 C4
KMS `Sign` reconciliation (L2)	Every `Sign` call has one record and every signed record one call	A `Sign` call with no record pages immediately and stops the domain	E16 §08
Anchor lag (L2)	Unanchored head and interval per tenant	Published SLO breach	E17 §04

A nightly job that silently does not run is worse than one that fails, so the write-to-record join pings a dead-man switch on success and silence pages [ETS §10]. Chain integrity is its own status-page component, flipped by a missed nightly job "without anyone pressing anything" [E09 §02 row 4]. Who owns that component when the nightly job is itself the thing that failed is still open [E09 OQ5].

22.5 Paging, on-call and escalation

22.5.1 What pages and what raises a ticket

Pages (wake someone)	Raises a ticket
Outbox-to-record lag over SLO	Write API latency, including p99
Any non-zero on the three numbers	Bundle build time (L2)
Anchor service unable to reach a TSA for two consecutive intervals (L2)	Studio availability
Platform-class refusal rate above baseline; every platform-class refusal	Connector failures
An eval failure	Meter-line divergence in the nightly report
Invoice reconciliation that cannot complete or shows a discrepancy	—

Sources: EBB EB-11 §2; E02 §06; E17 §06 step 3; E09 §07 step 1.

The last row of the left column comes from the onboarding flow book, which says a non-zero discrepancy at period close "stops the invoice and pages us" [E09 §07]. The Build Book classes meter-line divergence as "a billing incident and a ticket" [EBB EB-11 §8]. The two are compatible only if the ticket is the nightly finding and the page is the period-close block; this memorandum adopts that reading.

22.5.2 A severity ladder for the kernel

The sources name individual P0s but give the kernel no complete ladder. rung1 uses P0–P3 with the desk setting severity [R12 §3]; stack1 uses P1–P4 with the consumer setting it [S02 §7]. **(Build recommendation)** The kernel adopts rung1's four levels, because its consumers are ERP tenants and because the kernel's P0 class is about evidence rather than availability:

Sev	Definition for the kernel	First response	Contain / restore	Who is woken
P0	Evidence integrity or security: a broken chain link or `seq` gap; a committed write with no record; a record mutated outside the writer; `CHAIN_HEAD_CONFLICT`; a governed write that proceeded with no decision; suspected key compromise or an unaccounted KMS `Sign`; two TSA tokens disagreeing about a root; a tenant isolation breach	15 min, 24×7	Tenant or domain stopped inside the response time; CERT-In clock if reportable	Primary, secondary, elan1 lead, founder
P1	Kernel unavailable to one or more tenants (platform-class refusals above baseline); outbox-to-record lag over SLO; a declaration version resolving wrongly; an Enforced write type showing an unrecorded write; a period-close reconciliation block	15 min in hours; 30 min out of hours once a third on-call name exists (§22.5.3)	4 h	Primary
P2	Degraded: one TSA failing two intervals; latency SLO breached for over an hour; a nightly job late but not missed; sandbox provisioning over its published p95	2 business hours	1 business day	Primary, next business hour
P3	Questions, documentation defects, anything with a workaround	1 business day	5 business days	Nobody

The P0 row is assembled from source designations: a broken link [E01 §02 #7]; non-zero writes without a record [E01 §06 #3]; a `seq` gap and two chain writers [E02 §3 seal]; a key incident [E16 §07]; an unaccounted `Sign` call [E16 §08]; disagreeing tokens [E17 decision 2]; a run that proceeded without a decision [R12 RB-4]. The response times are rung1's [R12 §3]. Mapping to stack1 when the runtime is the cause follows the rule the rung1 memorandum proposes — kernel P0 to stack1 P1 with the security flag, P1 to P1, P2 to P2, P3 to P3/P4 — with both identifiers recorded on the incident [RTM Ch.19 §19.4; S02 §7].

Two escalation rules are adopted from the consumer and runtime sides. A P1 open at four hours becomes a P0 for notification purposes and the founder is told [R12 §3]. A consumer may escalate a level after the stated response time has elapsed without asking permission [S02 §6].

22.5.3 The rota, stated as it is

The Build Book specifies "one rotation, two people, follow-the-sun only after W8", because a rotation added before volume justifies it "produces oncall engineers who have never seen an incident" [EBB EB-11 §3]. E09 requires "a named human on a rota" and refuses any production applicant who will not name a production owner to receive incident notices: "give us a person, not a distribution list" [E09 §02 row 5, §08].

> **Two names is a rota that fails when one is on a flight.** stack1's GA requires 24×7 P1/P2 cover with at least three names and paging tested at 03:00 IST twice [S13 5.1, 5.4]; the kernel pages for more serious things with a smaller rota. With eleven people in the company in October 2026 [IM §15.2], this memorandum proposes that **L1 GA publishes no 24×7 P1 promise** until a third trained name exists, and that P0 cover is by named escalation to the elan1 lead and founder **(build recommendation)**. rung1 and stack1 ask the same staffing question [R12 OQ1; S09 OQ2].

Incident command is delegated to brand leads at Phase 2 close [O11 §08]; until then the founder is the third call. The kernel has its own on-call, and rung1 diagnoses kernel failures "with them, not around them" [R12 RB-4 step 4].

22.5.4 The incident lifecycle

The kernel uses the lifecycle both sibling runbooks share [S09 §11; R12 §2–4], with two kernel-specific additions in bold:

Stage	What happens	Clock
Notice	Alert, probe, nightly job, dead-man silence, or a report. The clock starts at noticing, not at confirming	—
Declare	Incident opened; timestamp recorded automatically; severity set against §22.5.2	Immediately
First response	A human acknowledges and names the runbook	Per §22.5.2
Contain	For anything touching credentials, keys or data, containment runs before diagnosis is complete. **The kernel's containment is the tenant or domain stop, verified by an attempted write**	Inside first response
Communicate	Status page component and affected production owners; cadence stated and held	P0/P1 every 30 min
Preserve	**Before any restart or repair: snapshot the chain head, the outbox, the writer lock table and the KMS audit window for the affected range**	Before repair
Verify	The runbook's last step returns its expected output	Before close
Close	Only after verification	—
Post-incident note	Written and published to affected tenants for every P0 and P1 within five working days; a P0 without a published note is itself a P1	5 working days

The five-working-day rule is the kernel's own [E09 §02 row 6]; rung1 (48 hours) [R12 §4] and stack1 (two working days) [S02 §7] are stricter, and where an incident spans brands the stricter clock applies **(build recommendation)**. The stack1 postmortem template is adopted as written [S09 §12], with one kernel field added: **the `seq` range affected per tenant and its state after recovery** (continuous, continuous with a declared gap record, or not checked).

22.6 Incident classes and runbooks

The Build Book names twelve kernel runbooks [EBB EB-11 §4]. The set below keeps its numbering, maps each to the v3 source that now governs it, and marks the level at which it becomes live.

RB	Runbook	Governing source	Live from
RB1	Record writer (outbox consumer) is behind	E02 §3 seal; EBB	L1
RB2	Anchor service cannot reach a TSA	E17 Flow 2	L2
RB3	A write committed without a record	E01 §06 #3; EBB	L1
RB4	A record was mutated outside the kernel	E01 §06 #4	L1
RB5	Key rotation, planned	E16 Flow 1	L2
RB6	Key compromise, unplanned	E16 Flow 2	L2
RB7	A tenant must be stopped	E04 rung 0b; O05 Flow A	L1
RB8	An erasure request arrives	O05 Flow B	L1 (salt destruction from L2)
RB9	A bundle fails verification at a customer	E15; EBB	L2
RB10	A declaration version must be withdrawn	E03; E09 Flow 2	L1
RB11	Residency migration rehearsal	O05 §1	before any second domain
RB12	Break-glass access to a record store	E02 decision 9; O05 §3	L1

Six incident classes are the ones a diligence reader should expect to see handled. Each is set out below as trigger, severity, containment, steps and verification. Two further classes — meter reconciliation at period close and an Enforced write type showing an unrecorded write — are folded into the classes they most resemble.

22.6.1 Proof-chain gap (RB1, RB3, RB4)

Trigger. Any of: the nightly verifier reports a broken link, a `seq` gap, a duplicate or a digest that does not recompute; the two-way reconciliation reports a committed write with no record or a record with no write; `PLATFORM.CHAIN_HEAD_CONFLICT` [E01 §02 #7; E02 §3; EBB EB-11 §8]. **Severity** P0. A writer that is merely behind (RB1) is P1 until the lag exceeds the outbox-to-record SLO, then P0 if the outbox row is lost rather than late.

Steps.

1. Confirm the outbox lag and decide whether to stop intake for the affected tenant or domain. "Stopping intake is the safe direction and it is the default"; the first instruction is never "scale it up" [EBB RB1].
2. Set the tenant stop at rung 0b and verify it with an attempted write [O05 Flow A].
3. Preserve: chain head, outbox contents, writer-lock history and the deployment log for the window (§22.5.4).
4. Run `verify` over the full range, not a sample, and record the first failing `seq`, field and reason, which the verify call returns [E02 §4].
5. Classify. A late outbox row (the rung-7 transaction committed, the record not yet appended) is recovered by the chain writer consuming it in order — no gap exists [E02 §05 failure paths]. A **lost** record, where the write committed and no outbox row survives, "should be impossible"; if it happens the transaction boundary is broken [EBB RB3].
6. For a lost record, **do not back-fill**. "A seal minted after the fact with a synthesised `sealed_at` is a forgery, even a well-intentioned one." Write a **gap record**: a governed write of a declared type stating that a write with this `write_id` committed without a record, at this time, for this reason [EBB RB3]. For a mutated record (RB4), the chain already shows the break; append a governed incident record naming the affected `seq` and the forensic finding, and never rewrite the chain.
7. Clear the stop only after step 8 passes.
8. **Verify.** `verify` over the tenant's full range returns PASS with the gap record present, or "break at N" documented by an incident record; reconciliation returns zero on both write/record lines; `promotions in the window = records in the window` on the consumer side [R12 RB-4 step 8].

"A documented gap is recoverable. An undocumented one is indistinguishable from tampering, forever" [EBB RB3]. The gap write type must exist in the kernel's own operator declaration **before** L1 GA, because declaring it during the incident would be a declaration change under pressure **(build recommendation)**.

22.6.2 Meter/proof mismatch

Trigger. The nightly reconciliation reports a meter event with no write, a write with no meter event, or a refusal whose meter event has non-zero quantity; or the period-close reconciliation cannot complete [EBB EB-11 §8; E09 §07]. **Severity** P2 as a nightly finding; P1 when it blocks a period close. The quantity-zero line should be unreachable because a `CHECK` constraint rejects the insert [E04 rung 9; E05 §06]; a non-zero there means the constraint was dropped or bypassed, which is a P0 under §22.6.1.

Steps. Hold the invoice for the affected period; no invoice is issued over an unreconciled period [E09 decision 4]. Tell the consumer the period is open and why [E09 §07 failure paths]. Identify the direction of the mismatch by `write_id` and `seq`. The proof chain is authoritative: where two counts differ, "the kernel's count is the count and the other is a report defect" [O11 §06]. Repair the meter by appending corrective meter events that reference existing `write_id`s — the meter is append-only, so a repair is an insertion, never an update [E01 §02 #9]. Where a customer disputes a line, hold that `seq` range and let the rest of the invoice proceed [E09 §07 step 4]. **Verify:** reconciliation returns zero discrepancies over the period, with scope and count printed, and every invoice line traces to a record [E09 §07].

22.6.3 Key compromise (RB6, L2)

Trigger. Any of: the KMS audit shows a `Sign` call the platform cannot account for; a credential with `Sign` permission is exposed; an insider event; a vendor advisory. Suspicion is enough [E16 §07]. **Severity** P0.

Steps [E16 Flow 2]. Declare the incident and record `T_suspect`, "always the earliest defensible time, never the most convenient one". Stop signing in the affected domain; the chain writer continues to write and chain records unsigned at `assurance_level: 1`, and the Assurance door says so. Generate and register a successor under dual control; set the predecessor's `valid_to` to `T_suspect`, leaving a declared unsigned interval rather than a fictitious continuous one. Mark the predecessor `revoked` with reason and instant, append-only. Notify every affected tenant in writing **within 24 hours**, using fixed template wording, naming the instant, the record range signed by the revoked key, and what remains true. Use the anchors as the recovery instrument: every record signed and anchored before `T_suspect` has an independent witness. Re-enable signing under the successor. **Verify:** the key register shows abutting-or-gapped windows with no overlap; the boundary and counts are rendered; the `Sign` reconciliation reads zero for the successor.

What is forbidden during this incident is as important as what is done: re-signing the revoked range is refused by the kernel; back-dating a revocation requires the founder's written approval; and there is no spare key "in a drawer", because a spare key is the thing being protected against [E16 §07, §10]. A revoked key's records still verify and are **never** reported as a verification failure [E16 decision 7]. At L1 this runbook does not exist, because no key material exists anywhere [E01 §07].

22.6.4 Timestamp-authority outage (RB2, L2)

Trigger. A token request times out, errors or returns a malformed token [E17 Flow 2]. **Severity** P2 at two consecutive failed intervals for one authority (SRE paged); P1 when both authorities fail for four consecutive intervals and the tenant's Assurance door changes from a count to a banner ("Anchoring degraded since 09:14 — 88 records unanchored"); P0 if two tokens disagree about a root, or if one authority returns an imprint mismatch twice in a row [E17 §05–06, decision 2].

Steps. Retry the failing authority with backoff for the rest of the interval; if the other answered, mark the batch `single_anchor` — degraded, not failed. Open the next batch on schedule; never hold a batch open waiting. On recovery, anchor outstanding batches oldest first, each with its own root and true token time; never merge skipped batches, because merging loses interval boundaries an auditor is entitled to read. Write up any period in which both authorities failed for more than one interval, available to every affected tenant [E17 Flow 2]. If an authority returns tokens that fail elan1's own verification, remove it from the pair by an operational decision and run with one while a replacement is contracted [E17 decision 10]. **Verify:** unanchored head returns to within the published interval; every batch in the window has at least one verifying token; the degraded interval appears with its cause and duration in the anchor log.

The runbook never refuses a customer's write [E17 decision 5]. That is the difference between this class and a KMS outage, and it is deliberate.

22.6.5 Declaration mis-publish (RB10)

Trigger. A certified declaration version resolves in production with a wrong gate order, a wrong blast radius, a wrong channel mode or a loosened gate; or a promoted Enforced write type shows a successful write to the target with no record [E09 §09 failure paths]. **Severity** P1; P0 if the mis-publish loosened a never-skippable gate. (The six never-skippable gates cannot be omitted — validation fails, and "if a validator bug let it through the kernel runs it anyway" [E04 rung 3] — so a P0 here means both defences failed.)

Constraints. A certified version is immutable; nothing resolves to "latest"; a governed action written under version N resolves against N forever [E05 §10; E02 I7; IM §6.11]. Registering a version is itself a governed write under dual control [E09 §04 step 6]. Loosening a gate requires a new version and an ADR [IM §6.11]. There is therefore no "unpublish".

Steps (build recommendation, consistent with those constraints). Stop the affected write types for the affected tenants, not the whole tenant, by publishing an emergency version N+1 that removes them from `write_types` under the normal dual-control certification — the only route by which a write type becomes unreachable. Mark version N `withdrawn` in the registry: it remains resolvable for `verify` and for every record already written under it, and it becomes unresolvable for new intents, which then refuse with `POLICY.DECLARATION_UNRESOLVED` at rung 2 [E04 rung 2]. List every record written under N in the window, by `seq`, and assess each against the corrected rules; any record that the corrected version would have refused is reported to the tenant as a record written under a defective version, never relabelled. For an Enforced write type with an unrecorded write, revert the mode to Attested advisory immediately, tell the tenant, and render the boundary

date on the Assurance door in both directions [E09 decision 7]. **Verify:** the conformance suite passes against version N+1 on the reference tenant; C13 rejects an intent pinned to N; the tenant's Assurance door shows the version boundary with counts.

22.6.6 Tenant isolation breach

Trigger. Any evidence that one tenant's data, record, chain or token was readable or writable by another: a cross-tenant read test going red in CI or production; a cache, index or vector store returning another tenant's entry; a token carrying the wrong tenant; a call accepted without tenant attribution [O05 §2]. **Severity** P0, and a presumptive CERT-In reportable incident.

Steps [O05 Flow A, adapted to the kernel]. Declare; the six-hour clock starts at noticing. Contain before diagnosing: revoke the affected tokens and credentials; set the stop for every tenant on the suspected path, and verify each by an attempted write. Page the named CERT-In responder, who assesses reportability and files within six hours; if unclear, file. Run the DPDP assessment in parallel — whose personal data, and does the tenant have to be told — and notify through the DPA contact. Preserve access logs for the window before any restart. Check the proof chains of the affected tenants with `verify`: isolation of the chain (one per tenant, its own `seq` from 1) is independent of isolation of the data, and a clean chain is evidence that no cross-tenant **write** was governed, not that no **read** occurred [E16 §04]. **Verify:** the cross-tenant read test, run in the environment where the breach occurred, fails to read another tenant's row; `verify` passes on every affected chain; the written tenant update has gone out within 72 hours.

The structural controls that make this class rare — RLS on every tenant table, partitioned caches, unattributable calls rejected — are covered in Chapter 18 [O05 §2].

22.7 CERT-In and DPDP reporting timelines

CERT-In Directions 2022 are in force now: incident reporting within six hours, 180-day logs retained in India, and clocks synchronised to NPL or NIC [O05 §01]. DPDP main obligations commence on 13 May 2027 [O05 §01]. The clocks that apply to a kernel incident:

Clock	Starts	Duration	Owner	Source
CERT-In report	Noticing, not confirming	6 h	Named CERT-In responder; founder if unreachable	O05 Flow A
Key-compromise notice to tenants	`T_suspect` declared	24 h	Incident commander	E16 Flow 2 step 5
Written update to affected tenant	Declaration	72 h	On-call	O05 Flow A step 6
Erasure request	Receipt	72 h end to end	Two named approvers	O05 Flow B; O05 decision 5
Post-incident note to affected tenants	Close	5 working days	Incident owner	E09 §02 row 6; O05 Flow A
Both-TSA outage write-up	Recovery	Not fixed; available to affected tenants	SRE	E17 §06 step 6

The minimum bar before any brand's first external user includes a named CERT-In responder and the six-hour path tested once [O05 §12]. Both are open. The responder and deputy were due to be named by 30 September 2026 [O05 OQ2]; rung1 carries the same question to 31 January 2027 [R12 OQ6]. The Information Memorandum adds that no primary CERT-In URL was captured and one should be before the obligation is relied on contractually [IM App. E.5].

(Build recommendation) Record every incident in a control-plane table under the same row-level security as every tenant table, so that the timeline is data, not narrative, and the six-hour and 72-hour deadlines are computed rather than remembered:

```
CREATE TABLE kernel_incident (
  incident_id     uuid PRIMARY KEY,
```

```
  severity        text NOT NULL CHECK (severity IN ('P0','P1','P2','P3')),
  incident_class  text NOT NULL,        -- chain_gap | meter_mismatch | key_compromise | tsa
                                        -- | decl_mispublish | isolation | bundle_fail | other
  noticed_at      timestamptz NOT NULL, -- the clock starts here
  declared_at     timestamptz NOT NULL DEFAULT now(),
  certin_due_at   timestamptz GENERATED ALWAYS AS (noticed_at + interval '6 hours') STORED,
  certin_decision text CHECK (certin_decision IN ('filed','not_reportable','pending')),
  certin_ref      text,
  t_suspect       timestamptz,          -- key incidents only; earliest defensible instant
  domains         text[] NOT NULL,      -- residency domains affected
  seq_ranges      jsonb,                -- per tenant: {tenant_ref: [from, to, state]}
  contained_at    timestamptz,          -- tenant/domain stop verified by attempted write
  verified_at     timestamptz,          -- runbook's final check returned expected output
  closed_at       timestamptz CHECK (closed_at IS NULL OR verified_at IS NOT NULL)
);
```

The last constraint is the "cannot be closed on a theory" rule expressed as a `CHECK` [S09 decision 7].

22.8 Backup, disaster recovery, RPO and RTO

22.8.1 What the sources fix

Control	Requirement	Source
Region	ap-south-1 (Mumbai), three availability zones; no second region, **including for DR**, until a residency decision is written and counsel-confirmed	O05 §1; O05 decision 2
Database	Postgres primary with a synchronous standby in a second AZ; point-in-time recovery on	O05 §1, §6
Backups	Nightly, encrypted, retained 30 days	O05 §6
Evidence	Insert-only tables plus object storage with Object Lock (WORM)	O05 §5
Restore drill	At least quarterly, into a clean environment, timed; includes restoring the evidence store, re-running `verify` on a sample of records, and running the conformance suite against the restored stack	O05 §6
Go-live bar	No go-live for any brand without one recorded successful restore drill	O05 decision 6
Keys	Never backed up, escrowed or copied between domains — "including 'for disaster recovery'"	E16 §01, §10
RPO / RTO	"Not yet set; set before go-live"	O05 §6; O05 OQ1 (15 Oct 2026)

stack1's restore runbook shows the procedure the kernel inherits: restore the tenant store and the evidence store into a clean environment, never over a live one; run the blocking conformance classes; then re-run a sample of proof-record verifications "through the elan1 layer's own verify path" and report a pass count, not a row count [S09 RB-07]. stack1 GA requires this to have been done for real, with the RPO and RTO actually achieved written down whether or not they looked good [S13 §4].

22.8.2 What DR means for each kernel store

The absence of a second region is a deliberate residency choice with a consequence that should be stated plainly: **the loss of the Mumbai region is not recoverable inside any published RTO**. Within the region, the synchronous standby gives zone-failure protection. **(Build recommendation)** The targets below are proposed for adoption against O05 OQ1; they are not company decisions.

Store	Failure	RPO	RTO	Why
Proof chain and outbox (Postgres, evidence schema)	AZ loss	0	≤ 30 min	Synchronous standby; a committed write is on two AZs before `COMMIT` returns
Proof chain	Logical corruption	0 for records; restore to a clean env for forensics only	Stop-and-verify, not restore-over	A chain is never restored over itself; the live chain plus gap/incident records is the recovery
Evidence objects (WORM)	Accidental deletion	0	n/a	Object Lock prevents it

Store	Failure	RPO	RTO	Why
Declaration registry	AZ loss / corruption	0	≤ 30 min	Append-only, every version forever [E10 §07]
Meter ledger	Corruption	≤ 15 min	≤ 2 h	Rebuildable from the chain, which is authoritative
Snapshots for 30-day rollback	Loss	24 h	≤ 2 h	Loss removes the fast undo path; the forward path remains [E01 §02 #6]
Signing keys (L2)	KMS regional outage	n/a	Duration of the outage	No backup by design; the domain refuses writes (§22.3.1)
Whole region	Region loss	Last off-site export, if any is permitted	Undefined	No second region until a written residency decision

The 15-minute and two-hour figures are rung1's own targets for its control plane [R12 §6], used here so that the consumer and the kernel do not carry different numbers for the same event.

22.8.3 Retention classes, and where the sources disagree

The baseline retains audit and proof records seven years in WORM storage and evidence bundles 24 months [O05 §5]; rung1 retains the proof chain seven years [R12 §6]. The evidence bundle specification retains "the chain and its anchors indefinitely", because a bundle must be checkable in ten years, and does **not** retain bundles at all — they are generated on request [E15 §04]. The pricing model costs a ten-year carry of every record [E10 §07]. And the first pack exists to serve an MCA audit-trail obligation whose preservation floor is eight years [IM §6.7; IM §11.5]. A seven-year proof retention cannot serve an eight-year statutory floor. This memorandum treats **indefinite retention of the chain and anchors** [E15 §04] as the operative rule, because it is the only one of the four that satisfies the first pack, and records the conflict for closing.

22.9 Capacity management

Throughput is not the constraint at planned volumes: 180,000 governed actions a month at tranche 1 [IM §18.2] is about 0.07 a second on average (our arithmetic). Structure is.

- **The single writer serialises each tenant.** One process mutates a target and one chain writer assigns `seq` per tenant [E02 I2, §3], so a tenant's write rate is bounded by rung 7 plus the sandbox-first applies. Scale across tenants, never within one **(build recommendation: partition chain writers by tenant hash, with the writer lock as the only coordination)**.
- **Cost is fixed per residency domain** — footprint, registry, on-call share, and at L2 key custody and anchoring [E10 §07; EBB EB-11 §6]. The ten-tenants-per-domain density assumption has "nothing behind it" and is due for measurement by 30 June 2027 [E10 OQ4].
- **Storage never compacts.** About 4 KB per record with gate results at L1 [E10 §07]; about 640 bytes per record in an L2 bundle [E15 §04]. At tranche-1 volume that is roughly 8.6 GB a year of evidence rows (our arithmetic): small, but permanent.
- **Anchoring scales with intervals, not records** — about 2,880 tokens per authority a month for a continuously writing tenant, none for an idle interval [E17 §07].
- **Quotas.** Sandbox: 10 writes a minute, 2 concurrent, 5,000 a month, 429 with `Retry-After` on breach [E09 §02 row 3]. Production quotas are per contract and cannot be set until a design partner produces load [E05 OQ2]. A 429 has no proof entry and no meter event and is never reported as a refusal [E05 decision 2].
- **The runtime underneath** refuses at the edge above declared peak plus 40% [S09 RB-08]; the kernel's per-domain probes should alert before a consumer sees that refusal **(build recommendation)**.

22.10 Support model for design partners and third parties

rung1 and its design partners. rung1's customers are rung1's to support: its desk takes the ticket, sets severity and speaks to the tenant [R12 §2–3]. The kernel supports rung1, engineer to engineer, through its own on-call; internally that is "one engineer, one channel" [E10 §04]. When the kernel is the cause, rung1 diagnoses with the elan1 on-call [R12 RB-4]. rung1 owns the words the tenant reads; the kernel owns the refusal codes and operator actions those words are rendered from [E02 §06].

Third parties before GA. Phase 1 has no self-serve production access; third parties work under an explicit pre-GA agreement [E05 §01, §11]. A design-partner pilot does not include a production key by default and goes through the E09 production review if it needs one [E10 §03].

Third parties at L1 GA. The external consumer receives "a rota, a status page, an SLA" rather than a shared channel [E10 §04]. The path is fixed by E09: sandbox key self-serve; a certified declaration; the conformance result file attached to a production request; a four-point human review promised in two working days; a named production owner who receives incident notices; the first invoice with the refusal count on its face at ₹0 [E09 Flows 1–5]. Who staffs the two-working-day review at twenty requests a week is open to 31 March 2027 [E09 OQ1]. Support tiers, hours and response promises for external kernel consumers are not defined in any source. **(Build recommendation)** Publish one tier at L1 GA — business hours IST for P1–P3 at the §22.5.2 response targets, P0 by named escalation at any hour — and add a paid 24×7 tier only after the third on-call name exists (§22.5.3).

Promotion to Enforced is a support event as much as an engineering one: per write type, after at least 30 days of Attested advisory traffic, with a bypass proof against the real target and a rollback drill, recorded as a certified declaration version [E09 §09]. Whether 30 days has any basis is itself open [E09 OQ2].

22.11 Operational readiness before L1 GA

L1 GA opens only when every readiness row reads "done, measured rather than asserted"; a partial GA is not a GA [E09 decision 2]. Combining the E09 checklist with the elan1 column of the group minimum bar [O05 §12]:

Item	Done means	Status at 23 Sep 2026
Documentation	E02–E05, E12, E13 at stable URLs; every refusal code has a public page	Not started
Sandbox keys	Self-serve, tenant in < 60 s, p95 published	Not built
Quotas	Published and load-tested quarterly	Not set
Status page	Per component, chain integrity its own component, history never edited	Not built; host undecided [R12 OQ3]
Support path	One address, named human on a rota	Rota of two specified [EBB EB-11 §3]
Incident process	Ladder, page targets, five-day post-incident notes	This chapter proposes one
Versioning	`/v1`, 12-month sunset, codes never reused	Specified [E05 §10]
Conformance	C1–C14 runnable by a consumer; fails on a broken implementation	Specified [E02 §08]
Residency and isolation	Mumbai, three AZ; RLS with cross-tenant test in CI	Specified [O05 §1–2]
Restore drill	One successful drill on record, incl. `verify` sample	Not run; RPO/RTO unset
CERT-In path	Named responder; six-hour path tested once	Responder unnamed
Break-glass	Tested and alerting within one minute	Not built

Items 4, 6 and 8 are, in E09's words, the ones that get deferred under schedule pressure and "the three a serious integrator checks first" [E09 §02].

23. Technical risks, open questions and diligence answers

This chapter is written for the technical diligence team first and the build team second. It gathers in one place what could make the technical design fail, what the company has not yet decided, and the answers to the questions a sceptical reviewer should ask. The sources are the Information Memorandum's risk chapter and its appendix of open items and contradictions [IM §18; IM App. E], the "Open questions" table that closes every E-series document, the group decision register [O01], the group competitor watch [O08], and the rescored competitive matrix [E08]. Where this chapter assigns a likelihood or an impact, or proposes a response the sources do not record, the judgement is this memorandum's and is marked **(build recommendation)**.

> **The company's own rule applies to this chapter.** "Designed" is not "yes" [O01 DR-010]. The corpus's instruction to a reader who finds a problem its own appendix did not name is to "treat that as the more serious finding" [IM App. E]. This chapter tries to name every problem first, including the ones that weaken the case.

23.1 The state of the evidence

Before any risk is weighed, the reviewer should hold the facts about what exists.

Claim a reader might assume	Position at 23 September 2026	Source
elan1 has customers	None. No customer, no signed order, no invoice; governed actions committed or refused: 0	IM App. E.1
The kernel runs somewhere	A specification corpus and "an implementation of the L1 kernel against a single system of record"; no production tenant	IM §4.9
Latency and cost are known	Modelled only; two unreconciled latency models (≈ 64 ms and ≈ 91 ms)	IM §4.9; E04 §02; E10 §07
Records are signed	No. L1 has no key material anywhere; L2 does not exist	E01 §03, §07; IM §4.9
The seal format is proven	Frozen with 64 vectors (28 negative); the two-implementation rule has **not** been satisfied; the earlier published digest was retired as computed against a superseded encoding table	IM §4.9; E12 §01
An outside party can verify anything	Not at L1, by design; the offline verifier arrives at L2	E01 §08; E07
The company is certified	"We hold no certification of any kind"	IM App. E.1; S13 7.4
Operations are ready	RPO and RTO unset; CERT-In responder unnamed; TSA unnamed and unnegotiated	O05 OQ1–2; IM App. E.1
The later phases have begun	The Phase 2–4 folders were "in place and empty" on the date the Phase 2–4 documents carry	IM App. E.1, E.2 #16
The competitive gap is ours	elan1's own row in the competitive matrix reads No in all four capability columns	E08 §03, decision 2

The last row is the company's own scoring, and the matrix gives its reason in a sentence a reviewer can test: "we have the design and nobody's books" [E08 §04].

23.2 The technical risk register

Likelihood (L) and impact (I) are rated High, Medium or Low. The ratings are this memorandum's **(build recommendation)**; the risks, mitigations, owners and triggers are sourced. "Trigger" means the observable event that should make the risk's owner act; where the trigger is also a kill criterion or falsifier, it says so.

#	Risk	L	I	Mitigation	Owner	Trigger / falsifier
T1	Kernel latency envelope not held once the sandbox-first applies are real	M	H	Rung spans measured at Gate 1; target-system time given its own SLO; no rung skippable [E04 §02, decision 3; ETS §10]	Kernel engineer	p50 > 120 ms or p99 > 450 ms at ten design partners' load — **kill criterion** [IM §18.2]
T2	Marginal cost of a governed action far above model because the shadow sandbox apply is 10× the ₹0.15 assumed	M	H	Cost published monthly beside the refusal rate; transfer price reset quarterly; no price below measured cost [IM §18.1; E10 OQ2]	Kernel lead	Measured cost > ₹0.40 for two consecutive months — **falsifier four** [IM §18.3]
T3	Enforcement cannot be held outside the system of record for anyone but rung1 hosted	H	H	Mode A credential custody; per-write-type promotion with a bypass proof on the real target [E09 §09; O03 §02]	elan1 lead	No deployment mode lets the kernel hold the credential — **kill criterion** [IM §18.2; O03 OQ1]
T4	Attested advisory sold or described as enforcement	M	H	`enforcement_mode` declared and sealed per write type; C14 claim check in the consumer's CI; caller-performed flag in every record [E05 decision 1; E02 C14; O03 decision 3]	Founder	Any "blocked", "prevented" or "cannot" in advisory copy [O03 §02]
T5	L1 chain rewritten or truncated by whoever controls the store	M	H	Claim limited to "tamper-evident to us and the tenant"; nightly verifier; anchors at L2 detect a cut tail [E04 rung 8; E17 §01]	Evidence lead	A buyer who needs third-party verification before L2 [E01 §04]
T6	Two-implementation rule delays L2 or does not converge	M	H	Spec text frozen early; second implementation possibly a paid external team; disagreement log kept [E12 §08; IM §18.1]	Founder	No convergence on all 64 vectors by 30 Sep 2027 [IM §16.6 item 1]
T7	Timestamp authority: no contract, unknown cost, no India-resident authority	H	H	Two authorities, one India-based; Merkle batching; `single_anchor` degraded mode; founder owns the negotiation [E17 §03–07; IM §16.6]	Founder	Contract not signed by 31 Oct 2027; price unquoted by 30 Jun 2027 [IM §16.6; E17 OQ1]
T8	Cloud KMS insider can alter the key policy	M	H	`Sign` granted to one principal; three independent records of key use; nightly two-way reconciliation; limitation disclosed unprompted [E16 §08–09]	Evidence lead	A `Sign` call with no record [E16 §08]
T9	DPDP erasure is held not to be satisfied by salt destruction on an append-only chain	M	H+	Salted F14; salt destroyed on erasure; verifier reports "PASS — payload unbound" [O05 Flow B; E15]	Founder, with counsel	Adverse counsel reading, or a data-subject request before the reading [IM §18.1; O05 OQ4]
T10	A model reaches the decision path	L	H	Import-graph check (C1); rungs 3–6 and 8 unreachable from any model [E02 I1, C1]	Kernel lead	C1 failure; see the O01 contradiction in §23.5
T11	Determinism breaks (a gate reads a clock or an undeclared source)	L	M	`as_of` supplied, never read; nightly replay of every reference-tenant refusal [E02 I8; E01 §06 #5]	Kernel engineer	Any replay divergence [E02 C4]
T12	The refusal register and fixtures map the gate set for an attacker	M	L	Publication decided explicitly [E01 OQ3; E02 OQ4; E09 OQ3]	elan1 lead	Evidence of probing by refusal pattern
T13	stack1 runtime slips, taking key custody and the anchor job with it	M	H	The runtime is replaceable behind contracts; stack1 slippage "reaches elan1 and never reaches the product" [IM §16.6]	stack1 lead	stack1 GA later than 30 Sep 2027; `fusion.run/v1` not blocking-green by 30 Nov 2027 [IM §16.6]
T14	Temporal is the heaviest operational item and every governed write runs inside it	M	M	Postgres-backed Temporal; idempotency keys; no replay without a key [O01 DR-002; STS; S09 RB-05]	stack1 lead	A replay that repeats a side effect [S09 decision 4]
T15	A licence moves under a pinned component	M	M	Licence register, quarterly re-read, CI check on pinned SHAs [O04; IM §18.1]	Runtime lead	A LICENSE diff at a pinned commit [O08 D7]
T16	Single region; region loss unrecoverable in any RTO	L	H	Three AZs with synchronous standby; no second region until a residency decision [O05 §1]	stack1 lead	Region-level outage; a customer contract demanding DR
T17	Applied cryptography and regulatory engineering cannot be hired at plan	H	H	Highest bands (₹38 and ₹30 lakh); equity-heavy packages; paid external second implementation [IM §18.1, §15.2]	Founder	Either function more than one quarter behind plan [IM §18.1]

#	Risk	L	I	Mitigation	Owner	Trigger / falsifier
T18	Platform absorption by Microsoft, ServiceNow, AWS, Salesforce, SAP	M	H	Price per action, never per seat; enter through the auditor's budget; sell what a vendor will not produce about itself [IM §12.7; E08 decision 4]	Founder	Governance included free in any two of Microsoft, ServiceNow, SAP, Boomi [O08 C1]
T19	Frappe ships a governed write path inside ERPNext	M	H	Compliance depth and channel; route to a second system of record [O08 B1–B2]	Founder	Flow GA under a permissive licence with an approval step [O08 B1]
T20	The auditor will not rely on the bundle	M	H	Auditor's necessity statement taken on day 0 and bound into the profile [IM §18.1]	Founder	Day-80 verdict names something not on the roadmap — **falsifier three** [IM §18.3]
T21	The receipt format is defined by someone else first	M	M	Profile SCITT (RFC 9943) and COSE Receipts (RFC 9942) rather than compete; win on production [IM §18.1]	Standards lead	A rival format acquires two independent implementations [IM §18.1]
T22	The corpus contradicts itself on build-relevant points	H	M	Every conflict recorded (Appendix A); E/O series prevail over earlier material	elan1 lead	A contradiction found in build that no register names

Three of these deserve reading together. T2 is the risk most likely to fire, because "a competitor's price collapse is a threat; our own cost exceeding our own price is a fact, and it arrives with a date on it" [O08 C6; IM §18.1]. T9 is the risk that would do most damage, because an adverse answer "does not compress a margin — it invalidates the artefact" [IM §18.1]. And T7 is the one that no amount of engineering shortens: "a negotiation with an entity we have not yet named, on someone else's calendar" [IM §16.6].

23.3 Platform absorption, read technically

The Information Memorandum calls platform absorption "the strongest argument against this company", and dates its beginning to 2026 [IM §12.6]. The commercial reading is in the IM. The technical reading asks a narrower question of each platform: what exactly does it govern, where does its authority stop, and what single shipped capability would close the gap elan1 occupies.

Platform	What shipped in 2026	Where it stops	What would close the gap	Watch
Microsoft Agent 365	GA 1 May 2026 at US$15 per user per month; agent discovery, policy controls on agent actions, runtime blocking of coding agents, "audit-ready evidence generation" [IM §12.6]	Coverage of non-Microsoft agents "expanding but newer and partial"; the audit log is attested by Microsoft [IM §12.7]	Cross-platform business-write evidence with an independently verifiable receipt [IM §12.7 #1]	Purview for Agent 365 docs and audit-log schema, monthly [O08 A2]
ServiceNow AI Control Tower	Gartner MQ Leader; kill switch; 30-plus discovery integrations incl. SAP, Oracle, Workday; offered free for a year [IM §12.6]	"Strongest for what runs through ServiceNow"; per-action inside its own workflows; signs nothing [IM §12.7; E08 §03]	Approvals or write operations into a system of record ServiceNow does not own [O08 A1]	Connector catalogue listing ERP writes, monthly [O08 A1]
AWS Bedrock AgentCore / Guardrails	Policy as a separately deployed artefact enforced at a gateway; governs model calls and tool access [BOOMI §19.2; IM §12.3]	"Stops outside the AWS account boundary"; rescored "specified, not yet shipped" for business actions [IM §12.7; IM §12.1]	A per-action record of a business write to a non-AWS system of record	Not on the O08 list — **gap in the watch list**
Salesforce Agentforce Operations	Announced 29 Apr 2026: "permanent audit trail by recording every AI action against relevant workflow blueprints" [IM §12.6]	Permanent inside Salesforce's object model [IM §12.7]	The same trail for writes outside Salesforce, exportable and checkable	C2 per-action price watch [O08 C2]
Palantir AIP	Governs the action, holds the credential and records it — on Foundry objects [E08 §03]	Single vendor, no portability, no third-party verifiability [E08 §03]	AIP action governance offered against non-Foundry systems of record [O08 A5]	Quarterly [O08 A5]

Platform	What shipped in 2026	Where it stops	What would close the gap	Watch
Boomi Agent Control Tower	Registry, guardrails, kill switch; AI Gateway via Lunar.dev [E08 §03]	Observes and routes agents; no per-action record of the business write; signs nothing [E08 §03]	A per-action record of a business write [O08 A4]	Quarterly [O08 A4]
SAP / Oracle	SAP Agent Hub is an MQ Niche Player and the system of record elan1 would govern [IM §12.7]	Inside its own estate [E08 §03]	Native write-path approval gating with signed evidence — "the existential scenario" [IM §12.7 #3]	Not separately listed in O08

The table supports three technical conclusions.

The gap is architectural, and it is narrow. Every platform above enforces inside its own estate or on the model interaction; none produces a per-action record of a write into a system of record it does not own that a party who distrusts the vendor can check [E08 §04; IM §12.7]. That property depends on holding the credential to somebody else's system of record, which is why the quadrant is empty — and why elan1's own cell in it is also empty [E08 §04].

The defence is not a feature race. The design deliberately consumes identity rather than competing on it [IM §13.2], and treats MCP's 2026 in-band `input_required` affordance as a way for an approval gate to become "a protocol behaviour any MCP client already understands" [IM §13.1]. **(Build recommendation)** The kernel should therefore be buildable *under* these platforms: a Mode B MCP server that Agent 365, Control Tower or AgentCore gateways can call, with the record produced by elan1 rather than by the calling platform. That turns a Class A trigger into a partnership question, which the watch list already asks: at what point is a partnership the right answer rather than a competitive response [O08 OQ5].

The watch list has holes. O08 watches Microsoft, ServiceNow, IBM, Boomi, Palantir, Airia/Zenity and ERP incumbents, but has no row for AWS AgentCore or Google, and no row for SAP or Oracle shipping signed write-path gating, which the IM names as the existential case [O08 §03; IM §12.7]. **(Build recommendation)** Add both rows with monthly cadence. DR-010 applies to these rows as to any other: an announcement is not a trigger; shipped capability is [O08 §07].

23.4 The open-questions register

Every E-series document ends with the questions it leaves open, each with an owner and a date. They are consolidated below and de-duplicated where two documents ask the same thing. Dates are as the source gives them; "first DP" means first design partner. Questions whose date has passed or falls before 31 December 2026 are the ones a diligence team should ask to see closed. Group-document questions that bear on the build (O01, O03, O05) are included where they block kernel work.

23.4.1 Questions that block L1 (due by mid-2027)

#	Question (abridged)	Source	Owner	Needed by
Q1	Who is the named CERT-In responder and deputy?	O05 OQ2; R12 OQ6	Founder	30 Sep 2026
Q2	Does `block_doors` → `channels` need a schema version bump?	O01 OQ1	Kernel lead	30 Sep 2026
Q3	Which of the 28 codes active at L1 are reachable with only ERPNext declared?	E04 OQ5	Kernel engineer	Oct 2026
Q4	Which of the 34 MCP tools ship in Phase 1?	E05 OQ5	Kernel engineer	Oct 2026
Q5	Where does an `acted_unproven` action go?	O03 OQ3	Kernel lead	15 Oct 2026
Q6	Agreed RPO and RTO, and are they affordable?	O05 OQ1; S09 OQ1	stack1 lead	15 Oct 2026
Q7	Does `assert` return synchronously through rung 9?	E02 OQ1	elan1 lead	31 Oct 2026
Q8	Is `GATE.SANDBOX_DIVERGED` gate-class or platform-class?	E02 OQ3	elan1 lead, rung1 lead	31 Oct 2026
Q9	Who owns the reference tenant and its adversarial declaration?	E02 OQ5	Founder	31 Oct 2026
Q10	Is a dual-control threshold a declaration constant or per tenant?	E03 OQ1	elan1 lead, rung1 lead	31 Oct 2026

#	Question (abridged)	Source	Owner	Needed by
Q11	Does the 24-hour IRN cancellation window belong in the declaration or the adapter?	E03 OQ2	rung1 lead	31 Oct 2026
Q12	Can hosted ERPNext accept only kernel-issued tokens (Enforced in Mode B)?	O03 OQ1; E05 OQ3	rung1 lead; ERPNext engineer	31 Oct / Dec 2026
Q13	Does an unsigned L1 record satisfy what a CA expects under the MCA audit-trail rule?	E01 OQ1; E11 OQ1	Founder, with two CAs	31 Oct / 31 Dec 2026
Q14	Does retaining a record with erased identifiers satisfy DPDP erasure?	O05 OQ4	Counsel	30 Nov 2026
Q15	Measured p50 of a full governed write on Mumbai hosting	E01 OQ2	elan1 lead	30 Nov 2026
Q16	Is the 30-day rollback a constant, per tenant or per write type?	E01 OQ4; E04 OQ4	elan1 lead	Nov 2026
Q17	Does running all gates after the first failure leak the gate set?	E02 OQ4	elan1 lead	30 Nov 2026
Q18	Salting scheme for F14 that survives the L2 retrofit	E02 OQ2; E12 OQ1	elan1 lead	30 Nov 2026 / 31 Mar 2027
Q19	How rich should `links[]` be; does the lexicon live inside or beside a declaration?	E03 OQ4–5	elan1 lead	30 Nov 2026
Q20	Publish the HMAC webhook scheme or adopt a standard header?	E05 OQ1	Kernel engineer	Nov 2026
Q21	Which ERPNext write types may be approved over WhatsApp?	E06 OQ1	rung1 lead	Nov 2026
Q22	Is a WCAG 2.2 AA audit in Phase 1 scope?	E06 OQ5	elan1 lead	Nov 2026
Q23	Does the shadow sandbox's second gate run need a separate third-party SLO?	E05 OQ6	Kernel engineer	Dec 2026
Q24	Publish the refusal register publicly at L1?	E01 OQ3	Founder	31 Dec 2026
Q25	Does an external L1 SKU exist in Phase 2, or is elan1 internal until L2?	E01 OQ5	Founder	31 Jan 2027
Q26	Who may author a declaration in Phase 2 — elan1 only or a certified partner?	E03 OQ3	Founder	31 Jan 2027
Q27	Publish the conformance suite's expected-failure fixtures?	E09 OQ3	elan1 lead	28 Feb 2027
Q28	Who owns the chain-integrity status component when the nightly job fails?	E09 OQ5	elan1 lead	28 Feb 2027
Q29	Is ₹0.15 for the shadow sandbox apply right, or 10× that?	E10 OQ2	Kernel engineer	31 Mar 2027
Q30	Who is the production reviewer at twenty requests a week?	E09 OQ1	Founder	31 Mar 2027
Q31	Is a sandbox tenant ever deleted, and what happens to its chain?	E09 OQ4	Kernel engineer	31 Mar 2027
Q32	Per-regime retention profiles at L1, or indefinite retention only?	E11 OQ3	elan1 lead	31 Mar 2027
Q33	Does the 30-day advisory observation before promotion have a basis?	E09 OQ2	elan1 lead	30 Apr 2027
Q34	Wording for an L1 record whose payload retention has expired	E11 OQ4	elan1 lead	30 Apr 2027
Q35	Ten tenants per shared residency domain — what is the real density?	E10 OQ4	elan1 lead	30 Jun 2027
Q36	Measured p50 per rung; measured p95 of the ERPNext adapter at Mumbai	E04 OQ1–2	Kernel / ERPNext engineer	Gate 1
Q37	Default obligation expiry (4 h asserted); default production rate limits	E04 OQ3; E06 OQ3; E05 OQ2	elan1 lead	first DP
Q38	Bulk approve for identical low-value items?	E06 OQ2	elan1 lead	first DP
Q39	Does the Starter tier survive a real buyer?	E10 OQ3	Founder	first external consumer
Q40	Tamil interface in Phase 1 or Phase 2?	E06 OQ4	rung1 lead	Dec 2026

#	Question (abridged)	Source	Owner	Needed by
Q41	Twelve-month sunset for consumers under a regulator's change control?	E05 OQ4	elan1 lead	Phase 2

23.4.2 Questions that block L2 (due mid-2027 to 2028)

#	Question (abridged)	Source	Owner	Needed by
Q42	Which regime gets the first verifier profile — MCA, DPDP or EU AI Act Art. 12?	E07 OQ2; E11 OQ2; E13 OQ1	Founder / elan1 lead	31 Mar–30 Jun 2027
Q43	Is one certified profile enough to ship L2?	E14 OQ2	Founder	31 Mar 2027
Q44	Who is P11's second reader before a compliance hire exists?	E13 OQ4	Founder	31 Mar 2027
Q45	AWS KMS or Google Cloud HSM for the `in` domain?	E16 OQ1	elan1 lead	31 Mar 2027
Q46	Does any Indian auditor care that the token is India-based?	E17 OQ6	Founder, with two CAs	31 Mar 2027
Q47	Is there an Indian route to a reading of the MCA binding without a formal submission?	E26 OQ4	Founder	Q2 2027
Q48	Who writes the second seal implementation — internal pair or paid external team?	E12 OQ2; E20 OQ3	Founder / elan1 lead	30 Apr 2027
Q49	Publish vector inputs before convergence?	E12 OQ5	elan1 lead	30 Apr 2027
Q50	Who is the air-gapped witness for L2 done-item 7?	E14 OQ3	Founder	30 Apr 2027
Q51	Out-of-band channel for fingerprint publication	E16 OQ2	elan1 lead	30 Apr 2027
Q52	`not_measurable` at check level as well as clause level?	E13 OQ3	elan1 lead	30 Apr 2027
Q53	Who is the second named human for upgrade stage 2 in a two-person company?	E19 OQ2	Founder	30 Apr 2027
Q54	Which second TSA with eMudhra, and does the pair survive a procurement review on independence?	E17 OQ2	elan1 lead	31 May 2027
Q55	Who holds the release key, and what happens when one holder leaves?	E16 OQ4	Founder	31 May 2027
Q56	Is fourteen days the right upgrade notice?	E19 OQ1	Founder	31 May 2027
Q57	Is `customer.update` in scope for an audit-trail pack?	E18 OQ3	Profile author	31 May 2027
Q58	What does an RFC 3161 token cost at modelled volume, and does the L2 add-on survive it?	E07 OQ1; E10 OQ1; E14 OQ1; E17 OQ1	elan1 lead	30 Jun 2027
Q59	Is 64 vectors enough once a third party implements?	E12 OQ3	Kernel engineer	30 Jun 2027
Q60	Who may author a profile or a pack; what stops a bad pack carrying our name?	E13 OQ2; E18 OQ5	Founder	30 Jun 2027
Q61	Does the profile document ship inside the bundle by default?	E13 OQ5; E15 OQ2	elan1 lead	30 Jun 2027
Q62	Which rung1 customer and which auditor for the ledger1 pilot?	E18 OQ1	Founder	30 Jun 2027
Q63	Does a shared-domain tenant need its own key before L2 GA?	E14 OQ5	elan1 lead	31 Jul 2027
Q64	Is ≤ 15 minutes right for every tier, and does a shorter paid interval contradict decision 3?	E17 OQ3	Founder	31 Jul 2027
Q65	Does anyone want anchoring without signing (an "L1.5")?	E07 OQ4; E19 OQ3	elan1 lead	Q1 / 31 Jul 2027
Q66	Second publication location for the verifier	E15 OQ3	Verifier engineer	31 Jul 2027
Q67	Is ~640 bytes per record right once inclusion paths are real?	E15 OQ4	Evidence team	31 Aug 2027
Q68	Export size limit — constant or tier property?	E15 OQ1	elan1 lead	30 Sep 2027
Q69	Full-chain replay duration at a million records	E19 OQ4	Kernel engineer	30 Sep 2027
Q70	Does annual rotation survive an auditor asking for quarterly?	E16 OQ3	Founder	30 Sep 2027

#	Question (abridged)	Source	Owner	Needed by
Q71	Should a `single_anchor` batch block a non-repudiation profile claim?	E17 OQ5	elan1 lead	30 Sep 2027
Q72	How many packs can be in review with one certifier?	E18 OQ4	elan1 lead	30 Sep 2027
Q73	Will any auditor run E11 test 3 on real data before Phase 3?	E11 OQ5	Founder	30 Sep 2027
Q74	Does field-level disclosure interest a real auditor?	E23 OQ3	Founder, with pack auditor	Q4 2027
Q75	Who is the first regulator conversation with?	E26 OQ3	Founder	Q4 2027
Q76	Re-anchor historical roots on a schedule?	E17 OQ4	elan1 lead	31 Dec 2027
Q77	Stream the KMS audit log into the tenant chain?	E16 OQ5	Kernel engineer	31 Dec 2027
Q78	Does a bundle need a member for the target system's audit trail?	E15 OQ5	elan1 lead	31 Dec 2027
Q79	May a customer switch signing off permanently?	E19 OQ5	Founder	31 Dec 2027
Q80	Does the auditor's reporting duty create willingness to pay?	E18 OQ2	Founder, with two CAs	31 Dec 2027
Q81	If the pilot auditor says L2 is insufficient, what exactly would they need?	E18 OQ6	Founder	Q1 2028
Q82	Has any of the twenty vendors shipped a signed decision?	E08 OQ1	elan1 lead	quarterly

23.4.3 Questions that belong to L3 and beyond

#	Question (abridged)	Source	Owner	Needed by
Q83	Who counter-signs at L3, and will they do it commercially?	E07 OQ5; E20 OQ1	Founder	2028
Q84	Does earned autonomy belong at L3, or is it an L1 capability deferred for the wrong reason?	E20 OQ6; E24 OQ6	elan1 lead	Q1 2028
Q85	Thresholds in `score_v1`; supervised sampling rate; auditor notified on graduation?	E24 OQ1–3	elan1 lead / Founder	Q1–Q2 2028
Q86	Does precedent belong to elan1 or to rung1's product surface?	E24 OQ4	elan1 lead, rung1 lead	Q1 2028
Q87	Cross-tenant benchmark on request? (Answer recorded: no)	E24 OQ5	Founder	Q3 2028
Q88	Does any usable trust anchor exist in India by 2029?	E22 OQ1	elan1 lead	Q2 2028
Q89	Receiving side for a non-tenant; blinding `seq`; who pays; receipt expiry	E22 OQ2–5	elan1 lead / Founder	Q1–Q3 2028
Q90	Disclosure profiles in or beside the declaration; padding low-entropy fields; derived fields at rung 4	E23 OQ1–2, OQ5	elan1 lead	Q1–Q2 2028
Q91	Who holds the salt store under customer-held keys; what is owed after an accidental loss?	E23 OQ4; E20 OQ5	elan1 lead / Founder, with counsel	Q2 2028
Q92	Commercial shape of cross-organisation verification	E20 OQ2	Founder	Q2 2028
Q93	Is sovereign deployment built at all?	E20 OQ4	Founder	Q3 2028
Q94	Who may author a Regulation; import customer control registers?	E21 OQ1–2	Founder / elan1 lead	Q2–Q3 2028
Q95	Mid-period clause change: one report or two? Severity driving exit codes?	E21 OQ3–4	elan1 lead, with pack auditor	Q1–Q2 2028
Q96	Publish Regulation objects and profiles under the standard's licences?	E21 OQ5; E25 OQ6	Founder	Q4 2028
Q97	SCITT transparency service as an anchor; media type; editorship destination	E25 OQ1, OQ3–4	elan1 lead / Founder	Q2–Q4 2028

#	Question (abridged)	Source	Owner	Needed by
Q98	Who is the second implementer of Action Receipt v1, and how are they paid without compromising independence?	E25 OQ2	Founder	Q1 2028
Q99	What may be said publicly about a competitor's inadequate conformance claim?	E25 OQ5	Founder, with counsel	Q3 2028
Q100	Cost and value of ISO 42001 and SOC 2 Type II; is an EU entity needed?	E26 OQ1–2, OQ6	Founder	Q1–Q2 2028
Q101	Threshold to build a capability a supervisor asks for	E26 OQ5	Founder, with elan1 lead	Q3 2028
Q102	If no customer pays for evidence separately by Q4 2028, fold SKU 2 into SKU 1?	E14 OQ4	Founder	Q4 2028
Q103	If Microsoft, ServiceNow or Boomi bundle governance at zero, does the floor survive?	E10 OQ6	Founder	Phase 3
Q104	At what revenue does single-tenant HSM custody stop being premature?	E16 OQ6	Founder	Phase 4

Two patterns in the register matter more than any single row. **The founder owns a large share of the questions**, including most that decide whether a level ships, which is the concentration risk the IM names without a technical mitigation [IM §18.1]. And **at least six questions ask for a buyer's or an auditor's answer rather than an engineer's** (Q13, Q46, Q73, Q74, Q80, Q81): they cannot be closed by building, only by asking, and several of them are the evidence for Gate 3 and falsifier three [IM §18.3; E10 §06].

23.5 Contradictions in the sources that change the build

The conflicts block at the end of this chapter lists the source contradictions a diligence reader should know about, with the treatment this memorandum adopts; the editor carries them into Appendix A. Three change what gets built and deserve a sentence here.

- **Whether a model may be called at rung 4.** The group glossary's Right Path table marks rung 4 `evaluate()` as "the only rung at which a model may be called" and says "eight of the ten rungs make no model call" [O01 §02]. The kernel contract, the scope freeze, the flow book and the earned-autonomy specification all say no model is in the decision path at any level, ever, and that rungs 3–6 and 8 are unreachable from a model, enforced by an import-graph check [E02 I1; E01 §02 #2; E04 rung 4; E24 rule 2]. This memorandum follows the E-series. The O01 row is the most dangerous sentence in the corpus, because it contradicts the one claim the company says it can make "on day one that our competitors cannot make on any day" [E01 §08], and it should be superseded by a numbered register entry.
- **Whether a consumer may call `seal`.** The kernel contract says `seal` is callable only by the kernel and "must never … be callable by a consumer" [E02 §3, decision 3]. The integration contract's Mode B flow has the caller report its outcome "through `seal`" [O03 §04]. The build needs a separate outcome-report call for Mode B (or an `assert` follow-up carrying the outcome) so that the kernel remains the only chain writer.
- **When L2 arrives.** The customer-facing sentences in E01, E09 and E10 say L2 is "Q3 2027" [E01 §08; E09 §08; E10 §06]. The plan puts L2 GA in April 2028, with Phase 3 running Q3 2027 to Q1 2028 [IM §4.9; E14 §01]. A support engineer reading E09's refusal script today would promise a date the plan does not hold.

23.6 Diligence questions and answers

The questions below are the ones a technical diligence team should ask, including the uncomfortable ones. Each answer is direct and sourced; where the honest answer is "not known", it says so.

23.6.1 What exists

D1. What is built and running today? A specification corpus and, per the IM, an implementation of the L1 kernel against one system of record, with no production tenant, no customer and no governed action committed or refused [IM §4.9; IM App. E.1]. The seal specification is frozen with 64 vectors but has not passed the two-implementation rule, and the

previously published test-vector digest has been retired [IM §4.9]. By the company's own scoring every capability cell in its row of the competitive matrix reads No [E08 decision 2]. A reviewer should inspect the repository and run the published conformance suite rather than rely on any document, including this one.

D2. Has any latency, cost or throughput figure been measured? No. Every latency figure is a design target marked for measurement [E04 §02; O03 §08]; the cost per governed action is modelled at ₹0.20 marginal [E10 §07]; the two internal latency models disagree (≈ 64 ms and ≈ 91 ms) and are unreconciled [IM §6.10]. The first measured cost per governed action will come from rung1's load, which begins paying from 1 January 2027 [IM §2.2].

D3. The later-phase documents are all dated the same day. Why should we believe a plan written in one pass? You should treat it as a plan, and the company says so: the Phase 2–4 folders were empty on the day those documents are dated, and "every phase after the first describes work not begun" [IM App. E.2 #16]. What distinguishes the corpus is internal discipline — every document ends in decisions and open questions, and the IM reproduces sixteen unresolved contradictions verbatim [IM App. E.2]. That is evidence of method, not of delivery.

D4. How do you stop your own material overstating what exists? Three mechanisms. DR-010 scores "designed" as zero in every matrix, including the company's own row [O01 DR-010]. The L1 claim boundary is a table of permitted and banned sentences, with "tamper-proof" banned outright [E01 §08, decision 8]. And the boundary is enforced in code: conformance test C14 fails a consumer's build if its user-facing copy says "signed", "tamper-proof" or "independently verifiable" while running at L1 [E02 C14].

23.6.2 Enforcement and the two modes

D5. What exactly does "enforced" mean, and who can claim it today? It means no write reaches the system of record without a gate decision and a proof record, and it may be claimed only where the kernel holds the write credential or the target accepts only kernel-issued tokens [O03 §02, decision 2]. In Phase 1 that is true for rung1 Enterprise in hosted mode and nothing else [O03 §02; IM App. E.2 #8].

D6. In Mode B, what physically stops an agent writing after a refusal? Nothing. "MCP alone gives advice, not enforcement" [E01 §05]. The caller holds the credential; if it ignores a refusal it still writes, and elan1 holds only a record that it was told not to, plus a reconciliation that catches the unmatched write afterwards [O03 §04]. That is why Mode B is labelled Attested advisory, why its records carry a caller-performed flag, and why the permitted sentence is "every write intent is evaluated against policy and recorded" [O03 §02–04].

D7. Can an ERPNext site be made to accept only kernel-issued tokens, so that Mode B becomes enforced? Not known. It is an open question [O03 OQ1; E05 OQ3], and whether enforcement can be held outside the system of record at all is kill criterion four for the kernel [IM §18.2; IM App. E.2 #8]. The documented alternative for third parties is promotion per write type after a bypass proof against the real target [E09 §09].

D8. If the SDK runs in the caller's process, what stops the caller using the credential directly? The credential is not in the caller's process. Target credentials live in a per-tenant credential broker and are minted per call with a ten-minute life, scoped to one write, for the single writer [O03 §06; E02 I2; O05 §3]. Service tokens can never reach KernelPort [O03 §06]. Conformance test C2 must show every other path to the target failing, with the attempts recorded [E02 C2]. **(Build recommendation)** Run the single writer as a separate process with its own identity even in Mode A, so that "in-process" describes the decision call and never the credential.

D9. What happens when the kernel is down? In Mode A the write does not happen; in Mode B it does not happen only if the caller honours the contract, which treats an `assert` timeout as a refusal [O03 §02, §04]. Inside the kernel every dependency failure is a typed, paging, unbilled platform-class refusal, and conformance test C5 proves none of them allows [E02 §06, C5]. Chapter 22 sets out the full matrix (§22.3).

D10. Is there any administrative, batch, emergency or cached path around the rungs? No, by invariant: "no fast path, no batch variant, no admin mode, no decision cache, no second implementation" [E02 I5]. Break-glass exists but is itself a declared write type under dual control, recorded in the operator's own chain and visible to the tenant [E02 decision 9]. At L2 there is no administrative "sign this" surface either; building one "would be the shortest available route to destroying the product" [E16 §08].

D11. Is there a model anywhere in the decision? Not in the E-series design: a model may produce a write intent and may never produce an allow; `evaluate()` is versioned rules over declared fields; rungs 3–6 and 8 are unreachable from any model, checked by the import graph [E02 I1; E04 rung 4]. The group glossary contradicts this for rung 4 [O01 §02], which is a documentation defect a reviewer should see corrected before close (§23.5).

23.6.3 Evidence and cryptography

D12. What does an L1 record prove, and to whom? That a governed write or refusal happened in this order, under this declaration and policy version, by this actor kind, and that any later alteration of the chain would be detected by someone holding it [E01 §08]. It is "tamper-evident to us and to the tenant", and "not yet tamper-evident to a stranger who distrusts us, because nothing is signed" [E04 rung 8]. That is why the L1 buyer is a software vendor, not a compliance officer, and why an applicant wanting a signature or a regulator claim at L1 is refused onboarding [E01 §04; E09 §08].

D13. Could your own staff rewrite an L1 chain? In principle, yes. The IM states it: "whoever controls the store could in principle recompute the whole chain at L1, and a hash chain cannot detect a truncated tail" [IM §18.1]. The controls are operational — insert-only grants to one principal, nightly verification, the proof chain held by the tenant — and the cryptographic fix is L2's signatures and external anchors [E04 rung 8; E17 §01].

D14. At L2, could an insider sign whatever they like? Not without leaving evidence, but a sufficiently privileged insider could alter the KMS key policy. The company discloses this unprompted: they "could not do so without the provider's configuration history recording it, without the dual-control record being absent, and without the `Sign`-call reconciliation showing an excess — but they could do it" [E16 §09 step 7]. The custody model that removes it is customer-held keys at L3 [E16 §09; E20].

D15. Why one signing key per residency domain rather than per tenant? Because per-tenant keys multiply rotation, window bookkeeping and fingerprint distribution by the tenant count, and "the isolation it buys is mostly imaginary while we hold every key in the same account" [E16 §04]. The consequence is stated: a tenant cannot prove its records were signed by a key no other tenant's records used; a tenant who needs that takes a dedicated domain [E16 §04, decision 2]. Note that the group baseline still says per-tenant keys with 90-day rotation; that is recorded as a conflict [O05 §3].

D16. What does a timestamp authority prove, and what if one disappears? That a digest existed in that form no later than that instant, according to a disinterested party — nothing more; not truth, not completeness, not honesty [E17 §01]. It is the only mechanism that catches a truncated tail [E17 §01]. A token remains checkable after the authority ceases to exist because the certification path and revocation data are captured at anchor time; a distrusted root is why there are two authorities [E17 §08].

D17. Does an Indian-resident RFC 3161 authority exist, what will anchoring cost, and what if it is late? eMudhra is the recommended India-based authority, but no authority is contracted or named in a contract [E17 §03; IM App. E.1]. Price at 10^8 records a year is not publicly quoted and "is either trivial or fatal" [E17 §07]. The contract is the one dependency that moves everything: a quarter's slip pushes Gate 1 into Phase 4 and removes the first two quarters of L2 revenue [IM §16.6]. Whether any Indian auditor cares that the token is India-based is itself unasked [E17 OQ6].

D18. What is the two-implementation rule, what does it cost, and has it passed? No seal record leaves the company until two implementations, written by two people from the specification text alone with no shared code or channel, produce identical bytes on all 64 vectors; a disagreement is fixed in the text, never by making one implementation match the other [E12 §01, §08]. It has not passed [IM §4.9]. The cost is people: the cryptography function grows from one to three by March 2027 and nine by March 2028 at the highest band, and the second implementation may be a paid external team [IM §15.2; IM §18.1]. How to pay a team that must not read the source is open [E20 OQ3].

D19. What happens if the seal specification has a defect after bundles are in the field? There is no v6.1: a change is a new domain string and a new verifier [E12 §09]. A bundle failing at a customer is reproduced on an air-gapped machine first; if the verifier is right and the specification wrong, the defect is disclosed to every holder of affected bundles [EBB RB9]. Records under the old domain string remain verifiable by the old verifier.

D20. How does DPDP erasure work against an append-only chain, and is it lawful? The payload is committed through a salted hash (F14); erasure destroys the salt, so the payload is unrecoverable "including to elan1" while every digest still verifies, and the verifier reports "PASS — payload unbound" [O05 Flow B; E15 exit codes]. Whether that satisfies DPDP erasure has not been answered by counsel [O05 OQ4]. The IM calls it "the sharpest unresolved legal/architectural tension" and the risk that would invalidate the artefact rather than compress a margin [IM App. E.3; IM §18.1]. At L2 elan1 still holds the salts, so "we cannot see your data" may not be said until customer-held keys [IM App. E.3].

D21. What happens to records signed by a compromised key? They still verify; they are no longer exclusive after the suspected instant; and those anchored before it keep an independent witness [E16 §07]. Revocation limits what may be signed afterwards and never invalidates what came before; re-signing a revoked range is refused by the kernel [E16 §07].

23.6.4 Dependencies and related parties

D22. How much of elan1's revenue is related-party, and how should we read it? rung1 pays ₹0.40 per committed governed action from 1 January 2027, refusals at zero, reset quarterly against measured cost: ₹0.52 crore in FY2027-28 and never more than 4.9% of revenue by FY2030-31 [IM §2.2]. The IM asks investors to treat it as "engineering validation, not commercial traction" and presents it as its own line so it can be removed in one subtraction [IM §2.2]. The pricing model forbids presenting it as elan1 revenue anywhere [E10 §04 rule 2]. The price is set by the same founder on both sides [IM §18.1].

D23. Is rung1 a captive customer whose use validates nothing? It validates load, not demand. rung1 is the only consumer in Phase 1 and builds no governance of its own [E01 decision 7], so every rung1 write to a customer's books is a real kernel write under a real ERP, and it produces the first measured cost per governed action [IM §2.2]. It does not validate that an outsider will integrate. That is tested separately and early: the kernel terminates if no third party has integrated against KernelPort under a signed agreement by 30 June 2027 [IM §18.3 falsifier one].

D24. What is stack1's role, and what if it fails? stack1 hosts the processes and never decides what is allowed; "the kernel never lives inside the runtime" [O03 §01]. It is billed at observed cost plus 15% and appears in elan1's accounts only as cost of revenue [IM §2.2]. If stack1 slips, key custody per domain and the batched anchor job slip with it, and so does the L2 chain [IM §16.6 items 2, 4]. If stack1's own round fails, elan1's exposure is "a supplier substitution, not a capital call" [IM §2.2] — a claim that depends on the `fusion.*` contracts being as substitutable as S13's GA checklist requires [S13 §1–2].

D25. How exposed is the kernel to Frappe? Heavily in Phase 1, by choice. The only system of record declared is ERPNext, written through Frappe's own API so hooks and permissions run [E04 rung 7; IM §6.10]. If Frappe ships a permission-aware, approval-recorded change path under a permissive licence, the group's technical position is "twelve to eighteen months of compliance depth plus whatever channel we have built" [O08 §04]. The kernel itself is system-of-record-neutral by construction — products vary only in declarations and adapters [O01 DR-001] — but a second adapter does not exist yet.

D26. What open-source licence exposure does the kernel carry? Permissive licences only inside the paid service; copyleft behind a contract as a separate network service; components pinned at exact commits with a licence register re-read quarterly; n8n, Inngest and Windmill excluded by decision [IM §18.1; O01 DR-003–004; O04]. The register records one favourable licence move in seven over 24 months among candidate components [IM §18.1].

23.6.5 Competition

D27. Microsoft Agent 365 at US$15 per seat advertises "audit-ready evidence". Why does elan1 survive it? Because the evidence is Microsoft's own attestation of activity it governs, and its coverage of non-Microsoft agents is partial [IM §12.6–12.7]. elan1 never prices per seat, so it does not sit in the same procurement line [IM §12.7]. The honest limit: if Agent 365 extends its evidence to cross-platform business writes with an independently verifiable receipt, "L1 becomes unsellable in Microsoft estates the week that ships" [IM §12.7 #1]. At a fifteen-user firm the Agent 365 rate also exceeds rung1's own top band, which cuts both ways [IM App. E.2 #13].

D28. ServiceNow AI Control Tower is a Leader and free for a year. Why is it not the answer? Inside ServiceNow workflows it sits in the path and its record is per action; it cannot govern a write to a system it does not own, and it signs nothing [E08 §03]. The company names it the primary threat [E08 decision 3]: if it gains write operations into third-party systems of

record, "we are a feature" [O08 A1].

D29. AWS AgentCore and the hyperscalers? They enforce policy at a gateway for model calls and tool access, and their visibility "stops outside the AWS account boundary" [BOOMI §19.2; IM §12.7]. For business actions the IM rescores AgentCore as "specified, not yet shipped" [IM §12.1]. The group watch list has no AgentCore row, which this memorandum recommends adding (§23.3).

D30. Palantir already does this. Why build it again? Palantir AIP governs the action, holds the credential and records it — on Foundry objects, for Foundry customers, with no portability and no third-party verifiability [E08 §03]. The company treats AIP as "the row that proves the design is right rather than the row that proves it is ours" [E08 §03]. The threat is AIP offered against non-Foundry systems of record [O08 A5].

D31. If IETF drafts and SCITT already define signed receipts, what is the moat? Not the format. SCITT is published as RFC 9943 and COSE Receipts as RFC 9942; an individual draft already specifies hash-chained, RFC 3161-anchored receipts; the company has done no liaison work [IM §12.8; IM §18.1; IM App. E.1]. The claimed moat is that elan1's receipt is produced by the system that actually refused, gated and executed the write, "which no format-only effort can do" [IM §12.8]. The IM also records that the moat "assumes the packs and profiles are genuinely hard, which is a hypothesis we have not tested" [IM App. E.1].

23.6.6 Operations, people and process

D32. Who is on call, and is it enough? The specified rota is one rotation of two people [EBB EB-11 §3], in a company of eleven in October 2026 [IM §15.2]. It is not enough for a 24×7 promise, and Chapter 22 recommends that L1 GA publishes none until a third trained name exists (§22.5.3). The named CERT-In responder is unappointed [O05 OQ2].

D33. What are the RPO and RTO, and is there a DR region? Not set [O05 OQ1]. There is no second region, by decision, until a residency decision is written and counsel-confirmed [O05 decision 2]; the loss of the Mumbai region is therefore not recoverable inside any RTO. Signing keys are never backed up, by design [E16 §10]. Chapter 22 proposes per-store targets (§22.8.2).

D34. Does the two-implementation rule make the evidence team too expensive to build? It makes it more expensive and slower, deliberately: the rule's purpose is to find ambiguities in the specification before a third party finds them "during an audit two years later" [E12 §01]. The plan funds the cryptography function at the highest band and allows the second implementation to be bought externally [IM §15.2; IM §18.1]. The risk is schedule, not feasibility: the L2 chain is six items in series with about one quarter of float [IM §16.6].

D35. Do refusals being free create a perverse incentive? Not for revenue, which excludes refusals; but cost per governed action rises with the refusal rate, because refusals consume every rung except the act and appear in cost but not in billed volume [IM App. E.2 #4]. The mitigation is publication: the refusal rate is published beside cost per governed action monthly so that a falling cost cannot hide a loosened gate [IM App. E.2 #4]. Refusals were made free against an earlier rule that billed them at 0.3×, to be revisited "only if a customer shows us refusal-farming" [E01 §09].

D36. Which certifications do you hold? None: no SOC 2, ISO/IEC 27001, ISO/IEC 42001 or NIST AI RMF assessment, and nobody is permitted to improve on that answer in a meeting [IM §4.9]. The certification dossier records three management-system gaps as "Nothing yet. Real gap" and has not yet costed ISO 42001 or SOC 2 Type II [IM App. E.1; E26 OQ1].

D37. If the first auditor will not rely on the bundle, what then? The consequence is written in advance. If the day-80 verdict names L3, it is a schedule problem; if it names something not on the roadmap, the pack programme stops taking candidates, the verdict is published internally verbatim, and the next quarter is spent building what the auditor named or establishing that it cannot be built [IM §18.3 falsifier three; E18 OQ6].

23.7 What would falsify the technical thesis

The technical thesis, in one sentence: a single kernel outside the system of record can hold the write credential, decide every consequential write by versioned rules at an acceptable latency and cost, and produce a record that — from L2 — a party who distrusts elan1 can check offline. Each clause of that sentence has an observable that would falsify it. Most are already written into the company's kill criteria and falsifiers; this table collects them by the clause they test.

Clause of the thesis	Falsifier (observable)	By when	Response recorded in advance	Source
The kernel can hold the credential outside the system of record	No deployment mode lets the kernel hold the write credential; enforcement reduces to advice	O03 OQ1, 31 Oct 2026; continuous	Kernel terminates as a separately funded venture	IM §18.2 kill 4
It is a platform, not one product's feature	No third party has integrated against KernelPort under a signed agreement	30 Jun 2027	Team folds into the product company as an internal control layer; remaining tranches not drawn	IM §18.3 falsifier one
It holds latency at real load	p50 > 120 ms or p99 > 450 ms at ten design partners' load, measured	At ten design partners	Kernel terminates	IM §18.2 kill 1
It holds cost	Measured cost per governed action > ₹0.40 for two consecutive months	Monthly from Jan 2027	Transfer price resets; unit repriced within two quarters, not explained	IM §18.3 falsifier four
The record format is unambiguous	Two independent implementations do not converge on all 64 vectors	30 Sep 2027 (L2 chain item 1)	Specification text fixed first; L2 slips rather than ships unconverged **(build recommendation for the date test)**	E12 §08; IM §16.6
An independent time witness is obtainable	No RFC 3161 contract signed; no India-based authority available	Contract 31 Oct 2027	Anchor outside India "and explain it"; Gate 1 moves if late	IM §16.6
The evidence plane is lawful	Counsel reads DPDP erasure as reaching retained proof records	30 Nov 2026 (question due)	Not recorded — **no pre-committed response exists**	O05 OQ4; IM §18.1
Someone outside will rely on the record	Day-80 pilot verdict names something not on the roadmap	First pack pilot	Pack programme stops taking candidates; build what was named or show it cannot be built	IM §18.3 falsifier three
Evidence has a buyer of its own	No draft contract anywhere contains an evidence line	Q4 2027	Evidence folded into the platform floor; L3 stopped, not deferred	IM §18.3 falsifier two
Platforms will not produce this about themselves	A platform vendor ships an external, exportable, hash-chained governed write path across systems of record elan1 does not own	Continuous	Kernel terminates	IM §18.2 kill 3

Two falsifiers sit above the company and have no management response beyond noticing early: that enterprises conclude "do not give agents write access" rather than "gate the writes", and that auditors, regulators and courts keep accepting ordinary logs, in which case "the price of cryptographic assurance collapses to the price of a logging library" [IM §18.3; IM App. E.6].

The DPDP row is the gap in this table. Every other falsifier has a response written before the event; the one the IM calls the most damaging does not [IM §18.1]. **(Build recommendation)** Write it now: if counsel's reading is adverse, the fallback architecture is to keep payloads, salts and identifiers out of the append-only structure entirely — the chain commits only to a per-record salted digest whose salt lives in a deletable store — and to restate the retention of proof records per regime rather than indefinitely. That fallback changes the verifier's outputs and the evidence bundle's retention promise, and both should be specified before L2 code is written rather than after an adverse opinion.

23.8 What a diligence team should check first

(Build recommendation) Eight checks, in order, each answerable in a working day by someone with repository access:

1. Run the published conformance suite (C1–C14) against the reference tenant from a clean clone; confirm it fails against a deliberately broken implementation [E09 §02 row 8].

2. Run the import-graph check and confirm no module reachable from rungs 3–6 or 8 imports a model client [E02 C1].
3. Attempt a write to the reference target by every path other than the single writer, and confirm each fails and is recorded [E02 C2].
4. Make the registry, rule engine, autonomy table and target unavailable in turn; confirm every case refuses and pages [E02 C5].
5. Insert a refusal row with a non-zero meter quantity directly into the database; confirm the constraint rejects it [E04 rung 9].
6. Flip one octet, reorder a pair, duplicate a `seq` and delete a record in a 10,000-record chain; confirm `verify` names each failure with `seq` and field [E02 C9].
7. Ask for the disagreement log from the two-implementation process; its absence means the rule has not been run [E12 §08].
8. Ask to see the tenant stop set, verified by an attempted write, and a restore drill record that re-verifies proof records [O05 Flow A; O05 §6].

If any of the first six fails, the design's central claims are not yet implemented, whatever the documents say.

Appendix A. Source conflicts and how this memorandum treats them

The source documents were written over several months by different owners, and later documents revise earlier ones. Where two sources disagree on something that changes how the system is built, the conflict is recorded. Each chapter recorded its own conflicts as it was written, so the register below is grouped by chapter and some conflicts appear in more than one chapter, seen from a different side. The principal recurring conflicts are consolidated first. None of the treatments below is a decision the Company has recorded; each is this memorandum's reading, and each names who has to close it.

A.1 Principal recurring conflicts

#	Topic	What the sources say	Treatment in this memorandum	Needs closing by
P1	L2 Attested date	E01, E07, E09 and E14: "Phase 3, Q3 2027". IM: L2 GA 30 April 2028	L2 build starts in Q3 2027; GA is 30 April 2028. E09's applicant-facing text must be corrected	elan1 lead, before any third-party onboarding
P2	Signing keys and rotation	O05, S12 and IM §10: per-tenant keys, 90-day rotation, published key list. E16: one Ed25519 key per residency domain in managed KMS, 12-month rotation, fingerprints out of band	E16 is followed. No signing key exists at L1	elan1 kernel team; update O05 and IM §10
P3	Proof-record retention	O05: 7 years (bundles 24 months). E11 and E15: chains indefinitely, no bundles kept by elan1. MCA Rule 3(1): at least 8 years; RBI MRM draft: 10 years	Chains retained indefinitely, never below the statutory floor; O05's 7 years must be raised	Security owner with counsel
P4	Who performs the live write at rung 7	E02, E04 and O03 Mode A: the kernel's single writer. R04: rung1's promoter. S03: the consumer workflow via fusion.tools/v1	The kernel's rung-7 executor holds the single-use credential; consumer adapters are declared, not writers	elan1 kernel team with rung1 and stack1 leads
P5	Semantics of define, assert and seal	E02: define registers a write type at a declaration version, assert runs the rungs, only the kernel calls seal. O03 and R04 use them differently; O03 has the Mode B caller report through seal	E02 is followed throughout	elan1 kernel team; update O03
P6	A model at rung 4	O01 §02: evaluate() is the one rung that may call a model. E01, E02, E04, IM: no model anywhere in the decision path	No model in rungs 3–8; O07's "inference at rung 4" cost is the consumer's	elan1 kernel team; update O01 and O07
P7	Third token class	E05 and IM §6.9: "ops" (read-only). O03 and O05: "service"	Treated as one class; the name must be fixed before the SDK freezes	elan1 kernel team
P8	Approval (obligation) expiry	E04 and E06: 4 hours. O03: 24 hours. R04: 72 hours; R02: 7 days	Kernel default 4 hours, overridable per write type in the Declaration	elan1 kernel team with rung1 lead
P9	Rung and refusal-register counts	IM and E02: nine rungs; E04 and O01 count ten (0b STOP). E02 names 43 codes, 28 active at L1. O01 spells codes REF-nn; E02 uses POLICY., *GATE.*, PLATFORM.*	Nine rungs plus the 0b stop check; E02's register and spelling	elan1 kernel team; update O01
P10	Latency models	E04: ≈64 ms kernel p50. E10 / pricing model: ≈91 ms. O03: p50 ≤120 ms, p99 ≤450 ms end to end	All are unmeasured targets; one budget must be published after the first measurement	elan1 kernel team
P11	Second seal implementation	E07: at L3. E12, E14 and IM Gate A: required before any seal leaves the building at L2	E12 is followed: two independent implementations before L2	elan1 evidence team; update E07
P12	F14 erasure salt at L1	E01: no erasure salt at L1. E02, E03, E11 and E12: F14 salted per record from L1	Salt from the first L1 record; the F14 scheme must be fixed before the first real-tenant record	elan1 evidence team with counsel
P13	Build order	EBB and the Agenthon: seal and verifier first (TV-001 at 311 octets). E01, O02 and IM: L1 kernel first; E12 retires 311 for 329 octets	Kernel first; record layout and digest vectors frozen early; 329 octets	elan1 lead; retire EBB figures

#	Topic	What the sources say	Treatment in this memorandum	Needs closing by
P14	Hosting substrate	ETS: AWS ECS Fargate, Secrets Manager, S3 Object Lock, DR in ap-south-2. S03, STS, DR-002: stack1 Kubernetes, OpenBao, Temporal. O05: no second region	elan1 runs on stack1's runtime in ap-south-1 only; vendor question OD-28 open	elan1 lead with stack1 lead and counsel
P15	Price books	E10: six monthly tiers, verifier profile ₹6 lakh, bundle export free. IM Appendix A: annual floors by level, profile ₹3 lakh, bundles priced beyond an allowance	Rating is built on price tables that can hold either; the IM is the plan of record	Finance owner
P16	L3 timing and pre-work	E20: no L3 work, including design, before an outside auditor accepts an L2 bundle. E22–E24 already exist; IM Gate B requires Action Receipt v1 by June 2029; IM plans L3 entry October 2029, O12 "2028"	L3 is event-gated; existing specifications are frozen text only	Founder and board

A.2 Full register by chapter

The rows below are reproduced from each chapter as written. Numbering runs A1 onwards.

Chapter 1

#	Topic	What the sources say	Treatment	Needs closing by
A1	L2 date	E01 §08, E07 §03 and E09 §08 say L2 arrives "Q3 2027"; E14 §01 and O09 §02 put Phase 3 at Q3 2027–Q1 2028 with signing live 30 Sep 2027; IM §4.9 and §16.5 put L2 GA at 30 Apr 2028	"Q3 2027" is read as the start of Phase 3 and L2 signing, not GA. L2 GA is 30 Apr 2028 after E14 done-measurements 1–8 close. Sales copy that says "L2 is Q3 2027" (E09 §08 refusal script) must be corrected	Before L1 GA copy is published, 30 Jun 2027
A2	Content of L2 and L3	O01 §02 glossary: L2 = seal, RFC 3161, bundle; L3 = verifier profiles, offline verification, Action Receipt. E07 §02–03: L2 includes verifier and profiles; L3 = second seal implementation, published vectors, standard, non-elan1 counter-signature. E14/E20/IM §1.3: L3 = cross-org verification, selective disclosure, customer-held keys, sovereign deployment, precedent, earned autonomy	Follow E14, E20 and IM. The second independent seal implementation is an L2 hard gate (E12, E14 done item 1, IM Gate A), not an L3 addition; E07's L3 list is read as "kept in agreement" at L3 (as E20 §03 does)	O01 superseding entry before L2 build starts, 30 Jun 2027
A3	What is built	IM §4.9 says an implementation of the L1 kernel against a single system of record exists; IM §2.5 and App. E.1 say zero governed actions; HFS/HV2 (derived 9–12 Sep 2026) show a prototype chokepoint whose acts insert only into elan1's own Postgres, a 13-field seal without `seq`, no signing and no export; GIP §0 speaks of "ten signed orders" and 8,186 commits, while IM and O02 say no signed order	Existing code is described as a prototype; the E02 L1 kernel is not built; there is no customer or signed order (IM and O02 prevail over GIP). A dated, derived inventory of the tree should replace all three descriptions	With the six coding blockers, 31 Oct 2026
A4	Build order: seal first or kernel first	EBB/EPA §19 and EAP put the seal specification, vectors and offline verifier first on the critical path (W0) and the Agenthon (17–21 Sep 2026) built seal A, seal B and a verifier; E01 decision 1, O02 decision 4 and IM §16.2 build L1 first and allow only L2 specification in Phase 1	Kernel first. Seal and verifier work in Phase 1 is limited to validating the E12 text (disagreement log, vectors); no implementation counts toward L2 gates, and no key, TSA or bundle is built before Phase 3	Founder ruling on the Agenthon output, Oct 2026
A5	Genesis vector size	ETR §02 and EAP §01 use TV-001 = 311 octets with a published digest; IM §7.3 says 311 and its digest are retired under the current encoding table and the genesis preimage is 329 octets	329 octets; the 311-octet vector and its digest are not quoted anywhere, and any Agenthon code built to it is re-run against the current table	Before E12 is frozen
A6	Kernel latency model	E04 §02: ≈64 ms p50 with proof at 12 ms; EPA §07 and the pricing cost model: ≈91 ms with proof at 40 ms; O03 §08 calls p50 ≤120 ms / p99 ≤450 ms the "governed write, end to end", while E04 sets end to end at ≤2 s p95 including applies	64 ms is the per-rung design budget, 120/450 ms the kernel-side envelope (and kill criterion 1), 2 s p95 the end-to-end objective including target applies; all unmeasured	E01 open question 2, 30 Nov 2026; Gate 1 readings
A7	Refusal register counts	E02 §06 says 43 codes (policy 14, gate 19, platform 10) with 28 active at L1, but its L1 table lists 32 codes (12 policy, 13 gate, 7 platform) and `define` raises three further policy codes not in the table; E05 §07 says ten are platform class	The register lint (E01 done item 6) is the single source; the wire table is completed to 43 with an explicit `active_at` level per code before KernelPort is frozen	30 Nov 2026

Chapter 2

#	Topic	What the sources say	Treatment	Needs closing by
A8	KernelPort call semantics at the seam	E02 §05: `define` registers a write type at a declaration version (control token); `assert` runs rungs 0b–9 and returns committed/refused/escalated with seq and digest; `seal` is kernel-only. O03 §03–§05: `define` registers a Governed Action (state proposed) with the intent; `assert` runs 0b–6 and returns allow/refuse/needs_approval; kernel then executes rung 7; in Mode B the caller performs the write and reports the outcome through `seal` (scope rw). IM §4.2 follows E02; IM §16.2.3 (December demonstration) follows O03	E02 governs the port. `needs_approval` is a screen label for `escalated`. In Mode B the kernel records the decision (E05 §02 "chain contains the decision"); an optional caller outcome report is a separate declared write type, not a consumer call to `seal` (build recommendation). O03 §03–§05 to be amended to match	E02 and O01 signed, 31 Oct 2026; port frozen 30 Nov 2026
A9	Token classes and scopes	O03 §06: tenant, control, service ("no fourth"); service tokens for stack1-to-stack1 calls, never reaching KernelPort. E05 §05 and IM §6.9: tenant, control, ops (read-only, no tenant data). E02 §04: tenant-token scopes ro/stage/act; O03 §05 uses scopes rw/ro	The kernel accepts three classes — tenant, control, ops; stack1's service token exists below the port and is rejected by it; scopes are E02's ro/stage/act (build recommendation)	31 Oct 2026
A10	Entity structure	O11 §02–03: Holdco elan1 holds the kernel, P3 and below, and all trademarks, and owns 100% of Opco rung1 and Opco stack1; O10 §02–03: stack1 sits "inside elan1" with no separate identity. IM §2.1–2.2: elan1 Private Limited is the kernel and packs only; rung1 and stack1 are separate ventures under common founder control raising their own rounds, dealing at arm's length	Follow the IM perimeter. Repository ownership, the KernelPort and Declaration-schema licence to consumers, and trademark ownership must be re-papered to match; O11's "what must never share an entity" rules still apply	Before term sheet, 28 Feb 2027
A11	stack1 transfer price in Phase 1	O03 §07 and S02 §9 decision 8: stack1 bills nobody in Phase 1 and issues a statement at observed unit cost. IM §2.2 and O11 §05: stack1 billed at observed internal cost + 15%	Phase 1 statement at observed cost (no invoice); cost + 15% from the date the related-party agreement is signed; the kernel meter is unaffected	With the related-party agreement, 31 Mar 2027
A12	Model at rung 4	O01 §02 Right Path table: rung 4 `evaluate()` is "the only rung at which a model may be called" (8 of 10 rungs make no model call). E01 §02 item 2, E02 I1, E04 rung 4 and IM §4.3: no model at any level, ever	Follow E02: no model is reachable from rungs 3–8 at any level; C1 enforces it	O01 superseding entry, 31 Oct 2026
A13	Voice on plane P6	O01 §02 planes: "Voice is parked with verna1". O09 §03 item 9 and decision 7: rung1 Voice (inbound approvals) is a channel on P6 needing a `voice` channel kind in Declaration block 7, and does not reopen verna1	rung1 Voice is an inbound approval channel in the consumer; elan1 owes a `voice` channel kind in block 7; verna1 outbound remains parked and outside the perimeter	Channel kind in a Declaration schema major by 30 Sep 2027 (O09 item 9)
A14	"rung1 Enterprise" and the naming standard	O10 §06 forbids "Enterprise" (and Pro, Plus, Premium) in a product name and names the line by its earlier product name; the group naming instruction renames it rung1 Enterprise; IM App. E.2 item 1 records no dated amendment	Use rung1 Enterprise and `rung1-enterprise`; O01/O10 need a dated superseding entry	31 Oct 2026
A15	elan1-built apps and Gate 2	EPA §14 and the Build Book give elan1 twenty apps (`@elan1/app-*`), ten packs and thirty compiled screens, with Gate 2 "app three renders with zero screen code"; IM §2.3 and §6.2 forbid elan1 building an ERP, a CRM or a UI shell for another product, while O09 §07 and IM §16.5 keep Gate 2 in the gate calendar	Apps are consumers' declarations; elan1 builds only the shared P5 renderers and its own Platform console; Gate 2 is read as a test of the renderers driven by a consumer's declaration, built by a team that did not build apps one and two	Before Gate 2 is staffed, 30 Jun 2027 (O09 open question 4)
A16	Meaning of ledger1	EPA §14 lists ledger1 as a "SEC 17a-4 audit-trail alternative" among ten US/EU evidence packs (claim1, carbon1, gxp1…); E18 and IM §1.4 define ledger1 as the MCA Rule 3(1)/11(g) audit-trail pack for Indian companies, first in October 2027	E18/IM definition; the product-folder pack list is superseded	Already superseded; record in O01

Chapter 3

#	Topic	What the sources say	Treatment	Needs closing by
A17	Who performs the physical write at rung 7	E02 §05 and E04 rung 7: the kernel's single writer applies sandbox then live with a minted ten-minute credential. O03 §03 step 6: "the kernel executes rung 7 ACT itself, using the credential it holds". S03 §04 Flow A steps 9–10: the consumer's stack1 workflow calls `fusion.tools/v1` with the ERPNext MCP tool and a sealed credential reference, and "only the Promoter" holds live credentials (S03 B6); the elan1 layer then writes its proof record	The rung-7 executor is elan1 kernel code; the target adapter is consumer-supplied and declared; the credential broker is elan1 logic over stack1 secrets and mints only against an allow bound to `write_id` (build recommendation). `fusion.tools/v1` may be the transport beneath the adapter, but a write that does not pass through the rung-7 executor is attested advisory, not Enforced	31 Oct 2026 (with O03 open question 3 and S03 open question 5)
A18	Commit and seal ordering	EPA §07 and §16 C15: "no seal, no commit", seal synchronous, record change rolled back if S11 fails (EPA §20 note 5 flags that the diagram commits first). E02 §05: rung-7 transaction commits; a failed chain append leaves the outbox row and the writer retries in order. O03 §03: proof failure after the act marks the action `acted_unproven`	E02: commit record + outbox, then ordered append; the caller waits for rung 8 up to an SLO and otherwise sees "applied, proof pending"; a committed write without a record beyond the SLO is a P0 in the nightly join. A live ERPNext write cannot be rolled back by a database transaction, so C15 as written is not adopted	15 Oct 2026 (O03 open question 3)
A19	Repository layout	O03 §12 and O10 §06: `elan1-core` and `elan1-declarations` as separate repositories, one writer per file; EPA §14 / Build Book EB-09: one monorepo with eleven shared packages, apps, packs, `/spec` and `/vectors`; the prototype is a single `elan1-platform` tree	O03's two repositories are the ownership boundary; add `elan1-packs`, `elan1-conformance`, `elan1-console`, `elan1-spec` now and `elan1-verify`, `elan1-seal-b` in Phase 3 (build recommendation)	31 Oct 2026
A20	Signing-key custody	S03 §01: OpenBao with non-exportable per-tenant Ed25519 keys, 90-day rotation. O05 baseline rows: per-tenant keys in managed KMS, 90-day rotation. E16 §03–§05: one signing key per residency domain in managed KMS, annual rotation, "never … software keys in OpenBao, Vault transit or a file"	No key material at L1 (E01 §07). For L2, E16 governs; OpenBao holds only target credentials and webhook secrets. O05 and S03 to be amended	Before key-custody build, 31 Aug 2027; decision by 31 Mar 2027
A21	Who hosts and operates the elan1 planes	ETS §01: elan1 runs its own per-zone, per-domain AWS VPCs (Terraform/OpenTofu, ECS Fargate, KMS, S3 Object Lock), hosting decision OD-28 open; EPA §03 draws elan1-owned zones. S02/S03: stack1 hosts the processes in `ap-south-1`; elan1 is a consumer; IM §2.2: stack1 bills elan1 as supplier	stack1 operates the runtime and databases; elan1 owns schemas, grants and the network absences as requirements on stack1's deploy specification; OD-28 is closed in stack1's favour for Phase 1	31 Dec 2026 (stack1 customer-zero gate)
A22	Physical home of the proof store	S03 open question 2 and S02 open question 4 leave open whether the proof record lives in "the evidence plane stack1 runs, or storage elan1 owns outright"; S03 §06 lists an "Evidence plane (P3)" Postgres run by stack1; EPA §03 places it in an elan1 Z4 per domain	elan1 owns the schema, the single INSERT principal and the grants; stack1 operates a dedicated instance with a separate credential set; proof never lives in trace storage	31 Oct 2026
A23	Environments	EPA §17: `dev`, `ref`, `stage`, `prod`. S02 §2: `dev`, `ci`, `stage`, `prod` and "no fifth environment"	Four S02 environments; the reference tenant and sandbox tenants are tenants in `prod` with chains marked `mode: sandbox`	With E09 readiness, Q1 2027
A24	Mode A package	E05 §01: rung1 and stack1 link "`@elan1/kernel`" (an npm-scoped name) in-process; rung1 runs on Frappe (Python); ETS §04 and the prototype put the kernel in Python 3.12	Python package `elan1_port` for the in-process port and `elan1_kernel` behind it; TypeScript and Python clients for Mode B (build recommendation)	KernelPort freeze, 30 Nov 2026
A25	Port import rule wording	E02 §01: "Nothing above L0 imports a kernel type"; O01 §02 restates it as "nothing above P3"; EPA §20 note 7 flags the ambiguity	Nothing above P3; E02 wording to be corrected	31 Oct 2026
A26	Size of the API surface	EPA §15: 115 endpoints in thirteen families; E05 §04: eight families in Phase 1	Eight families at L1; the rest arrive with L2 or fold into `ctl` families	L1 GA scope, 31 Mar 2027

Chapter 4

#	Topic	What the sources say	Treatment	Needs closing by
A27	Meaning of `define`	E02 §05.1 and R04 §02: `define` publishes a declaration version. O03 §03, §05: `define` registers a per-action Governed Action in state `proposed` and returns a GovernedActionRef	E02/R04 followed; the per-action "proposed" step is served by `stage.write`	elan1 kernel lead; before SDK v1 freeze (Oct 2026)
A28	Reach of `assert` and approval return	E02 §05.2: `assert` runs rungs 0b–9 and returns allow/refuse/escalate with seq; approval is a separate governed write followed by resubmission. O03 §03: `assert` runs 0b–6, returns `needs_approval`; after approval `assert` is called again with the approval reference and rung 7 follows. R04 §06: layer "returns allow against that decision id"	E02/E04 followed; `needs_approval` read as `escalate`; resubmission carries `obligation_ref`	elan1 kernel lead with rung1 lead; Oct 2026
A29	Who calls `seal`	E02 §05.3 and decision 3: kernel only. O03 §04: Mode B caller reports outcome through `seal`. R04 §02–03: rung1 calls `seal` after promotion and on rollback	`seal` kernel-only; Mode B uses an `act`-scoped outcome report the kernel then seals; rung1 does not call `seal`	elan1 kernel lead; Oct 2026
A30	Token scope for `assert`	E02 §04: `act`. O03 §05 table: `assert` scope `ro`	`act`	elan1 kernel lead; Oct 2026
A31	Third token class	E05 §05: ops token (read-only, no tenant data). O03 §06 and IM §6.9: service token (stack1 components, never reaches KernelPort)	Kernel recognises tenant/control/ops; stack1 service tokens presented to KernelPort are rejections and alerts	elan1 kernel lead and stack1 lead; Nov 2026
A32	Record fields for `mode`, `enforcement_mode`, caller-performed flag	E05 §02, §05 say `mode` and `enforcement_mode` are sealed into every record; O03 §04 requires a caller-performed flag on Mode B records; E02 §03 freezes sixteen fields, none of which is any of these, and forbids layout change at v1	Bound as reserved kernel-owned members in the canonicalised payload behind F14 (build recommendation); no seventeenth field	elan1 lead; before seal layout is used in production (Q1 2027)
A33	`actor_kind` source	E02 §03: required intent field. E02 §05.2 and E04 rung 1: taken from the token, never supplied by caller	Token authoritative; intent value cross-checked; mismatch is a rejection and alert	elan1 kernel lead; Oct 2026
A34	rung1 `actor_id`	R04 §04: `actor_id` is "the human who raised the request — never an agent identity". E02 §03 and E04 rung 1: `actor_kind` may be `agent`, and the security review depends on it	rung1 registers agents as actors with `on_behalf_of` = requesting human	rung1 lead with elan1 kernel lead; Oct 2026
A35	Idempotency after a refusal	E02 §05.2: a repeat `intent_id` returns the original decision (C7). Same section: after a refusal "the user resubmits with the same `intent_id` once the cause is fixed"	Committed/governed-refused/escalated keys are terminal; platform-class refusals are retryable under the same key; resubmission after a governed refusal or approval uses a new `intent_id`	elan1 lead; Oct 2026
A36	Refusal register count	E02 §06: 43 codes (policy 14, gate 19, platform 10), 28 active at L1; the same section enumerates 32 codes and `define` adds 3 policy codes (35 named)	Register file is authoritative with `active_from`; CI asserts counts; prose corrected to the file	elan1 lead; Oct 2026
A37	`GATE.SANDBOX_DIVERGED` rung and class	E02 §06: gate class, rung 3. E04 rung 7 and E09 §06: raised at rung 7 after live re-run. E02 OQ3: class undecided	Gate class, raised at rung 7, register entry to say rung 7	elan1 and rung1 leads; 31 Oct 2026
A38	Refusal code spelling	E02 §06: `POLICY.*` / `GATE.*` / `PLATFORM.*`. O01 DR-011: `REF-nn`. R04 uses `APPROVAL_EXPIRED` (not registered)	Each entry carries a permanent `ref` and a dotted `wire` spelling; `APPROVAL_EXPIRED` maps to `POLICY.OBLIGATION_EXPIRED` raised by the kernel	elan1 lead and rung1 lead; Oct 2026

#	Topic	What the sources say	Treatment	Needs closing by
A39	Model at rung 4	O01 §02 rung table: rung 4 is "the only rung at which a model may be called". E02 I1, E04 rung 4, IM §4.3: no model at rung 4 at any level	No model at rung 4; O01 table to be corrected	Founder (O01 owner); Oct 2026
A40	Notice for a new refusal code	E05 §10: new code in existing class is additive, ships without notice. E02 §07: register version increments, 30 days' notice. O06 §04: 30 days internal, 60 days external	O06 followed	elan1 kernel lead; before L1 GA
A41	Non-existent pinned version	E02 C13: rejected, not defaulted. O06 §05: `"latest"` is a rejection; a non-existent integer refuses with `POLICY.DECLARATION_UNRESOLVED`	`"latest"`/non-integer → 4xx rejection; unresolved integer → rung-2 refusal; C13 asserts both	elan1 kernel lead; Oct 2026

Chapter 5

#	Topic	What the sources say	Treatment	Needs closing by
A42	Rung count	IM §4.3, E02 C3, WD: nine rungs plus the stop set at 0b. E04 §03: "The ten rungs". O01 §02: "Eight of the ten rungs"	Nine rungs and a stop set; C3 harness counts ten stage executions per commit	elan1 lead; before the conformance suite is published (Q1 2027)
A43	Dual control versus rung order	E02 §06/E04 rung 3: G4 fails with `GATE.DUAL_CONTROL_PENDING` when no second approver exists, and a rung-3 failure goes straight to rung 8 as a refusal. E04 Flow 3: dual-control write escalates at rung 6 and the resubmission passes G4	G4 records `pending` on a first submission (no refusal code); rung 6 escalates; resubmission must pass G4 (build recommendation)	elan1 lead; before rung 3/6 implementation (Oct 2026)
A44	`policy_exempt`	E03 §02 block 4: `policy_exempt` skips rung 4 with an ADR. E04 §02: "no configuration that skips a rung"; E02 I5	Rung 4 still executes and records `allow` with rule id `exempt:ADR-nnnn`	elan1 lead; before Declaration schema v1 freeze
A45	Obligation expiry default	E04 Flow 3 and E06 OQ3: 4 hours. O03 §03: open 24 hours when no approver. R04 §06: 72 hours	Per-write-type declaration parameter, 4-hour kernel default; rung1 may declare 72 hours	elan1 lead with rung1 lead; first design partner
A46	Who performs the live write for rung1	E02 I2, E04 rung 7, O03 §03 step 6: kernel single writer, credential held by kernel. R04 §03 steps 7–8: rung1's promoter mints the token and applies, then calls `seal`. S03 §04 step 9: rung1 workflow writes through `fusion.tools/v1` ERPNext MCP tool	Kernel's per-tenant writer workflow applies; rung1's promoter output becomes rung 7 input	elan1 kernel lead, rung1 lead, stack1 lead; before Gate 1
A47	Approval as durable state	S04 §02, S03 §04: approval is a durable Temporal signal inside the run. E04 decision 4: resubmitted, never resumed; no suspended state in the kernel	Durable wait in the consumer's workflow; kernel stateless across approval; resubmission with `obligation_ref`	elan1 kernel lead with stack1 lead; Oct 2026
A48	Proof synchronous versus pending	E04 rung 8: synchronous, caller sees `seq` on every outcome. E02 §05.2: append failure retried from outbox. O03 §03: `acted_unproven`, "Applied, proof pending". R04 §07: `SEAL_PENDING`, P1 after 5 minutes	Response `committed` with `record.status: pending` after deadline; `seq` by webhook; P1 at 5 minutes; never a platform refusal code	elan1 kernel lead; 15 Oct 2026 (with O03 OQ3)
A49	Kernel and proof latency	E04 §02: ≈64 ms kernel, 12 ms proof. Pricing model (IM §6.10, ETS §04): ≈91 ms, ~40 ms proof. O03 §08: p50 ≤ 120 ms/p99 ≤ 450 ms end to end; proof p50 ≤ 40 ms. E04: end to end ≤ 2 s p95 with target	64 ms/12 ms used for L1; O03 120 ms read as kernel-side; all targets, none measured	Kernel engineer; Gate 1 measurements
A50	Autonomy table unavailable	E02 C5 takes the autonomy table down and expects refusal; E02 §06 register has no platform code for it	Register `PLATFORM.AUTONOMY_UNAVAILABLE` at rung 6 (build recommendation)	elan1 lead; Oct 2026

#	Topic	What the sources say	Treatment	Needs closing by
A51	Reversal method names	E03 §02: `snapshot_restore`, `compensating_entry`, `target_native_undo`. E18 §06: `compensating_instrument`	E03 enum used; E18 value maps to `compensating_entry`	Pack author with elan1 lead; before ledger1 build (Phase 3)

Chapter 6

#	Topic	What the sources say	Treatment	Needs closing by
A52	Granularity of `enforcement_mode`	E05 §02, decision 1 and IM §4.6: per write type. E03 §02 block 7 and decision 8: `channels[].mode`, per channel. E09 §09: promotion per write type "because `channels[].mode` is declared per channel"	Effective mode is that of the (channel, write type) pair; channel default, per-write-type promotion; fixed at rung 2	elan1 lead with founder (E03 decision 8 owner); before Declaration schema v1 freeze
A53	Where the single writer runs in Mode A	E05 §01: in-process consumers link `@elan1/kernel`. O03 §02: library call inside the caller's process, kernel holds the credential. E02 I2: any consumer path to the target makes the consumer advisory	SDK in the consumer process holds port types and client only; rungs 0b–9 and the writer run in the kernel service and per-tenant writer worker; broker grants the writer identity only	elan1 kernel lead; before Gate 1
A54	stack1 as SDK or MCP consumer	E05 §01: rung1 and stack1 are in-process SDK consumers. O03 §02, §04: stack1 tooling uses Mode B over MCP	stack1 tooling uses Mode B; stack1 must build nothing on the governed write path (O03 §10)	elan1 kernel lead with stack1 lead; Oct 2026
A55	L2 availability date quoted to applicants	E09 §08: "You need L2. It is Q3 2027." IM §4.9 and brief: L2 GA targeted April 2028	April 2028 (IM); E09 script to be corrected	elan1 lead; before L1 GA
A56	Number of Declaration semantic rules	E09 §04 and the E03 §05 heading: "nine semantic rules". E03 §05 body lists L0-S01 to L0-S11 (two added in v3.0); IM §4.5, §6.6: eleven	Eleven; Studio Validate runs L0-S01 to L0-S11	elan1 lead; before Studio build
A57	Webhook event set	E05 §09: seven events. ETS §08: "Webhooks (10 events)"	E05's seven; any additional event is additive under O06	Kernel engineer; Nov 2026 (with E05 OQ1)
A58	Webhook signature scheme	E05 §09: elan1 HMAC headers `X-Elan1-Signature` / `X-Elan1-Timestamp`; E05 OQ1 open. ETS §08: Standard Webhooks signatures recommended	Adopt Standard Webhooks (build recommendation)	Kernel engineer; Nov 2026

Chapter 7

#	Topic	What the sources say	Treatment	Needs closing by
A59	Timing of the Controls and Regulations families	E03 §09 dates both "Phase 3, with L2"; E14 freezes Phase 3 without them; E21 §01 and E13 §08 D10 move them to Phase 4	Phase 4, per E21 (later and more specific); not in L1 or L2 build scope	Closed in source; the plan in Chapter 21 must not schedule them before L3 entry
A60	Reversal method for `sales_invoice.submit`	E03 §04 and E18 §06 declare `target_native_undo` (cancel IRN and invoice) for 24 h then `compensating_entry`; E21 §08 worked example states `reversal_plan.method: compensating_entry`	E03/E18 form used; E21's example read as describing the after-window path	elan1 lead, when declaration v1 for rung1 is certified
A61	Name of the sales-invoice declaration and write type	E03 §04 slug `sales_invoice`, write type `sales_invoice.submit`; E21 §08 declaration `erp_sales_invoice` at version 7; E07 §01 writes `erp.sales_invoice.submit`	E03 grammar `<slug>.<verb>` is canonical; `erp_sales_invoice` used only when quoting E21	elan1 lead with rung1 lead before first certification (Oct 2026)

#	Topic	What the sources say	Treatment	Needs closing by
A62	C-AT-01 coverage vs declared write types	E21 §08 C-AT-01 covers `sales_invoice.amend`, which no E03/E18 declaration declares (L0-C02 would fail); IM §6.7 figure binds C-AT-01 to `journal_entry.submit` with dual control + precondition + retention, E21 to precondition + retention only	Worked example follows E21 but notes the IM variant; `amend` must be declared or removed from coverage before the control could validate	Profile/control author, Phase 4
A63	Number of semantic rules	E03 v3.0 and IM §4.5/§6.6: eleven (L0-S01–S11); E09 §04 and EBB EB-15 §6: nine	Eleven	Closed; E09 and EBB text should be updated
A64	Validation-time refusal codes	E09 §04 uses `POLICY.DECLARATION_INVALID`, `POLICY.VERSION_NOT_SEQUENTIAL`, `POLICY.NOT_CERTIFIED`; none is in E02's closed register of 43, which uses `POLICY.DECLARATION_UNRESOLVED` for uncertified versions	Validation failures treated as control-plane rejections, not runtime refusals; `POLICY.DECLARATION_UNRESOLVED` used at rung 2	Kernel lead — register decision before L1 GA
A65	Database mirror of L0-S03	E03 §05 "exactly one of" mirrored as a CHECK; EBB EB-15 §6 gives `CHECK (irreversible OR reversal_plan IS NOT NULL)`, which admits both	Exclusive CHECK recommended (§7.6.1)	Kernel engineer, at DDL review
A66	"L0" identifiers after DR-011	O01 DR-011 retires "L0 schema"; E03 keeps rule ids `L0-S01…`, `L0-C…`, `L0-R…` and field `l0_version`	Wire identifiers kept as spelt; prose says "declaration schema"	elan1 lead; any rename is a breaking schema change under O06
A67	Audit-trail gate codes	R03 §06 (rung1 G8) raises `GATE_AUDIT_TRAIL_DISABLED`, `_GAP`, `_MUTATED`; E02's closed register has no such codes	Expressed in the kernel as a precondition predicate raising `GATE.PRECONDITION_FAILED`; rung1 names are consumer-side sub-reasons	Kernel lead with rung1 lead — before ledger1 T4 fixtures are written
A68	MCA preservation period and clause wording	IM §3.6/§10 state eight-year preservation under Rule 3(1) and the Rule 11(g) reporting duty as PUBLISHED; R03 §06 marks both "unverified — confirm with counsel"; E11 §08 declines to assert the clause reference	Stated with the counsel caveat; Regulation object must quote the instrument verbatim	Counsel, before the in-mca-audit-trail Regulation object is certified

Chapter 8

#	Topic	What the sources say	Treatment	Needs closing by
A69	Where G4 dual control produces an approval	E02 §06 lists `GATE.DUAL_CONTROL_PENDING` as a rung 3 refusal ("Request approval"), which would end the ladder at rung 3; E04 Flow 3 has all gates green at rung 3 for a ₹2,40,000 invoice above the ₹2 lakh threshold, escalation at rung 6, and G4 satisfied on resubmission	Follows E04: where an approval route exists, G4-pending is carried to an obligation; `DUAL_CONTROL_PENDING` is the refusal when none exists (build recommendation)	Kernel engineer — before rung 3/6 are implemented
A70	WhatsApp approval rights	E03 §02 block 7 has only per-channel `may_approve`; E03 §04 sets WhatsApp `may_approve: true` for `sales_invoice.submit` (one_counterparty); E06 §08/D5 allows reply-approval only for write types declared `whatsapp_approve: true`, low-value, reversible, `one_record` — a field absent from the E03 schema	Per-channel `may_approve` means approval may start on that channel via deep link; reply-approval needs a per-write-type opt-in, added as a new optional field (additive under O06)	elan1 lead with rung1 lead, Nov 2026 (E06 Q1)
A71	Door for approvals	E06 §01–§02: approvals live in Work · Approvals; R19 §04 places pending items in "Assurance → Awaiting approval"	E06 governs the kernel's Platform module; rung1's console placement is rung1's to align	rung1 lead

#	Topic	What the sources say	Treatment	Needs closing by
A72	Status of voice	O01 §03 plane P6: "Voice is parked with verna1"; R19 specifies rung1 Voice inbound approvals (Phase 3) and E03 includes `voice` as a channel kind	Inbound approval voice is a rung1 consumer channel; outbound voice remains verna1's and parked (DR-007)	Founder — O01 P6 wording
A73	Approval with amendment	E02 §03 approval `decision` is `approve · decline` only; E24 §04 counts `approved_amended` ("a human looked and changed something")	At L1 an approver cannot amend; a changed payload is a new intent by the human, and the counter derives from comparing applied and proposed payloads	elan1 lead, before E24 score function
A74	Quorum for tenant/cross-boundary radius	E04 §03 G4 needs "a second named actor"; R19 §07 speaks of changes needing "two approvers"	`quorum` field recommended (default 1; 2 for tenant/cross-boundary)	elan1 lead
A75	Pausable timers vs kernel expiry	E06 §09 item 7: timers pausable so no obligation is lost to an unperceived countdown; E04/E06 expiry is a kernel property	Single declared extension as a governed event (build recommendation)	Renderer lead with kernel engineer

Chapter 9

#	Topic	What the sources say	Treatment	Needs closing by
A76	Retention of proof records	O05 §5: audit and proof records 7 years, WORM, India. E15 §04 and decision 4: chains and anchors retained indefinitely. E11 §7: "we hold records indefinitely". IM §3.6/§10: MCA Rule 3(1) requires at least eight years; RBI draft MRM ten years	Indefinite retention with eight years as the contractual floor; O05's 7-year row read as a floor that is too short and should be amended	E11 open question 3 (per-regime retention at L1), 31 Mar 2027; O05 amendment before first external user
A77	Retention of evidence bundles	O05 §5: evidence bundles 24 months, WORM. E15 §04, decision 4: bundles are not retained; only the manifest digest and the export's governed record are kept	E15 followed; O05 row to be removed or re-scoped to the export record	Before L2 build starts
A78	Erasure salt at L1	E01 §03 and §07: "salted erasure" is an L2 capability and "no erasure salt" is a Phase 1 non-goal. E12 §02, E02 §03, E11 §2: F14 is over the salted payload, salted per record, in every L1 record; O05 Flow B destroys the subject's salt	Salt store treated as an L1 component, because F14 is frozen and written from the first record; the L1 → L2 retrofit depends on it	Salting scheme: E12 open question 1, 31 Mar 2027; must precede the first L1 record in any real tenant
A79	Signature storage on the record	EBB EB-01 sheet E02 lists `signature` as a column of the `seal` table. E12 §04: at L1 the signature member is absent, "not null … not stubbed"; a verifier fails an L1 record carrying one (negative vector set, E12 §07)	No signature, key or anchor column on `proof_record`; L2 adds separate insert-only tables; L1 has none	Before the L2 migration is written
A80	F08 and F15 on early refusals	E12 §03: no optional fields; only F12/F13 zero conventions are defined. A refusal at rungs 0b–2 precedes policy resolution (rung 4), and `POLICY.DECLARATION_UNRESOLVED` precedes lexicon resolution (E02 §05–§06). Sources are silent on the values	Recommended: `policy_version = 0` when rung 4 not reached; `lexicon_hash` = 32 zero octets when no declaration version resolved; F11 records the resolved grant (A0 if missing). Must become published vectors	Before seal implementations A and B start (W0)
A81	Tenant-stopped refusal and the record	E05 §07: `POLICY.TENANT_STOPPED` is "the only refusal without" a proof entry, and is not metered. E11 §1: every governed action produces exactly one record; E02 §05–§06: `refuse` records a full sixteen-field record	Record written (E02/E11), metered at zero; E05 row to be corrected	Before L1 GA
A82	Sandbox "mode" in the proof entry	E05 §05: "The mode field is sealed into the proof entry". E12 §02: sixteen fields, no mode field, "a seventeenth field does not exist"; enforcement mode deliberately outside the preimage (E12 §03)	Sandbox runs as its own tenant with its own chain and `tenant_id` (E05 §05 "its own chain"); mode held on `core.tenant` and rendered in every export; nothing added to the preimage	Before L1 GA

#	Topic	What the sources say	Treatment	Needs closing by
A83	Signing keys at L1	O05 §3 and §12: elan1 requires signing keys in managed KMS, per-tenant, 90-day rotation, before its first external user. E01 §07 and E12 §06: no key material anywhere at L1. IM §7.7: one key per residency domain, 12-month rotation, from L2	No keys at L1; key register created by the L2 migration; custody per Chapter 12	O05 amendment before first external user

Chapter 10

#	Topic	What the sources say	Treatment	Needs closing by
A84	When the second seal implementation is required	E07 §02–§03 and §06: a second independent seal implementation and the published test vectors are L3 additions ("L2 → L3 requires the second seal implementation agreeing with the first"). E12 §01, §08 and decision 8; E14 §06–§07 item 1; IM §7.3: no record may leave the building, and L2 is not done, until two implementations agree on all 64 vectors	E12/E14 followed: two implementations agree before any seal is shown outside the company and before L2 GA; E07 to be amended	Before the L2 build starts (WD Phase 2 01: "signed before L2 build starts")
A85	TV-001 octet count and digest	EBB EB-01 sheet and EB-05 print TV-001 at 311 octets with digest a376103f…2b280d31; E12 §07 and decision 3: under the v3.0 encoding table TV-001 is 329 octets and the 311 figure and digest are retired and may not be quoted	329 octets; the old digest is not reproduced; the v3.0 digest is written only after two implementations agree	EBB reissue now; before W0 implementation starts
A86	F14 salting scheme deadline	E12 open question 1 leaves the F14 salting scheme open until 31 Mar 2027, framed as an L2 concern. F14 is written into every L1 record (E12 §02, E11 §2), and L1's definition of done includes a real governed write to a real ERPNext site (E01 §06 item 10); E01 §07 lists "no erasure salt" as a Phase 1 non-goal	The scheme (and payload canonicalisation) must be frozen with vectors before the first real-tenant record, which is earlier than E12's date; a per-field salted-commitment construction is recommended	Before rung1's first governed write to a customer's books (Phase 1)
A87	Vector numbering inside E12	E12 §07 lists 23 negative cases for NV-01…NV-28; §04 cites "Vector NV-06" as the Ed25519-strictness vector while the listing order puts prev_digest mismatch at 6 and the strictness cases at 16–17; §06 cites PV-07 as the key-rotation boundary although rotation sits in the chain group PV-13…PV-24	Groups and counts taken as normative; individual numbers treated as provisional; five unnamed negatives to be specified	At specification freeze (E12 Flow 3 step 1)
A88	Exit class for missing-member negatives	E13 §06 P4: each of the 28 negative vectors exits 1 FAIL_INTEGRITY. E15 §06: a missing referenced declaration, lexicon or anchor member exits 3 INCOMPLETE, which is what NV-20/NV-21 ("bundle missing a referenced declaration version / lexicon") describe	E15 followed; P4 to read "exits with the class named in the vector's expected file"	Before the first profile is certified (E13 decision 9)
A89	How the record format may change	E02 decision 2: the sixteen-field layout and encoding "may not change without a new port major version". E12 §01, §09: a change is not a version of the suite at all but "a new domain string and a new verifier"; there is no v6.1	E12 followed; any change needs a new domain string, new vectors and a new verifier, and (because the record is a port object) a new port major version as well	Next revision of E02

Chapter 11

#	Topic	What the sources say	Treatment	Needs closing by
A90	Full-chain profile over an upgraded tenant	E13 §07: `in-mca-audit-trail` v1 requires `continuity_scope: full_chain`, `min_assurance_level: 2`, `applies_to_records_from: 2027-07-01`. E07 decision 2, E19 §05: records before the upgrade stay L1 forever. E15 §05: an export whose range is partly outside the profile's window is refused and split. Taken together, no tenant that began at L1 can produce a passing full-chain bundle under the profile	Profile checks (including minimum assurance level) apply only to records inside the applicability window; records outside are integrity-checked and printed as "outside profile window — integrity only"; export does not refuse a full-chain range on that ground	Before `in-mca-audit-trail` v1 is certified (E14 §07 item 6)

#	Topic	What the sources say	Treatment	Needs closing by
A91	Earliest L2 record date	E13 §07: `applies_to_records_from: 2027-07-01 (the earliest L2 record)`. E14 §01 and E07 §02: Phase 3 / L2 from Q3 2027. IM §4.9 and §10 timeline: L2 GA April 2028; ledger1 pilot from October 2027	Date treated as provisional; set equal to the first L2 record actually written (the pilot tenant's boundary), per E19	When the first tenant is upgraded (E19)
A92	Exit class of negative vectors in profile test P4	E13 §06 P4: each of the 28 negatives exits 1 FAIL_INTEGRITY. E15 §06: missing declaration, lexicon or anchor members exit 3 INCOMPLETE, and NV-20/NV-21 are missing-member cases (E12 §07)	E15 followed; P4 asserts the class named in each vector's expected file	Before the first profile is certified
A93	Verifier behaviour on a range spanning a profile boundary	IM §7.8: the verifier "refuses the single-profile run and offers the two sub-ranges". E15 §06: "a verifier does not refuse, it reports". E13 §05: the report "prints both scopes with their counts"	Splitting is an export-side refusal (E15 §05); the verifier reports, per E13/E15, and never refuses	Next revision of E13
A94	Residency refusal code at export	E15 §05 uses `GATE.RESIDENCY_VIOLATION`; the closed register in E02 §06 has `GATE.RESIDENCY_MISMATCH` and no `RESIDENCY_VIOLATION`	Use the registered code `GATE.RESIDENCY_MISMATCH`, or register the new code under E02's "adding a code takes a decision" procedure	Before L2 export is built
A95	Meter basis for an export	E15 §05: export "metered at the one_tenant basis". E02 §03 and E05 §06: the blast-radius and basis enum is `one_record · one_counterparty · batch · tenant · cross_boundary`; there is no `one_tenant`	Read as `tenant`	Next revision of E15
A96	Pilot day 1 versus the upgrade preconditions	IM §16.5 gate calendar: pilot day 0 complete by 31 Oct 2027; anchor interval "met over a measured month" by 31 Dec 2027. E18 §07: day 1 is the L1 → L2 upgrade. E19 §03 item 3 and E07 §06: the upgrade may not start until the anchor SLO is published and met over a measured month	Day 1 held until all seven E19 preconditions are true; the calendar moves, not the precondition	Before pilot day 0 (Oct 2027)
A97	Whether exports are free and unlimited	E14 §02 item 4: "Export is unlimited and free; rationing it teaches the customer not to use it"; E15 §05: the bundle is priced ₹0. IM §7.9 price book: Rs 2,500 per bundle beyond an included allowance; the IM names the tension as an open commercial item	The product imposes no export quota; any per-bundle charge is a billing-side line computed from export records, never a gate on export	Commercial decision before L2 GA (Chapter 19)

Chapter 12

#	Topic	What the sources say	Treatment	Needs closing by
A98	Signing-key granularity, rotation cadence and key publication	O05 §04, §12 and decision 4 require per-tenant signing keys, ninety-day rotation and a published JWKS; E16 §04–§05 and decisions 2 and 4 fix one key per residency domain, annual rotation with a 30-day successor, and fingerprints distributed out of band only (E15 §01: no key in any bundle)	Built to E16 (later, more specific phase document); O05 to be revised; E22's L3 issuer key document is the only JWKS-like publication	elan1 lead and stack1 lead (O05 owner), before L2 build starts (Q3 2027)
A99	Behaviour when the KMS is unavailable	E12 §06, E16 §06 (Flow 1) and E19 §04 say an unavailable custody module makes every write in the domain refuse (fail closed); E16 §07 and decision 6 say that when signing stops (compromise, or regional KMS outage preventing a successor) the domain keeps writing unsigned at assurance_level 1 and "writes never stop because proof degraded"	Build recommendation: undeclared KMS error on an active key fails closed and pages; a declared unsigned interval, opened as a dual-controlled governed write by the incident commander, lets writes resume at level 1 with the boundary rendered	Kernel engineer and founder, in the L2 runbook set before L2 GA (April 2028)
A100	Number of timestamp-authority contracts on the critical path	O09 §03 item 3 requires "a signed contract with a named timestamp authority" by 31 Oct 2027; E14 §07 item 5 and E17 decision 1 require two authorities contracted, one India-based, with a single-authority outage drill before L2 is done	Plan for two contracts by 31 Oct 2027; O09 item 3 read as the first of two	Founder (non-delegable item), 31 Oct 2027

#	Topic	What the sources say	Treatment	Needs closing by
A101	India-based versus India-resident authority	E17 §03 recommends eMudhra as the India-based authority; O09 §12 Q1 still asks whether an Indian-resident TSA meets RFC 3161 and DPDP residency together "or do we anchor outside India and explain it"; E17 Q6 asks whether any Indian auditor cares	eMudhra plus one global authority assumed; residency of the second authority is not required since only a hash and nonce leave the evidence plane (E17 §02)	Founder with elan1 kernel lead, 31 Mar 2027
A102	"No entry" in the key register for customer-held keys	E20 §07 item 7 requires elan1's own key register to show "no entry" for a customer-held key; E12 §06, E15 §02 require keys.json windows and fingerprints for every signing key a verifier must resolve	Build recommendation: register holds public fingerprint and window with an external-custody marker and no custody reference; "no entry" read as "no private-key reference"	elan1 lead, in the L3 custody specification after the L3 entry condition
A103	Legal standing of an India-licensed token	IM §13.4 says a CCA-licensed token "has its time attested under the IT Act" and labels TSA costs "MEASURED — FINANCIAL MODEL"; E17 §01 says elan1 does "not opine" on legal standing, E14 §05 bans "court-ready" and "legally admissible", and IM §7.6 labels the same costs [modelled]	E17/E14 wording governs product and sales material; costs treated as modelled; IM wording flagged for counsel	Founder with counsel, before any L2 material is issued

Chapter 13

#	Topic	What the sources say	Treatment	Needs closing by
A104	ledger1 timing vs L2	IM §5.4/§5.6: ledger1 live Oct 2027, L2 GA April 2028; E13 §07 and E18 §06 set `applies_to_records_from: 2027-07-01` ("the earliest L2 record"); E07 §02 and E11 §06 put L2 at "Q3 2027"; E18 §07 pilot requires the customer at L2; ESB Ch.11 §7: no pack ships before an auditor has verified the previous pack's bundle	ledger1 before April 2028 = L1 configuration with dry-run reports; pilot on an early L2 tenant; the 2027-07-01 date treated as a profile parameter to be reset to the first real L2 record	Founder with elan1 lead — before the Chapter 21 plan is frozen
A105	Pack composition	IM §1.4: declarations, policy rules, regulatory bindings, evaluation set, approver model; E18 §04: eleven blocks with no approver-model or evaluation-set block	Mapped in §13.1; explicit `approver_model` section recommended	elan1 lead
A106	The two unnamed rejections	E18 §02 and IM §5.2: two rejections "held in the portfolio register", not named; ESB Ch.11 §4, ERD §1.6 and EBA name them: hr1 (G-A) and "a general AI governance product" (G-E)	Named, citing the product-folder sources at lower weight	IM text should be corrected
A107	Reversal vocabulary in ledger1	E18 §06 uses `compensating_instrument`, not in E03's `reversal_plan.method` enum; `sales_invoice.cancel` described as reversible for 24 h then irreversible but tabled as irreversible	`compensating_entry`; cancel as `reversal_plan` with `after_window: not_reversible`	Pack author, before T1
A108	Retention clause	IM §1.4/§10: Rule 3(1) requires eight-year preservation; E13 §07 profile binds four clauses and no `retention` row	Preservation engineered (96-month payload floor) and a Phase 4 `not_measurable` retention row; period subject to counsel (R03 §06)	Profile author with counsel, before ledger1 certification
A109	IRN request radius	E03 §04/E18 §06: IRN requested within `sales_invoice.submit` at `one_counterparty`; R03 §07: pushing a payload to the IRP is `cross_boundary` (dual control + proof binding); R03 also names the smallest radius `self`, not E03's `one_record`	Separate `e_invoice.request_irn` at `cross_boundary` in invoice1; E03 enum governs	elan1 lead with rung1 lead, before invoice1 build
A110	collect1 irreversible dials	E27 §02: `call.attempt` irreversible, automated; E03 §02, E01 §02: irreversible bars auto-approval; E05 §11: no reversal plan means no A3+	Dual-controlled `dial_list.approve` precondition recommended; requires a recorded decision	Founder — before collect1 enters the build queue
A111	collect1 gate count	E27 §03: fourteen gates; IM §5.6: nine gates, "a new regime parameterises them rather than adding to them"	Fourteen expressed on nine gates plus rules and write-type preconditions (§13.7.3)	elan1 lead, at collect1 admission
A112	collect1 refusal codes	E27 §05: sixteen reasons "drawn from verna1"; E02 §06: closed register of 43 with none of them	Map to register; propose a few new gate-class codes by register decision	Kernel lead, 30-day notice per E02 §07

#	Topic	What the sources say	Treatment	Needs closing by
A113	collect1 runtime	IM §5.5: collect1 Oct 2028; O01 DR-007: verna1 (outbound voice) parked until a paid NBFC pilot; R19 §01: rung1 Voice inbound only	collect1 date conditional on a voice runtime and carrier partner	Founder
A114	First-pack choice	ESB Ch.11 §7: first pack ledger1 (then SEC 17a-4) or trace1; E18 §05: ledger1 against MCA	E18 governs; SEC 17a-4 is a later profile under ledger1	Closed by E18

Chapter 14

#	Topic	What the sources say	Treatment	Needs closing by
A115	L3 design work before the entry condition	E20 §08 forbids any L3 work before an outside auditor accepts an L2 bundle, "including design work, including a slide"; WD says each L3 specification is "only started once L2 evidence has been accepted outside the company"; yet E22, E23 and E24 exist (21 Sep 2026) with open questions dated Q4 2027–Q3 2028, before Gate 3 (end Q4 2028) can close	E22–E24 treated as frozen specifications that fix L2-facing choices; no further L3 design, staffing or commitment before entry; open-question dates to be re-based to after entry	Founder, at the next document revision and before Phase 3 entry (Q3 2027)
A116	When L3 starts	E20 §01 says "Phase 4 runs from 2028"; O12 §02 shows Phase 4 "2028 on"; IM §8.1 and §9 model L3 entry in October 2029, after Gate B (July 2029)	IM date used for planning (modelled); 2028 references read as the earliest possible start, conditional on the entry event	Founder, in the plan of record before Series A close (Mar 2027)
A117	Whether the Series A funds L3 and the standard	O12 §09 tranche table: "D — L3, the second seal implementation, the Action Receipt standard — Not this money"; IM §1.8/§8.1 Gate B (tranche 3, July 2029) requires Action Receipt v1 published with at least two named external implementers, and the plan carries L3 entry, sovereign deployments and a standards function inside the horizon	Treated as funded by tranche 3 of the elan1 Series A per the IM; O12 read as the group narrative written before the elan1 entity's round was sized	Founder with counsel, before any term sheet
A118	Redaction mechanism in product documents	ETS §06/§13 (SD-JWT RFC 9901 library in the bundle service, wave W6), ETA ("Redaction by SD-JWT (RFC 9901)"), EPA and ERD specify SD-JWT for redaction; E23 §03 adopts SD-JWT's construction in elan1's LP encoding and rejects the SD-JWT JSON container as reintroducing JSON canonicalisation into the digest path	Built to E23; no SD-JWT library in the digest or bundle path; L2 redaction remains per-export redaction_profile	elan1 lead, when the W6 tooling ADR is written
A119	Supervised-period sampling key	E24 §05 samples 1 in 10 writes "chosen by the record's own digest", but the routing decision is taken at rung 6, before the record and its digest exist	Build recommendation: key the sample on the digest of the canonical write intent available at rung 6	elan1 lead, with the score thresholds (Q2 2028, re-based after entry)
A120	Scope of graduation	WD precedent brief outlines "Graduation: A0 to A5"; E24 §05 permits graduation only A2 → A3 in Phase 4, with A4/A5 as declaration decisions with an ADR	Built to E24	elan1 lead; WD superseded
A121	What counts as the "second independent seal implementation"	E14 §07 item 1 and IM Gate A require two independent implementations agreeing on 64 vectors at L2 GA (April 2028); ETS §06 plans Seal A (Python) and Seal B (TypeScript); E20 §07 item 2 requires, for L3, a second implementation "by a team with no access to the first's source" on a full production chain, and E20 Q3 asks who writes it	Build recommendation: staff Seal B as a clean-room team from the start so it can satisfy both; otherwise L3 needs a third implementation	elan1 lead, before Seal B is staffed (Phase 3 entry, Q3 2027)

Chapter 15

#	Topic	What the sources say	Treatment	Needs closing by
A122	When Action Receipt v1 is published	IM §16.5.2 Gate B item 3 requires v1 published with two named external implementers (one producing a receipt elan1's verifier accepts) measured at 30 Jun 2029; IM §9 says "publication belongs to Phase 4" with L3 opening Oct 2029; E20 §08 forbids any L3 work, including design, before the entry condition; E25 §07 requires a 60-day comment period and two implementations before a version ships	v1 track starts the day the entry condition is met (itself Gate B item 1), leaving at most two quarters; Oct 2029 read as entry for the other L3 capabilities; the compression is flagged as a Gate B schedule risk	Founder, in the plan of record before Series A close (Mar 2027)
A123	"Certified" partner firms versus no certification in v0	IM §9, §11.6, §16.4 and §16.5.2 (Gate B item 4: 28 "certified partner firms") and WD overall Phase 3 02 ("trained and certified", "certification levels") describe certified partners; E25 §08 and decision 6 say there is no certification body or mark in v0 and elan1 "may not certify" an implementation	Build recommendation: partner "accreditation" for commercial delivery, kept separate from self-asserted conformance; rename before external use	Founder, before the first partner agreement (plan: 6 firms by Mar 2028)
A124	Serialisation of the receipt: COSE/SCITT profile or elan1 suites	IM §9 recommends "in its own voice" that v1 be a SCITT profile on COSE with disclosure as a v1 suite; E25 §02 specifies elan1-seal-v6 / rfc3161-6962 / elan1-sd-v1 with LP encoding and no COSE; E23 §03 bans JSON containers in the digest path	E25 governs; a COSE-profiled seal suite treated as a registry entry to be proposed in the v1 comment period, never a change to elan1-seal-v6	elan1 lead, before the v1 draft is published
A125	OpenTelemetry identifiers as "first-class receipt fields"	IM §9 and §13.8 say the receipt carries OTel GenAI trace and span IDs as first-class fields; E25 §02 fixes the member list and E12/E20 forbid a seventeenth sealed field	Build recommendation: commit OTel IDs through F14 as declared proof_binding fields; no new sealed field or receipt member	elan1 lead, with the declaration template for packs (before L2 GA, Apr 2028)
A126	Missing envelope-signature member	E25 §02 rule 4 and §03 step 9 require a separate envelope signature over digest, audience, nonce and expiry; the E25 member table (13 members) has no member to carry it	Build recommendation: add envelope_signature as a CL-4 member	elan1 lead, before the v1 comment period
A127	Certification spend before the certification trigger	E26 §02 and decision 2: certify only to the standard the first paying evidence customer's auditor names, never speculatively (Gate 3 is end Q4 2028); IM §10 plans Rs 0.66 crore certification and audit spend in FY2027-28	FY2027-28 line read as readiness work on the three ISO 42001 gaps and quotes, not an audit engagement	Founder, with the Q1 2028 quotes
A128	Which certificate the plan targets	WD overall Phase 3 01 asks for "a dated plan for … holding ISO/IEC 42001"; E26 §02 says the first certificate is whichever standard the first evidence customer's auditor names, ISO/IEC 42001 or SOC 2 Type I/II	Built to E26; the dated plan in §15.14.1 carries both branches	Founder, when the group Standard and Certification Plan is written

Chapter 16

#	Topic	What the sources say	Treatment	Needs closing by
A129	L2 availability date	E01 §08, E07 §02–03, E09 §08 and E11 §06 say L2 is "Q3 2027" (E09 tells refused applicants "It is Q3 2027"); IM §16.5 has L2 signing live 30 Sep 2027 and L2 GA 30 Apr 2028; E14 says Phase 3 runs Q3 2027–Q1 2028	Q3 2027 = start of L2 build and internal signing; 30 Apr 2028 = GA and the only date given to customers. E01/E07/E09 customer-facing sentences to be amended	elan1 lead, before L1 GA copy is published (30 Jun 2027)
A130	L3 start date	E07 §02 "Phase 4, 2028 onward"; E20 §01 "Phase 4 runs from 2028"; E20 §02 says L3 begins on an event, not a date; IM §16.5 L3 entry 31 Oct 2029 with the condition required by Gate B at 30 Jun 2029	Event-gated; 31 Oct 2029 shown as a modelled plan date only; "2028" removed from customer copy	Founder, at E07 re-issue
A131	Second seal implementation: L2 or L3	E07 §02–03 lists "a second independent seal implementation" as an L3 addition; E12, E14 §06–07 done-item 1, and IM §16.5 Gate A require two independent implementations agreeing on all 64 vectors before any L2 bundle is shown	Two-implementation rule applies from L2; L3 adds agreement on a full production chain by a team with no access to the first's source (E20 done-item 2). E07 table to be corrected	elan1 lead, before the 30 Sep 2027 signing gate

Chapter 17

#	Topic	What the sources say	Treatment	Needs closing by
A132	Who executes the write	S03 §4 Flow A and S15 §05 have the consumer's fusion.run/v1 workflow call the ERPNext MCP tool via fusion.tools/v1 with a sealed credential after approval, and elan1 writes the proof afterwards; E01 §02, E04, E05 §02 and IM §6.10 have the kernel's single writer at rung 7 mint the credential, apply to the shadow sandbox, re-run gates and apply to live	E-series prevails for Enforced mode: rung 7 single writer (a kernel-owned Temporal activity) performs the write; the S03/S15 flow describes Attested advisory callers only. S03 §4 and S15 §05 to be amended	elan1 lead and stack1 lead, before the first real governed write (31 Dec 2026)
A133	stack1 as an in-process consumer	E05 §01 says rung1 and stack1 link @elan1/kernel in-process; S03 B1/B2 forbid any stack1 import of or manifest reference to an elan1 package, and S03 §2 says the kernel is never linked into stack1	stack1 tooling is a Mode B consumer; the Mode A SDK is an in-process client library and rungs execute in the kernel service (build recommendation), preserving the single writer	elan1 lead, at E05 sign-off (Oct 2026)
A134	Compute substrate and secrets store	ETS §07 recommends ECS Fargate and AWS Secrets Manager for elan1; STS, S12 and S15 run every consumer on Kubernetes namespaces with OpenBao paths, and the kernel is hosted as P4	Kernel runs on stack1 Kubernetes (P4) with OpenBao; Fargate only if used for the Phase A walking skeleton	elan1 lead with stack1 lead, with OD-28 (before stage build-out)
A135	Disaster-recovery region	ETS §07 recommends ap-south-2 Hyderabad as an in-domain DR region; O05 §02, S03 decision 7 and S17 decision 7 forbid any second region, including for DR, until a written counsel-confirmed residency decision, and forbid cross-region failover	Single region, restore in place; ap-south-2 only after a written residency decision	Founder with counsel, before first external tenant (Jul 2027)
A136	Latency of the proof rung and the envelope	E04 §02 ≈64 ms kernel p50 with rung 8 at 12 ms; E10 §07 ~91 ms with rung 8 ~40 ms; O03 §08 and S17 §9 envelope p50 ≤120 ms, p99 ≤450 ms, proof write p50 ≤40 ms	E04 as design budget, O03 as ceiling; L2 rung-8 budget set from a measured KMS Sign call	Kernel engineer, at Gate 1 measurement
A137	Proof store location and trace retention	ETS places the evidence plane (Postgres + S3 Object Lock) in elan1-owned per-domain zones; S03 OQ2 leaves open whether it lives in stack1's evidence plane; S02 OQ4 asks whether 90-day trace retention suits proof binding	Proof records in elan1-owned storage inside the P4 deployment; proof never depends on a trace span (build recommendation)	elan1 lead and stack1 lead, 31 Oct 2026
A138	Tenancy mapping and hosted capacity	S16 decisions 2–3 cap stack1 Cloud at 25 tenants and sell Standard+ customer-VPC; S15 sizes P4 at 1–50 consumer tenants; IM §16.4 plans 148 elan1 tenants by Mar 2029 and 545 by Mar 2031 on hosted domains; E10 assumes ten tenants per shared domain	One elan1 residency domain = one stack1 consumer tenant on P4; elan1 tenants isolated inside it by RLS, per-tenant chains and leases (build recommendation)	elan1 lead and stack1 lead, before first external tenant (Jul 2027)
A139	Sovereign deployment sequencing	E14 names it L3 or later; E20 §08 forbids promising L3 capabilities before the entry condition and forbids selling sovereign before a defect-propagation process exists (no plan date); IM §16.5.2 Gate B requires a signed sovereign or on-prem order by 30 Jun 2029 and IM §8.5 models the first deployment in Jul 2029; WD labels the guide "Phase 3"	Treated as an optional deployment profile; defect-propagation design given as build recommendation; Gate B condition 6 flagged as dependent on E20 preconditions	Founder, before tranche terms are fixed (Feb 2027)

Chapter 18

#	Topic	What the sources say	Treatment	Needs closing by
A140	Signing-key granularity	O05 §04 and §12 ("per-tenant") and S12 §1, §3 ("per-tenant non-exportable Ed25519 signing keys") vs E16 §04 and IM §10 ("one signing key per residency domain, not per tenant")	E16 prevails for elan1 domain keys; per-tenant key only via a dedicated domain (H2) or customer-held keys (L3); O05 wording to be amended	elan1 lead and stack1 lead, before the key-custody gate (31 Aug 2027)
A141	Signing-key rotation period	O05 §04, S12 §3 and IM §10: ninety-day rotation; E16 §02, §05: domain key every 12 months, successor registered 30 days ahead	E16 annual rotation for elan1 domain keys; stack1's own keys keep 90 days	elan1 lead, before the key-custody gate (31 Aug 2027)
A142	Signing keys in the go-live bar at L1	O05 §12 requires elan1 signing keys in managed KMS before the first external user; E01 §07 and E16 §01 forbid any key material at L1, and L1 GA precedes L2	Row read as not applicable until L2; O05 §12 to be amended	stack1 lead (O05 owner), before L1 GA (30 Jun 2027)

#	Topic	What the sources say	Treatment	Needs closing by
A143	Erasure salt at L1	E01 §07 non-goal "no erasure salt"; E19 §02 salt retained from L2; E12 F14 defined over the salted payload and frozen at L1; O05 §11–12 require salt-destroying erasure tested before the first external user	Generate and store a per-record salt from the first L1 record in the P0 salt store (build recommendation); E01 non-goal narrowed to "no salt-based erasure claim at L1"	elan1 lead with counsel, with the record-shape freeze (30 Nov 2026) and no later than 13 May 2027
A144	Token classes	O03 §06 and O05 §03: "three token classes and no fourth" — tenant, control, service; E05 §05: tenant, control, ops	Four classes (tenant, control, service, ops), each with its own audience; "no fourth" rule amended (build recommendation)	elan1 lead and stack1 lead, at O03/E05 sign-off (Oct 2026)
A145	Proof-record retention	O05 §06: audit and proof records 7 years; MCA Rule 3(1): 8 years (IM §10); RBI draft MRM: 10 years; E10 §07 costs a ten-year carry; E11 §06: records held "indefinitely"	Per-tenant retention at the longest bound regime, never below 8 years for ledger1 tenants (build recommendation); O05 row to be corrected	elan1 lead with counsel, before ledger1 production (31 Oct 2027)
A146	Break-glass recording	E05 §05, E09 §09 and E16 §08: break-glass is a declared governed write with dual control, recorded in the chain and visible to the tenant; S12 §5: a stack1 support engineer's break-glass read is recorded only as an event in the tenant's next attestation	Any break-glass touching kernel data is an elan1 governed write in the operator chain, in addition to stack1's attestation entry	elan1 lead and stack1 lead, before L1 GA

Chapter 19

#	Topic	What the sources say	Treatment	Needs closing by
A147	Quantity-zero CHECK form	EBB EB-02 and EPA write `CHECK (outcome <> 'refused' OR quantity = 0)`, which allows a non-zero escalated row; E05 §07 says escalations meter zero and E02 §03 says quantity 1 only for a committed write	Constraint written positively: non-zero only when committed, production, governed_action (§19.5.2)	elan1 lead, before meter go-live
A148	Inference in cost per governed action	O07 §06 lists "inference at rung 4 evaluate() — the only rung that calls a model" as component 1; E02 §05/C1 and E01 §02 item 2 forbid any model in rungs 3–8 and describe rung 4 as model-free rules	Kernel inference cost is zero by construction; O07 component 1 read as the consumer's upstream inference attributed via run_id (§19.13.2)	Founder / elan1 lead, O07 revision
A149	Cost-per-GA denominator	O07 §06: committed actions only; R13 §6: all governed actions including refusals, PLATFORM excluded	Both computed from the same rows and labelled; O07 committed-only as headline (§19.13.3)	Founder, before first publication
A150	Two price books	E10 §03/§06: six monthly tiers (₹40,000–₹10,00,000+ floor, ₹1.50–6.00 overage), verifier profile ₹6,00,000, bundle export ₹0, L2 add-on ₹2,00,000/mo; IM §14.5 and Appendix A: annual floors by assurance level (L1 ₹1,80,000), overage ₹1.60/1.20/0.90, verifier profile ₹3,00,000, bundle ₹2,500 beyond allowance	Rating built on effective-dated price-book tables supporting either; neither hard-coded (§19.10.3)	Founder, before L1 GA (Q2 2027)
A151	Transfer-price review cadence	E10 §04 rule 3 and decision 4: reviewed once a year, ₹0.40 through Phase 2; O07 §05 and decision 3: reset quarterly against measured cost	Effective-dated price row supports either; quarterly memo recommended for the related-party file (§19.12)	Founder, 31 Dec 2026 (with O07 Q1)
A152	Evidence SKU start date	E10 §06 and E09 §08: evidence from L2 "Q3 2027"; E07/brief and IM: L2 GA April 2028, signing live 30 Sep 2027	Evidence metering and entitlements treated as L2 scope, dated to L2 GA; no evidence line before L2 GA	elan1 lead
A153	Export meter basis spelling	E15 §05: "one_tenant" basis; E05 §06 enum value is "tenant"	Read as `tenant`; unknown basis values rejected at define	elan1 lead, E15 revision
A154	Kernel latency assumption in cost model	E10 §07: ~91 ms p50 with ~40 ms at rung 8; E04 §02: ≈64 ms kernel total with 12 ms at rung 8	Both unmeasured; Gate 1 measurements replace both	elan1 lead, E01 Q2 (30 Nov 2026)
A155	Platform refusal meter event	E02 §04 and C8: every refusal, including platform class, has a record and a quantity-0 meter event; E05 §07 error taxonomy: platform failure meter "no"	Emit quantity-0 event with the PLATFORM code so C10 reconciles exactly (§19.3)	elan1 lead, before meter go-live

#	Topic	What the sources say	Treatment	Needs closing by
A156	Metering of approvals, define and exports	E02 §03/§05 and E15 §05 call them governed writes; no source says whether they meter; R13 §1 says the approval step must never be charged	New meter_class with quantity 0 (§19.3)	elan1 lead, before meter go-live

Chapter 20

#	Topic	What the sources say	Treatment	Needs closing by
A157	Compute platform	ETS §07 and ETA recommend ECS Fargate on AWS; STS §02/§07 fixes managed Kubernetes per region, and S03 §02 makes elan1 a consumer of the stack1 runtime	elan1 production runs on stack1 Kubernetes in ap-south-1; Fargate dropped	elan1 lead with stack1 lead, before stage (OD-28)
A158	Secrets	ETS §07 recommends AWS Secrets Manager; STS §06 fixes OpenBao and S03 decision 5 makes secrets stack1's non-substitutable code; E05 §02 requires a secrets broker minting per-write credentials	stack1 secrets broker on OpenBao	elan1 lead, before first production key
A159	Evidence object store	ETS §05 recommends S3 Object Lock; STS §05 makes SeaweedFS/Ceph/Garage load-bearing and allows S3 only in stack1 Cloud	S3 Object Lock in hosted mode; Garage/Ceph/SeaweedFS in sovereign and air-gapped modes	Evidence and cryptography lead, before L2 build
A160	Workflow engine	ETS omits Temporal and routes the write path through a synchronous kernel plus outbox relay; DR-002 and S04 require every governed write inside a Temporal workflow	Temporal for consumer runs, approval signals and the anchor job; rungs 0b–9 synchronous inside one activity (§20.2.4)	elan1 lead with stack1 lead, before rung 7 build
A161	Signing and key custody at L1	ETA, EPA and ERD §7.1/§7.4 seal with Ed25519 at rung 8 from the first write and require HSM keys before first seal; E01 §03/§07 and E12 §09 forbid signing and key material at L1	L1 unsigned hash chain; KMS at L2, HSM at L3	Closed by E01 (record only)
A162	Verifier and vectors as first epics	EBB EB-10 puts seal spec, vectors and verifier on the W0 critical path; E01 §07 excludes the verifier at L1 and IM §16.2.2 limits E12 to specification in Nov 2026	Spec and vectors drafted; nightly integrity checker at L1 in Go; offline verifier at L2	elan1 lead, Nov 2026
A163	TV-001 target	EAP §05 and ERD §4.7 target TV-001 at 311 octets with a known digest; E12 §07 and decisions 3–4 retire 311 octets and the digest (329 octets under v3.0)	Any Agenthon artefact reproducing 311 octets is discarded; v3.0 digest withheld until two implementations agree	Evidence and cryptography lead, before L2 build
A164	Voice ownership	ETS §08 and EBB EB-10 route voice via verna1 CarrierPort; DR-007 parks verna1 and O09 items 9–10 make inbound voice rung1 Voice on P6 with a kernel-defined `voice` kind in block 7	Kernel supplies the block-7 channel kind; rung1 Voice builds the channel	elan1 kernel lead, 30 Sep 2027
A165	Repository perimeter	EBB EB-09 §2 puts 20 apps and 10 packs in one elan1 monorepo; O03 §12 gives apps to consumers and the kernel repo to P3	elan1 repos hold kernel, declarations and pack configuration only	elan1 lead, Oct 2026
A166	Licence register coverage	O04 §03 lists 25 rows; PostgreSQL, Go, pyca/cryptography, MCP SDK, React, Grafana stack and SD-JWT are absent though they are in elan1's path	Rows to be added at pinned SHAs before any paying customer	Engineering leads, next quarterly review
A167	Proof record location	S03 Q2 leaves open whether proof records live in stack1's evidence plane or elan1-owned storage; O11 §02 puts P3 and below in the holding company	elan1-owned Postgres instance inside the stack1 footprint	elan1 lead, stack1 lead, 31 Oct 2026

Chapter 21

#	Topic	What the sources say	Treatment	Needs closing by
A168	Gate 1 definition and date	EBB EB-12 §1–§2 and ERD §8.1: end Q2 2027, a walking skeleton (one app, one pack, one bundle) verifies offline; O09 §07 decision 3 and IM §16.3.4/§16.5: 31 Jan 2028 (O09 also says "end Q1 2028"), a paying customer's write verifies offline	O09/IM Gate 1 on 31 Jan 2028 with the raised bar	Closed by O09 decision 3 (record only)

#	Topic	What the sources say	Treatment	Needs closing by
A169	Build order: seal and verifier first vs L1 first	EBB EB-10 §2/§10 and EB-14 §5 put the seal spec, 64 vectors and offline verifier on days 1–75 and forbid "anything before E01"; E01 §03/§07 exclude signing and the verifier at L1 and IM §16.2.2 limits E12 to specification in Nov 2026	L1 kernel first; preimage and digest vectors frozen before rung 8; nightly integrity checker as the L1 second implementation; seal A/B and verifier built in Phase 3	elan1 lead, Oct 2026
A170	Agenthon scope and TV-001	EAP §05/§07 and ERD §4.7 target seal A/B and a verifier, with TV-001 at 311 octets and a known digest as Day 1's exit; E12 §07 and decisions 3–4 retire 311 octets and the digest (329 octets under v3.0; digest withheld until two implementations agree)	Harness and practices kept; 311-octet artefacts archived; Agenthon seal/verifier authors not used as the independent second implementer of code they have read	Evidence and cryptography lead, before Phase 3
A171	Team size and shape	EBB EB-14 §1: 3–4 founding four growing to 30–48 across waves W0–W11; IM §15.4/§16.2: 11 in Oct 2026, 27 in Mar 2027, 95/170/243/305 to Mar 2031 across ten functions	IM people plan	Closed by IM (record only)
A172	Gate 2 ownership	O09 §07 and IM §16.5 keep Gate 2 (app three renders with zero screen code) as an elan1 gate owned by product and design; O03 §10/§12 give apps and screens to consumers and do not assign the surface compiler (P5) to an entity; O11 §02 puts only P3 and below in the holding company	Treated as an elan1 gate on the shared renderers; app three built by a consumer team; entity ownership of P5 open	Founder, 30 Jun 2027 (with O09 Q4)
A173	Signing keys in the Jan 2027 baseline	O05 minimum bar and IM §16.2.4: tenant signing keys in a managed KMS with 90-day rotation, closing Jan 2027; E01 §07: no key material at L1; E16 §02: per-domain signing key rotating every 12 months, at Phase 3	No signing keys at L1; the O05 row is satisfied at L1 by token-signing keys only and applied to record signing from L2 under E16	elan1 lead with security owner, Jan 2027
A174	First real write scope	IM §16.2.3 describes `assert` running rungs 0b–6 and returning "needs_approval", then rung 7 executed by the kernel; E02 §05 has assert return committed, refused or escalated after all nine rungs	E02 outcome names and rung sequence	elan1 lead, Oct 2026

Chapter 22

#	Topic	What the sources say	Treatment	Needs closing by
A175	Kernel behaviour when signing is unavailable (L2)	E16 §06 and IM §6.10: KMS unavailable at a key cut instant → every write in the domain refuses. E16 §07 step 2 and decision 6: on suspected compromise the domain stops signing and keeps writing unsigned at assurance_level 1	Both adopted as distinct states: involuntary KMS outage = fail closed; declared compromise = recorded downgrade with a declared unsigned interval. Chain-writer state machine must specify both	Before L2 build starts; with E16 OQ1, 31 Mar 2027
A176	Name and handling of a write applied but not yet recorded	R12 RB-4: SEAL_PENDING, P1 after 5 min. O03 §03: acted_unproven, alarms, never silently completed; O03 §04 acted_unattested for Mode B. E02 §05: outbox row survives and chain writer retries in order, "no gap"	Kernel term (unconsumed outbox row) used; paging threshold = outbox-to-record lag SLO; acted_unattested kept for Mode B only	O03 OQ3, 15 Oct 2026
A177	Signing-key rotation cadence and scope	O05 §3 and decision 4: 90-day rotation, per-tenant signing keys in KMS; IM §18.1 repeats 90 days. E16 §04–05 and decisions 2, 4: annual rotation, one key per residency domain, not per tenant	E16 adopted for runbooks (annual, per domain, dedicated domain for an exclusive key); O05 checklist rows must be amended	Before L2 GA; E16 OQ3 (30 Sep 2027)
A178	Proof-record and bundle retention	O05 §5: proof records 7 years WORM, bundles 24 months WORM. R12 §6: proof chain 7 years. E15 §04: chain and anchors retained indefinitely; bundles not retained at all. E10 §07: ten-year carry. MCA audit-trail floor served by ledger1 is 8 years (IM §6.7, §11.5)	Indefinite retention of chain and anchors (E15) treated as operative; bundles not retained; O05 and R12 rows to be amended	Before L1 GA; with E11 OQ3 (31 Mar 2027)
A179	Meter reconciliation discrepancy severity	EBB EB-11 §8: meter-line divergence is a billing incident and a ticket. E09 §07: a non-zero discrepancy at period close stops the invoice and pages	Nightly finding = ticket (P2); period-close block = page (P1)	Before L1 GA

#	Topic	What the sources say	Treatment	Needs closing by
A180	RPO and RTO	O05 §6 and OQ1: not yet set for any brand. R12 §6: rung1 states RPO 15 min, RTO 2 h	This memorandum proposes per-store targets (§22.8.2), aligned to R12 for the meter and snapshots, marked build recommendation	O05 OQ1, 15 Oct 2026
A181	Kernel latency model	E04 §02: ≈ 64 ms p50 kernel total, rung 8 at 12 ms. E10 §07 and EBB EB-11 §6: ≈ 91 ms, rung 8 ≈ 40 ms. O03 §08: envelope p50 ≤ 120 ms / p99 ≤ 450 ms, proof p50 ≤ 40 ms	O03 envelope used as the SLO; E04 and E10 figures shown as unreconciled estimates	Gate 1 measurement; E04 OQ1

Chapter 23

#	Topic	What the sources say	Treatment	Needs closing by
A182	Model at rung 4	O01 §02 Right Path table: rung 4 evaluate() "the only rung at which a model may be called — Yes"; "eight of the ten rungs make no model call". E01 §02 #2, E02 I1, E04 rung 4, E24 rule 2: no model in the decision path at any level; rungs 3–6 and 8 unreachable from a model	E-series followed; O01 row treated as a defect to be superseded by a numbered register entry	Before first external demonstration; next O01 revision
A183	Who may call seal	E02 §3 and decision 3: seal callable only by the kernel, never by a consumer. O03 §04 Mode B: caller reports its outcome "through seal"; O03 §05 lists seal with rw scope	Kernel remains sole chain writer; Mode B needs a distinct outcome-report call	Before Mode B build; with O03 OQ4 (30 Nov 2026)
A184	L2 date in customer-facing text	E01 §08, E09 §08, E10 §06: L2 "Q3 2027". E14 §01: Phase 3 runs Q3 2027–Q1 2028. IM §4.9 and §18.2: L2 GA April 2028	April 2028 GA used; Q3 2027 read as Phase 3 start; E09 refusal script and E01 claim table to be amended	Before L1 GA support scripts are published
A185	Level at which verifier profiles and offline verification arrive	O01 §02 Assurance Levels: L2 = seal, anchoring, bundle; L3 = verifier profiles, offline verification, Action Receipt standard. E07, E14 §07 and E15: offline verifier and one certified profile are L2 done-items	E07/E14 followed (L2)	Next O01 revision
A186	Refusal code naming	O01 §03 and DR-011: refusal register becomes REF-nn; O03 §03–04 uses "REF code". E02 §06 and E05 §07: wire spellings are POLICY.* / GATE.* / PLATFORM.*, fixed and never renumbered	E02 wire spellings used; REF-nn treated as a document index only	Before the public refusal pages (E09 row 1)
A187	Default obligation / approval expiry	E04 walkthrough and E06 OQ3: 4 hours, asserted. O03 §03: a proposed action with no approver configured "stays open for 24 hours"	4 h used as the obligation default; O03's 24 h treated as the no-approver case, pending E04 OQ3	First design partner
A188	stack1 GA contract set, on the L2 critical path	S13 §01 and decision 1: GA is nine contracts, fusion.memory/v1 limited availability, GA 1 Oct 2027. IM §16.6 item 11: stack1 GA = all ten contracts blocking-green by 30 Sep 2027	S13 followed for stack1 GA; the elan1 dependency is only fusion.run/v1 (item 4) and key custody topology (item 2)	stack1 go/no-go, 17 Sep 2027
A189	First verifier profile deadline	Q2 2027 (E07 OQ2), 30 Jun 2027 (E11 OQ2), 31 Mar 2027 (E13 OQ1); IM App. E.2 notes the three dates	Earliest date (31 Mar 2027) used for planning	31 Mar 2027
A190	CA reading of an unsigned L1 record	E01 OQ1: due 31 Oct 2026. E11 OQ1: same question due 31 Dec 2026	Earlier date used	31 Oct 2026
A191	F14 salting scheme	E02 OQ2: due 30 Nov 2026. E12 OQ1: due 31 Mar 2027	Earlier date used, since the scheme must be fixed before L1 writes real records	30 Nov 2026
A192	Ownership of the TSA relationship	IM §16.6: the TSA contract "is not delegated, at any point, to anybody". O11 §08: the TSA relationship is delegated to the elan1 kernel lead after Phase 3, once the first contract is signed	Contract negotiation founder-owned; operation of the relationship delegable after signature	Before the first TSA contract, 31 Oct 2027
A193	Platform-absorption watch coverage	IM §12.7 names AWS AgentCore's boundary and SAP/Oracle signed write gating as the existential scenario. O08 §03 has no AgentCore, Google or SAP/Oracle row	Rows recommended for addition (build recommendation)	Next quarterly watch, Jan 2027

Appendix B. Source documents

All sources are held in the "elan1 project details" folder, except the Information Memorandum, which is held in the "Elan1 Full Product Plan" folder. The phase documents are in the *elan1 details*, *overall details*, *rung1 details* and *stack1 details* folders. The product-folder documents are in *elan1 product* and *stack1 product*, and the strategy and research documents are at the top level of the folder. Where a product-folder or strategy document disagrees with the E-, O-, S- or R-series, the series document prevails and the conflict is recorded in Appendix A.

elan1 documents

ID	Title	Phase
E01	Scope Freeze - elan1 L1 Guarded	1
E02	Kernel - Layer Contract v1	1
E03	Declaration Spec v1 - the elan1 Ontology	1
E04	Right Path Runtime Flow Book	1
E05	MCP and SDK Interface Spec	1
E06	Console and Approval UX Flow Book	1
E07	Assurance Level Definition	1
E08	Competitive Matrix v3	1
E09	L1 GA Readiness and Third-Party Onboarding	2
E10	Pricing Model v1	2
E11	Evidence and Audit Note	2
E12	Seal Spec v6 and Test Vectors	2
E13	Verifier Profile Spec	2
E14	Scope Freeze - L2 Attested	3
E15	Evidence Bundle Spec and Offline Verifier	3
E16	Key Management and Custody	3
E17	Timestamp Authority Memo	3
E18	Pack Definition Template and the First India Pack	3
E19	L1 to L2 Upgrade Runbook	3
E27	collect1 Pack Specification Sketch	3
E20	Scope Freeze - L3 Proven	4
E21	Declaration Full Spec - Controls and Regulations	4
E22	Cross-Organisation Verification Spec	4
E23	Selective Disclosure Spec	4
E24	Precedent and Earned Autonomy Spec	4
E25	Action Receipt - Open Standard Draft	4
E26	Certification and Regulator Engagement Dossier	4
WD	Working-draft briefs for the elan1 and group documents (Phase 1–3 markdown notes)	1–3

Group (overall) documents

ID	Title	Phase
O01	Vocabulary and Decision Register	1
O02	Portfolio and Gates Memo	1
O03	Three-Brand Integration Contract	1
O04	Open-Source Licence Register	1
O05	Security, Residency and DPDP Baseline	1
O06	Version and Compatibility Policy	2
O07	Shared Pricing and Metering Model	2
O08	Group Competitor Watch	2
O09	Cross-Product Roadmap and Dependency Map	3
O10	Brand Architecture and Naming Standard	3
O11	Group Operating Model	4
O12	Investor and Board Narrative Pack	4
P1-00 to P4-00	Phase 1–4 Document Indexes	1–4

stack1 documents

ID	Title	Phase
S01	Scope Freeze and Customer Zero	1
S02	Internal Platform Contract (Phase 1, revised in Phase 2)	1–2
S03	Runtime Architecture and Boundary Note	1
S04	Workflow Engine Decision Record	1
S05	Operator User-Flow Book	1
S06	Conformance Suite Spec	1
S07	Competitor Comparison v3	1
S08	Price Decision Memo	2
S09	Day-2 Runbook Set	2
S10	Substitution Protocol Spec	2
S11	External Pilot Pack	2
S12	Security and Tenant Isolation Note	2
S13	GA Readiness Checklist	3
S14	OEM and Systems-Integrator Partner Pack	3
S15	Reference Architectures by Profile	3
S16	Cost and Capacity Model	3
S17	Scale and Multi-Region Plan	4
S18	Spin-Out or Absorb Decision Memo	4
S19	Open-Core Release Plan	4

rung1 documents (the kernel's first consumer)

ID	Title	Phase
R01	Scope Freeze - rung1 Enterprise v1 (issued as "rung1 ERP v1")	1
R02	Product User-Flow Book	1
R03	Gate Specification	1
R04	elan1 Integration Spec	1
R05	Frappe and ERPNext Technical Standards	1
R06	Requirement to Deployment Checklist	1
R07	Design Partner Pack	1
R08	Competitive Matrix v3	1
R09–R13	Attach Mode scope, connector spec, Tally migration, support runbook, pricing and meter model	2
R14–R19	rung1 Revenue reopen memo, scope, CRM adapters, gates, platform-terms risk; rung1 Voice spec (inbound approvals)	3
R20–R24	rung1 Commerce reopen memo, scope, adapters, gates; agentic protocol status memo	4

Other sources

ID	Document	Note
IM	elan1 — Series A Information Memorandum (22 September 2026)	The investor companion to this memorandum
RTM	rung1 Technology Memorandum (September 2026)	Consumer-side detail only
ETS	elan1 Technology Stack and Alternatives	elan1 product folder
EPA	elan1 Platform Architecture End-to-End Flow	elan1 product folder
ETA	elan1 technical architecture flow	elan1 product folder
ECB	Claude Build Dossier	elan1 product folder
EBB	ELAN1 Build Book	elan1 product folder; earlier build order
ERD	elan1 Requirement-to-Deployment Checklist	elan1 product folder
ETR	elan1 Team Readiness Kit Rev2	elan1 product folder
EAP	elan1 Agenthon Acceleration Playbook	elan1 product folder
ESB	elan1 Strategy Blueprint	elan1 product folder
ETP	elan1 Technical Pitch	elan1 product folder
EBA	elan1 Business Architecture Flow V2	elan1 product folder
ECM	elan1 Competitive Matrix A3 v2	elan1 product folder
EPD	elan1 Business Pitch Deck	elan1 product folder
GIP	elan1 Governed Intelligence Platform	Strategy; earlier platform concept
GAM	elan1 Governed Action Model (deck)	Strategy
HFS	elan1 Horizon — Finished Shape	Strategy and code audit, 9 Sept 2026
HV2	elan1 Horizon v2 deep research report	Research, 12 Sept 2026
BOOMI	Boomi Deep Research Report, September 2026	Research
MMS	elan1 Milk-Mountain-Moonshot Strategy	Strategy
MMM	*Milk*, *Mountain*, *Moonshot* — M K Elango	The founder's book
DS30	elan1 30-Slide Deck Script	Strategy
STS	stack1 Technology Stack and Alternatives	stack1 product folder

ID	Document	Note
SPA	stack1 Platform Architecture End-to-End Flow	stack1 product folder
SBB	stack1 Build Book	stack1 product folder

Appendix C. Glossary

Terms are listed alphabetically, with symbols and code-style identifiers sorted by their first letter. Each meaning carries the source that defines the term. Where the corpus retired an earlier name, the retired name is given so that older material can be read.

Term	Meaning
A0–A5 (autonomy ladder)	The level an actor holds for a write type: A0 Observe, A1 Draft, A2 Propose, A3 Act reversible, A4 Act bounded, A5 Act broad. A property of the actor, not the tool; a missing grant resolves to A0. Written L0–L5 before v3.0 [O01 §02; E04 rung 6]
`acted_unattested`	Mode B state in which the kernel allowed a write but never received the caller's outcome; reconciled and flagged [O03 §04]
`acted_unproven`	Mode A state in which the act reached the target but the proof write failed; alarms, never silently completed [O03 §03]. See outbox
Action Receipt	The open standard for a single governed-action record — one record, one signature, one anchor, one procedure a stranger can implement from the document alone; its licence grant sits irrevocably in Holdco [E25; O11 §02]
Actor kind	`human`, `agent` or `system` (field F10); set by the kernel, never supplied or inferred [E02 §03; E04 rung 1]
Anchor	An RFC 3161 timestamp token over a Merkle root of record digests, proving the records existed in that form no later than the token's time; the only mechanism that detects a truncated chain tail. L2 [E17 §01–02]
Anchor client key	The key that authenticates elan1 to a timestamp authority; in the evidence path but signs nothing a reader verifies [E16 §02]
Anchor interval	The maximum time a record may sit unanchored: ≤ 15 minutes, published as an SLO per tier, never negotiated per customer [E17 §04, decision 3]
`as_of`	The business instant against which gates evaluate; supplied in the intent, never read from a clock [E02 §03, I8]
`assert`	The KernelPort call that runs the Right Path for a write intent and returns allow, refuse or escalate (needs approval) [E02 §04; O03 §05]
Assurance door	The fourth of the Four Doors; where proof records, mode boundaries, key windows and anchoring state are shown to the tenant [E06; E09 §06]
Assurance Level	A property of the proof, never of the policy: L1 Guarded, L2 Attested, L3 Proven [E07; O01 §02]
Attested advisory	The mode in which the caller holds the target credential; elan1 evaluates and records every intent but cannot prevent a write. May not be described as blocking or preventing [E01 §05; O03 §02]
Blast radius	The declared scope of a write type — `one_record`, `one_counterparty`, `batch`, `tenant`, `cross_boundary`; checked at rung 5 and the normative meter basis [E04 rung 5; E05 §06]
Break-glass	Emergency access to a record store; a declared write type under dual control, recorded in the operator's own chain and visible to the tenant — never an admin mode [E02 decision 9; O05 §3]
Bundle (evidence bundle)	The exportable, offline-checkable package for a tenant's `seq` range: eight members (`manifest.json`, `seals.jsonl`, `chain.json`, `anchors/`, `keys.json`, `declarations/`, `lexicon/`, `VERIFY.txt`), format string `elan1.evidence.bundle.v1`. L2 [E15 §02]
C1–C14	The KernelPort conformance tests a consumer must pass in its own CI before claiming "runs on the elan1 layer" — import graph, bypass attempt, rung completeness, determinism, fail-closed matrix, missing grant, idempotency, refusal shape, chain integrity, meter reconciliation, approval separation, rollback drill, version pinning, claim boundary [E02 §08]
Certify	The act of a named human, never the author, making a declaration version resolvable; itself a governed write under dual control [E09 §04]
Chain writer	The single kernel process per tenant that consumes the outbox in order, assigns `seq`, chains `prev_digest` and appends the record; at L2 the only principal that may invoke the signing key [E02 §3; E16 §02]
`channels`	Declaration block 7: where a module is reachable (web, API, messaging), with `mode` per channel. Formerly "doors" [O01 §02; E09 §04]
Claim boundary	The table of sentences elan1 may and may not say at each level; enforced for consumers by C14 [E01 §08; E02 C14]

Term	Meaning
collect1	Pack sketch for RBI collections conduct: four obligation layers (conduct, telecom, data, sector) and fourteen gates in three phases; Attested advisory where the lender's core system owns the write [E27; IM App. D]
Commitment (per-field salted)	The selective-disclosure construction: each field committed with its own salt in SD-JWT's shape, so an issuer can reveal one field without the others. L3 [E23]
Controls family	The Declaration family holding controls — one control, one assertion — that establish regulation clauses. Phase 4 [E21 §04]
Credential broker	The per-tenant store of target-system credentials, which mints a single-use credential per write with a ten-minute life [O03 §06; O05 §3]
Credential custody	The Enforced mechanism in which elan1 holds the target credential; the alternative is token acceptance [E09 §09]
Cross-organisation verification	L3 protocol by which one organisation issues a receipt another checks against a trust anchor and binds into its own chain as a governed write [E22; E14 decision 2]
Customer-held keys	L3 custody in which the customer holds the signing key; removes the cloud-KMS insider limitation [E16 §03, §09; E20]
Decide screen	The console screen where a named human approves or declines, showing every declared gate's result [E06 decision 9; E04 Flow 3]
Declaration	The elan1 ontology: one machine-readable object saying what a module is and may do, in seven blocks; policy as data [O01 §02; E03]
Declaration registry	The append-only store of every certified declaration version, never compacted; a version is immutable and resolves forever [E09 §04; E10 §07]
Declaration schema	The schema declarations conform to; formerly "L0 schema" [O01 §03]
`define`	KernelPort call registering a declaration or a governed action reference [E02; O03 §05]
Designed is not shipped	DR-010: only shipped, demonstrable capability scores "yes" in any competitive matrix, including elan1's own row [O01 DR-010; E08 §01]
Domain signing key	The Ed25519 key that signs record digests at rung 8 from L2; one per residency domain, held in a managed KMS [E16 §02, §04]
DOMAIN string	`elan1.proof.seal.v6`, the first length-prefixed group of every preimage; a format change is a new domain string, never a minor version [E12 §03, §09]
DR-xxx	Numbered entries in the group decision register (DR-001 to DR-013); closed unless superseded by a new numbered entry [O01 §05]
Dual control	Gate G4: a second named actor, never the requester and never an agent, must approve the `write_id` [E04 rung 3; E02 decision 7]
Earned autonomy	L3 mechanism by which an action class graduates (for example A2 to A3) through a dual-control governed write against a deterministic precedent score; revocable by one person [E24]
Enforced	The mode in which no write reaches the system of record without a gate decision and a record, because the kernel holds the credential or the target accepts only kernel-issued tokens [O03 §02; E01 §05]
Escalate	The third outcome of `assert`: a human must decide and the write does not proceed until they do; "the honest version of warn" [E04 rung 4; E02 I6]
Evidence SKU	SKU 2: evidence sold on its own line per verifier profile, per regime, per year, from L2 [E10 §06]
Exit codes (verifier)	0 PASS (with "payload unbound" and "unsigned, assurance level 1" variants), 1 FAIL_INTEGRITY, 2 FAIL_PROFILE, 3 INCOMPLETE, 4 UNSUPPORTED, 5 USAGE [E15]
F01–F16	The sixteen frozen record fields, from `tenant_id` and `seq` to `payload_hash`, `lexicon_hash` and `recorded_at`; no optional fields [E12 §02]
F14 `payload_hash`	Digest over the salted, canonicalised payload; destroying the salt unbinds the payload while the digest still verifies [E12 §02; O05 Flow B]
Fail closed	Every stage's default is refuse; an unavailable dependency is a platform-class refusal, never a silent allow (invariant I3) [E02 I3; O05 §01]
Fast path / forward path	The two undo routes inside the 30-day window: snapshot restore, or a compensating instrument written as a new governed write; the system chooses [E01 §02 #6]
Five-link chain	Clause → control → gate → write type → proof field: how a regulation compiles to a runtime gate [E21; IM §6.7]

Term	Meaning
Four Doors	The module UX grammar: Work · Records · Agents · Assurance, rendered from the declaration with no hand-written screen code [O01 §02; E06]
`fusion.*/v1`	The ten stack1 runtime contracts — chat, gateway, inference, retrieval, vectors, trace, tools, run, guard, memory; nine at stack1 GA, memory limited [S13 §01; O03]
G1–G9	The nine pre-gates at rung 3: consent, residency, retention, dual control, rate, budget, counterparty, schedule, precondition [E04 rung 3]
G-A–G-E (pack gates)	The five pack admission gates: a dated statutory hook, a named auditor, reversible writes, evidence separately saleable, elan1 owns the write path [E18 §02]
Gap record	A governed write stating that a write committed without a record, when and why; the only permitted repair — never a back-filled seal [EBB RB3]
Gate 1, Gate 2, Gate 3	Product gates: a paying customer's write verifies offline (31 Jan 2028); app three renders from its declaration (31 Mar 2028); a signed contract with evidence on its own line (end Q4 2028) [IM §16.3; E10 §06]
Gate A, Gate B	The investor tranche gates for tranches 2 (Oct 2028) and 3 (Jul 2029) [IM §18.2]
Governed Action	The unit that is permitted, executed, recorded and billed — the same unit at all four points [O01 §02; O03 §07]
Import-graph check	CI check that no consumer imports a kernel type and no module reachable from rungs 3–8 imports a model client (C1) [E02 §01, C1]
Insider limitation	The disclosed fact that at L2 a privileged insider in elan1's cloud account could alter the KMS key policy, leaving traces but not prevented [E16 §09]
Invariants I1–I8	The kernel contract's rules: model never produces an allow; single writer; fail closed; refusals free and typed; no fast path; no warn; nothing resolves to "latest"; determinism [E02 §02]
invoice1	The GST e-invoicing pack [IM §5]
Kernel	Plane P3: the one write function, the nine rungs, `evaluate()`, proof and meter; "shut" to change [O01 §02]
KernelPort	The single seam to the kernel: six calls (`define`, `assert`, `seal`, `verify`, `scope`, `refuse`), five objects, one writer, a closed refusal register [E02; O01 §02]
Key register / key window	Registry of `key_id`, `valid_from`, `valid_to`, `successor_id` and status (active, retired, revoked); windows abut and never overlap [E16 §05]
Kill criteria	Fifteen pre-signed termination conditions, five per venture; termination needs no further approval [IM §18.2]
L1 Guarded	Gates, approval, sandbox, 30-day rollback, single writer, hash-chained log and metering; no key material. Being built; GA targeted Q2 CY2027 [E01; IM §4.9]
L2 Attested	Ed25519 signing of the same sixteen fields, dual RFC 3161 anchoring, evidence bundles, the offline verifier and one certified profile. Frozen specification; GA targeted April 2028 [E14; IM §4.9]
L3 Proven	Cross-organisation verification, selective disclosure, customer-held keys, precedent and earned autonomy; begins on an auditor's written acceptance of an L2 bundle [E20; IM §4.9]
ledger1	The first pack: the MCA Rule 3(1) / 11(g) audit trail for accounting software, with profile `in-mca-audit-trail` [E18; IM §5]
Lexicon	The term set in force for a declaration; committed by `lexicon_hash` (F15) and shipped verbatim in a bundle [E12 §02; E15 §02]
MEASURE THIS	Label on a number derived by decomposing the architecture; may not appear in a customer document until measured under real load [O03 §08; EBB EB-11]
Merkle batch / inclusion path	The per-tenant, per-interval tree (RFC 6962, odd node promoted) whose root is anchored; a record's path of $\log_2(n)$ sibling digests proves it alone [E17 §04]
Meter event	Exactly one per governed write, append-only, referencing a `write_id`; refusals at quantity zero enforced by a database `CHECK` [E04 rung 9; E01 §02 #9]
Meter basis	The billing unit derived normatively from the declared blast radius; never a per-customer pricing parameter [E05 §06; E09 §08]
Mode A	SDK in-process integration; the kernel holds the credential; Enforced [O03 §02]
Mode B	MCP or HTTP integration; the caller holds the credential unless the target accepts only kernel tokens; Attested advisory by default [O03 §02]

Term	Meaning
Never-skippable gates	Consent, residency, retention, dual control, counterparty and precondition; a declaration omitting one fails validation and the kernel runs it anyway [E01 §02 #1; E04 rung 3]
`not_measurable` / `out_of_scope`	The two permitted non-values in a declaration or profile field: with a reason and `would_change_if`, or with an ADR reference [E09 §04; E13]
Obligation	The row created when a write escalates: `write_id`, level, expiry and resolver set; its resolution or expiry is itself a governed write [E04 rung 6, Flow 3]
Offline verifier	Zero-dependency verifier with an empty lockfile that shares no code with the chain writer and never phones home; free to anyone. L2 [E12 §08; E15]
Outbox	The row enqueued in the same transaction as the record version at rung 7; the chain writer consumes it in order, so a failed append loses no write [E02 §05; E04 rung 7]
Pack	A regulatory product built as configuration, not code: declarations, policy rules, regulatory bindings, an eval set and an approver model, admitted through five pack gates [E18; IM §5]
Pack template	The eleven blocks every pack declares, from `pack_identity` to `pilot` [E18 §04]
Planes P0–P8	The nine architectural strata: residency and tenancy, runtime, system of record, kernel, declaration, surfaces, channels, apps, packs. Formerly L0–L8 [O01 §02]
Platform floor	The fixed monthly charge per tier that pays for per-domain fixed cost; no discount may reach it [E10 §05]
Precedent	A versioned deterministic score computed inside one tenant's own chain; nothing crosses a tenant boundary. L3 [E24]
Preimage	The length-prefixed concatenation of the DOMAIN string and the sixteen encoded fields; the record digest is SHA-256 over it [E12 §03]
Proof record	The rung-8 record: sixteen fields chained by `prev_digest`; hash-chained at L1, signed from L2; refusals produce one with a zero record identifier [E04 rung 8; E02 §3]
`proof_binding`	Declaration block 6: which fields are bound into the proof record [O01 §02]
Reference tenant	The synthetic tenant against which consumers simulate and run conformance; the control plane cannot read production records [E05 §11; E09 §05]
Refusal classes	POLICY (a rule said no), GATE (a declared gate failed) and PLATFORM (elan1 failed; pages, never billed) [E02 §06]
Refusal register	The closed enumeration of 43 codes, 28 active at L1; each maps to one rung, one class and one operator action; never renumbered or reused. Indexed as REF-nn in O01 [E02 §06; O01 DR-011]
Release key	Offline hardware key held by two named humans that signs verifier binaries and vector sets [E16 §02]
Residency domain	The deployment boundary for data and keys; one signing key per domain; Phase 1 domain `in` in ap-south-1 [E16 §04; O05 §1]
Revocation / `T_suspect`	On suspected key compromise, the key's window closes at the earliest defensible instant; earlier records still verify [E16 §07]
Right Path	The nine rungs of the one write function, in fixed order, with no fast path [O01 §02; E04]
Rungs 0b–9	0b STOP · 1 IDENTITY · 2 WRITE DOOR · 3 PRE-GATES · 4 `evaluate()` · 5 GUARDRAIL · 6 AUTONOMY · 7 ACT · 8 PROOF · 9 METERED [E04 §03]
Sandbox key / shadow sandbox	Distinct terms: `sk_sbx_…` is a self-serve key to an isolated sandbox tenant; the shadow sandbox is the per-tenant copy rung 7 applies to before live [E05 decision 7; E09 §03]
`scope`	Advisory KernelPort call returning the write types and autonomy level available to an actor; `assert` always re-resolves [E02 decision 4; O03 §05]
`seal`	KernelPort call that writes and chains the record — at L1 a hash-chain receipt, at L2 also signed; callable only by the kernel [E02 §3; O01 DR-013]
Seal Spec v6	The frozen specification of the sixteen fields, their encoding and the preimage, with 64 test vectors [E12]
Selective disclosure	L3 capability to reveal chosen fields of a record to a recipient while the rest stay committed [E23]
Seven blocks	The Declaration's structure: identity · shape · permissions · write_types · gates · proof_binding · channels [O01 §02; E03]
`single_anchor`	Marking on a batch anchored by only one authority; degraded, not failed, and carried into every bundle covering it [E17 §06]

Term	Meaning
Single writer	Invariant I2: exactly one process mutates the target per tenant, holding a single-use, ten-minute, one-write credential [E02 I2]
Sovereign deployment	Deployment operated outside elan1's own hosting for a customer or jurisdiction; L3-era and possibly declined [E20 OQ4; IM §8.5]
Stop set / tenant stop	Rung 0b: tenant kill switch, revocation, blacklist, regulator directive; the only refusal with no chain entry [O01 §02; E04 rung 0b]
Telos Council	The group body (founder plus two) that alone may move capital between layers; owns the objective, layer ratios and Coherence Matrix [O11 §07]
Test vectors	The 64 published seal vectors, 28 of them negative, that any implementation must reproduce or reject octet for octet [E12; IM §4.9]
Three numbers	The operations wall: writes committed without a record, records mutated outside the kernel writer, and dependencies in the verifier — all must read zero [EBB EB-12 §7]
Three-Brand Integration Contract	The written boundary between rung1 (product), elan1 (kernel) and stack1 (runtime): what each may never build [O03]
Token acceptance	The Enforced mechanism in which the target accepts only kernel-issued tokens for declared write types [E09 §09; O03 §02]
Token classes	Tenant (one tenant, one actor, one autonomy level), control (platform operations, never writes a tenant record) and service (stack1 components, never reaches KernelPort) — no fourth [O03 §06; O05 §2]
Transfer price	The internal ₹0.40 per committed governed action rung1 pays elan1, refusals at zero; not revenue traction [E10 §04; IM §2.2]
Trust anchor	A third party's published statement binding an organisation to its key, which neither party to a receipt owns; "the load-bearing and least-solved part" of L3 [E22 §04; IM App. E.1]
TSA	Timestamp authority issuing RFC 3161 tokens; two are required, independent, at least one India-based [E17 §03]
Two-implementation rule	No seal record leaves the company until two independent implementations written from the text alone produce identical bytes on all vectors; disagreements fix the text [E12 §01, §08]
Unanchored head	The count of records not yet covered by an anchor; a required manifest field where 0 is a value [E15 §02; E17 §05]
`verify`	KernelPort call that recomputes digests and walks the chain over a range, returning PASS or the first failure with `seq`, field and reason [E02 §4]
Verifier profile	An eight-block document binding a regime's clauses to checks, authored and certified by different named people; passes P1–P12 [E13]
Write intent	A request, never a permission: the object a consumer (possibly a model) submits to `assert` [E02 §03]
Write type	A declared `<slug>.<verb>` write with blast radius and exactly one of reversal plan or `irreversible: true`; there is no generic write [E04 rung 2; O01 §02]